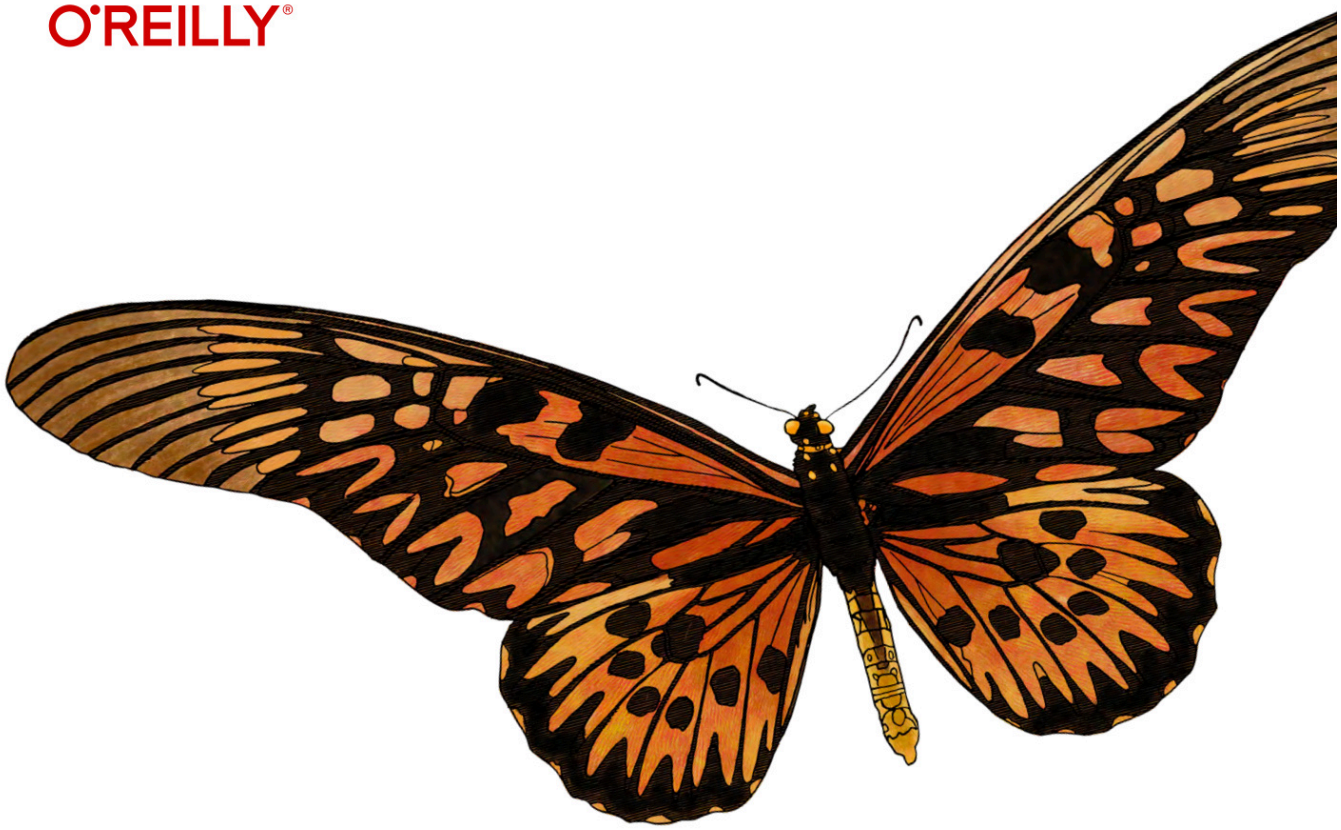


动手实践生成式AI：Transformers与扩散模型

**本pdf由开源项目fast_pdf_trans翻译（经过人工校正）**

Omar Sanseviero, Pedro Cuenca, Apolinário Passos & Jonathan Whitaker “这是开发者掌握过去十年中最大AI革命背后工具和概念的必备指南。” Lewis Tunstall，Hugging Face的机器学习工程师，《自然语言处理与Transformers》合著者 “这本书正是你开始生成式AI之旅所需的一切：从全面的解释到深思熟虑的技巧和动手练习，应有尽有。”

Luba Elliott，AI艺术策展人，elluba.com

动手实践生成式AI：Transformers与扩散模型

通过这本实用的动手实践书籍，学习使用生成式AI技术创建新颖的文本、图像、音频甚至音乐。读者将了解最先进的生成模型如何工作，如何微调和适应它们以满足自己的需求，以及如何结合现有的构建块来创建新模型和不同领域的创意应用。

这本必备书籍首先介绍理论概念，随后是指导性的实际应用，包含大量代码示例和易于理解的插图。你将学习如何使用开源库来利用Transformers和扩散模型，进行代码探索，并研究多个现有项目以指导你的工作。

• 构建和定制能够生成文本和图像的模型
• 探索使用预训练模型与微调自己模型之间的权衡
• 创建和利用能够生成、编辑和修改任何风格图像的模型
• 为多种创意目的定制Transformers和扩散模型
• 训练能够反映你独特风格的模型

Omar Sanseviero曾是Hugging Face的首席Llama官及平台和社区负责人。他在开源、产品、研究和技术团队的交叉领域拥有丰富经验。

Pedro Cuenca是Hugging Face的机器学习工程师。

Apolinário Passos是Hugging Face的艺术家和机器学习艺术工程师，为创意和艺术社区提供与AI模型互动的技术和工具。

Jonathan Whitaker是数据科学家和深度学习研究员。除了在answer.ai的研发工作外，他还通过DataScienceCastnet YouTube频道和各种免费在线资源分享知识。

对《动手实践生成式AI：Transformers与扩散模型》的赞誉

一本不可或缺的技术指南，提供了清晰的动手实践指导，用于实现稳定扩散和微调语言模型。任何AI开发者书架上必备的书籍。

—Vicki Reyzelman，Mave Sparks首席AI解决方案架构师

作为一本全面且实用的指南，适合任何渴望掌握生成式AI的人，这本书将理论与实际应用相结合。从语言模型和扩散技术的基础知识到微调和创建文本到图像应用等高级主题，作者提供了可操作的Python代码和清晰的见解，使读者能够构建、创新并在这个快速发展的领域中保持领先。他们的专业知识和动手实践方法使这本书成为初学者和经验丰富的从业者的宝贵资源。

—Anil Sood，安永美国高级经理

一本无价的指南，揭开了生成式AI的神秘面纱，将实用见解与涵盖多个领域的动手技术和示例相结合。对AI未来感兴趣的人必读之作。

—Vishwesh Ravi Shrimali，汽车行业工程师

这本书是一本制作精良的指南，使复杂的AI概念对广泛的读者群体变得易于理解。作者为Transformers和扩散模型带来了清晰度，使其成为任何真正希望理解推动当今生成式AI的基本构建块的人的绝佳读物。

—Sai M Vuppalapati，Tubi TV数据与AI/ML平台产品经理

这本书是任何对AI生成内容潜力感到好奇的人的宝库。通过专注于解决相关的现实问题并提供动手指导，它巧妙地弥合了复杂概念，使生成式AI对爱好者和专业人士都变得易于接近。对于任何准备深入这个动态领域并探索生成式AI力量的人来说，这是一本必读之作。

—Lipi Deepaakshi Patnaik，Zeta高级软件开发人员

这本书正是你开始生成式AI之旅所需的一切：从全面的解释到深思熟虑的技巧和动手练习，应有尽有。对于任何希望学习如何使用、适应和评估生成式AI模型的人来说，这是一本优秀的指南。

—Luba Elliott，AI艺术策展人，elluba.com

Omar、Pedro、Apolinário和Jonathan展示了技术深度与直观指导的令人印象深刻的结合，使读者能够以清晰和目的性将创新的生成式AI解决方案变为现实。通过对Transformers和扩散模型的清晰解释，以及它们在文本、图像和音频中的深入开发和应用，他们使复杂的AI世界变得易于理解和操作。这部作品为下一代创新者提供了自信应对生成式AI技术、伦理和实践挑战的能力。

—Aditya Goel，AI顾问

这本书非常适合任何开始生成式AI之旅的人。作者以简单直观的方式引导我们了解这个复杂的主题。

—Zygmunt Lenyk，Odyssey研究工程师

《动手实践生成式AI：Transformers与扩散模型》提供了对生成式AI核心概念和应用的全面且易于理解的指南。作者巧妙地涵盖了从Transformers和扩散模型到创意应用的基本主题，使其成为任何希望掌握生成式AI技术的人的必读之作。

—Gourav Singh Bais，Allianz Services高级数据科学家和高级技术内容撰稿人

开发者掌握过去十年中最大AI革命背后工具和概念的必备指南。这本书是我自己书籍的强劲竞争对手，我担心我们的版税！

—Lewis Tunstall，Hugging Face的机器学习工程师，《自然语言处理与Transformers》合著者

动手实践生成式AI：Transformers与扩散模型

Omar Sanseviero, Pedro Cuenca, Apolinário Passos, 和 Jonathan Whitaker

《动手实践生成式AI：Transformers与扩散模型》由Omar Sanseviero, Pedro Cuenca,

Apolinário Passos, 和 Jonathan Whitaker撰写

版权 ® 2025 Omar Sanseviero, Pedro Cuenca, Apolinário Passos, 和 Jonathan Whitaker。保留所有权利。

在美国印刷。

由O'Reilly Media, Inc.出版，地址：1005 Gravenstein Highway North, Sebastopol, CA 95472。

O'Reilly书籍可用于教育、商业或销售推广用途。大多数书籍的在线版本也可在https://oreilly.com上获取。如需更多信息，请联系我们的公司/机构销售部门：800-998-9938或corporate@oreilly.com。

责任编辑：Nicole Butterfield 开发编辑：Jill Leonard 制作编辑：Gregory Hyman 校对编辑：Krsta Technology Solutions 校对员：Sharon Wilkey

索引员：BIM Creatives, LLC 内部设计师：David Futato 封面设计师：Karen Montgomery 插图师：Kate Dullea

2024年12月：第一版

第一版修订历史

2024年11月22日：首次发布

有关发布详情，请参见http://oreilly.com/catalog/errata.csp?isbn ＝ 9781098149246。

O'Reilly徽标是O'Reilly Media, Inc.的注册商标。《动手实践生成式AI：Transformers与扩散模型》、封面图像及相关商业外观是O'Reilly Media, Inc.的商标。

本书中表达的观点为作者的观点，不代表出版商的立场。尽管出版商和作者已尽最大努力确保本书中包含的信息和说明准确无误，但出版商和作者对任何错误或遗漏不承担任何责任，包括但不限于因使用或依赖本书而导致的损害责任。使用本书中的信息和说明需自行承担风险。如果本书中包含或描述的任何代码示例或其他技术受开源许可或他人知识产权的约束，你有责任确保你的使用符合此类许可和/或权利。

# 目录

前言. xi

第一部分. 利用开放模型

1. 生成式媒体简介.

生成图像 4
生成文本 7
生成音频片段 8
伦理与社会影响 8
我们走过的路与现状 9
生成式AI模型是如何创建的？ 10
总结 12

2. 变压器. . . 13

语言模型的实际应用 14
文本分词 14
预测概率 17
生成文本 20
零样本泛化 28
少样本泛化 30
变压器块 33
变压器模型谱系 35
序列到序列任务 35
仅编码器模型 37
预训练的力量 40
变压器回顾 43
局限性 45
超越文本 46
项目时间：使用语言模型生成文本 50
总结 50
练习 52
挑战 52

参考文献 53

3. 压缩与信息表示. . . . 55

自编码器 57
数据准备 57
编码器建模 61
解码器 64
训练 65
探索潜在空间 70
可视化潜在空间 74
变分自编码器 78
VAE编码器与解码器 79
从编码器分布中采样 80
训练VAE 83
VAE用于生成建模 93
CLIP 93
对比损失 94
使用CLIP的逐步指南 95
使用CLIP进行零样本图像分类 101
零样本图像分类流程 102
CLIP用例 103
CLIP的替代方案 104
项目时间：语义图像搜索 105
总结 106
练习 108
挑战 108
参考文献 109

4. 扩散模型. 111

关键洞察：迭代优化 112
训练扩散模型 115
数据 116
添加噪声 118
UNet 119
训练 121

采样 123
评估 124
深入探讨：噪声调度 126
为什么要添加噪声？ 126
从简单开始 128
数学原理 130
输入分辨率和缩放的影响 135
深入探讨：UNet及其替代方案 137
简单的UNet 138
改进UNet 141
替代架构 142
深入探讨：扩散目标 143
项目时间：训练你的扩散模型 145
总结 145
练习 146
挑战 147
参考文献 148

5. 稳定扩散与条件生成. . . . 149

添加控制：条件扩散模型 149
数据准备 150
创建类别条件模型 152
训练模型 153
采样 157
提高效率：潜在扩散 159
稳定扩散：组件深入解析 160
文本编码器 161
变分自编码器 164
UNet 167
稳定扩散XL 169
FLUX、SD3与视频 171
无分类器引导 172
整合所有：带注释的采样循环 173
开放数据，开放模型 176
挑战与LAION-5B的终结 177

替代方案 178
公平与商业使用 178
项目时间：使用Gradio构建交互式ML演示 179
总结 180
练习 181
挑战 181
参考文献 182

第二部分. 生成模型的迁移学习

6. 微调语言模型. 185

文本分类 186
识别数据集 187
定义使用的模型类型 189
选择一个好的基础模型 190
预处理数据集 190
定义评估指标 192
训练模型 194
仍然相关吗？ 202
生成文本 203
选择合适的生成模型 204
训练生成模型 208
指令 212
适配器简介 216
量化简介 221
整合所有 225
深入探讨评估 230
项目时间：检索增强生成 233
总结 235
练习 236
挑战 236
参考文献 237

7. 微调稳定扩散. . 239

完整稳定扩散微调 239

准备数据集 240
微调模型 242
推理 246
DreamBooth 248
准备数据集 250
先验保留 250
DreamBooth模型 251
推理 252
训练LoRAs 253
赋予稳定扩散新能力 256
修复 256
特殊条件的额外输入 256
项目时间：自己训练SDXL DreamBooth LoRA 257
总结 258

练习 挑战 参考文献

第三部分. 更进一步

8. 文本到图像模型的创意应用. 263

图像到图像 263
修复 265
提示权重与图像编辑 267
提示权重与合并 268
通过语义引导编辑扩散图像 270
通过反转进行真实图像编辑 274
使用LEDITS $^{++}$ 编辑 276
通过指令微调进行真实图像编辑 277
ControlNet 279
图像提示与图像变体 283
图像变体 283
图像提示 285
项目时间：你的创意画布 288
总结 289
练习 289
参考文献 290

9. 生成音频. . 291

音频数据 293
波形 297
频谱图 298
使用基于变压器的架构进行语音到文本转换 307
基于编码器的技术 308
编码器-解码器技术 312
从模型到管道 315
评估 318
从文本到语音到生成音频 324
使用序列到序列模型生成音频 324
超越语音的Bark 329
AudioLM与MusicLM 332
AudioGen与MusicGen 335
音频扩散与Riffusion 336
Dance Diffusion 339
更多关于生成音频的扩散模型 340
评估音频生成系统 340
下一步是什么？ 341
项目时间：端到端对话系统 342
总结 342
练习 345
挑战 345
参考文献 346

10. 生成式AI的快速发展领域. . . . 349

偏好优化 349
长上下文 352
专家混合 354
优化与量化 356
数据 358
一统天下的模型 359
计算机视觉 360
3D计算机视觉 362

视频生成 363
多模态 365
社区 367
A. 开源工具. 369
B. LLM内存需求. . . 373
C. 端到端检索增强生成. . 377
索引. . . . 385

# 前言

生成式AI是一项革命性技术，它已迅速从实验室演示过渡到实际应用，影响了数十亿人。它可以通过学习现有数据的模式来创建新内容——图像、文本、音频、视频等，从而增强创造力、扩充数据或协助完成许多任务。例如，一个经过音乐训练的生成式AI模型可以创作新的旋律，而一个经过文本训练的模型可以生成故事甚至编程代码。

这本书不仅适合专家——它适合任何想了解这个迷人新领域的人。我们不会专注于从头构建模型或直接深入复杂的数学。相反，我们将利用现有模型解决现实世界的问题，帮助你建立对这些技术如何工作的坚实直觉，并为你继续探索提供基础。

我们希望这种实践方法能帮助你快速高效地掌握生成式AI。你将学习如何使用预训练模型，根据你的需求进行调整，并用它们生成新数据。你还将学习如何评估生成数据的质量，并探索使用生成式AI可能引发的伦理和社会问题。这种接触将使你能够跟上新模型的步伐，并帮助你确定你可能想深入探索的领域。

# 谁应该读这本书

鉴于你可能已经看到的关于生成式AI的令人印象深刻的产品和新闻，对它感到兴奋或担忧是正常的！无论你是对程序如何生成图像感到好奇，想训练一个模型以你的风格发推文，还是希望更深入地理解像ChatGPT这样的产品，这本书都适合你。通过生成式AI，我们可以做到所有这些以及许多其他事情，包括：

撰写新闻文章摘要 · 根据描述生成图像

- 提高图像质量
- 转录会议

- 生成以你的声音风格合成的语音
- 将新主题或风格融入图像生成模型，比如生成“你的猫穿着宇航服”的图像

无论你的理由是什么，你已经决定学习生成式AI，这本书将引导你完成这一过程。

# 先决条件

本书假设你熟悉Python编程，并对机器学习有基础理解，包括PyTorch或TensorFlow等框架的基本使用。虽然不需要有训练模型的实践经验，但它将有助于更深入地理解内容。以下资源为本书涵盖的主题提供了良好的基础：

- 《Scikit-Learn、Keras 和 TensorFlow 机器学习实战》，第二版，作者 Aurélien Géron（O’Reilly）
- 《使用 fastai 和 PyTorch 进行深度学习的程序员指南》，作者 Jeremy Howard 和 Sylvain Gugger（O’Reilly）

如果你对先决条件感到畏惧，别担心！这本书旨在增强你的直觉，并提供一种实践方法，帮助你入门。

# 你将学到什么

本书分为三个部分：

- 在第一部分“利用开源模型”中，我们将介绍生成式 AI 的基本构建模块。你将学习如何使用预训练模型生成文本和图像。这一部分将帮助你理解该领域的基础知识并掌握整体概况。
- 第二部分“生成模型的迁移学习”将重点介绍微调，展示如何将现有模型调整以满足你的需求。我们将引导你如何教扩散模型一个新概念，定制一个 Transformer 模型以分类文本并在对话中回复，并探索在有限硬件上处理大型模型的高级技术。如果你是第一次听说 Transformer 或扩散模型，不用担心；你很快就会了解它们。
- 在第三部分“更进一步”中，我们将扩展前几部分的内容，生成新的模态（如音频）并探索新的创意应用。读完本书后，你将扎实掌握构建生成式应用的方法和技术。

# 如何阅读本书

我们设计本书时希望你能按顺序阅读，但我们也尽量使各章自成一体，以便你可以跳转到最感兴趣的部分。本书中涵盖的许多思想适用于多种模态，因此即使你只对某个特定领域（如图像生

成）感兴趣，浏览其他章节仍然可能对你有价值。

我们在书中穿插了练习和代码示例，旨在帮助你动手实践。建议你在阅读过程中尝试完成这些练习，并在可能的情况下，看看是否能将这些示例调整到你的用例中。亲自动手尝试将帮助你更深入地理解内容。

最后，大多数章节的总结部分列出了进一步阅读的额外资源。我们鼓励你探索这些资源，以加深对书中涵盖主题的理解。你不需要在进入新章节之前阅读这些资源；你可以在准备好深入研究感兴趣的主题时再回来阅读。

# 软件和硬件要求

为了充分利用本书，我们强烈建议你在阅读时运行代码示例。通过修改代码并探索不同场景进行实验，将增强你的理解。处理 Transformer 和扩散模型可能会对计算资源要求较高，因此拥有一台配备 NVIDIA GPU 的计算机将非常有益。虽然 GPU 不是必需的，但它将显著加快训练速度。

你可以使用多种在线选项，例如 Google Colaboratory 和 Kaggle Notebooks。按照以下说明设置你的环境并跟随学习：

使用 Google Colab 大多数代码应该可以在任何 Google Colab 实例上运行。我们建议你在包含训练循环的章节中使用 GPU 运行时。

在本地运行代码 要在你的计算机上运行代码，请使用你首选的方法创建一个 Python 3.10 虚拟环境。例如，你可以使用 conda 这样做：

```
conda create -n genaibook python
conda activate genaibook
```

为了获得最佳性能，我们建议使用兼容 CUDA 的 GPU。如果你不知道 CUDA 是什么，不用担心，我们会在书中解释。

书中使用了许多支持工具和辅助函数。要访问它们，请安装 genaibook 包：

```
pip install genaibook
```

这将安装运行 Transformer 和扩散模型所需的库，以及 PyTorch、matplotlib、numpy 和其他必

备工具。

所有代码示例和补充材料都可以在本书的 GitHub 仓库中找到。你可以在 Jupyter Notebooks 中交互式地运行所有示例，并且仓库将定期更新最新资源。

# 本书使用的约定

本书使用了以下排版约定：

*斜体*

表示新术语、URL、电子邮件地址、文件名和文件扩展名。

`等宽字体`

用于程序列表，以及在段落中引用程序元素，如变量或函数名、数据库、数据类型、环境变量、语句和关键字。

此元素表示提示或建议。

此元素表示一般说明。

此元素表示警告或注意事项。

# 使用代码示例

补充材料（代码示例、练习等）可在 https://oreil.ly/handsonGenAIcode 下载。

如果你有技术问题或在使用代码示例时遇到问题，请发送电子邮件至 support@oreilly.com。

本书旨在帮助你完成工作。一般来说，如果本书提供了示例代码，你可以在你的程序和文档中使用它。除非你复制了大量代码，否则无需联系我们以获得许可。例如，编写一个使用本书中几段代码的程序不需要许可。出售或分发 O'Reilly 书籍中的示例需要许可。通过引用本书并引用示例代码来回答问题不需要许可。将本书中的大量示例代码纳入你的产品文档中需要许可。

我们感谢但不强制要求署名。署名通常包括书名、作者、出版商和 ISBN。例如："《Hands-On Generative AI with Transformers and Diffusion Models》，作者 Omar Sanseviero、Pedro Cuenca、Apolinário Passos 和 Jonathan Whitaker（O'Reilly）。版权所有 2025 Omar Sanseviero、Pedro Cuenca、Apolinário Passos 和 Jonathan Whitaker，ISBN 978-1-098-14924-6。"

如果你认为你对代码示例的使用超出了合理使用范围或上述许可，请随时通过 permissions@oreilly.com 联系我们。

# 如何联系我们

请将有关本书的评论和问题发送给出版商：

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-889-8969（美国或加拿大）
707-827-7019（国际或本地）
707-829-0104（传真）
support@oreilly.com
https://oreilly.com/about/contact.html

我们为本书设立了一个网页，列出了勘误、示例和任何其他信息。你可以访问此页面：https://oreil.ly/handsonGenAI。

有关我们的书籍和课程的最新消息和信息，请访问 https://oreilly.com。

在 LinkedIn 上找到我们：https://linkedin.com/company/oreilly-media。

在 YouTube 上观看我们：https://youtube.com/oreillymedia。

# 最先进的技术：一个不断变化的目标

最先进的技术（SOTA）用于描述在特定任务或领域中当前实现的最高性能水平。在生成式 AI 领域，随着新模型的开发和新技术的发现，SOTA 不断变化。本书将为你提供生成式 AI 基础知识的扎实基础，但当你阅读本书时，可能已经发布了比我们讨论的模型更优秀的新模型。

与其试图追逐不断变化的最佳技术，我们尝试专注于帮助你理解模型工作原理的通用原则，这些原则即使在该领域继续发展时仍然有用。新模型很少凭空出现，通常建立在先前模型的思想之上。通过理解基础知识，你将更好地理解最新发展。

# 致谢

我们想向令人难以置信的 O'Reilly 团队，特别是 Jill Leonard，表达最深切的感谢，感谢她在整个过程中的出色指导和支持。特别感谢 Nicole Butterfield、Karen Montgomery、Kate Dullea、Gregory Hyman 和 Kristen Brown，感谢他们从最初的规划到制作精美封面和插图的宝贵建议和贡献。

我们非常感谢我们的技术审稿人：Vishwesh Ravi Shrimali、David Mertz、Lipi Deepaakshi Patnaik、Luba Elliott、Anil Sood、Sai Vuppalapati、Ranjeeta Bhattacharya、Rajat Dubey、Bryan Bischof、Vladislav Bilay、Gourav Singh Bais、Aditya Goel、Lakshmanan Sethu Sankaranarayanan、Zygmunt Lenyk、Youssef Hosn、Vicki Reyzelman、Lewis Tunstall、Sayak Paul 和 Vaibhav Srivastav。他们的深刻反馈对本书的成型起到了关键作用。

我们还想向 Hugging Face 团队表达我们的感激之情，感谢他们的启发和合作，特别是 Clémentine Fourrier 在模型评估方面的见解，Sanchit Gandhi 在音频相关主题上的指导，以及 Leandro von Werra 和 Lewis Tunstall 在书籍写作过程中的帮助。Hugging Face 团队以其才华和善意继续激励我们，帮助这个项目得以实现。

衷心感谢无数朋友、合作者和贡献者，他们塑造了我们引以为豪的开源生态系统。我们感激整个机器学习社区，他们推动了构成本书核心的研究、工具和资源的进步。这项工作是在 Jupyter Notebooks 中完成的，我们特别感谢 Jeremy Howard、Hamel Husain 以及所有为 Quarto 和 nbdev 做出贡献的人，使这一切成为可能。

## Jonathan

我非常感激那些分享想法并推动可能性的研究者和黑客社区。感谢 Jeremy Howard、Tanishq Abraham 以及 fastdiffusion 团队的其他人，他们一起学习了这些想法的所有内容。还有我出色的合著者们，没有他们，这本书就不可能完成！

## Apolinário

我感谢我的合著者 Omar、Pedro 和 Jonathan 共同创作了这本书。将技术教育与创造力结合起来是一个有趣且充满挑战的任务。我感谢那些理解并支持我的朋友，即使我带着笔记本电脑来聚会时，他们依然包容我。我也感谢 Hugging Face 的同事们一直以来的支持。

## Pedro

写书是一件非常有趣的事情，但它不公平地要求你所爱的人做出牺牲。我非常幸运，有我的生活伴侣 María José 的支持。她让我能够轻松地投入工作，当我遇到瓶颈时，她用常识性的推理帮助我，坦白说，这种常识并不常见。我向我的父母道歉，因为我每次拜访时都带着笔记本电脑；向我的儿子 Pablo 道歉，因为我们没有像我们希望的那样探索海拉鲁或艾欧泽亚；向我的儿子 Javier 道歉，因为我有时谈论工作太多，谈论生活太少。他们是最棒的。

我真正受到我出色的合著者们的启发。我钦佩并仰望他们，不敢相信自己每天都能从他们身上学到东西，这让我感到非常幸运。这种启发也延伸到了 Hugging Face 的团队，他们的热情和谦逊为事情的发生提供了原始的环境，也延伸到了整个开放的机器学习社区，他们的工作始终在推动这个领域的发展，但并不总是得到应有的认可。

谢谢你们。

## Omar

感谢 Michelle，你在这个过程中一直给予我鼓励，感谢所有的头脑风暴会议，以及过去两年里的支持。没有你，我无法完成这个项目。徒步旅行又可以提上日程了！

感谢我的父母 Ana 和 Walter，感谢你们从一开始就培养了我对书籍的热爱，并支持我成为今天的我。

最后，我要感谢我出色的合著者——Pedro、Poli 和 Jonathan。这段旅程非常有趣，我非常感激我们能够一起完成这个项目。

# 第一部分 利用开放模型

## 第1章 生成式媒体简介

近年来，生成模型变得非常流行。如果你正在阅读这本书，你可能已经与生成模型有过互动。也许你使用过 ChatGPT 生成文本，在 Instagram 等应用中使用过风格迁移，或者看过那些成为头条新闻的深度伪造视频。这些都是生成模型的实际应用！

在这本书中，我们将探索生成模型的世界，从两种生成模型家族（Transformer 和扩散模型）的基础知识开始，逐步深入到更高级的主题。我们将介绍生成模型的类型、它们的工作原理以及如何使用它们。在本章中，我们将回顾一些历史，了解我们是如何走到今天的，并看看一些模型提供的功能，这些功能我们将在书中更深入地探讨。

那么，生成式建模到底是什么？其核心是教一个模型生成与其训练数据相似的新数据。例如，如果我在一个猫的图像数据集上训练一个模型，那么我可以使用该模型生成新的猫的图像，这些图像看起来就像来自原始数据集一样。这是一个强大的想法，它有广泛的应用，从创建新颖的图像和视频到生成具有特定风格的文本。

在本书中，你将发现一些流行的工具，这些工具使得使用现有的生成模型变得非常简单。机器学习（ML）的世界提供了许多开放访问的模型，这些模型在大型数据集上训练，任何人都可以使用。从头开始训练这些模型可能既昂贵又耗时，但开放访问模型提供了一种实用且高效的替代方案。这些预训练模型可以生成新数据、分类现有数据，并可以适应新的应用。最受欢迎的开放访问模型平台之一是 Hugging Face，该平台拥有超过两百万个模型，适用于许多机器学习任务，包括图像生成。

### 生成图像

作为一个开源库的示例，我们将从 diffusers 开始。这个流行的库提供了对最先进的（SOTA）扩散模型的访问。它是一个强大而简单的工具箱，可以让我们快速加载和训练扩散模型。

通过访问 Hugging Face Hub 并过滤出基于文本提示生成图像的模型（文本到图像），我们可以找到一些最受欢迎的模型，例如 Stable Diffusion 和 SDXL。我们将使用 Stable Diffusion 1.5，这是一个能够生成高质量图像的扩散模型。如果你浏览模型网站，你可以阅读模型卡，这是一个用于可发现性和可重复性的重要文档。在那里，你可以了解模型、它是如何训练的、预期的用例等等。

既然我们有了一个模型（Stable Diffusion）和一个使用模型的工具（diffusers），我们现在可以生成我们的第一张图像了！当我们加载模型时，我们需要将它们发送到特定的硬件设备，例如 CPU（cpu）、GPU（cuda 或 cuda:0）或称为 Metal（mps）的 Mac 硬件。我们在前言中提到的 genaibook 库有一个实用函数，可以根据你运行示例代码的位置选择适当的设备。例如，以下代码将分配 cuda 给设备变量，如果你有 GPU 的话：

```
from genaibook.core import get_device
device = get_device()
print(f"Using device: {device}")
```

```
Using device: cuda
```

接下来，我们将加载 Stable Diffusion 1.5。diffusers 库提供了一个方便的高级封装器，称为 StableDiffusionPipeline，非常适合这个用例。现在不用担心所有的参数——重点包括以下内容：

- 有许多具有 Stable Diffusion 架构的模型，所以我们需要指定我们想要使用的模型。我们将使用 stable-diffusion-v1-5/stable-diffusion-v1-5，这是 RunwayML 发布的原始 Stable Diffusion 1.5 模型的镜像。
- 我们需要指定加载模型的精度。精度是你稍后会学到的东西。在高层次上，模型由许多参数组成（数百万或数十亿个）。每个参数都是在训练期间学习到的数字，我们可以用不同的精度存储这些参数（换句话说，我们可以使用更多的比特来存储模型）。更高的精度允许模型存储更多的信息，但它也需要更多的内存和计算。另一方面，我们可以通过设置 torch_dtype = float16 来使用较低的精度，并使用比默认的 float32 更少的内存。在进行推理（一种花哨的说法，表示“执行”模型）时，使用 float16 通常是可以的。[1]

第一次运行此代码时，可能会花费一些时间：毕竟，管道下载的是一个几 GB 的模型！如果你第二次加载管道，它只会在 Hugging Face 上托管模型的远程仓库发生变化时重新下载模型。2 Hugging Face 库将模型本地存储在缓存中，这使得后续加载速度更快：

```
import torch
from diffusers import StableDiffusionPipeline
pipe = StableDiffusionPipeline.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    variant="fp16",
).to(device)
```

现在模型已经加载，我们可以定义一个提示——模型将接收的文本输入。然后我们可以将提示传递给模型，并根据该文本生成我们的第一张图像！尝试输入以下提示：

```
prompt = "a photograph of an astronaut riding a horse"
pipe(prompt).images[0]
```

令人兴奋！只需几行代码，我们就生成了一张新图像。尝试修改提示并生成新的图像。你可能会注意到两件事。首先，运行相同的代码每次都会生成不同的图像。这是因为扩散过程本质上是随机的，意味着它使用随机性来生成图像。我们可以通过设置种子来控制这种随机性：

```
import torch
torch.manual_seed(0)
```

其次，生成的图像并不完美。它们可能会有伪影、模糊，或者与提示完全不匹配。我们将在后面的章节中探讨这些局限性以及如何提高生成图像的质量。例如：

- 第4章和第5章深入探讨了扩散模型背后的所有组件，以及如何从文本生成新图像。它们依赖于像自动编码器这样的基础方法——在第3章中介绍——这些方法可以从输入数据中学习高效的表示，并减少构建扩散模型和其他生成模型的计算需求。
- 在第7章中，你将学习如何向 Stable Diffusion 教授新概念。例如，我们可以向 Stable Diffusion 教授“我的狗”的概念，以生成作者狗的新场景图像，例如“我的狗访问月球”。
- 第8章展示了扩散模型如何用于不仅仅是图像生成，例如通过提示编辑图像或填充图像的空白部分。

# 生成文本

正如 diffusers 是一个非常方便的扩散模型库，流行的 transformers 库对于运行基于 transformer 的模型和适应新的用例也非常有用。它为各种任务提供了标准化的接口，例如生成文本、检测图像中的物体以及将音频文件转录为文本。

transformers 库提供了不同层次的抽象。例如，如果你不关心所有内部细节，最简单的方法是使用 pipeline，它抽象了获取预测所需的所有处理。我们可以通过调用 `pipeline()` 函数并指定要解决的任务（如文本分类）来实例化一个 pipeline：

```
from transformers import pipeline
classifier = pipeline("text-classification", device=device)
classifier("This movie is disgustingly good!")
```

[{'label': 'POSITIVE', 'score': 0.9998536109924316}]

模型正确地预测了输入文本中的情感是积极的。默认情况下，文本分类 pipeline 使用情感分析模型，但我们也可以指定其他基于 transformer 的文本分类模型。

同样，我们可以将任务切换到文本生成（text-generation），通过它我们可以基于输入提示生成新文本。默认情况下，pipeline 使用 GPT-2 模型。transformer pipeline 使用默认的最大生成字数，因此如果输出被截断，不要感到惊讶。稍后你将学习如何更改这一点：

```
from transformers import set_seed

// 设置种子确保每次运行此代码时都能得到相同的结果
set_seed(10)

generator = pipeline("text-generation", device=device)
prompt = "It was a dark and stormy"
generator(prompt)[0]["generated_text"]
```

It was a dark and stormy year, and my mind went blank," says the 27-year-old, who has become obsessed with art, poetry and music since moving to France. "I don't really know why, but there are things

尽管 GPT-2 按照今天的标准并不是一个很好的模型，但它为我们提供了一个使用小型模型时 transformer 生成能力的初步示例。你从 GPT-2 中学到的相同概念可以应用于 Llama 或 Mistral 等模型，这些模型是（在撰写本文时）最强大的开放访问模型之一。在本书中，我们将在模型的质量和大小之间取得平衡。通常，较大的模型具有更高质量的生成。同时，我们希望拥有消费级计算机或访问免费服务（如 Google Colab）的人能够通过运行代码创建新的生成：

- 第 2 章将教你 transformer 模型在底层是如何工作的。我们将深入探讨不同类型的 transformer 模型以及如何使用它们生成文本。
- 第 6 章将教你如何继续训练 transformer 模型以适应不同的用例。这将使我们能够创建类似于你可能使用过的 ChatGPT 或 Gemini 的对话模型。我们还将讨论高效的训练方法，以便你可以在自己的计算机上训练 transformer 模型。

## 生成音频片段

生成模型不仅限于图像和文本。模型可以生成视频、短歌曲、合成语音、蛋白质提案等等！

第 9 章深入探讨了可以用机器学习解决的音频相关任务，例如转录会议和生成音效。现在，我们可以将自己限制在熟悉的 transformers pipeline 中，并使用 Meta 发布的 MusicGen 的小型版本来生成基于文本的音乐：

```
pipe = pipeline("text-to-audio", model="facebook/musicgen-small", device=device)

data = pipe("electric rock solo, very intense")

print(data)
```

{'audio': array([0.12342193, 0.11794732, 0.14775363, ..., 0.0265964 , 0.02168683, 0.03067675], dtype=float32), 'sampling_rate': 32000}

稍后，你将学习音频数据是如何表示的以及这些数字的含义。当然，我们无法在书中直接打印音频文件！最好的替代方法是在我们的笔记本中显示一个查看器，或者将音频保存到文件中，以便我们可以使用我们最喜欢的音频应用程序播放。例如，我们可以使用 `IPython.display()` 来实现这一点：

```
import IPython.display as ipd

display(ipd.Audio(data["audio"][0], rate=data["sampling_rate"]))
```

# 伦理和社会影响

虽然生成模型提供了显著的能力，但它们的广泛采用引发了关于伦理和社会影响的重要考虑。在我们探索生成模型的能力时，重要的是要牢记这些因素。以下是几个关键领域需要考虑：

### 隐私和同意

生成模型能够基于极少的数据生成逼真的图像和视频，这对隐私构成了重大挑战。例如，从个人的一小部分真实图像中创建合成图像，引发了关于未经同意使用个人数据的问题。这也增加了创建深度伪造的风险，深度伪造可能被用来传播错误信息或伤害个人。

### 偏见和公平性

生成模型是在包含偏见的大型数据集上训练的。这些偏见可能会被生成模型继承和放大，我们将在第 2 章中探讨这一点。例如，用于训练图像生成模型的偏见数据集可能会生成刻板或歧视性的图像。重要的是要考虑减轻这些偏见，并确保生成模型被公平和道德地使用。

### 监管

鉴于生成模型相关的潜在风险，越来越多的人呼吁进行监管监督和问责机制，以确保负责任的开

发。这包括透明度要求、伦理指南和法律框架，以解决生成模型的滥用问题。

重要的是要以深思熟虑和道德的心态来对待生成模型。在我们探索这些模型的能力时，我们还将考虑伦理影响以及如何负责任地使用它们。

## 我们的历程和现状

生成模型的研究和开发始于几十年前，最初的努力集中在基于规则的系统上。随着计算能力和数据可用性的增加，生成模型逐渐演变为使用统计方法和机器学习。随着深度学习作为机器学习中的强大范式的出现，以及图像和语音识别领域的突破，生成模型取得了显著进展。尽管卷积神经网络（CNNs）和循环神经网络（RNNs）是几十年前发明的，但在过去十年中它们变得非常流行。CNNs 彻底改变了图像处理任务，而 RNNs 带来了序列数据建模能力，使得翻译文本和生成文本等任务成为可能。

2014 年，Ian Goodfellow 提出了生成对抗网络（GANs），以及诸如深度卷积 GANs（DCGANs）和条件 GANs 等变体，开启了生成模型的新时代。GANs 被用于生成高质量图像，并应用于风格迁移等任务，使用户能够以惊人的真实感将艺术风格应用于他们的图像。尽管 GANs 非常强大，但近年来扩散模型的质量已经超越了 GANs。

同样，尽管 RNNs 曾经是语言建模的首选工具，但包括 GPT 在内的 transformer 模型在自然语言处理（NLP）中达到了最先进的性能。这些模型在语言理解、文本生成和机器翻译等任务中展示了显著的能力。特别是 GPT，因其生成连贯且上下文相关的文本的能力而变得非常流行。不久之后，一波生成语言模型涌现出来。

由于近年来研究、资源和开发的迅速扩展，生成 AI 领域比以往任何时候都更加容易接触。一个对该领域感兴趣的不断增长的社区、丰富的开源生态系统以及促进部署的研究，导致了广泛的应用和用例。自 2023 年以来，新一代能够生成高质量图像、文本、代码、视频等的模型已经出现；例如 ChatGPT、DALL·E、Imagen、Stable Diffusion、Llama、Mistral 等等。

## 生成式 AI 模型是如何创建的?

通常，AI 模型的创建归结为巨额预算或开源。

过去几年中，一些最令人印象深刻的生成模型是由大型私营公司中有影响力的研究实验室创建的。OpenAI 开发了 ChatGPT、DALL·E 和 Sora；Google 构建了 Imagen、Bard 和 Gemini；而 Meta 则创建了 Llama 和 Code Llama。

这些模型的发布方式存在不同程度的开放性。有些可以通过特定的用户界面使用，有些通过开发

者 API 访问，有些则仅作为研究报告发布，完全没有公开访问权限。在某些情况下，代码和模型权重也会发布：这些通常被称为开源发布，因为它们是运行模型所必需的关键工件。然而，出于战略原因，它们通常被隐藏起来。

与此同时，一个不断增长、充满活力和热情的社区将开源模型作为他们创造力的基石。包括研究人员、工程师、修补匠和业余爱好者在内的各类从业者，在彼此的工作基础上进行构建，提出新颖的解决方案和巧妙的想法，推动该领域向前发展，一次提交一个。其中一些想法进入了研究人员所借鉴的理论知识体系，一段时间后，使用这些想法的新模型就会问世。

大型模型，即使被隐藏，也能为社区提供灵感，他们的工作成果为整个领域服务。

这个循环之所以能够运作，是因为一些模型是开源的，可以被社区使用。发布开源模型的公司并非出于利他主义，而是因为他们发现这种策略具有经济价值。通过提供被社区采用的代码和模型，他们获得了公众的审查，包括错误修复、新想法、衍生模型架构，甚至是与发布模型配合良好的新数据集。由于所有这些贡献都基于他们发布的资产，这些公司可以迅速采用它们，从而比独自行动更快地前进。当 Meta 发布 Llama（最受欢迎的语言模型之一）时，一个繁荣的生态系统自然而然地围绕它发展起来。

包括 Meta、Stability AI（Stable Diffusion）和 Mistral AI 在内的老牌和新公司，都将不同程度的开源作为其业务战略的一部分。这与那些更愿意将商业机密保密的竞争公司的战略一样合法（即使这些公司也可以从开源社区中汲取灵感）。

在这一点上，我们想澄清的是，模型发布很少是真正开源的。与软件世界不同，源代码不足以完全理解一个机器学习系统。模型权重也不够：它们只是模型训练过程的最终输出。要完全复制现有模型，需要用于训练模型的源代码（不仅仅是建模代码或推理代码）、训练方案和参数，以及最关键的是所有用于训练的数据。这些通常都不会发布，尤其是数据。

如果能够访问这些细节，社区和公众就有可能理解模型的工作原理，探索可能影响它的偏见，并更好地评估其优势和局限性。访问权重和模型代码提供了对这些知识的不完美估计，但实际的数据会更好。除此之外，即使模型公开发布，它们通常也会附带一个不符合开源倡议定义的特殊许可证。这并不是说这些模型没有用，或者发布它们的公司没有做好事，但这是一个需要牢记的重要背景，也是我们经常说开放访问而不是开源的原因之一。

尽管如此，现在从未有比现在更好的时机来构建生成模型或使用生成模型。你不需要成为顶级研究实验室的工程师，就能提出解决你感兴趣的问题的想法，或为该领域做出贡献。我们希望这些页面在你的旅程中有所帮助！

## 总结

希望在你生成第一张图片、第一段文本和第一段音频后，你会兴奋地学习扩散模型和 Transformer 的工作原理，如何将它们适应新的用例，以及如何将它们用于不同的创意应用。尽管本章专注于高级工具，但我们将为生成之旅打下坚实的基础和直觉。让我们继续学习生成模型的原理吧！

# 第 2 章 Transformer

许多人将生成式 AI 的最新进展追溯到 2017 年引入的一类称为 Transformer 的模型。它们最著名的应用是强大的大型语言模型（LLM），如 Llama 和 GPT-4，每天有数亿人使用。Transformer 已成为现代 AI 应用的支柱，为从聊天机器人和搜索系统到机器翻译和内容摘要的一切提供动力。它们甚至超越了文本，在计算机视觉、音乐生成和蛋白质折叠等领域掀起波澜。在本章中，我们将探讨 Transformer 的核心思想及其工作原理，重点介绍最常见的应用之一：语言建模。

在我们深入探讨 Transformer 的细节之前，让我们先退一步，了解什么是语言建模。语言模型（LM）本质上是一个概率模型，它学习根据前面的或周围的单词预测序列中的下一个单词（或标记）。这样做可以捕捉语言的基本结构和模式，使模型能够生成逼真且连贯的文本。例如，给定句子“我开始我的一天吃”，LM 可能会以高概率预测下一个单词为“早餐”。

那么，Transformer 如何融入这一图景？Transformer 旨在高效且富有表现力地处理长距离依赖性和单词之间的复杂关系。例如，假设你想使用 LM 来总结一篇新闻文章，该文章可能包含数百甚至数千个单词。传统的 LM，如 RNN，在处理长上下文时表现不佳，因此摘要可能会跳过文章开头的关键细节。然而，基于 Transformer 的 LM 在这项任务中表现出色。除了高质量的生成外，Transformer 还具有其他特性，如高效的训练并行化、可扩展性和知识转移，使其在多项任务中广受欢迎且适用。这一创新的核心是一种称为自注意力的机制，它允许模型在序列的整个上下文中权衡每个单词的重要性。

为了帮助我们建立对 LM 工作原理的直觉，我们将使用与现有模型交互的代码示例，并在找到相关部分时进行描述。让我们开始吧。

## 语言模型实战

在本节中，我们将加载并交互一个现有的小型（预训练）Transformer 模型，以对其工作原理有一个高层次的理解。近年来，公司、研究实验室和开源社区发布了数千个你可以使用的开放模

型。

我们将选择一个你可以在硬件上直接运行的小型模型，但请考虑同样的原则适用于后来发布的更大（超过 100 倍！）且更强大的模型。以下是一些小型模型的好例子：

GPT-2 (137M)

该模型在 2019 年因其（当时）令人印象深刻的文本生成能力而成为头条新闻。尽管按今天的标准来看，它很小且几乎过时，但 GPT-2 展示了这些 LM 的工作原理。

Qwen2 (494M)

这是阿里巴巴的模型，属于 Qwen 系列。Qwen 系列有从 5 亿到超过 1000 亿参数的模型。

SmolLM (135M)

这是 Hugging Face 使用非常高质量数据训练的模型。作者发布了 1.35 亿、3.6 亿和 17 亿参数的模型。

第 6 章将提供更多关于为你的用例选择合适的模型的见解。现在，我们建议探索上述列表中的一个模型（或全部！）。

## 文本分词

让我们从基于初始输入生成文本的旅程开始。例如，给定短语“it was a dark and stormy”，我们希望模型能够生成后续的词语。模型不能直接接收文本作为输入；它们的输入必须是表示为数字的数据。为了将文本输入模型，我们必须首先找到一种将序列转换为数字的方法。这个过程称为分词，它是任何自然语言处理（NLP）管道中的关键步骤。

一个简单的选择是将文本拆分为单个字符，并为每个字符分配一个唯一的数字ID（图2-1）。这种方案可能对像中文这样的语言有帮助，因为每个字符都携带大量信息。在像英语这样的语言中，这会创建一个小的词汇表，并且在运行推理时，未知的标记（训练期间未找到的字符）会很少。然而，这种方法需要许多标记来表示一个字符串，这对性能不利，并且会抹去文本的一些结构和意义——这对准确性是一个缺点。每个字符携带的信息很少，使得模型难以学习文本的潜在结构。

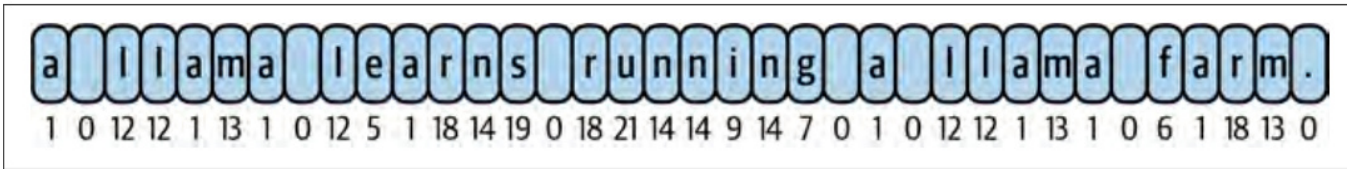


图2-1. 在字符级分词中，每个字母都有自己的ID，所有相同字母的实例都具有相同的ID。在这个例子中，ID对应于它们在字母表中的位置。

另一种方法是将文本拆分为单个单词（图2-2）。虽然这让我们能够捕捉每个标记的更多意义，但它也有缺点，即我们需要处理更多的未知单词（例如，拼写错误或俚语），我们需要处理同一单词的不同形式（例如，“run”、“runs”和“running”），并且我们可能会得到一个非常大的词汇表，对于像英语这样的语言，词汇表可能轻松超过50万个单词。

图2-2. 在单词级分词中，相同的单词总是具有相同的ID

现代分词策略在这两个极端之间取得了平衡，将文本拆分为子词，既能捕捉文本的结构和意义，又能处理未知单词和同一单词的不同形式（图2-3）。通常一起出现的字符（如最常见的单词）可以被分配一个代表整个单词或词组的标记。长或复杂的单词，或有许多屈折变化的单词，可能会被拆分为多个标记，每个标记通常代表单词的一个有意义的部分。

没有单一的最佳分词器；每个语言模型（LM）都有自己的分词器。分词器之间的区别在于支持的标记数量和分词策略。例如，GPT-2分词器平均每个单词有1.3个标记。

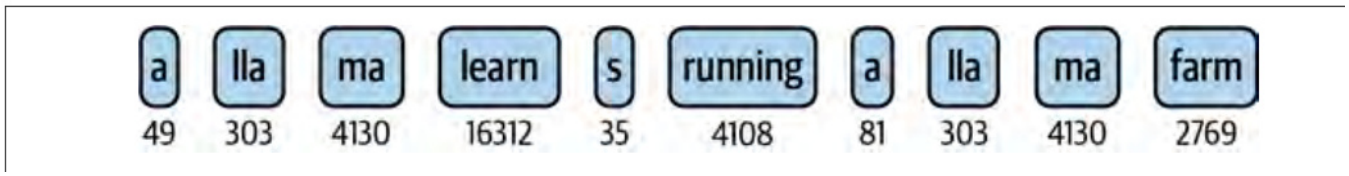



图2-3. 在这个子词分词的例子中，llama被拆分为两个标记，因为它可能不是用于创建分词器的数据中的常见单词

让我们看看Qwen分词器如何处理一个句子。我们将首先使用transformers库加载与Qwen对应的分词器。然后，我们将输入文本（也称为提示）通过分词器运行，将字符串编码为表示标记的数字。为了演示目的，我们将使用decode()方法将每个ID转换回其对应的标记：

```python
from transformers import AutoTokenizer

# 使用你想使用的模型的ID
# GPT-2 "openai-community/gpt2"
# Qwen "Qwen/Qwen2-0.5B"
# SmolLM "HuggingFaceTB/SmolLM-135M"

prompt = "It was a dark and stormy"
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2-0.5B")
input_ids = tokenizer(prompt).input_ids
input_ids

[2132, 572, 264, 6319, 323, 13458, 88]


for t in input_ids:
    print(t, "\t:", tokenizer.decode(t))

2132         : It
572         :  was
264         :  a
6319         :  dark
323         :  and
13458         :  storm
88         : y
```

如图所示，分词器将输入字符串拆分为一系列标记，并为每个标记分配一个唯一的ID。大多数单词由一个标记表示，但在使用Qwen和GPT-2分词器时，“stormy”由两个标记表示：一个用于“storm”（包括单词前的空格），另一个用于后缀“y”。这使得模型能够学习到“stormy”与“storm”相关，并且后缀“y”通常用于将名词转换为形容词。另一方面，SmolLM分词器在这个特定句子中没有拆分任何单词。每个模型通常与其自己的分词器配对，因此在使用模型时始终使用正确的分词器。本节中的三个模型的词汇表从50,000到150,000个标记不等，这使得它们能够表示几乎任何输入文本。

尽管我们通常谈论训练分词器，但这与训练模型无关。模型训练本质上是随机的（非确定性的），而我们使用统计过程训练分词器，该过程识别出对于给定数据集来说哪些子词是最佳选择。如何选择子词是分词算法的一个设计决策。因此，分词训练是确定性的。我们不会深入探讨不同的分词策略，但一些最流行的子词方法是字节级字节对编码（BPE，用于GPT-2）、WordPiece和SentencePiece。

## 预测概率

GPT-2、Qwen 和 SmolLM 是作为因果语言模型（也称为自回归模型）进行训练的，这意味着它们被训练为在给定前序 token 的情况下预测序列中的下一个 token。transformers 库提供了高级工具，使我们能够快速使用此类模型生成文本或执行其他任务。通过直接检查模型在此语言建模任务中的预测方式，有助于理解模型如何做出预测。我们首先加载模型：

```
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2-0.5B")
```

注意使用了 AutoTokenizer 和 AutoModelForCausalLM。transformers 库支持数百种模型及其对应的 tokenizer。我们不需要学习每个 tokenizer 和模型类的名称，而是使用 AutoTokenizer 和 AutoModelFor*。对于自动模型类，我们需要指定模型用于的任务，例如分类（AutoModelForSequenceClassification）或目标检测（AutoModelForObjectDetection）。在 Qwen2 的情况下，我们将使用与因果语言建模任务对应的类。使用自动类时，transformers 将根据模型的配置选择一个合适的默认类。例如，在底层，transformers 将使用 Qwen2Tokenizer 和 Qwen2ForCausalLM。

如果我们将上一节中 token 化的句子输入模型，我们会得到输入字符串中每个 token 的 151,936 个值的结果：

```
# 我们再次进行 token 化，但指定 tokenizer 返回 PyTorch 张量，这是模型期望的，
# 而不是整数列表
input_ids = tokenizer(prompt, return_tensors="pt").input_ids
outputs = model(input_ids)
outputs.logits.shape  # 每个输入 token 的输出
torch.Size([1, 7, 151936])
```

输出的第一个维度是批次数量（1，因为我们只运行了一个序列）。第二个维度是序列长度，即输入序列中的 token 数量（本例中为 7）。第三个维度是词汇表大小。我们为原始序列中的每个 token 得到了一组约 150,000 个数字。这些是原始模型输出，或称为 logits，对应于词汇表中的 token。对于每个输入 token，模型预测词汇表中每个 token 继续序列的可能性。以我们的示例句子为例，模型将预测“It”、“It was”、“It was a”等的 logits。较高的 logit 值意味着模型认为相应的 token 更有可能继续序列。表 2-1 显示了输入序列、最可能的 token ID 及其对应的 token。

Logits 是模型的原始输出（一组数字，如 [0.1, 0.2, 0.01, …]）。我们可以使用 logits 来选择最有可能继续序列的 token。然而，我们也可以将 logits 转换为概率，我们很快就会这样做。

表 2-1. 根据 Qwen2 模型，最有可能继续输入序列的 token

| Input sequence | ID of most likely next token | Corresponding token |
|---|---|---|
| It | 374 | is |
| It was | 264 | a |
| It was a | 2244 | great |
| It was a dark | 323 | and |
| It was a dark and | 13458 | storm |
| It was a dark and | 88 | y |
| It was a dark and stormy | 3729 | (let's figure this one) |

让我们关注整个提示的 logits，并学习如何预测序列的下一个词。我们可以使用 `argmax()` 方法找到具有最高值的 token 的索引：

```
final_logits = model(input_ids).logits[0, -1]  # 最后一组 logits
final_logits.argmax()  # 最大值的索引
tensor(3729)
```

3729 对应于模型认为最有可能跟随输入字符串“It was a dark and stormy”的 token 的 ID。解码这个 token，我们可以发现这个模型知道一些故事套路：

```
tokenizer.decode(final_logits.argmax())
' night'
```

因此，“ night”是最有可能的 token。考虑到我们提供的输入句子的开头，这是有道理的。模型通过一种称为自注意力的机制学习如何关注其他 token，这是 transformer 的基本构建块。直观地说，自注意力允许模型识别每个 token 对序列意义的贡献程度。

Transformer 模型包含许多这样的注意力层，每一层都专注于输入的某些方面。与启发式系统不同，这些方面或特征是在训练过程中学习的，而不是预先指定的。

现在，让我们通过使用 `topk()` 选择前 10 个值来找出其他可能的候选 token：

```
import torch

top10_logits = torch.topk(final_logits, 10)
for index in top10_logits.indices:
    print(tokenizer.decode(index))
night
evening
day
morning
winter
afternoon
Saturday
Sunday
Friday
October
```

我们需要将 logits 转换为概率，以更好地理解模型对每个预测的置信度。我们可以通过将每个值与所有其他预测值进行比较并进行归一化，使所有数字之和为 1。这正是 `softmax()` 操作所做的。以下代码使用 `softmax()` 打印出模型认为最有可能的前 10 个 token 及其相关概率：

```
top10 = torch.topk(final_logits.softmax(dim=0), 10)
for value, index in zip(top10.values, top10.indices):
    print(f"{tokenizer.decode(index):<10} {value.item():.2%}")

 night     88.71%
 evening   4.30%
 day       2.19%
 morning   0.49%
 winter    0.45%
 afternoon 0.27%
 Saturday  0.25%
 Sunday    0.19%
 Friday    0.17%
 October   0.16%
```

在继续之前，我们建议你尝试前面的代码。以下是一些你可以尝试的想法：

更改几个词

尝试更改输入字符串中的形容词（例如，“dark”和“stormy”），看看模型的预测如何变化。预测的词是否仍然是“night”？概率如何变化？

更改输入字符串 尝试不同的输入字符串，并分析模型的预测如何变化。你同意模型的预测吗？

语法

如果你提供一个语法不正确的字符串，会发生什么？模型如何处理它？查看前几个预测的概率。

## 生成文本

一旦我们知道了如何获取模型对序列中下一个 token 的预测，通过反复将模型的预测结果反馈给它自身，生成文本就变得很容易。我们可以调用 model(ids)，生成一个新的 token ID，将其添加到列表中，然后再次调用该函数。为了方便生成多个单词，transformers 自回归模型提供了一个 generate() 方法，非常适合这种情况。让我们来看一个例子：

```
output_ids = model.generate(input_ids, max_new_tokens=20)
decoded_text = tokenizer.decode(output_ids[0])

print("Input IDs", input_ids[0])
print("Output IDs", output_ids)
print(f"Generated text: {decoded_text}")
```

Input IDs tensor([ 2132, 572, 264, 6319, 323, 13458, 88])
Output IDs tensor([ 2132, 572, 264, 6319, 323, 13458, 88, 3729, 13, 576, 12884, 572, 6319, 323, 279, 9956, 572, 1246, 2718, 13, 576, 11174, 572, 50413, 1495, 323, 279])
Generated text: It was a dark and stormy night. The sky was dark and the wind was howling. The rain was pouring down and the

当我们在上一节中运行 model() 的前向方法时，它返回了词汇表中每个 token 的 logits 列表（151,936）。然后，我们必须计算概率并选择最可能的 token。generate() 方法抽象了这一逻辑。它进行了多次前向传递，反复预测下一个 token，并将其附加到输入序列中。该方法为我们提供了最终序列的 token ID，包括输入和新的 token。然后，通过 tokenizer 的 decode() 方法，我们可以将 token ID 转换回文本。

我们可以使用多种策略来执行生成。我们刚刚所做的，选择最可能的 token，被称为贪婪解码（图 2-4）。尽管这种方法简单直接，但有时可能会导致次优的结果，尤其是在生成长文本序列时。贪婪解码可能会出现问题，因为它不考虑句子的整体概率，而只关注下一个单词。例如，给定起始词“Sky”和下一个单词的选择“blue”和“rockets”，贪婪解码可能会倾向于“Sky blue”，因为“blue”在“Sky”之后最初看起来更有可能。然而，这种方法可能会忽略一个更连贯且更可能的整体序列，如“Sky rockets soar”。因此，贪婪解码有时可能会错过最可能的整体序列，导致文本生成效果不佳。

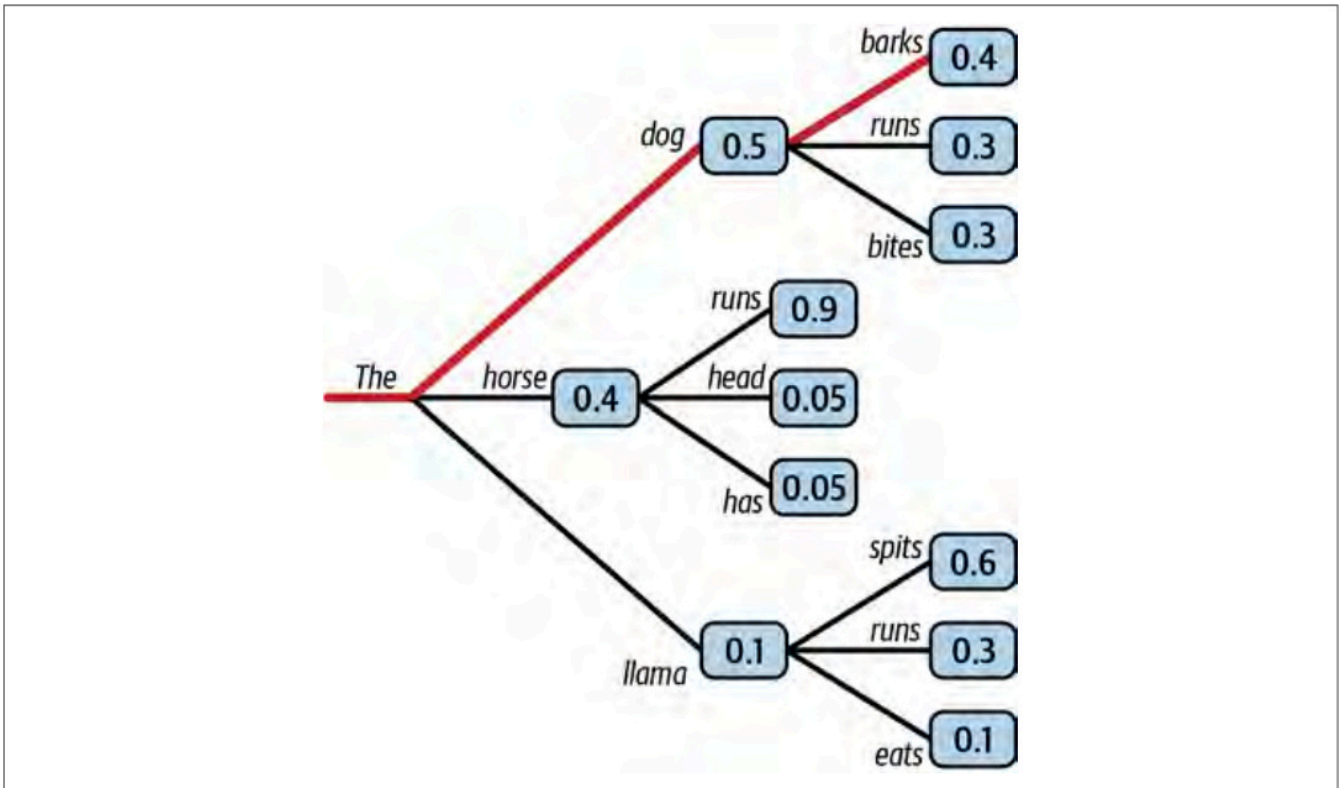



图 2-4. 这个贪婪解码示例生成了“The dog barks”，因为“dog”是最可能的第二个 token，概率为 0.5，而“barks”是“The dog”之后最可能的 token

与一次选择一个 token 不同，诸如束搜索（见图 2-5）等技术会探索序列的多个可能延续，并返回最可能的延续序列。如下面的代码示例所示，这种方法在生成过程中保留最可能的 `num_beams` 个假设，并选择最可能的一个。

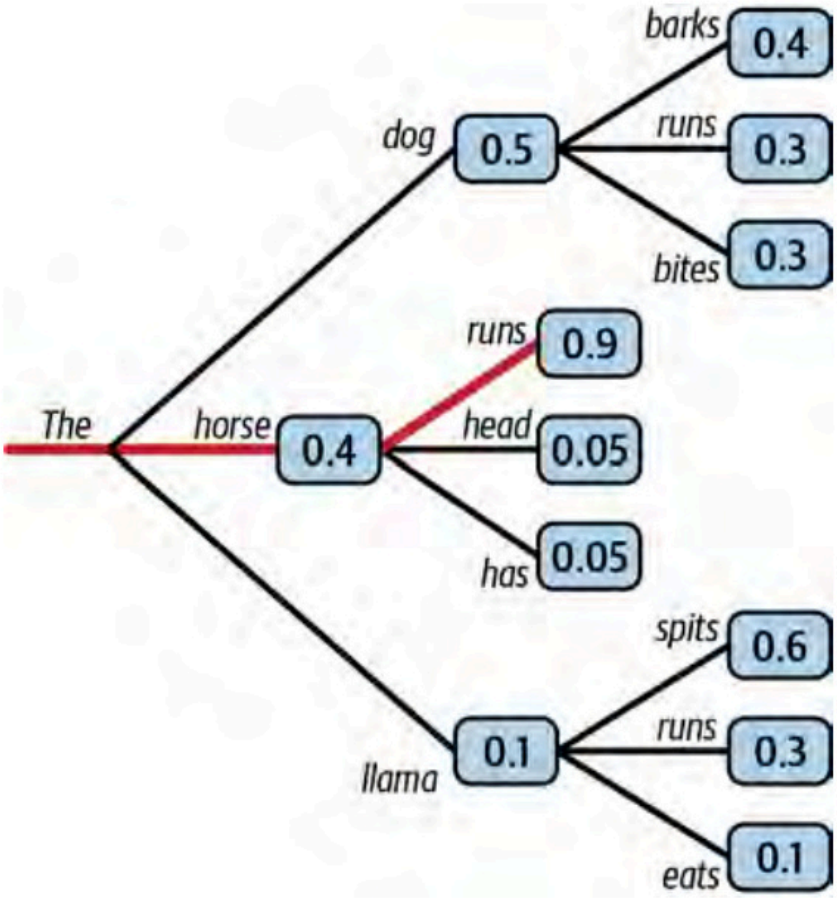



图 2-5. 这个束搜索示例找到了一个更可能的序列：“The dog barks”的总概率为 $0.5 \times 0.4 = 0.2$，而“The horse runs”的概率为 $0.4 \times 0.9 = 0.36$

```
beam_output = model.generate(
    input_ids,
    num_beams=5,
    max_new_tokens=30,
)

print(tokenizer.decode(beam_output[0]))
```

It was a dark and stormy night. The wind was howling, and the rain was pouring down. The sky was dark and gloomy, and the air was filled with the

根据模型的不同，你可能会得到一些重复的结果。虽然不经常使用，但我们可以控制多个参数来减少重复生成。让我们考虑两个例子：

`repetition_penalty`

对已经生成的 token 进行多少惩罚，以避免重复。一个好的默认值是 1.2。

`bad_words_ids`

不应生成的 token 列表（例如，避免生成冒犯性词汇）。

让我们通过惩罚重复来探索我们可以实现的效果：

```
beam_output = model.generate(
    input_ids,
    num_beams=5,
    repetition_penalty=2.0,
    max_new_tokens=38,
)

print(tokenizer.decode(beam_output[0]))
```

It was a dark and stormy night. The sky was filled with thunder and lightning, and the wind howled in the distance. It was raining cats and dogs, and the streets were covered in puddles of water.

使用哪种生成策略？正如在机器学习中经常遇到的情况一样，这取决于具体情况。当文本的期望长度可以预测时，束搜索效果很好。这对于摘要或翻译等任务来说是如此，但对于开放式生成则不然，因为输出长度可能会有很大差异，导致重复。虽然我们可以抑制模型以避免重复，但这样做也可能导致其表现更差。还要注意，束搜索会比贪婪搜索慢，因为它需要同时为多个束运行推理，这对于大型模型来说可能是一个问题。

当我们使用贪婪搜索和束搜索生成时，我们推动模型生成具有高概率下一个单词分布的文本，如图 2-6.3 所示。有趣的是，高质量的人类语言并不遵循类似的分布。人类文本往往更加不可预测。一篇关于这一反直觉观察的优秀论文的作者推测，人类语言不喜欢可预测的单词——人们会优化以避免陈述显而易见的内容。该论文提出了一种称为核采样的方法。

在讨论核采样之前，让我们先讨论采样。通过采样，我们从下一个 token 的概率分布中采样来选择下一个 token。这意味着采样不是一个确定性的生成过程。如果下一个可能的 token 是“night” $(60\%)$ 、“day” $(35\%)$ 和“apple” $(5\%)$ ，而不是选择“night”（通过贪婪搜索），我们将从分布中采样。换句话说，即使“apple”是一个低概率 token 并导致无意义的生成，也会有 $5\%$

的机会选择它。采样避免了创建重复的文本，从而生成更多样化的结果。

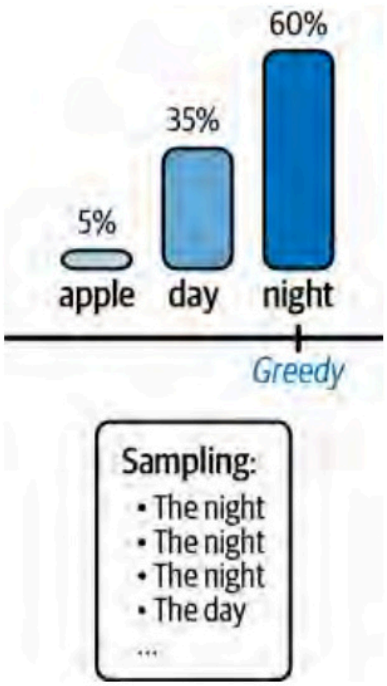



图 2-6. 贪婪生成将始终选择最可能的下一个 token，而采样将从概率分布中采样选择下一个 token

在 transformers 中，通过使用 `do_sample` 参数进行采样：

```
from transformers import set_seed

# 设置种子确保每次运行此代码时都能得到相同的结果
set_seed(70)
sampling_output = model.generate(
    input_ids,
    do_sample=True,
    max_new_tokens=34,
    top_k=0,  # 我们稍后会回到这个参数
)

print(tokenizer.decode(sampling_output[0]))
```

```
It was a dark and stormy night. Kevin said he was going to stay up all night, staring at the cloudless stars, wondering, what if I lost my dream.He'd been teasing her about
```

我们可以在采样之前操纵概率分布，使用温度参数使其更尖锐或更平坦。大于 1 的温度将增加分布的随机性，我们可以用它来鼓励生成不太可能的 token。0 到 1 之间的温度将减少随机性，增加更可能 token 的概率，并避免可能过于意外的预测。温度为 0 时，所有概率将转移到最可能的下一个 token，这相当于贪婪解码，如图 2-7 所示。

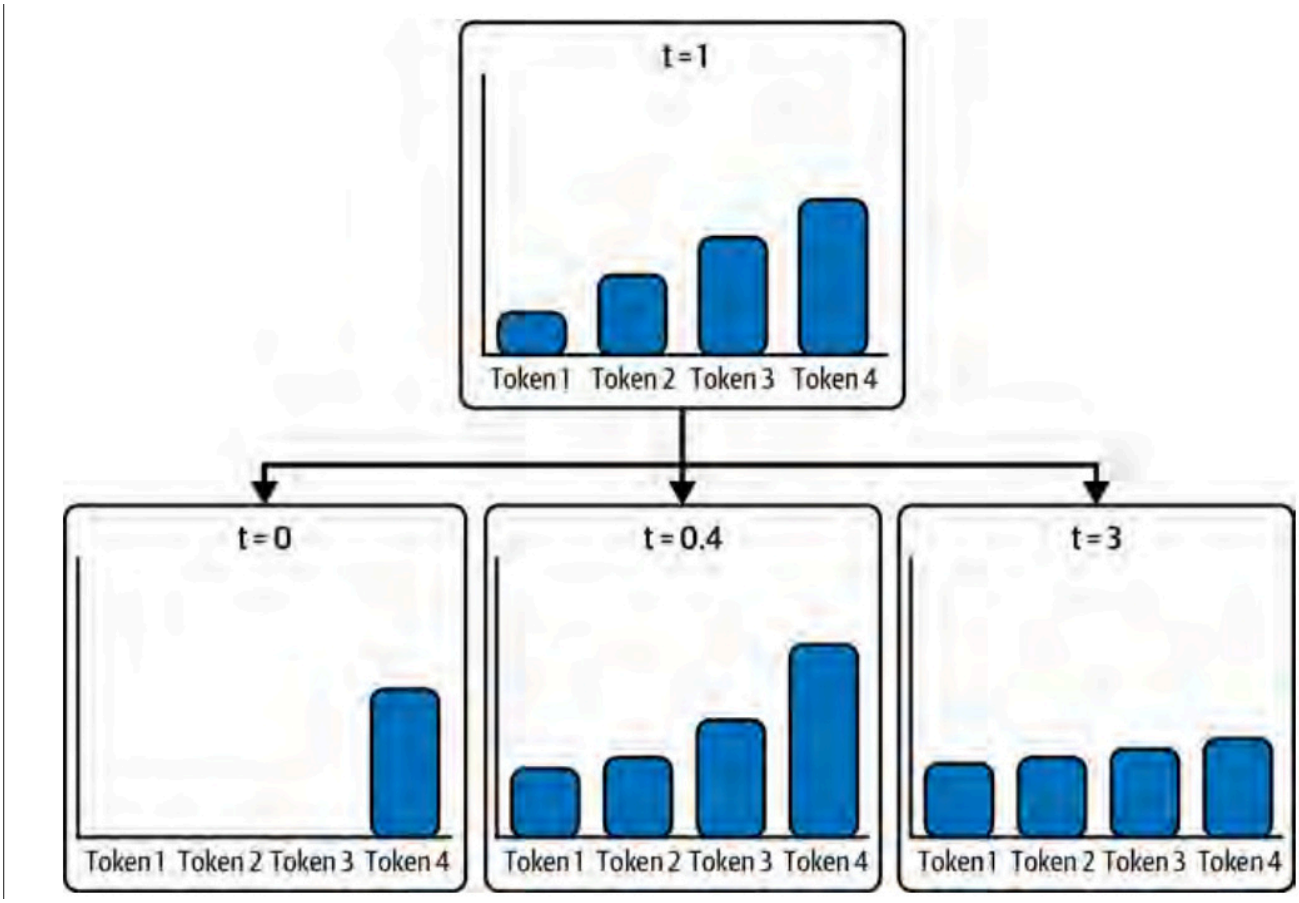



图 2-7. 温度对 token 概率分布的影响

比较以下示例中温度参数对生成文本的影响：

```
sampling_output = model.generate(
    input_ids,
    do_sample=True,
    temperature=0.4,
    max_new_tokens=40,
    top_k=0,
)

print(tokenizer.decode(sampling_output[0]))
```

It was a dark and stormy night in 1878. The only light was the moon, and the only sound was the distant roar of the thunder. The only thing that could be heard was the sound of the storm

```
sampling_output = model.generate(
    input_ids,
    do_sample=True,
    temperature=0.001,
    max_new_tokens=40,
    top_k=0,
)

print(tokenizer.decode(sampling_output[0]))
```

那是一个风雨交加的夜晚。天空漆黑，狂风呼啸。大雨倾盆而下，闪电划破天际。天空漆黑，狂风呼啸。大雨倾盆而下

```
sampling_output = model.generate( input_ids, do_sample=True, temperature=3.0,
    max_new_tokens=40, top_k=0,
)

print(tokenizer.decode(sampling_output[0]))
```

那是一个风雨交加的清晨一步 女人们都 BL 咻任何时候 نب صت attendees*sinruitmentโลก uresindi
ambassadors eventData 原来是 exaлENCE hemisphere worldsω.Anyar 久‡ Sous dapat HVışísdía.inventory emptiedfuncpping {\Sex

嗯，第一个测试比第二个测试连贯得多。第二个测试使用了非常低的温度，结果是重复的（类似于贪婪解码）。最后，第三个样本使用了极高的温度，生成了无意义的文本。

你可能注意到的一个参数是 `top_k`。它是什么？Top-K 采样是一种简单的采样方法，其中只考虑 $K$ 个最可能的下一个 token。例如，使用 `top_k=5`，生成方法会首先过滤出最可能的五个 token，并重新分配概率，使它们的总和为 1：

```
sampling_output = model.generate( input_ids, do_sample=True, max_new_tokens=40,
    top_k=5,

print(tokenizer.decode(sampling_output[0]))
```

那是一个风雨交加的夜晚，纽约城正处于一场猛烈风暴的边缘。天空被明亮的红色和橙色所覆盖。这是一个征兆，但风暴已经来临

嗯……这可能会更好。Top-K 采样的一个问题是，实际中相关候选 token 的数量可能会有很大差异。如果我们定义 `top_k=5`，一些分布仍然会包括概率非常低的 token，而其他分布则只包含高概率的 token。

我们将探讨的最后一个生成策略是 Top-p 采样（也称为核采样）。与采样 $K$ 个最高概率的单词不同，我们将使用所有最可能的单词，其累积概率超过给定值。如果我们使用 `top_p=0.94`，我们首先会过滤掉只保留累积概率为 0.94 或更高的最可能单词，如图 2-8 所示。然后我们重新分配概率并进行常规采样。让我们看看实际操作中的效果。

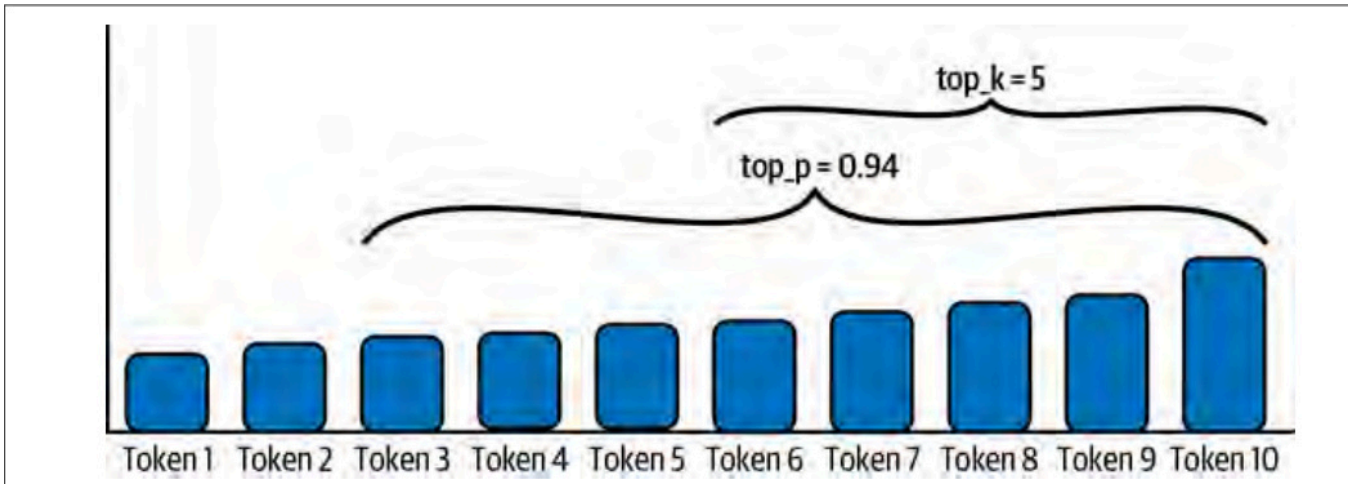



图 2-8. top_k 和 top_p 对 token 概率分布的影响。使用 `top_k=5`，只考虑五个最可能的 token。使用 `top_p=0.94`，我们包括所有 token，直到它们的累积概率达到 0.94。

在 transformers 中，我们可以使用 Top-p 采样并通过 `top_p` 参数修改概率：

```
sampling_output = model.generate( input_ids, do_sample=True, max_new_tokens=40,
    top_p=0.94, top_k=0,
)

print(tokenizer.decode(sampling_output[0]))
```

那是一个风雨交加的夜晚，在 Morrowind 的天空下，一个特别无情的战士决定这些在 Carrabine 的粗心游客应该以最大的残忍对待。这次重要行动的首领被任命为

Top-K 和 Top-p 在实践中都很常用。它们甚至可以结合使用，以过滤掉低概率的单词，但具有更多的生成控制。随机生成方法的问题是生成的文本不一定具有连贯性。

我们已经探讨了三种生成方法：贪婪搜索、束搜索解码和采样（通过温度、Top-K 和 Top-p 提供进一步的控制）。这些方法很多！如果你对模型的生成感到不满意，请考虑到这是一个拥有数亿参数的模型；它能够生成连贯的文本已经令人印象深刻了！新模型拥有数十亿甚至数千亿个参数，并使用更高质量的数据进行训练，因此切换到更现代或更大的模型将带来更好的结果。你可以在线找到一个交互式演示，可视化温度、`top_p` 和 `top_k` 对生成分布的影响。

如果你想进一步实验生成，以下是一些建议：

- 尝试不同的参数值。增加束的数量如何影响生成的质量？如果你减少或增加 `top_p` 值会发生什么？
- 减少束搜索中重复的一种方法是引入对 n-gram（$n$ 个单词的序列）的惩罚。这可以通过 `no_repeat_ngram_size` 来配置，它避免重复相同的 n-gram。例如，如果你使用 `no_repeat_ngram_size=4`，生成将永远不会包含完全相同的四个连续单词。
- Top-K 可能会导致丢弃高质量的 token，而 Top-p 可能会导致包括一些低概率的 token。对于更动态的方法，你可以使用 Min-P（`min_p`），它将 `min_p` 乘以最高 token 的概率，然后只包括高于该百分比的 token。换句话说，Min-P 根据最高 token 的概率定义了一个动态阈值。
- 一种较新的方法，对比搜索，可以生成长且连贯的输出，同时避免重复。这是通过同时考虑模型预测的概率和与上下文的相似性来实现的。这可以通过 `penalty_alpha` 和 `top_k` 来控制。

如果这一切听起来太经验主义，那是因为它确实是。生成是一个活跃的研究领域，新的论文提出了不同的建议，例如更复杂的过滤。我们将在最后一章简要讨论这些。没有单一的规则适用于所有模型，因此始终重要的是尝试不同的技术。

## 零样本泛化

生成语言是 transformers 的一个有趣且令人兴奋的应用，但编写关于独角兽的假文章并不是它们如此受欢迎的原因。为了很好地预测下一个 token，这些模型必须学习相当多的关于世界的知识。我们可以利用这一点来执行各种任务。例如，与其训练一个专门用于翻译的模型，我们可以用一个足够强大的语言模型（LM）来提示，输入如下：

将以下句子从英语翻译成法语：输入：The cat sat on the mat.
翻译：

我在 GitHub Copilot 激活的情况下输入了这个例子，它友好地建议了“Le chat était assis sur le tapis”作为提示的延续——这完美地展示了 LM 如何执行未明确训练的任务。模型越强大，它无需额外训练就能执行的任务就越多。这种灵活性使得 transformers 非常强大，并使其在近年来如此受欢迎。

为了亲自验证这一点，让我们使用 Qwen 作为分类模型。具体来说，我们将电影评论分类为正面或负面——这是 NLP 领域的一个经典基准任务。我们将使用零样本方法使事情变得有趣，这意味着我们不会为模型提供任何标注数据。相反，我们将用评论的文本提示模型，并要求它预测情感。

我们可以通过多种方式使用生成模型作为分类器。通常，我们首先将电影评论插入到提示模板中，为模型提供上下文。这个提示模板可以指示模型简单地返回评论的情感（并要求将其限制为正面或负面）。另一种技巧，特别是对于像 GPT-2 或小型 Qwen 这样的小模型，是查看它对下一个 token 的预测，并找出哪个可能的 token 被分配了更高的概率：正面还是负面？让我们采用这种方法，并找到与这些 token 对应的 ID：

```
// 检查单词 ' positive' 和 ' negative' 的 token ID
//（注意单词前的空格）
tokenizer.encode(" positive"), tokenizer.encode(" negative")
```

([6785], [8225])

一旦我们有了 ID，我们就可以运行模型推理并根据概率生成标签：

```
def score(review):
    """预测评论是正面还是负面

    此函数通过一些巧妙的提示来预测评论是正面还是负面。它查看 ' positive' 和
    ' negative' token 的logits，并返回得分最高的标签。"""

    prompt = f"""问题：以下评论对电影是正面还是负面？评论：{review} 答案："""

    input_ids = tokenizer(prompt, return_tensors="pt").input_ids
    final_logits = model(input_ids).logits[0, -1]

    if final_logits[6785] > final_logits[8225]:
        print("Positive")
    else:
        print("Negative")
```

对提示进行 tokenize。

获取词汇表中每个 token 的 logits。请注意，我们使用的是 model() 而不是 model.generate()，因为 model() 返回词汇表中每个 token 的 logits，而 model.generate() 只返回选择的 token。

检查正面 token 的 logit 是否高于负面 token 的 logit。

我们可以在一些假评论上尝试这个零样本分类器，以评估它的表现：

```
score("This movie was terrible!")
```

Negative

```
score("That movie was great!")
```

Positive

```
score("A complex yet wonderful film about the depravity of man")  # 一个错误
```

Negative

在本书的 GitHub 仓库中，你会找到一个标注评论的数据集和代码，用于评估这种零样本方法的

准确性。你能调整提示模板以提高模型的性能吗？你能想到可以使用类似方法执行的其他任务吗？

最近模型的零样本能力已经改变了游戏规则。随着模型的改进，它们可以开箱即用地执行更多任务，使它们更易于访问和使用，并减少了对每个任务专门模型的需求。

## 少样本泛化

尽管 ChatGPT 的发布和对完美提示的追求，零样本泛化（或提示调优）并不是让强大的 LM 执行任意任务的唯一方法。

零样本是一种称为少样本泛化的技术的极端应用，在这种技术中，我们为 LM 提供一些关于我们想要它执行的任务的示例，然后要求它为我们提供类似的答案。我们不是训练模型，而是展示一些示例，通过增加后续文本遵循与我们的提示相同的结构和模式的概率来影响生成。让我们尝试一个例子。除了提供示例外，提供模型应该做什么的简短描述，例如“将英语翻译成西班牙语”，将有助于生成更高质量的文本：

```
prompt = """ 将英语翻译成西班牙语：

英语：I do not speak Spanish.
西班牙语：No hablo español.

英语：See you later! 西班牙语：⍰Hasta luego!

英语：Where is a good restaurant? 西班牙语：¿Dónde hay un buen restaurante?

英语：What rooms do you have available? 西班牙语：¿Qué habitaciones tiene disponibles?


英语：I like soccer
西班牙语："""
inputs = tokenizer(prompt, return_tensors="pt").input_ids
output = model.generate(inputs, max_new_tokens=10)

print(tokenizer.decode(output[0]))
```

将英语翻译成西班牙语：

英语：I do not speak Spanish.

西班牙语：No hablo español.

英语：See you later! 西班牙语：¡Hasta luego!

英语：Where is a good restaurant? 西班牙语：¿Dónde hay un buen restaurante?

英语：What rooms do you have available? 西班牙语：¿Qué habitaciones tiene disponibles?

英语：I like soccer 西班牙语：Me gusta el fútbol

英语：

我们陈述了我们想要实现的任务，并提供了四个示例来为模型设置上下文。因此，这是一个四样本泛化任务。然后我们要求模型生成更多文本以遵循模式并提供请求的翻译。以下是一些可以探索的想法：

- 这能用更少的示例工作吗？
- 没有任务描述它能工作吗？
- 其他任务呢？
- GPT-2 和 SmolLM 在这种设置下表现如何？

> GPT-2 由于其大小和训练过程，在少样本任务上表现不佳，在零样本泛化上甚至更差。你怎么可能像上一节那样用它进行情感分类呢？我们有点作弊：我们没有查看模型生成的文本，而是检查了“正面”的概率是否大于“负面”的概率。理解模型的工作原理可以解锁强大的应用，即使使用小模型也是如此。记住要思考你的问题；不要害怕探索。

GPT-2 和 Qwen 0.5B 是基础模型的例子。一些类似 Qwen 风格的基础模型具有我们可以在推理时使用的零样本和少样本能力。另一种方法是微调模型：我们采用基础模型并在特定领域或任务的数据上继续训练一段时间。我们并不总是需要世界上最强大的模型所展示的极端泛化能力；如果你只想解决一个特定的任务，通常更便宜且更好的是微调并部署一个专门用于单个任务的较小模型。

同样重要的是要注意，基础模型不是对话式的；尽管你可以编写一个漂亮的提示来帮助使用基础模型制作聊天机器人，但通常更方便的是用对话数据微调基础模型，从而提高模型的对话能力。这正是我们将在第 6 章中做的事情。最近的 LLM 发布往往包括一个基础模型和一个具有对话能力的官方模型。在小型 Qwen 模型的情况下，这将是 Qwen2-0.5B-Instruct。

## Transformer 模块

在我们使用语言模型进行简短实验之后，我们准备介绍一个基于 Transformer 的语言生成模型的架构图，如图 2-9 所示。

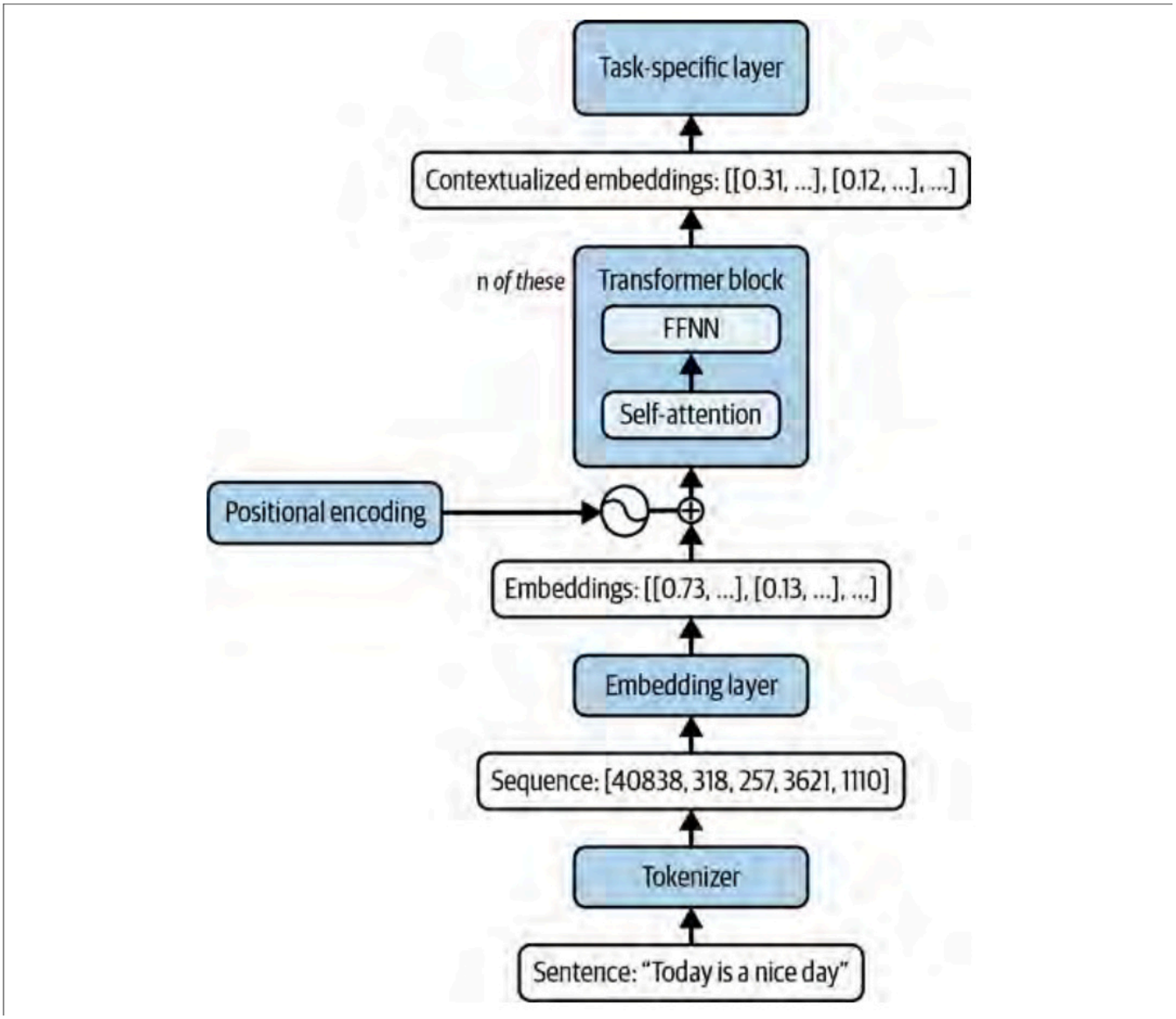



图 2-9. 基于 Transformer 的语言模型架构

涉及的高级组件包括以下内容：

### 分词

输入文本被分解为单独的 token。每个 token 都有一个对应的 ID，用于索引 token 嵌入。

### 输入 token 嵌入

token 被表示为称为嵌入的向量。这些嵌入作为数值表示，捕捉每个 token 的基本信息。你可以将向量视为一个（长的）数字列表，其中每个数字对应于 token 意义的某个特定方面。在训练过程中，模型学习如何将每个 token 映射到其对应的嵌入。无论 token 在输入序列中的位置如何，每个 token 的嵌入始终相同。

### 位置编码

Transformer 模型没有顺序的概念，因此我们需要用位置信息来丰富 token 嵌入。这是通过向 token 嵌入添加位置编码来实现的。该编码是一组向量，用于编码输入序列中每个 token 的位置。这使得模型能够根据 token 在序列中的位置来区分它们，这在同一个 token 出现在不同位置时可能具有不同含义的情况下非常有用。

### Transformer 模块

Transformer 模型的核心是 Transformer 模块。Transformer 的强大之处在于堆叠多个模块，使模型能够学习输入 token 之间越来越复杂和抽象的关系。它由两个主要组件组成：

### 自注意力机制

该机制允许模型在整个序列的上下文中权衡每个 token 的重要性。它帮助模型理解输入中 token 之间的关系。自注意力机制是 Transformer 能够处理长距离依赖和单词之间复杂关系的关键，并有助于生成连贯且上下文合适的文本。

### 前馈神经网络

自注意力的输出通过前馈神经网络传递，进一步细化输入序列的表示。

### 上下文嵌入

Transformer 模块的输出是一组上下文嵌入，捕捉输入序列中 token 之间的关系。与每个 token 固定的输入嵌入不同，上下文嵌入在 Transformer 模型的每一层根据 token 之间的关系进行更新。嵌入捕捉了 token 在其出现的上下文中的丰富而复杂的语义信息。

### 预测

一个额外的层将最终表示处理为任务相关的最终输出。在文本生成的情况下，这涉及一个线性层，将上下文嵌入映射到词汇空间，然后通过 softmax 操作预测序列中的下一个 token。

当然，这是对 Transformer 架构的简化。深入探讨自注意力机制的工作原理或 Transformer 模块的内部结构超出了本书的范围。然而，理解 Transformer 模型的高级架构有助于掌握这些模型的工作原理以及如何将它们应用于各种任务。这种架构使 Transformer 在各种任务和领域中取得了前所未有的性能，你会在本书的其余部分以及整个学科中反复看到它们的身影。

# Transformer 模型谱系

在本章的开头，我们使用 Qwen 进行了自回归生成文本的实验。Qwen 是一个基于解码器的 Transformer 示例，它有一个单一的 Transformer 模块堆栈来处理输入序列。这是当今流行的方法，但多年来也开发了其他架构。本节简要概述了 Transformer 模型的谱系。

## 序列到序列任务

最初的 Transformer 论文使用了一种看似更复杂的架构，称为编码器-解码器架构（图 2-10）。尽管编码器-解码器架构在 2023 年之前一直很流行，但在大多数研究实验室中已被解码器取代。

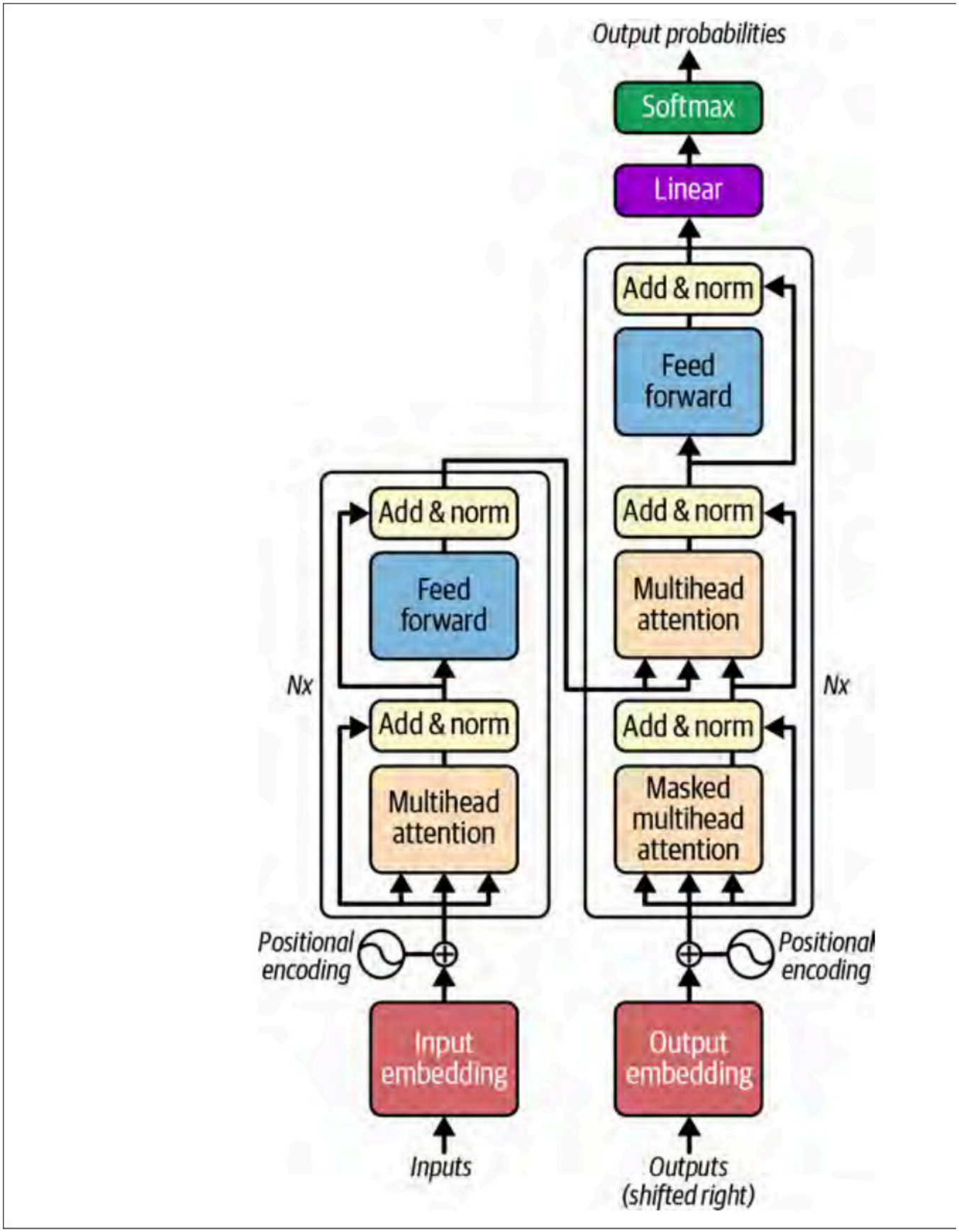

Output probabilities
Softmax
Linear
Add & norm
Feed forward
Add & norm
Multihead attention
Add & norm
Masked multihead attention
Add & norm
Feed forward
Add & norm
Multihead attention
Add & norm
Multihead attention
Nx
Nx
Positional encoding
Positional encoding
Input embedding
Output embedding
Inputs
Outputs (shifted right)

图 2-10. 编码器-解码器 Transformer（改编自原始 Transformer 论文中的图像）

Transformer 论文以机器翻译为例的序列到序列任务。当时，机器翻译的最佳结果是由 RNN（如长短期记忆网络（LSTM）和门控循环单元（GRU））实现的——如果你不熟悉它们，不用担心。论文通过专注于注意力方法展示了更好的结果，并表明可扩展性和训练要容易得多。出色的性能、稳定的训练和易于扩展等重要因素是 Transformer 起飞并被适应于多个任务的原因，下一节将更深入地探讨这一点。

在编码器-解码器模型中，如论文中描述的原始 Transformer 模型，一个称为编码器的 Transformer 模块堆栈将输入序列处理为一组丰富的表示，然后将其输入到另一个称为解码器的 Transformer 模块堆栈中，解码器将其解码为输出序列。这种将一个序列转换为另一个序列的方法称为序列到序列（seq2seq），自然非常适合翻译、摘要或问答等任务。

例如，你将一个英语句子输入翻译模型的编码器，编码器生成一个捕捉输入意义的丰富嵌入。然后解码器使用该嵌入生成相应的法语句子。生成过程在解码器中逐个 token 进行，正如我们在本章前面生成序列时所看到的那样。然而，每个连续 token 的预测不仅由正在生成的序列中的前一个 token 提供信息，还由编码器的输出提供信息。

编码器侧的输出被纳入解码器堆栈的机制称为交叉注意力。它类似于自注意力，只是输入中的每个 token（由解码器处理的序列）关注的是编码器的上下文，而不是其序列中的其他 token。交叉注意力层与自注意力层交错，允许解码器在其序列中使用上下文以及来自编码器的信息。

在 Transformer 论文之后，现有的序列到序列模型（如 Marian NMT）将这些技术作为其架构的核心部分。使用这些思想开发了新的模型。一个值得注意的模型是 BART（双向自回归 Transformer 的缩写）。在预训练期间，BART 破坏输入序列并尝试在解码器输出中重建它们。之后，BART 针对其他生成任务（如翻译或摘要）进行微调，利用预训练期间获得的丰富序列表示。顺便说一下，输入破坏是扩散模型背后的关键思想之一，我们将在第 4 章讨论。

## 仅编码器模型

正如我们所讨论的，最初的Transformer模型基于编码器-解码器架构，这种架构在BART或T5等模型中得到了进一步探索。此外，编码器或解码器可以独立训练和使用，从而形成了不同的Transformer家族。本章的前几节探讨了仅解码器或自回归模型。这些模型专门用于使用我们描述的技术进行文本生成，并展示了令人印象深刻的性能，如ChatGPT、Claude、Llama和Gemma所展示的那样。

另一方面，编码器模型专门用于从文本序列中获取丰富的表示，并可用于分类任务或为检索系统中的大量文档准备语义嵌入。最著名的Transformer编码器模型可能是BERT，它引入了掩码语

言模型（MLM）目标，后来被BART采用并进一步探索。

因果语言建模根据前面的标记预测下一个标记——这就是我们使用Qwen所做的。模型只能关注给定标记左侧的上下文。编码器模型中使用的另一种方法称为掩码语言建模（MLM）。MLM在著名的BERT论文中提出，预训练模型以学习“填空”。给定输入文本，我们随机掩码一些标记，模型必须预测被掩码的标记，如图2-11所示。与因果语言建模不同，MLM使用掩码标记左侧和右侧的序列，因此BERT名称中的“B”代表双向。这有助于创建给定文本的强表示。在底层，这些模型使用Transformer架构的编码器部分。

我们刚刚讨论了编码器-解码器和仅解码器架构。一个常见的问题是，如果像Qwen和Llama这样的仅解码器模型可以显示出良好的结果，为什么还需要编码器-解码器模型来进行翻译等任务。编码器-解码器模型旨在将整个输入序列翻译为输出序列，使其非常适合翻译。相比之下，仅解码器模型专注于预测序列中的下一个标记。最初，像GPT-2这样的仅解码器模型在零样本学习场景中的能力不如像GPT-4这样的最新模型，但这不仅仅是因为缺少编码器。像GPT-4这样的高级模型在零样本能力上的提升还得益于更大的训练数据、更好的训练技术和增加的模型规模。虽然seq2seq模型中的编码器在理解输入序列的完整上下文方面起着至关重要的作用，但仅解码器模型的进步使其更加有效和多功能，甚至适用于传统上依赖seq2seq模型的任务。

让我们看一些代码。与其使用我们之前使用的AutoModel和AutoTokenizer类，不如介绍一个更高级的transformers API，称为pipeline。这个API允许你轻松加载用于给定任务的模型。pipeline API负责所有的预处理和后处理，因此，它是快速尝试模型的好方法：

```
from transformers import pipeline
fill_masker = pipeline("fill-mask", model="bert-base-uncased")
fill_masker("The [MASK] is made of milk.")
```

```
[{'score': 0.19546695053577423, 'token': 9841, 'token_str': 'dish', 'sequence':
    'the dish is made of milk.'},
{'score': 0.1290755718946457, 'token': 8808, 'token_str': 'cheese', 'sequence':
    'the cheese is made of milk.'},
{'score': 0.10590697824954987, 'token': 6501, 'token_str': 'milk',
'sequence': 'the milk is made of milk.'},
{'score': 0.04112089052796364, 'token': 4392, 'token_str': 'drink',
'sequence': 'the drink is made of milk.'},
{'score': 0.03712352365255356, 'token': 7852, 'token_str': 'bread', 'sequence':
    'the bread is made of milk.'}]
```

很高兴知道牛奶是由牛奶制成的！在底层发生了什么？编码器接收输入序列并为每个标记生成上下文表示。这种表示是一个数字向量，它捕捉了标记在整个序列上下文中的含义。编码器通常后面跟着一个特定任务的层，该层使用这些表示来执行分类、问答或掩码语言建模等任务，如图2-11所示。编码器被训练生成对需要良好理解输入的任务有用的表示。

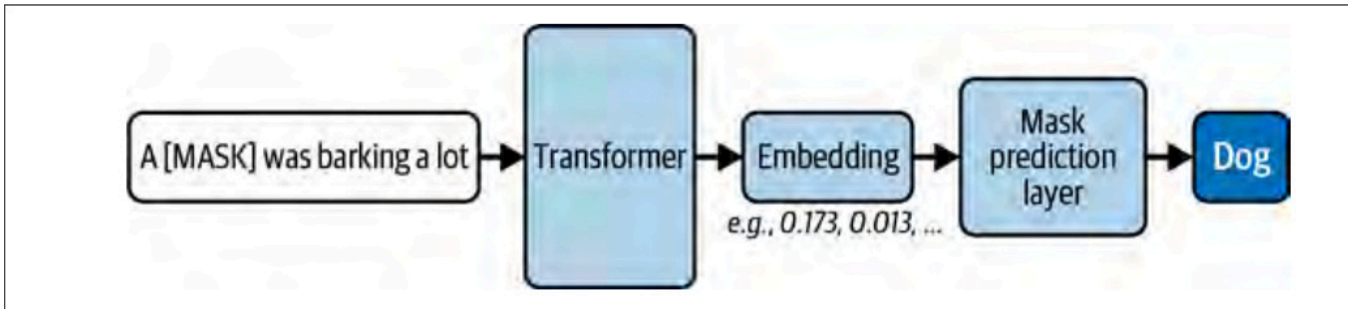



图2-11. 编码器模型输出语义嵌入，可用于解决预测序列中间标记等任务

在仅编码器、仅解码器和编码器-解码器模型之间，公司和研究实验室发布了大量新的开源和闭源语言模型，如GPT-4、Mistral、Falcon、Llama、Qwen、Yi、Claude、Bloom、Gemma等数百个。图2-12展示了Transformer模型的非详尽家谱，展示了它们截至2024年对NLP领域的丰硕影响。

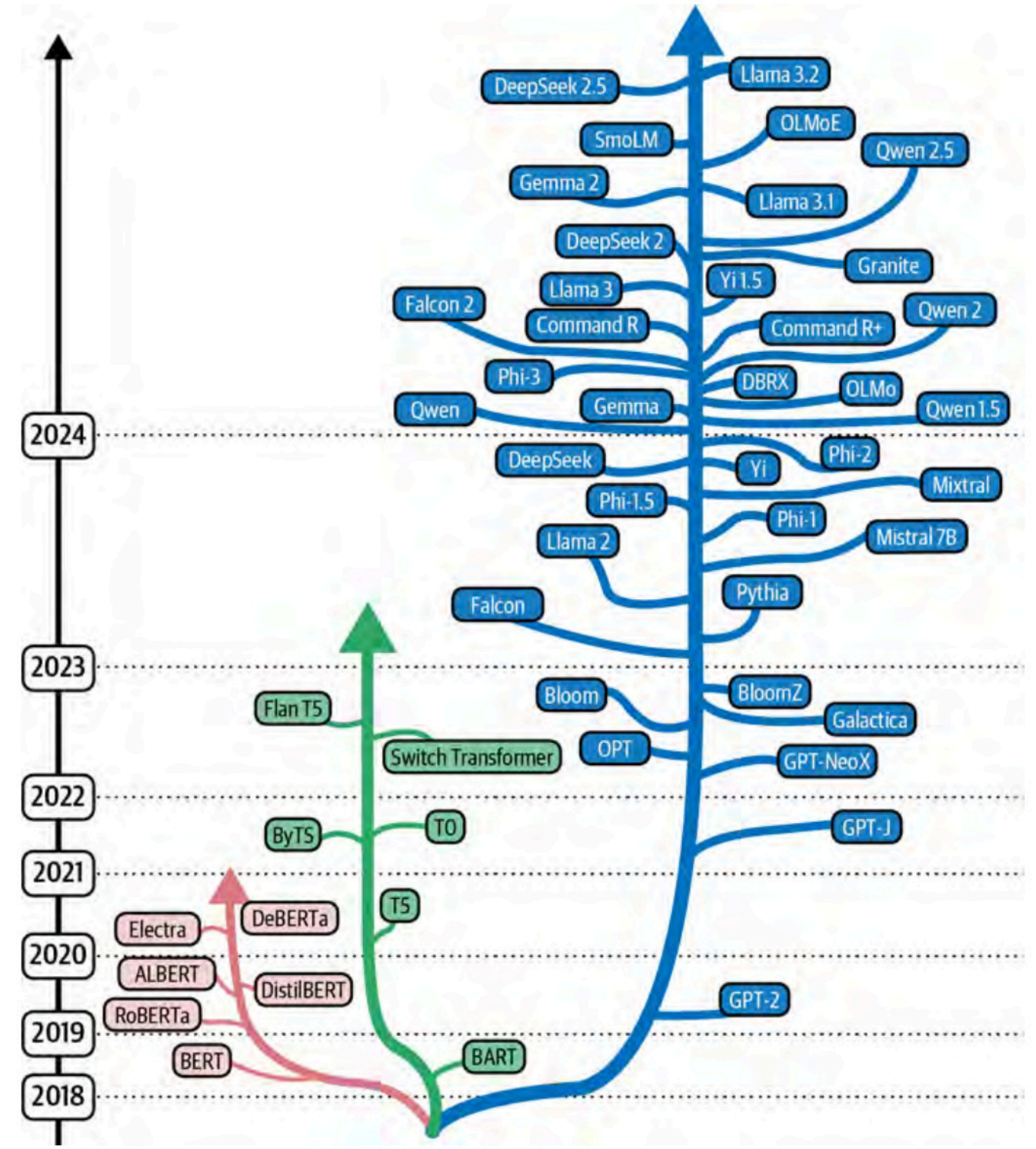



图2-12. 开源仅编码器（红色）、编码器-解码器（绿色）和仅解码器（蓝色）模型发布的时间线

## 预训练的力量

使用现有模型非常强大。Transformer 模型在许多其他语言任务中表现出色，例如文本分类、机

器翻译和基于输入文本回答问题。为什么 Transformer 如此有效?

第一个关键点是对注意力机制的使用，正如本章所暗示的那样。注意力机制使 Transformer 模型能够处理长序列并学习长距离关系。换句话说，Transformer 可以估计某些 token 与其他 token 的相关性。

第二个关键方面是它们的扩展能力。Transformer 架构的实现针对并行化进行了优化，研究表明这些模型可以扩展到处理高复杂性和大规模的数据集。尽管最初是为文本数据设计的，但 Transformer 架构足够灵活，可以支持不同的数据类型并处理不规则的输入。

第三个关键点是预训练和微调的能力。传统任务方法，如电影评论分类，受限于标注数据的可用性。模型会从头开始在大量标注样本上进行训练，试图直接从输入文本中预测标签。这种方法通常被称为监督学习。然而，它有一个显著的缺点：需要大量标注数据才能有效训练。这是一个问题，因为标注数据既昂贵又耗时。在许多领域，甚至可能没有任何可用的数据。

为了解决对大量标注数据的需求，研究人员开始寻找一种方法，在现有数据上预训练模型，然后针对特定任务进行微调（或调整）。这种方法被称为迁移学习，是现代机器学习在许多领域（如自然语言处理和计算机视觉）的基础。自然语言处理中的早期工作主要集中在为语言模型预训练阶段寻找特定领域的语料库，但像 ULMFiT 这样的论文表明，即使在通用文本（如维基百科）上进行预训练，当模型在下游任务（如情感分析或问答）上进行微调时，也能产生令人印象深刻的结果。这为 Transformer 的崛起奠定了基础，Transformer 被证明非常适合学习丰富的语言表示。

预训练的思想是在大量未标注的数据集上训练模型，然后将其微调到新的目标任务，这需要更少的标注数据（图 2-13）。在进入自然语言处理之前，迁移学习已经在构成现代计算机视觉核心的卷积神经网络（CNN）中取得了巨大成功。在这种情况下，首先在分类任务中使用大量标注图像训练一个大型模型。通过这个过程，模型学习到可以在不同但相关的问题上利用的通用特征。例如，我们可以在数千个类别上预训练一个模型，然后将其微调以分类图片是否为热狗。

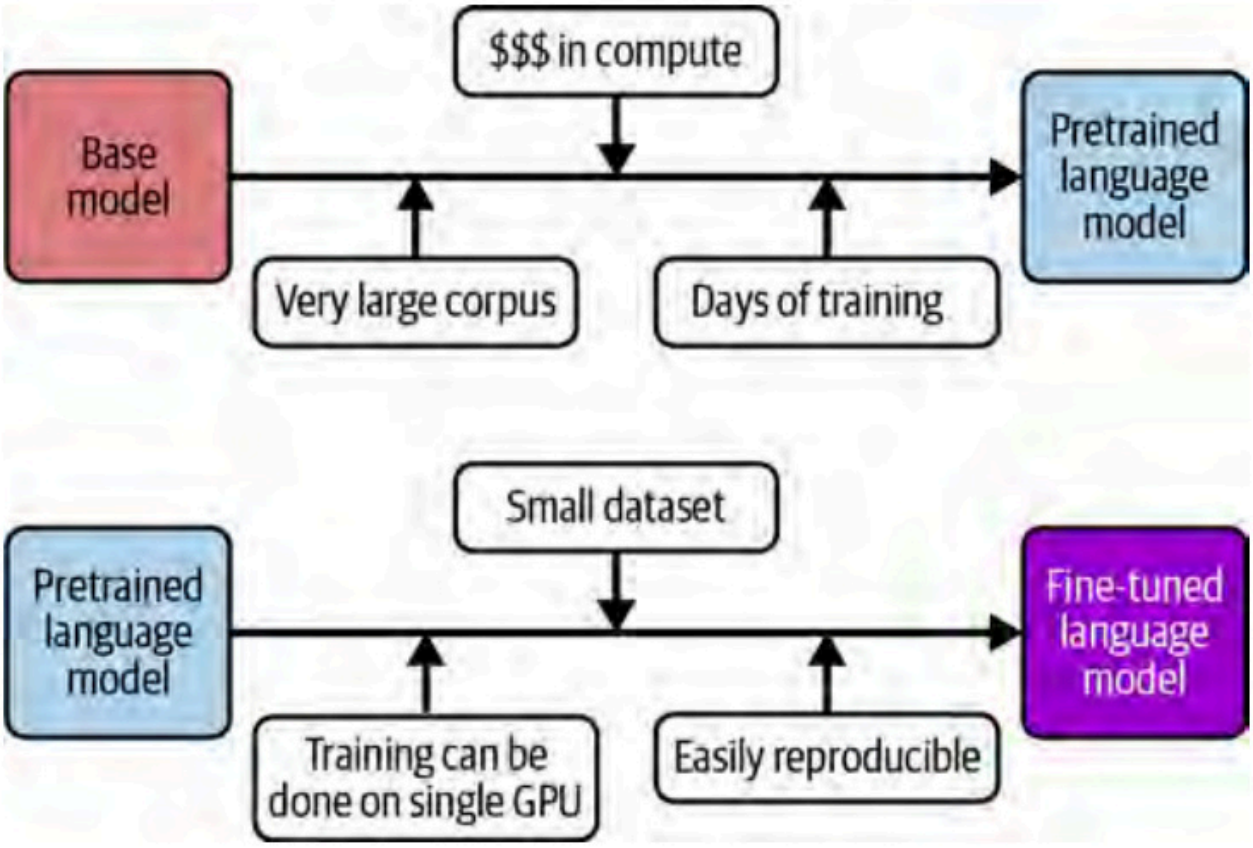



图 2-13. 虽然预训练基础模型可能需要大量资源，但在新任务或领域上微调现有模型的成本要低得多

对于 Transformer，事情更进一步，采用了自监督预训练（使用未标注数据进行训练）。我们可以在大量未标注的文本数据上预训练模型。如何做到呢？让我们想想像 GPT 这样的因果模型。模型预测下一个 token 是什么。好吧，我们不需要任何标签来获取训练数据！给定一个文本语料库，我们可以屏蔽序列后的 token，并训练模型学习预测它们。就像在计算机视觉中一样，预训练为模型提供了对底层文本的有意义表示。然后，我们可以微调模型以执行其他任务，例如生成与我们推文风格或特定领域（例如，公司聊天）相符的文本。鉴于模型已经学习了语言的表示，微调所需的数据量将远少于从头开始训练。

对于许多任务，输入的丰富表示比预测下一个 token 更重要。例如，如果你想微调一个模型来预测电影评论的情感，掩码语言模型（MLM）会更强大。像 GPT-2 这样的模型是为优化文本生成而设计的，而不是为了构建强大的文本表示。另一方面，像 BERT 这样的模型非常适合这项任务。如前所述，编码器模型的最后一层输出输入序列的密集表示，称为嵌入。然后，可以通过在编码器顶部添加一个简单的小网络并针对特定任务微调模型来利用这个嵌入。作为一个具体示例，我们可以在 BERT 编码器输出之上添加一个简单的线性层来预测文档的情感。我们可以采用这种方法来解决广泛的任务：

Token 分类

识别句子中的每个实体，例如人、地点或组织。

抽取式问答

给定一个段落，回答特定问题并从输入中提取答案。

语义搜索

编码器生成的特征对于构建搜索系统非常有用。给定一个包含一百个文档的数据库，我们可以为每个文档计算嵌入。然后，在推理时，我们可以将输入嵌入与文档的嵌入进行比较，从而识别数据库中与输入最相似的文档。

前面的列表并不详尽，还包括文本相似性、异常检测、命名实体链接、推荐系统和文档分类。

让我们使用一个基于 BERT 的模型，该模型经过微调以执行序列分类，确定文本的情感是正面还是负面。再次，我们将使用 pipeline API 加载模型并执行分类：

```
from transformers import pipeline

classifier = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english",
)
classifier("This movie is disgustingly good !")
```

[{'label': 'POSITIVE', 'score': 0.9998536109924316}]

这个分类模型可以分析评论并预测其情感，正如我们在第 28 页的“零样本泛化”和第 30 页的“少样本泛化”中所做的那样。第 52 页的“挑战”展示了如何评估分类模型并比较零样本设置和这个微调模型。

# Transformer 回顾

我们讨论了三种类型的架构：

基于编码器的架构

基于编码器的架构，如 BERT、DistilBERT 和 RoBERTa，非常适合需要理解整个输入的任务。

这些模型输出捕获输入序列含义的上下文嵌入。然后，我们可以在这些嵌入之上添加一个小网络，并针对依赖于语义信息的新特定任务进行训练。

基于解码器的架构

基于解码器的架构，如 GPT-2、Qwen、Gemma 和 Llama，非常适合生成新文本。

编码器-解码器架构

编码器-解码器架构，或序列到序列（seq2seq）架构，如 BART 和 T5，非常适合需要基于给定输入生成新句子的任务，例如摘要或翻译。

“等等，”你可能会说，“我可以用 ChatGPT 或 Llama 处理所有这些任务。”这是真的。鉴于训练数据、计算能力和训练优化的巨大且不断增长，生成模型的质量显著提高。与几年前相比，它们的零样本能力已经取得了长足的进步。

这里有两种主要的思想流派。一种观点认为，如果有资源，针对你的特定任务和领域微调模型将比使用通用的预训练模型产生更好的结果。例如，如果你想使用类似 GPT 的模型在游戏中实时生成角色对话，事先用类似数据进行微调可能会提高性能。同样，如果你需要一个模型从化学论文数据集中提取实体，用相关化学文本微调基于编码器的模型可能是有意义的。

另一方面，随着高质量、低成本且在各种任务上表现良好的通用模型的兴起，一些人认为对于大多数用例来说，微调可能是不必要的。相反，提示工程可能是一种更有效且更便宜的方法。

序列到序列模型最初之所以成功，是因为它们可以将可变长度的输入序列编码为总结输入信息的嵌入。解码器然后使用这个上下文生成输出。最近，仅解码器模型因其简单性、可扩展性、效率和并行化而受到欢迎。在实践中，根据任务的不同，会使用不同类型的模型；没有一种“万能”模型适用于所有任务。

面对超过一百万个开放模型，你可能会想知道该使用哪一个。第 6 章将帮助你在这个领域中导航，提供如何根据任务和需求选择合适模型的指南，以及如何为你的特定需求微调模型。

## 局限性

此时，你可能会对 Transformer 的潜在问题产生疑问。让我们简要地讨论一些局限性：

Transformer 模型非常庞大

研究一致表明，更大的模型表现更好。虽然这令人兴奋，但也带来了担忧。首先，一些最强大的模型需要数千万美元的训练成本——仅计算资源就如此。这意味着只有少数机构能够训练非常大

的基础模型，限制了那些没有这些资源的机构所能进行的研究。其次，使用如此大量的计算资源也可能带来生态影响——数百万 GPU 小时的计算当然需要大量的电力支持。第三，即使其中一些模型是开源的，运行它们也可能需要许多 GPU。第 6 章将探讨一些技术，即使你家里没有多个 GPU，也可以使用这些大语言模型（LLM）。即便如此，在资源受限的环境中部署它们仍然是一个常见的挑战。

顺序处理

如果你回想一下解码器部分，我们必须为每个新 token 处理所有之前的 token。这意味着生成序列中的第 10,000 个 token 将比生成第一个 token 花费更长的时间。在计算机科学术语中，Transformer 的时间复杂度相对于输入长度是二次的。这意味着随着输入长度的增加，处理时间呈二次增长，这使得将它们扩展到非常长的文档或在某些实时场景中使用这些模型变得具有挑战性。尽管 Transformer 在许多任务中表现出色，但在生产环境中使用时，其计算需求需要仔细考虑和优化。话虽如此，已经有很多研究致力于使 Transformer 在极长序列中更高效，例如巧妙的缓存技术、环形注意力（ring attention）和无限注意力（Infiniattention）。

固定输入大小

Transformer 模型可以处理的最大 token 数量取决于基础模型。模型可以处理的 token 数量称为上下文窗口，在选择预训练模型时，了解这一点至关重要。你不能简单地将整本百科全书传递给 Transformer，期望它们能够进行总结，但这种情况正在迅速改变。虽然一些预训练模型只能处理最多 512 个 token，但现在能够处理多达 32,000 个 token 的模型更为常见。新技术甚至允许扩展到数十万甚至数百万个 token！例如，Llama 3.1 可以处理 131,000 个 token，这接近本书的长度！

有限的解释性
Transformer 经常因其缺乏解释性而受到批评。

所有这些局限性都是当前的研究领域。人们一直在探索如何用更少的计算资源训练和运行模型（例如 QLoRA，我们将在第 6 章中探讨）、使生成更快（例如 Flash Attention 和辅助生成）、实现无限制的输入大小（例如 RoPE 和注意力池），以及解释注意力机制。

一个主要的问题是模型中存在的偏见。如果用于预训练 Transformer 的训练数据包含偏见，模型可能会学习并延续这些偏见。这是机器学习中的一个普遍问题，但在 Transformer 中尤为突出。让我们重新看一下填充掩码（fill-mask）管道。假设我们想要预测最可能的职业。正如你在以下示例中所看到的，使用“男人”和“女人”这两个词时，结果会有所不同：

```
unmasker = pipeline("fill-mask", model="bert-base-uncased")
result = unmasker("This man works as a [MASK] during summer.")
print([r["token_str"] for r in result])

result = unmasker("This woman works as a [MASK] during summer.")
print([r["token_str"] for r in result])
```

['farmer', 'carpenter', 'gardener', 'fisherman', 'miner']
['maid', 'nurse', 'servant', 'waitress', 'cook']

为什么会发生这种情况？为了实现预训练，研究人员通常需要大量数据，这导致他们抓取所有能找到的内容。这些内容的质量可能参差不齐，包括有毒内容（可以在一定程度上过滤掉）。基础模型在微调时可能会最终内化并延续这些偏见。类似的问题也存在于对话模型中，最终模型可能会生成从预训练数据集中学到的有毒内容。

## 超越文本

Transformer 已被用于许多将数据表示为文本的任务。一个明显的例子是代码生成：我们可以使用大量代码来训练语言模型，而不是使用英语数据，通过我们刚刚学到的相同原理，它将学会如何自动补全代码。另一个例子是使用 Transformer 从表格（如电子表格）中回答问题。

由于 Transformer 模型在文本领域取得了巨大成功，其他领域也产生了极大的兴趣，希望将这些技术应用于其他模态。这导致 Transformer 模型被用于图像识别、分割、目标检测、视频理解等任务，如图 2-14 所示。

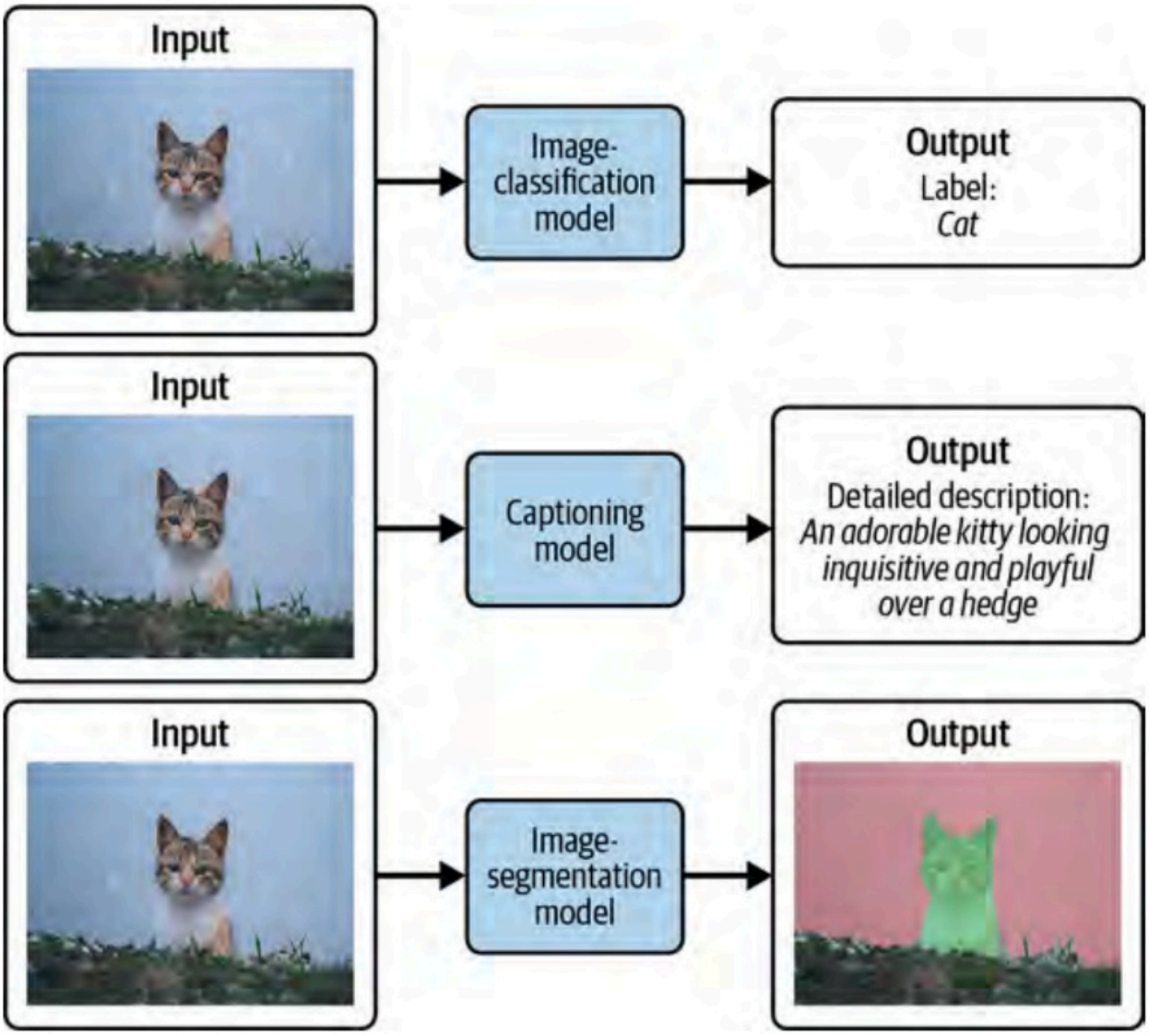



图 2-14. Transformer 模型可用于分类图像、检测目标和分割图像等任务

CNN 一直是大多数计算机视觉技术的首选 SOTA 模型。随着视觉 Transformer（ViTs）的引入，近年来人们开始探索如何用注意力和基于 Transformer 的技术解决视觉任务。ViTs 并没有完全抛弃 CNN：在图像处理管道中，CNN 提取图像的特征图以检测高级边缘、纹理和其他模式。从 CNN 中获得的特征图随后被划分为固定大小的、不重叠的 patch。这些 patch 可以像 token 序列一样处理，因此注意力机制可以学习不同位置 patch 之间的关系。

不幸的是，ViTs 需要比 CNN 更多的数据（3 亿张图像！）和计算资源才能获得良好的结果。近年来，进一步的工作已经展开；例如，DeiT 模型能够利用基于 Transformer 的架构处理中等规模的数据集（120 万张图像），这得益于使用了 CNN 中常见的增强和正则化技术。其他模型如 DETR、SegFormer 和 Swin Transformer 进一步推动了该领域的发展，支持图像分类、目标检测、图像分割、视频分类、文档理解、图像恢复、超分辨率等任务。

基于 Transformer 的图像模型的一个强大示例是零样本图像分类。与在固定类别集上训练的传统图像分类器不同，零样本图像分类允许在推理时指定类别。这提供了使用单一模型处理各种图像分类任务的灵活性，即使是那些它没有明确训练过的任务。为了演示，让我们首先使用 PIL 库加载一张图像，这是一个广泛用于视觉相关预处理的工具：

```
import requests
from PIL import Image
from genaibook.core import SampleURL
// 下载一张图像并使用 PIL 库加载它
url = SampleURL.CatExample
image = Image.open(requests.get(url, stream=True).raw)
image
```

现在，让我们使用高级管道来使用模型执行给定任务：

```
pipe = pipeline(
    "zero-shot-image-classification", model="openai/clip-vit-base-patch32"
)
labels = ["cat", "dog", "zebra"]
pipe(image, candidate_labels=labels)
```

[{'score': 0.9936687350273132, 'label': 'cat'}, {'score': 0.006043245084583759, 'label': 'dog'}, {'score': 0.0002880473621189594, 'label': 'zebra'}]

1 加载 openai/clip-vit-base-patch32 模型。

定义我们希望在推理时使用的类别。

将图像和标签通过管道传递以获取模型预测。

正如我们将在第 9 章中探讨的那样，Transformer 模型也可以用于音频任务，例如转录音频或生成合成语音或音乐。在底层，预训练和注意力机制的基本原理仍然存在，但每种模态具有不同的数据类型，需要不同的方法和修改。

Transformer 正在探索的其他模态包括：

图

一个很好的入门读物是 Clémentine Fourrier 的《图机器学习导论》。使用 Transformer 处理图数据仍然非常具有探索性，但有一些令人兴奋的早期结果。涉及图数据的任务示例包括预测分子的毒性、预测系统的演化或生成新的合理分子。

3D 数据

对可以表示为 3D 的数据进行分割，例如自动驾驶中的 LiDAR 点云或用于器官分割的 CT 扫描。另一个示例是估计物体的六自由度，这在机器人应用中可能很有用。

时间序列

分析股票价格或进行天气预报。

多模态

一些 Transformer 模型设计用于处理或输出多种类型的数据（如文本、图像和音频）。这开辟了新的可能性，例如多模态系统，你可以说话、书写或提供图片，并由单一模型处理它们。另一个

示例是视觉问答，模型可以回答有关提供图像的问题。

# 项目时间：使用语言模型生成文本

我们在第20页的“生成文本”中使用了generate()方法来执行各种解码技术。为了更好地理解其背后的工作原理，现在是时候自己实现它了。我们将以generate()为参考，但从头开始实现它。

你的目标是填写以下函数中的代码。与使用model.generate()不同，我们的想法是迭代调用model()，将之前的token作为输入传递。你需要在do_sample = False时实现贪婪搜索，在do_sample = True时实现采样，以及在do_sample = True且top_k不为None时实现Top-K采样。这将是一个具有挑战性的任务，所以如果你没有很快想出解决方案，不要担心。我们建议你从实现贪婪搜索开始，然后在此基础上进行扩展：

```
def generate( model, tokenizer, input_ids, max_length=50, do_sample=False,
    top_k=None
): """生成一个序列而不使用model.generate()
    参数:
    model: 用于生成的模型。
    tokenizer: 用于生成的分词器。
    input_ids: 输入的ID
    max_length: 序列的最大长度。
    do_sample: 是否使用采样。
    top_k: 采样的token数量。
    """
    # 在这里编写你的代码
    # 从最简单的方法开始，贪婪解码。
    # 然后添加采样，最后是top-k采样。
```

# 总结

你现在已经学会了加载和使用transformers来完成各种任务。本章还介绍了transformer模型如何对文本等序列数据进行处理，以及这一特性如何让它们学习到有价值的表示，我们可以利用这些表示来生成或分类新的序列。随着这些模型的规模增加，它们的能力也随之增强——以至于现在拥有数千亿参数的大型模型可以完成许多以前被认为计算机无法完成的任务。

我们可以选择强大的现有预训练模型，并通过微调将其修改为特定领域和用例。向更大、更强大的模型发展的趋势已经改变了人们使用它们的方式。特定任务的模型通常被通用的大型语言模型（LLM）所超越，大多数人现在通过API、托管解决方案、本地部署或直接通过基于聊天的用户

界面与这些模型进行交互。与此同时，像Llama这样的大型、强大的开放访问模型的发布，激发了研究人员和实践者在消费级计算机上直接运行高质量模型的强烈运动，从而产生了以隐私为先的解决方案。这一趋势不仅限于推理：近年来，出现了允许个人在没有大量计算资源的情况下微调这些模型的新颖训练方法。第6章将进一步探讨这一点，并深入探讨传统和新颖的微调技术。

尽管我们介绍了transformer的工作原理，并且我们将深入探讨它们的训练，但探索这些模型的内部机制（例如注意力机制背后的数学原理）或如何从头开始预训练模型超出了本书的范围。幸运的是，有一些优秀的资源可以学习这些内容：

- Jay Alammar的“The Illustrated Transformer”是一份精美的视觉指南，以详细且直观的方式解释了transformer。
- 如果你想更深入地了解如何为多个特定任务微调这些模型的内部机制，我们推荐阅读Lewis Tunstall等人所著的《Natural Language Processing with Transformers》（O’Reilly出版）。
- Hugging Face有一个免费的开源课程，教授如何解决不同的NLP任务。

如果你想更深入地了解GPT系列模型，我们建议阅读以下论文：

“Improving Language Understanding by Generative Pre-training”

这是原始的GPT论文，由Alec Radford等人于2018年发表。它提出了使用基于transformer的模型在大量文本语料库上进行预训练以学习通用语言表示，然后在特定下游任务上进行微调的想法。该论文还展示了GPT模型在当时多个自然语言理解基准测试中取得了SOTA（最先进）的结果。

“Language Models Are Unsupervised Multitask Learners”

由Alec Radford等人于2019年发表，该论文介绍了GPT-2，这是一个拥有15亿参数的基于transformer的模型，预训练于一个名为WebText的大型网络文本语料库。该论文还展示了GPT-2可以在无需微调的情况下在多种自然语言任务上表现出色，例如文本生成、摘要、翻译、阅读理解和常识推理。最后，它讨论了大规模语言模型潜在的伦理和社会影响。

“Language Models Are Few-Shot Learners”

由Tom B. Brown等人于2020年发表，该论文表明，扩大语言模型的规模显著提高了它们从少量示例或简单指令中执行新语言任务的能力，而无需微调或梯度更新。

该论文还介绍了GPT-3，这是一个拥有1750亿参数的自回归语言模型，在许多NLP数据集和任务上表现出色。

# 练习

1. 注意力机制在文本生成中的作用是什么?
2. 在哪些情况下会优先使用基于字符的分词器?
3. 如果你使用与模型不同的分词器会发生什么?
4. 在生成时使用no_repeat_ngram_size的风险是什么? （提示：考虑城市名称。）
5. 如果你将束搜索和采样结合起来会发生什么?
6. 想象你正在使用一个通过采样生成代码的LLM，在代码编辑器中，哪种方式更方便：低温还是高温?
7. 微调的重要性是什么? 它与零样本生成有何不同?
8. 解释编码器、解码器和编码器-解码器transformer的区别和应用。

你可以在本书的GitHub仓库中找到这些练习和以下挑战的解决方案。

# 挑战

1. 摘要生成。使用摘要生成模型（你可以使用pipeline("summarization")）生成段落的摘要。它与使用零样本的结果相比如何? 通过提供少量示例能否超越它?
2. 情感分析。在零样本补充材料中，我们使用零样本分类计算了一些指标。探索使用distilbert-base-uncased-finetuned-sst-2-english编码器模型进行情感分析。你得到了什么结果?
3. 语义搜索。让我们构建一个FAQ系统！句子transformer是强大的模型，可以测量语义文本相似性。虽然transformer编码器通常为每个token输出一个嵌入，但句子transformer为整个输入文本输出一个嵌入，使我们能够根据它们的相似性得分判断两个文本是否具有相似的含义。让我们看一个使用sentence_transformers库的简单示例：

```
from sentence_transformers import SentenceTransformer, util
sentences = ["I'm happy", "I'm full of happiness"]
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# 计算两个列表的嵌入
embedding_1 = model.encode(sentences[0], convert_to_tensor=True)
embedding_2 = model.encode(sentences[1], convert_to_tensor=True)

util.pytorch_cos_sim(embedding_1, embedding_2)

tensor([[0.6003]], device='cuda:0')
```

编写一个包含五个问题和答案的列表，主题自选。你的目标是构建一个系统，当给定一个新问题时，可以为用户提供最可能的答案。我们如何使用句子transformer来解决这个问题？补充材料中包含解决方案，但尽管具有挑战性，我们建议你先尝试解决，然后再查看答案。

> 一种称为检索增强生成（RAG）的强大技术结合了文本生成和嵌入来检索相关文档。附录C展示了一个如何构建最小RAG管道的端到端示例。在此之前，我们建议阅读第6章，该章介绍了微调以及如何使用它来使模型适应特定任务。

## 参考文献


Brown, Tom B., 等. “Language Models Are Few-Shot Learners.” arXiv, 2020年7月22日. http://arxiv.org/abs/2005.14165.

Devlin, Jacob, 等. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding.” arXiv, 2019年5月24日. http://arxiv.org/abs/1810.04805.

Dosovitskiy, Alexey, 等. “An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale.” arXiv, 2021年6月3日. http://arxiv.org/abs/2010.11929.

Fourrier, Clémentine. “Introduction to Graph Machine Learning.” Hugging Face 博客, 2023年1月3日. https://oreil.ly/TK9vt.

Gao, Leo, 等. “Scaling and Evaluating Sparse Autoencoders.” arXiv, 2024年6月6日. https://arxiv.org/abs/2406.04093.

Holtzman, Ari, 等. “The Curious Case of Neural Text Degeneration.” arXiv, 2020年2月14日. http://arxiv.org/abs/1904.09751.

Howard, Jeremy, 和 Sebastian Ruder. “Universal Language Model Fine-Tuning for Text Classification.” arXiv, 2018年5月23日. http://arxiv.org/abs/1801.06146.

Lan, Tian. “Generating Human-Level Text with Contrastive Search in Transformers.” Hugging Face 博客, 2022年11月8日. https://oreil.ly/VFc42.

Lewis, Mike, 等. “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.” arXiv, 2019年10月29日. http://arxiv.org/abs/1910.13461.

Radford, Alec, 等. “Improving Language Understanding by Generative Pretraining.” OpenAI 博客, 2018年6月11日. https://oreil.ly/ND5Bk.
Radford, Alec, 等. “Language Models Are Unsupervised Multitask Learners.” OpenAI 博客 1, no. 8 (2019): 9. https://oreil.ly/m5bBi.
Raffel, Colin, 等. “Exploring the Limits of Transfer Learning with a Unified Text-toText Transformer.” arXiv, 2020年7月28日. http://arxiv.org/abs/1910.10683.
Vaswani, Ashish, 等. “Attention Is All You Need.” arXiv, 2017年6月12日. https:// arxiv.org/abs/1706.03762.
Yang, Jingfeng, 等. “Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond.” arXiv, 2023年4月27日. http://arxiv.org/abs/2304.13712.

# 第3章 信息压缩与表示

本章介绍了用于学习高效数据表示的机器学习模型和技术，这些技术适用于涉及图像、视频或文本的任务。为什么高效表示很重要？我们希望在保持数据本质特征的同时，减少需要存储和处理的信息量。丰富的表示使得训练专注于特定任务的模型成为可能，而紧凑的表示则减少了训练和处理数据密集型模型的计算需求。例如，在图像的向量嵌入上进行训练可能比直接在其像素上训练更高效且更具表现力。

传统的压缩方法如ZIP或JPEG专注于特定数据类型，并使用手工设计的算法来减小文件大小。虽然这些方法在其预期用途上非常有效，但它们缺乏学习型压缩技术的灵活性和适应性。例如，ZIP擅长通过识别和编码重复模式来实现通用数据的无损压缩。另一方面，JPEG专门为图像压缩设计，通过丢弃不太明显的视觉信息来实现显著的尺寸缩减。然而，这些传统方法不会从它们压缩的数据中学习，也无法自动适应不同类型的内容或优化除尺寸缩减之外的特定任务。这就是机器学习模型可以发挥作用的地方。

我们将从探索自动编码器（AutoEncoders）开始，这是一类由“压缩”数据的编码器和仅使用表示来重建数据的解码器组成的机器学习模型。编码器学习数据中需要关注的关键特征，这使得解码器能够逆转这些变换，如图3-1左侧所示。这种训练方法是一种自动构建压缩器的方式，无需依赖手工设计的算法。压缩信息（即使是有损的）本身就有帮助，但一旦我们有了紧凑的数据集表示，还可以做一些其他有趣的事情。

如果我们的系统训练正确，并且解码器可以从表示中恢复原始数据，这意味着学习到的表示已经捕捉到了关键信息。因此，在表示上进行操作等同于处理原始数据，但需要的内存和计算资源要少得多。这是诸如Stable Diffusion等模型的关键设计方面之一——正如我们将在第5章中看到的，我们可以生成和操作大型图像，但大部分计算发生在表示所在的较小潜在空间中。

由于学习到的表示捕捉了关键信息，我们可以在自动编码器训练后将编码器和解码器分开，并将编码器用作特征提取组件。在编码器输出上添加一个小型网络，如图3-1中间所示，使我们能够为不同任务（如文本或图像分类）训练模型。这些小型网络并不对整个输入图像进行操作，而是对编码器获得的关键特征进行操作。

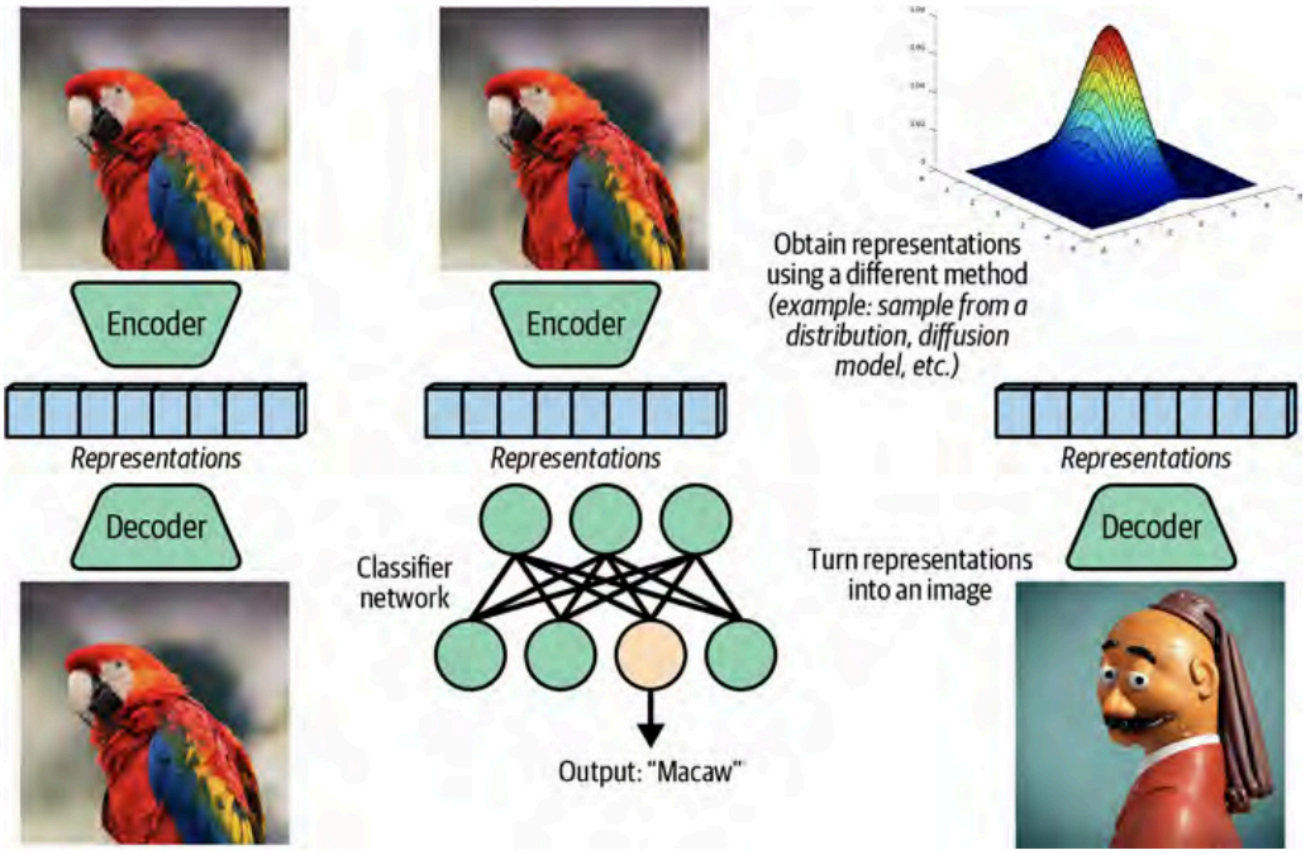



图3-1. 高效数据表示方法（左）可用于其他任务，如分类（中），或生成新内容（右）

我们还可以将不同数据类型编码到相同的潜在空间表示中。正如我们在第2章中看到的，序列到序列语言模型使用编码器-解码器架构来执行各种任务，如翻译或摘要。尽管在设计此类系统时需要考虑更多细节，但一个关键见解是编码器的任务是捕捉携带足够语义信息的输入文本的关键特征。这也适用于跨模态任务：例如，图像描述模型的工作是将图像表示转换为文本描述，使用潜在空间作为内部工作数据。

自动编码器的最后一个用途是生成建模（图3-1右侧）。在训练完编码器-解码器对后，我们可以丢弃编码器，并通过从潜在空间中的随机分布中采样来生成新数据。这是变分自动编码器（VAEs）的基础。我们将在本章的第二部分中看到一个如何实现这一点的示例。

我们将使用图像数据来展示自动编码器和VAE的工作原理，但这些技术可以应用于任何数据，而不仅仅是图像。本章的最后一部分探讨了多模态表示学习系统（如CLIP）如何弥合文本和图像

之间的差距，并可用于非常有趣的用例，如语义搜索、数据过滤、文本到图像生成等。

# 自动编码器

自动编码器由两个模型拼接而成——即编码器和解码器，如图3-2所示。这两个模型一起训练，目标是编码器生成中间表示，解码器随后使用这些表示来重建输入数据。如果训练成功，自动编码器将学会从输入数据中提取关键特征。

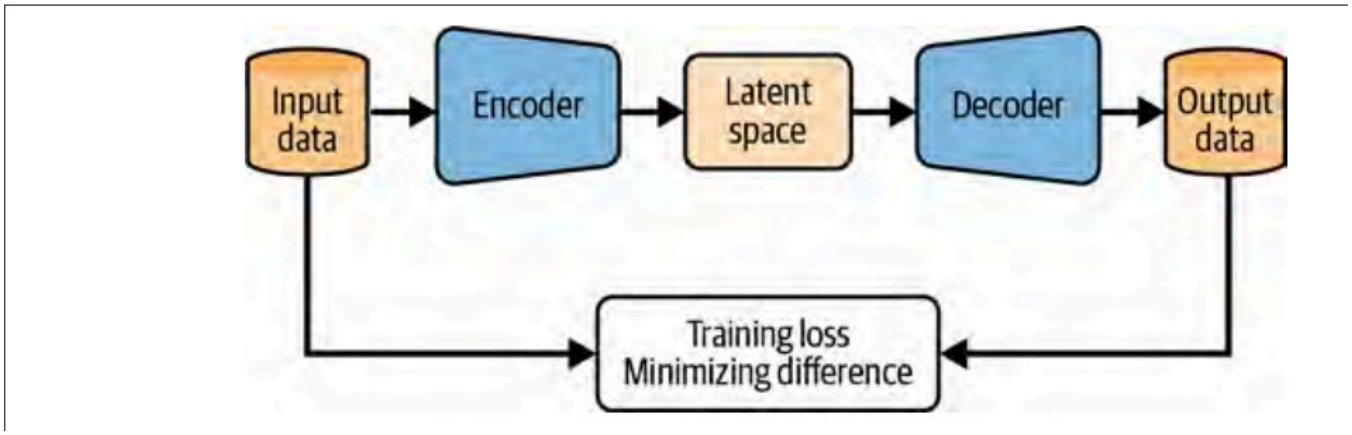



图3-2. 自动编码器架构

## 准备数据

在本节中，我们将使用 MNIST 数据集构建一个简单的自动编码器（AutoEncoder）。MNIST 是一个经典的数据集，包含 70,000 张低分辨率 $(28 \times 28)$ 的黑白手写数字图像。我们将从 Hugging Face 上托管的该数据集的分发版本中下载它。为了下载它，我们将使用一个名为 `datasets` 的库，它提供了一个统一的 API 来访问数千种任何类型的数据集。关于它如何工作的细节现在并不重要；只需注意它将负责下载和缓存以供后续使用。它提供了两个数据集分割——一个包含 60,000 张图像的训练集和一个包含剩余 10,000 张图像的测试集：

```
from datasets import load_dataset
mnist = load_dataset("mnist")
mnist
DatasetDict({ train: Dataset({ features: ['image', 'label'], num_rows: 60000 })
    test: Dataset({ features: ['image', 'label'], num_rows: 10000 })
})
```

如你所见，数据集包含一个名为 `image` 的列，其中包含手写图像，以及 `label` 列，其中包含图像所表示的数字。由于我们要训练一个自动编码器来压缩和重建图像，我们实际上根本不需要标

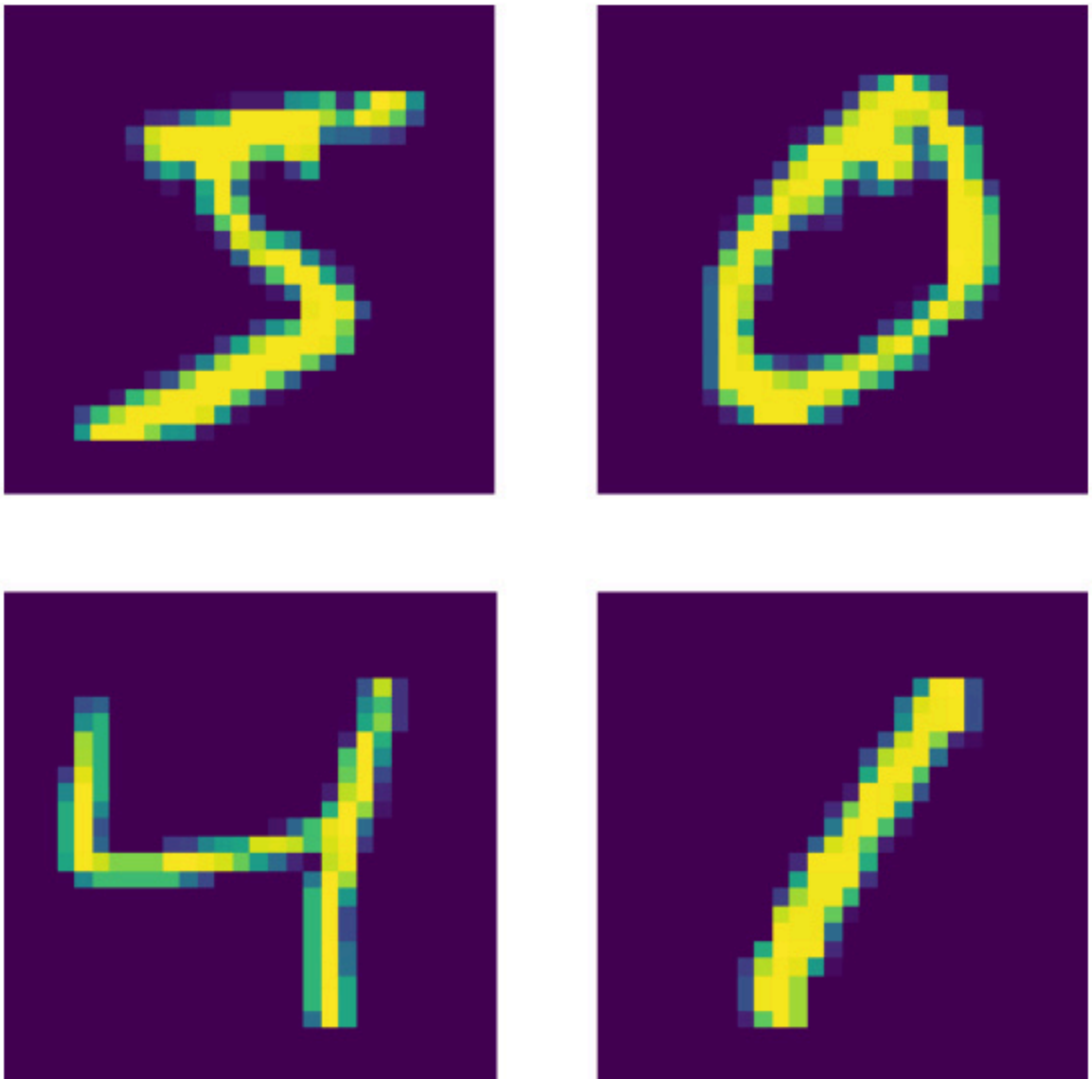
签数据：我们将向编码器提供随机样本的批次，而解码器的任务是生成与输入相似的图像。因为输入数据包含了训练所需的所有信息，而不依赖于外部标注信息，所以自动编码器训练是自监督学习过程的一个例子。

我们将忽略标签进行训练，但稍后将使用它们进行可视化。与往常一样，在训练模型之前，让我们先探索一下数据集：

```
mnist["train"]["image"][1]
```

以今天的标准来看，分辨率仅为 $28 \times 28$ 的图像非常小。我们将使用一个辅助函数 `show_images()` 以更高的分辨率显示它们。该函数基于 Python 的 `matplotlib` 库，默认情况下，它使用高对比度的调色板来表示单色图像数据：

```
from genaibook.core import show_images show_images(mnist["train"]["image"][:4])
```



由于原始图像是黑白的，我们将配置 `matplotlib` 仅使用灰色。我们选择“反转灰色”（`gray_r`）以获得黑色数字在白色背景上的效果。请注意，原始图像是相反的（表示数字的像素是白色的，而背景是全零，意味着黑色）：

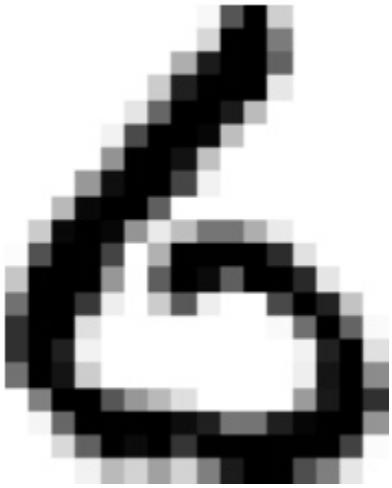
```
import matplotlib as mpl mpl.rcParams["image.cmap"] = "gray_r" show_images(
    mnist["train"]["image"][:4])
```

在下面的示例中，我们将图像转换为 PyTorch 张量并打乱训练数据集。我们将使用 torchvision 库中的 ToTensor() 将输入像素从 [0, 255] 范围转换为 PyTorch 张量，范围从 0 到 1。我们不应用任何其他操作。为了方便起见，show_images() 也可以绘制表示图像的张量。

```
from torchvision import transforms

def mnist_to_tensor(samples):
    t = transforms.ToTensor()
    samples["image"] = [t(image) for image in samples["image"]]
    return samples
mnist = mnist.with_transform(mnist_to_tensor)
mnist["train"] = mnist["train"].shuffle(seed=1337)
```

让我们检查数据集中的一张图像，并确认输入像素的范围是从 0 到 1：

```
x = mnist["train"]["image"][0]
x.min(), x.max()
(tensor(0.), tensor(1.))
show_images(mnist["train"]["image"][0])
```



通过以下代码，我们现在创建一个 PyTorch DataLoader 来准备训练数据。由于训练自动编码器是一个自监督过程，我们将使用数据集的 image 列并忽略标签。稍后，我们将返回并使用标签来可视化结果。DataLoader 是一个抽象，其主要任务是对输入进行整理，这意味着将单个样本收集并组合成具有相同形状的训练批次。在我们的例子中，所有图像的大小相同，因此所有张量具有相同的形状，因此 DataLoader 会将它们连接在一起。在更复杂的情况下，DataLoader 可能需要通过填充或截断等策略来处理不规则的输入形状。

```
from torch.utils.data import DataLoader bs = 64 train_dataloader = DataLoader(
    mnist["train"]["image"], batch_size=bs)
```

## 编码器的建模

首先，我们将为自编码器的编码器部分创建一个模型定义。由于我们处理的是图像数据，一个自然的选择是使用卷积层，因为它们擅长捕捉图像特征。对于这个问题，我们还可以考虑许多其他替代方案：线性层、Transformer 模块、使用残差跳跃连接等。我们将使用一个基于优秀 pythae 库中的卷积自编码器实现的简单卷积编码器。这是一个很好的探索起点！

卷积层是一组应用于输入图像不同区域的小型 2D 滤波器。这些滤波器可以检测诸如线条或圆形区域等模式。传统上，2D 滤波器被用于数字图像处理，其中它们被精心手工设计以匹配输入图像中的特定特征。卷积层的主要区别在于滤波器不是预先准备好的——相反，卷积层将它们作为网络训练过程的一部分进行学习。通过堆叠多个卷积层，模型可以逐步从输入图像中提取更抽象的特征，学习能够有效解决任务的滤波器。有一个令人兴奋的解释和可解释性领域，旨在可视化和理解模型层的内部工作原理，并且已经证明，神经网络学习的滤波器有时类似于手动设计的用于检测边缘、颜色或轮廓的经典滤波器。

要深入了解 CNN，我们推荐 Jeremy Howard 和 Sylvain Gugger 所著的《Deep Learning for Coders with fastai and PyTorch》（O'Reilly 出版）的第 13 章。

由于我们将堆叠多个卷积层，我们将编写一个简单的辅助函数 `conv_block()` 来创建它们。我们的 `conv_block()` 辅助函数首先进行 2D 卷积，然后附加一个批量归一化层和一个非线性激活函数（在这个例子中，我们将使用 ReLU 激活函数）。在训练过程中，批量归一化使用当前批次的均值和标准差来归一化输入数据，使其保持在可预测的范围内，这通常会导致训练更平滑和更快。

让我们实现这个函数：

```
from torch import nn

def conv_block(in_channels, out_channels, kernel_size=4, stride=2, padding=1):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size=kernel_size,
            stride=stride, padding=padding),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(),
    )
```

如前所述，编码器实现将是一系列卷积层。每一层逐步减少图像分辨率，同时将表示的通道数增加到 1,024。最后，我们将在末尾附加一个线性层，以创建 16 维的向量表示。`forward()` 方法

中的注释显示了输入数据在通过各层时的形状变化：

```
class Encoder(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.conv1 = conv_block(in_channels, 128)
        self.conv2 = conv_block(128, 256)
        self.conv3 = conv_block(256, 512)
        self.conv4 = conv_block(512, 1024)
        self.linear = nn.Linear(1024, 16)

    def forward(self, x):
        x = self.conv1(x)  # (batch size, 128, 14, 14)
        x = self.conv2(x)  # (bs, 256, 7, 7)
        x = self.conv3(x)  # (bs, 512, 3, 3)
        x = self.conv4(x)  # (bs, 1024, 1, 1)
        x = self.linear(x.flatten(start_dim=1))  # (bs, 16)
        return x
```

让我们验证一下我们是否可以通过编码器运行我们的输入图像。它们的形状为 [1, 28, 28]，因为它们只包含一个通道的（黑色或白色）像素数据。然而，请注意，我们以通用的方式编写了编码器；我们也可以将其用于三通道图像：

```
mnist["train"]["image"][0].shape
torch.Size([1, 28, 28])
```

让我们选择一个图像并将其放入一个批次中，使用 PyTorch 的 `None` 索引创建一个新维度。我们还需要将编码器设置为评估模式。此模式将模型配置为推理而不是训练。如果我们不这样做，最后一个 `BatchNorm2d` 层将失败，因为它将接收到形状为 [1, 1024, 1, 1] 的张量，并且它无法计算单个样本的均值和标准差：

```
in_channels = 1
x = mnist["train"]["image"][0][None, :]
encoder = Encoder(in_channels).eval()
encoded = encoder(x)
encoded.shape
torch.Size([1, 16])
```

编码器模型有效！它将图像（每个 784 像素）转换为仅包含 16 个数字的向量。如果我们能够有效地训练它，编码器计算的表示将比原始像素数据具有更低的维度。

当然，这些表示目前是毫无意义的，因为模型尚未训练：

```
encoded
tensor([[-0.0145, -0.0318, -0.0109,  0.0080, -0.0218,  0.0305,  0.0183, -0.0294,
     0.0075,  0.0178, -0.0161, -0.0018, 0.0208, -0.0079,  0.0215,  0.0101]],
    grad_fn=<AddmmBackward0>)
```

让我们看看它是否能够处理 64 个图像的批次：

```
batch = next(iter(train_dataloader))
encoded = Encoder(in_channels=1)(batch)
batch.shape, encoded.shape
(torch.Size([64, 1, 28, 28]), torch.Size([64, 16]))
```

这完成了我们的编码器模型，它将图像转换为中间表示。现在让我们转向解码器。

## 解码器

解码器从编码器获得的潜在表示（16 维向量）开始，并将它们转换为原始大小的图像。

解码器架构不必是编码器的反向；它可以是任何能够“理解”编码器表示并能够将其转换为图像的东西。在我们的例子中，我们将创建一个与编码器大致对称的网络：我们将应用转置卷积来增加分辨率，同时减少通道数，直到我们达到所需的 $28 \times 28$ 像素输出分辨率。

我们将在转置卷积之前附加一个线性层，以创建 16,384 个 $1024 \times 4 \times 4$ 像素的张量。该层将被重塑为 $4 \times 4$ 分辨率（[1024,4,4]），这将是第一个转置卷积的输入。从那里，我们逐步减少通道并增加分辨率，直到达到原始图像形状。还有其他方法可以实现相同的目标；请记住，输入是一个具有 16 通道的平坦向量，输出必须由 1 个通道和 $28 \times 28$ 像素组成。

这是我们的解码器实现：

```
def conv_transpose_block(
    in_channels, out_channels, kernel_size=3, stride=2, padding=1,
        output_padding=0, with_act=True,
):
    modules = [
        nn.ConvTranspose2d(
            in_channels, out_channels, kernel_size=kernel_size, stride=stride,
                padding=padding, output_padding=output_padding,
        )
    ]

    if with_act:  # 控制这一点将在以后派上用场
        modules.append(nn.BatchNorm2d(out_channels))

    modules.append(nn.ReLU())
    return nn.Sequential(*modules)

class Decoder(nn.Module):
    def __init__(self, out_channels):
        super().__init__()
        self.linear = nn.Linear(16, 1024 * 4 * 4)  // 注意它在 forward 中被重塑
        self.t_conv1 = conv_transpose_block(1024, 512)
        self.t_conv2 = conv_transpose_block(512, 256, output_padding=1)
        self.t_conv3 = conv_transpose_block(256, out_channels, output_padding=1)


    def forward(self, x):
        bs = x.shape[0]
        x = self.linear(x)  # (bs, 1024*4*4)
        x = x.reshape((bs, 1024, 4, 4))  # (bs, 1024, 4, 4)
        x = self.t_conv1(x)  # (bs, 512, 7, 7)
        x = self.t_conv2(x)  # (bs, 256, 14, 14)
        x = self.t_conv3(x)  # (bs, 1, 28, 28)
        return x

decoded_batch = Decoder(x.shape[0])(encoded)
decoded_batch.shape
torch.Size([64, 1, 28, 28])
```

## 训练

到目前为止，我们已经创建了**编码器**，它降低了输入图像的维度，以及**解码器**，它将低维潜在表示扩展到原始图像分辨率。除了用随机权重初始化外，这两个组件目前是完全不连接的。我们需要一起训练它们，以便它们都能理解相同的潜在表示。

为此，我们将创建一个**自动编码器**模型，该模型将输入数据依次通过编码器和解码器。我们将训练它以最小化输出解码图像与作为输入提供的原始图像之间的差异。如果我们成功了，输出图像将类似于输入图像。

这个过程很有用，因为它允许数据压缩，但当我们意识到在训练后可以单独使用这两个组件时，它就变得更加有趣了。这将使我们在接下来的章节中能够做许多令人兴奋的事情。例如，我们可以使用编码器将任意图像转换为更压缩的表示，这些表示可以用作其他模型的输入。我们还可以使用解码器生成与训练数据集中的图像相似的新图像。

兴奋吗？让我们开始训练我们的自动编码器：

```
class AutoEncoder(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.encoder = Encoder(in_channels)
        self.decoder = Decoder(in_channels)

    def encode(self, x):
        return self.encoder(x)

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        return self.decode(self.encode(x))

model = AutoEncoder(1)
```

我们可以使用 `torchsummary` 库来打印模型的摘要，它显示了每层的参数数量和输出形状。这是一个有用的工具，可以检查模型是否正确定义，并理解模型的架构：

```
import torchsummary

torchsummary.summary(model, input_size=(1, 28, 28), device="cpu")
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1          [-1, 128, 14, 14]           2,176
       BatchNorm2d-2          [-1, 128, 14, 14]             256
              ReLU-3          [-1, 128, 14, 14]               0
            Conv2d-4            [-1, 256, 7, 7]         524,544
       BatchNorm2d-5            [-1, 256, 7, 7]             512
              ReLU-6            [-1, 256, 7, 7]               0
            Conv2d-7            [-1, 512, 3, 3]       2,097,664
       BatchNorm2d-8            [-1, 512, 3, 3]           1,024
              ReLU-9            [-1, 512, 3, 3]               0
           Conv2d-10           [-1, 1024, 1, 1]       8,389,632
      BatchNorm2d-11           [-1, 1024, 1, 1]           2,048
             ReLU-12           [-1, 1024, 1, 1]               0
           Linear-13                 [-1, 16]          16,400
          Encoder-14                 [-1, 16]               0
           Linear-15              [-1, 16384]         278,528
  ConvTranspose2d-16            [-1, 512, 7, 7]       4,719,104
      BatchNorm2d-17            [-1, 512, 7, 7]           1,024
             ReLU-18            [-1, 512, 7, 7]               0
  ConvTranspose2d-19          [-1, 256, 14, 14]       1,179,904
      BatchNorm2d-20          [-1, 256, 14, 14]             512
             ReLU-21          [-1, 256, 14, 14]               0
  ConvTranspose2d-22            [-1, 1, 28, 28]           2,305
      BatchNorm2d-23            [-1, 1, 28, 28]               2
```

ReLU-24 [-1, 1, 28, 28] 0
Decoder-25 [-1, 1, 28, 28] 0
总参数：17,215,635
可训练参数：17,215,635
不可训练参数：0

输入大小 (MB)：0.00
前向/反向传播大小 (MB)：2.86
参数大小 (MB)：65.67
估计总大小 (MB)：68.54

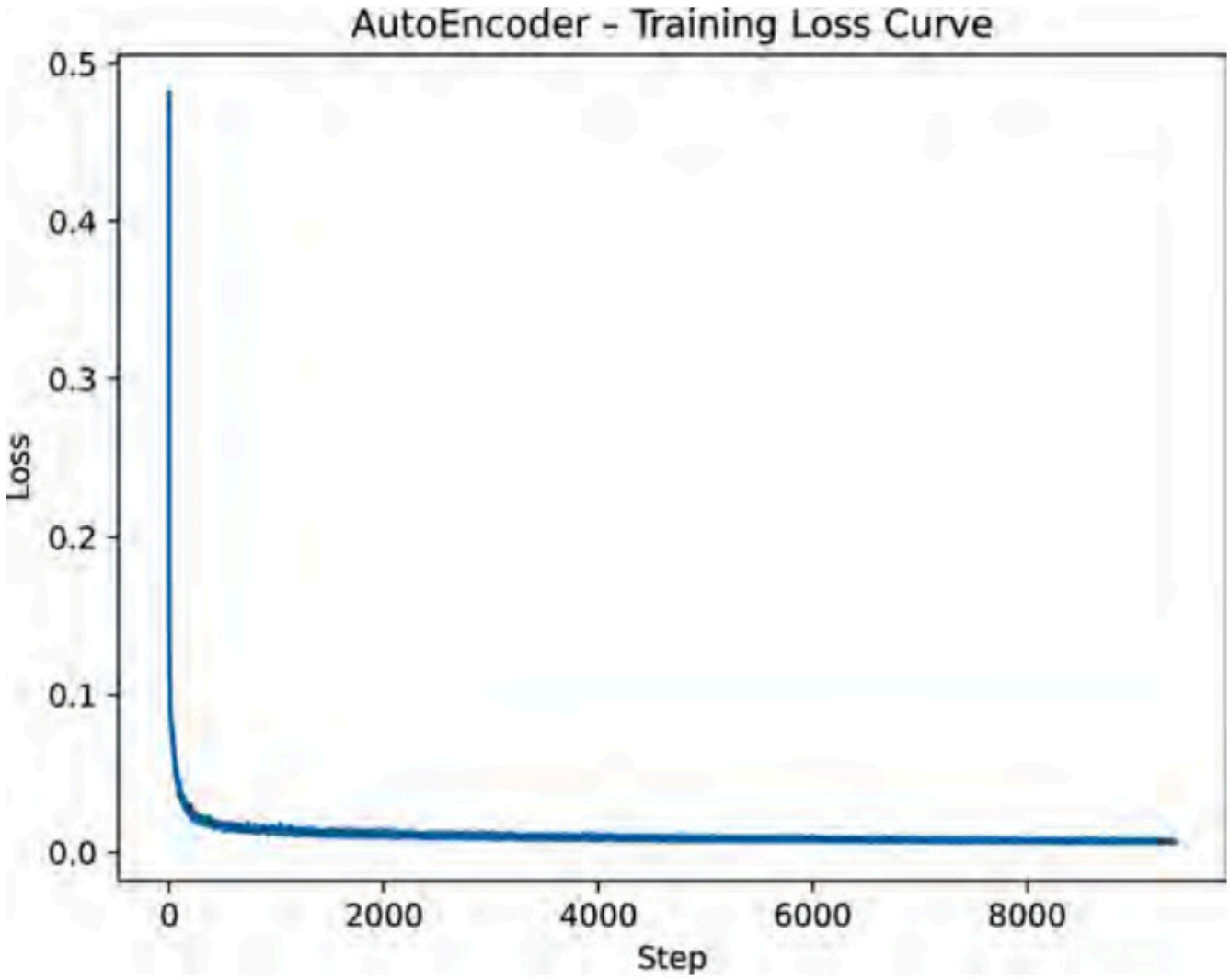
在接下来的代码中，我们创建了一个简单的训练循环，它反复遍历训练数据并使用恒定的学习率。为了专注于核心内容，我们不会在测试集上运行验证（但鼓励你这样做以练习！）。我们使用流行的`tqdm`库来显示进度，但为了简洁起见，我们不会在这里描述它；我们将在其他章节中看到更多示例。不需要理解所有正在发生的事情；只需注意高层次的操作：

1. 从`Dataloader`中加载一个批次。
2. 获取模型预测。
3. 计算与原始图像的损失。
4. 执行优化器步骤以更新模型权重。

```
import torch

from matplotlib import pyplot as plt
from torch.nn import functional as F
from tqdm.notebook import tqdm, trange
from genaibook.core import get_device

num_epochs = 10
lr = 1e-4

device = get_device()
model = model.to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)
losses = []  # 用于存储损失值的列表，以便绘图
for _ in (progress := trange(num_epochs, desc="Training")):
    for _, batch in (inner := tqdm(enumerate(train_dataloader), total=len(
        train_dataloader))):
        batch = batch.to(device)
        // 通过模型并获取重建图像
        preds = model(batch)

        // 将预测与原始图像进行比较
        loss = F.mse_loss(preds, batch)

        // 显示损失并存储以绘图
        inner.set_postfix(loss=f"{loss.cpu().item():.3f}")
        losses.append(loss.item())

        // 根据此损失使用优化器更新模型参数
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    progress.set_postfix(loss=f"{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
```

让我们绘制损失曲线，看看训练过程如何：

```
plt.plot(losses)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("AutoEncoder - 训练损失曲线")
plt.show()
```




我们在训练循环中没有进行验证，但我们可以看到自动编码器在测试集上的表现。使用视觉数据进行训练是学习和迭代的好方法，因为我们可以看到最终结果并自己判断。

我们将从测试集中创建一个包含16个样本的批次，将它们通过训练好的编码器和解码器，并显示重建结果。让我们开始创建评估 `DataLoader`：

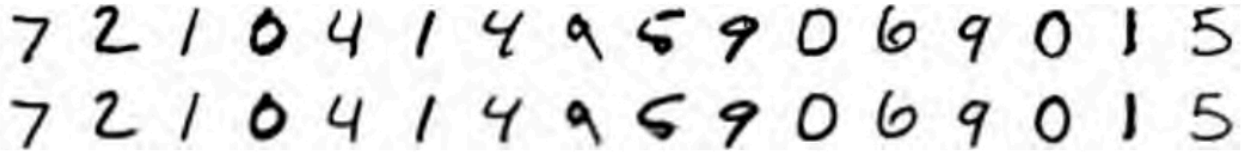
```
eval_bs = 16
eval_dataloader = DataLoader(mnist["test"]["image"], batch_size=eval_bs)
```

我们将使用`model.eval()`将模型置于评估模式（在我们的情况下，它将禁用`BatchNorm`更新），并使用`inference_mode`上下文管理器关闭梯度计算：

```
model.eval()
with torch.inference_mode():
    eval_batch = next(iter(eval_dataloader))
    predicted = model(eval_batch.to(device)).cpu()
```

现在我们有了预测结果，让我们显示原始图像及其重建结果：

```
batch_vs_preds = torch.cat((eval_batch, predicted))
show_images(batch_vs_preds, imsize=1, nrows=2)
```



这里显示的结果，第一行图像代表原始的MNIST图像，第二行图像代表我们自动编码器的重建结果，看起来相当不错！请记住，第二行中的数字是解码器从简洁的向量表示中获得的原始图像的近似值。

在这一点上，我们建议确保你理解刚刚介绍的概念。我们还建议尝试获得更好的重建结果。以下是一些实验想法：

- 逐步降低学习率。
- 尝试不同的批次大小。
- 在解码器的末端使用`sigmoid`函数，以鼓励最终像素值为黑色或白色。请注意，我们的输入数据在0到1之间，因此`sigmoid`输出必须匹配该范围。
- 调整网络的深度或拓扑结构。

我们还建议通过在训练期间添加和记录评估损失来实验训练循环。

## 探索潜在空间

AutoEncoder 的一个重要超参数是我们用于表示编码输入的维度数。我们随意选择了 16，结果显示这似乎足以表示 MNIST 数据集中各种手写数字。

在下一个实验中，我们将仅使用两个维度来表示潜在空间中的向量。我们将强制编码器将尽可能多的关于输入图像的信息压缩到仅两个浮点数中，并看看这是否足以恢复输入。此外，使用两个维度非常便于可视化。在训练新模型后，我们可以在二维空间中绘制一些有趣的图表，这将帮助我们获得更多的直觉。

我们将对代码进行一些重构，具体更改如下：

- 我们将潜在空间的维度作为一个超参数包含进来。
- 我们使用一个容器（`nn.Sequential`）来存储卷积层，以便在以后想要调整网络深度时更容易操作。
- 我们将最终解码器卷积后的激活函数替换为 sigmoid 函数。我们希望鼓励解码器生成黑色或白色的像素，而 sigmoid 激活函数比 ReLU 更适合这一目的。这是因为 sigmoid 函数将输出压缩到 (0, 1) 的范围，与图像中的像素范围相同。

这也是将我们的训练循环放入函数中的好时机。

不要一开始就尝试编写具有许多选项和参数的代码。最好从你能编写的最简单的有效代码开始，然后根据需要逐步丰富它（如果需要的话）。

图 3-3 展示了 ReLU 与 sigmoid 函数的对比图。通过使用 sigmoid 作为激活函数，我们确保每层后的输出都在 (0, 1) 的范围内，与输入图像使用的范围相同。这并不是网络学习的严格必要条件；我们之所以这样做，是因为我们知道期望的输出范围。

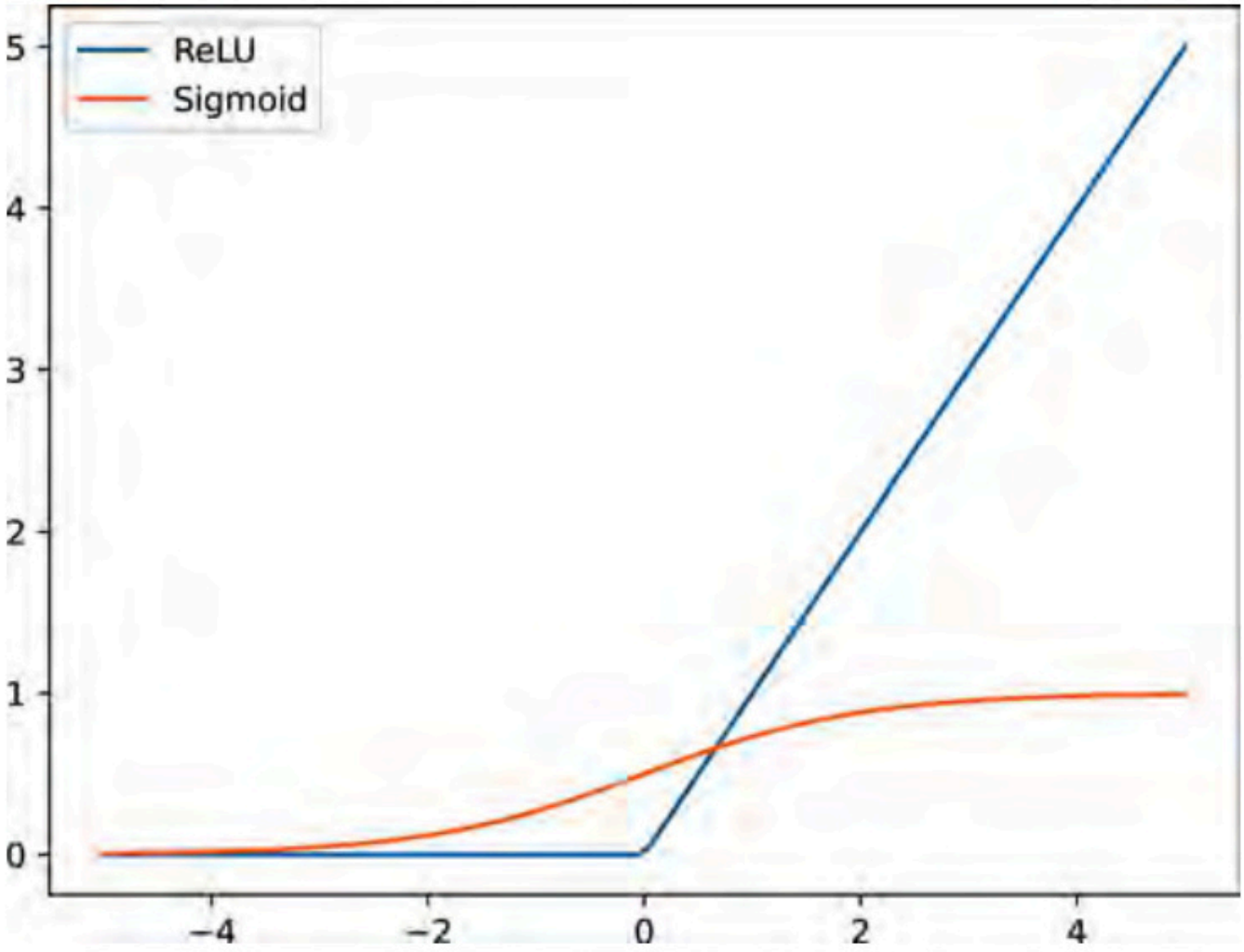



图 3-3. 激活函数：ReLU 与 Sigmoid

经过这些更改后的重构代码如下：

```
class Encoder(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()
        self.conv_layers = nn.Sequential(
            conv_block(in_channels, 128),
            conv_block(128, 256),
            conv_block(256, 512),
            conv_block(512, 1024),
        )
        self.linear = nn.Linear(1024, latent_dims)

    def forward(self, x):
        bs = x.shape[0]
        x = self.conv_layers(x)
        x = self.linear(x.reshape(bs, -1))
        return x
```

```
class Decoder(nn.Module):
    def __init__(self, out_channels, latent_dims):
        super().__init__()
        self.linear = nn.Linear(latent_dims, 1024 * 4 * 4)
        self.t_conv_layers = nn.Sequential(
            conv_transpose_block(1024, 512),
            conv_transpose_block(512, 256, output_padding=1),
            conv_transpose_block(256, out_channels, output_padding=1,
                with_act=False),
        )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        bs = x.shape[0]
        x = self.linear(x)
        x = x.reshape((bs, 1024, 4, 4))
        x = self.t_conv_layers(x)
        x = self.sigmoid(x)
        return x
```

新版本的 `AutoEncoder` 类与之前的几乎相同：它在编码器的输出上调用解码器。它只是接受了

一个额外的参数 `latent_dims`，以便我们可以指定表示的期望维度：

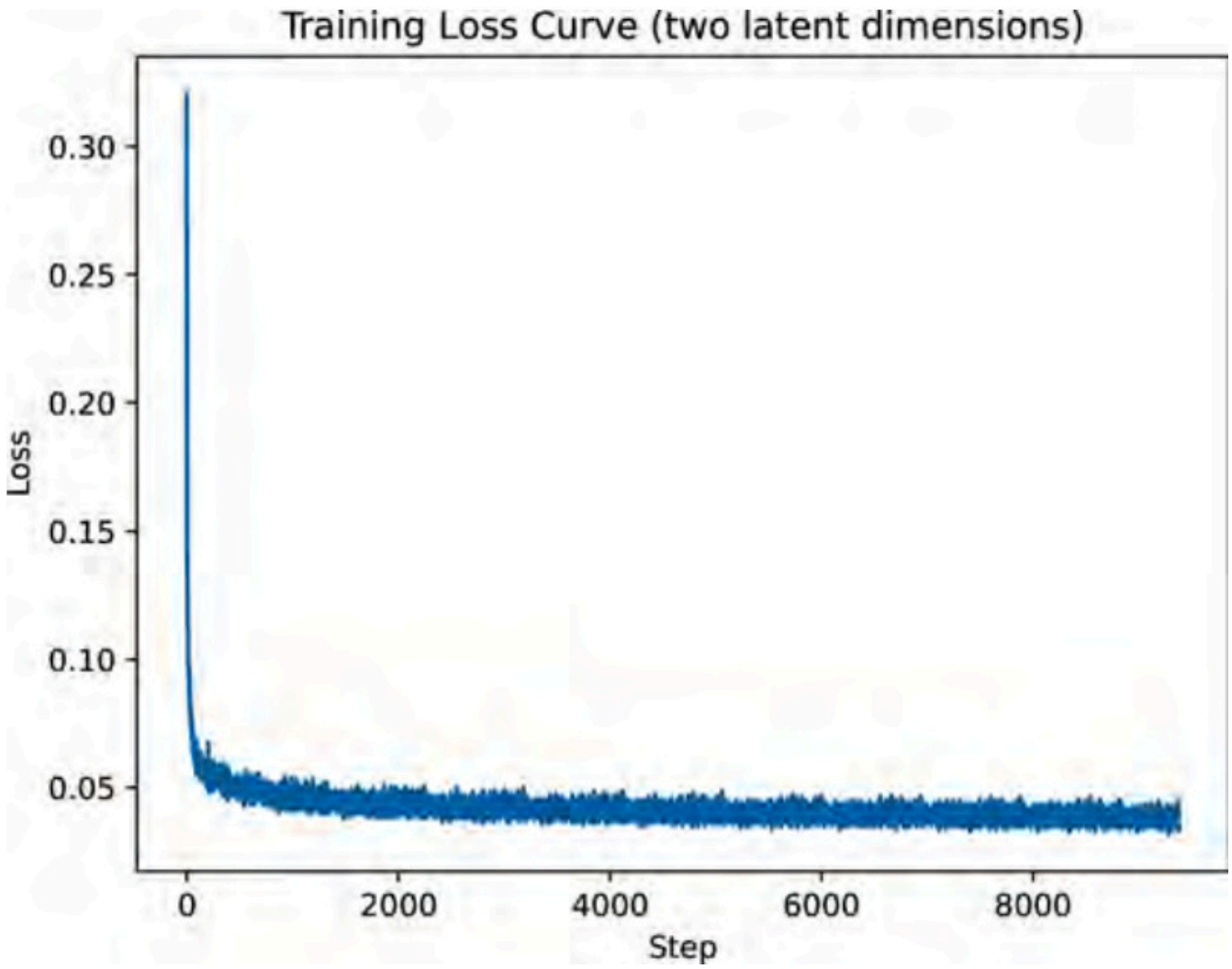
```
class AutoEncoder(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()
        self.encoder = Encoder(in_channels, latent_dims)
        self.decoder = Decoder(in_channels, latent_dims)

    def encode(self, x):
        return self.encoder(x)

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        return self.decode(self.encode(x))
```

训练循环与之前相同，但我们将其放入一个函数中以便重用并在需要时调用：

```
def train(model, num_epochs=10, lr=1e-4):
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)
    model.train()  # 将模型设置为训练模式
    losses = []
    for _ in (progress := trange(num_epochs, desc="Training")):
        for _, batch in (inner := tqdm(enumerate(train_dataloader), total=len(
            train_dataloader))):
            batch = batch.to(device)
            // 通过模型并获取另一组图像
            preds = model(batch)
            // 将预测与原始图像进行比较
            loss = F.mse_loss(preds, batch)

            // 显示损失并存储以绘制图表
            inner.set_postfix(loss=f"{loss.cpu().item():.3f}")
            losses.append(loss.item())

            // 根据此损失使用优化器更新模型参数
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
        progress.set_postfix(loss=f"{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
    return losses
```

我们创建并训练一个仅有两个潜在变量的 AutoEncoder：

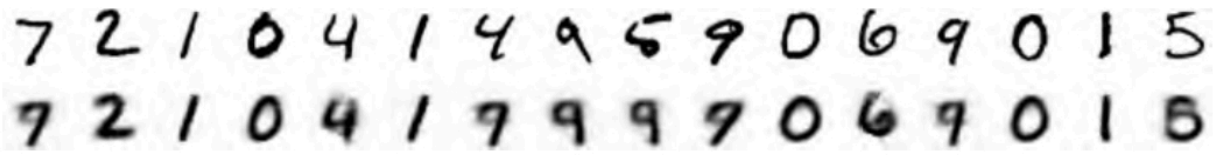
```
ae_model = AutoEncoder(in_channels=1, latent_dims=2)
ae_model.to(device)
losses = train(ae_model)
plt.plot(losses)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("Training Loss Curve (two latent dimensions)")
plt.show()
```




让我们再次加载训练好的模型并查看一些重建结果：

```
ae_model.eval()
with torch.inference_mode():
    eval_batch = next(iter(eval_dataloader))
    predicted = ae_model(eval_batch.to(device)).cpu()

batch_vs_preds = torch.cat((eval_batch, predicted))
show_images(batch_vs_preds, imsize=1, nrows=2)
```

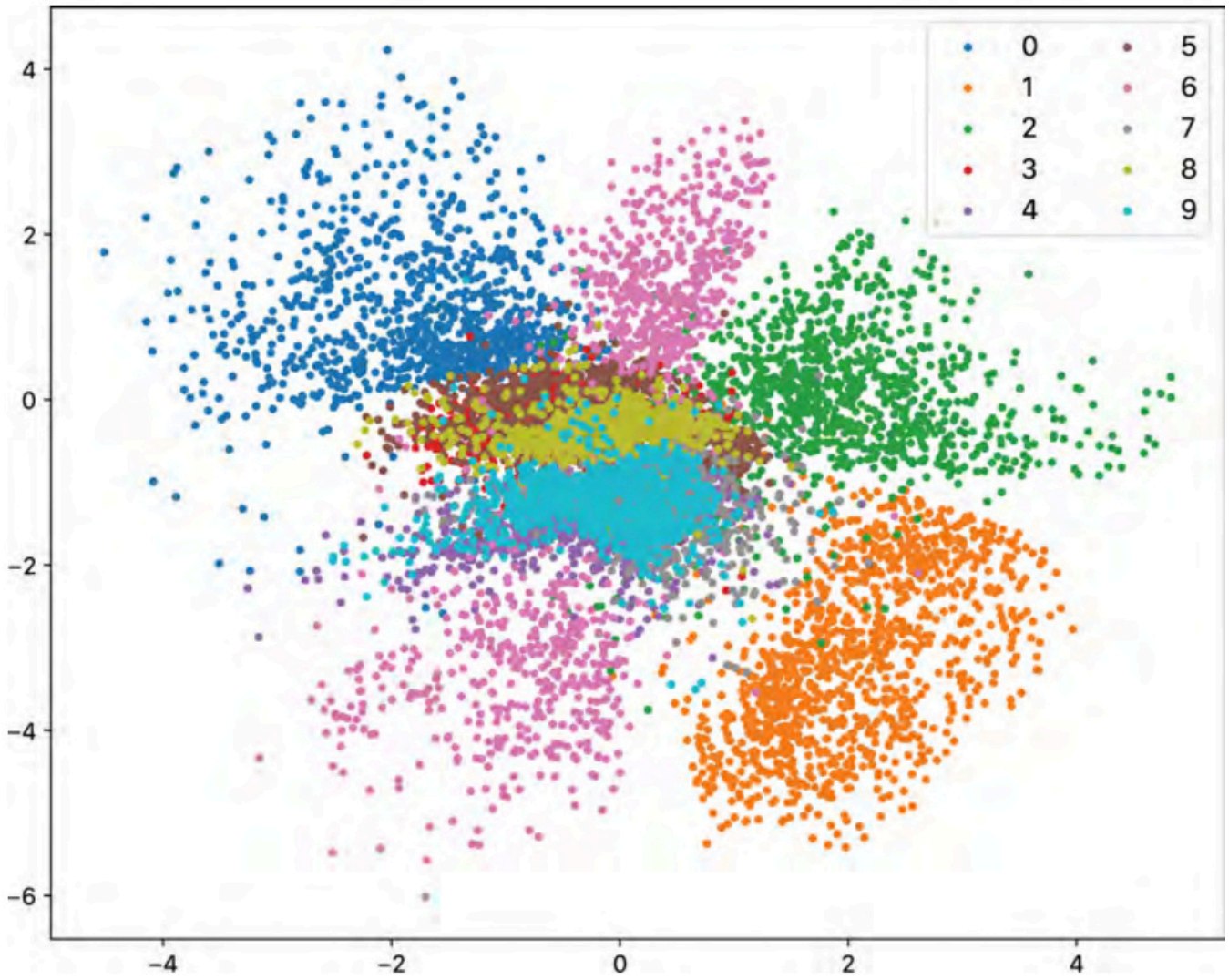
结果不如之前好，但请记住，我们现在仅使用两个浮点数来表示一个 $28 \times 28$ 的手写图像。我们看到一些 4、5 和 9 的混淆，但总体而言，恢复的图像与输入图像非常相似！



## 可视化潜在空间

我们仅使用两个维度来表示潜在空间，以便轻松可视化其结构。现在，我们将使用测试数据集中的所有编码向量，并使用标签列为每个类别分配不同的颜色。编码向量的第一个值将显示在x轴上，第二个值将显示在y轴上：

```
images_labels_dataloader = DataLoader(mnist["test"], batch_size=512)
import pandas as pd
df = pd.DataFrame( { "x": [], "y": [], "label": [], }
)
for batch in tqdm( iter(images_labels_dataloader), total=len(
    images_labels_dataloader)
): encoded = ae_model.encode(batch["image"].to(device)).cpu() new_items = { "x":
     [t.item() for t in encoded[:, 0]], "y": [t.item() for t in encoded[:, 1]],
    "label": batch["label"], } df = pd.concat([df, pd.DataFrame(new_items)],
    ignore_index=True)
plt.figure(figsize=(10, 8))
for label in range(10): points = df[df["label"] == label] plt.scatter(
    points["x"], points["y"], label=label, marker=".")
plt.legend();
```




自编码器在将潜在空间的不同区域分离方面做得很好，尤其是针对我们数据集中的各种图像。例如，请注意表示数字0的深蓝色点与表示数字1的橙色点之间的距离。请记住，在训练过程中没有使用任何关于图像标签的信息，但即便如此，数据点还是根据其视觉特征自动分组到不同的区域。然而，这个过程是完全无约束的，因此无法保证潜在空间的形状或结构。

因此，潜在空间足够丰富，能够捕捉到我们数据集中的相关图像特征，但我们仍然不清楚如何将其用于生成目的。理想情况下，为了生成与MNIST中图像相似的新图像，我们希望丢弃编码器，并向解码器提供来自潜在空间的随机样本。然而，我们可以在图中看到一些问题：

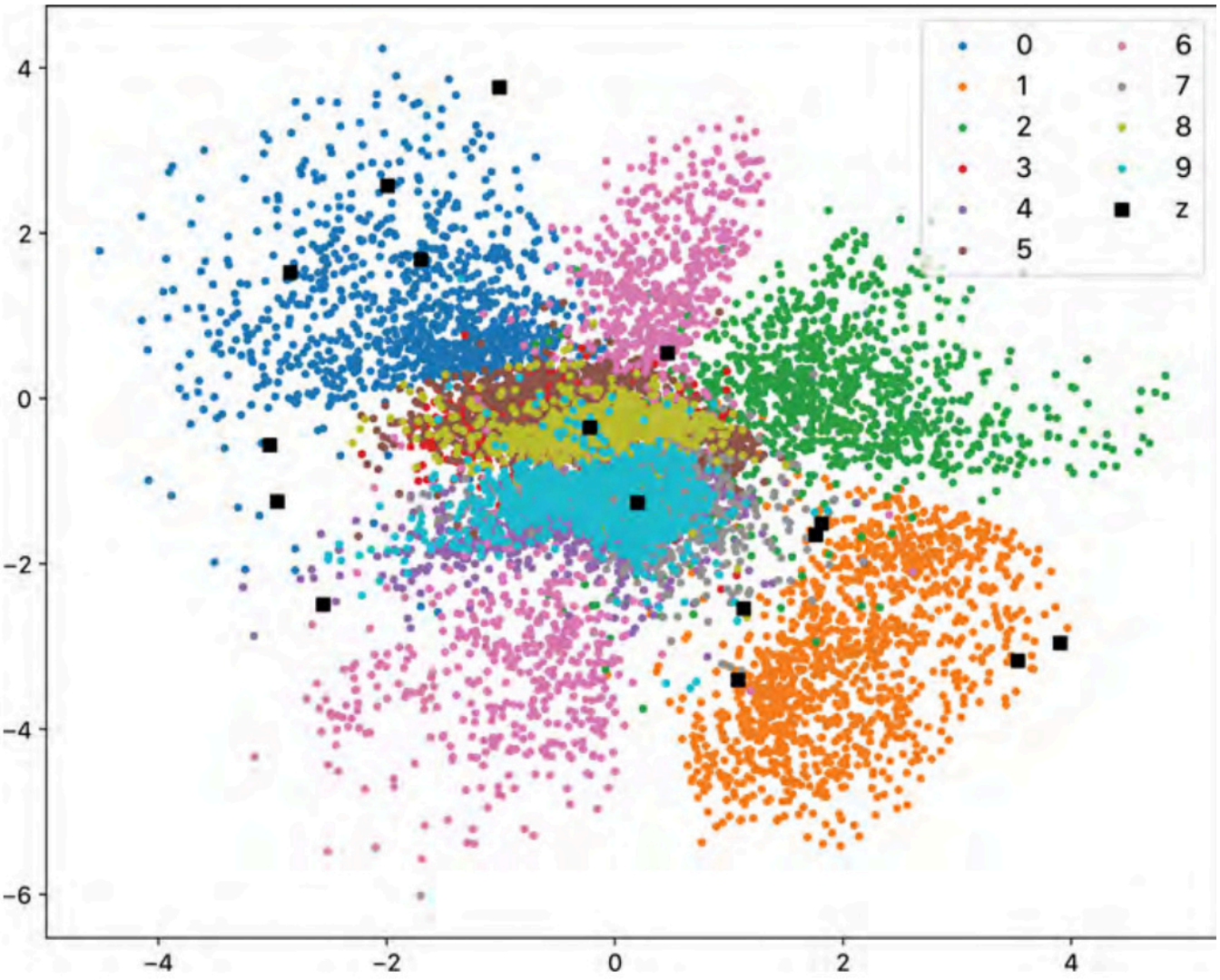
- 表示的空间在各个方向上分散。
- 中心区域有许多重叠，且存在大片空白区域。
- 图是非对称的：y轴的负值比正值使用得更多。

这使得选择潜在空间中的适当区域以生成高质量的图像变得具有挑战性。让我们看一个使用解码器生成图像的示例。我们首先生成随机的潜在样本（通常表示为z）：

```
N = 16  # 我们将生成16个点
z = torch.rand((N, 2)) * 8 - 4
```

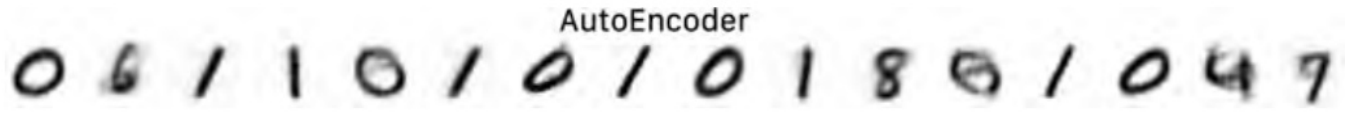
让我们将生成的潜在样本叠加到之前展示的潜在空间表示上：

```
plt.figure(figsize=(10, 8))
for label in range(10): points = df[df["label"] == label] plt.scatter(
    points["x"], points["y"], label=label, marker=".")
plt.scatter(z[:, 0], z[:, 1], label="z", marker="s", color="black")
plt.legend();
```




最后，我们让解码器从刚刚创建的潜在样本中生成图像：

```
ae_decoded = ae_model.decode(z.to(device))
show_images(ae_decoded.cpu(), imsize=1, nrows=1, suptitle="AutoEncoder")
```




生成的图像在样本接近模型在潜在空间中划分的某个区域时是合理的，但当它们位于这些区域之外时，效果就不那么令人信服了。此外，请注意，某些数字会被过度表示，因为它们在潜在空间中的分配区域更大。

下一节将讨论如何使用不同类型的自编码器在潜在空间中施加一些秩序，以及这如何使生成变得更加容易。

在继续之前，这里有一些（逐渐更具挑战性的）练习，你可以现在（或在章节结束时）完成，以加强对概念的理解：

1. 如果模型使用16个潜在维度进行训练，生成效果如何？
2. 使用我们使用的相同参数再次训练模型（只需运行本章中显示的代码），但使用不同的随机数初始化，并可视化潜在空间。很可能形状和结构会有所不同。这是你预期的吗？为什么？
3. 编码器提取的图像特征有多好？丢弃自编码器的解码器部分，并在编码器之上构建一个数字分类器。例如，你可以训练几个线性层，并在它们之间加入非线性。最终的线性层应输出一个10维的向量，表示数据集中的10个标签。仅训练这些层，而不更新编码器的权重。你能达到什么准确率？使用16个潜在维度的模型与仅使用两个的模型相比如何？

# 变分自编码器

在上一节中，我们探讨了简单的自编码器如何在低维潜在空间中学习输入数据的高效表示。自编码器可以忠实地编码任何样本，并在稍后恢复（或解码）它。这对于特征提取或数据表示非常有效，但不适合生成新样本。

如前所述，原因是自编码器没有被激励将表示分离到潜在空间的一致部分中。正如我们所看到的，相似输入的表示通常聚集在一起，但存在大量的重叠和空白区域，以及每个类别在潜在空间中分配的空间量存在显著差异。如果我们在潜在空间中选择一个随机点并将其传递给解码器，我们无法忠实地预测会出现什么结果。

变分自编码器（VAE）通过学习潜在空间中每个特征的概率分布来解决这个问题。VAE不是将输入映射到特定点，而是用高斯分布表示每个特征，捕捉该特征在数据中的变异性。

例如，考虑一个由多种狗和猫的图像组成的数据集。我们不知道编码器会提取哪些特征，但我们可以想象，其中一些特征可能用于表示诸如毛皮斑块、眼睛、耳朵、腿或尾巴等特征。这些特征可能在数据集中的所有图像中有很大程度的重叠（所有这些动物都有两只耳朵、四条腿和一条尾巴），但狗和猫的耳朵外观也存在变异性。我们的“耳朵”特征可以用一个高斯分布来表示，该分布涵盖了所有这些变异性，分布的均值代表动物耳朵的平均形状。如果我们从均值向不同方向移动，我们将获得向不同耳朵形状的连续且均匀的过渡，这些形状可能出现在各种品种中。

这种方法在图3-4中进行了概念性表示，它创建了一个更有结构的潜在空间，从这些分布中采样可以生成新的、合理的实例。与自编码器一样，VAE通常也不使用类别信息。让我们看看如何编写和训练它们。

---

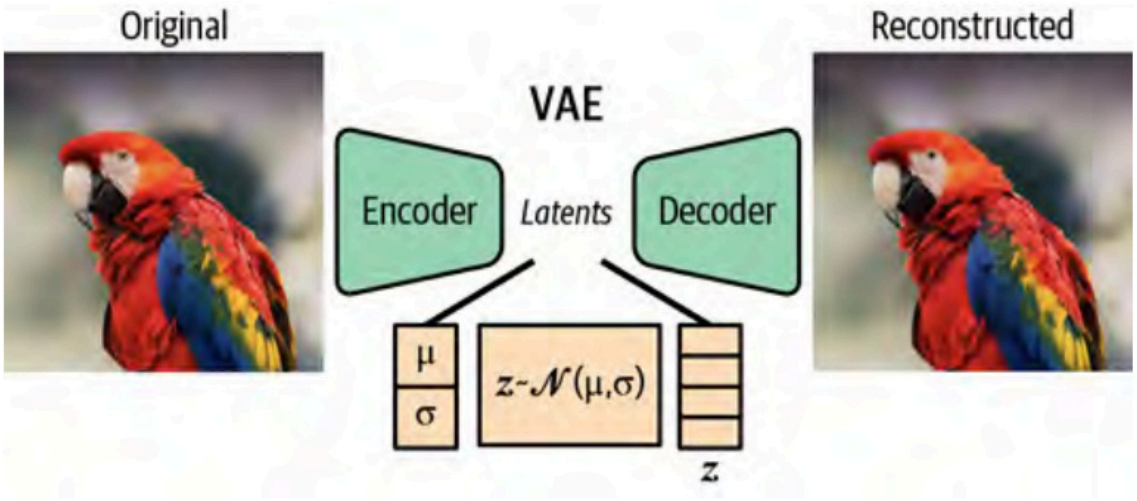



图3-4. VAE学习特征的高斯表示，并通过其均值和方差来描述它们。潜在空间中的点$z$是从预测的高斯分布中采样的。

## VAE 编码器与解码器

VAE 编码器与上一节中的基本编码器非常相似。在我们的示例中，我们使用了一些卷积层和一个线性层来投影到潜在表示的期望大小。

为了创建我们的第一个 VAE 编码器，我们将使用相同的架构。唯一的区别是，我们不再使用线性层来预测图像的潜在空间，而是使用线性层来学习分布。分布由两个参数表征，即均值和方差，因此我们需要两个线性层：

- 其中一个线性层将表示我们试图学习的分布的均值。
- 另一个线性层将学习分布的方差。

在代码方面，我们的第一个 VAE 编码器如下所示：

```
class VAEEncoder(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()
        self.conv_layers = nn.Sequential(
            conv_block(in_channels, 128),
            conv_block(128, 256),
            conv_block(256, 512),
            conv_block(512, 1024),
        )
        // 定义用于均值和对数方差的全连接层
        self.mu = nn.Linear(1024, latent_dims)
        self.logvar = nn.Linear(1024, latent_dims)

    def forward(self, x):
        bs = x.shape[0]
        x = self.conv_layers(x)
        x = x.reshape(bs, -1)
        mu = self.mu(x)
        logvar = self.logvar(x)
        return (mu, logvar)
```

如果你将这段代码与上一节中的编码器示例进行比较，你会发现差异很小。我们使用两个线性层而不是一个，从卷积层提取的相同表示中计算两个不同的值，并在 `forward()` 方法中返回这两个值。

这两个计算值的目的是表示概率分布的均值和方差。然而，它们最初只是两个相同的线性层。挑战在于确保它们在训练过程中学会表示我们想要的内容——均值和方差，正如我们将展示的那样。

在此之前，请注意计算值 `mu` 表示均值，`logvar` 表示方差的对数。我们使用 `mu` 来表示均值，源自希腊字母 $\mu$，在数学符号中经常用于表示正态分布的均值。使用 `logvar` 而不是直接输出方差的原因主要是数值稳定性，我们稍后会解释。

那么解码器呢？事实证明，我们不需要对它进行任何更改。VAE 和简单自编码器之间的区别在于我们找到潜在空间中的一个点来表示输入项的方式，但解码器的任务是相同的：给定潜在空间中的一个点（z），显示其编码表示与 z 最相似的像素。在自编码器的情况下，z 是卷积层提取

特征的线性投影。当我们使用 VAE 编码器时，我们得到一个正态分布，然后从该分布中采样以获得 z。因此，我们可以使用上一节中使用的相同 `Decoder` 类，但我们需要修改模型以从分布中采样。

## 从编码器分布中采样

我们更新后的 VAE 编码器返回一个正态分布的均值和方差，该分布试图匹配输入数据的表示。为了获得解码输出，我们必须从该分布中采样，如下面的代码片段所示：

```
class VAE(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()
        self.encoder = VAEEncoder(in_channels, latent_dims)
        self.decoder = Decoder(in_channels, latent_dims)

    def encode(self, x):
        // 返回 mu, log_var
        return self.encoder(x)

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        // 获取正态（高斯）分布的参数
        mu, logvar = self.encode(x)
        // 从分布中采样
        std = torch.exp(0.5 * logvar)
        z = self.sample(mu, std)
        // 将潜在点解码为像素空间
        reconstructed = self.decode(z)
        // 返回重建的图像，以及 mu 和 logvar
        // 以便我们可以计算分布损失
        return reconstructed, mu, logvar

    def sample(self, mu, std):
        // 重参数化技巧
        // 从 N(0, I) 中采样，然后平移和缩放
        eps = torch.randn_like(std)
        return mu + eps * std
```

1 我们使用 `latent_dims` 维度来表示分布的均值和对数方差。
编码器现在计算两个变量：均值和对数方差。
从对数方差计算标准差。
通过使用计算出的均值和标准差从分布中采样。
解码器将样本转换为图像。
我们不仅返回重建的图像，还返回均值和对数方差。
重参数化技巧：从标准正态分布中采样，然后平移和缩放。

到目前为止，我们通过直觉解释了 VAE 的工作原理。如果你允许我们稍微偏离主题，回顾一些统计概念，我们可以快速回顾前面的代码，同时尝试在术语上更加精确。你可以跳过本节或稍后再回来阅读。理解它并不是使用或训练 VAE 的必要条件，但如果你想深入研究并阅读有关该主题的论文，它可能会有所帮助。

首先，请注意我们使用多维高斯分布，而不仅仅是实值的一维正态曲线。在这个 VAE 编码器示例中，我们使用 `latent_dims` 来表示均值和方差。我们可以使用任意数量的维度，比如我们第一个 MNIST 自编码器使用的 16 维，或者 2 维以便于可视化。实值（一维）正态分布表示为 $\mathcal{N}(\mu, \sigma^2)$，并由两个量定义：$\mu$，分布的均值，和 $\sigma_i$，标准差，它是方差 $\sigma^2$ 的平方根。

正态分布的一个有用特性是，所有正态分布都可以通过平移和缩放标准正态分布来表示，标准正态分布的均值为 0，方差为 1：

$$\mathcal{N}(\mu, \sigma^2) = \mu + \sigma\mathcal{N}(0, 1)$$

这意味着，要从任意正态分布 $\mathcal{N}(\mu, \sigma^2)$ 中获取样本，我们可以改为从 $\mathcal{N}(0, 1)$ 中采样，然后乘以 $\sigma$ 并加上 $\mu$。这被称为重参数化，当我们研究扩散模型时，这将非常有用。

多维高斯分布被称为多元高斯分布。它们仍然可以由两个参数定义，区别在于 $\mu$ 是一个向量，$\sigma$（协方差矩阵，现在用 $\mathbf{\Sigma}$ 表示）是一个矩阵。因此，分布定义为 $\mathcal{N}(\mu, \Sigma)$。如果分布在所有维度上都是独立的，意味着每个变量与其他变量不相关且具有相同的方差，则称为各向同性。在各向同性多元高斯分布中，协方差矩阵 $\Sigma$ 是一个对角矩阵，其中对角线上的所有元素都相等，可以表示为 $\sigma^2\mathbf{I}$, 其中 I 是单位矩阵。标准多元高斯分布则表示为 $\mathcal{N}(0, \mathbf{I})$。

我们前面的 VAE 示例建模了一个多元、各向同性的高斯分布，因为没有理由认为样本坐标相互依赖（而且更简单！）。这意味着我们可以使用所谓的重参数化技巧从标准高斯分布中采样，然后平移和缩放以获得我们将解码的潜在空间向量。

重参数化技巧不仅仅是为了方便——它是训练的关键要素。当我们使用表达式 `mu + eps * std` 时，其中 `eps` 是标准高斯分布的样本，模型输入的梯度与随机过程（从分布中采样）无关，因

此可以计算。这使得我们可以使用熟悉的梯度下降方法来训练模型，就像我们训练任何神经网络一样。

说到稳定性，我们的模型预测方差的对数而不是方差，以增加数值稳定性并促进训练。在数学上，计算其中一个或另一个没有区别。在实践中，我们知道方差始终是一个正数，通常接近 0。然而，当我们开始训练时，模型没有理由产生正且小的值。此外，数字以浮点格式表示，这使得很难区分非常接近的值。通过取对数，我们得到了两个好处：

- 我们将可接受值的范围扩展到 $-\infty$，因此模型在表达浮点值结果时有更大的自由度。
- 我们确保方差始终为正，因为它是 `logvar` 的指数。

高斯分布经常用于机器学习的许多其他领域，有时是为了方便，因为它们的数学特性众所周知。在第 4 章中，我们将探讨它们用于建模噪声污染的使用，这是扩散模型的重要组成部分。

## 训练 VAE

训练 VAE 的关键在于损失函数。在第 57 页的“AutoEncoders”中，我们使用的损失函数衡量了重建图像与原始图像之间的差异。我们仍然希望重建图像尽可能接近原始图像，但现在我们为损失函数引入了第二个因素，以施加我们一直在讨论的 VAE 约束：我们希望特征（或多或少）遵循高斯分布。

我们实现这一目标的方式是使用分布之间的 Kullback-Leibler 散度，也称为相对熵。Kullback-Leibler 散度，或 KL 散度（KLD），是一种衡量一个概率分布与另一个分布差异的方法。在多变量各向同性高斯分布的情况下，可以证明 KLD 可以按如下方式计算：

$$D_{KL}[\mathcal{N}\left(\mu, \sigma^2\right)||\mathcal{N}(0,1)] = -\frac{1}{2}\sum\left(1 + log\left(\sigma^2\right) - \mu^2 - \sigma^2\right)$$

为了结合这两个损失因素，我们创建了一个名为 `vae_loss()` 的损失函数，它接收原始图像和编码器的输出，并执行以下操作：

- 计算重建损失，作为解码器生成的像素与原始图像之间的均方误差（MsE）。这个损失因素

与我们用于训练 AutoEncoders 的损失因素相同。

- 根据我们刚刚提出的公式计算 KLD 项。
- 将它们相加。我们可以为其中一个或另一个分配更多的重要性，以平衡重建保真度与高斯分布的符合性。我们现在将它们相加，但调整这个平衡是一个值得尝试的实验。

损失函数返回三个值：总损失、重建损失和 KLD 项。我们只需要总损失来进行训练，但我们保留其他两个值以进行可视化和分析：

```
def vae_loss(batch, reconstructed, mu, logvar):
    bs = batch.shape[0]
    // 重建损失，来自像素 - 每张图像 1 个
    reconstruction_loss = F.mse_loss(
        reconstructed.reshape(bs, -1),
        batch.reshape(bs, -1),
        reduction="none",
    ).sum(dim=-1)
    // KL 散度损失，每张输入图像
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=-1)
    // 结合两个损失并计算图像的平均值
    loss = (reconstruction_loss + kl_loss).mean(dim=0)
    return (loss, reconstruction_loss, kl_loss)
```

现在我们已经定义了损失函数，可以继续训练模型。我们将使用总损失来更新模型权重，但我们也会跟踪重建损失和 KLD 项，以了解模型的学习情况：

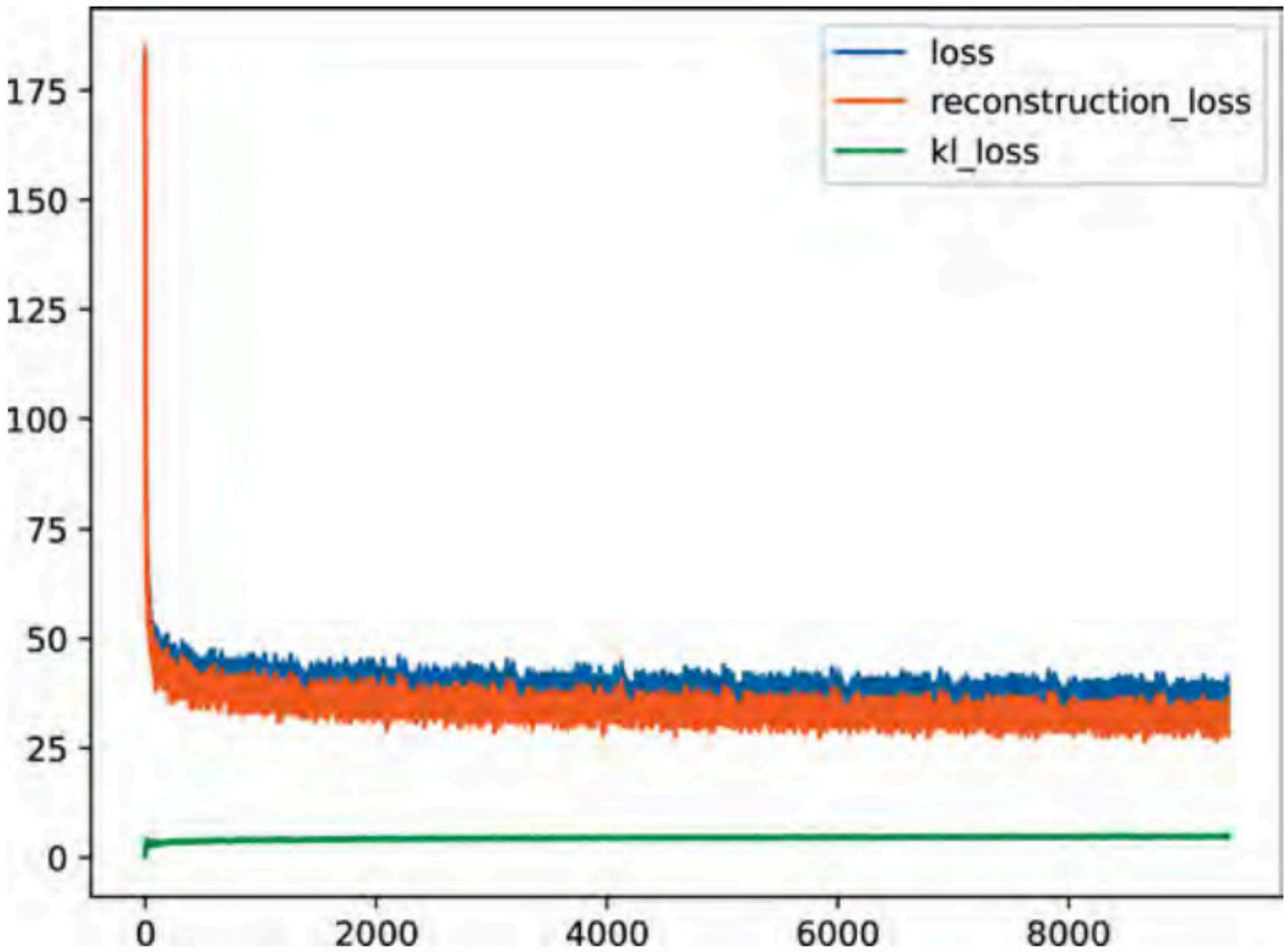
```
def train_vae(model, num_epochs=10, lr=1e-4):
    model = model.to(device)
    losses = {
        "loss": [],
        "reconstruction_loss": [],
        "kl_loss": [],
    }
    model.train()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)
    for _ in (progress := trange(num_epochs, desc="Training")):
        for _, batch in (
            inner := tqdm(
                enumerate(train_dataloader),
                total=len(train_dataloader)
            )
        ):
            batch = batch.to(device)
            // 通过模型传递
            reconstructed, mu, logvar = model(batch)
            // 计算损失
            loss, reconstruction_loss, kl_loss = vae_loss(
                batch, reconstructed, mu, logvar
            )
            // 显示损失并存储以供绘图
            inner.set_postfix(loss=f"{loss.cpu().item():.3f}")
            losses["loss"].append(loss.item())
            losses["reconstruction_loss"].append(
                reconstruction_loss.mean().item()
            )
            losses["kl_loss"].append(kl_loss.mean().item())
            // 根据总损失更新模型参数
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        progress.set_postfix(loss=f"{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
    return losses
ae_model = VAE(in_channels=1, latent_dims=2)
losses = train_vae(vae_model, num_epochs=10, lr=1e-4)
```

让我们分析一下在训练期间存储的三个损失项：

重建损失 衡量输出图像与原始图像的相似程度
KLD 衡量特征遵循高斯分布的程度
总损失 前两个损失的总和

现在让我们绘制训练 VAE 时的损失组件。总损失是重建损失加上 KL 项的总和：

```
for k, v in losses.items():
    plt.plot(v, label=k)
    plt.legend();
```




尽管总损失是这两个损失的总和，但在本例中，它主要由重建损失主导，因为其幅度远大于 KLD。让我们分别绘制损失组件，看看它们在训练期间如何演变，如图 3-5 所示。

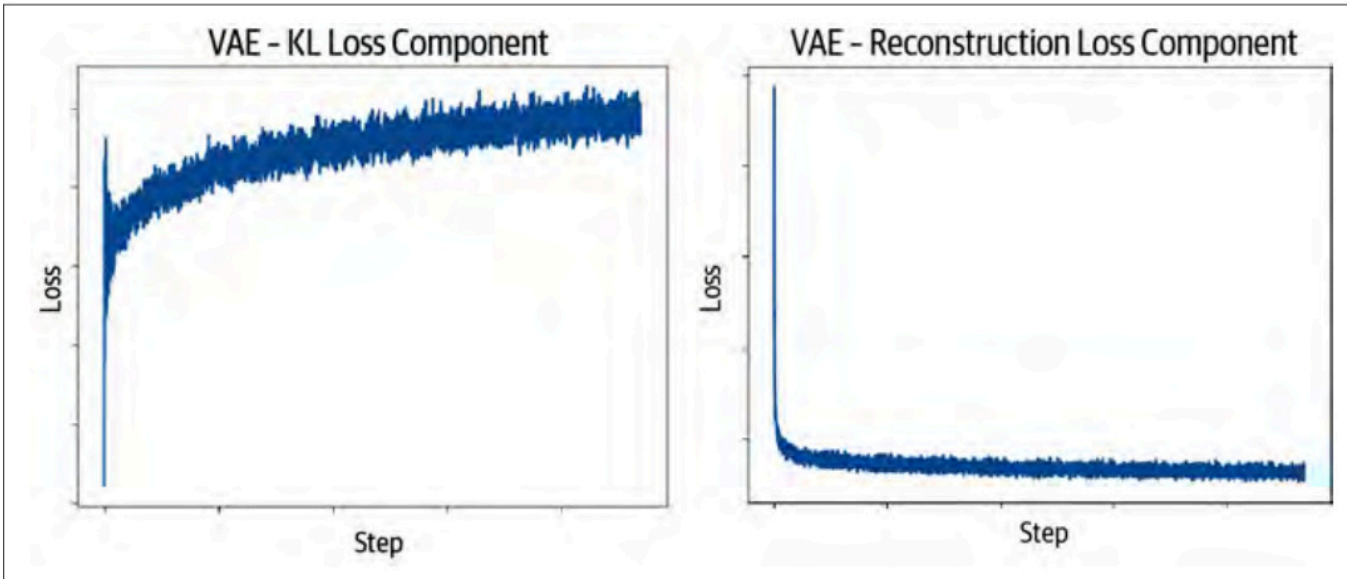


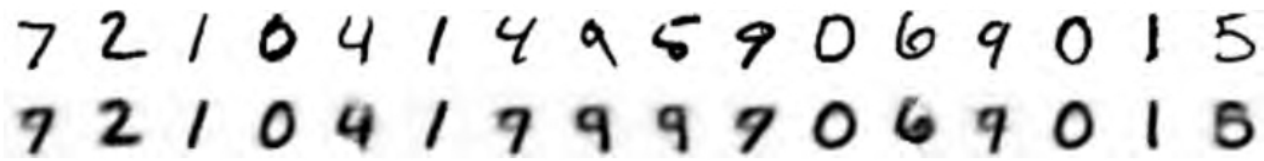

图 3-5. VAE 训练期间的 KL 损失和重建损失组件

请注意，KL 损失图有一个特殊的形式：它在训练开始时飙升，然后下降，然后再次缓慢上升。为什么会发生这种情况？我们认为这些是 KLD 在训练中经历的概念阶段：

- 当训练开始时，VAE 编码器和解码器用随机权重初始化，模型对输入数据一无所知。因此，输出类似于随机分布，KLD 非常低。
- 当网络只看到几批数据时，重建质量会很低，但不再是随机的，KLD 会飙升。
- 在训练的早期阶段，模型只知道如何表示输入数据的平均特征。KLD 项驱动模型产生越来越接近高斯分布的输出，从而降低 KLD 损失。
- 随着编码器和解码器学会产生更忠实的表示，在仍然匹配高斯分布的同时提高质量变得更加困难。重建损失占主导地位，KLD 增加，但两者之间存在平衡。如果我们在损失项中增加 KLD 的重要性，我们可以实现更好的高斯符合性，但代价是像素表示更差。

让我们重建一些图像，并可视化从 MNIST 样本中 VAE 重建的结果：

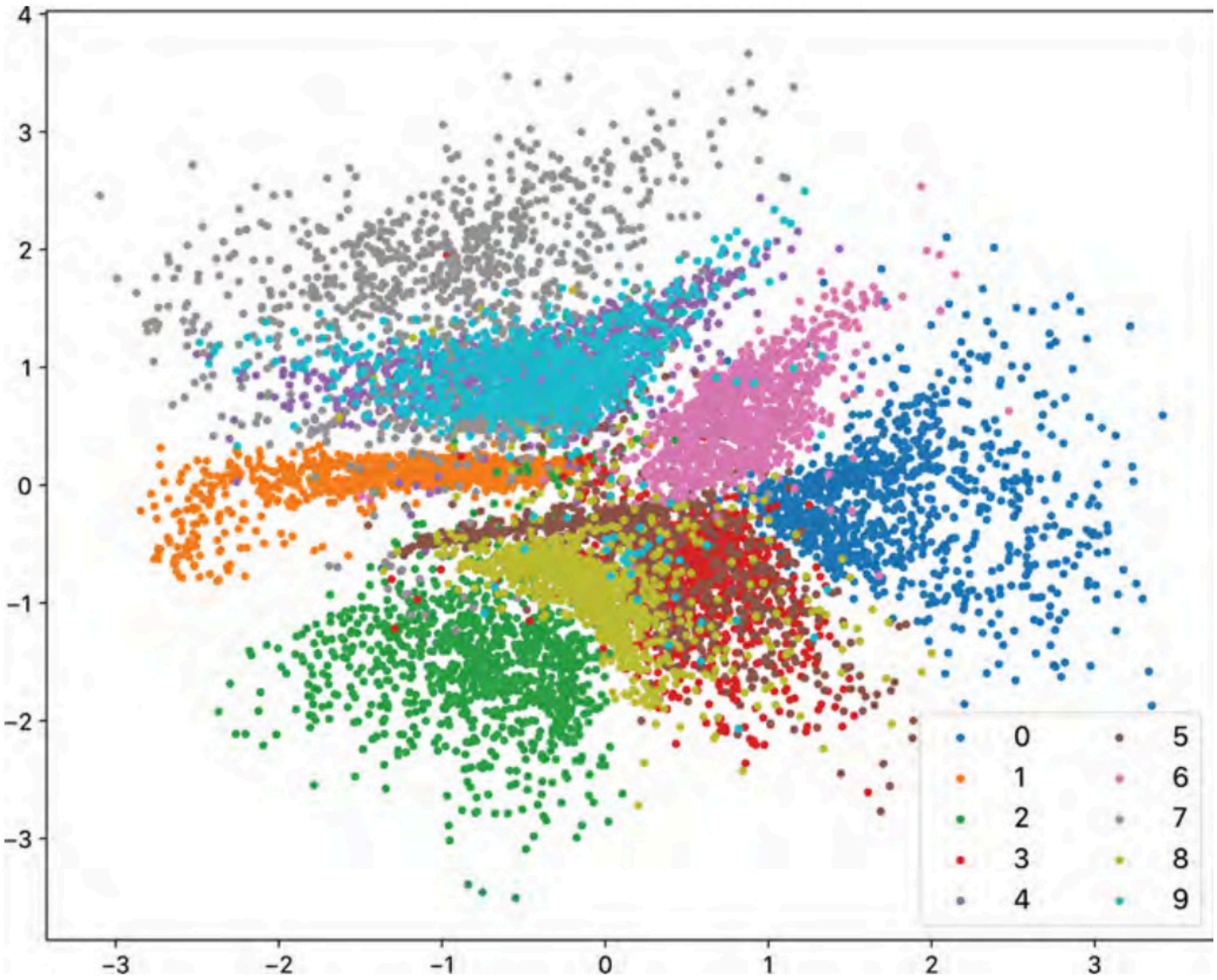
```
vae_model.eval()
with torch.inference_mode():
    eval_batch = next(iter(eval_dataloader))
    predicted, mu, logvar = (v.cpu() for v in vae_model(eval_batch.to(device)))

batch_vs_preds = torch.cat((eval_batch, predicted))
show_images(batch_vs_preds, imsize=1, nrows=2)
```



视觉结果比 AutoEncoder 的情况更差，因为模型不仅需要学习如何编码输入图像，还受到约束，试图避免偏离正态分布太远。让我们探索一下在添加了这个新目标后发生了什么。

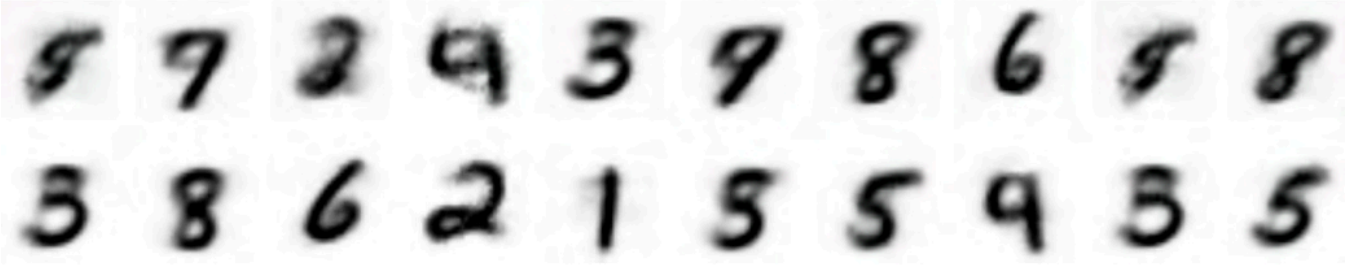
如果我们绘制模型为测试集编码的标准分布的均值，我们会得到一个比 AutoEncoder 情况下稍微更好的结果，如下图所示。结果现在更集中在零附近，并且不像 AutoEncoder 情况下那样远离。不同类别占据的区域大小相似，尽管相似数字之间仍然存在重叠：

```
df = pd.DataFrame(
{
    "x": [],
    "y": [],
    "label": [],
}
for batch in tqdm(
    iter(images_labels_dataloader),
    total=len(images_labels_dataloader)
):
    mu, _ = vae_model.encode(batch["image"].to(device))
    mu = mu.to("cpu")
    new_items = {
        "x": [t.item() for t in mu[:, 0]],
        "y": [t.item() for t in mu[:, 1]],
        "label": batch["label"],
    }
    df = pd.concat([df, pd.DataFrame(new_items)], ignore_index=True)
plt.figure(figsize=(10, 8))
for label in range(10):
    points = df[df["label"] == label]
    plt.scatter(points["x"], points["y"], label=label, marker=".")
plt.legend();
```




尝试将编码器拟合到正态分布的主要优势是，我们现在应该能够从分布中随机采样数据，并希望获得类似于我们输入数据集的图像。让我们看看它在 AutoEncoder 和 VAE 中的效果如何：

```
z = torch.normal(0, 1, size=(10, 2))
ae_decoded = ae_model.decode(z.to(device))
vae_decoded = vae_model.decode(z.to(device))
show_images(ae_decoded.cpu(), imsize=1, nrows=1)
show_images(vae_decoded.cpu(), imsize=1, nrows=1)
```



我们从正态分布中采样纯随机数据，然后使用自编码器（上排）和变分自编码器（下排）的解码器来展示这些点如何被重建。当然，我们会看到不同的重建结果，因为自编码器和变分自编码器是分别训练的，并且它们为每个类别分配了不同的潜在空间部分。

变分自编码器的结果比自编码器的结果更像数字。这是因为变分自编码器的训练过程鼓励编码器不要偏离正态分布太远，而自编码器没有这样的限制。你能多次运行采样代码并亲自观察吗？

一个有趣的练习是展示当我们沿着潜在空间的二维路径移动时，表示如何变化。我们可以在X=-0.8处固定一条垂直线，如图3-6所示，并探索这条线上的不同点。

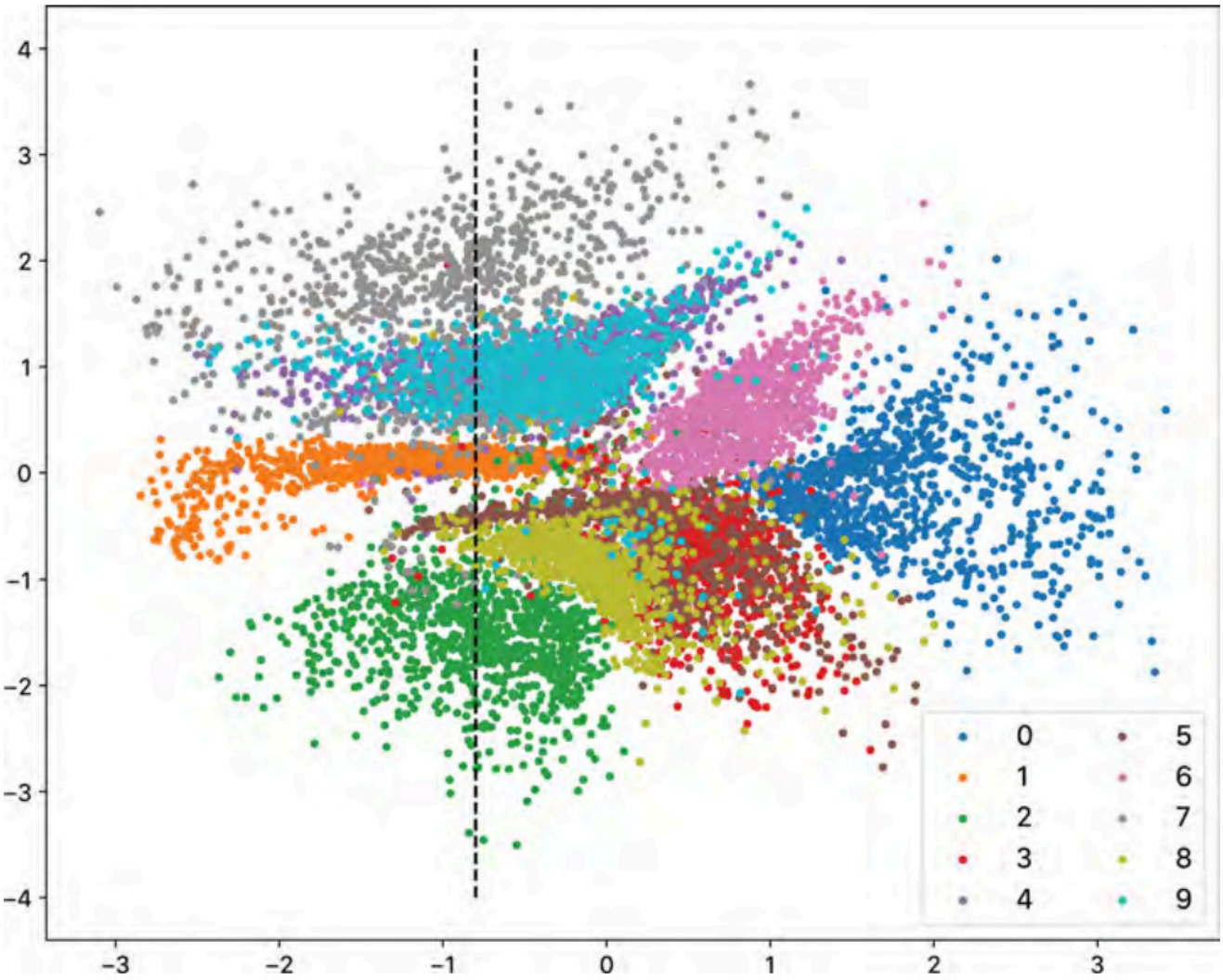
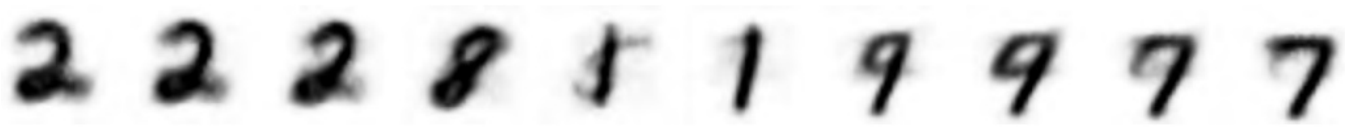



图3-6. 变分自编码器潜在空间，聚焦于x=-0.8的样本

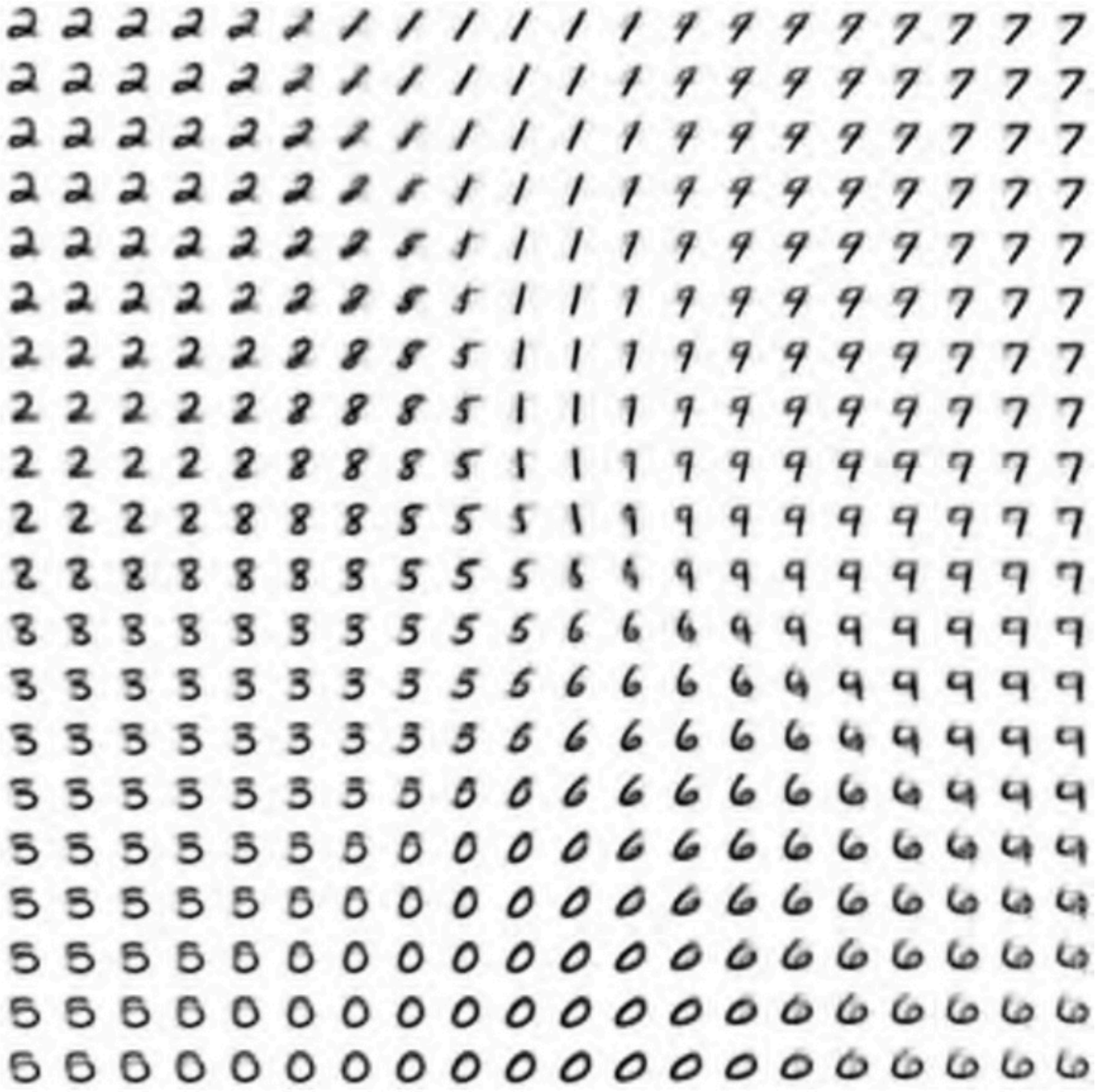
如果我们从y=-2到y=2选择点，我们会看到重建结果与表示各种数字的潜在空间区域相匹配：

```
import numpy as np

with torch.inference_mode():
    inputs = []
    for y in np.linspace(-2, 2, 10):
        inputs.append([-0.8, y])
    z = torch.tensor(inputs, dtype=torch.float32).to(device)
    decoded = vae_model.decode(z)
    show_images(decoded.cpu(), imsize=1, nrows=1)
```



让我们将探索潜在空间的想法扩展到二维网格。这是模型学习到的视觉表示，有趣的是，过渡并不太疯狂！

```
inputs = []
for x in np.linspace(-2, 2, 20):
    for y in np.linspace(-2, 2, 20):
        inputs.append([x, y])
z = torch.tensor(inputs, dtype=torch.float32).to(device)
decoded = vae_model.to(device).decode(z)
show_images(decoded.cpu(), imsize=0.4, nrows=20)
```



再次，这里有一些练习可以深入探讨这些主题：

1. 当我们训练变分自编码器时，我们添加了重建损失和KL散度损失。然而，两者的尺度不同。如果我们更重视其中一个，会发生什么？你能运行一些实验并解释结果吗？
2. 我们在本节中探索的变分自编码器仅使用两个维度来表示分布的均值和logvar。你能使用16个维度进行类似的探索吗？
3. 人类经过训练可以轻松识别面部特征中的不真实部分。你能为包含面部的数据集训练一个自编码器和一个变分自编码器并分析结果吗？你可以从变分自编码器论文中使用的Frey Face数据集开始——这是一个同质的单色图像集，展示了同一个人的不同面部表情。如果你想更

有野心，可以尝试使用CelebFaces Attributes数据集，该数据集可以从Hugging Face Hub轻松获取。另一个有趣的例子是Oxford pets数据集，同样可以在Hugging Face Hub上找到。

### 用于生成建模的VAEs

训练一个编码器使其接近某个分布是VAEs的一个关键见解，也是生成建模的基石之一。通过AutoEncoder，我们可以学习数据集的高效表示。然而，模型学习到的潜在空间并不能保证帮助我们生成与原始数据相似的新数据。通过旨在学习一个分布，VAEs允许我们从潜在空间中的随机点开始生成合理的新图像。扩散模型将这种从随机噪声中采样的想法更进一步，通过将迭代优化引入到这一过程中。我们将在未来的章节中详细讨论它们。

## CLIP

到目前为止，我们主要关注的是图像数据。通过对比语言-图像预训练（CLIP），我们将远离AutoEncoder/VAE方法，探索一种不同的技术来匹配图像和文本。这个过程在某种意义上与我们的目标相似，即从输入数据中创建丰富的表示，但方法不同，更重要的是，它可以同时处理图像和文本。

我们的数据集现在由两种模态组成：图像和描述这些图像的文本。给定这些训练数据，CLIP的目标是创建一个模型，用于衡量我们提供的任意文本-图像对中文本描述图像内容的准确性。关键，一如既往，在于我们使用的损失函数。

### 对比损失

CLIP由OpenAI于2021年推出。它是用于创建最初的DALL·E的工具之一，DALL·E是一个令人印象深刻的文本到图像模型，席卷了全球。尽管DALL·E没有开源（模型权重仍然是私有的），但CLIP是开源的。这是一个非凡的消息，因为将图像与文本关联起来的能力带来了许多技巧。CLIP和类似CLIP的模型自此成为生成领域中不可或缺的工具。

CLIP使用了一种称为对比损失的损失函数。其工作原理如图3-7所示。训练数据集由数百万张图像及其相关的描述或标题组成。对于每个图像-标题对，我们使用任何图像编码器对图像进行编码，以获得编码器潜在空间中的嵌入向量。图中的$I_I, I_2, ..., I_N$框表示不同图像的嵌入向量。文本也被编码，通常使用我们在第2章中看到的Transformer模型。关键的是，我们使用的编码器使得图像和文本的嵌入向量的维度相同。这样，我们可以计算文本和图像嵌入之间的内积（或点积）来确定它们的接近程度。

训练过程通过在同一个批次中提供大量图像-文本对来进行。我们计算批次中所有图像嵌入与所

有文本嵌入的点积，并尝试最大化来自同一对的项目的乘积（即图像中的蓝色对角线），同时最小化其余部分。这样，相似的文本和图像将由潜在空间中接近的向量表示，而不同的概念将在其他区域远离。

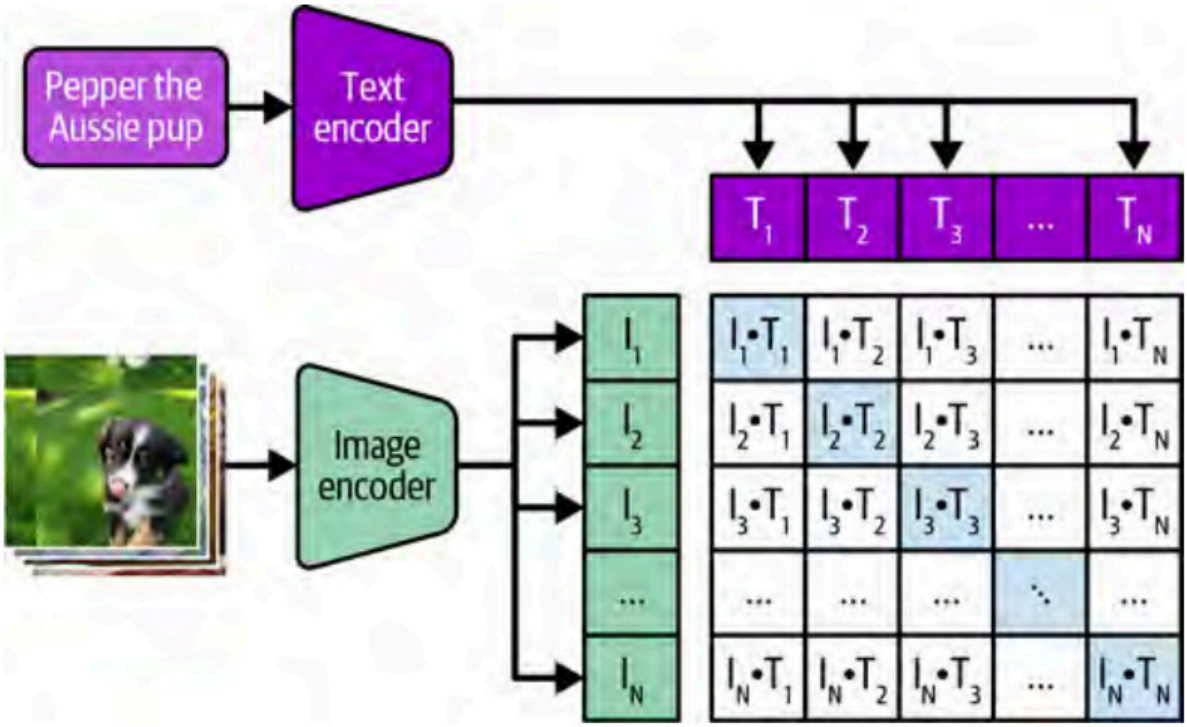



图3-7. CLIP（改编自OpenAI的图像）

为什么使用点积?

如果你不记得或以前没有学过微积分，向量空间中的一个重要关系如下：

$$A \cdot B = |A|\,|B| \cos(\theta)$$

这意味着两个向量之间的点积等于两个向量长度的乘积乘以它们之间的夹角。这可以使用欧几里得几何中的余弦定理和点积的定义$A \cdot B = \Sigma_{n=1}^{N} a_i b_i$.来证明。在我们简短的讨论中，我们没有提到向量嵌入被归一化为单位长度；因此，它们的点积是向量之间的夹角。这被称为余弦相似度，它衡量两个向量之间的接近程度。尽管向量有很多维度，但点积只是一个标量（即实数），可以用作相似度评分。

训练CLIP模型需要大量的数据和计算资源。OpenAI发布的原始CLIP模型使用了4亿个图像-文本对的专有数据集，并且批次大小为32,000对。从那时起，已经有多项努力来训练CLIP或类似CLIP的模型，包括支持音频等其他模态：

OpenCLIP

CLIP的一个开源实现。它被用于训练多个模型，使用不同的数据集、图像分辨率和模型大小。

CLAP（对比语言-音频预训练）

允许获取音频的表示，而不是图像。这可以用于训练生成音频的模型，我们将在第9章中探讨。

## 使用 CLIP，逐步操作

我们现在将使用预训练的 CLIP 模型来直观了解它们的工作原理，并展示一些它们可以用于的示例。在以下示例中，我们使用 `clip-vit-large-patch14`，这是 OpenAI CLIP 模型之一。它使用 Vision Transformer (ViT) 作为图像编码器（其他版本使用 ResNet 卷积架构）。还有更大和更小的版本；你可以尝试几个，看看它们在硬件上的速度、内存和质量表现如何。

考虑图 3-8 中的照片，这是来自 Pixabay 的免版税资源，展示了一只可爱的狮子幼崽从树枝后面看着我们。让我们看看如何将其与 CLIP 一起使用。

图 3-8. 树枝后面的可爱狮子幼崽照片

到目前为止，我们只使用 transformers 加载文本模型，但我们也可以使用它们来处理其他模

态。正如我们在前一章中使用 `GPT2LMHeadModel` 一样，我们可以使用 `CLIPModel`。让我们试试：

```
import requests
from PIL import Image
from transformers import CLIPModel, CLIPProcessor
from genaibook.core import SampleURL
clip = CLIPModel.from_pretrained("openai/clip-vit-large-patch14").to(device)
processor = CLIPProcessor.from_pretrained("openai/clip-vit-large-patch14")
url = SampleURL.LionExample
image = Image.open(requests.get(url, stream=True).raw)
```

我们加载的 CLIP 模型 `clip` 包含两个组件：一个用于编码图像的视觉模型和一个用于编码文本的文本模型。`CLIPProcessor`，你可以将其视为等效的分词器，它准备输入数据以匹配模型训练期间使用的预处理步骤：图像调整大小、归一化等。这对于确保我们为推理提供的输入具有与模型训练时看到的数据相同的特征至关重要。让我们先处理图像：

```
image_inputs = processor(images=image, return_tensors="pt")
pixel_values = image_inputs["pixel_values"]
pixel_values.shape, pixel_values.min(),
pixel_values.max()
```

(torch.Size([1, 3, 224, 224]), tensor(-1.7923), tensor(2.0179))

图像已被调整为 224 像素的正方形并进行了归一化。你可以检查图像处理器以查看完整的应用变换集。请注意，调整大小的策略是中心裁剪（即选择图像中心周围的方形块，然后将其缩小到 $224 \times 224$）。这种方法会从横向图像中裁剪左右部分，或从纵向图像中裁剪上下部分。请注意，如果你想要检查的某些主体位于这些区域，可能会导致信息丢失：

```
processor.image_processor
```

CLIPImageProcessor { "crop_size": { "height": 224, "width": 224 }, "do_center_crop": true, "do_convert_rgb": true, "do_normalize": true, "do_rescale": true, "do_resize": true, "image_mean": [ 0.48145466, 0.4578275, 0.40821073 ], "image_processor_type": "CLIPImageProcessor", "image_std": [ 0.26862954, 0.26130258, 0.27577711 ], "resample": 3, "rescale_factor": 0.00392156862745098, "size": { "shortest_edge": 224 }

}

让我们验证一下我们的狮子幼崽照片能否在中心裁剪中幸存：

```
width, height = image.size
crop_length = min(image.size)

left = (width - crop_length) / 2
top = (height - crop_length) / 2
right = (width + crop_length) / 2
bottom = (height + crop_length) / 2

cropped = image.crop((left, top, right, bottom))
cropped
```

我们照片中的主体完全保留了下来。根据你的数据，你可能需要在将源图像传递给处理器之前对其进行裁剪，以确保主体可见。

我们现在将从预处理后的图像中获取嵌入向量。我们使用存储在 `clip` 实例中的视觉模型来完成此操作。这个子组件有时被称为视觉塔：

```
with torch.inference_mode():
    output = clip.vision_model(pixel_values.to(device))

image_embeddings = output.pooler_output
image_embeddings.shape
```

torch.Size([1, 1024])

视觉模型返回一个包含最后隐藏状态和池化输出的字典。这个形状为 [1, 1024] 的向量表示编码过程的结果。现在让我们将注意力转向模型的语言部分。我们将遵循类似的过程来获取两个文本提示的嵌入：“a photo of a lion” 和 “a photo of a zebra”。我们的最终目标是比较图像嵌入与每个提示嵌入之间的余弦相似度。希望狮子的描述能更好地匹配图像！

```
prompts = [ "a photo of a lion", "a photo of a zebra",
]

// Padding makes sure all inputs have the same length
text_inputs = processor(text=prompts, return_tensors="pt", padding=True)
```

```
{'attention_mask':
    tensor([[1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1]]),
'input_ids':
    tensor([[49406, 320,  1125, 539,   320,  5567, 49407],
        [49406,   320,  1125, 539, 320, 22548, 49407]])}
```

文本处理将输入字符串分词，就像前一章一样。我们现在可以通过模型的语言塔部分传递分词后的文本输入，以获取提示嵌入：

```
text_inputs = {k: v.to(device) for k, v in text_inputs.items()}
with torch.inference_mode():
    text_output = clip.text_model(**text_inputs)
    text_embeddings = text_output.pooler_output
    text_embeddings.shape
```

torch.Size([2, 768])

我们通过使用两个输入提示的批次得到了两个向量。然而，每个向量有 768 维，而图像的嵌入

有 1,024 维。请记住，要计算两个向量的点积，它们必须具有相同的维度数，我们在本节介绍中强调过，文本编码器和图像编码器必须生成具有相同维度的嵌入。我们之前没有提到的一个额外步骤是，我们可以选择生成相同维度的编码器模型，或者我们可以使用任意的文本和图像编码器，并计算投影到具有相同维度的向量。这些投影是在 CLIP 训练过程中学习的，也是我们之前下载的 `clip` 模型包装器的一部分。

在这种情况下，学习的投影只是将输入映射到 768 维向量的线性层。文本编码器有一个投影，视觉模型部分有另一个投影：

```
print(clip.text_projection)
print(clip.visual_projection)
Linear(in_features=768, out_features=768, bias=False)
Linear(in_features=1024, out_features=768, bias=False)
with torch.inference_mode(): text_embeddings = clip.text_projection(
    text_embeddings) image_embeddings = clip.visual_projection(image_embeddings)

text_embeddings.shape, image_embeddings.shape
```

(torch.Size([2, 768]), torch.Size([1, 768]))

我们几乎准备好计算余弦相似度了。我们只需要记住使用单位范数的归一化向量，我们通过缩放嵌入并除以它们各自的范数来实现这一点。然后我们可以通过矩阵乘法一次性计算两个点积：

```
text_embeddings = text_embeddings / text_embeddings.norm( p=2, dim=-1,
    keepdim=True
)
image_embeddings = image_embeddings / image_embeddings.norm( p=2, dim=-1,
    keepdim=True
similarities = torch.matmul(text_embeddings, image_embeddings.T)
similarities
tensor([[0.2171], [0.1888]], device='cuda:0')
```

在训练期间，这些余弦相似度被解释为要输入到交叉熵损失分类器的 logits，该分类器预测每个图像-文本对的标签。由于 CLIP 训练使用了 32 大小的批次，标签是批次中 32,768 个位置中的每一个。如果你再次参考本节开头的 CLIP 图像，一旦模型被训练，交叉熵过程将为 $I_1$ 选择“类” $T_1$，为 $I_2$ 选择类 $T_2$，依此类推。

不过，还有一个最后的细节。由于向量是归一化的，logits 只能位于 [-1, 1] 的范围内，由于浮点

数格式的限制，这个范围可能不足以有效地捕捉 32K 个类别的概率分布。作者使用了一个可学习的温度参数来缩放 logits，使其具有更宽的范围。然而，为了数值稳定性，他们也将这个缩放值限制在最大 100。在所有的训练运行中，他们发现缩放值总是达到了最大值 100。因此，CLIP 推理在将 logits 解释为概率之前，使用了 100 的缩放因子。

让我们将这个修正应用到之前的代码片段中，然后我们可以将缩放后的相似性 logits 转换为概率。这些概率表示模型对图像与我们使用的两个文本描述之一对应的置信度：

```
similarities = 100 * torch.matmul(text_embeddings, image_embeddings.T)
similarities.softmax(dim=0).cpu()
```

```
tensor([[0.9441],[0.0559]])
```

模型以 $94.4\%$ 的置信度将提示“a photo of a lion”与图像匹配。

## 使用CLIP进行零样本图像分类

我们在前一节中遵循的过程是实现CLIP零样本分类任务所需的详细步骤。幸运的是，像transformers或OpenCLIP这样的软件库提供了更高层次的抽象，使这一过程变得更加简单。

为什么这被称为零样本分类?

分类是机器学习中的典型问题之一：给定一个数据点和一组预定义的类别，估计该点对应于其中一个类别的概率。根据这个定义，类别集必须事先固定，因此我们训练的模型只会知道这些类别，而不会知道其他内容。例如，ImageNet数据集包含来自20,000个类别的图像。多年来，ImageNet大规模视觉识别挑战赛（ILSVRC）一直是计算机视觉系统的测试基准，其目标是对属于ImageNet中1,000个类别的对象进行分类。2012年，一个名为AlexNet的深度CNN轻松赢得了当年的挑战赛，这引发了一场革命，随着设计出越来越好的深度学习模型，准确率逐年提高。

零样本分类指的是模型在没有针对我们要求的类别进行显式训练的情况下，能够正确分类数据的能力。在第2章中，我们看到了使用语言模型进行零样本情感分类的示例，而前一节则是这一能力的另一个主要展示。CLIP被训练用于匹配图像-标题对，但如果我们构建能够合理描述我们想要分类的图像的标题，我们可以利用这种行为进行分类。例如，如果我们想要分类猫和狗，我们的提示可以是“一张猫的照片”和“一张狗的照片”。CLIP将为我们提供的图像匹配最佳提示，从而给出我们想要的分类结果。

让我们使用更高级的transformers API来复制前一节中的示例。我们以与之前相同的方式加载CLIP模型、处理器和测试图像：

```
clip = CLIPModel.from_pretrained("openai/clip-vit-large-patch14").to(device)
processor = CLIPProcessor.from_pretrained("openai/clip-vit-large-patch14")
image = Image.open(requests.get(SampleURL.LionExample, stream=True).raw)
```

我们可以利用处理器同时计算图像和文本提示的输入。我们还可以方便地调用CLIP模型，使用完整的输入集来检索图像与每个提示之间的logits（或缩放的余弦相似度）。让我们使用更多的提示来增加趣味性：

```
prompts = [
    "a photo of a lion",
    "a photo of a zebra",
    "a photo of a cat",
    "a photo of an adorable lion cub",
    "a puppy",
    "a lion behind a branch",
]
inputs = processor(
    text=prompts,
    images=image,
    return_tensors="pt",
    padding=True
)
inputs = {k: v.to(device) for k, v in inputs.items()}
outputs = clip(**inputs)
logits_per_image = outputs.logits_per_image
probabilities = logits_per_image.softmax(dim=1)
probabilities = probabilities[0].cpu().detach().tolist()
for prob, prompt in sorted(zip(probabilities, prompts), reverse=True):
    print(f"{100*prob: =2.0f}%: {prompt}")
```

输出结果：

```
89%: a photo of an adorable lion cub
9%: a lion behind a branch
2%: a photo of a lion
0%: a photo of a zebra
0%: a photo of a cat
0%: a puppy
```

同样，我们可以在同一个输入批次中提供多个图像和提示，并同时获得所有分类概率。请随意探索并适应您的用例。

## 零样本图像分类管道

现在您已经了解了CLIP的工作原理以及如何使用它进行零样本图像分类，我们可以使用更高级的API来简化操作。我们将使用管道抽象，这已经在第1章中介绍过。在那种情况下，我们展示了如何将其用于文本分类任务，但我们也可以将其应用于许多其他任务，包括零样本图像分类。

要实例化一个管道，我们只需提供任务名称（在这种情况下是zero-shot-image-classification）和我们想要使用的模型：

```
from transformers import pipeline
classifier = pipeline(
    "zero-shot-image-classification",
    model="openai/clip-vit-large-patch14",
    device=device,
)
```

管道为我们处理所有细节：分词、图像预处理和logits后处理。我们只需要使用我们想要分类的图像和一组候选标签调用管道实例。管道返回一个按分数排序的字典，包含与我们提供的标签相关的所有分数：

```
scores = classifier(
    image,
    candidate_labels=prompts,
    hypothesis_template="{}",
)
```

`hypothesis_template`是一个Python格式字符串，应用于每个候选标签以构建分类的文本提示。

如果我们省略`hypothesis_template`参数，管道将自动使用"This is a photo of a {}"，这适用于格式化由其名称指示的类别标签，例如“cat”或“lion”。由于我们已经构建了与CLIP配合良好的提示，我们使用"{}"来保持我们的标签不变：

```
[
    {'label': 'a photo of an adorable lion cub', 'score': 0.886413037776947},
    {'label': 'a lion behind a branch', 'score': 0.09321863204240799},
    {'label': 'a photo of a lion', 'score': 0.018809959292411804},
    {'label': 'a photo of a zebra', 'score': 0.0011134858941659331},
    {'label': 'a photo of a cat', 'score': 0.0004198708338662982},
    {'label': 'a puppy', 'score': 2.4912407752708532e-05}
]
```

## CLIP的用例

CLIP最初设计的用例是零样本图像分类。结果令人印象深刻：CLIP实现了与专门为ImageNet分类训练的模型相似的性能，而从未在训练中使用ImageNet标签。因此，在许多其他数据集上，性能同样强劲，无需进行微调，正如您可以在原始博客文章中读到的那样。

前面的部分已经表明，解决零样本图像分类的基础是能够计算任意图像与文本提示之间的相似性，这是可能的，因为图像和文本嵌入都捕捉了数据的本质语义。CLIP的工作方式使得社区能够将其用于许多任务，而不仅仅是零样本分类。

能够计算文本和图像之间的相似性使得诸如语义搜索等应用成为可能，这使得可以根据自然语言描述的内容搜索照片，或找到与我们提供的示例图像相似的图像。这些技术的应用存在于多个领域，包括消费硬件（如手机）、医疗系统、时尚等。在第2章中，我们提出了一个类似的挑战，但它仅限于文本数据：通过计算语言模型嵌入输出之间的相似性来构建FAQ系统。在本章末尾，我们提出了一个使用CLIP进行语义搜索的挑战。

CLIP还可以用于获取下游任务的丰富嵌入。例如，一些文本到图像模型使用CLIP来获取用户提供的提示的语义丰富的表示。

CLIP也被采用为生成用例的重要工具。由Ryan Murdock、Katherine Crowson等人开发的CLIP Guidance方法使用CLIP作为损失来引导模型梯度朝向所需的表示（通过提示表达）。这种方法在生成艺术社区中引发了创意爆炸。后来，CLIP条件化成为诸如Stable Diffusion等模型中的关键部分，我们将在第5章中探讨。

CLIP的评分能力也被用于过滤和评分从互联网来源爬取的大规模图像-标题对数据集，如

LAION。依赖CLIP允许数据集创建者选择图像和标题之间相似性得分高的对，并丢弃其他对。这使得构建和精炼数十亿项数据集成为可能，这些数据集可以用于构建更好的模型，从而更精确地精炼数据集。真是元数据！

## CLIP的替代方案

由于CLIP在工业和研究中被广泛使用，围绕该模型提出的想法以及如何使其更好、更快或适应其他任务，已经有很多活动。为了使CLIP完全开源和可复现，已经做出了重大努力。OpenCLIP存储库提供了实现CLIP和多种架构变体的开源代码。使用OpenCLIP代码库和我们在前一节中提到的庞大的LAION数据集，LAION团队训练了非常强大的各种规模的CLIP模型，并将所有检查点公开供大家使用。

最近的研究表明，更好的文本理解可以带来更好的文本-图像模型。BLIP、CoCa 和 CapPa 证明了图像描述任务（生成图像内容的详细描述）可以产生能够生成优秀图像表示并解决广泛视觉-语言任务的模型。在并行研究中，使用不同的损失函数（SigLIP 中使用的 sigmoid 损失，而不是 softmax 归一化）可以使训练更容易，并缓解 CLIP 需要大批量才能有效训练的问题。

另一个有前景的方向是构建更小、更快的模型，可以在个人计算机和移动设备上运行。苹果的 MobileCLIP 就是一个例子，它通过更小（且更快）的模型实现了 OpenAI 的 CLIP 模型的性能。苹果的另一项努力，数据过滤网络（Data Filtering Networks），旨在提高文本-图像数据集的质量，并使用这些数据集训练了多个 CLIP 变体。

正如你将在本书的其余部分看到的，CLIP 是图像生成系统的关键组成部分。这种对更强大、更高效、更快的类 CLIP 模型的健康研究让我们对未来非常乐观。

## 项目时间：语义图像搜索

一个有趣的项目是为你的照片构建一个语义搜索引擎。完成后，你应该能够通过简单描述照片内容来搜索你的照片库（例如：“炎炎夏日里狗跳入水中”或“打着伞的女人走在繁忙的街道上”），而不必试图记住这些照片存储在收藏中的哪个位置。你也可以使用任何你喜欢的其他图像数据集，但使用对你有意义的内容会让你更有成就感，并允许你评估系统的效果。这也可能会激发你思考改进的想法，或与家人分享这些想法。

以下是一些建议的步骤来应对这个项目：

1. 选择一个文本-图像模型，例如CLIP，它可以为图像和文本描述生成嵌入向量。你可以探索其他替代方案，但CLIP应该是一个不错的起点。选择一个具有多种尺寸变体的模型系列，这样你可以轻松地替换它们。这样，你可以在工作时使用较小的模型以更快地迭代，并查看

使用较大模型时性能提升的程度。

2. 找到一定数量的照片并将它们全部复制到计算机上的一个文件夹中。几百或一千张照片应该足够了。
3. 编写一个循环，使用你选择的模型从照片中创建嵌入向量：
   a. 从磁盘读取照片，裁剪和/或调整大小，使它们具有相同的尺寸，并创建一个批次。你可以使用PyTorch的DataLoader，就像我们在本章的训练循环中所做的那样。对于预处理，你可以手动使用torchvision.transforms，或者如果模型有内置的预处理器，也可以利用它。选择一个适合你硬件的批次大小。
   b. 通过CLIP模型的图像部分运行每批图像。使用推理模式（不需要计算梯度，因为你不会训练任何东西）。
   c. 从输出中获取嵌入向量并将其保存到磁盘。你将获得每个图像文件的多维向量。你可以将它们转换为numpy数组并将它们全部写入同一个文件。别忘了存储原始照片的名称或路径；稍后你需要它们来检索照片。
   此时，你有一个numpy格式的向量数组。你现在可以使用模型的文本部分来运行查询：
   a. 编写一个函数，接收输入提示并使用模型的文本塔生成嵌入向量。
   b. 计算该向量与图像嵌入表中所有向量的余弦相似度。如果你有足够的RAM，你可以简单地使用PyTorch的matmul操作来完成。
   c. 对输出进行排序并选择前几个。
   d. 找到与最高分数相关的图像并可视化它们。它们是否与你使用的提示匹配？

额外任务：

- 你能尝试找到与另一张图像相似的图像吗？（即基于输入图像的语义搜索，而不是文本描述。）
- 如果你有很多照片，计算分数可能会遇到问题。你如何解决这个问题？是否有任何框架或服务可以帮助解决这个问题？需要多少张照片才能达到你计算机的极限？
- 预训练模型对你照片中出现的主题一无所知。你可以做些什么来能够通过个人姓名或地点进行搜索？
- 如果你有冒险精神，你可以使用MobileCLIP在手机上运行搜索引擎。这本身就是一个巨大的挑战；不要低估它的难度！

# 总结

本章展示了如何从输入数据中学习压缩表示是捕捉数据集本质特征的一种方式，以及这些表示如何有效地用于许多额外的下游任务。我们通过查看一个经典系统——自动编码器（AutoEncoder）——开始了这次探索，其目标是将输入样本编码到降维的潜在空间中，然后从

潜在表示中恢复原始数据点。

通过将自动编码器拆分为两个组件——编码器和解码器——我们可以想象出超越重建的新应用。例如，编码器可以用作特征提取器。因为它学习了输入数据集的基本特征，我们可以使用它来训练其他系统，例如分类器，其输入数据是潜在表示。我们还探索了使用解码器进行生成的想法。如果潜在空间是原始数据集的表示，我们是否可以移动到任意点并查看我们得到的输出？然而，自动编码器由于训练方式的原因，在这项任务中存在一些局限性。

变分自动编码器（VAE）是一种特殊类型的自动编码器，它试图在潜在空间中实现“行为更好”的表示。通过尝试使潜在特征匹配概率分布，我们可以从所需分布中采样以获得新的随机潜在特征。如果我们将这些特征输入解码器，我们可以生成看起来像来自原始数据集的数据点。这对于生成应用来说是一个关键结果。进一步探索，潜在空间是数据的紧凑表示这一事实是诸如Stable Diffusion等生成系统背后的核心思想。正如我们将在第5章中更详细地看到的那样，我们可以在表示图像的潜在空间中进行计算，而不是在原始图像数据上进行计算。这使得能够在消费级硬件上快速高效地运行高质量图像生成系统的训练成为可能。

本章的最后一部分聚焦于CLIP，这是由OpenAI开发并发布的一个极具影响力的模型，它将图像和文本数据编码到同一个潜在空间中。使用CLIP可以实现一些以前难以解决的新技巧。例如，给定一张图像和几个句子，我们可以测量哪个句子与图像更匹配。相反，给定一个标题和几个图像候选，我们可以选择与标题最匹配的图像。CLIP是作为OpenAI的DALL·E图像生成项目（该项目本身并未发布）的一部分发布的，并引发了生成研究的一场革命。类似CLIP的模型是Stable Diffusion和其他文本到图像模型的关键组件，但它们也用于许多其他应用：自然语言图像检索、语义搜索、基于内容或风格提取相似的图像，以及更多。

所有这些都源于最初的认知，即学习如何压缩数据等同于学习数据本身。这些概念的许多变体使用不同类型的数据提取和表示技术：CNN、transformer，或结合两者的最佳系统，如VQGAN。我们只是对这些基础模块背后的动机和思想进行了简要介绍，希望它们能对探索这个丰富而迷人的领域有所帮助。

# 练习

大多数练习与我们在本章中提出的相同，为了方便起见，这里进行了整理。根据你的学习风格，你可能喜欢在阅读本章时进行练习，或者在第一次阅读后尝试所有练习：

1. 如果自动编码器模型使用16个潜在维度进行训练，生成是如何工作的？你能比较使用16个潜在维度的模型和仅使用2个潜在维度的模型之间的生成结果吗？
2. 使用我们使用的相同参数再次训练模型（只需运行本章中显示的代码），但使用不同的随机

数初始化，并可视化潜在空间。很可能形状和结构会有所不同。这是你预期的吗？为什么？
3. 编码器提取的图像特征有多好？通过在编码器之上训练一个数字分类器来探索它。
4. 当我们训练VAE时，我们添加了重建损失和KL散度损失。然而，两者的尺度不同。如果我们更重视其中一个而不是另一个，会发生什么？你能运行一些实验并解释结果吗？
5. 我们训练的VAE仅使用两个维度来表示分布的均值和logvar。你能使用16个维度进行类似的探索吗？

你可以在本书的GitHub仓库中找到这些练习及其后续挑战的解决方案。

# 挑战

1. 使用 BLIP-2 进行搜索。语义图像搜索的实践项目相当具有挑战性，但这里还有一个想法。你能像我们在本章中使用 CLIP 那样，将 BLIP-2 模型用于相似性任务吗？你会如何实现，它与 CLIP 相比如何？你还能用 BLIP-2 解决哪些其他任务？
2. 训练你自己的自编码器（AutoEncoder）或变分自编码器（VAE）。人类经过训练可以观察面部并轻松识别出不真实特征。你能尝试为包含面部的数据集训练一个自编码器和一个变分自编码器，并看看结果如何吗？你可以从 VAE 论文中使用的 Frey Face 数据集开始——这是一个同质的单色面部数据集，来自同一个人，展示了不同的面部表情。如果你想更具挑战性，可以尝试 CelebFaces Attributes 数据集，该数据集也托管在 Hugging Face Hub 上。另一个有趣的例子是尝试 Oxford pets 数据集，该数据集同样可在 Hub 上找到。

参考文献

Chollet, François. “变分自编码器。” Keras 实现的 VAE。2024 年 4 月 24 日。https://oreil.ly/TShML。

Clément, Chadebec。pythae 库 GitHub 仓库。2022 年。https://oreil.ly/XUGjo。

Ermon, Stefano 等。 “变分自编码器。” 在 CS 228 - 概率图模型课程笔记中。https://oreil.ly/pc0EP。

Esser, Patrick, Robin Rombach 和 Bjorn Ommer。 “驯服 Transformer 进行高分辨率图像合成。” arXiv，2021 年 6 月 23 日，https://arxiv.org/abs/2012.09841。

Fang, Alex 等。 “数据过滤网络。” arXiv，2023 年 11 月 6 日。https://arxiv.org/abs/2309.17425。

Floret, François。深度学习课程材料（2024 年版）。https://oreil.ly/Csb_F。修订版 VAE 讲义（PDF）在 https://oreil.ly/456Vq。

Foster, David。第 3 章，“变分自编码器。” 在《生成式深度学习》第二版中。O’Reilly，2023 年。https://oreil.ly/TH2aO。

Howard, Jeremy 和 Sylvain Gugger。 《使用 fastai 和 PyTorch 进行深度学习的程序员指

南》。O’Reilly，2020 年。https://oreil.ly/n1M6C。
Kingma, Diederik P. 和 Max Welling。 “自编码变分贝叶斯。” arXiv，2022 年 12 月 10 日。https://arxiv.org/abs/1312.6114。
LAION（多位作者）。 “在 LAION-2B 上训练的大规模 OpenCLIP。” LAION 博客，2022 年 9 月 15 日。https://oreil.ly/FBlmH。
Li, Junnan 等。 “BLIP：引导语言-图像预训练以实现统一的视觉-语言理解和生成。” arXiv，2022 年 2 月 15 日。https://arxiv.org/abs/2201.12086。
Li, Junnan 等。 “BLIP-2：使用冻结图像编码器和大型语言模型引导语言-图像预训练。” arXiv，2023 年 6 月 15 日。https://arxiv.org/abs/2301.12597。
Loong, Jackie。MNIST 的变分自编码器 GitHub 仓库。https://oreil.ly/-IRhy。
Maucher, Johannes。 “卷积和反卷积的动画。” 在《机器学习和概述》课程笔记中，2022 年。https://oreil.ly/KL8VU。
ML Foundations（多位作者）。OpenCLIP GitHub 仓库。https://oreil.ly/Hkjbj。
PyTorch 团队。PyTorch GitHub 仓库中的 PyTorch VAE 示例，2017 年 1 月。https://oreil.ly/ZchyF。
Radford, Alec 等。 “从自然语言监督中学习可迁移的视觉模型。” arXiv，2021 年 2 月 26 日。https://arxiv.org/abs/2103.00020。
斯坦福大学。 “卷积层。” 在 CS231n 卷积神经网络视觉识别课程笔记中，2022 年 11 月。https://oreil.ly/Ohh6z。
Tschannen, Michael 等。 “图像描述器也是可扩展的视觉学习者。” arXiv，2023 年 12 月 21 日。https://arxiv.org/abs/2306.07915。
Vasu, Pavan Kumar Anasosalu 等。 “MobileCLIP：通过多模态强化训练实现快速图像-文本模型。” arXiv，2024 年 4 月 1 日。https://arxiv.org/abs/2311.17049。代码仓库在 https://oreil.ly/DmtWF。
Whitaker, Jonathan。 “深入探讨 OpenAI 的 OpenCLIP。” W&B Fully Connected，2022 年 11 月 7 日。https://oreil.ly/9SSo1。
Yanagisawa, Chiaki。 “Conv2d 和 ConvTransposed2d。” PowerPoint 演示文稿，2021 年 2 月 19 日。https://oreil.ly/JEEl0。
Yu, Jiahui 等。 “Coca：对比描述器是图像-文本基础模型。” arXiv，2022 年 6 月 14 日。https://arxiv.org/abs/2205.01917。
Zhai, Xiaohua 等。 “用于语言图像预训练的 Sigmoid 损失。” arXiv，2023 年 9 月 27 日。https://arxiv.org/abs/2303.15343。

# 第 4 章 扩散模型

图像生成领域在 Ian Goodfellow 于 2014 年提出生成对抗网络（GANs）后变得广受欢迎。GANs 的核心思想催生了一个能够快速生成高质量图像的模型家族。然而，尽管 GANs 取得了成功，它们也带来了挑战，需要大量参数并且难以有效泛化。这些限制引发了并行研究，推动了扩散模型的探索——这类模型重新定义了高质量、灵活图像生成的格局。

2020 年底，一种鲜为人知的模型类别——扩散模型开始在机器学习界引起轰动。研究人员发现如何使用这些扩散模型生成比 GANs 更高质量的图像。随后，一系列论文提出了改进和修改，进一步提升了生成质量。到 2021 年底，像 GLIDE 这样的模型在文本到图像任务中展示了令人难以置信的结果。仅仅几个月后，这些模型通过 DALL·E 2 和 Stable Diffusion 等工具进入了主流。这些模型使得任何人都可以通过输入文本描述轻松生成图像。

在本章中，我们将深入探讨这些模型的工作原理。我们将概述使它们如此强大的关键见解，使用现有模型生成图像以感受其工作方式，然后训练我们自己的模型以进一步加深理解。该领域仍在快速发展，但本章涵盖的主题将为你提供一个坚实的基础，这将在第 5、7 和 8 章中进一步扩展。

扩散模型的高层思想是，它们接收带有噪声的模糊图像，并学习去噪，输出清晰的图像。在训练扩散模型时，数据集中包含不同噪声量的图像（即使输入是纯噪声）。在推理时，我们可以从纯噪声开始，模型将生成与训练分布匹配的图像。模型通过多次迭代完成这一过程，自我修正，从而生成令人印象深刻的高质量图像。

## 关键洞察：迭代优化

那么，是什么让扩散模型如此强大？之前的技术，如VAE或GAN，通过模型的一次前向传递生成最终输出。这意味着模型必须在第一次尝试时就做到完美。如果它犯了错误，就无法回头修正。而扩散模型则通过多次迭代生成输出。这种迭代优化允许模型在之前的步骤中纠正错误，并逐步改进输出。为了说明这一点，图4-1展示了一个扩散模型的实际运行示例。

图4-1. 渐进去噪过程

我们可以使用Hugging Face的diffusers库加载一个预训练的扩散模型。该库提供了一个高级管道，可以直接用于生成图像。我们将加载ddpm-celebahq-256模型，这是最早用于图像生成的共享扩散模型之一。该模型使用CelebA-HQ数据集进行训练，这是一个当时流行的高质量名人图像数据集，因此它将生成看起来像来自该数据集的图像。我们将使用该模型从噪声中生成一张图像：

```
import torch
from diffusers import DDPMPipeline
from genaibook.core import get_device
// 设置使用的设备为GPU或CPU
device = get_device()
// 加载管道
image_pipe = DDPMPipeline.from_pretrained("google/ddpm-celebahq-256")
image_pipe.to(device)

// 生成一张图像
image_pipe().images[0]
```

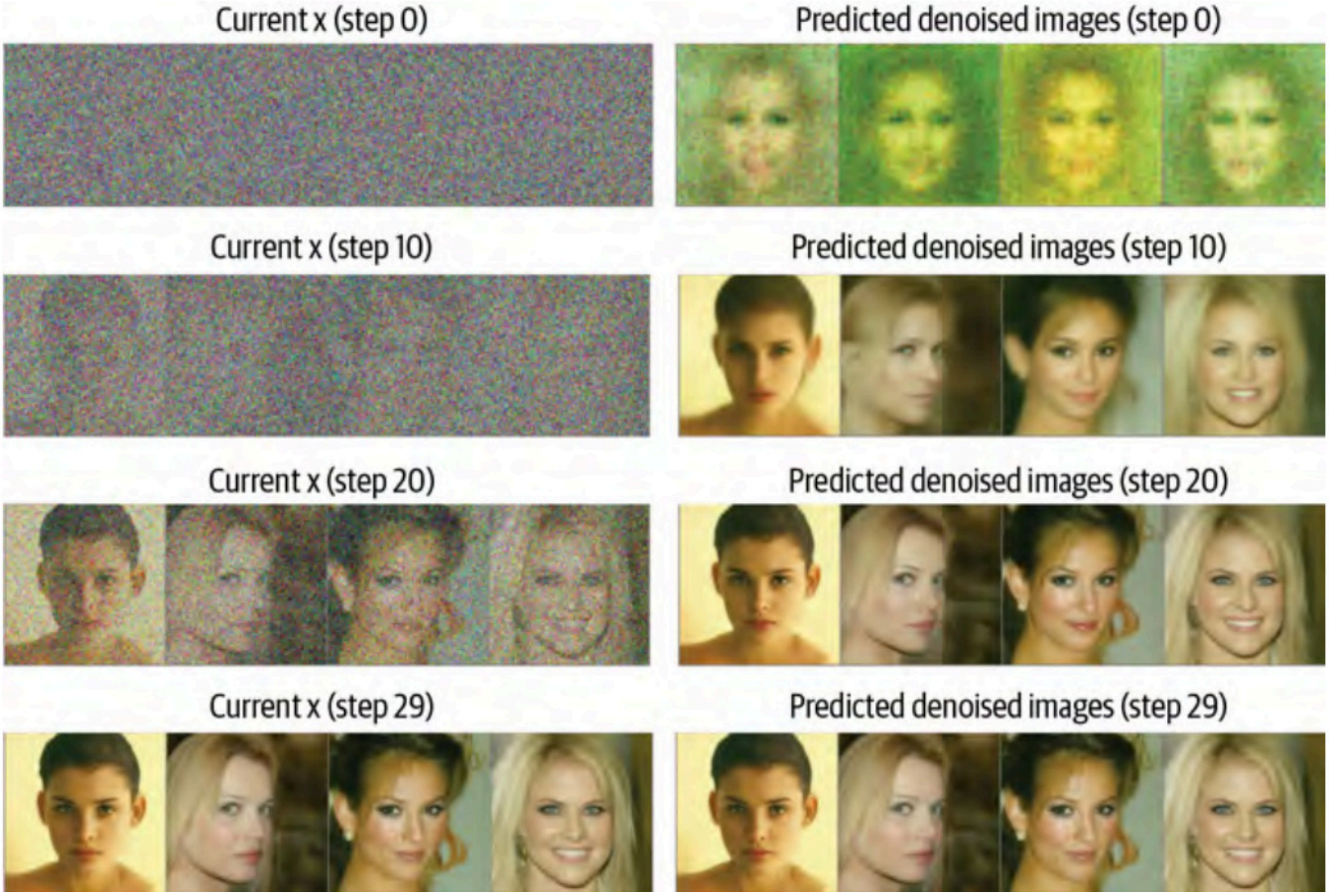
管道并没有展示其内部的工作原理，因此让我们深入其内部。如果你运行这段代码，你会注意到生成过程需要1,000步。这个扩散管道必须经过1,000次优化步骤（和前向传递）才能得到最终图像。这是与GAN相比，普通扩散模型的主要缺点之一——它们需要许多步骤来生成高质量图

像，使得模型在推理时速度较慢。

我们可以逐步重新创建这个采样过程，以更好地理解其内部发生了什么。在扩散过程的开始，我们用一批四张随机图像初始化样本 x（换句话说，我们采样一些随机噪声）。我们将运行30步，逐步对输入图像进行去噪，最终得到一个来自真实分布的样本。

让我们生成一些图像！在下面图像的左侧，你可以看到给定步骤的输入（从随机噪声开始）。你可以在右侧看到模型对最终图像的预测。第一行的结果并不特别好。在给定的扩散步骤中，我们不会直接跳到最终的预测图像，而是只对输入x（显示在左侧）进行小幅度的修改，朝着预测的方向。然后我们将这个新的、稍微更好的x再次输入模型进行下一步，希望得到一个稍微改进的预测，可以用来进一步更新x，依此类推。经过足够的步骤，模型可以生成一些令人印象深刻的逼真图像：

```
from genaibook.core import plot_noise_and_denoise
// 随机起点是一批4张图像
// 每张图像是3通道（RGB）256x256像素图像
image = torch.randn(4, 3, 256, 256).to(device)

// 设置特定的扩散步骤数
image_pipe.scheduler.set_timesteps(num_inference_steps=30)

// 遍历采样时间步
for i, t in enumerate(image_pipe.scheduler.timesteps):
    // 获取给定当前样本x和时间步t的预测
    // 由于我们正在运行推理，不需要计算梯度，
    // 因此我们可以使用torch.inference_mode()。
    with torch.inference_mode():
        // 我们需要传入时间步t，以便模型知道当前的时间步。我们将在接下来的章节中了解更多。
        noise_pred = image_pipe.unet(image, t)["sample"]

    // 使用调度器计算更新后的样子
    scheduler_output = image_pipe.scheduler.step(noise_pred, t, image)

    // 更新x
    image = scheduler_output.prev_sample
    // 偶尔显示x和预测的去噪图像
    if i % 10 == 0 or i == len(image_pipe.scheduler.timesteps) - 1:
        plot_noise_and_denoise(scheduler_output, i)
```




如果这段代码看起来令人生畏，不要担心——我们将在本章中解释这一切是如何工作的。现在请专注于这个想法。

这种学习如何迭代优化噪声输入的核心思想可以应用于广泛的任务。本章将专注于无条件图像生成，生成与训练数据分布相似的图像。例如，我们可以使用蝴蝶数据集训练一个无条件图像生成模型，使其也能生成新的高质量图像。这个模型无法生成与训练数据集分布不同的图像，所以不要指望它能生成恐龙。

在第5章中，我们将深入探讨基于文本的扩散模型，但我们还可以做许多其他事情。扩散模型已经应用于音频、视频、文本、3D对象、蛋白质结构等领域。虽然大多数实现使用我们在这里介绍的某种去噪方法，但新兴的方法应用了不同类型的“破坏”（总是与迭代优化相结合），可能会推动该领域超越当前对去噪扩散的关注。

## 训练扩散模型

在本节中，我们将从头开始训练一个扩散模型，以更好地理解它们的工作原理。我们将从使用diffusers库中的组件开始。随着本章的进展，我们将逐步揭开每个组件的工作原理。与其他生成模型相比，训练扩散模型相对简单。要训练一个模型，我们重复以下步骤：

1. 从训练数据中加载一些图像。
2. 添加不同数量的噪声。记住，我们希望模型能够很好地估计如何“修复”（去噪）极度噪声的图像和接近完美的图像，因此我们需要一个具有不同噪声量的数据集。
3. 将噪声版本的输入输入模型
4. 评估模型在去噪这些输入时的表现。
5. 使用这些信息更新模型权重。

要使用训练好的模型生成新图像，我们从完全随机的输入开始，并反复将其输入模型，根据模型预测在每次迭代中对输入进行小幅更新。正如我们将看到的，有几种采样方法可以简化这一过程，以尽可能少的步骤生成良好的图像。

## 数据

在这个示例中，我们将使用来自 Hugging Face Hub 的图像数据集——具体来说，是一个包含 1,000 张蝴蝶图片的集合。稍后，在第 145 页的“项目时间：训练你的扩散模型”中，你将看到如何使用自己的数据。让我们加载蝴蝶数据集：

```
from datasets import load_dataset

dataset = load_dataset("huggan/smithsonian_butterflies_subset", split="train")
```

在使用数据训练模型之前，我们必须对其进行预处理。图像通常表示为像素网格。与前一章中使用的灰度图像不同，这些图像是彩色的。每个像素由三个颜色通道（红色、绿色和蓝色）的 0 到 255 之间的颜色值表示。为了处理这些图像并使它们准备好进行训练，我们执行以下操作：

1. 将它们调整为固定大小。这是必要的，因为模型期望所有图像具有相同的尺寸。
2. （可选）通过随机水平翻转图像来增加一些数据增强，使模型更加鲁棒，并允许我们使用更多数据进行训练。数据增强（图 4-2）是计算机视觉任务中的常见做法，因为它有助于模型更好地泛化到未见过的数据。翻转只是图像数据增强的一种技术。其他技术包括平移、缩放和旋转。

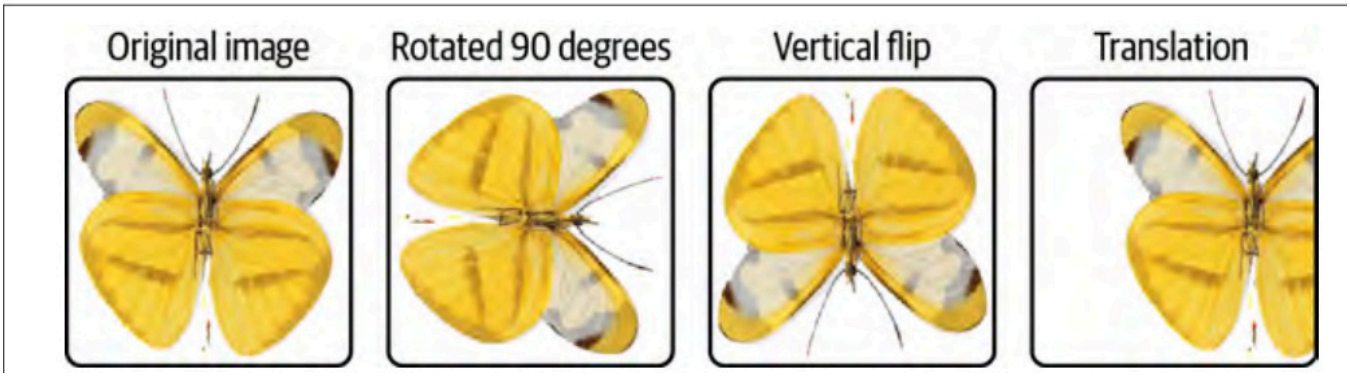



图 4-2. 数据增强从训练数据集中创建更多数据，提高泛化能力

3. 将它们转换为 PyTorch 张量（将颜色值表示为 0 到 1 之间的浮点数）。模型输入必须始终格式化为多维矩阵或张量。
4. 将它们归一化，使其均值为 0，值在 -1 到 1 之间。这是训练深度学习模型的常见做法，因为它有助于模型更快、更有效地学习。

我们可以使用 `torchvision.transforms` 来定义这些转换：

```
from torchvision import transforms
image_size = 64
# 定义转换
preprocess = transforms.Compose(
    [
    transforms.Resize((image_size, image_size)), # 调整大小
    transforms.RandomHorizontalFlip(),  # 随机翻转（数据增强）
    transforms.ToTensor(),  # 转换为张量 (0, 1)
    transforms.Normalize([0.5], [0.5]),  # 映射到 (-1,1)
    ]
)
```

`datasets` 库提供了一个方便的方法 `set_transform()`，它允许我们指定在数据使用时动态应用的转换。最后，我们可以将数据集包装在 `DataLoader` 中，这是一个加载工具，可以轻松地迭代数据批次，简化我们的训练代码：

```
def transform(examples):
    examples = [preprocess(image) for image in examples["image"]]
    return {"images": examples}

dataset.set_transform(transform)
batch_size = 16
train_dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size,
    shuffle=True)
```

我们可以通过加载一个批次并检查图像来验证这是否有效。以下是训练集中的一个示例批次：

```
from genaibook.core import show_images
batch = next(iter(train_dataloader))

# 当我们归一化时，我们将 (0, 1) 映射到 (-1, 1)
# 现在我们将映射回 (0, 1) 以进行显示
show_images(batch["images"][:8] * 0.5 + 0.5)
```

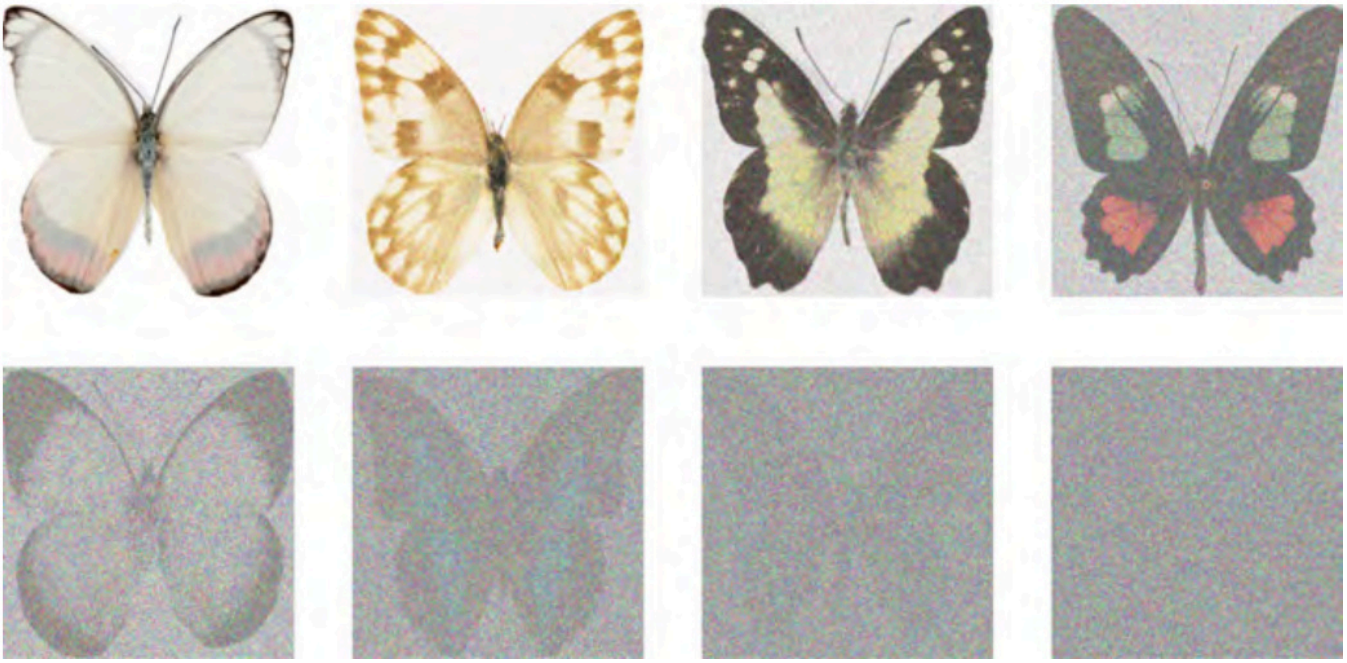
## 添加噪声

我们如何逐渐破坏数据？最常见的方法是在图像中添加噪声。我们将在训练数据中添加不同数量的噪声，因为目标是训练一个鲁棒的模型，无论输入中有多少噪声，都能去噪。我们添加的噪声量由噪声调度控制，这是扩散模型的一个关键方面。不同的论文和方法以不同的方式处理这个问

题。

现在，让我们探索基于 DDPM 论文的一种常见方法。在 `diffusers` 中，添加噪声由一个称为 `Scheduler` 的类处理，它接收一批图像和一个时间步列表，并确定如何创建这些图像的噪声版本。我们将在本章后面探讨其背后的数学原理，但现在让我们看看它在实践中是如何工作的。以下代码片段对每个输入图像应用逐渐增大的噪声量：

```
from diffusers import DDPMScheduler
# 我们将在接下来的部分中了解 beta_start 和 beta_end
scheduler = DDPMScheduler( num_train_timesteps=1000, beta_start=0.001,
    beta_end=0.02
)
# 创建一个包含从 0 到 999 的 8 个均匀间隔值的张量
timesteps = torch.linspace(0, 999, 8).long()
# 我们从数据集中加载 8 张图像并 # 对它们添加逐渐增大的噪声
x = batch["images"][:8]

noise = torch.rand_like(x)
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0, 1))
```



在训练过程中，我们将随机选择时间步。调度器接收一些参数（`beta_start` 和 `beta_end`），它使用这些参数来确定给定时间步应该存在多少噪声。我们将在第 126 页的“深入：噪声调度”中

更详细地介绍调度器。

## UNet

UNet 是一种为图像分割等任务发明的 CNN，其中期望的输出与输入具有相同的形状。例如，UNet 用于医学成像中以分割不同的解剖结构。

如图 4-3 所示，UNet 由一系列减少输入空间大小的下采样层组成，随后是一系列再次增加输入空间范围的上采样层。下采样层通常后跟跳跃连接，将下采样层的输出连接到上采样层的输入。这使得上采样层能够结合来自早期层的更精细细节，在去噪过程中保留重要的高分辨率信息。

`diffusers` 库中使用的 UNet 架构比 2015 年提出的原始 UNet 更先进，增加了注意力和残差块等功能。我们稍后会详细讨论，但这里的关键思想是它可以接收输入并生成相同形状的预测。在扩散模型中，输入可以是噪声图像，输出可以是预测的噪声。有了这些信息，我们现在可以对输入图像进行去噪。

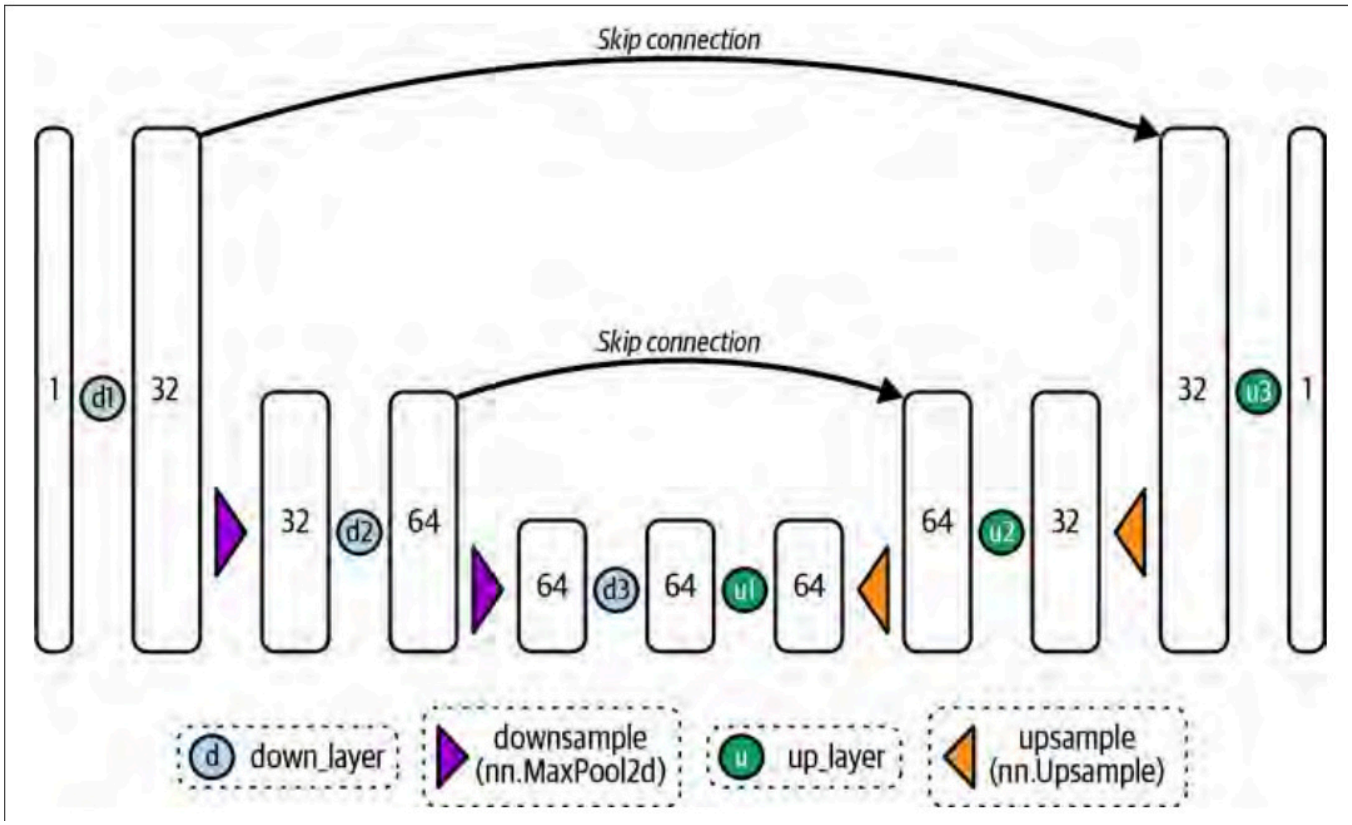



图 4-3. 简化 UNet 的架构

以下是我们如何创建 UNet 并将一批噪声图像输入其中的示例：

```
from diffusers import UNet2DModel

model = UNet2DModel(
    in_channels=3,  # 3个通道用于 RGB 图像
    sample_size=64,  # 指定我们的输入大小
    # 每个块的通道数影响模型大小
    block_out_channels=(64, 128, 256, 512),
    down_block_types=(
        "DownBlock2D",
        "DownBlock2D",
        "AttnDownBlock2D",
        "AttnDownBlock2D",
    ),
    up_block_types=("AttnUpBlock2D", "AttnUpBlock2D", "UpBlock2D", "UpBlock2D"),
).to(device)

# 传递一批数据以确保其正常工作
with torch.inference_mode():
    out = model(noised_x.to(device), timestep=timesteps.to(device)).sample

print(noised_x.shape)
print(out.shape)

torch.Size([8, 3, 64, 64])
torch.Size([8, 3, 64, 64])
```

注意，输出与输入的形状相同，这正是我们想要的。

## 训练

现在我们已经准备好了数据和模型，让我们开始训练它。对于每个训练步骤，我们执行以下操作：

1. 加载一批图像。
2. 向图像添加噪声。添加的噪声量取决于指定的时间步数：时间步数越多，噪声越多。如前所述，我们希望我们的模型能够对带有少量噪声和大量噪声的图像进行去噪。为了实现这一点，我们将添加随机量的噪声，因此我们将随机选择时间步数。
3. 将带有噪声的图像输入模型。
4. 使用均方误差（MSE）计算损失。MSE 是回归任务中常见的损失函数，包括 UNet 模型的

噪声预测。它测量预测值与真实值之间的平均平方差，对较大的误差进行更严厉的惩罚。在 UNet 模型中，MSE 是在预测噪声和实际噪声之间计算的，通过最小化损失帮助模型生成更逼真的图像。这被称为噪声或 epsilon 目标。

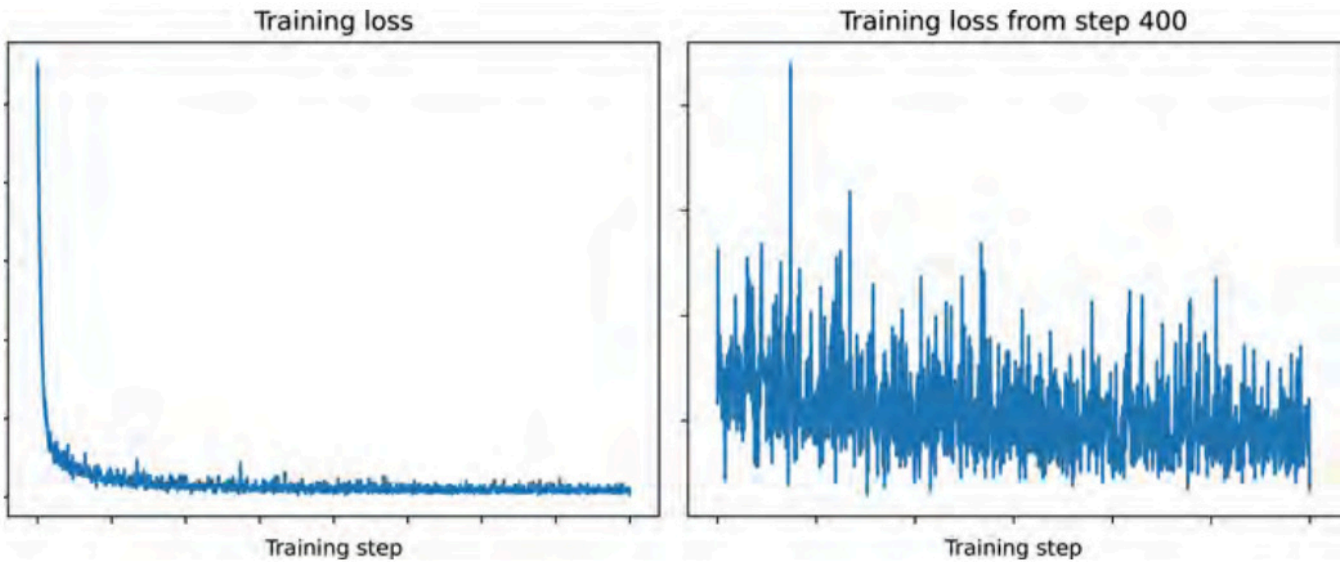
5. 反向传播损失并使用优化器更新模型权重。

以下是代码中的实现。训练需要一段时间，所以这是一个暂停、回顾本章内容或吃点东西的好时机：

```
from torch.nn import functional as F

num_epochs = 50  # 我们应该对数据进行多少次遍历？
lr = 1e-4  # 我们应该使用什么学习率
optimizer = torch.optim.AdamW(model.parameters(), lr=lr) []  # 用于存储损失值以便后续绘图
// 训练模型（这需要一段时间）
for epoch in range(num_epochs):
    for batch in train_dataloader:
        // 加载输入图像
        clean_images = batch["images"].to(device)
        // 采样噪声添加到图像中
        noise = torch.randn(clean_images.shape).to(device)
        // 为每张图像随机采样一个时间步
        timesteps = torch.randint(
            0, scheduler.config.num_train_timesteps, (clean_images.shape[0],),
                device=device,
        ).long()
        // 根据每个时间步的噪声幅度向干净图像添加噪声
        noisy_images = scheduler.add_noise(clean_images, noise, timesteps)
        // 获取模型对噪声的预测
        // 模型还使用时间步作为输入以进行额外的条件处理
        noise_pred = model(noisy_images, timesteps, return_dict=False)[0]
        // 将预测值与实际噪声进行比较
        loss = F.mse_loss(noise_pred, noise)
        // 存储损失以便后续绘图
        losses.append(loss.item())
        // 根据此损失使用优化器更新模型参数
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

现在模型已经训练好了，让我们绘制训练损失：

```
from matplotlib import pyplot as plt
plt.subplots(1, 2, figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(losses)
plt.title("训练损失")
plt.xlabel("训练步数")

plt.subplot(1, 2, 2)
plt.plot(range(400, len(losses)), losses[400:])
plt.title("从第 400 步开始的训练损失")
plt.xlabel("训练步数")
```

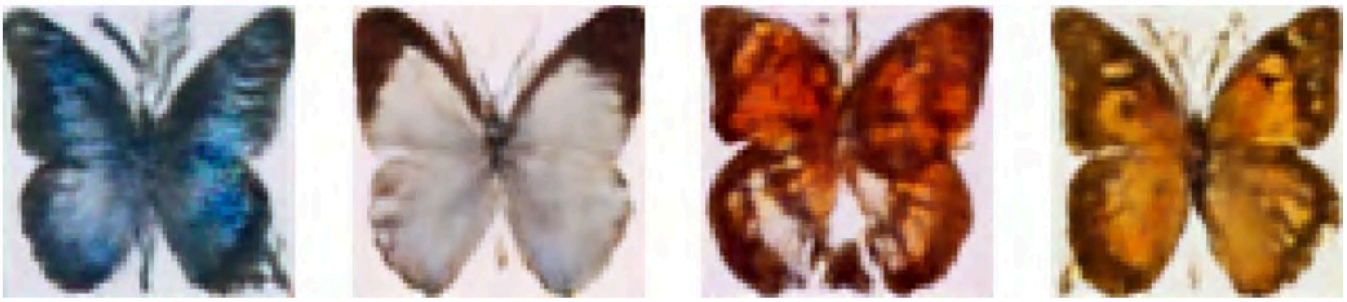



左侧的损失曲线显示了所有步骤，而右侧的曲线跳过了前 400 步。随着模型学会对图像进行去噪，损失曲线呈下降趋势。曲线有些波动——损失不太稳定。这是因为每次迭代使用了不同的噪声时间步数。通过查看噪声预测的 MSE 很难判断这个模型是否擅长生成样本，所以让我们继续下一节，看看它的表现如何。

## 采样

现在我们有了一个模型，让我们进行推理并生成一些图像。diffusers 库使用管道的概念将生成扩散模型样本所需的所有组件捆绑在一起。我们可以使用管道来测试我们刚刚训练的 UNet；以下展示了几次生成的结果：

```
pipeline = DDPMPipeline(unet=model, scheduler=scheduler)
ims = pipeline(batch_size=4).images
show_images(ims, nrows=1)
```



将生成样本的工作交给管道并没有展示其内部的工作原理。因此，让我们根据管道 call() 方法中的代码进行一个简单的采样循环，展示模型如何逐步细化输入图像：

```
// 随机起点（4 张随机图像）：
sample = torch.randn(4, 3, 64, 64).to(device)

for t in scheduler.timesteps:
    // 获取模型预测
    with torch.inference_mode():
        noise_pred = model(sample, t)["sample"]
    // 使用步骤更新样本
    sample = scheduler.step(noise_pred, t, sample).prev_sample
show_images(sample.clip(-1, 1) * 0.5 + 0.5, nrows=1)
```

这是我们在本章开头用来说明迭代细化概念的相同代码，但现在你更好地理解了这里发生的事情。如果你查看 diffusers 库中 DDPMPipeline 的实现，你会发现其逻辑与我们之前的代码片段非常相似。

我们从一个完全随机的输入开始，然后模型在一系列步骤中对其进行细化。每个步骤都是对输入

的一个小更新，基于模型在该时间步对噪声的预测。我们仍然抽象了 pipeline.scheduler.step() 调用背后的一些复杂性；稍后，我们将深入探讨不同的采样方法及其工作原理。

## 评估

评估生成模型是复杂的——这本质上是一项主观任务。例如，给定一个输入提示“戴着太阳镜的猫的图像”，有许多潜在的正确答案。一种常见的方法是将定性评估（例如，通过让人类比较生成结果）与定量指标结合起来，后者提供了评估的框架，但不一定对应于高图像质量。

Fréchet Inception Distance (FID) 分数可以评估生成模型的性能。FID 分数比较两个图像数据集的相似性。使用预训练的神经网络（如图 4-4 所示），它们通过比较从两个数据集提取的特征图之间的统计量来测量生成样本与真实样本的接近程度。分数越低，给定模型生成的图像质量和真实感越好。FID 分数之所以受欢迎，是因为它们能够为不同类型的生成网络提供“客观”的比较指标，而不依赖于人类判断。

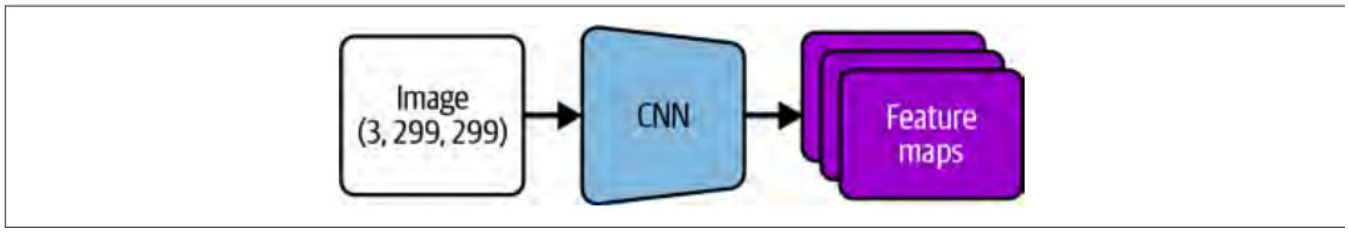



图 4-4. 用于从图像中提取特征图的 CNN 网络

尽管 FID 分数很方便，但有一些重要的注意事项需要了解（其他评估指标可能也是如此）：

- FID 分数旨在比较两个分布。因此，它假设我们可以访问一个源数据集进行比较。第二个问题是你无法计算单个生成的 FID 分数。如果我们只有一张图像，就无法计算其 FID 分数。
- 给定模型的 FID 分数取决于用于计算它的样本数量，因此在比较模型时，我们需要确保两个报告的分数是使用相同数量的样本计算的。通常的做法是使用 50,000 个样本，尽管为了节省时间，你可以在开发期间评估较少的样本，并在准备发布结果时进行完整评估。
- FID 可能对许多因素敏感。例如，不同的推理步骤数会导致非常不同的 FID。调度器（在本例中为 DDPM）也会影响 FID。
- 在计算 FID 时，图像会被调整为 $299 \times 299$ 的图像。这使得它作为极低或极高分辨率图像的指标不太有用。不同深度学习框架之间的调整大小处理方式也存在微小差异，这可能导致 FID 分数的轻微差异。
- 用于 FID 的特征提取器网络通常是在 ImageNet 分类任务上训练的模型。[6] 当生成不同领域的图像时，该模型学习到的特征可能不太有用。更准确的方法是首先在特定领域数据上训练分类网络，这使得比较不同论文和技术之间的分数变得更加困难。目前，ImageNet 模型是

标准选择。
- 如果你保存生成的样本以供后续评估，格式和压缩可能会影响 FID 分数。尽可能避免使用低质量的 JPEG 图像。

即使你考虑了所有这些注意事项，FID 分数也只是一个粗略的质量衡量标准，并不能完美捕捉到使图像看起来更“真实”的细微差别。生成模型的评估是一个活跃的研究领域。像核初始距离（KID）和初始分数（Inception Score）这样的标准指标与 FID 存在类似的问题。因此，使用这些指标来了解一个模型相对于另一个模型的表现，但也要查看每个模型生成的实际图像，以便更好地比较它们。

由 FID 或 KID 衡量的图像质量只是我们用来评估文本到图像模型性能的指标之一。诸如**文本到图像模型整体评估（HEIM）** 等尝试考虑了文本到图像模型的额外理想特性，例如提示遵循性、原创性、推理能力、多语言支持、无偏见和毒性等。

在最终相当主观的领域中，人类偏好仍然是质量的黄金标准。例如，**Parti Prompts** 数据集包含 1,600 个不同难度和类别的提示，并允许比较文本到图像模型，例如我们将在第 5.7 章中探讨的模型。

# 深入探讨：噪声调度

在前面的训练示例中，其中一个步骤是“添加不同量的噪声”。我们通过在0到1,000之间随机选择一个时间步长，然后依赖调度器来添加适当量的噪声来实现这一点。同样，在推理过程中，我们再次依赖调度器来告诉我们使用哪些时间步长以及如何根据模型预测从一个时间步长移动到下一个时间步长。选择添加多少噪声是一个关键的设计决策，它可能会极大地影响给定模型的性能。在本节中，我们将探讨为什么会出现这种情况，并探索实践中使用的不同方法。

## 为什么要添加噪声？

在本章的开头，我们说过扩散模型背后的关键思想是迭代优化。在训练过程中，我们通过不同程度的破坏来损坏输入。在推理过程中，我们从最大程度损坏的输入（即纯噪声图像）开始，并逐步去噪，期望最终得到一个良好的结果。

到目前为止，我们专注于一种特定的破坏方式：添加高斯噪声。高斯噪声是一种遵循正态分布的噪声，正如我们在第3章中看到的，大多数值集中在均值附近，随着距离均值越远，值越少。这种关注的一个原因是扩散模型的理论基础，它假设使用高斯噪声——如果我们使用不同的破坏方法，从技术上讲，我们就不再是进行扩散了。

然而，《Cold Diffusion》论文表明，我们不一定仅仅为了理论上的方便而将自己限制在这种方

法中。作者展示了（图4-5）类似扩散模型的方法适用于许多破坏方法。这意味着我们可以使用其他图像变换而不是噪声。例如，Muse、MaskGIT和Paella等模型使用了随机标记掩码或替换作为等效的破坏方法。

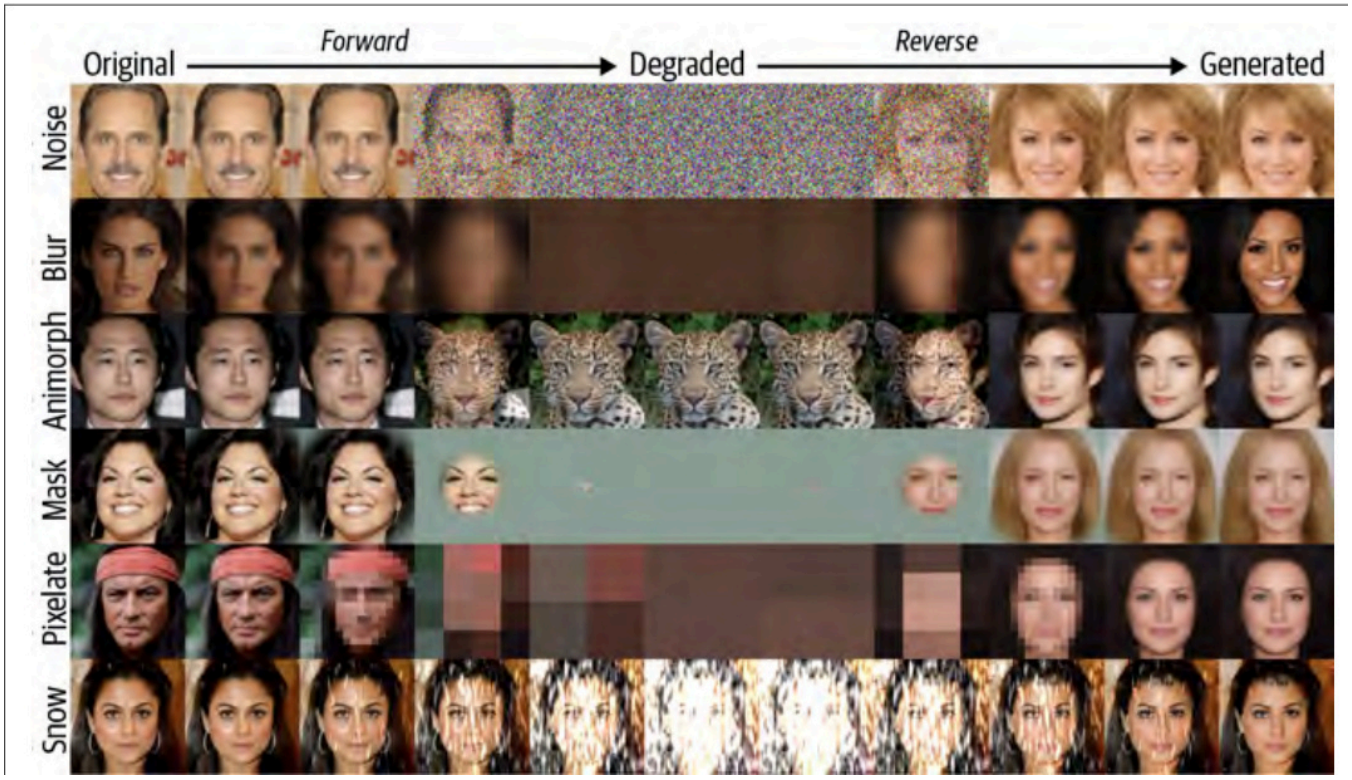



图4-5. 扩散的一般原理适用于其他类型的破坏，而不仅仅是高斯噪声（改编自《Cold Diffusion》论文中的图像）

尽管如此，添加噪声仍然是最流行的方法，原因如下：

- 我们可以轻松控制添加的噪声量，从而实现从“完美”到“完全损坏”的平滑过渡。这与降低图像分辨率等方法不同，后者可能会导致“离散”的过渡。
- 我们可以有许多有效的随机推理起点，而某些方法可能只有有限数量的可能初始（完全损坏）状态，例如完全黑色的图像或单像素图像。

因此，目前我们将添加噪声作为我们的破坏方法。接下来，让我们探讨如何向图像中添加噪声。

## 从简单开始

我们有一些图像$x$，我们想向它们添加一些随机噪声。我们使用`torch.rand_like()`生成与输入图像相同维度的纯高斯噪声：

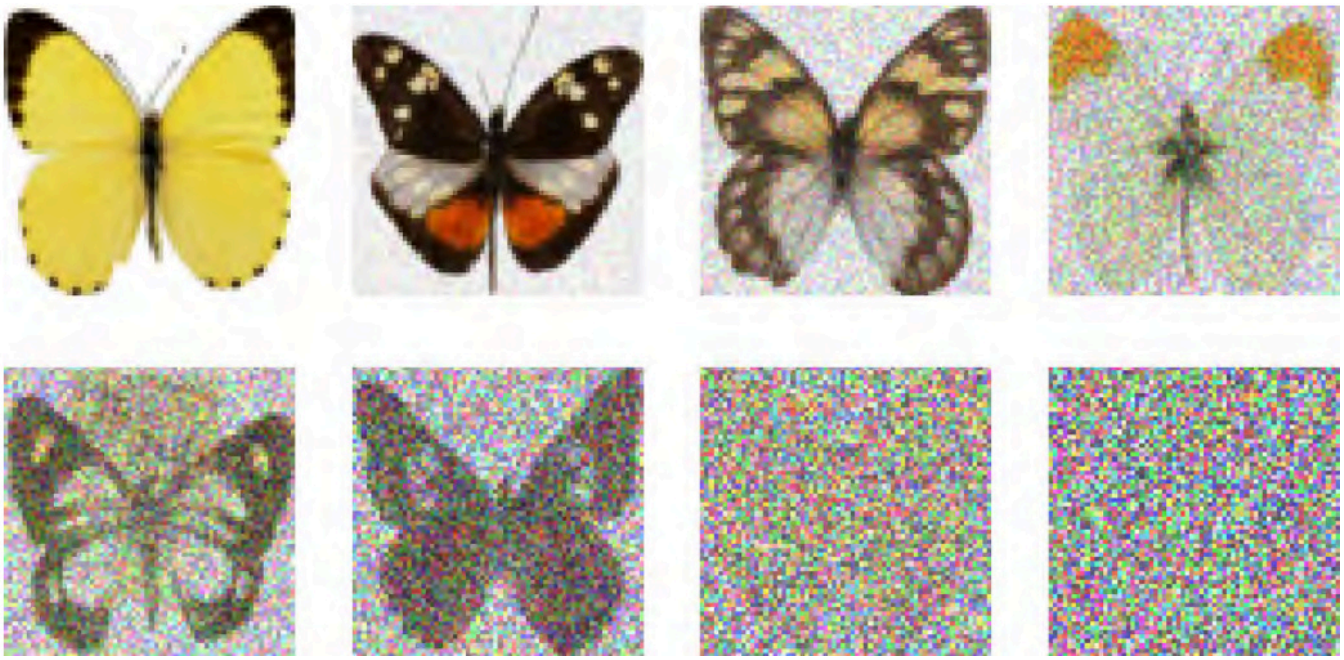
```
x = next(iter(train_dataloader))["images"][:8]
noise = torch.rand_like(x)
```

一种添加不同量噪声的方法是线性插值（简称“lerp”）图像和噪声之间的某个量。这为我们提供了一个函数，随着量的变化从0到1，它从原始图像$x$平滑过渡到纯噪声：

```
def corrupt(x, noise, amount):
    // 重塑amount，使其与原始数据正确配合
    amount = amount.view(-1, 1, 1, 1)  # 确保它可以广播
    // 根据量混合原始数据和噪声
    return ( x * (1 - amount) + noise * amount ) # 等价于x.lerp(noise, amount)
```

让我们在一批数据上看看这个效果，噪声量从0到1变化：

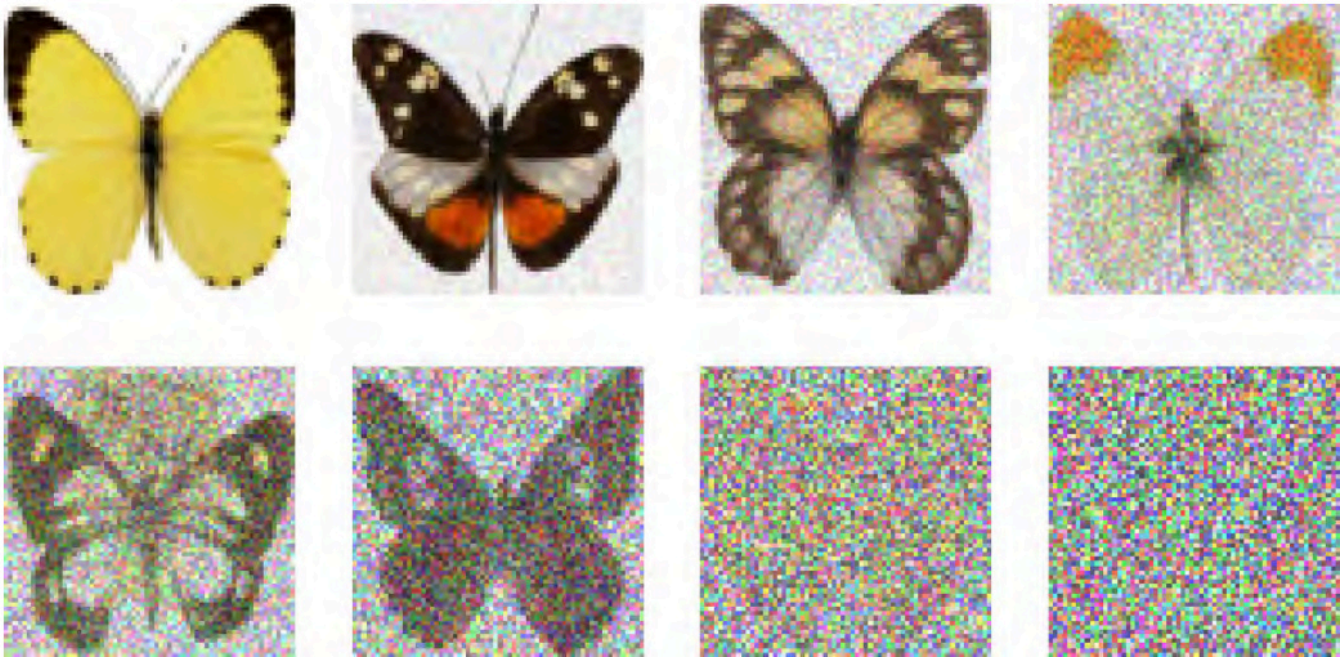
```
amount = torch.linspace(0, 1, 8)
noised_x = corrupt(x, noise, amount)
show_images(noised_x * 0.5 + 0.5)
```



这实现了我们想要的效果：从原始图像平滑过渡到纯噪声。我们使用连续时间方法创建了一个噪声调度，其中我们在0到1的时间尺度上表示完整路径。其他方法使用离散时间方法，使用大量整数时间步长来定义噪声调度器。我们可以将我们的函数包装成一个类，将连续时间转换为离散时间步长，并适当地添加噪声：

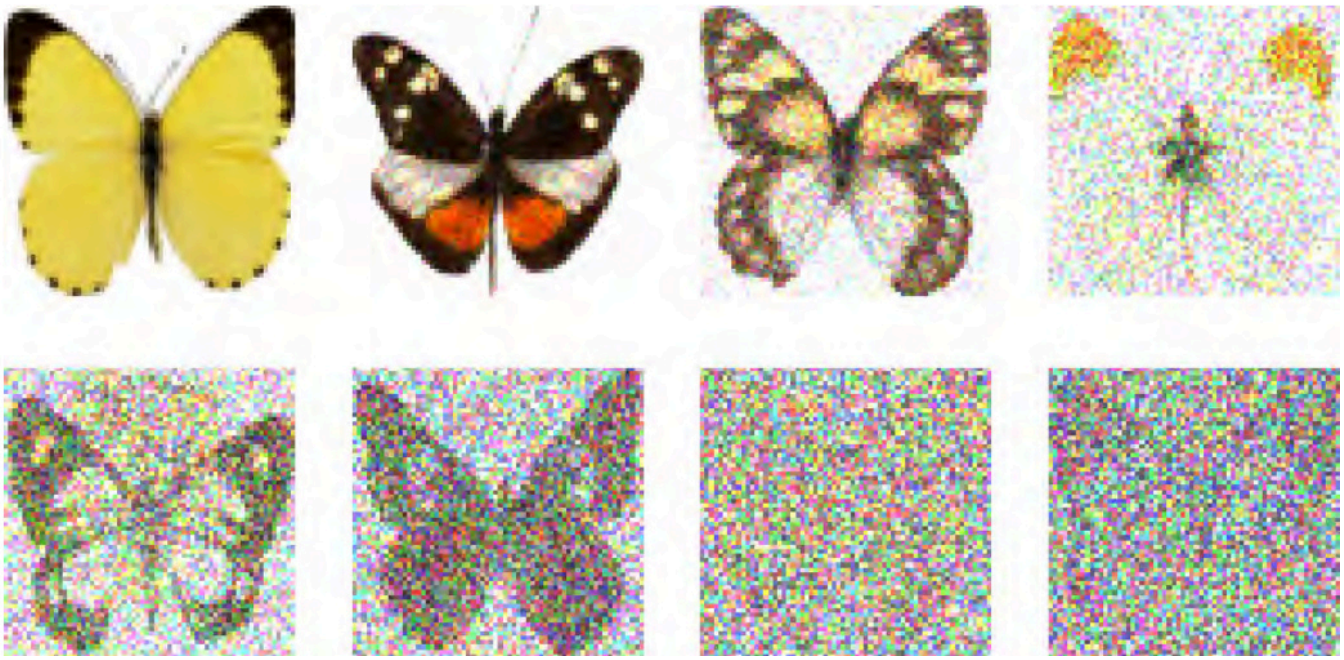
```
class SimpleScheduler:
    def __init__(self):
        self.num_train_timesteps = 1000
    def add_noise(self, x, noise, timesteps):
        amount = timesteps / self.num_train_timesteps
        return corrupt(x, noise, amount)
```

```
scheduler = SimpleScheduler()
timesteps = torch.linspace(0, 999, 8).long()
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images(noised_x * 0.5 + 0.5)
```



现在我们有了可以直接与 `diffusers` 库中使用的调度器（如我们在训练期间使用的 `DDPMScheduler`）进行比较的东西。让我们看看它的比较结果：

```
scheduler = DDPMScheduler(beta_end=0.01)
timesteps = torch.linspace(0, 999, 8).long()
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0, 1))
```



如果你将我们的调度器的结果与`DDPMScheduler`的结果进行比较，你可能会注意到它们并不完全相同，但它们足够相似，可以探索使用我们的噪声调度器训练模型。

## 数学原理

让我们深入探讨一下解释如何向原始图像添加噪声的数学原理。需要记住的一点是，文献中有许多不同的符号和方法。例如，在某些论文中，噪声调度是连续参数化的，因此 $t$ 从 0（无噪声）到 1（完全损坏）运行，就像我们在 corrupt 函数中所做的那样。其他论文使用离散时间方法，其中时间步是整数，从 0 到一个较大的数 $T$，通常是 1,000。可以通过我们 SimpleScheduler 类的方式在这两种方法之间进行转换——在比较不同模型时，请确保保持一致。我们将在这里坚持使用离散时间方法。

深入理解数学的一个好起点是 DDPM 论文或“Annotated Diffusion Model”博客文章。如果你觉得这部分内容过于密集，可以先关注高层次的概念，稍后再回到数学部分。

让我们从定义如何从时间步 $t-1$ 到时间步 $t$ 进行单步噪声添加开始。如前所述，其思想是添加高斯噪声 $(\epsilon)$。噪声具有单位方差，控制噪声值的分布。通过将此噪声添加到上一步的图像中，我们逐渐损坏原始图像，这是扩散模型训练过程的关键部分：

$$\mathbf{x}_t = \mathbf{x}_{t-1} + \epsilon$$

为了控制每一步添加的噪声量，我们引入 $\beta_t$。该参数为所有时间步 $t$ 定义，并指定每一步应添加多少噪声。换句话说，$\mathbf{x}_t$ 是 $\mathbf{x}_{t-1}$ 和由 $\beta_t$ 缩放的随机噪声的混合。这使我们能够随着时间步的推移逐渐增加添加到图像中的噪声量，这是扩散模型训练过程的关键部分：

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}$$

我们可以进一步将噪声添加过程定义为一种分布，其中带噪 $\mathbf{x}_t$ 的均值为 $\sqrt{1-\beta_t}\mathbf{x}_{t-1}$，方差为 $\beta_t$。这种分布帮助我们更准确地建模噪声添加过程。以下是分布形式的公式：

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}\right)$$

我们现在已经定义了一个分布，用于在给定前一个值的情况下对 $x$ 进行采样。为了在时间步 $t$ 获得噪声输入，我们可以从 $t=0$ 开始并重复应用此单步，但这将非常低效。相反，我们可以通过重参数化技巧找到一个公式，直接跳到任何时间步 $t$。其思想是预先计算由 $\beta_t$ 值定义的噪声调度。然后我们可以定义 $\alpha_t = 1-\beta_t$ 和 $\bar{\alpha}$ 为所有 $\alpha$ 值到时间 $t$ 的累积乘积，可以表示为 $\bar{\alpha}_t = \prod_{s=1}^{t}\boldsymbol{\alpha}_s$。使用这些工具和符号，我们可以重新定义分布以及如何在特定时间进行采样。新的分布 $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ 的均值为 $\overline{\alpha}_t\mathbf{x}_{t-1}$，方差为 $(1-\overline{\alpha}_t)\mathbf{I}$：

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t; \overline{\alpha}_t\mathbf{x}_{t-1}, \left(1-\overline{\alpha}_t\right)\mathbf{I}\right)$$

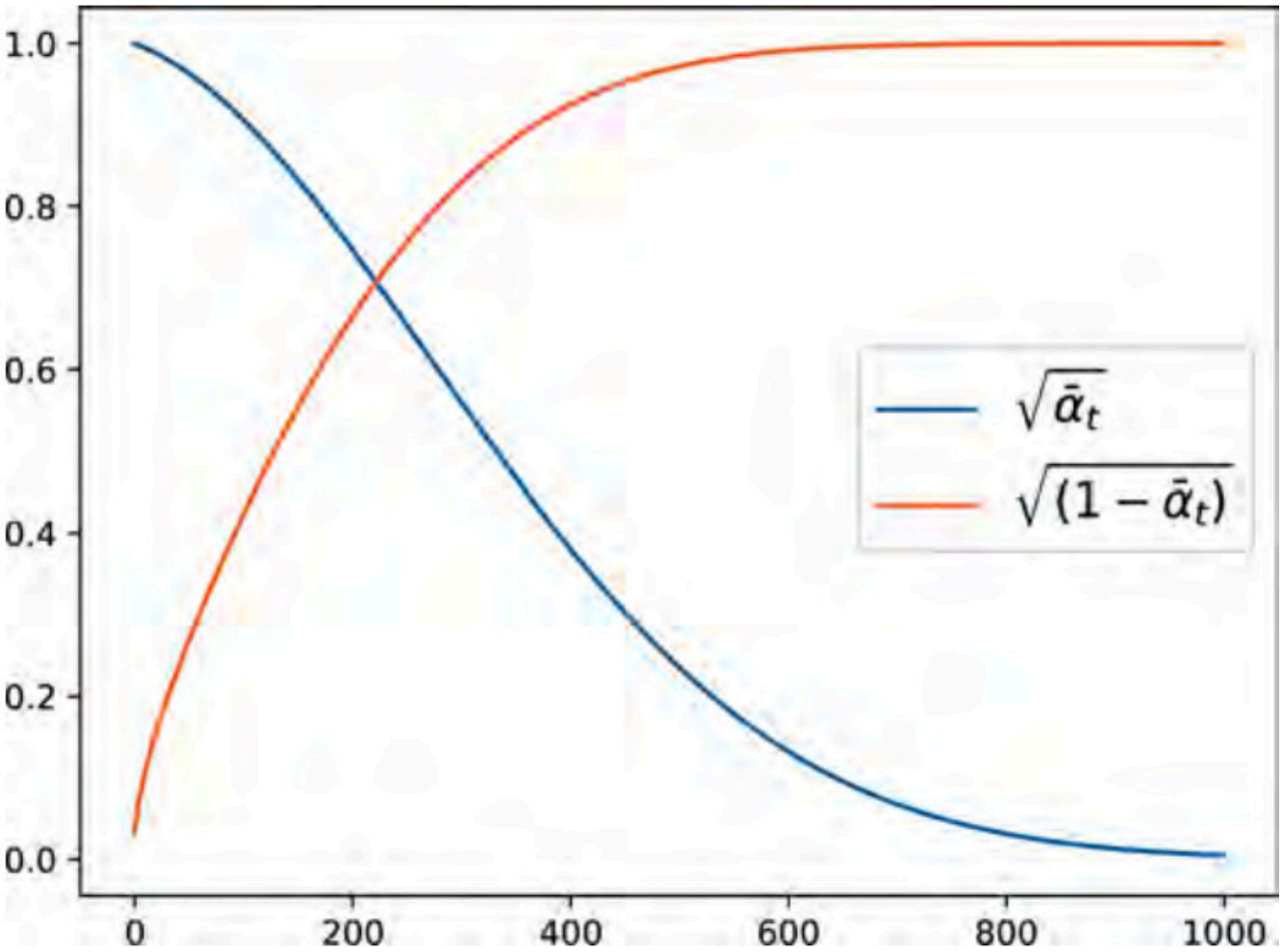
探索这个重参数化技巧是本章末尾挑战的一部分。我们现在可以使用以下公式在时间步 $t$ 采样噪声图像：

$$x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\epsilon$$

$x_t$ 的方程表明，时间步 $t$ 的噪声输入是原始图像 $x_0$（由 $\sqrt{\overline{\alpha}_t}$ 缩放）和 $\epsilon$（由 $\sqrt{1-\overline{\alpha}_t}$ 缩放）的组合。请注意，我们现在可以直接计算样本，而无需遍历所有先前的时间步，这使得训练扩散模型更加高效。

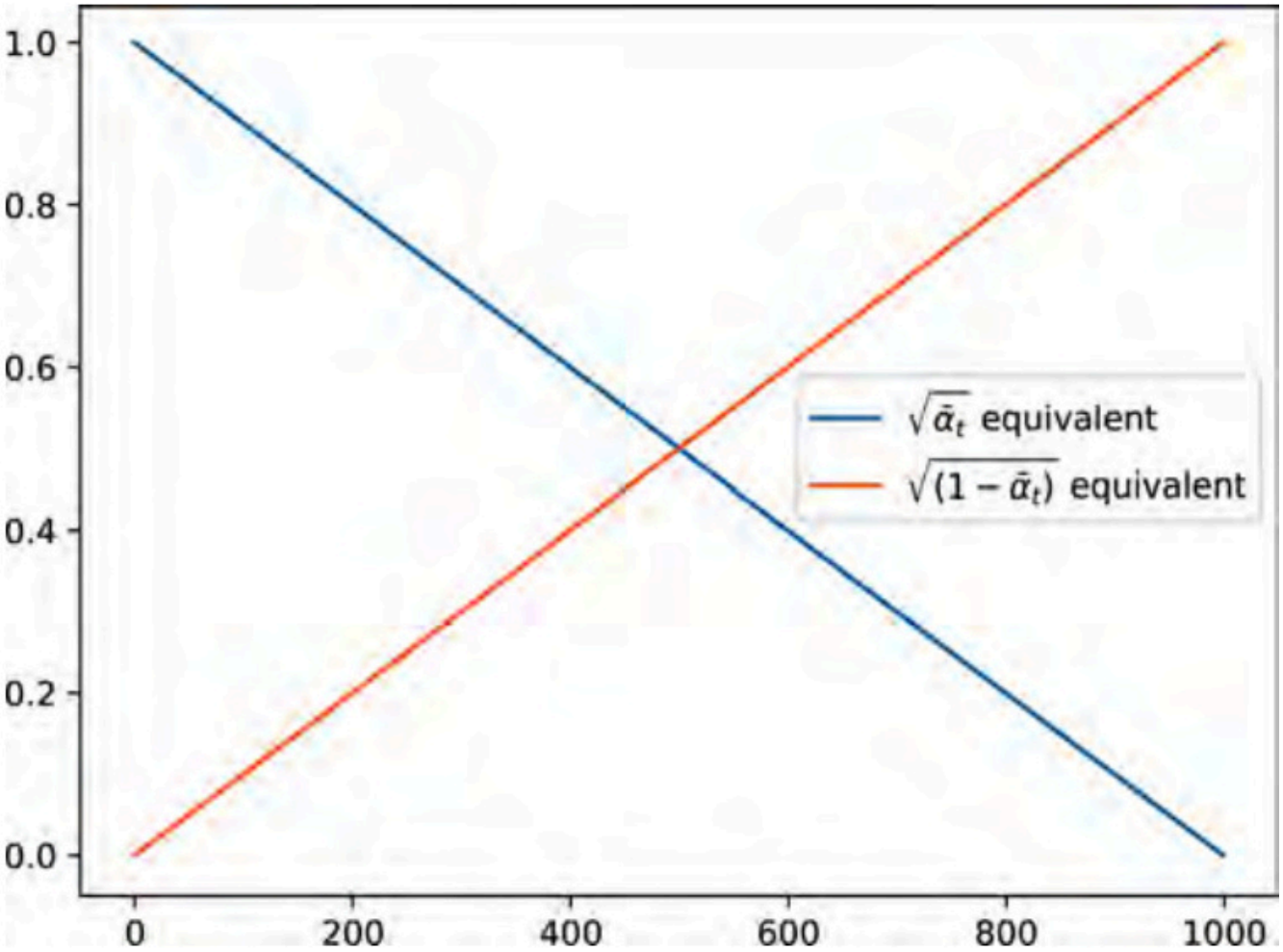
在 diffusers 库中，$\bar{\alpha}$ 值存储在 scheduler.alphas_cumprod 中。知道这一点后，我们可以绘制给定调度器下原始图像 $x_0$ 和噪声 $\epsilon$ 在不同时间步的缩放因子。diffusers 库允许我们通过定义其初始值（beta_start）、最终值（beta_end）以及值的步进方式（例如线性 beta_schedule="linear"）来控制 beta 值。以下 DDPMScheduler 的图描述了添加到输入图像中的噪声量（橙色线）和输入图像（蓝色线）。我们可以看到，随着时间步的增加，噪声的缩放比例也增加，正如预期的那样：

```
from genaibook.core import plot_scheduler
plot_scheduler( DDPMScheduler(beta_start=0.001, beta_end=0.02,
    beta_schedule="linear")
```




我们的 `SimpleScheduler` 只是线性混合原始图像和噪声，正如我们绘制缩放因子（相当于 DDPM 情况下的 $\sqrt{\alpha_t}$ 和 $\sqrt{(1-\alpha_t)}$）时可以看到的那样：

```
plot_scheduler(SimpleScheduler())
```

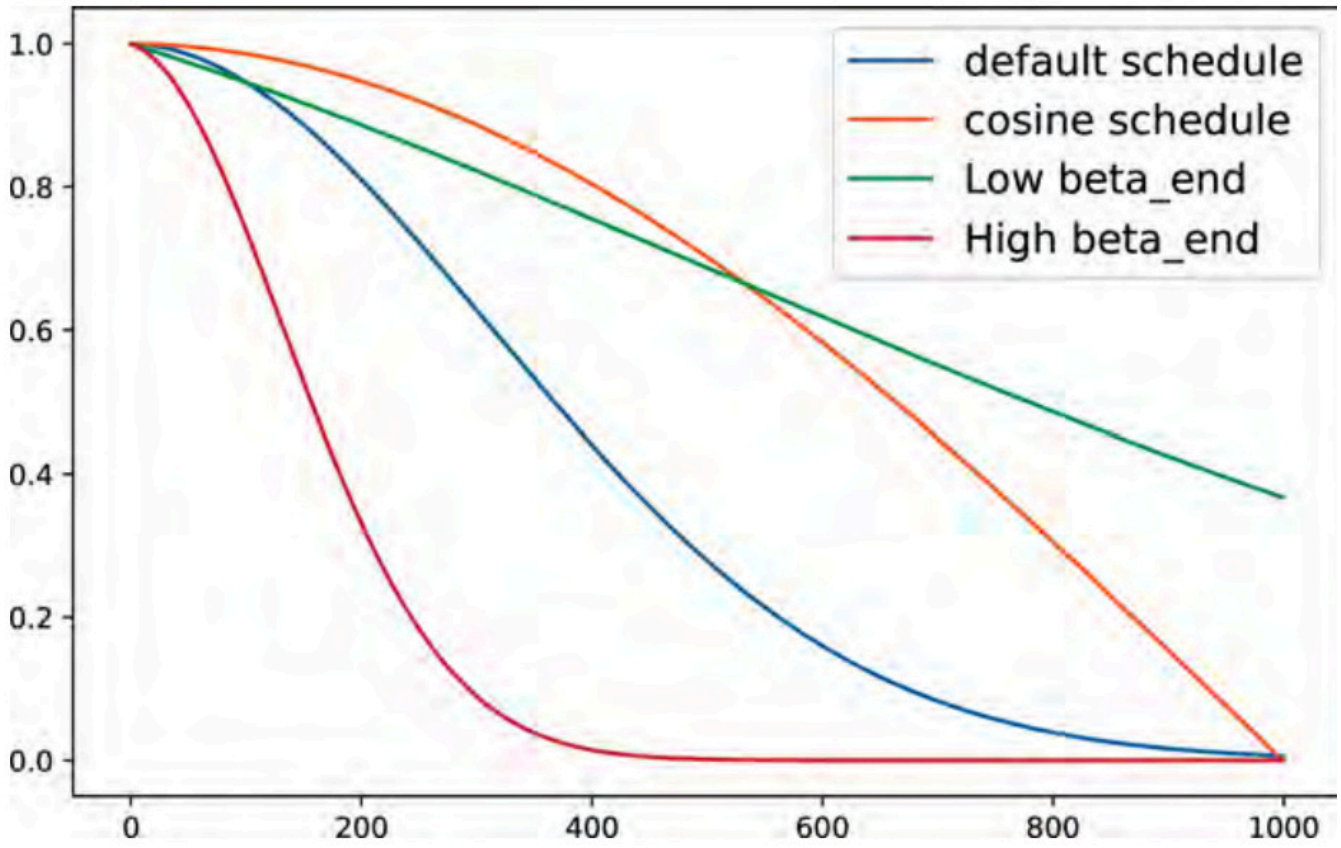



一个好的噪声调度将确保模型看到不同噪声水平的图像混合。最佳选择将根据训练数据而有所不同。可视化更多选项时，请注意以下几点：

- 将 `beta_end` 设置得太低意味着我们永远不会完全损坏图像，因此模型永远不会看到像推理起点那样的随机噪声。
- 将 `beta_end` 设置得极高意味着大多数时间步都花在几乎完全噪声上，导致训练性能不佳。
- 不同的 beta 调度给出不同的曲线。余弦调度很受欢迎，因为它从原始图像平滑过渡到噪声。

让我们通过以下图来可视化不同 DDPMScheduler 调度器的比较，变化超参数和调度：

```
fig, (ax) = plt.subplots(1, 1, figsize=(8, 5))
plot_scheduler(
    DDPMScheduler(beta_schedule="linear"),
    label="default schedule",
    ax=ax,
    plot_both=False,
)
plot_scheduler(
    DDPMScheduler(beta_schedule="squaredcos_cap_v2"),
    label="cosine schedule",
    ax=ax,
    plot_both=False,
)
plot_scheduler(
    DDPMScheduler(beta_start=0.001, beta_end=0.003, beta_schedule="linear"),
    label="Low beta_end",
    ax=ax,
    plot_both=False,
)
plot_scheduler(
    DDPMScheduler(beta_start=0.001, beta_end=0.1, beta_schedule="linear"),
    label="High beta_end",
    ax=ax,
    plot_both=False,
)
```



这里显示的所有调度都称为方差保持（VP），这意味着模型输入的方差在整个调度过程中保持接近 1。你可能还会遇到方差爆炸（VE）公式，其中噪声以不同量添加到原始图像中（导致高方差输入）。我们的 `SimpleScheduler` 几乎是一个 VP 调度，但由于线性插值，方差并未完全保持。

在一篇题为“常见扩散噪声计划和采样步骤存在缺陷”的论文中，探讨了将模型暴露于良好混合的噪声图像（包括纯噪声，这是推理的初始状态）的重要性。该论文表明，由于训练计划未覆盖所有状态，一些扩散模型无法生成过亮或过暗的图像。与许多扩散相关主题一样，不断有新的论文在探索噪声计划这一主题，因此在你阅读本文时，可能会有大量可供尝试的选择。[9]

## 输入分辨率和缩放的影响

直到最近，噪声调度中一个被大多数忽视的方面是输入大小和缩放的影响。许多论文在小规模数据集和低分辨率下测试潜在的调度器，然后使用表现最好的调度器在更大的图像上训练他们的最终模型。如果我们向两张不同大小的图像添加相同量的噪声，这个问题就会显现出来，如图4-6所示。

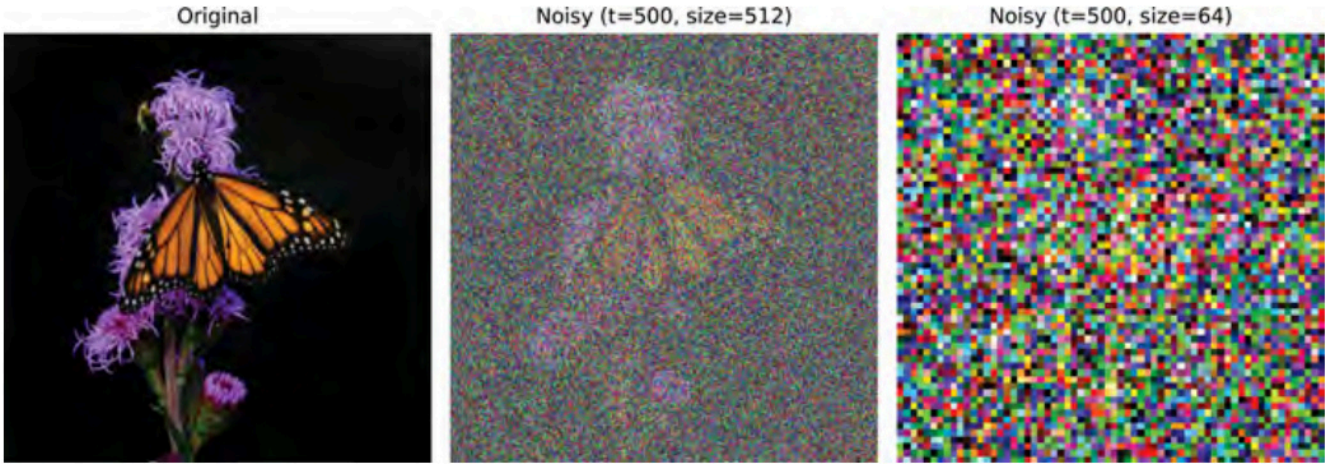
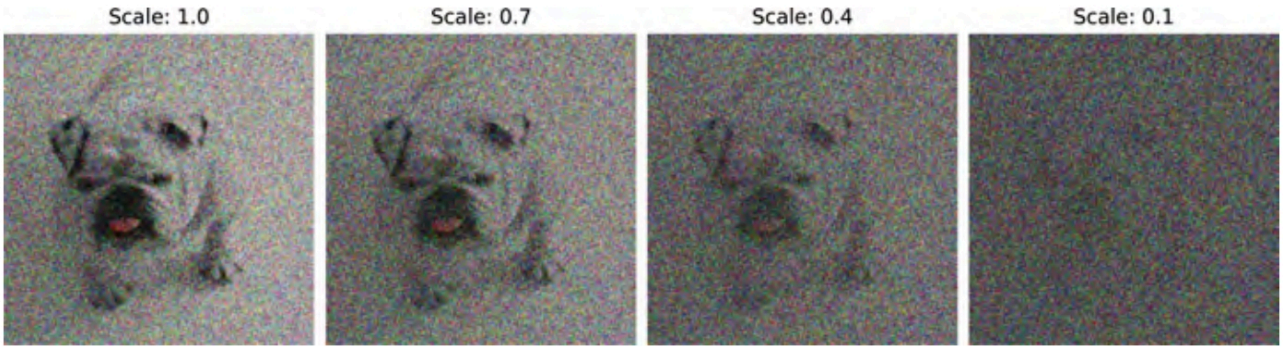



图4-6. 向不同分辨率的图像应用相同量的输入噪声

高分辨率图像往往包含大量冗余信息。这意味着即使单个像素被噪声遮挡，周围的像素也有足够的信息来重建原始图像。这与低分辨率图像不同，在低分辨率图像中，单个像素可能包含大量有用信息。向低分辨率图像添加相同量的噪声将导致图像比向高分辨率图像添加等量噪声时更加损坏。

2023年初的两篇独立论文深入研究了这一现象。每篇论文都利用新的见解训练了能够生成高分辨率输出的模型，而无需使用之前必要的任何技巧。简单扩散引入了一种基于输入大小调整噪声调度的方法，允许将在低分辨率图像上优化的调度适当地修改为新的目标分辨率。另一篇论文进行了类似的实验，并指出了另一个关键变量：输入缩放。即，我们如何表示图像？如果图像表示为0到1之间的浮点数，它们的方差将低于噪声（通常为单位方差）。因此，对于给定的噪声水平，信噪比将低于图像表示为$^{-1}$到1之间的浮点数（我们在前面的训练示例中使用）或其他表示方式的情况。缩放输入图像会改变信噪比，因此在训练较大图像时，修改这种缩放量是另一种调整方式。事实上，这篇论文建议将输入缩放作为适应不同图像大小训练的简单方法。也可以根据分辨率调整噪声调度，但由于涉及多个超参数，找到最佳调度会更加困难。这里我们看到了输入缩放的影响：

```
import numpy as np
from genaibook.core import SampleURL

scheduler = DDPMScheduler(beta_end=0.05, beta_schedule="scaled_linear")
image = load_image(
    SampleURL.DogExample,
    size=((512, 512)),
    return_tensor=True,
)

t = torch.tensor(300)  # The timestep we're noising to
scales = np.linspace(0.1, 1.0, 4)

images = [image]
noise = torch.randn_like(image)
for b in reversed(scales):
    noised = (
        scheduler.add_noise(b * (image * 2 - 1), noise, t).clip(-1, 1) * 0.5
        + 0.5
    )
    images.append(noised)

show_images(
    images[1:],
    nrows=1,
    titles=[f"Scale: {b}" for b in reversed(scales)],
    figsize=(15, 5),
)
```




所有图像都应用了相同的输入噪声，对应于步骤$t = 300$，但我们乘以不同的缩放因子。随着缩

放对图像的影响更大，噪声更加明显。缩放还减少了动态范围（或方差），导致输入看起来更暗。[10]

# 深入探讨：UNet及其替代方案

让我们来讨论进行所有重要预测的实际模型。回顾一下，该模型必须能够接收噪声图像并输出其噪声，从而实现对输入图像的去噪。这需要一个能够接收任意大小的图像并输出相同大小图像的模型。此外，该模型应该能够在像素级别进行精确预测，同时捕捉图像的高级信息。一种流行的方法是使用称为UNet的架构。UNet于2015年发明，用于医学图像分割，此后成为各种图像相关任务的热门选择。

与我们在前一章中看到的自动编码器和VAE类似，UNet由一系列下采样和上采样块组成。下采样块负责减小图像大小，而上采样块负责增加图像大小。下采样块通常由一系列卷积层组成，后跟一个池化或下采样层。[11]上采样块通常包括一系列卷积层，后跟一个上采样或转置卷积层。转置卷积层是一种特殊类型的卷积层，它增加图像的大小而不是减小它。

常规的自动编码器和VAE不适合此任务，因为它们从低维潜在空间重建图像时，在像素级别进行精确预测的能力较差。在UNet中，下采样和上采样块通过跳跃连接相连，这使得信息可以直接从下采样块流向上采样块。这使得模型能够在像素级别进行精确预测，同时还能捕捉图像整体的高级信息。

## 一个简单的UNet

为了更好地理解UNet的结构，让我们从头开始构建一个简单的UNet。
图4-7展示了一个基本UNet的架构图。

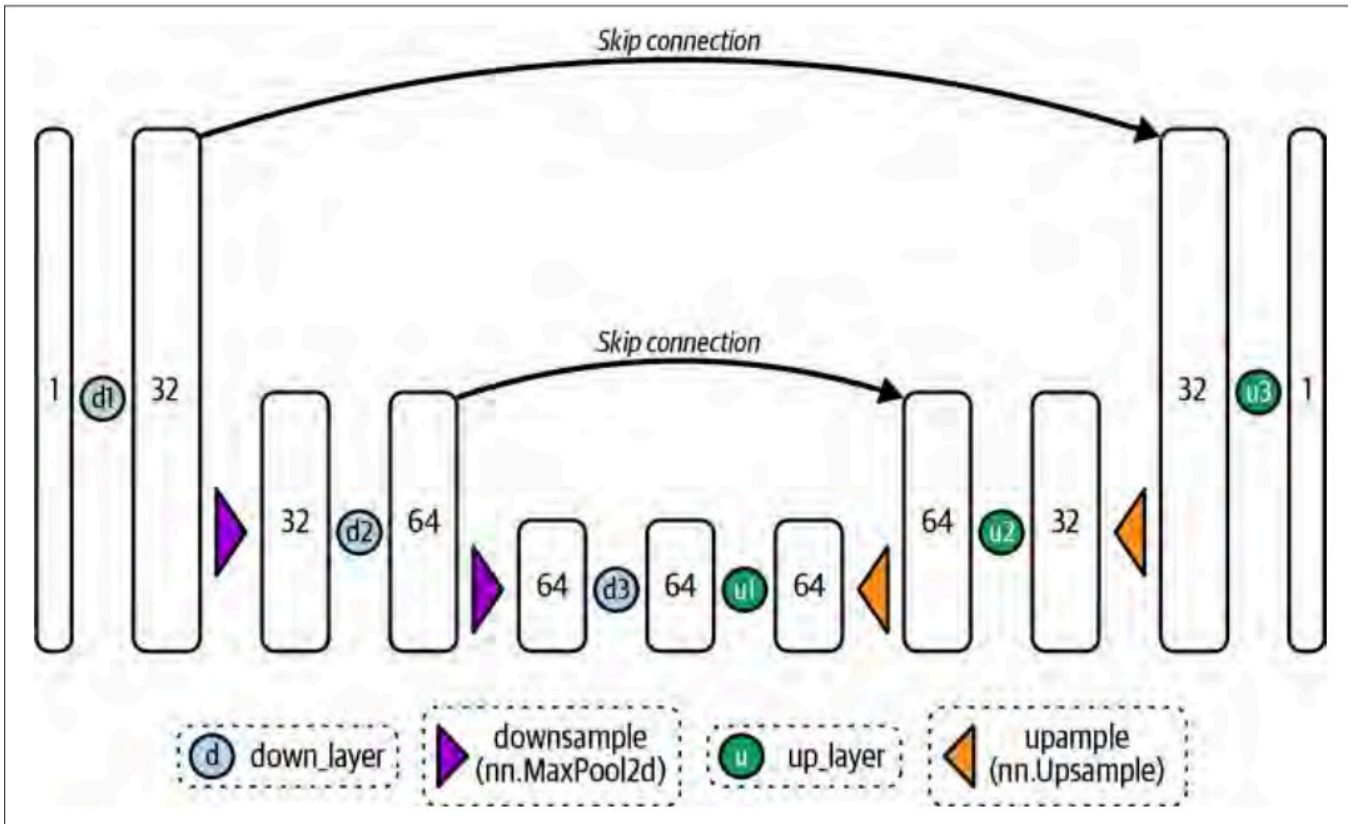



图4-7. 基本UNet的架构

我们将设计一个适用于单通道图像（例如灰度图像）的UNet，可以用来为MNIST等数据集构建扩散模型。我们将在下采样路径中使用三层，在上采样路径中使用另外三层。每一层由一个卷积层、一个激活函数和一个上采样或下采样步骤组成，具体取决于它们是在编码路径还是解码路径中。如前所述，跳跃连接直接将下采样块与上采样块连接起来。实现跳跃连接有多种方法。

我们将使用的一种方法是将下采样块的输出添加到相应上采样块的输入中。另一种方法是将下采样块的输出与上采样块的输入进行拼接。我们甚至可以在跳跃连接中添加一些额外的层。

现在让我们保持简单，采用初始方法。以下是这个网络的代码实现：

```
from torch import nn

class BasicUNet(nn.Module):
    """A minimal UNet implementation."""
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down_layers = nn.ModuleList(
            [
                nn.Conv2d(in_channels, 32, kernel_size=5, padding=2),
                nn.Conv2d(32, 64, kernel_size=5, padding=2),
                nn.Conv2d(64, 64, kernel_size=5, padding=2),
            ]
        )
        self.up_layers = nn.ModuleList(
            [
                nn.Conv2d(64, 64, kernel_size=5, padding=2),
                nn.Conv2d(64, 32, kernel_size=5, padding=2),
                nn.Conv2d(32, out_channels, kernel_size=5, padding=2),
            ]
        )
        // 使用SiLU激活函数，它已被证明效果良好
        // 由于其不同的特性（平滑性、非单调性等）。
        self.act = nn.SiLU()
        self.downscale = nn.MaxPool2d(2)
        self.upscale = nn.Upsample(scale_factor=2)

    def forward(self, x):
        h = []
        for i, l in enumerate(self.down_layers):
            x = self.act(l(x))
            if i < 2:  // 除了第三层（最后一层）下采样层：
                h.append(x)  // 存储输出用于跳跃连接
                x = self.downscale(x)  // 下采样，准备进入下一层
        for i, l in enumerate(self.up_layers):
            if i > 0:  // 除了第一层上采样层
                x = self.upscale(x)  // 上采样
                x += h.pop()  // 获取存储的输出（跳跃连接）
            x = self.act(l(x))
        return x
```

如果你输入一个形状为(1, 28, 28)的灰度图像，模型的处理路径如下：

1. 图像通过下采样块。第一层是一个具有32个滤波器的2D卷积，将其形状变为[32, 28, 28]。
2. 然后图像通过最大池化下采样，形状变为[32, 14, 14]。MNIST数据集包含在黑色背景上绘制的白色数字（黑色由数字零表示）。我们选择最大池化以选择区域中的最大值，从而聚焦于最亮的像素。
3. 图像通过第二个下采样块。第二层是一个具有64个滤波器的2D卷积，将其形状变为[64, 14, 14]。
4. 经过另一次下采样后，形状变为[64, 7, 7]。
5. 下采样块中有第三层，但这次没有下采样，因为我们已经使用了非常小的$7 \times 7$块。这将保持[64, 7, 7]的形状。
6. 我们以相反的顺序进行相同的处理，上采样到[64, 14, 14]，[32, 14, 14]，最后到[1, 28, 28]。

使用这种架构在MNIST上训练的扩散模型生成的样本如图4-8所示（代码包含在补充材料中，但为简洁起见在此省略）。

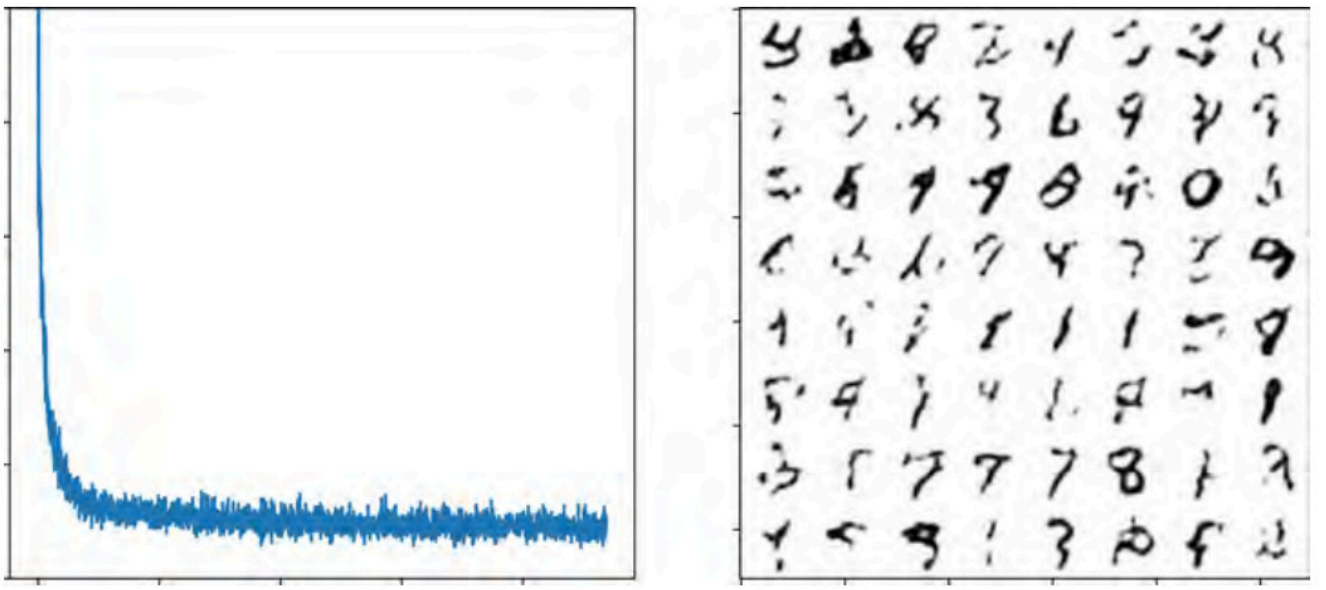


图4-8. 基本UNet的损失和生成结果

## 改进UNet

这个简单的UNet适用于这个相对简单的任务。我们如何处理更复杂的数据？以下是一些选项：

增加更多参数

这可以通过在每个块中使用多个卷积层、在每个卷积层中使用更多的滤波器或使网络更深来实现。

添加归一化，例如批量归一化

批量归一化可以通过确保每层的输出以0为中心并具有标准差为1，帮助模型更快、更可靠地学习。

添加正则化，例如dropout

Dropout有助于防止对训练数据的过拟合，这在处理较小数据集时尤为重要。

添加注意力机制

引入自注意力层允许模型在不同时间聚焦于图像的不同部分，这可以帮助UNet学习更复杂的函数。添加类似Transformer的注意力层还可以让我们增加可学习参数的数量。缺点是注意力层在高分辨率下的计算成本比常规卷积层高得多，因此我们通常只在较低分辨率下使用它们（例如UNet中的低分辨率块）。

作为对比，图4-9展示了使用diffusers库中的UNet实现（包含上述改进）在MNIST上的结果。

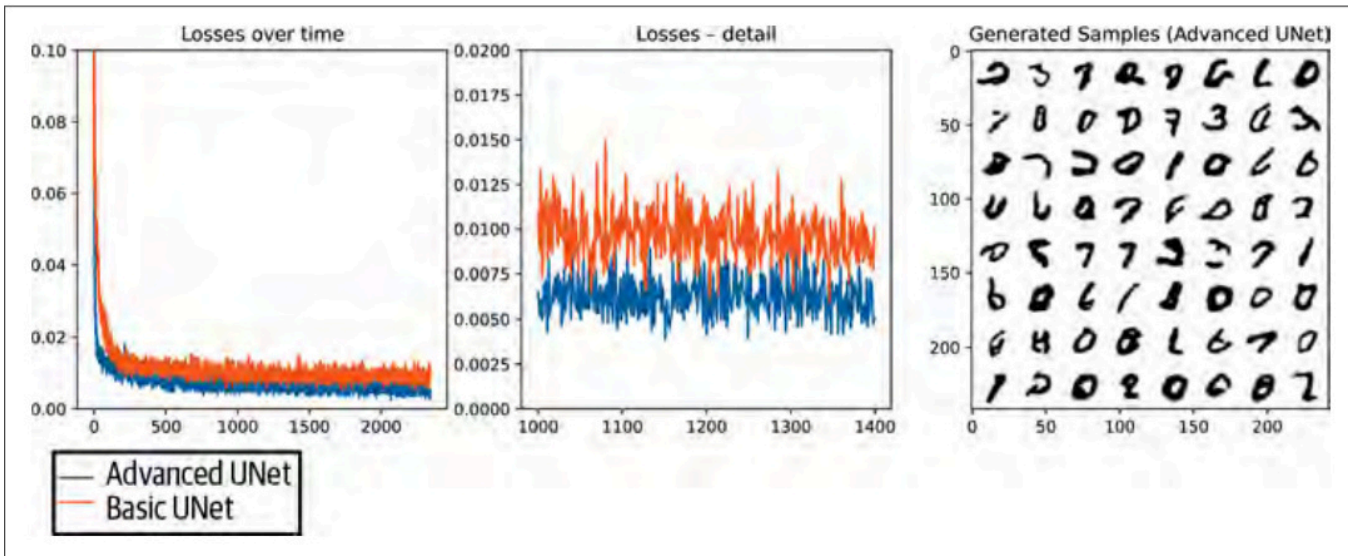



图4-9. 使用diffusers UNet的损失和生成结果，相比基本架构有多个改进

## 替代架构

最近，针对扩散模型提出了几种替代架构（图4-10）。

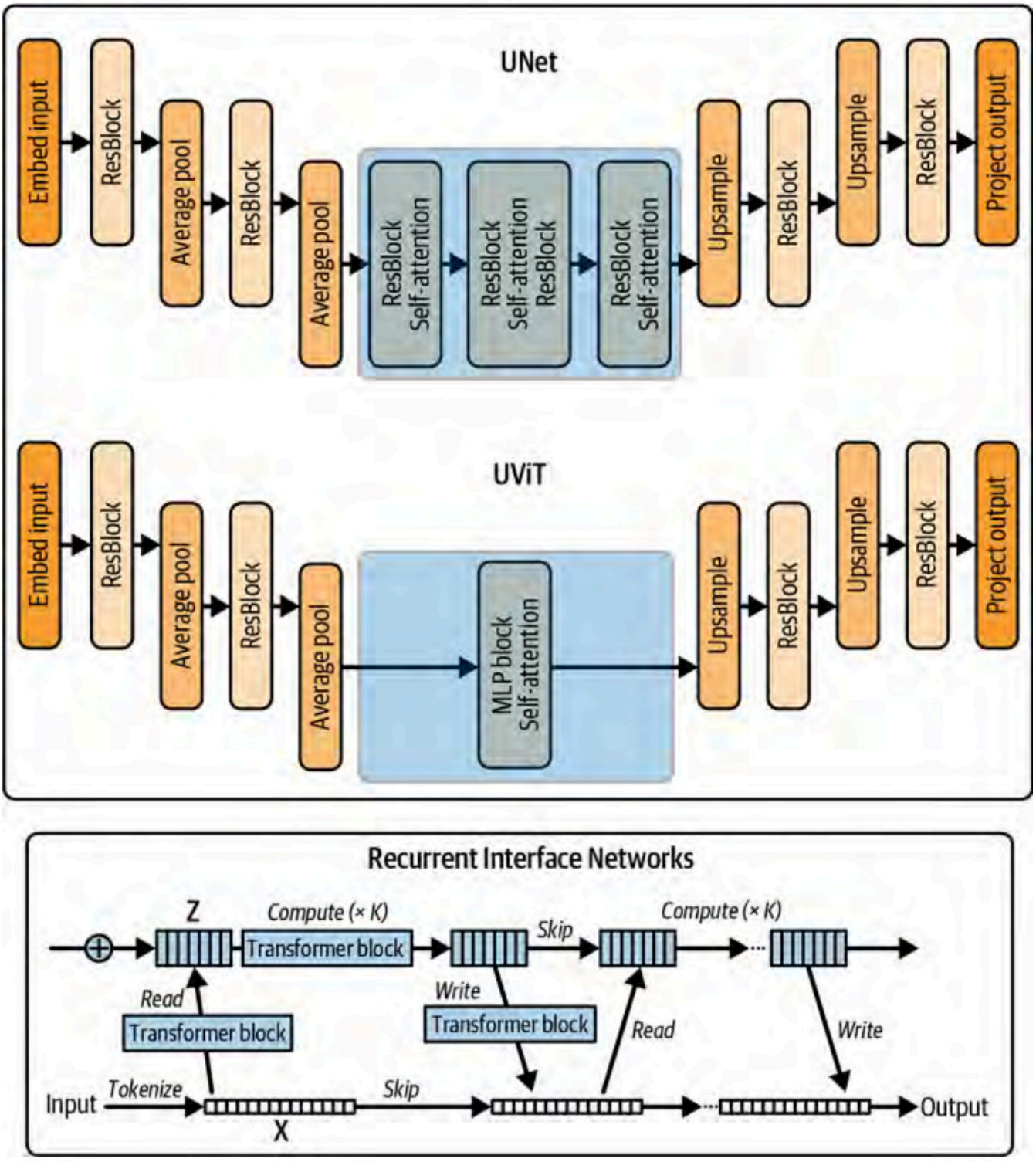



图4-10. UNet与UViT和RIN的比较

这些架构包括以下内容：

Transformers

《Diffusion Transformers》论文表明，基于Transformer的架构可以训练出效果极佳的扩散模型。然而，Transformer架构的计算和内存需求对于非常高分辨率仍然是一个挑战。

UViT

《Simple Diffusion》论文中的UViT架构旨在通过将UNet的中间层替换为大量Transformer块来获得两全其美的效果。该论文的一个关键见解是，将大部分计算集中在UNet的低分辨率块上，可以更高效地训练高分辨率扩散模型。对于非常高的分辨率，他们使用称为小波变换的预处理方法来降低输入图像的空间分辨率，同时通过额外的通道尽可能保留信息，从而减少在高空间分辨率上的计算量。

Recurrent Interface Networks (RINs)

RIN论文采用了类似的方法，首先将高分辨率输入映射到更易处理的低维潜在表示，然后通过一堆Transformer块进行处理，最后解码回图像。此外，RIN论文引入了循环的概念，其中信息从先前的处理步骤传递给模型。这可以有益于扩散模型设计的迭代改进。

一些高质量的扩散Transformer模型包括Flux、Stable Diffusion 3、PixArt-Σ以及文本到视频的Sora。目前尚不清楚基于Transformer的方法是否会完全取代UNet作为扩散模型的首选架构，或者像UViT和RIN架构这样的混合方法是否会更有效。

# 深入探讨：扩散目标

我们已经讨论了扩散模型如何接收噪声输入并学习去噪。乍一看，你可能会认为网络的自然预测目标是图像的去噪版本，我们称之为x0。然而，我们在代码中将模型预测与用于创建噪声版本的单位方差噪声（通常称为 epsilon 目标，eps）进行了比较。两者在数学上看起来是相同的，因为如果我们知道噪声和时间步长，我们可以推导出x0，反之亦然。虽然这是事实，但目标选择在不同时间步长下对损失的大小有一些微妙的影响，从而影响模型在哪些噪声水平下去噪效果最好。预测噪声对模型来说比直接预测目标数据更容易。这是因为噪声在每一步都遵循已知的分布，预测两个步骤之间的差异通常比预测目标数据的绝对值更简单。

为了获得一些直观的理解，让我们可视化不同时间步长下的不同目标。图 4-11 中的输入图像和随机噪声是相同的（图中的前两行），但第三行中的噪声图像根据时间步长添加了不同量的噪声。

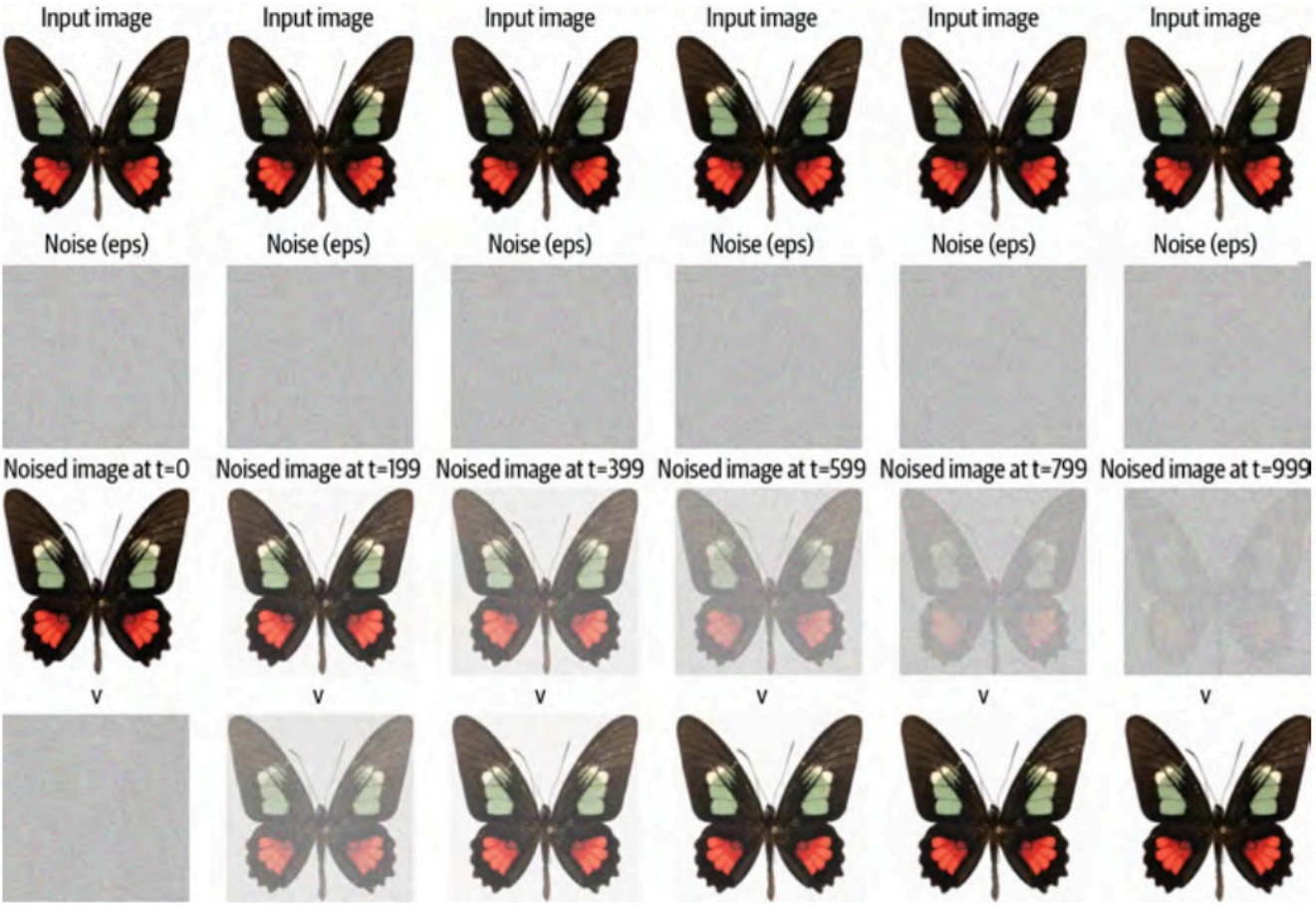



图 4-11. 比较 eps、x0 和 v 目标：eps 尝试预测每个时间步长添加的噪声，x0 预测去噪图像，v 使用两者的混合

在极低噪声水平下，x0目标非常简单（噪声图像几乎与输入相同），而准确预测噪声几乎是不可能的。同样，在极高噪声水平下，eps 目标非常简单（噪声图像几乎等于添加的纯噪声），而准确预测去噪图像几乎是不可能的。如果我们使用x0目标，我们的训练将减少对较低噪声水平的权重。

这两种情况都不理想，因此引入了额外的目标，使模型在不同时间步长下预测x0和eps的混合。图中最后一行所示的速度（v）目标就是其中之一，其定义为 $v = \sqrt{\overline{\alpha}} \cdot \epsilon + \sqrt{1 - \overline{\alpha}} \cdot x_0$。eps 目标仍然是最受欢迎的方法之一，但了解其缺点和其他目标的存在也很重要。

> NVIDIA 的一组研究人员致力于将扩散模型的不同公式统一到一个一致的框架中，并明确分离设计选择。这使得他们能够识别采样和训练过程中的变化，从而获得更好的性能，最终形成了所谓的k-diffusion。如果你有兴趣了解更多关于扩散模型公式的不同目标、缩放和细微差别，我们建议阅读 EDM 论文以进行更深入的讨论。

## 项目时间：训练你的扩散模型

好了，理论已经讲得够多了。现在是时候训练你的无条件扩散模型了。和之前一样，你将训练一个模型来生成新图像。这个项目的主要挑战是创建或找到一个可以用于此的良好数据集。

如果你想使用现有数据集，一个很好的起点是在 Hugging Face Hub 上筛选图像分类数据集，并选择一个你喜欢的。你将需要回答的主要问题之一是，你想使用数据集的哪一部分进行训练。你会像以前一样使用整个数据集，以便模型生成数字吗？还是你会使用特定类别（例如，猫，以便我们得到一个猫专家模型）？或者你会使用数据集的子集（例如，仅使用具有特定分辨率的图像）？

如果你想上传一个新数据集，第一步将是找到并访问数据。要共享数据集，最直接的方法是使用 datasets 库的 ImageFolder 功能。然后你可以将数据集上传到 Hugging Face Hub 并在你的项目中使用它。

一旦你有了数据，考虑一下预处理步骤、模型定义和训练循环。你可以使用本章的代码作为起点，并根据你的数据集进行修改。

## 总结

我们本章开始时使用高级管道运行扩散模型的推理。最终我们从零开始训练了我们的扩散模型，并深入探讨了每个组件。让我们做一个简短的回顾。

目标是训练一个模型，通常是 UNet，它接收噪声图像作为输入，并可以预测该图像的噪声部分。在训练我们的模型时，我们根据随机的时间步长以不同的幅度添加噪声。我们看到的挑战之一是，要在高步数（例如 900）下添加噪声，我们需要进行大量的噪声迭代。为了解决这个问

题，我们使用重参数化技巧，它允许我们直接获得特定时间步长的噪声输入。模型被训练为最小化噪声预测与实际输入噪声之间的差异。对于推理，我们进行一个迭代的细化过程，模型在其中细化初始的随机输入。我们不是保留单个扩散步骤的最终预测，而是根据该预测的方向对输入x进行小幅度的迭代修改。当然，这是使用扩散模型进行推理往往较慢的原因之一，也是与 GAN 等模型相比的主要缺点之一。

扩散领域发展迅速，因此存在许多进展（例如，调度器、模型、训练技术等）。本章重点介绍了基础内容，这些内容将使我们能够跳转到条件生成（例如，根据输入提示生成图像），并为你深入探索扩散领域提供背景知识。本章中的一些阅读材料可以帮助你更深入地了解。

对于额外的阅读材料，我们建议查看以下内容：

- “Annotated Diffusion Model” 博客文章，它对 DDPM 论文进行了技术性解读
- Lilian Weng 的文章，非常适合深入探讨数学
- “Denoising Diffusion Probabilistic Models” 论文本身
- Karras 关于统一扩散模型公式的工作
- “Simple Diffusion: End-to-End Diffusion for High Resolution Images”，它解释了如何为不同尺寸调整采样计划

# 练习

1. 解释扩散推理算法。
2. 噪声调度器的作用是什么？
3. 在创建图像训练数据集时，哪些特征需要注意？
4. 为什么我们要随机翻转训练图像？
5. 我们如何评估扩散模型的生成结果？
6. beta_end 的值如何影响扩散过程？
7. 为什么我们使用 UNet 而不是 VAE 作为扩散的主要模型？
8. 将来自 transformer 的技术（如注意力层或基于 transformer 的架构）引入扩散模型时，面临哪些好处和挑战？

你可以在本书的 GitHub 仓库中找到这些练习和以下挑战的解决方案。

# 挑战

1. 重参数化技巧。证明

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}$$

等价于

$$\mathbf{x}_t = \sqrt{\overline{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\epsilon$$

请注意，这不是一个简单的例子，也不是使用扩散模型所必需的。我们建议查看“A Beginner’s Guide to Diffusion Models: Understanding the Basics and Beyond”以获取指导。重要的是要知道如何合并两个高斯分布：如果你有两个具有不同方差的高斯分布，$\mathcal{N}\left(\mu_1, \sigma_1^2\right)$和$\mathcal{N}\left(\mu_2, \sigma_2^2\right)$，合并后的高斯分布是$\mathcal{N}\left(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2\right)$。

2. DDIM 调度器。本章使用了 DDPM 调度器，有时需要数百或数千步才能获得高质量的结果。最近的研究探索了用尽可能少的步骤（甚至一到两步）实现良好的生成效果。diffusers 库包含多个调度器，例如来自“Denoising Diffusion Implicit Models”论文的 DDIMScheduler。使用 DDIMScheduler 创建一些图像。本章的采样部分使用 DDPMScheduler 需要 1,000 步。你需要多少步才能生成质量相似的图像？尝试将调度器切换为 google/ddpm-celebahq-256 并比较两个调度器。

# 参考文献


Bansal, Arpit, 等. “Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise.” arXiv, 2022年8月19日. http://arxiv.org/abs/2208.09392.

Chen, Ting. “On the Importance of Noise Scheduling for Diffusion Models.” arXiv, 2023年5月21日. http://arxiv.org/abs/2301.10972.

Ho, Jonathan, 等. “Denoising Diffusion Probabilistic Models.” arXiv, 2020年12月16日. http://arxiv.org/abs/2006.11239.

Hoogeboom, Emiel, 等. “Simple Diffusion: End-to-End Diffusion for High Resolution Images.” arXiv, 2023年12月12日. http://arxiv.org/abs/2301.11093.

Jabri, Allan, 等. “Scalable Adaptive Computation for Iterative Generation.” arXiv, 2023年6月13日. http://arxiv.org/abs/2212.11972.

Karras, Tero, 等. “Elucidating the Design Space of Diffusion-Based Generative Models.” arXiv, 2022年10月11日. http://arxiv.org/abs/2206.00364.

Lee, Tony, 等. “Holistic Evaluation of Text-to-Image Models.” arXiv, 2023年11月7日. https://arxiv.org/abs/2311.04287.

Lin, Shanchuan, 等. “Common Diffusion Noise Schedules and Sample Steps Are Flawed.” arXiv, 2024年1月23日. https://arxiv.org/abs/2305.08891.

Peebles, William, 和 Saining Xie. “Scalable Diffusion Models with Transformers.” arXiv, 2023年3月2日. http://arxiv.org/abs/2212.09748.

Rogge, Niels, 和 Kashif Rasul. “The Annotated Diffusion Model.” Hugging Face 博客, 2022年6

月7日. https://oreil.ly/mFHxe.
Ronneberger, Olaf, 等. “U-Net: Convolutional Networks for Biomedical Image Segmentation.” arXiv, 2015年5月18日. http://arxiv.org/abs/1505.04597.
Song, Jiaming, 等. “Denoising Diffusion Implicit Models.” arXiv, 2022年10月5日. https://arxiv.org/abs/2010.02502.

# 第5章. Stable Diffusion 与条件生成

在上一章中，我们介绍了扩散模型及其背后的迭代优化思想。到章节结束时，我们已经可以生成图像，但训练模型非常耗时，并且我们无法控制生成的图像。在本章中，我们将探讨如何从这一点过渡到基于文本描述的高效生成图像的文本条件模型，并以 Stable Diffusion 模型为例进行案例研究。不过，在深入 Stable Diffusion 之前，我们将先了解条件模型的工作原理，并回顾一些导致我们今天所拥有的文本到图像模型的创新。

## 添加控制：条件扩散模型

在我们解决从文本描述生成图像的挑战之前，让我们从稍微简单一些的内容开始。我们将探讨如何将模型输出引导到特定类型或类别的图像。我们可以使用一种称为条件化的方法，其核心思想是要求模型生成的不是任意图像，而是属于预定义类别的图像。在这种情况下，条件化指的是通过在生成过程中提供额外信息（如标签或提示）来引导模型的输出。

模型条件化是一个简单但有效的概念。我们将从第4章中使用的扩散模型开始，并引入一些变化。首先，我们将不再使用蝴蝶数据集，而是切换到一个有类别的数据集。我们将使用 Fashion MNIST，这是一个包含数千张服装图像的数据集，每张图像都与10个类别中的某个标签相关联。然后，关键的是，我们将为模型提供两个输入：(1) 图像，和之前一样，以及 (2) 每张图像所属的类别标签。通过这样做，我们期望模型能够学习图像与标签之间的关联，帮助它理解毛衣、靴子和其他服装项目的独特特征。

请注意，我们并不关心解决分类问题——我们不希望模型告诉我们图像属于哪个类别。我们仍然希望它执行与第4章相同的任务：请生成看起来像是来自该数据集的合理图像。唯一的区别是我们为它提供了关于这些图像的额外信息。我们将使用相同的损失函数和训练策略，因为这与之前的任务相同。

### 准备数据

我们需要一个具有不同图像组的数据集。用于计算机视觉分类任务的数据集非常适合此目的。我

们可以从像 ImageNet 这样的数据集开始，它包含数百万张图像，涵盖1,000个类别。然而，在这个数据集上训练模型将花费极长的时间。在解决新问题时，从一个较小的数据集开始是一个好主意，以确保一切按预期工作。这可以保持反馈循环的简短，以便我们能够快速迭代并确保我们走在正确的轨道上。

我们可以选择 MNIST 作为此示例，就像在第4章中一样。为了使事情稍微不同，我们将选择 Fashion MNIST。Fashion MNIST 由 Zalando 开发并开源，是 MNIST 的替代品，具有相似的特征：紧凑的尺寸、黑白图像和10个类别。主要区别在于类别对应于不同类型的服装，而不是数字，并且图像包含比简单手写数字更多的细节。

就像在第3章中一样，我们需要配置 matplotlib 以使用反转的灰度颜色来匹配 Fashion MNIST 数据集。让我们看一些示例：

```
import matplotlib as mpl
from datasets import load_dataset
from genaibook.core import show_images
mpl.rcParams["image.cmap"] = "gray_r"
fashion_mnist = load_dataset("fashion_mnist")
clothes = fashion_mnist["train"]["image"][:8]
classes = fashion_mnist["train"]["label"][:8]
show_images(clothes, titles=classes, figsize=(4, 2.5))
```

因此，类别0对应T恤，2是毛衣，9是靴子。[1] 我们准备数据集和 DataLoader 的方式与第4章类似，主要区别在于我们还将类别信息作为输入包含在内。与第4章中的调整大小不同，我们将图像输入从 $28 \times 28$ 像素填充到 $32 \times 32$ 像素。这将保留原始图像质量，有助于 UNet 做出更高

质量的预测。[2] 填充有助于避免操作可能裁剪或扭曲边缘信息的问题：

```
import torch
from torchvision import transforms

preprocess = transforms.Compose(
    [
        transforms.RandomHorizontalFlip(),  # 随机翻转（数据增强）
        transforms.ToTensor(),  # 转换为张量 (0, 1)
        transforms.Pad(2),  # 在所有边添加2像素
        transforms.Normalize([0.5], [0.5]),  # 映射到 (-1, 1)
    ]
)


def transform(examples):
    images = [preprocess(image) for image in examples["image"]]
    return {"images": images, "labels": examples["label"]}


train_dataset = fashion_mnist["train"].with_transform(transform)

train_dataloader = torch.utils.data.DataLoader(
    train_dataset, batch_size=256, shuffle=True
)
```

- 定义一系列将应用于数据集中图像的转换（翻转、转换为张量、填充和归一化）。
- 使用转换处理一批图像。
- 返回包含处理后的图像及其对应标签的字典。
- 加载数据集的训练部分。通过使用 with_transform，确保在应用转换后返回项目。
- 创建一个 DataLoader，它将构建批次并打乱数据，简化我们的代码。

## 创建类条件模型

diffusers 库中的 UNet 允许提供自定义的条件信息。在这里，我们创建了一个类似于第 4 章中使用的模型，但我们在 UNet 构造函数中添加了一个 `num_class_embeds` 参数。这个参数告诉模型我们希望使用类标签作为额外的条件。我们将使用 10，因为这是 Fashion MNIST 中的类别数量：

```
from diffusers import UNet2DModel

model = UNet2DModel(
    in_channels=1,  # 1 通道用于灰度图像
    out_channels=1,
    sample_size=32,
    block_out_channels=(32, 64, 128, 256),
    num_class_embeds=10,  # 启用类条件
)
```

要使用此模型进行预测，我们必须将类标签作为额外输入传递给 `forward()` 方法：

```
x = torch.randn((1, 1, 32, 32))
with torch.inference_mode():
    out = model(x, timestep=7, class_labels=torch.tensor([2])).sample
out.shape
torch.Size([1, 1, 32, 32])
```

> 我们还向模型传递了其他条件信息：时间步长。没错，即使是第 4 章中的模型也可以被视为条件扩散模型。我们根据时间步长对其进行条件化，期望知道我们在扩散过程中的位置将有助于生成更逼真的图像。

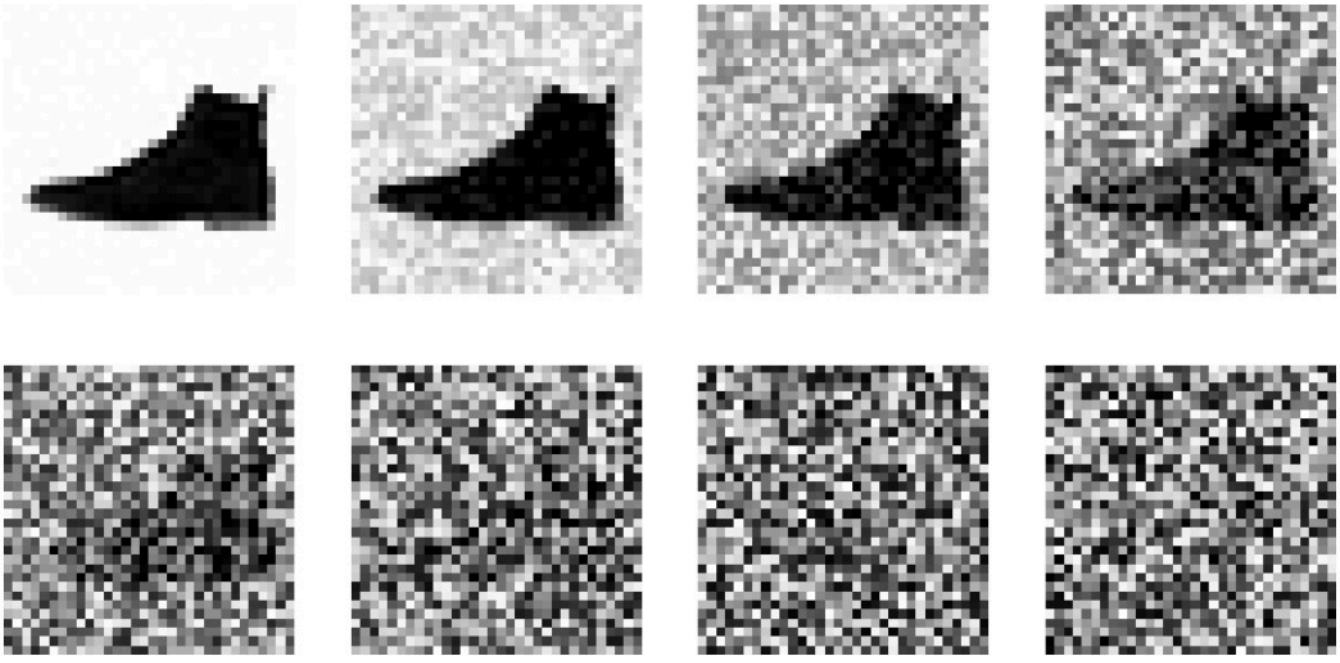
在内部，时间步长和类标签被转换为嵌入，模型在前向传播过程中使用这些嵌入。在 UNet 的多个阶段中，这些嵌入被投影到与给定层中通道数相匹配的维度。然后，这些嵌入被添加到该层的输出中。这意味着条件信息被馈送到 UNet 的每个块中，使模型有充分的机会学习如何有效地使用它。

嵌入在扩散模型中非常有效，因为它们提供了条件信息的紧凑且密集的表示，例如时间步长和类标签，模型可以轻松地将其集成到 UNet 架构中。嵌入的灵活性还允许有效处理不同类型的条件输入，无论是连续的（如时间步长）、分类的（如类标签）还是基于文本的（如提示）。

## 训练模型

添加噪声在灰度图像上的效果与第 4 章中的蝴蝶图像一样好。让我们看看随着噪声时间步长的增加，噪声的影响：

```
from diffusers import DDPMScheduler
scheduler = DDPMScheduler( num_train_timesteps=1000, beta_start=0.001,
    beta_end=0.02
)
timesteps = torch.linspace(0, 999, 8).long()
batch = next(iter(train_dataloader))
// 我们从数据集中加载 8 张图像并
// 向它们添加越来越多的噪声
x = batch["images"][0].expand([8, 1, 32, 32])
noise = torch.rand_like(x)
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0, 1))
```



我们的训练循环几乎与第 4 章中的相同，只是我们现在传递类标签作为条件。请注意，这只是模型的额外信息，但它不会以任何方式影响我们定义损失函数的方式。我们还将使用 `tqdm` 在训练期间显示进度条。

这是一个启动训练并喝杯茶、咖啡或你喜欢的饮料的好时机。不要被以下代码吓到：它与我们在无条件生成中所做的类似。我们强烈建议将此代码与前一章中的代码并排查看。你能找到所有不

同之处吗？[3]

该脚本执行以下操作：

1. 加载一批图像及其对应的标签（使用 `train_dataloader` 和 `tqdm` 进行迭代）
2. 根据时间步长向图像添加噪声（使用 `scheduler.add_noise()`）
3. 将噪声图像与类标签一起馈送到模型中（使用 `model()`）
4. 计算损失
5. 反向传播损失并使用优化器更新模型权重

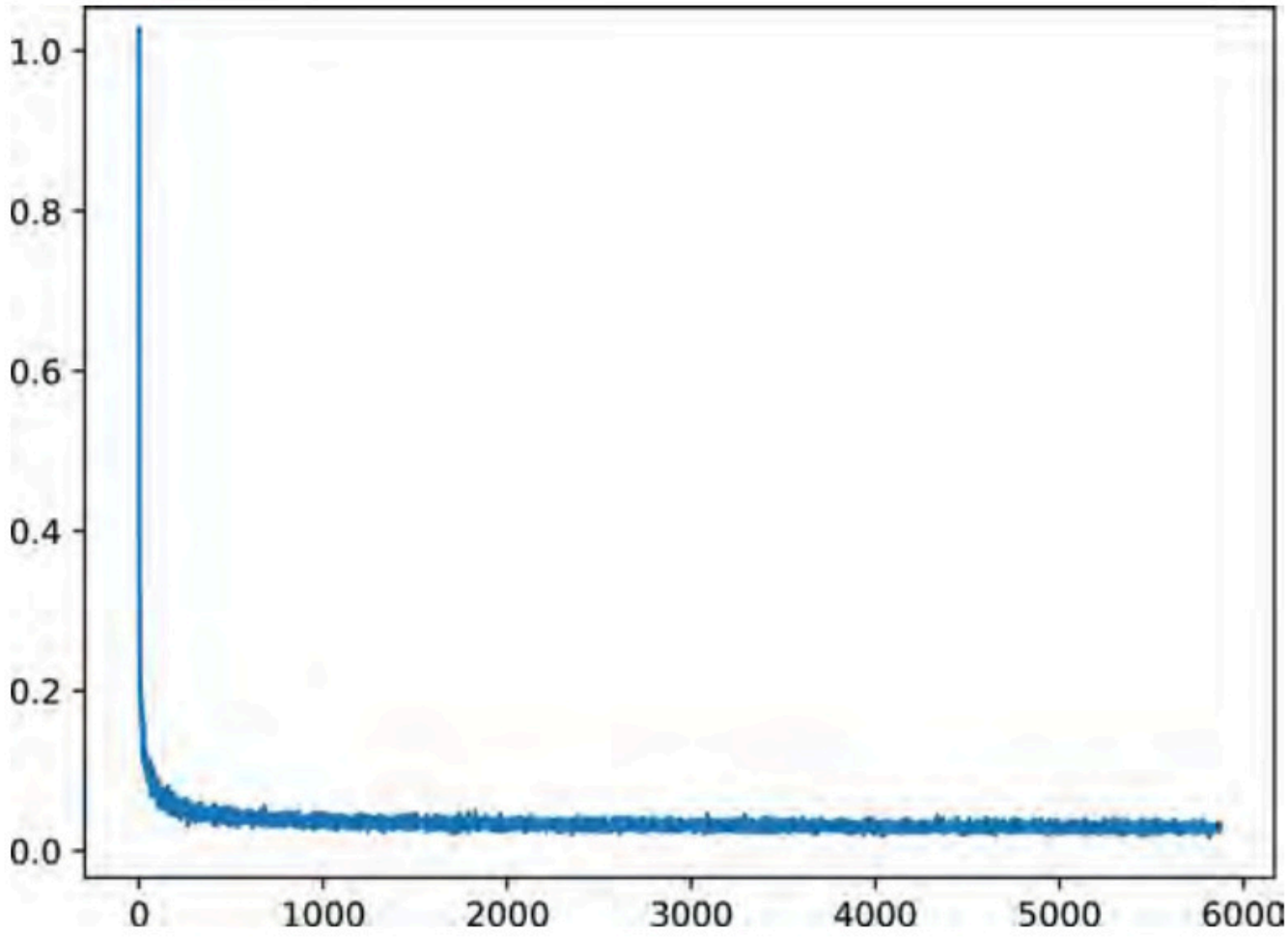
```
from torch.nn import functional as F
from tqdm import tqdm

from genaibook.core import get_device

# 初始化调度器
scheduler = DDPMScheduler(
    num_train_timesteps=1000, beta_start=0.0001, beta_end=0.02
)

num_epochs = 25
lr = 3e-4
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)
losses = []  # 用于存储损失值以进行绘图

device = get_device()
model = model.to(device)

# 训练模型（这需要一些时间！）
for epoch in (progress := tqdm(range(num_epochs))):
    for step, batch in (
        inner := tqdm(
            enumerate(train_dataloader),
            position=0,
            leave=True,
            total=len(train_dataloader),
        )
    ):
        # 加载输入图像和类别
        clean_images = batch["images"].to(device)
        class_labels = batch["labels"].to(device)

        # 采样噪声以添加到图像中
        noise = torch.randn(clean_images.shape).to(device)

        # 为每张图像采样一个随机时间步长
        timesteps = torch.randint(
            0,
            scheduler.config.num_train_timesteps,
```

```
            (clean_images.shape[0],),
            device=device,
        ).long()

        # 根据每个时间步长的噪声幅度向干净图像添加噪声
        noisy_images = scheduler.add_noise(clean_images, noise, timesteps)

        # 获取模型对噪声的预测
        # 注意使用class_labels
        noise_pred = model(
            noisy_images,
            timesteps,
            class_labels=class_labels,
            return_dict=False,
        )[0]

        # 将预测与实际噪声进行比较
        loss = F.mse_loss(noise_pred, noise)

        # 更新损失显示
        inner.set_postfix(loss=f"{loss.cpu().item():.3f}")

        # 存储损失以便稍后绘图
        losses.append(loss.item())

        # 反向传播和优化
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

训练完成后，我们可以绘制训练损失图以查看模型的表现：

```
import matplotlib.pyplot as plt
plt.plot(losses)
```

```
t(losses)
```



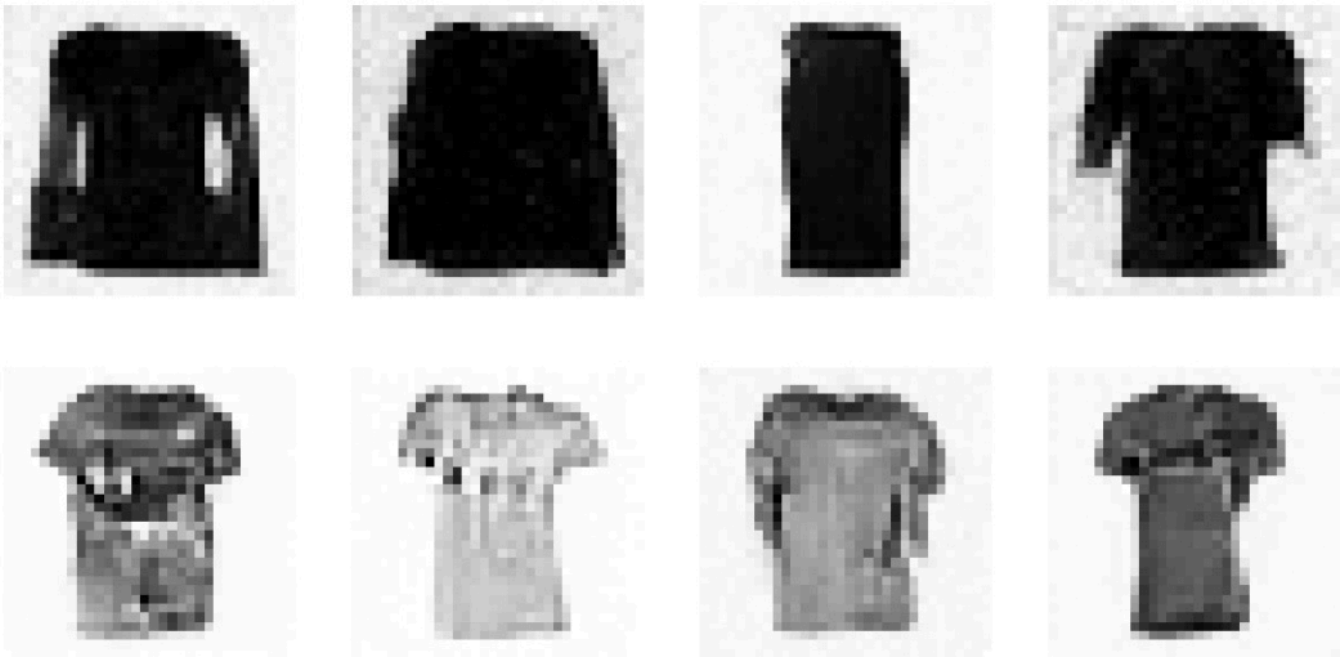

## 采样

我们现在有一个在预测时期望两个输入的模型：图像和类标签。我们可以从随机噪声开始，然后迭代去噪，传递我们想要生成的类标签来创建样本：

```
def generate_from_class(class_to_generate, n_samples=8):
    sample = torch.randn(n_samples, 1, 32, 32).to(device)
    class_labels = [class_to_generate] * n_samples
    class_labels = torch.tensor(class_labels).to(device)

    for _, t in tqdm(enumerate(scheduler.timesteps)):
        # 获取模型预测
        with torch.inference_mode():
            noise_pred = model(sample, t, class_labels=class_labels).sample

        # 更新样本
        sample = scheduler.step(noise_pred, t, sample).prev_sample

    return sample.clip(-1, 1) * 0.5 + 0.5
```

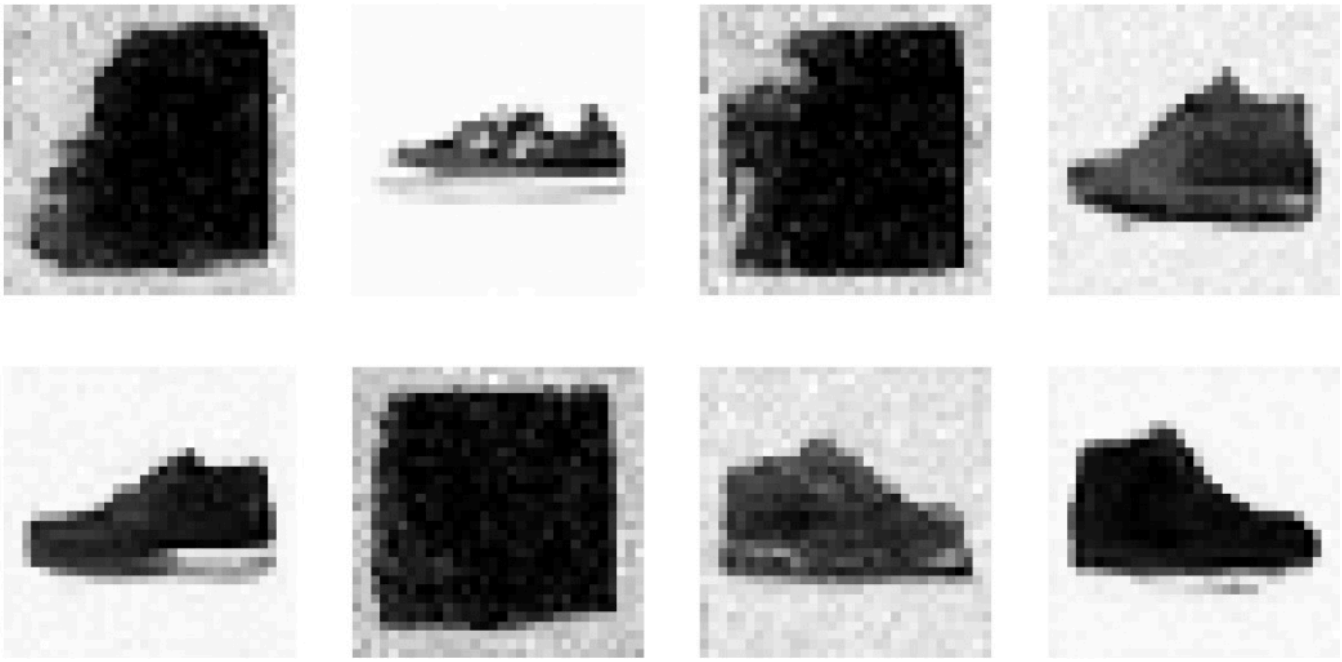
我们可以生成一些 T 恤（类别 0）：

```
images = generate_from_class(0)
show_images(images, nrows=2)
```



现在，我们可以生成一些运动鞋（类别 7）：

```
images = generate_from_class(7)
show_images(images, nrows=2)
```



或者，最后，我们可以生成一些靴子（类别 9）：

```
images = generate_from_class(9)
show_images(images, nrows=2)
```

如你所见，生成的图像仍然包含一些噪声。如果我们探索模型架构、进行超参数调整并训练更长时间，它们可能会变得更好。无论如何，令人惊讶的是，模型学会了不同类型服装的形状，并意识到形状 9 与形状 0 不同，仅仅是通过在训练数据中传递这些信息。换句话说，模型习惯于看到数字 9 伴随着靴子。当我们要求它生成图像并提供 9 时，它会生成一只靴子。

## 提高效率：潜在扩散

既然我们可以训练一个条件模型，我们只需要将其扩展并用文本而不是类别标签作为条件……对吗？对吗？嗯，不完全是这样。随着图像尺寸的增长，处理这些图像所需的计算能力也会增加。这在自注意力机制中尤为明显，其中操作的数量随着输入数量的增加呈二次方增长。一张 $128 \times 128$ 的图像比一张 $64 \times 64$ 的图像多四倍的像素，在自注意力层中需要 16 倍的内存和计算量。这对于任何想要生成高分辨率图像的人来说都是一个难题。

潜在扩散试图通过使用一个单独的变分自编码器（VAE，图 5-1）来缓解这个问题。正如我们在第 2 章中看到的，VAE 可以将图像压缩到更小的空间维度。其基本原理是图像往往包含大量冗余信息。给定足够的训练数据，VAE 可以学习生成输入图像的更小表示，然后基于这个小的潜在表示以高保真度重建图像。Stable Diffusion 中使用的 VAE 接收三通道图像并生成四通道的潜在表示，每个空间维度的压缩因子为 8。一张 $512 \times 512$ 的输入图像 ($3 \times 512 \times 512 = 786{,}432$ 个值) 将被压缩为一个 $4 \times 64 \times 64$ 的潜在表示（16,384 个值）。

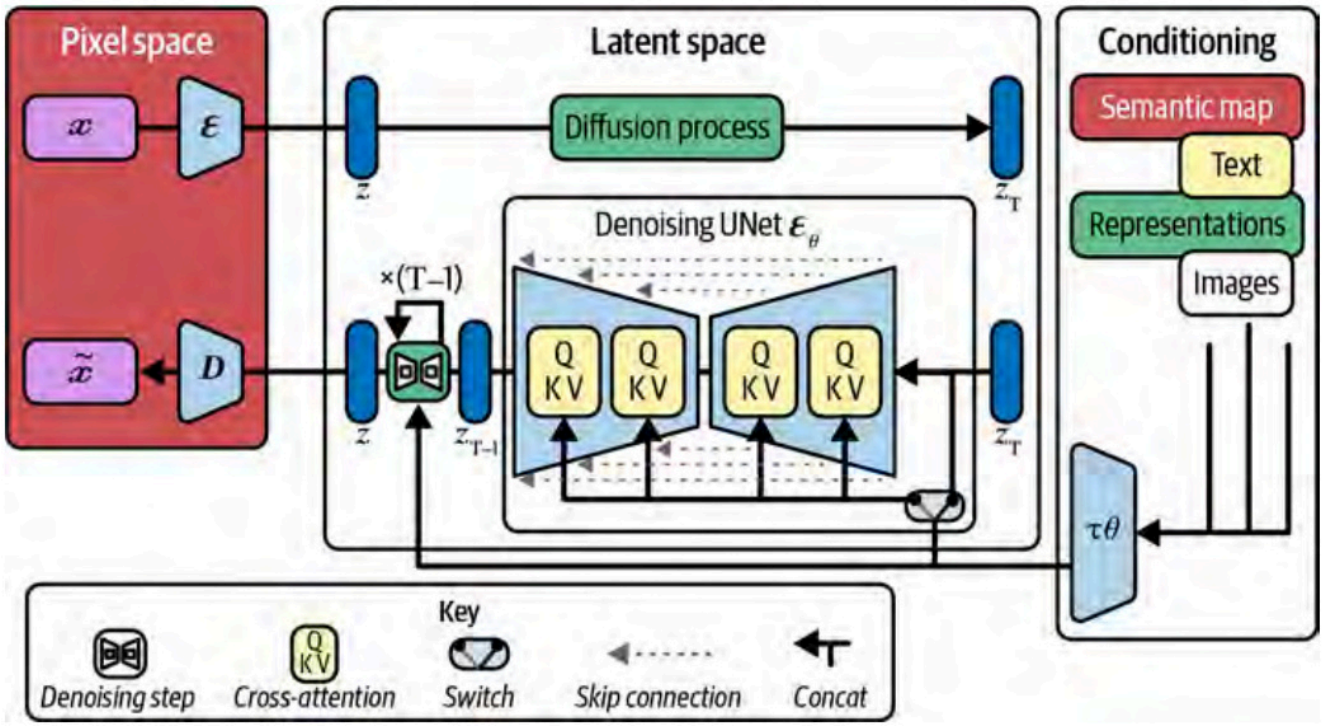



图 5-1. 潜在扩散过程。注意左侧的 VAE 编码器和解码器，用于在像素空间和潜在空间之间进行

转换。

通过在这些较小的潜在表示上应用扩散过程，而不是在全分辨率图像上，我们可以获得使用较小图像的许多好处（更低的内存使用、UNet 中需要的层数更少、生成时间更快等），并且一旦我们准备好查看结果，仍然可以将其解码回高分辨率图像。这一创新大大降低了训练和运行这些模型的成本。提出这一想法的论文《潜在扩散模型》通过训练以分割图、类别标签和文本为条件的模型展示了这种技术的强大能力。令人印象深刻的结果促使作者与 LAION、StabilityAI、RunwayML 和 EleutherAI 等合作伙伴进一步合作，训练了一个更强大的模型版本，即 Stable Diffusion。

## Stable Diffusion：深入剖析组件

Stable Diffusion 是一种潜在扩散模型，可以根据文本提示生成图像。它还可以用于许多其他事情，例如修改图像，你将在第 9 章中了解更多。

由于其受欢迎程度，数百个网站和应用程序允许你在不需要技术知识的情况下使用它来创建图像。它也得到了像 diffusers 这样的库的很好支持，这些库让我们可以使用用户友好的管道来采样图像，就像我们在第 1 章中所做的那样：

```
from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    variant="fp16",
).to(device)
pipe("Watercolor illustration of a rose").images[0]
```

本节将探讨实现这一目标的所有组件。

> 正如 diffusers 有 StableDiffusionPipeline 一样，它还提供了对来自不同家族的数十个其他

> 模型（Würstchen、AuraFlow、Flux 等）的访问。此外，它还提供了用于其他任务的类似管道，例如修复（StableDiffusionInpaint）和超分辨率（StableDiffusionLatentUpscale）。第 8 章将进一步探讨其中一些模型和管道。我们将首先专注于理解整个 Stable Diffusion 管道。

## 文本编码器

那么，Stable Diffusion 是如何理解文本的呢？早些时候，我们解释了如何向 UNet 提供额外信息，以便我们能够对生成的图像类型进行一些控制。给定一个噪声版本的图像，模型的任务是根据额外的线索（如类别标签）预测去噪版本。在 Stable Diffusion 的情况下，额外的线索是文本提示。在推理时，我们输入我们想要生成的图像的描述和一些纯噪声作为起点，模型会尽力将随机输入去噪为与描述匹配的内容。

为了实现这一点，我们需要创建一个文本的数字表示，以捕捉其描述的相关信息。为此，我们将使用一个文本编码器，将输入字符串转换为文本嵌入，然后将其与时间步长和噪声潜在表示一起输入到 UNet 中，如图 5-2 所示。

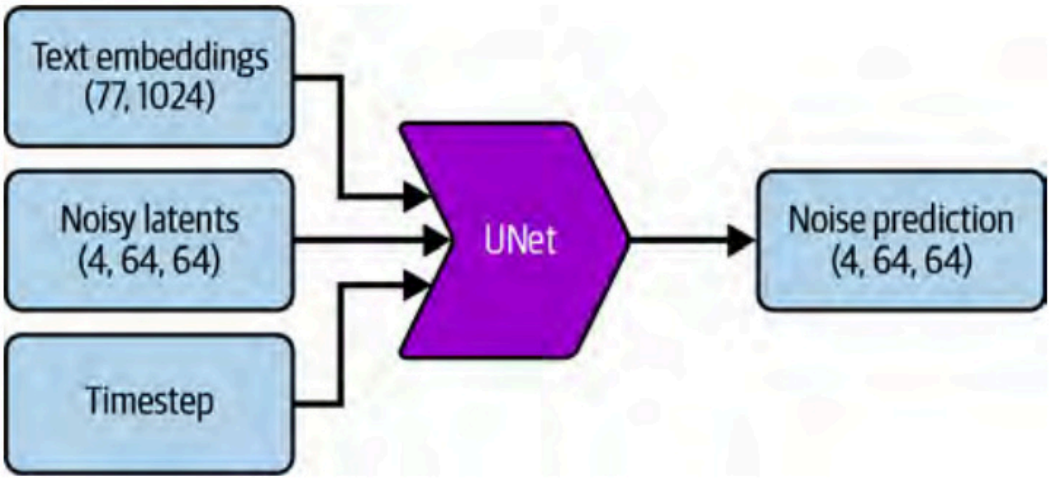



图 5-2. UNet 可以以多个输入为条件，例如时间步长、类别标签或文本嵌入

为此，Stable Diffusion 利用了基于 CLIP 的预训练 transformer 模型，该模型在第 3 章中介绍。文本编码器是一个 transformer 模型，它接收一系列标记并为每个标记生成一个向量，如图 5-3 所示。在 Stable Diffusion 的第一个版本（Stable Diffusion 1 到 1.5）中，他们使用了 OpenAI 的原始 CLIP，文本编码器映射到 768 维向量。由于 CLIP 的原始数据集未知，社区训练了一个开源版本，称为 OpenCLIP。Stable Diffusion 2 使用了 OpenCLIP 的文本编码器，它为每个标记生成 1,024 维向量。

我们没有将所有标记的向量组合成一个单一的表示，而是将它们分开，并将它们用作 UNet 的条

件。这使得 UNet 可以分别使用每个标记中的信息，而不仅仅是提示的整体含义。因为我们是从 CLIP 模型的内部表示中提取这些文本嵌入的，所以它们通常被称为编码器隐藏状态。让我们深入了解文本编码器的工作原理。这与我们在第 2 章中讨论的编码器模型的过程相同。

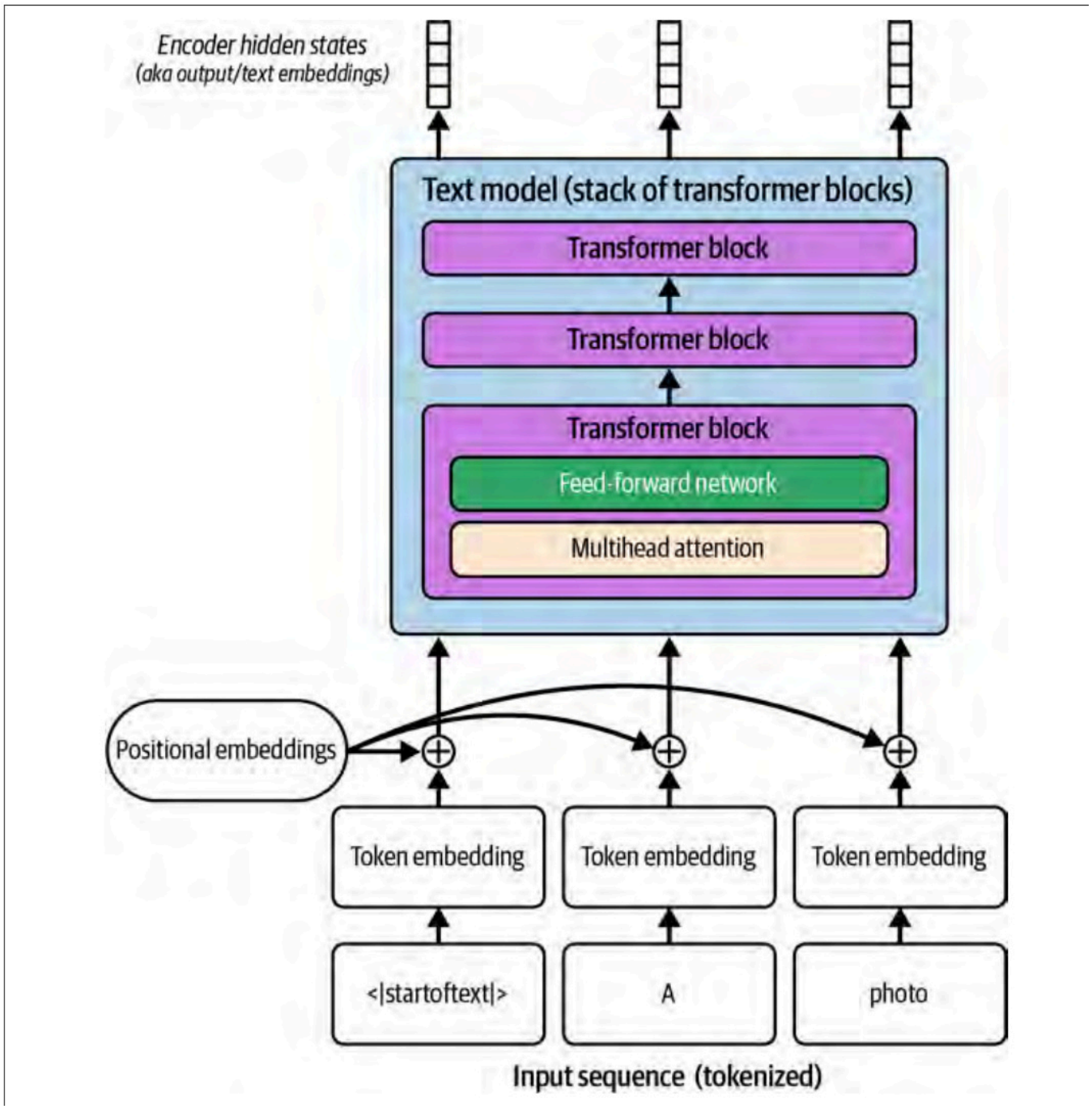



图 5-3. 文本编码过程将输入提示转换为一组文本嵌入（编码器隐藏状态），然后可以作为条件输入到 UNet 中

编码文本的第一步是执行标记化，将字符序列转换为数字序列，正如你在第 2 章中学到的那

样。在下面的示例中，我们看到了如何使用 Stable Diffusion 的标记器对短语进行标记化。提示中的每个标记都被分配了一个唯一的标记编号（例如，“photograph”在标记器的词汇表中恰好是 8853）。还有一些特殊标记提供了额外的上下文，例如句子结束的位置：

```
prompt = "A photograph of a puppy"

# 将文本转换为标记序列:
text_input = pipe.tokenizer(
    prompt,
    return_tensors="pt",
)

# 输出每个标记及其对应的 ID
for t in text_input["input_ids"][0]:
    print(t, pipe.tokenizer.decoder.get(int(t)))

tensor(49406) <|startoftext|>
tensor(320) a</w>
tensor(8853) photograph</w>
tensor(539) of</w>
tensor(320) a</w>
tensor(6829) puppy</w>
tensor(49407) <|endoftext|>
```

一旦文本被标记化，我们就可以将其通过文本编码器，得到最终的文本嵌入，这些嵌入将被输入到 UNet 中：

```
text_embeddings = pipe.text_encoder(text_input.input_ids.to(device))[0]
print("Text embeddings shape:", text_embeddings.shape)
```

Text embeddings shape: torch.Size([1, 77, 768])

## 变分自编码器 (VAE)

VAE 的任务是将图像压缩为更小的潜在表示，并再次重建它们。如图 5-4 所示，VAE 是 Stable Diffusion 模型的关键组成部分，确实令人印象深刻。我们不会在这里深入讨论训练细节，但除了第 3 章中描述的常规重建损失和 KL 散度之外，VAE 还使用了一种基于 patch 的判别器损失，以帮助模型生成逼真的细节和纹理。这有助于避免之前 VAE 中典型的略微模糊的输出。与

文本编码器一样，VAE 通常是单独训练的，并在扩散模型的训练和采样过程中作为冻结组件使用。

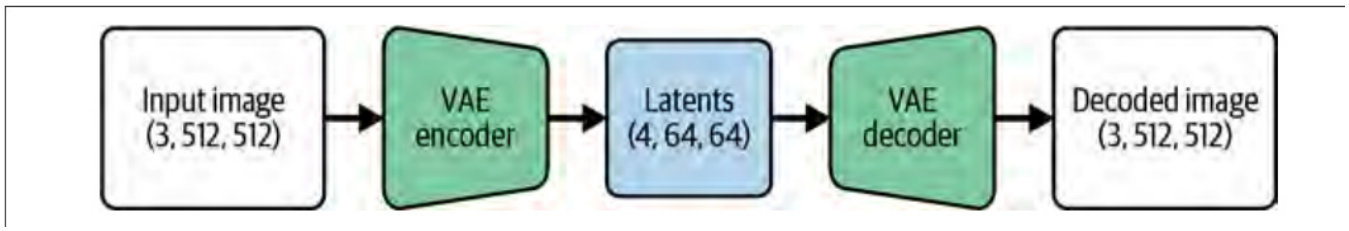



图 5-4. VAE 架构

让我们加载一张图像，看看它经过 VAE 压缩和解压缩后的样子。首先，我们来看看原始图像：

```
from genaibook.core import load_image, show_image, SampleURL

im = load_image( SampleURL.LlamaExample, size=(512, 512),
)
show_image(im);
```

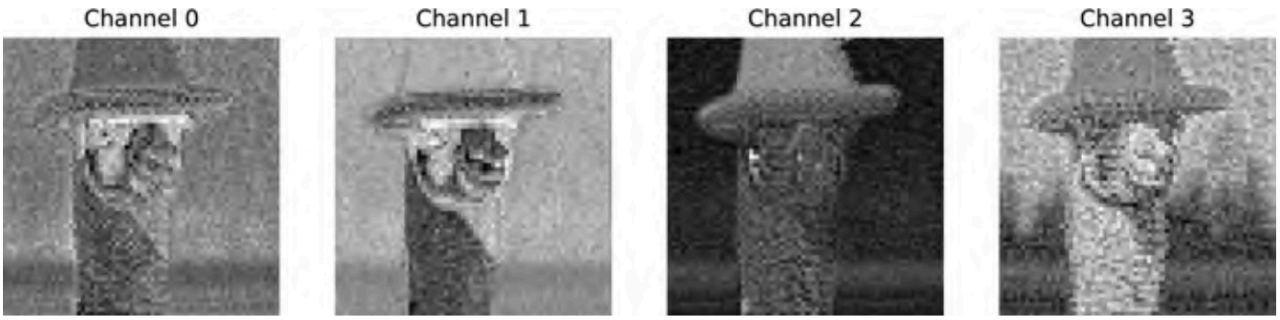
现在，让我们将图像通过 VAE：

```
with torch.inference_mode():
    // 处理图像
    tensor_im = transforms.ToTensor()(im).unsqueeze(0).to(device) * 2 - 1
    tensor_im = tensor_im.half()

    // 编码图像
    latent = pipe.vae.encode(tensor_im)

    // 从潜在分布中采样
    latents = latent.latent_dist.sample()
    latents = latents * 0.18215

latents.shape
torch.Size([1, 4, 64, 64])
```

- 图像被转换为符合 VAE 输入期望的格式。我们将其转换为张量，并使用 `unsqueeze(0)` 添加一个维度，使其成为大小为 1 的批次。最后，我们将图像归一化到范围 [–1, 1] 以匹配 VAE 的输入范围。
- 图像从 float32 转换为 float16，因为我们以 `torch.float16` 精度加载了管道。
- 图像通过 VAE 的编码器。正如第 3 章所讨论的，VAE 输出一个分布，我们可以从中采样。
- 我们可以访问 VAE 编码器生成的分布对象，从中生成一个样本。
- 将潜在向量缩放为固定大小 0.18215。Stable Diffusion 的作者引入了这个 0.18215 的缩放因子，以确保潜在空间的方差接近 1，与扩散过程中添加的噪声的近似尺度匹配。它可以在 `vae.config.scaling_factor` 中访问。

原始图像是一个大小为 $512 \times 512$ (786,432 值) 的三通道图像。VAE 将此图像压缩为一个大小为 $64 \times 64$ (16,384 值) 的四通道潜在表示。我们可以绘制潜在表示中的每个通道：

```
show_images(
    [l for l in latents[0]],
    titles=[f"Channel {i}" for i in range(latents.shape[1])],
    ncols=4,
)
```




现在我们已经将图像编码为潜在表示，我们可以将其解码回图像。在理想情况下，解码后的图像将与原始图像完全相同。实际上，VAE 可能会引入一些噪声和伪影。让我们看看解码后的图像是什么样子：

```
with torch.inference_mode():
    image = pipe.vae.decode(latents / 0.18215).sample

image = (image / 2 + 0.5).clamp(0, 1)
show_image(image[0].float())
```

当从头生成图像时，我们创建一组随机潜在作为起点。我们迭代地优化这些噪声潜在，生成一个样本，然后使用 VAE 解码器将这些最终潜在解码为我们可以查看的图像。编码器仅在我们希望从现有图像开始该过程时使用，这将在第 8 章中探讨。

## UNet

Stable Diffusion 中使用的 UNet 与第 4 章中用于生成图像的 UNet 类似。不同的是，我们输入的是一个四通道潜在，而不是三通道图像。时间步嵌入的输入方式与本章开头的示例中的类别条件相同。但这个 UNet 还需要接受文本嵌入作为额外的条件。UNet 中散布着交叉注意力层。UNet 中的每个空间位置都可以关注文本条件中的不同标记，从提示中引入相关信息。图 5-5 展示了如何在不同点输入文本条件（以及基于时间步的条件）。

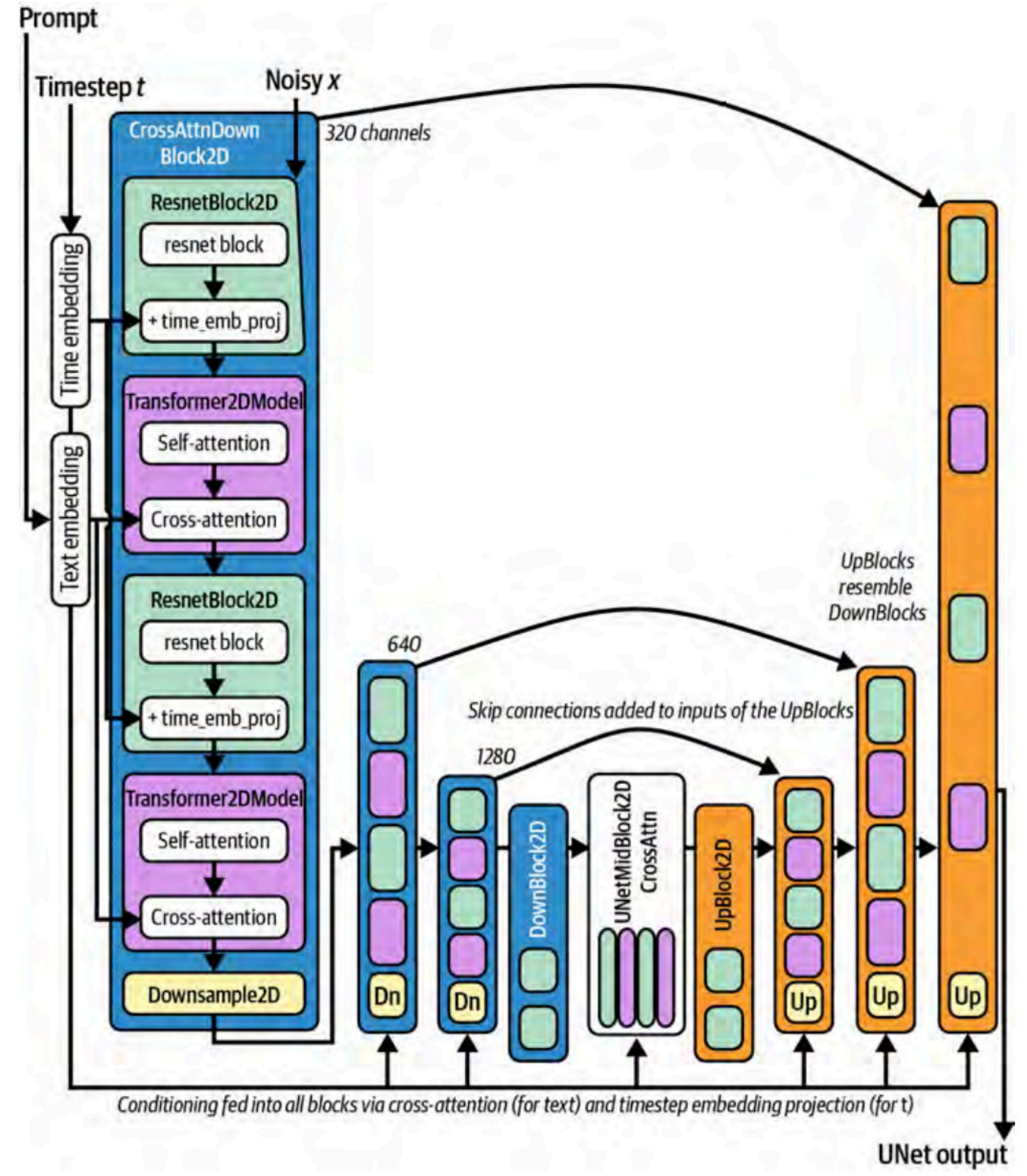



图 5-5. 条件化 UNet 架构

在图 5-5 的左侧，你可以找到模型输入：噪声 $x$，在这种情况下是四通道潜在。时间步和提示也

以嵌入的形式在架构的不同点输入到模型中。与所有之前的 UNet 一样，该模型有一系列层，用于对输入进行下采样，然后将其上采样回原始大小。网络还具有跳跃连接，允许模型访问来自早期块的信息。

Stable Diffusion 版本 1 和 2 的 UNet 大约有 8.6 亿个参数。较新的 Stable Diffusion XL (SDXL) 中的 UNet 参数更多，约为 26 亿，并且使用了额外的条件信息。

## Stable Diffusion XL

2023 年夏季，发布了新版本的 Stable Diffusion：Stable Diffusion XL。它使用了本章描述的相同原理，并在所有系统组件中进行了各种改进。一些最令人兴奋的变化如下：

更大的文本编码器以捕捉更好的提示表示
它使用两个文本编码器的输出并将嵌入连接起来。

条件化一切

除了时间步（携带有关噪声量的信息）和文本嵌入之外，SDXL 还使用以下额外的条件信号：

原始图像大小

在训练集中，小图像几乎占用于训练 SDXL 的总训练数据的 $40\%$，小图像被放大并用于训练，而不是丢弃它们。然而，模型也会接收到有关它接收到的图像大小的信息。这样，它学会了放大伪影不应该是大图像的一部分，并在推理过程中鼓励生成更好的质量。

裁剪坐标

输入图像通常在训练期间随机裁剪，因为批次中的所有图像必须具有相同的大小。随机裁剪可能会产生不良效果，例如切割主题头部或完全从图像中移除主题，即使它们可能在文本提示中描述。在模型训练后，如果我们请求未裁剪的图像（通过将裁剪坐标设置为 $(0, 0)$），模型更有可能生成位于画面中心的主题。

目标宽高比

在初始预训练于方形图像后，SDXL 在各种宽高比上进行了微调，并将原始宽高比的信息用作另一个条件信号。与其他条件情况一样，这使得生成比以前更逼真的风景和肖像图像，且伪影更少。

更大的分辨率

SDXL 设计用于生成分辨率为 $1,024 \times 1,024$ 像素的图像（或总像素数约为 $1,024^2$ 的非方形图像）。与宽高比一样，此功能是在微调阶段实现的。

UNet 大约大了三倍
交叉注意力上下文更大，以适应条件量的增加。

改进的 VAE

它使用与原始 Stable Diffusion 相同的架构，但在更大的批次大小上训练，并使用指数移动平均 (EMA) 技术更新权重。

精炼模型

除了基础模型外，SDXL 还包含一个额外的精炼模型，该模型与基础模型在相同的潜在空间中工作。然而，该模型仅在噪声调度前 $20\%$ 的时间内对高质量图像进行了训练。这意味着它知道如何从带有少量噪声的图像中生成高质量的纹理和细节。

由于原始的 Stable Diffusion 是开源的，其他研究人员和开源社区已经探索了许多这些技术。SDXL 结合了这些想法，实现了图像质量的显著提升（见图 5-6），但运行模型的成本更高，速度更慢，且占用更多内存。

图 5-6. 使用 SDXL 生成的图像

## FLUX、SD3 和视频

扩散模型领域的发展并未显示出放缓的迹象。2024 年 6 月，Stability AI 发布了 Stable Diffusion 3，2024 年 8 月，Black Forest Labs 发布了 Flux 系列模型。这些模型背后的原理与你在本章中学到的相同。然而，它们包含了一种不同类型的调度器（修正流匹配调度器）和一种不同的架构：diffusion transformer 而不是 UNet。

这两个模型系列都有多种尺寸，支持不同的分辨率，并且它们改进了提示理解和文本渲染能力。

相同的架构也可以扩展到支持视频生成。CogVideoX 是一系列将这些想法应用于视频生成领域的模型。这些模型表明，我们涵盖的原理（特别是条件控制）是指导生成模型行为的强大通用工具，而开源发布可以加快探索速度。

## 无分类器引导

尽管我们尽最大努力使文本条件控制尽可能有用，但模型在做出预测时仍然倾向于主要依赖噪声输入图像而不是提示。在某种程度上，这是有道理的：许多标题与其相关图像的联系并不紧密，因此模型学会了不过分依赖描述。然而，在生成新图像时，这是不可取的：如果模型不遵循提示，我们可能会得到与描述无关的图像。

为了缓解这个问题，我们引入了引导。引导是任何提供对采样过程更多控制的方法。我们可以应用的一种选择是修改损失函数以偏向特定方向。例如，如果我们希望生成偏向某种颜色，我们可以更改损失函数以衡量我们与目标颜色的平均距离。另一种选择是使用模型，如 CLIP 或分类器，来评估结果并将其损失信号作为生成过程的一部分。例如，使用 CLIP，我们可以比较提示文本和生成图像嵌入之间的差异，并引导扩散过程以最小化这种差异。第 181 页的“练习”展示了如何使用这种技术。

另一种选择是使用一种称为无分类器引导（Classifier-Free Guidance, CFG）的技巧，它结合了条件扩散模型和无条件扩散模型的生成。在训练期间，文本条件有时保持空白，迫使模型学习在没有文本信息的情况下对图像进行去噪（无条件生成）。然后，我们在推理时进行两次预测：一次使用文本提示作为条件，一次不使用。然后，我们可以使用这两个预测之间的差异来创建一个最终的组合预测，该预测根据缩放因子（引导比例）进一步推向文本条件预测指示的方向，希望生成更符合提示的图像。

为了引入引导，我们可以通过类似 noise_pred = noise_pred_uncond + guidance_scale * (noise_pred_text - noise_pred_uncond) 的方式修改噪声预测。这个小改动效果出奇地好，使我们能够更好地控制生成。我们将在本章后面深入探讨实现细节，但让我们先看看如何使用它。查看提示“An oil painting of a collie in a top hat”在 CFG 比例为 1、2、4 和 12 时的结果（输出图像从左到右）：

```
images = []
prompt = "An oil painting of a collie in a top hat"
for guidance_scale in [1, 2, 4, 12]:
    torch.manual_seed(0)
    image = pipe(prompt, guidance_scale=guidance_scale).images[0]
    images.append(image)

image_grid(images, 1, 4)
```

如你所见，较高的值会生成更符合描述的图像，但过高的值可能会导致图像过饱和。Google 的 Imagen 论文发现，这是因为进行中的预测超出了模型训练时使用的 [-1, 1] 范围。由于扩散模型是迭代运行的，它必须处理训练期间未见过的输入值，这可能导致过度饱和、海报化甚至生成失败。他们提出了一种称为动态阈值的方法，将值控制在范围内，并大大提高了高 CFG 比例下的质量。

## 综合应用：带注释的采样循环

现在你已经了解了每个组件的作用，让我们将它们结合起来生成图像，而不依赖管道。以下是我们将使用的设置：

```
// 一些设置
prompt = [ "Acrylic palette knife painting of a flower"
]  # 我们想要生成的内容
height = 512  # Stable Diffusion 的默认高度
width = 512  # Stable Diffusion 的默认宽度
num_inference_steps = 30  # 去噪步骤数
guidance_scale = 7.5  # 无分类器引导的比例
seed = 42  # 随机数生成器的种子
```

第一步是对文本提示“Acrylic palette knife painting of a flower”进行编码。因为我们计划进行CFG，所以我们将创建两组文本嵌入：一组是提示嵌入，另一组代表空字符串，即无条件输入。虽然我们在这里选择无条件输入，但这种设置提供了很大的灵活性。例如，我们可以执行以下操作：

编码一个负面提示而不是空字符串
添加负面提示可以引导模型避免朝某个方向发展。在本章的第 6 个练习中，你将尝试使用负面提示。

结合多个提示并赋予不同的权重
提示权重允许我们强调或弱化提示的某些部分。你将在第 8 章中了解更多关于提示权重的信息。

让我们实现文本嵌入的创建：

```
# 对输入进行标记化
text_input = pipe.tokenizer(
    prompt,
    padding="max_length",  # 填充到最大长度以确保输入具有相同的形状
    return_tensors="pt",
)

# 对无条件输入（空字符串）执行相同的操作
uncond_input = pipe.tokenizer(
    "",
    padding="max_length",
    return_tensors="pt",
)

# 将两个嵌入传给文本编码器
with torch.inference_mode():
    text_embeddings = pipe.text_encoder(text_input.input_ids.to(device))[0]
    uncond_embeddings = pipe.text_encoder(uncond_input.input_ids.to(device))[0]

# 连接两组文本嵌入
text_embeddings = torch.cat([uncond_embeddings, text_embeddings])
```

接下来，我们创建随机初始潜在变量，并设置调度器以使用所需的推理步骤数：

```
# 准备调度器
pipe.scheduler.set_timesteps(num_inference_steps)

# 准备随机起始潜在变量
latents = (
    torch.randn(
        (1, pipe.unet.config.in_channels, height // 8, width // 8),
    )
    .to(device)
    .half()
)
latents = latents * pipe.scheduler.init_noise_sigma
```

现在我们循环遍历采样步骤，在每个阶段获取模型预测，并使用它来更新潜在变量：

```
for t in pipe.scheduler.timesteps:
    // 创建两个潜在变量的副本以匹配两组文本嵌入（无条件和条件）
    latent_input = torch.cat([latents] * 2)
    latent_input = pipe.scheduler.scale_model_input(latent_input, t)

    // 预测无条件和条件潜在变量的噪声残差
    with torch.inference_mode():
        noise_pred = pipe.unet( latent_input, t,
            encoder_hidden_states=text_embeddings ).sample
    // 将预测分为无条件和条件版本
    noise_pred_uncond, noise_pred_text = noise_pred.chunk(2)
    // 执行无分类器引导
    noise_pred = noise_pred_uncond + guidance_scale * ( noise_pred_text -
        noise_pred_uncond
    )
    // 更新下一个时间步的潜在变量
    latents = pipe.scheduler.step(noise_pred, t, latents).prev_sample
```

注意 CFG 步骤。我们的最终噪声预测是 noise_pred_uncond + guidance_scale * (noise_pred_text - noise_pred_uncond)，将预测从无条件预测推向基于提示的预测。尝试更改引导比例以探索这如何影响输出。

在循环结束时，潜在变量应该代表一个与提示匹配的合理图像。最后一步是使用 VAE 将潜在变

量解码为图像，以便我们可以看到结果：

```
// 使用 VAE 缩放和解码图像潜在变量
latents = 1 / pipe.vae.config.scaling_factor * latents
with torch.inference_mode():
    image = pipe.vae.decode(latents).sample

image = (image / 2 + 0.5).clamp(0, 1)
show_image(image[0].float());
```

如果你探索 StableDiffusionPipeline 的源代码，你会注意到前面的代码与管道使用的 `call()` 方法非常匹配。希望这个带注释的版本表明幕后并没有发生什么太神奇的事情。当我们遇到在这个基础上添加技巧的其他管道时，请将其作为参考。

# 开放数据，开放模型

LAION-5B 数据集包含了超过 50 亿张图片的 URL 及其相应的描述（图片-描述对）。该数据集的创建过程是首先从 CommonCrawl（一个开放的网页抓取数据仓库，类似于谷歌为其搜索引擎索引互联网的方式）中提取所有图片 URL，然后使用 CLIP 筛选出文本与图片之间具有高相似度的图片-描述对。

该数据集由开放机器学习社区创建，并为该社区服务，他们看到了对这种开放访问数据集的需求。在 LAION 计划之前，只有少数大公司的研究实验室能够访问图片-文本对数据集。这些组织对其数据集的细节保密，使得他们的结果无法验证或复现。通过创建一个公开的 URL 和描述索引源，LAION 使得一波小型社区和组织能够训练模型并进行研究，否则这些研究将无法实现。

第一个潜在扩散模型（Latent Diffusion Model）就是这样一个模型，它由 CompVis 在 LAION 数据集的早期版本（包含 4 亿个图片-文本对）上训练而成。LAION 训练的潜在扩散模型的发布标志着整个研究社区首次能够使用一个强大的文本到图像模型。

潜在扩散的成功展示了这种方法的潜力，后续的 Stable Diffusion 工作进一步实现了这一潜力。训练像 Stable Diffusion 这样的模型需要大量的 GPU 时间。即使利用免费提供的 LAION 数据集，也只有少数人能够承担 GPU 时间的投入。这就是为什么模型权重和代码的公开发布如此重要：它标志着所有人首次能够使用一个与最佳闭源替代品具有相似能力的强大文本到图像模型。

Stable Diffusion 的公开可用性使其成为过去几年中研究人员和开发者探索该技术的首选。数百篇论文基于该基础模型，增加了新的功能或找到了创新的方法来提高其速度和质量。除了研究论文外，一个不一定来自机器学习背景的多样化社区一直在使用这些模型进行创新工作流程的探索、优化推理速度等等（见图 5-7）。

无数初创公司已经找到了将这些快速改进的工具集成到其产品中的方法，催生了一个全新的应用生态系统。

图 5-7. 文本到图像领域的创造力正在爆发

Stable Diffusion 推出后的几个月展示了公开分享这些技术的影响，许多进一步的质量改进和定制技术将在第 7 章和第 8 章中探讨。Stable Diffusion 在质量上与当时的商业替代品（如 DALL·E 和 Midjourney）竞争，成千上万的人花费时间改进它并基于这一开放基础进行构建。我们希望这个例子能鼓励其他人在未来效仿并与开源社区分享他们的工作。

> 除了用于训练 Stable Diffusion 外，LAION-5B 还被许多其他研究工作使用。一个例子是 Open-CLIP，这是 LAION 社区为训练高质量（SOTA）开源 CLIP 模型并复现与原始模型相似质量所做的努力。一个高质量的开源 CLIP 模型对许多任务有益，例如图像检索和零样本图像分类。在用于训练模型的数据中保持透明性也使得研究模型扩展的影响、正确复现

> 结果以及使研究更加易于访问成为可能。

## 挑战与 LAION-5B 的终结

然而，文本到图像生成模型及其下游商业应用的成功引发了人们对这些数据集的数据源和内容的担忧。

由于该数据集包含从互联网抓取的图片链接，它包含了数百万个指向可能包含受版权保护材料（如照片、艺术作品、漫画和插图）的图片的 URL。研究发现，这样的数据集还包括公开在线的私人敏感信息，例如个人可识别的医疗图像。

使用这样的数据集来训练生成式 AI 模型也可能使模型具备生成强化或加剧社会偏见内容的能力，并且它们可以用于生成明确的成人内容。此外，这些模型经常生成与训练数据非常相似的内容，这可能导致生成与受版权保护材料极为相似的内容。然而，这些开放模型是在开放数据集上训练的，因此可以研究、分析和缓解这些偏见和问题内容。

最近的研究表明，LAION-2B 数据集从互联网抓取时未能过滤掉与暴力和儿童安全相关的明确非法材料，这导致了数据集的停用。

## 替代方案

随着 LAION 数据集因儿童安全问题而停用，诸如 COYO-700M 和 DataComp-1B 等开放替代品填补了空白，它们遵循了类似的互联网级图片抓取公式。尽管它们不包含会导致其立即停用的内容，但它们仍然面临着与偏见、受版权保护材料和个人权利相同的挑战。CommonCanvas 是一个较小规模（7000 万张图片-文本对）的数据集，但仅包含开放 Creative Commons 许可的图片。

## 公平与商业使用

虽然一些国家在版权法中对研究用途有公平使用例外，而其他国家在使用抓取数据训练机器学习模型方面有看似有利的先例，但当研究模型被商业化和大规模用于生成式 AI 时会发生什么？这一复杂主题目前正在美国和欧洲多个司法管辖区的法院审理，涉及版权法、研究应用的公平使用、隐私、AI 工具对创意工作的经济影响等方面。我们并不声称对如此复杂的问题有答案，但这种法律灰色地带正在使研究和开源社区远离使用开放数据集；对于 Stable Diffusion XL，尽管模型权重是开源的，但用于训练它的数据集并未公开。

构建一个以同意、安全和许可为核心的新的大规模文本-图像数据集也将成为研究和开源社区的优秀资源，并为商业下游应用提供法律确定性。CommonCanvas 数据集展示了这一方向的道路。

# 项目时间：使用 Gradio 构建一个交互式 ML 演示

到目前为止，我们专注于使用开源库运行 Transformer 和扩散模型。这为我们提供了对模型的灵活性和控制，但也需要大量的工作来设置和运行。现实情况是，大多数人不知道如何编写代码，但可能对探索模型及其功能感兴趣。

在这个项目中，我们将构建一个简单的 ML 演示，允许用户通过使用 Stable Diffusion 从文本提示生成图像。演示可以轻松地向广大观众展示模型，并使你的工作和研究更易于访问。

构建 ML 演示的方法有很多。你可以使用 HTML、JavaScript 和 CSS。然而，这需要一些 Web 开发经验，并且不是一个简单的过程。或者，诸如 streamlit 和 gradio 等开源库使得使用 Python 构建交互式 ML 演示变得容易。我们将在本章中使用 gradio，这是一个非常简单和极简的库。

gradio 库可以在任何地方运行——在 Python IDE、Jupyter notebook、Google Colab 或 Hugging Face Spaces 等云环境中。构建 gradio 演示的最简单方法是使用其 Interface 类，该类有三个关键方面：

inputs

演示的预期输入类型，例如文本提示或图片

outputs

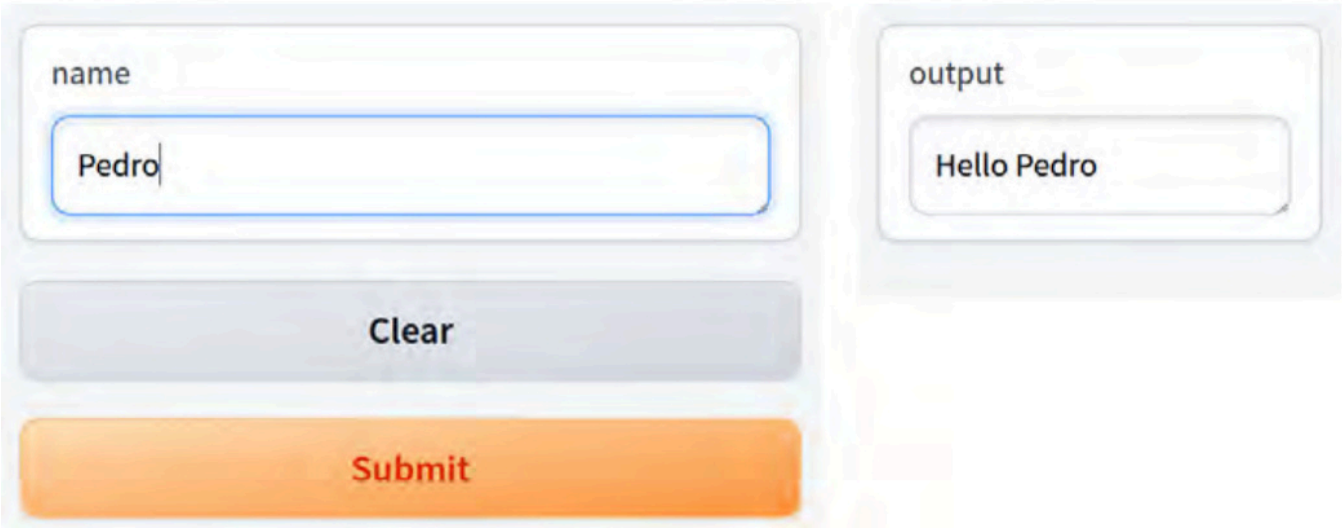
演示的预期输出类型，例如生成的图片

fn

当用户与演示交互时将调用的函数。这是魔法发生的地方。你可以在这里运行任何代码，包括使用 transformers 或 diffusers 运行模型。

让我们看一个例子：

```
import gradio as gr

def greet(name):
    return "Hello " + name

demo = gr.Interface(fn=greet, inputs="text", outputs="text")
demo.launch()
```




现在轮到你了！构建一个简单的演示，允许用户通过使用 Stable Diffusion 从文本提示生成图像。你可以使用上一节的代码作为起点。一旦你让演示运行起来，我们建议添加更多功能以使其更具互动性和趣味性。例如，你可以执行以下操作：

- 添加一个滑块来控制引导比例。
- 添加一个额外的文本字段以添加负面提示。
- 添加标题和描述，以便用户了解演示的内容。

如果你需要帮助，请记住查看官方文档和快速入门指南。

# 总结

本章展示了条件化如何为我们提供控制扩散模型生成图像的新方法。我们已经看到文本编码器如何根据文本提示条件化扩散模型，从而实现强大的文本到图像功能。并且我们通过深入采样循环并了解不同组件如何协同工作，探索了这一切如何在 Stable Diffusion 模型中结合在一起。

在第7章中，你将学习如何微调Stable Diffusion，以向模型添加新的知识或能力。例如，你将看到，通过展示你宠物的照片，Stable Diffusion可以学习“你的宠物”这一概念，并在新场景中生成

新颖的图像，比如“你的宠物在月球上”。

随后，在第8章中，我们将展示一些可以添加到扩散模型中的能力，使其超越简单的图像生成。例如，我们将探索修复技术，它允许我们遮罩图像的一部分，然后填充该部分。第8章还探讨了基于提示编辑图像的技术。

# 练习

1. 类条件扩散模型的训练过程与非条件模型有何不同，特别是在输入数据和使用的损失函数方面？
2. 时间步嵌入如何影响扩散过程中图像的质量和演变？
3. 解释潜在扩散与普通扩散的区别。使用潜在扩散的权衡是什么？
4. 文本提示是如何融入模型的？
5. 基于模型的引导与无分类器引导有何区别？无分类器引导的好处是什么？
6. 使用负提示的效果是什么？尝试使用 `pipe(, negative_prompt="")` 进行实验。你如何通过 Stable Diffusion 引导图像生成？
7. 假设你想从任何生成的图像中移除白色帽子。你如何使用负提示来实现这一点？首先尝试使用高级管道实现。然后，尝试调整端到端推理示例。（提示：只需修改无分类器条件中的随机部分。）
8. 在 SDXL 中，如果你使用 (256, 256) 而不是 (1024, 1024) 作为“原始大小”条件信号会发生什么？如果你使用 (0, 0) 以外的裁剪坐标会发生什么？你能解释原因吗？

你可以在本书的 GitHub 仓库中找到这些练习及其后续挑战的解答。

# 挑战

蓝色引导。假设我们希望将生成的图像偏向特定颜色，例如蓝色。我们该如何做到这一点？第一步是定义一个我们希望最小化的条件函数，在这种情况下，它将是一个颜色损失：

```python
def color_loss(images, target_color=(0.1, 0.5, 0.9)):
    """给定目标颜色 (R, G, B)，返回图像像素与该颜色平均距离的损失。"""
    # 将目标颜色映射到 (-1, 1)
    target = torch.tensor(target_color).to(images.device) * 2 - 1

    # 调整形状以与图像 (b, c, h, w) 兼容
    target = target[None, :, None, None]

    # 图像像素与目标颜色之间的平均绝对差异
    error = torch.abs(images - target).mean()
    return error
```

给定这个损失函数，编写一个采样循环（不需要训练），修改 `x` 以朝着损失函数的方向变化。

# 参考文献


Esser, Patrick, et al. “Scaling Rectified Flow Transformers for High-Resolution Image Synthesis.” arXiv, March 5, 2024. https://arxiv.org/abs/2403.03206.

Ho, Jonathan, and Tim Salimans. “Classifier-Free Diffusion Guidance.” arXiv, July 25, 2022. http://arxiv.org/abs/2207.12598.

Luccioni, Alexandra Sasha, et al. “Stable Bias: Analyzing Societal Representations in Diffusion Models.” arXiv, March 20, 2023. http://arxiv.org/abs/2303.11408.

Peebles, William, and Saining Xie. “Scalable Diffusion Models with Transformers.” arXiv, December 19, 2023. https://arxiv.org/abs/2212.09748.

Podell, Dustin, et al. “SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis.” arXiv, July 4, 2023. http://arxiv.org/abs/2307.01952.

Rombach, Robin, et al. “High-Resolution Image Synthesis with Latent Diffusion Models.” arXiv, April 13, 2022. http://arxiv.org/abs/2112.10752.

Saharia, Chitwan, et al. “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding.” Advances in Neural Information Processing Systems 35 (2022): 36479–36494. arXiv, May 24, 2022. https://arxiv.org/abs/2205.11487.

Schramowski, Patrick, et al. “Safe Latent Diffusion: Mitigating Inappropriate Degeneration in Diffusion Models.” arXiv, April 26, 2023. http://arxiv.org/abs/2211.05105.

Schuhmann, Christoph, et al. “LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models.” arXiv, October 15, 2022. http://arxiv.org/abs/2210.08402.

Xiao, Han, et al. “Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning

Algorithms.” arXiv, September 15, 2017. http://arxiv.org/abs/1708.07747.
Yang, Zhuoyi, et al. “CogVideoX: Text-to-Video Diffusion Models with an Expert Transformer.” arXiv, August 12, 2024. https://arxiv.org/abs/2408.06072.

# 第二部分. 生成模型的迁移学习

## 第6章. 微调语言模型

在第2章中，我们探讨了语言模型（LM）的工作原理以及如何将它们用于文本生成和序列分类等任务。我们看到，由于适当的提示和这些模型的零样本能力，LM 在许多任务中无需进一步训练即可发挥作用。我们还探索了社区提供的数十万个预训练模型中的一些。在本章中，我们将讨论如何通过在数据上微调 LM 来提高其在特定任务上的性能。

虽然预训练模型展示了显著的能力，但其通用训练可能并不适合某些任务或领域。微调通常用于使模型的理解适应其数据集或任务的细微差别。例如，在医学研究领域，一个在通用网络文本上预训练的 LM 在未经调整的情况下表现不佳，因此我们可以在医学文献数据集上对其进行微调，以增强其生成相关医学文本或从医疗文档中提取信息的能力。另一个例子是用于制作对话模型。尽管大型预训练模型可以生成连贯的文本，但它们通常无法生成高质量的对话文本或遵循指令。我们可以在包含日常对话和非正式语言结构的数据集上微调该模型，使其输出引人入胜的对话文本，就像你在 ChatGPT 等界面中期望的那样。

本章的目标是建立微调大型语言模型（LLM）的坚实基础，因此我们将涵盖以下内容：

- 使用微调的编码器模型对文本主题进行分类
- 理解编码器模型在现代 LLM 时代的作用
- 使用解码器模型生成特定风格的文本
- 通过指令微调解决多个任务
- 参数高效微调技术，使我们能够使用较小的 GPU 训练模型
- 使我们能够以更少的计算资源运行模型推理的技术

### 文本分类

在进入生成模型领域之前，了解微调预训练模型的一般流程是一个好主意。我们将从序列分类开始，其中模型为给定输入分配一个类别。序列分类是经典的机器学习问题之一。通过它，你可以解决诸如垃圾邮件检测、情感识别、意图分类和虚假内容检测等挑战。尽管这是一个简单的任

务，人们经常通过提示通用 LM 来解决，但它是理解微调过程及其步骤的良好起点。

我们将微调一个模型来对短新闻文章摘要的主题进行分类。正如我们很快会发现的那样，微调所需的计算和数据比从头开始训练模型要少得多。通常的流程如下（图6-1）：

1. 确定任务的数据集。
2. 定义所需的模型类型（编码器、解码器或编码器-解码器）。
3. 选择一个符合你需求的良好基础模型。
4. 预处理数据集。
5. 定义评估指标。
6. 训练模型。
7. 分享。

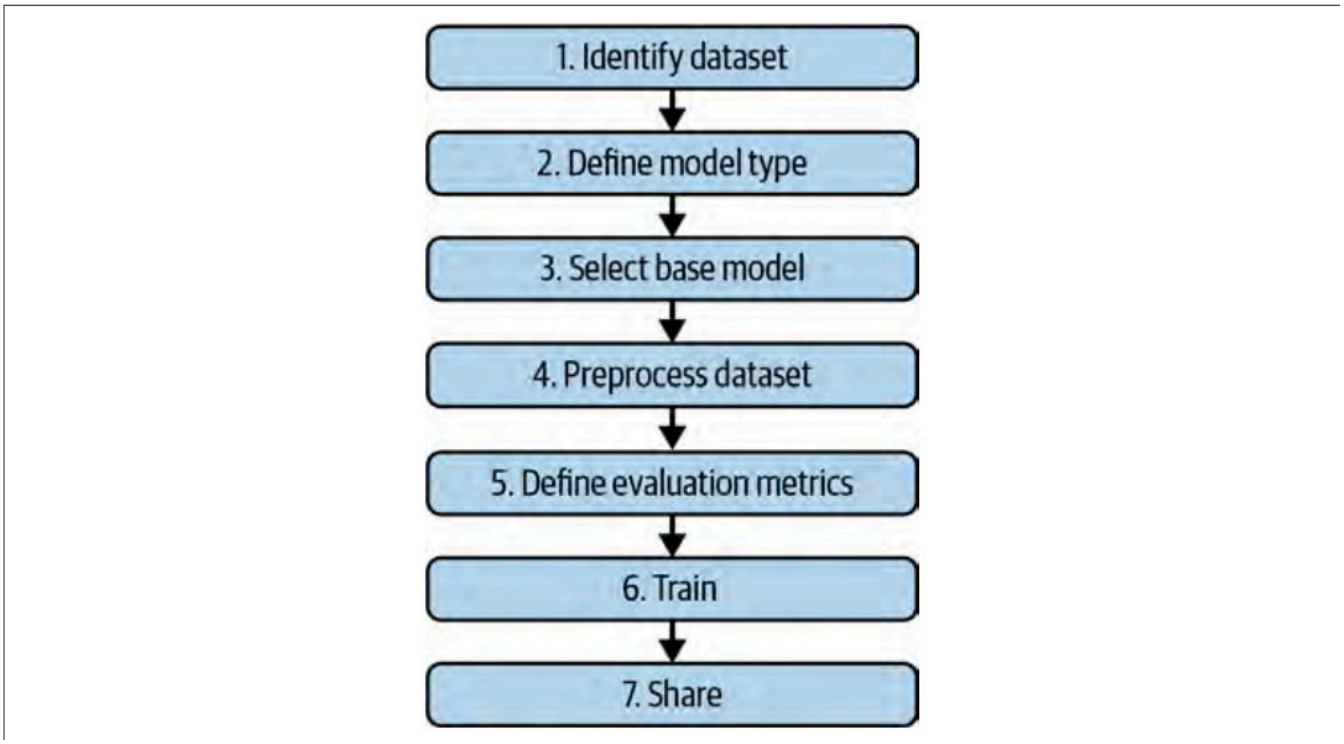



图6-1. 微调工作流的常规步骤

## 识别数据集

我们的目标是调整一个通用的预训练语言模型，使其能够作为文本分类器工作，因此我们需要教会它需要检测的类别。这导致我们需要一个用于序列分类的标注数据集。根据你的任务和用例，你可以使用公开的或私有的数据集（例如，来自你公司的数据集）。一些寻找公开数据集的好地方包括 Hugging Face Datasets、Kaggle、Zenodo 和 Google Dataset Search。由于有成千上

万的数据集，我们需要帮助找到适合我们用例的数据集。一种方法可以是在 Hugging Face 上筛选文本分类数据集。

在下载量最大的数据集中，AG News 数据集是一个著名的非商业数据集，用于基准测试文本分类模型以及研究数据挖掘、信息检索和数据流。

有时你可能希望与社区共享一个数据集。为此，你可以将其作为数据集仓库上传。datasets 库对常见数据类型（音频、图像、文本、CSV、JSON、pandas DataFrames 等）提供了开箱即用的支持。

首先应该探索数据集。如下面的代码所示，数据集包含两列：一列是文本，另一列是标签。数据集提供了 120,000 个训练样本，足够微调模型。与预训练模型相比，微调需要的数据量非常少，仅使用几千个样本就足以获得一个良好的基线模型：

```
from datasets import load_dataset
raw_datasets = load_dataset("fancyzhx/ag_news")
raw_datasets
DatasetDict({ train: Dataset({ features: ['text', 'label'], num_rows: 120000 })
    test: Dataset({ features: ['text', 'label'], num_rows: 7600 })
})
```

让我们看看一个具体的示例是什么样的：

```
raw_train_dataset = raw_datasets["train"]
raw_train_dataset[0]

{'label': 2,
'text': 'Wall St. Bears Claw Back Into the Black (Reuters) Reuters '
"- Short-sellers, Wall Street's dwindling\\band of "
'ultra-cynics, are seeing green again.'}
```

第一个样本包含文本和标签，标签是…2？2 指的是哪个类别？为了弄清楚这一点，我们可以检查数据集的特征及其标签字段：

```
print(raw_train_dataset.features)
{'label': ClassLabel(names=['World', 'Sports', 'Business', 'Sci/Tech'], id=None)
    , 'text': Value(dtype='string', id=None)}
```

因此，标签 0 表示世界新闻，1 表示体育新闻，2 表示商业新闻，3 表示科技新闻。弄清楚这一点后，让我们决定使用哪个模型。

## 定义使用哪种模型类型

让我们回顾一下第 2 章。根据我们试图解决的任务类型，我们可以使用三种类型的 transformer 之一：

### 编码器模型

它们从序列中获得丰富的语义表示，捕捉输入的含义，这些表示可以用于依赖于输入语义信息的各种任务（例如，识别文本中的实体或对序列进行分类）。可以在这些嵌入之上添加一个小型网络，以训练特定的下游任务。

### 解码器模型

这些模型旨在生成新的序列，例如文本。它们接受输入（通常是嵌入或上下文）并生成连贯的输出序列，使其非常适合文本生成任务。

### 编码器-解码器模型

这些模型非常适合需要将输入序列转换为不同输出序列的任务，例如机器翻译或摘要。编码器处理输入，而解码器生成相应的输出。

考虑到对短新闻文章摘要进行主题分类的任务，我们有三种可能的方法：

1. **零样本或少样本学习**

我们可以使用高质量的预训练模型，解释任务（例如，“分类为这四个类别”），然后让模型完成其余工作。这种方法不需要任何微调，并且在当今强大的预训练模型下非常常见——单个模型可以通过将任务表述为文本生成问题来解决许多任务。

2. **文本生成模型**

微调一个文本生成模型，使其在给定输入新闻文章时生成标签（例如，“商业”）。我们可以在这里使用解码器或编码器-解码器模型。

3. **带有分类头的编码器模型**

通过向嵌入添加一个简单的分类网络（称为头）来微调编码器模型。这种方法提供了一个专门且高效的模型，适合我们的用例，使其成为我们主题分类任务的首选。

基于上述内容，我们将选择第三种方法。

## 选择一个好的基础模型

我们需要一个满足以下条件的模型：

- 具有基于编码器的架构
- 足够小，可以在 GPU 上几分钟内完成微调
- 具有扎实的预训练结果
- 能够处理短文本序列

BERT 虽然较老，但它是微调的一个很好的基础编码器架构。鉴于我们希望快速训练模型且计算能力有限，我们可以使用 DistilBERT，它比 BERT 小 40%，速度快 60%，同时保留了 BERT 97% 的能力。给定基础模型，我们可以针对多个下游任务进行微调，例如回答问题或分类文本。

除了原始的 BERT 和 DistilBERT 之外，还有许多其他模型可以用作微调的基础模型。我们不会深入探讨每一个，但知道它们的存在很重要。一些例子包括 RoBERTa、ALBERT、Electra、DeBERTa、Longformer、LuKE、MobileBERT 和 Reformer。每个模型都有自己的训练过程，并基于原始的 BERT 模型构建。选择哪一个取决于你的具体需求，但鉴于我们的计算需求，使用 DistilBERT 是一个很好的起点。DeBERTa 在撰写本文时是 SOTA 之一。

## 预处理数据集

如第2章所述，每个语言模型（LM）都附带其对应的分词器。为了微调DistilBERT，我们必须确保整个数据集使用与预训练模型相同的分词器进行分词。我们可以使用AutoTokenizer加载适当的分词器，然后定义一个函数来对一批样本进行分词。transformers库要求批次中的所有输入具有相同的长度：通过设置padding=True，我们向样本中添加零，使它们与最长的输入样本具有相同的大小。

需要注意的是，Transformer模型有一个最大上下文大小——即LM在进行预测时可以使用的最多

token数。对于DistilBERT，这个限制是512个token，因此不要尝试将其用于整本书。幸运的是，我们的大多数样本都是简短的摘要，但有些样本可能仍然会超过这个token限制。为了处理这种情况，我们可以使用truncation=True，这将截断所有样本以使其适应模型的上下文长度。然而，这种方法有一个权衡：截断文本意味着可能会丢失一些潜在的有用信息。

处理长上下文的Transformer模型是一个活跃的研究领域。对于涉及基于编码器的模型和长上下文的场景，你可以尝试以下几种策略：

- 使用专门的长上下文Transformer模型，例如Longformer。
- 将文本分成较小的片段并分别处理。
- 使用滑动窗口方法分块处理文本。
- 在预处理步骤中总结文本，然后将摘要输入模型。

选择哪种策略取决于任务和模型。例如，如果你想分析一本书的情感，可以使用分块方法并分别分析书的不同部分。如果你想对一篇长文章进行分类，可以总结文章然后对摘要进行分类。

> 尽管模型自带一个开箱即用的上下文长度，即模型一次可以考虑的token数量，但诸如旋转嵌入（rotary embeddings）等技术允许我们使用更长甚至无限的上下文长度。我们稍后会详细讨论这一点。

让我们对两个样本进行分词以检查输出：

```
from transformers import AutoTokenizer
checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

def tokenize_function(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        padding=True,
        return_tensors="pt"
    )

tokenize_function(raw_train_dataset[:2])
```

```
{'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
     1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]),
'input_ids': tensor([[  101,  2813,  2358,  1012, 6468, 15020,  2067,  2046,
    1996,  2304,  1006, 26665, 1007, 26665, 1011,  2460, 1011, 19041, 1010,
    2813, 2395, 1005, 1055,  1040, 11101, 2989, 1032, 2316, 1997, 11087, 1011,
    22330, 8713, 2015, 1010, 2024, 3773, 2665, 2153, 1012, 102, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0], [  101, 18431, 2571, 3504, 2646, 3293, 13395,  1006,
    26665, 1007, 26665, 1011, 2797, 5211, 3813, 18431, 2571, 2177,  1010, 1032,
    2029, 2038,  1037,  5891, 2005,  2437,  2092,  1011, 22313,  1998,  5681,
    1032, 6801, 3248,  1999,  1996,  3639,  3068,  1010,  2038, 5168, 2872,
    1032,  2049, 29475,  2006,  2178,  2112, 1997, 1996,  3006,  1012,   102]])
    }
```

在这个例子中，tokenize_function()接受一批样本，使用DistilBERT分词器对其进行分词，并通过填充和截断确保统一长度。正如你所看到的，第一个元素比第二个元素短，因此它在末尾有一些额外的ID为0的token。这些零对应于[PAD] token，在推理过程中将被忽略。请注意，该样本的注意力掩码在末尾也有0；这确保模型只关注实际的token。

现在你已经了解了分词过程，我们可以使用map()方法对整个数据集进行分词。该方法并行地对数据集中的每个元素应用一个函数：

```
tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
tokenized_datasets
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label', 'input_ids', 'attention_mask'],
        num_rows: 120000
    })
    test: Dataset({
        features: ['text', 'label', 'input_ids', 'attention_mask'],
        num_rows: 7600
    })
})
```

## 定义评估指标

除了在训练过程中监控损失外，通常还建议定义一些下游指标，以更好地评估和监控模型的性能。我们将利用 `evaluate` 库，这是一个具有标准化接口的便捷工具，适用于各种指标。指标的选择取决于任务。对于序列分类，合适的候选指标可以是以下内容：

### 准确率

表示所有预测中正确预测的比例，提供了模型整体性能的高层次概览。对于平衡数据集来说，这是一个很好的指标，并且易于解释。然而，对于不平衡的数据集，它可能会产生误导。

### 精确率

这是正确标记的正例与所有预测为正例的实例的比率。它帮助我们理解模型对正例预测的准确性。

当误报的代价很高时，应使用精确率，例如垃圾邮件检测。

### 召回率

该指标表示模型正确预测的实际正例的比例。它反映了模型捕捉所有正例的能力，如果存在许多假阴性，召回率会较低。当漏报的代价很高时，应使用召回率，例如在医学诊断中。

### F1 分数

F1 分数是精确率和召回率的调和平均数，提供了一个平衡的度量，同时考虑了误报和漏报，并对精确率和召回率之间的强烈差异进行惩罚。F1 通常用于不平衡的数据集，并且是分类任务的一个很好的默认指标。

`evaluate` 中的指标提供了一个 `description` 属性和一个 `compute()` 方法，以在给定标签和模型预测的情况下获取指标：

```
import evaluate

accuracy = evaluate.load("accuracy")
print(accuracy.description)
print(accuracy.compute(references=[0, 1, 0, 1], predictions=[1, 0, 0, 1]))
```

('准确率是处理的案例总数中正确预测的比例。可以通过以下公式计算：'
'准确率 = (TP + TN) / $TP + TN + FP + FN$) '
' 其中：'
'TP: 真阳性'
'TN: 真阴性'
'FP: 假阳性'
'FN: 假阴性')
{'accuracy': 0.5}

让我们定义一个 `compute_metrics()` 函数，该函数在给定预测实例（包含标签和预测）的情况下，返回一个包含准确率和 F1 分数的字典。当我们在训练期间评估模型时，将自动使用此函数来监控其进度：

```
f1_score = evaluate.load("f1")

def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)

    # 计算准确率和 F1 分数
    acc_result = accuracy.compute(references=labels, predictions=preds)
    acc = acc_result["accuracy"]
    f1_result = f1_score.compute(references=labels, predictions=preds,
        average="weighted")
    f1 = f1_result["f1"]
    return {"accuracy": acc, "f1": f1}
```

- `compute_metrics()` 期望一个 `EvalPrediction` 实例。`EvalPrediction` 是 `Trainer` 使用的一个实用类，包含样本的标签和模型预测。
- 使用 `argmax` 获取概率最高的预测类别。
- 使用加载的 `accuracy` 计算标签和预测之间的准确率分数。回想一下，`accuracy` 输出一个带有 `accuracy` 键的字典。
- 重复计算 F1 分数。由于我们有多个类别，我们使用 `weighted` 参数。这意味着我们为每个类别计算 F1，然后根据每个类别的真实实例数量进行加权平均。
- 最后，通过构建一个字典返回这两个指标。

## 训练模型

是时候开始训练了。回想一下，DistilBERT 是一个编码器模型。如果我们直接使用原始模型，我们会得到嵌入，就像我们在第 2 章中所做的那样，因此我们不能直接使用这个模型。对于文本序列分类，我们将这些嵌入输入到一个分类头中（见图 6-2）。在微调时，我们不会使用固定的嵌入：所有模型参数、原始权重和分类头都是可训练的。这要求分类头必须是可微的，并引导我们在基础 Transformer 之上使用神经网络。这个头将接收嵌入作为输入并输出类别概率。为什么我们要训练所有的权重？通过训练所有参数，我们帮助使嵌入对这个特定的分类任务更有用。

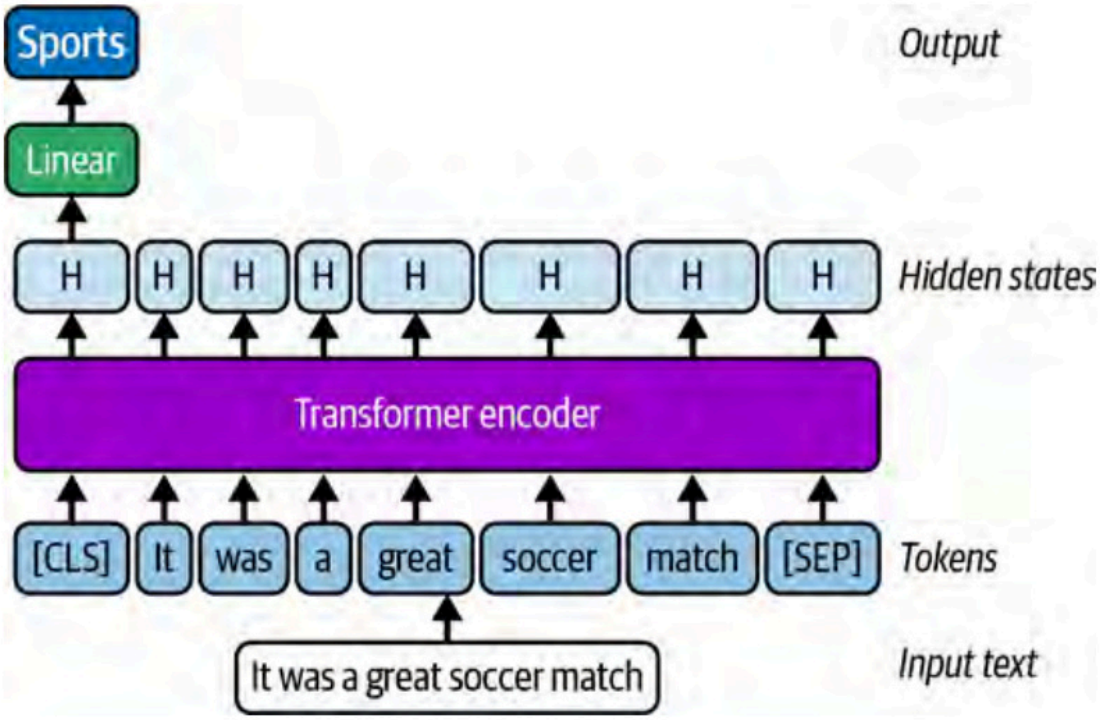



图 6-2. 带有分类头的 BERT。在实践中，CLS 对应的嵌入被用作池化嵌入，并可用于分类任务。

虽然我们将使用一个简单的前馈网络，但我们可以使用更复杂的网络作为头，甚至可以使用经典模型，如逻辑回归或随机森林（在这种情况下，我们将模型用作特征提取器并冻结权重）。使用一个简单的层效果很好，计算效率高，也是最常见的方法。

如果你在计算机视觉中进行过迁移学习，你可能熟悉冻结基础模型权重的概念。这在 NLP 中并不常见，因为我们的目标是使内部语言表示对下游任务更有用。在计算机视觉中，通常会冻结一些层，因为基础模型学习到的特征更通用，对许多任务都有用。例如，一些层捕捉到像边缘或纹理这样的通用特征，这些特征在视觉任务中广泛适用。是否冻结或解冻层取决于上下文，包括数据集大小、计算量以及预训练和微调任务之间的相似性。在本章后面，你将学习一种称为适配器的技术，它允许我们使用冻结的 LLM。

要使用分类头训练模型，我们可以使用 `AutoModelForSequenceClassification` 加载模型。这将做两件事：

- 加载指定的模型（在本例中为 DistilBERT），但不包括其掩码语言模型头。这是模型的编码器部分，它为每个标记输出一个嵌入。它还输出一个池化嵌入，该嵌入捕捉整个序列的信息。
- 在模型顶部添加一个随机初始化的分类头。这个头只是一个线性层，它接收池化嵌入并输出类别概率。

让我们看看这是如何实现的：

```
import torch
from transformers import AutoModelForSequenceClassification

from genaibook import get_device

device = get_device()
num_labels = 4
model = AutoModelForSequenceClassification.from_pretrained(
    checkpoint, num_labels=num_labels
).to(device)

('Some weights of DistilBertForSequenceClassification were not initialized from
    the model checkpoint at distilbert-base-uncased
and are newly initialized: ["classifier.bias", "classifier.weight",
    "pre_classifier.bias", "pre_classifier.weight"] You should probably TRAIN
    this model on a down-stream task to be able to use it for predictions and
    inference.')
```

你会收到一个关于一些权重被新初始化的警告。这是有道理的；我们有一个适合我们分类任务的新头，需要训练它。

随着我们的模型初始化完成，我们终于可以开始训练了。我们可以采取多种方法来训练模型。如果你熟悉 PyTorch，你可以编写自己的训练循环。或者，transformers 提供了一个名为 `Trainer` 的高级类，它简化了训练循环的复杂性。

在创建我们的 Trainer 之前，第一步是定义 TrainingArguments，如下面的代码示例所示。TrainingArguments 指定了用于训练的超参数，如学习率和权重衰减，确定每批样本的数量，设

置评估间隔，并决定是否通过将模型推送到 Hub 来与生态系统共享我们的模型。我们不会修改超参数，因为 TrainingArguments 提供的默认值通常表现良好。不过，我们鼓励你探索和实验它们。Trainer 类是一个强大而灵活的工具。

```
from transformers import TrainingArguments
batch_size = 32  # 如果你有一个大或小的 GPU，可以更改此值
training_args = TrainingArguments(
    "classifier-chapter4",
    push_to_hub=True,
    num_train_epochs=2,
    eval_strategy="epoch",
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
)
```

- 是否在每次保存模型时都将其推送至Hugging Face Hub。您可以通过调整save_strategy参数来改变模型保存频率，默认设置为每数百训练步保存一次。
- 训练的总轮次（epoch）数；一个epoch代表对训练数据的完整遍历。
- 验证集评估时机设置。默认每500训练步评估一次，若指定为epoch则会在每个训练轮次结束时进行评估。
- 每个计算核心的训练批次大小（batch size）。若GPU内存不足可调低该数值，或启用auto_find_batch_size=True自动寻找GPU能承受的最大批次大小。

我们现在拥有了所有需要的部分：

- 一个带有适当头的预训练模型，准备进行微调
- 训练参数
- 一个计算指标的函数
- 训练和评估数据集
- 一个分词器，我们添加它是为了确保它与模型一起被推送到 Hub

AG News 数据集包含 120,000 个样本，比我们需要的多。为了进行初始快速训练，我们将使用 10,000 个样本，但请随意调整这个数字——更多的数据应该会带来更好的结果。请注意，我们仍然会使用整个测试集进行评估：

```
from transformers import Trainer

# Shuffle the dataset and pick 10,000 examples for training
shuffled_dataset = tokenized_datasets["train"].shuffle(seed=42)
small_split = shuffled_dataset.select(range(10000))

# Initialize the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    compute_metrics=compute_metrics,
    train_dataset=small_split,
    eval_dataset=tokenized_datasets["test"],
    tokenizer=tokenizer,
)
```

一切准备就绪，`Trainer` 初始化后，是时候开始训练了：

```
trainer.train()
```

训练将报告损失、评估指标和训练速度的详细信息。表 6-1 提供了一个总结视图。

表 6-1. DistilBERT 在 AG News 数据集上微调的训练和评估指标

| Metric | Epoch 1 value | Epoch 2 value |
|---|---|---|
| eval_loss | 0.2624 | 0.2433 |
| eval_accuracy | 0.9117 | 0.9184 |
| eval_f1 | 0.9118 | 0.9183 |
| eval_runtime | 15.2709 | 14.5161 |
| eval_samples_per_second | 497.678 | 523.557 |
| eval_steps_per_second | 15.585 | 16.396 |
| train_runtime | - | 213.9327 |
| train_samples_per_second | - | 93.487 |
| train_steps_per_second | - | 2.926 |
| train_loss | - | 0.2714 |

希望这只需要一点点时间。最终的评估准确率和 F1 分数接近 $92\%$，这还不错，特别是考虑到我们使用了不到 $10\%$ 的可用训练数据。评估损失在 epoch 之间减少，这正是我们想要的。如果你想分享最终模型供其他人访问，你需要在最后调用 `push_to_hub`。你可以在 Hub 上找到我们的模型：

```
trainer.push_to_hub()
```

虽然使用 `Trainer` 可能看起来像一个黑盒子，但在底层，它只是像我们在第 3 章中训练简单扩散模型时那样进行常规的 PyTorch 训练循环。从头编写这样的循环看起来会像这样：

```
from transformers import AdamW, get_scheduler
optimizer = AdamW(model.parameters(), lr=5e-5)
lr_scheduler = get_scheduler("linear", ...)
for epoch in range(num_epochs):
    for batch in train_dataloader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
```

- 优化器保存模型的当前状态，并根据梯度更新参数。
- 学习率调度器定义了学习率在训练过程中如何变化。
- 对所有数据进行多次迭代（epoch）。
- 遍历训练数据中的所有批次（batch）。
- 将批次数据移动到设备上并运行模型。
- 计算损失并进行反向传播。
- 更新模型参数、调整学习率并将梯度重置为零。

如果你将模型推送到 Hub，其他人现在可以使用 `AutoModel` 或 `pipeline()` 访问它。让我们尝试一个例子：

```
# Use a pipeline as a high-level help
from transformers import pipeline

pipe = pipeline(
    "text-classification",
    model="genaibook/classifier-chapter4",
    device="cuda",
)
pipe(
    """The soccer match between Spain and
Portugal ended in a terrible result for Portugal."""
)

[{'label': 'Sports', 'score': 0.8631355166435242}]
```

预测似乎是正确的。在第一次尝试时，你可能会得到 `LABEL_1` 而不是 `Sports`。这是因为模型不知道内在的标签名称。要更新它们，你可以通过添加 `id2label` 和 `label2id` 映射来更新 `config.json` 文件。这将使预测更具可读性和可解释性。

让我们深入探讨一下指标。你可以使用 `Trainer.predict` 或 `pipe.predict` 来获取预测结果。`Trainer.predict` 方法返回一个 `PredictionOutput` 对象，其中包含预测结果、标签 ID 和指标，而 `pipe.predict` 返回一个包含预测结果和相应标签的字典列表。让我们通过查看前三个样本文本及其对应的预测和标签来确认一切是否合理。通过运行一些样本来确保一切正常总是很重要的：

```
// 获取所有样本的预测
model_preds = pipe.predict(tokenized_datasets["test"]["text"])
// 获取数据集标签
references = tokenized_datasets["test"]["label"]
// 获取标签名称列表
label_names = raw_train_dataset.features["label"].names
// 打印前3个样本的结果
samples = 3
texts = tokenized_datasets["test"]["text"][:samples]
for pred, ref, text in zip(model_preds[:samples], references[:samples], texts):
    print(f"Predicted {pred['label']}; Actual {label_names[ref]};")
    print(text)
```

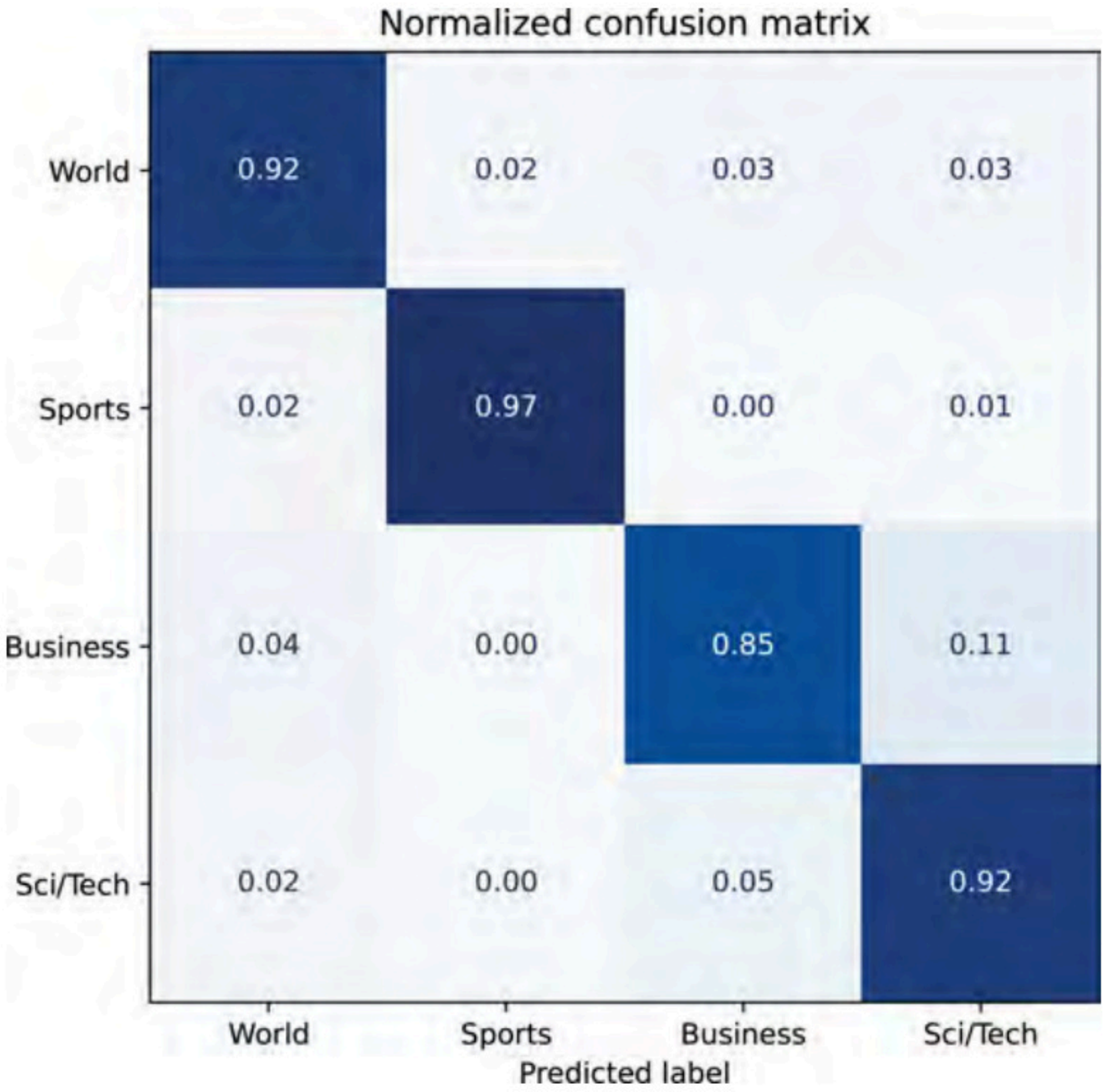
```
Predicted Business; Actual Business;
Fears for T N pension after talks Unions representing workers at Turner
Newall say they are 'disappointed' after talks with stricken parent firm
Federal Mogul.
Predicted Sci/Tech; Actual Sci/Tech;
The Race is On: Second Private Team Sets Launch Date for Human Spaceflight
(SPACE.com) SPACE.com - TORONTO, Canada -- A second\team of rocketeers
competing for the  #36;10 million Ansari X Prize, a contest for\privately
funded suborbital space flight, has officially announced the first\launch
date for its manned rocket.
Predicted Sci/Tech; Actual Sci/Tech;
Ky. Company Wins Grant to Study Peptides (AP) AP - A company founded by a
chemistry researcher at the University of Louisville won a grant to
develop a method of producing better peptides, which are short chains of
amino acids, the building blocks of proteins.
```

预测与参考一致，标签也合理。现在让我们深入探讨一下指标。

在机器学习分类任务中，混淆矩阵是一个总结模型性能的表格，描述了真正例、真反例、假正例和假反例的计数。对于多类分类，矩阵变成一个正方形，其维度等于类别数量，其中每个单元格表示标签和预测类别的组合的实例计数。行表示实际（真实）类别，而列表示预测类别。我们可以对矩阵进行归一化，使每一行的总和为1，从而更容易解释模型在不同类别中的表现。分析这个矩阵可以深入了解模型在区分特定类别时的优势和劣势。

我们将使用 `evaluate` 来加载和计算混淆矩阵，并使用 `sklearn` 中的 `ConfusionMatrixDisplay` 来可视化它。混淆矩阵将帮助我们理解模型在哪些地方出错，以及哪些类别更难预测。例如，通过查看以下混淆矩阵，我们可以检查商业文章经常被错误地标记为科技文章：

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
// 将预测标签转换为ID
label_to_id = {name: i for i, name in enumerate(label_names)}
pred_labels = [label_to_id[pred["label"]] for pred in model_preds]

// 计算混淆矩阵
confusion_matrix = evaluate.load("confusion_matrix")
cm = confusion_matrix.compute(
    references=references,
    predictions=pred_labels,
    normalize="true"
)["confusion_matrix"]
// 绘制混淆矩阵
fig, ax = plt.subplots(figsize=(6, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_names)
disp.plot(cmap="Blues", values_format=".2f", ax=ax, colorbar=False)
plt.title("Normalized confusion matrix")
plt.show()
```




## 还有价值吗?

在后ChatGPT时代，训练一个用于文本分类的编码器模型可能显得有些过时。难道我们不能直接提示一个廉价的模型“将文本分类为以下类别之一……”吗？虽然快速通用模型的便利性难以超越，但在某些情况下，小型、定制的分类器仍然很有帮助——尤其是在速度和效率是关键的应用中。

例如，这些模型在准备当今最大LLM的训练数据时表现出色，这涉及处理大量文本。在Llama 3论文中，作者应用了一种他们称之为“基于模型的质量过滤”的方法，其中训练了一个“质量分类

器”，然后用于对文档进行评分以进行质量过滤。作者指出：“出于效率考虑，我们使用DistilRoberta为每个文档生成质量分数。”这个质量模型的训练数据来自Llama 2的预测；直接使用Llama 2处理需要过滤的数万亿个token的数据将极其昂贵。Phi-3团队和FineWeb作者也采用了类似的方法，为他们的FineWeb Edu子集筛选最具教育意义的内容。作者写了一篇优秀的博客文章，介绍了他们的方法以及质量过滤的重要性。

小型且快速的基于编码器的模型的另一个有用之处是获取检索系统的嵌入，其目标是找到与给定查询最相似的文档。最后一个使用示例是护栏，即使用一个小型模型来检查模型的输入，以检测其是否恶意，或检查模型的输出以确保其无害。在所有这些情况下，小型模型的速度和效率是关键。

尽管如此，当今的趋势是远离这些小型、专用模型，转向更强大的通用模型。然而，仍然有定制的空间（因此有了这本书），所以让我们讨论如何应用类似的过程来生成文本：找到（或创建）一个数据集，选择一个模型，定义评估指标，并训练模型。

# 生成文本

我们刚刚微调了一个基于编码器的模型用于文本分类。现在，让我们深入探讨如何训练一个用于文本生成的模型。在文本分类中，我们的标签是一组离散选项（世界、体育、商业和科技），而训练和微调生成模型意味着执行下一个token预测任务，其中标签是文本输出。

例如，如果我们的目标是生成代码，我们可以收集大量允许的代码数据集（如The Stack）并从零开始训练模型。虽然这很有趣，但要获得不错的结果需要大量的计算资源（导致数周或数月的训练时间）。

与其从零开始训练一个用于开放式文本生成的模型，我们可以微调一个现有模型以生成特定风格的文本。这种方法使我们能够受益于模型对语言的已有知识，大大减少了对大量数据和计算能力的需求。例如，你可以使用几百条推文来生成具有你独特写作风格的新推文。

> **有标签还是无标签?**
> 在下一个token预测任务中，我们不需要像分类那样显式地为数据打标签；模型将根据输入文本学习预测下一个token。这使我们能够从网络上构建大规模数据集。
> 另一方面，一种新的技术家族——基于人类反馈的强化学习（RLHF）——允许我们通过提供反馈来引导模型的输出。这对于对话模型特别有用，因为你可以为模型的输出提供偏好或纠正。这就是为什么许多聊天机器人都有一个点赞/点踩按钮，或者提供并排生成的文本供用户选择最佳的一个。即便如此，偏好优化只是训练过程的一个组成部分，模型仍然需要从无监督的数据中学习。我们将在第10章中详细讨论RLHF。

让我们继续新闻主题，并微调一个模型以生成特定风格的新闻，例如商业新闻。我们可以使用相同的AG News数据集。首先，我们过滤出所有标记为商业的样本（标签为2），并删除不必要的标签列：

```
filtered_datasets = raw_datasets.filter(lambda example: example["label"] == 2)
filtered_datasets = filtered_datasets.remove_columns("label")
```

## 选择合适的生成模型

第二个问题是使用哪个基础模型。由于我们的目标是进行文本生成，因此需要一个解码器模型。面对成千上万的模型，我们需要选择一个符合我们需求的模型。让我们讨论一些可能影响我们决策的关键因素：

### 模型大小

在本地计算机上部署一个拥有600亿参数的模型是不现实的。模型大小的选择取决于预期的推理时间、硬件容量和部署需求等因素。在本章后面，我们将探讨一些技术，这些技术可以在相同的计算资源下运行更多参数的模型。

### 训练数据

微调模型的性能与基础模型的训练数据与推理数据的匹配程度相关。例如，如果要从一个预训练在代码上的模型开始，微调模型以生成符合你代码库风格的代码会更有效。考虑数据来源的特定性，尤其是当并非所有模型都公开其训练数据时。同样，对于韩语文本生成，你可能不希望使用一个主要基于英语的模型（例如多语言模型或专门针对韩语训练的模型）。并非所有模型都公开其数据来源，这可能会使识别这一点变得具有挑战性。

### 上下文长度

如前所述，不同的模型有不同的上下文长度限制。例如，如果上下文长度为1,024，模型可以使用最后1,024个标记进行预测。要生成长篇文本，你需要一个具有较大上下文长度的模型。我们将在本书后面探讨如何处理更长的上下文。

### 许可证

在选择基础模型时，许可证方面至关重要。考虑模型是否符合你的使用需求。模型可能具有商业或非商业许可证，并且开源和开放访问许可证之间存在区别。理解这些许可证对于确保遵守法律和使用限制至关重要。例如，尽管某些模型可能允许商业使用，但它们可以指定允许的使用场景

和不应使用模型的场景。在其他情况下，许可证可能会限制模型输出的使用方式（例如，禁止使用模型的输出来训练另一个模型）。

评估生成模型仍然是一个挑战，各种基准测试评估了特定方面。例如，ARC用于科学问题，HellaSwag用于常识推理，这些基准测试作为不同能力的代理。Hugging Face Open LLM Leaderboard收集了数千个模型的基准测试结果，并允许根据模型大小和类型进行过滤。然而，需要注意的是，这些基准测试是系统比较的工具，最终的模型选择应始终基于其在实际任务中的表现。Open LLM Leaderboard中使用的许多基准测试并不专注于对话，因此它们不应作为选择对话模型的主要标准。模型的选择取决于你的用例，基于单一指标选择模型是不推荐的。

Leaderboard考虑了一组具有挑战性的基准测试：[4]

**MMLU-Pro（知识, Massive Multitask Language Understanding - Professional）**

这是一个包含12,000个多项选择题的知识数据集。每个问题包括一段文字和10个答案选项。问题涉及数学、物理、经济学、心理学、商业等多个学科。

**GPQA（复杂知识, Graduate-Level Google-Proof Q&A）**

这是一个包含具有挑战性的研究生水平物理、化学和生物多项选择题的小型数据集。这些问题由领域专家（拥有或正在攻读相关领域的博士学位）设计，预计即使对非专家（来自其他领域且能访问互联网的熟练人类——GPQA中的GP代表“Google-Proof”）也具有挑战性。[5]

**MuSR（多步推理, Multistep Soft Reasoning）**

这包含每个问题约一千字的复杂问题，这对短上下文模型提出了挑战。问题可能包括谋杀谜案、团队分配和物体放置。

**MATH（问题解决, Mathematics Aptitude Test of Heuristics）**

这包含超过12,000个来自高中数学考试的问题。有不同难度级别，LLM Leaderboard仅使用最难的5级问题。

**BBH（混合, Big Bench Hard）**

这个基准测试包含一套23个需要多步推理的具有挑战性的任务。任务涵盖算法和算术推理（例如，布尔表达式、几何形状和导航）、自然语言理解（例如，讽刺检测和形容词排序）、世界知识（例如，理解体育和推荐电影）以及推理（翻译错误检测）。这个基准测试与人类偏好相关。

**IFEval（指令遵循, Instruction-Following Evaluation）**

这个数据集评估模型是否能够遵循诸如“至少提到关键词Y三次”或“用少于十个字写作”等指令。

在LLM Leaderboard使用的六个基准测试中，只有IFEval专门针对对话模型。我们将在本章后面讨论对话模型。我们当前的目标是以特定风格生成文本，因此我们将专注于这一点。表6-2展示了一些流行的开放访问预训练LLM。

表6-2. 一些流行的开放访问预训练LLM及其在Open LLM Leaderboard上的表现

| Model | Creator | Size | Training data | Open LLM performance | Context length | Vocab size | License |
|---|---|---|---|---|---|---|---|
| GPT-2 | OpenAI | 117M<br>380M<br>812M<br>1.6B | Unreleased<br>Up to 40 GB of text from a web scrape | 6.51<br>5.81<br>5.48<br>4.98 | 1,024 | 50,257 | MIT |
| GPT-Neo | EleutherAI | 125M<br>1.3B<br>2.7B | The Pile<br>300B tokens<br>380B tokens<br>420B tokens | 4.38<br>5.33<br>6.34 | 2,048 | 50,257 | MIT |
| Falcon | TII UAE | 7B<br>11B<br>40B<br>180B | Partially released<br>Refined Web built on top of CommonCrawl<br>1.5T tokens<br>1T tokens<br>3.5T tokens | 5.1<br>13.78<br>11.33<br>N/A | 8,192 | 65,024 | Apache 2.0 (7B and 40B)<br>Custom (11B and 180B) |
| Llama 2 | Meta | 7B<br>13B<br>70B | Unreleased<br>2T tokens | 8.72<br>10.99<br>18.25 | 4,096 | 32,000 | Custom |
| Llama 3 | Meta | 8B<br>70B | Unreleased<br>15T tokens | 13.41<br>26.37 | 8,192 | 128,256 | Custom |
| Llama 3.1 | Meta | 8B<br>70B<br>405B | Unreleased<br>15T tokens | 13.78<br>25.91<br>N/A | 131,072 | 128,256 | Custom |
| Mistral | Mistral | 7B | Unreleased | 14.5 | 8,192 | 32,000 | Apache 2.0 |
| Mixtral | Mistral | 8x7B<br>8X22B[a] | Unreleased | 19.23<br>25.49 | 32,768<br>65,536 | 32,000 | Apache 2.0 |
| Qwen 2 | Alibaba | 500M<br>1.5B<br>7B<br>72B | Unreleased | 7.06<br>10.32<br>23.66<br>35.13 | 131,072 | ~150,000 | Custom |
| Phi (1, 1.5, 2) | Microsoft | 1.42B<br>1.42B<br>2.78 | 54B tokens<br>150B tokens<br>1.4T tokens | 5.52<br>7.06<br>15.45 | 2,048 | 51,200 | MIT |

[a] 在“Mixtral”模型中，″8 × 7B″是什么意思？这意味着该模型是一个专家混合模型（MoE），这是一种特殊的模型架构，你将在第10章中了解更多。简而言之，它是一个包含多个较小模型的模型，内部机制决定每个标记使用哪些模型。比较MoE模型和常规密集模型的参数数量并不简单，你将在后面了解更多。

这个表格并不全面；还有许多其他开放的LLM，例如Google Gemma、Mosaic MPT和Cohere Command R+，当本书出版时，可能还会有更多。同样，这个表格不包括代码模型。对于这些模型，你可能需要查看Big Code Models Leaderboard，在那里你可以找到诸如CodeLlama（来自Meta的流行模型）和BigCode的模型（使用宽松许可证代码训练的模型）等模型。

此外，值得注意的是，这个表格偏向于主要在英语数据上训练的模型。然而，强大的中文模型如InternLM、ChatGLM和Baichuan也是预训练LM扩展领域中值得注意的贡献者。这些信息作为选择模型进行实验时的指南，而不是开源模型的详尽列表。

## 训练生成模型

鉴于我们希望在无需强大GPU的环境中快速训练少量数据，我们将微调SmolLM的最小变体。我们鼓励您尝试更大的模型和不同的数据集。在本章后面，我们将探讨使用更大模型进行推理和训练的技术。

和之前一样，我们首先加载模型和分词器。SmolLM的一个特点是它没有指定填充标记，但我们在分词时需要它，因为它用于确保所有样本具有相同的长度。我们可以将填充标记设置为与文本结束标记相同：

```
from transformers import AutoModelForCausalLM
model_id = "HuggingFaceTB/SmolLM-135M"
tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = (
    tokenizer.eos_token
) # Needed as SmolLM does not specify padding token.
model = AutoModelForCausalLM.from_pretrained(model_id).to(device)
```

我们将对数据集进行分词（但使用SmolLM的分词器）：

```
def tokenize_function(batch):
    return tokenizer(batch["text"], truncation=True)

tokenized_datasets = filtered_datasets.map(
    tokenize_function,
    batched=True,
    remove_columns=["text"],  # We only need the input_ids and attention_mask
)
```

```
tokenized_datasets

DatasetDict({ test: Dataset({ features: ['input_ids', 'attention_mask'],
    num_rows: 1900 }) train: Dataset({ features: ['input_ids',
    'attention_mask'], num_rows: 30000 })
})
```

在主题分类示例中，我们对所有样本进行填充和截断，以确保它们具有相同的长度。除了在分词阶段进行外，我们还可以使用数据整理器来完成。这些工具将样本组装成批次。transformers库为任务（如语言建模）提供了一些开箱即用的整理器。整理器将动态地将批次中的样本填充到最大长度。除了填充外，语言建模整理器还为语言建模任务构建输入，这比之前稍微复杂一些。在语言建模中，我们将输入移动一个元素并将其用作标签。例如，如果输入是“I love Hugging Face”，则标签是“love Hugging Face”。模型的目标是根据前面的标记预测下一个标记。实际上，数据整理器将创建一个带有输入副本的标签列。稍后，模型将负责移动输入和标签。

以下代码展示了如何为因果语言建模创建数据整理器：

```
from transformers import DataCollatorForLanguageModeling

# mlm corresponds to masked language modeling
# and we set it to False as we are not training a masked language model
# but a causal language model
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
```

让我们看看这对三个样本是如何工作的。如下所示，每个样本的长度不同（37、55和51）：

```
samples = [tokenized_datasets["train"][i] for i in range(3)]

for sample in samples:
    print(f"input_ids shape: {len(sample['input_ids'])}")

input_ids shape: 37
input_ids shape: 55
input_ids shape: 51
```

多亏了整理器，样本被填充到批次中的最大长度（55），并添加了一个标签列：

```
out = data_collator(samples)
for key in out:
    print(f"{key} shape: {out[key].shape}")

input_ids shape: torch.Size([3, 55]) attention_mask shape: torch.Size([3, 55])
    labels shape: torch.Size([3, 55])
```

最后，我们需要定义训练参数。通过调整TrainingArguments中的几个关键参数，例如以下参数，我们可以对模型训练进行一些控制：

**权重衰减**

这种正则化技术通过向损失函数添加惩罚项来防止模型过拟合。它阻止学习算法分配大权重。调整TrainingArguments中的权重衰减参数可以调整这种正则化效果，影响模型的泛化能力。

**学习率**

这个关键的超参数决定了优化步长。在TrainingArguments的上下文中，您可以指定学习率，影响训练过程的收敛速度和稳定性。仔细调整学习率可以显著影响模型的生成质量。

**学习率调度器类型**

学习率调度器决定了学习率在训练过程中如何演变。不同的任务和模型架构可能受益于特定的调度策略。TrainingArguments提供了定义学习率调度器类型的选项，使您可以尝试各种调度策略，例如恒定学习率、余弦退火等。

在此示例中，我们修改了一些参数以展示这种灵活性：

```
training_args = TrainingArguments( "business-news-generator", push_to_hub=True,
    per_device_train_batch_size=8, weight_decay=0.1, lr_scheduler_type="cosine",
     learning_rate=5e-4, num_train_epochs=2, eval_strategy="steps",
    eval_steps=200, logging_steps=200,
)
```

在完成所有这些设置后，与分类示例一样，最后一步是创建一个包含所有组件的Trainer实例。主要区别在于这次我们使用了数据整理器，并且我们使用了5,000个样本：

```
trainer = Trainer( model=model, tokenizer=tokenizer, args=training_args,
    data_collator=data_collator, train_dataset=tokenized_datasets["train"]
    .select(range(5000)), eval_dataset=tokenized_datasets["test"],
trainer.train()
```

表6-3总结了微调过程中的训练和评估损失。

表6-3. SmolLM在AG News数据集上微调的训练结果

| epoch | step | loss | grad_norm | learning_rate | eval_loss | eval_runtime | eval_samples_per_second | eval_steps_per_second |
|---|---|---|---|---|---|---|---|---|
| 0.32 | 200 | 3.2009 | 2.99705 | 0.0004690 | 3.31005 | 18.6024 | 102.137 | 12.794 |
| 0.64 | 400 | 2.8833 | 2.46037 | 0.0003839 | 3.21182 | 18.8513 | 100.789 | 12.625 |
| 0.96 | 600 | 2.7102 | 2.35531 | 0.0002656 | 3.09971 | 18.953 | 100.248 | 12.557 |
| 1.28 | 800 | 1.722 | 2.55815 | 0.0001435 | 3.24014 | 18.7631 | 101.262 | 12.684 |
| 1.6 | 1000 | 1.5371 | 1.89922 | 4.774e-05 | 3.224 | 18.7509 | 101.328 | 12.693 |
| 1.92 | 1200 | 1.4841 | 2.78178 | 1.971e-06 | 3.22884 | 18.5468 | 102.444 | 12.832 |

和之前一样，我们将模型推送到Hub：

```
trainer.push_to_hub()
```

现在我们可以使用pipeline()并指定任务（text-generation）来加载模型并运行推理：

```
from transformers import pipeline
pipe = pipeline( "text-generation", model="genaibook/business-news-generator",
    device=device,
)
print( pipe("Q1", do_sample=True, temperature=0.1, max_new_tokens=30)[0][
    "generated_text" ]
print( pipe("Wall", do_sample=True, temperature=0.1, max_new_tokens=30)[0][
    "generated_text" ]
print( pipe("Google", do_sample=True, temperature=0.1, max_new_tokens=30)[0][
    "generated_text" ]
)
('Q1: China #39;s Airline Pilots Union Says Unions May Block Planes '(Update1)
    China #39;s Air')
('Wall Street Seen Flat After Jobless Data  NEW YORK (Reuters) - ' 'Wall Street
    was expected to see a  slightly lower open on Friday')
('Google IPO Imminent Google #39;s long-awaited stock sale is 'imminent, and
    the company is already considering whether to sell ' 'its')
```

正如您所注意到的，生成的文本遵循与AG News商业部分相似的结构。然而，生成的内容可能并不总是表现出连贯性，这是可以理解的，因为我们使用了一个质量不高的小型基础模型，并且使用了很少的训练数据。使用Mistral 7B或Llama 3.1的70B变体等非常大的模型无疑会生成更连贯的文本，同时保持相同的格式。

# 说明

在本章的第一部分中，我们讨论了为特定文本分类任务（如主题分类）微调基于编码器的模型。然而，这种方法需要为每个不同的任务训练一个新模型。如果我们遇到一个未见过的任务，例如识别文本是否为垃圾邮件，我们将没有现成的预训练模型可用，因此需要为其微调一个模型。这促使我们探索其他技术，因此让我们简要讨论不同方法的优点、局限性和用途：

### 微调多个模型

我们可以为每个任务选择一个基础模型并进行微调，以构建一个专用模型。在微调过程中，所有模型权重都会更新，这意味着如果我们想解决五个不同的任务，最终将得到五个微调后的模型。

### 适配器

我们可以冻结基础模型，并训练一个称为适配器的小型辅助模型，而不是修改所有模型权重。我

们仍然需要为每个新任务使用不同的适配器，但它们的大小显著较小，这意味着我们可以轻松拥有多个适配器而不会增加开销。目前有积极的研究在生产环境中管理数百甚至数千个适配器，它们在实践和工业中都非常流行并被广泛使用。您将在下一节中了解适配器。

**提示工程**

正如第一章中所学，我们可以利用强大的预训练模型的零样本和少样本能力来解决不同的任务。在零样本方法中，我们编写一个详细解释任务的提示。在少样本方法中，我们添加解决任务的示例，以提高模型的性能。这些能力的表现取决于基础模型的强度。像 Llama 3.1 这样非常强大的模型可能会产生令人印象深刻的零样本结果，这对于处理各种任务（如撰写长篇电子邮件或总结书籍章节）非常有用。

**监督微调（SFT）**

SFT，也称为指令微调，是提高大语言模型（LLMs）零样本性能的一种替代且简单的方法。经典的指令微调将任务表述为指令，例如“这篇文章的主题是商业还是体育？”或“将‘你好吗’翻译成西班牙语”。这种方法主要涉及为许多任务构建指令数据集，然后使用这些混合的指令数据集微调预训练的语言模型，如图 6-3 所示。为指令微调创建数据集是一项复杂度可控的任务；例如，我们可以利用 AG News 并通过构建如下提示来将输入和标签结构化为指令：

文本属于“世界”、“体育”、“商业”或“科技”类别中的哪一个？用一个词回答：

文本：华尔街熊市重回盈利（路透社）路透社——华尔街日益减少的极端怀疑论者——卖空者，再次看到了绿色。

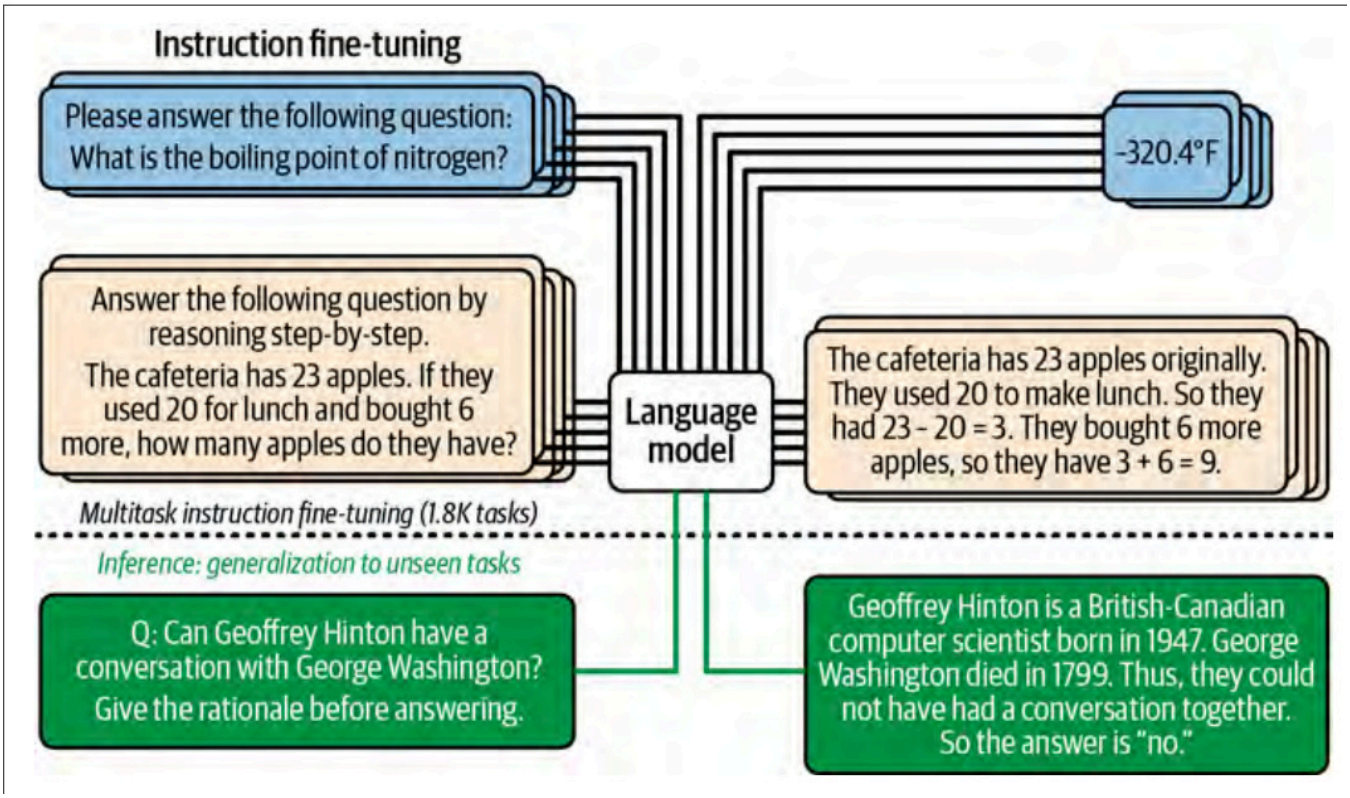



图 6-3. 通过指令微调，我们可以将许多带标签的数据集格式化为生成任务（改编自 Flan T5 论文中的图像）

通过构建足够大的多样化指令数据集，我们可以得到一个通用的指令微调模型，该模型可以解决许多任务，甚至新任务，这得益于跨任务泛化。这一理念是 Flan 模型的基础，该模型可以开箱即用地解决 62 个任务。这一概念在 Flan T5 模型中得到了进一步扩展，Flan T5 是一个开源的指令微调 T5 模型家族，可以解决超过 1,000 个任务。需要注意的是，该模型使用输入（指令）和输出（答案）文本进行训练；与 SmolLM 微调示例不同，这是一种监督训练技术。指令微调在 T5 或 BART 等编码器-解码器架构中非常流行，因为数据集的输入-输出结构。这一理念随后被扩展到大多数大语言模型中。

何时使用微调、指令微调还是提示工程？这再次取决于任务、可用资源、期望的实验速度等。通常，针对特定任务或领域的微调模型表现会更好。另一方面，它们不允许您开箱即用地处理任务。指令微调更具通用性，但定义数据集和结构需要额外的工作。提示工程是最灵活的快速实验方法，因为它不需要您开箱即用地训练模型。尽管如此，它需要一个更强大的基础模型，并且对生成的控制有限。

我们不会构建一个端到端的指令微调示例，因为它主要是一个数据集任务而不是建模任务，但如果您想深入研究这一主题，让我们讨论一些优秀的论文：

- 论文《Finetuned Language Models Are Zero-Shot Learners》（2022 年 2 月）的作者使用指令微调训练了一个名为 Flan 的模型，并超越了基础模型的零样本性能和其他模型的少样本性能。
- 在 Flan 之后，出现了一波新的数据集论文。《Cross-Task Generalization via Natural Language Crowdsourcing Instructions》（2022 年 3 月）介绍了 Natural Instructions，这是一个包含 61 个任务和 193,000 个输入-输出对的数据集，通过将现有的 NLP 数据集映射到统一模式生成。这样做的前提是，人类可以通过学习（以监督方式）其他任务的实例来遵循指令解决新问题。作者对 BART（一种编码器-解码器模型）进行了指令微调，与不使用指令相比，跨任务泛化能力提高了 $19\%$。模型训练的任务越多，表现越好。
- 《Multitask Prompted Training Enables Zero-Shot Task Generalization》（2022 年 3 月）遵循了类似的不同任务统一数据模式的概念。作者微调 T5 构建了 $\mathrm{T0}$，这是一个在多任务混合数据上训练的编码器-解码器模型，能够泛化到更多任务。其中一个令人兴奋的亮点是，数据中代表的任务越多，模型的中位性能越高，同时不会降低变异性。
- 随后，《Super-NaturalInstructions: Generalization via Declarative Instructions on $1600+$ NLP Tasks》（2022 年 10 月）进一步扩展了这一概念，这是一个包含超过 1,600 个任务和 500 万个示例的新数据集。这些项目的区别在于数据集的生成方式。T0 基于已有的任务实例回溯性地构建指令，而 Natural Instructions 则由 NLP 研究人员制作指令，并由众包工人构建数据集实例。

另一种方法是使用大语言模型生成输出：

- Unnatural Instructions（2022 年 12 月）是一个基于种子示例并请求第四个示例的自动生成示例的数据集。通过要求模型重新表述每个指令来增强数据集。
- Self-Instruct（2023 年 5 月）利用大语言模型自身的生成进行引导。其思想是让模型生成指令，然后生成输入（基于指令），最后生成输出。[7] 合成生成的数据集往往包含更多噪声。它们可能导致模型不如使用较少但更好策划的人工生成数据训练的模型稳健。
- LIMA（2023 年 5 月）是一个更小的英语指令数据集。尽管它只有一千个实例，但作者能够微调一个强大的 Llama 模型。这得益于强大的预训练模型和非常仔细的训练数据策划。

这些只是指令微调模型大规模爆炸中的一部分。Flan-T5 是一个使用 FLAN 数据集微调的 T5 模型。Alpaca 是一个在 InstructGPT 生成的指令数据集上微调的 Llama 模型。WizardLM 是一个在 Evol-Instruct 数据集上微调的 Llama 模型。ChatGLM2 是一个在英语和中文指令上训练的双语微调模型。我们不断发现相同的公式，即将强大的基础模型与多样化的指令数据集（可以是人工生成或模型生成）相结合。

《Learning to Generate Task-Specific Adapters from Task Description》（2021 年 6 月）是一种提高泛化能力的不同方法。作者不是为所有任务构建一个通用网络，而是生成称为适配器的任

务特定参数。尽管适配器已经存在多年，但最近它们在自然语言和图像生成中的采用变得广泛。在具有数十亿参数的语言模型中，许多人希望为其领域或任务进行微调。下一节将全面介绍适配器。

总结本节，指令微调的两个主要组成部分是强大的基础模型和高质量的指令数据集。毫不奇怪，指令数据集的质量对模型至关重要。该数据集可以是合成生成的（例如使用自指令）、人工生成的或两者的混合。一致的研究表明，训练数据中代表的任务越多，模型的表现越好。最后，指令模板对最终性能有很大影响。现有数据集最终在任务的数量和多样性之间进行权衡。

# 适配器的快速介绍

现在让我们深入探讨第四种方法：适配器。到目前为止，我们已经探索了微调 DistilBERT 进行文本分类和微调 SmolLM 以生成特定风格的文本。在这两种情况下，模型的所有权重都在微调过程中被修改。微调比预训练效率高得多，因为我们不需要太多数据或计算能力。然而，随着模型规模不断增大的趋势，在消费级硬件上进行传统的微调变得不可行。此外，如果我们想为不同的任务微调一个编码器模型，最终会得到多个模型，从而成倍增加存储和计算需求。

欢迎 PEFT！参数高效微调（Parameter-efficient fine-tuning，简称 PEFT）是一组技术，可以在不微调所有模型参数的情况下适应预训练模型。通常，我们会添加少量额外的参数，称为适配器，然后微调这些适配器，同时冻结原始的预训练模型。这会带来什么效果呢？

**更快的训练速度和更低的硬件需求**

在进行传统微调时，我们会更新许多参数。使用 PEFT 时，我们只更新适配器，适配器的参数数量相对于基础模型来说非常少。因此，训练完成得更快，并且可以在较小的 GPU 上进行。

**更低的存储成本**

微调模型后，我们只需要存储适配器，而不是为每次微调存储整个模型。当某些模型的存储空间超过 $100\text{GB}$ 时，如果每个下游模型都需要再次保存所有参数，这将无法很好地扩展。适配器的大小可能是原始模型的 $1\%$。如果我们对一个 $100\text{GB}$ 的模型进行 100 次微调，传统微调将需要 10,000 GB 的存储空间，而 PEFT 只需要 200 GB（原始模型和 100 个 1 GB 的适配器）。

**可比的性能**

PEFT 模型的性能往往与完全微调的模型性能相当。

**无延迟影响**

正如我们即将讨论的那样，训练后，适配器可以合并到预训练模型中，这意味着最终的大小和推理延迟将保持不变。

这听起来好得令人难以置信。它是如何工作的呢？有多种 PEFT 方法。其中最流行的是前缀调优（prefix tuning）、提示调优（prompt tuning）和低秩适应（LoRA），我们将在本章重点讨论 LoRA。LoRA 使用低秩分解将权重更新表示为两个较小的矩阵，称为更新矩阵。虽然这可以应用于 Transformer 模型中的所有块，但我们通常只将其应用于注意力块。

PEFT 是一个简单的库，用于在 transformers 和 diffusers 中使用这些技术。首先，让我们讨论如何构建上一节中 SmolLM 模型的适配器。在 LoRA 的情况下，我们可以控制多个方面，例如：

**秩 r**

这控制更新矩阵的大小。较大的秩允许适配器学习更复杂的模式，但需要更多的参数。

**lora_alpha**

这缩放更新矩阵。例如，如果 lora_alpha 为 32，r 为 8，梯度更新将放大 4 倍。这类似于在训练期间使用更高的学习率。

**lora_dropout**

LoRA 层的 dropout 概率，可以帮助防止过拟合。

**task_type**

任务类型，例如 SEQ_CLS（序列分类）或 CAUSAL_LM（因果语言模型）。这将决定适配器的架构。

**use_dora**

DoRA 是 LoRA 的一种变体，特别适合匹配完全微调的性能。我们在这个示例中不会使用它，但知道它的存在是很好的。

LoRA 中的更新矩阵不是普通的矩阵。它是一种特殊的矩阵，称为低秩矩阵。想象一下，你有一个包含大量信息的巨大矩阵。低秩矩阵的思想是使用更少的行和列来总结它，而不会丢失重要信息。对于那些对 LoRA 背后的（非常高级的）数学感兴趣的人，更新矩阵用低秩分解表示，其中 $W_0$ 是原始权重，$x$ 是输入：

$$h = W_0 x + \Delta W = W_0 x + \frac{\alpha}{r} BAx$$

在 LoRA 中，更新 $\Delta W$ 表示为两个低秩矩阵 $B$ 和 $A$ 的乘积，其中 $B$ 的行数较少，$A$ 的列数较少，比原始权重矩阵少。缩放因子 $\frac{\alpha}{r}$ 控制此更新的幅度。$B$ 的维度为 $d \times r$，$A$ 的维度为 $r \times k$，其中 $d$ 和 $k$ 分别是原始权重的行数和列数。

让我们进入代码部分。第一步是创建 PEFT 方法的配置：

```
from peft import LoraConfig, get_peft_model
peft_config = LoraConfig( r=8, lora_alpha=32, lora_dropout=0.05,
    task_type="CAUSAL_LM"
)
model = AutoModelForCausalLM.from_pretrained("HuggingFaceTB/SmolLM-135M")
peft_model = get_peft_model(model, peft_config)
peft_model.print_trainable_parameters()
```

trainable params: 460,800 || all params: 134,975,808 || trainable%: 0.3414

初始模型有近 1.35 亿个参数，但只有大约 46 万个参数会被训练。这只是原始模型大小的 $0.34\%$。PEFT 的思想是，我们可以训练这个小适配器，并获得与 300 倍大的模型相似的性能。

PEFT 在底层是如何工作的？当你微调一个基础模型时，你正在更新所有层。正如所讨论的那样，LoRA 用两个较小的矩阵来近似这些更新矩阵。例如，假设有一个包含 10,000 行和 20,000 列的更新矩阵。这意味着它包含 2 亿个值。假设我们使用秩为 8 的 LoRA。第一个矩阵 $A$ 将有 10,000 行和 8 列，而矩阵 $B$ 将有 8 行和 20,000 列（以确保相同的输入和输出大小）。$A$ 有 80,000 个值，$B$ 有 160,000 个值。我们从 2 亿个值减少到 24 万个值。这缩小了 800 倍！LoRA 假设这些矩阵可以很好地近似权重更新矩阵。

我们讨论了 $r$ 参数。如前所述，它控制 LoRA 矩阵的维度，这导致了能力和过拟合之间的权衡。秩过高会导致适配器过于复杂，容易过拟合。秩过低可能会导致性能不足，因为模型无法捕捉足够的复杂性。第二个关键参数是 alpha，它控制适配器对原始模型的影响程度。较高的 alpha 赋予适配器更多的重要性。选择 r 和 alpha 的值取决于问题和模型。对于 LLM 来说，一个好的起点是使用秩为 8，并始终使用 alpha 为秩的两倍。

微调后，我们可以将 LoRA 权重合并回原始模型，如图 6-4 所示。这意味着无论是否合并 LoRA 适配器，运行模型推理所需的延迟和计算量完全相同：

$$\text{weight} = \text{weight} + \text{scaling} \times (B \times A)$$

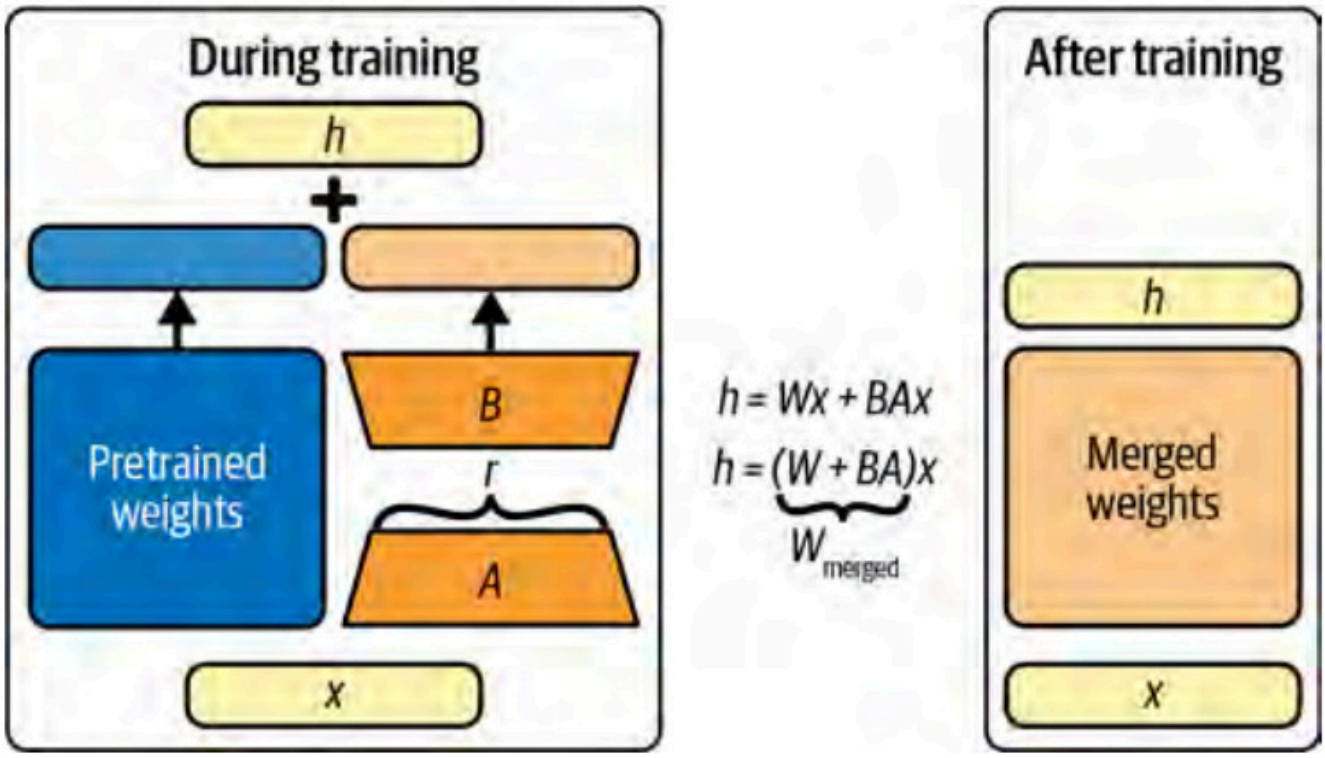



图 6-4. LoRA 减少了可训练权重的数量。训练后，LoRA 权重可以合并回原始模型。

LoRA 如此之小，这使得它们在生产中非常便携和实用。想象一个用例，用户期望聊天机器人或图像生成器以初始模型未知的 10 种风格生成内容。与其对初始模型进行 10 次微调并临时加载模型，我们可以根据需要加载和卸载适配器。最近的技术，如 LoRAX，允许在单个 GPU 上服务超过一百个微调适配器。

在某些用例中，你可能希望合并适配器。正如我们更新单个适配器一样，我们也可以对多个适配器进行更新：

$$\text{weight} = \text{weight} + \text{scaling}_1 \times (B_1 \times A_1)$$
$$\text{weight} = \text{weight} + \text{scaling}_2 \times (B_2 \times A_2)$$
$$\text{weight} = \text{weight} + \text{scaling}_3 \times (B_3 \times A_3)$$

最后一个问题是使用 LoRA 更新哪些参数。在更多块中使用 LoRA 通常会带来稍微更好的性能，但代价是训练期间需要更多的内存，这可能是一个值得的权衡，并且可以通过 target_modules 参数来实现。我们可以将 LoRA 用于注意力块以进行快速实验，这通常是 PEFT 库的默认设置。你也可以使用 target_modules = "all-linear" 来选择所有线性模块，不包括输出块。

尽管在本章中我们专注于文本生成的微调，但 PEFT 在其他领域也广泛使用，例如图像生成（我们将在第 7 章中探讨）、图像分割等。

# 量化简介

PEFT 允许我们以更少的计算和磁盘空间微调模型。然而，推理过程中模型的大小并不会减少。如果你要推理一个拥有 300 亿参数的模型，你仍然需要一个强大的 GPU 来运行它。例如，像 Llama 这样的 405B 模型将需要超过 8 个 A100 GPU，这些 GPU 非常强大且昂贵（每个价格超过 $\$15,000$）。在本节中，我们将讨论一种技术，它允许我们以不降低性能的方式在较小的 GPU 上运行这些模型。

每个参数都有一个数据类型或精度（见图 6-5）。例如，float32（FP32，也称为全精度）类型使用 32 位存储浮点数。FP32 允许以高精度表示广泛的数字范围，这对于预训练模型非常重要。然而，在许多情况下，并不需要如此广泛的范围。在这些情况下，我们可以使用 float16（或 FP16，也称为半精度）。FP16 的精度较低，数字范围也较小（最大可能数字为 64,000），这引入了新的风险：模型可能会溢出（如果数字不在可表示范围内）。然而，在推理过程中，使用 FP16 是没问题的；半精度的风险主要在训练期间显著。第三种数据类型是 brain floating-point，即 bfloat16。BF16 与 FP16 一样使用 16 位，但以不同的方式分配这些位，以便为较小的数字（如神经网络权重中常见的数字）获得更高的精度，同时仍然覆盖与 FP32 相同的总范围。

假设我们有一个 70 亿参数的模型；70 亿个参数，每个参数为 32 位，导致 2240 亿位；2240 亿位是 280 亿字节，即 $\sim 26$ GB。如果我们使用半精度，我们只需要 $13\text{GB}$。这是内存使用量的显著减少，可以带来更快的推理速度和更低的成本。附录 B 讨论了不同模型和精度的内存需求。

测试你的知识：

- 一个 135M 的模型在半精度下会占用多少内存？[10]
- 一个 405B 的模型在半精度下会占用多少内存？[11]

---

[10] 这将占用约260 MB的空间，体积极为轻量，甚至可以在网页浏览器中本地运行。

[11] 这将达到405B × 16 ~ 800 GB的规模，至少需要两个节点，每个节点配备8块A100 GPU。
译者注：为什么乘16？应该是乘2。

---

图 6-5. 不同精度的表示方式

在训练和推理中使用全精度通常会带来最佳结果，但速度明显较慢。对于训练，人们已经找到了进行混合精度训练的方法，这可以显著加快速度。在混合精度训练中，权重以全精度作为参考，但操作以半精度进行。半精度更新用于更新全精度权重。

精度对推理的影响不大，因此我们可以以半精度加载模型。PyTorch 默认以全精度加载所有模型，因此如果我们要使用 float16 或 bfloat16，我们需要在加载模型时通过传递 torch_dtype 来指定类型：

```
model = AutoModelForCausalLM.from_pretrained("gpt2", torch_dtype=torch.float16)
```

以每个参数 16 位而不是 32 位加载 7B 模型将需要 GPU，这对于一些消费级 GPU 可能效果很好；$\sim 7\mathrm{B}$ 模型如 Llama、Mistral 和 Gemma 已成为消费级 GPU 的流行解决方案，但还有更多参数的模型更具吸引力。例如，如果我们想以半精度使用 32B 模型，我们将需要 64 GB 的 GPU，这远非任何消费级 GPU 所能及。我们有什么办法可以使用这些模型吗？

直观上，我们可以天真地减少数字的范围或精度以达到四分之一精度（使用单个字节，每个参数 8 位）。不幸的是，这样做会导致显著的性能下降。我们可以通过 8 位量化实现四分之一精度。8 位量化技术的核心思想是将一个值从一种类型（例如 fp16）映射到 int8，后者将表示 [–127, 127] 或 [0, 256] 范围内的值。

有几种不同的 8 位量化技术。让我们探索简单的 absmax 量化。给定一个向量，我们首先计算其最大绝对值。然后我们将 127（最大可能值）除以这个最大值。这导致一个量化因子——当我

们将向量乘以这个因子时，我们保证最大值将为 127。我们可以对数组进行反量化以检索原始数字，但会丢失一些信息。通过运行一些代码可以更好地理解这一点：

```
import numpy as np
def scaling_factor(vector):
    // 获取向量的最大值
    m = np.max(np.abs(vector))
    // 返回缩放因子
    return 127 / m

array = [1.2, -0.5, -4.3, 1.2, -3.1, 0.8, 2.4, 5.4, 0.3]
alpha = scaling_factor(array)
quantized_array = np.round(alpha * np.array(array)).astype(np.int8)
dequantized_array = quantized_array / alpha

print(f"Scaling factor: {alpha}")
print(f"Quantized array: {quantized_array}")
print(f"Dequantized array: {dequantized_array}")
print(f"Difference: {array - dequantized_array}")
```

缩放因子: 23.518518518518515

量化数组: [ 28 -12 -101 28 -73 19 56 127 7]
反量化数组: [ 1.19055118 -0.51023622 -4.29448819 1.19055118 -3.10393701 0.80787402 2.38110236 5.4 0.2976378 ]
差异: [ 0.00944882 0.01023622 -0.00551181 0.00944882 0.00393701 -0.00787402 0.01889764 0. 0.0023622 ]

这些差异将导致性能下降。因此，经典的量化技术在数十亿参数的模型上失败了。LLM.int8() 是一种允许我们进行 8 位量化而不降低性能的技术。该技术的核心思想是提取异常值（即超出某些界限的值），并在 FP16 中计算这些异常值的矩阵乘法，同时对其余部分使用 int8。这种混合精度结构允许我们以 8 位管理 $99.9\%$ 的值，以全精度或半精度管理 $1\%$ 的值，并且没有性能下降。

有什么缺点？LLM.int8() 的主要目标是减少运行模型推理所需的大型 GPU。由于额外的转换开销，推理速度将比使用 fp16 慢 $15-30\%$。另外需要注意的是，尽管近年来所有 GPU 都提供了 int8 的张量核心，但一些较旧的 GPU 可能对此支持不佳。

低精度推理的边界正在被新的 4 位和 2 位量化技术推动。甚至有人探索使用低于 1 位的量化。随着模型变得越来越大，实现无性能下降的量化是一个备受关注的研究领域。在本节开始时，我们需要 28 GB 的 GPU 来加载一个 7B 参数的模型。我们现在可以以 7 GB 加载相同的模型，并且没有质量下降，代价是推理速度（但不会太多）。

transformers 库集成了不同的量化方法，如 AWQ、GPTQ 以及使用 bitsandbytes 的 4 位和 8 位量化。[12]

---

[12]如需了解更多模型量化技术，建议阅读 transformers 文档中的“量化”部分。

---

以 8 位加载模型就像创建一个 BitsAndBytesConfig 并指定 load_in_8bit 一样简单。然后你可以在加载模型时将其传递给模型：

```
from transformers import BitsAndBytesConfig
quantization_config = BitsAndBytesConfig(load_in_8bit=True)
model = AutoModelForCausalLM.from_pretrained(
    "gpt2", quantization_config=quantization_config
)
```

除了量化之外，我们还可以做一些其他事情来处理非常大的模型。一种流行的推理技术称为卸载，如图 6-6 所示。如果模型太大而无法放入你的 GPU，你可以将其拆分为多个检查点分片，这些分片由 transformers 自动处理。它有什么好处？如果模型太大，我们可以只加载适合的层或分片，并将其他操作卸载到你的 CPU RAM 中，这要慢得多。这允许我们处理任何大小的模型，但代价是推理速度，这对于许多大型模型来说是不可用的。如果模型太大以至于无法放入你的 CPU RAM，你可以将模型卸载到磁盘，这更慢，但应该允许你处理任何大小的模型（只要它适合你的磁盘）。

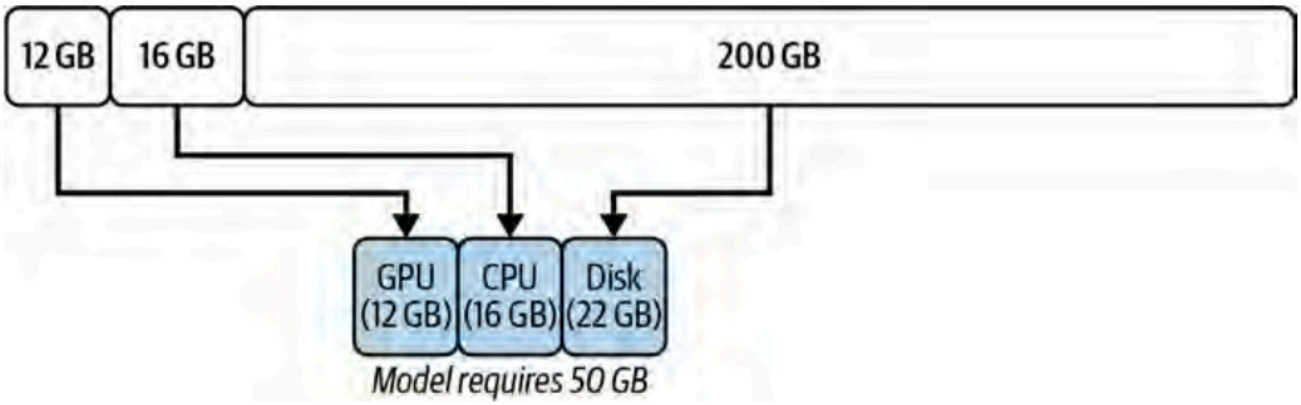



图 6-6. 模型卸载

## 整合所有内容

让我们回顾一下 PEFT 和量化：

- PEFT 允许我们通过添加适配器并冻结基础模型权重来使用更少的计算资源进行微调。由于只有少量权重可更新，这加速了训练过程。
- 量化允许我们使用比存储时更少的位数加载模型。这减少了加载模型并运行推理所需的 GPU 资源。

为什么不两者兼得呢？假设我们以 8 位精度训练一个模型。不幸的是，正如前一节所讨论的，在预训练或微调大型模型时，高精度非常重要。另一方面，PEFT 冻结了基础模型并仅使用一个小型适配器，因此我们可以在这里使用较低的精度，同时保持相同的性能。

QLoRA 允许我们使用较小的 GPU 微调大型模型。这种技术与 LoRA 非常相似，但加入了量化。首先，基础模型被量化为 4 位并冻结。然后，添加 LoRA 适配器（两个矩阵）并将其保持在 bfloat16 精度。在微调时，QLoRA 使用 4 位存储的基础模型和半精度 16 位模型进行计算。

以 4 位加载模型只需在创建 BitsAndBytesConfig 时更改为 load_in_4bit 参数。让我们用 Mistral 7B 来尝试一下，这是一个非常可靠的基础模型选择。由于这个模型比之前的模型大得多，我们还将指定 device_map="auto"。它会自动尝试填充 GPU 的所有空间，如果模型无法完全放入 GPU，则将权重卸载到 CPU（运行速度会慢得多，但模型仍会加载）。

在使用 Mistral 之前的最后一个细节：这个模型仓库是受限制的，这意味着作者需要明确同意其许可条款。要使用它，你必须在登录 Hugging Face 账户的情况下访问模型页面，阅读许可协议，并在同意条款后点击接受按钮。要以编程方式访问模型，如下面的代码片段所示，你必须使用相同的账户进行身份验证。最简单的方法是安装 huggingface_hub Python 包（它随

transformers 一起提供），并在终端会话中运行 huggingface-cli login。系统会要求你输入一个访问令牌，你可以在设置页面创建该令牌。如果你是从 Google Colab 会话下载模型，可以设置 HF_TOKEN 密钥并授予笔记本使用它的权限。

```
quantization_config = BitsAndBytesConfig(load_in_4bit=True)

model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.3",
    quantization_config=quantization_config,
    device_map="auto",
)
```

`BitsAndBytesConfig` 允许通过使用额外的参数来更精细地控制量化技术，例如更改计算类型、应用嵌套量化等。

QLoRA 只是我们工具箱中的一个工具，而不是万能解决方案。它在保持相同性能的同时显著减少了 GPU 需求，但也增加了训练模型所需的时间。PEFT 部分的所有优势仍然存在，使得 QLoRA 成为社区中快速微调 7B 模型的流行技术。

让我们进行一次 QLoRA 微调，以创建一个能够进行简单对话的生成模型。让我们逐一介绍每个组件：

**基础模型**

我们将使用 Mistral 模型。Mistral 是一个非常高质量的 7B 模型。我们使用 `load_in_4bit` 和 `device_map="auto"` 加载模型以进行 4 位量化。

**数据集**

我们将使用 Guanaco 数据集，其中包含 10,000 条人类与 OpenAssistant 模型之间的高质量对话。

**PEFT 配置**

我们将指定一个具有良好初始默认值的 `LoraConfig`：秩（r）为 8，alpha 为其值的两倍。

**训练参数**

和之前一样，我们可以配置训练参数（例如评估频率和训练轮数）以及模型超参数（学习率、权

重衰减或训练轮数）。

在之前的示例中，我们使用了 `TrainingArguments` 和 `Trainer`，这是 `transformers` 中的两个通用工具。在为自回归技术微调 LLM 时，`trl` 库的 `SFTConfig` 和 `SFTTrainer` 类是非常有用的工具。它们是围绕 `TrainingArguments` 和 `Trainer` 的封装，针对文本生成进行了优化。它们的功能包括：

- 易于使用的数据集加载和处理工具。我们无需自己处理数据集，而是可以使用 `dataset_text_field` 来指定包含训练数据的字段。此外，我们可以使用 `packing` 来连接多个序列，这对于高效的批处理非常有用。
- 支持开箱即用的常见对话和指令提示模板。
- 你可以直接将任何 `PeftConfig` 传递给 `SFTTrainer` 以使用 PEFT 技术。

和之前一样，我们可以传递现在已量化的模型和数据集（我们将仅传递 300 个样本以进行快速训练）。`SFTTrainer` 已经带有有用的默认整理器和数据集工具，因此无需对数据进行标记化和预处理：[13]

```python
from trl import SFTConfig, SFTTrainer

dataset = load_dataset("timdettmers/openassistant-guanaco", split="train")

peft_config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
)

sft_config = SFTConfig(
    "fine_tune_e2e",
    push_to_hub=True,
    per_device_train_batch_size=8,
    weight_decay=0.1,
    lr_scheduler_type="cosine",
    learning_rate=5e-4,
    num_train_epochs=2,
    eval_strategy="steps",
    eval_steps=200,
    logging_steps=200,
    gradient_checkpointing=True,
    max_seq_length=512,
    # New parameters
    dataset_text_field="text",
    packing=True,
)

trainer = SFTTrainer(
    model,
    args=sft_config,
    train_dataset=dataset.select(range(300)),
    peft_config=peft_config,
)

trainer.train()
trainer.push_to_hub()
```

前面的代码可能需要一个小时或更长时间才能运行。请记住，QLoRA 也会导致训练速度变慢。

在模型训练时，这是一个阅读更多关于数据集的好机会。如果你访问数据集页面，你会注意到它有以下格式：

```
### Human: Can you write a short introduction ....### Assistant: "Monopsony"
refers to a market ..### Human: Now explain it to a dog
```

- 每轮对话以 ### Human: 开头，后接一个空格，然后是用户的输入内容。
- 模型的回复以 ### Assistant: 开头，后接一个空格，接着是模型的输出内容。
- 对话可以包含多轮交互。

当你为对话任务微调模型时，通常会有一个聊天模板。所有细节都很重要。在聊天中添加新行、删除空格或添加额外的 `#` 都可能会降低模型的生成质量。这些期望来自于训练期间使用的训练格式。同样，如果模型仅使用单轮对话进行训练，它将难以生成高质量的多轮对话。了解提示格式很重要，因为我们需要使用它来生成高质量的对话。

一旦训练完成，让我们继续使用模型。当我们推送模型时，我们只推送了适配器。让我们使用模型和适配器运行推理：

```
# We load the base model just as before
tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.3")
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.3",
    torch_dtype=torch.float16,
    device_map="auto",
)

# You can load the adapter with `load_adapter`
model.load_adapter("genaibook/fine_tune_e2e")  # change with your adapter name

# Alternatively, you could just use `from_pretrained` with the adapter name and it
# will automatically take care of loading the base and adapter models.
# model = AutoModelForCausalLM.from_pretrained("genaibook/fine_tune_e2e"...

pipe = pipeline("text-generation", model=model, tokenizer=tokenizer)
pipe("### Human: Hello!### Assistant:", max_new_tokens=100)
```

前面的代码可能会输出如下内容：

```
### Human: Hello
### Assistant: Hello! How can I help you?

### Human: I want to know how to make a website.
### Assistant: Sure! Here are some steps to help you get started...
```

为了可读性，我们在人类和助手之间添加了一个新行，并在每一轮之间添加了一个额外的行。

令人印象深刻，我们刚刚微调了一个 7B 模型，使其能够进行对话，而无需巨大的 GPU。

随着对话模型变得越来越普遍，Hugging Face `transformers` 添加了一种方式，让模型创建者可以指定 `chat_template`。多亏了这一点，最终用户无需过多担心提示模板，而是可以专注于对话内容。例如，你可以简单地将消息传递给模型，标记器会自动处理格式化：

```
pipe = pipeline(
    "text-generation", "HuggingFaceTB/SmolLM-135M-Instruct", device=device
)
messages = [
    {
        "role": "system",
        "content": """You are a friendly chatbot who always responds
        in the style of a pirate""",
    },
    {
        "role": "user",
        "content": "How many helicopters can a human eat in one sitting?",
    },
]
print(pipe(messages, max_new_tokens=128)[0]["generated_text"][-1])
```

```
{'content': 'The number of helicopters that can be eaten in one '
            'sitting depends on the number of people in the room. If '
            'there are 10 people, then there are 10 helicopters that '
            'can be eaten in one sitting. If there are 15 people, '
            'then there are 15 helicopters that can be eaten in one '
            'sitting. If there are 20 people, then there are 20 '
            'helicopters that can be eaten in one sitting.\n'
            '\n'
            'The number of helicopters that can be eaten in one '
            'sitting depends on the number of people in the room. If '
            'there are 10 people, then there are 10 helicopters',
'role': 'assistant'}
```

如果你只想应用聊天模板但不将其传递给模型，你可以直接使用 `tokenizer.apply_chat_template()` 方法：

```python
tokenizer = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM-135M-Instruct")

chat = [
    {"role": "user", "content": "Hello, how are you?"},
    {
        "role": "assistant",
        "content": "I'm doing great. How can I help you today?",
    },
    {
        "role": "user",
        "content": "I'd like to show off how chat templating works!",
    },
]

tokenizer.apply_chat_template(chat, tokenize=False)
```

我们可以使用 `print()` 来获取完整的提示，但要注意传递给模型的原始字符串包含诸如 `\n` 这样的字符来标记新行：

```python
print(tokenizer.apply_chat_template(chat, tokenize=False))
```

```
<|im_start|>user
Hello, how are you?<|im_end|>
<|im_start|>assistant
I'm doing great. How can I help you today?<|im_end|>
<|im_start|>user
I'd like to show off how chat templating works!<|im_end|>
```

这对于在 `tokenizer_config.json` 中指定了 `chat_template` 的模型是有效的。要了解更多关于聊天模板的信息，我们建议阅读官方文档。

# 深入探讨评估

我们如何评估生成文本的质量？到目前为止，我们讨论了评估通用知识和推理能力的流行基准，但你可能想知道如何在更广泛的背景下评估生成文本的质量。首先要区分的是基础模型与微调或聊天模型的评估。对它们的期望是不同的，因此我们不应该以完全相同的方式评估它们。例如，我们不应该期望基础模型开箱即具备指令或聊天能力，因此在这些任务上评估它是不公平的。

让我们开始讨论一些评估基础模型的方法：

### 困惑度（Perplexity）

困惑度衡量语言模型对给定数据集的预测能力。较低的困惑度值表示性能更好，生成文本时的不确定性更小，表明模型可以更准确地预测下一个词。困惑度在基础模型的训练阶段尤为重要，因为它反映了模型学习有效词序列概率分布的能力。

### BLEU

BLEU 衡量生成文本与参考文本之间的相似性。它通过计算生成文本中与参考文本中相同的 n-gram 比例来实现。由于 BLEU 严重依赖精确的 n-gram 匹配，它可能无法捕捉自然语言的多样性，并且缺乏语义理解。

### ROUGE

与 BLEU 类似，ROUGE 衡量两个文本之间的重叠。然而，ROUGE 更注重召回率而非精确率，因此它在摘要等任务中非常有用。不过，它仍然缺乏语义理解，并且倾向于偏向较长的输出。

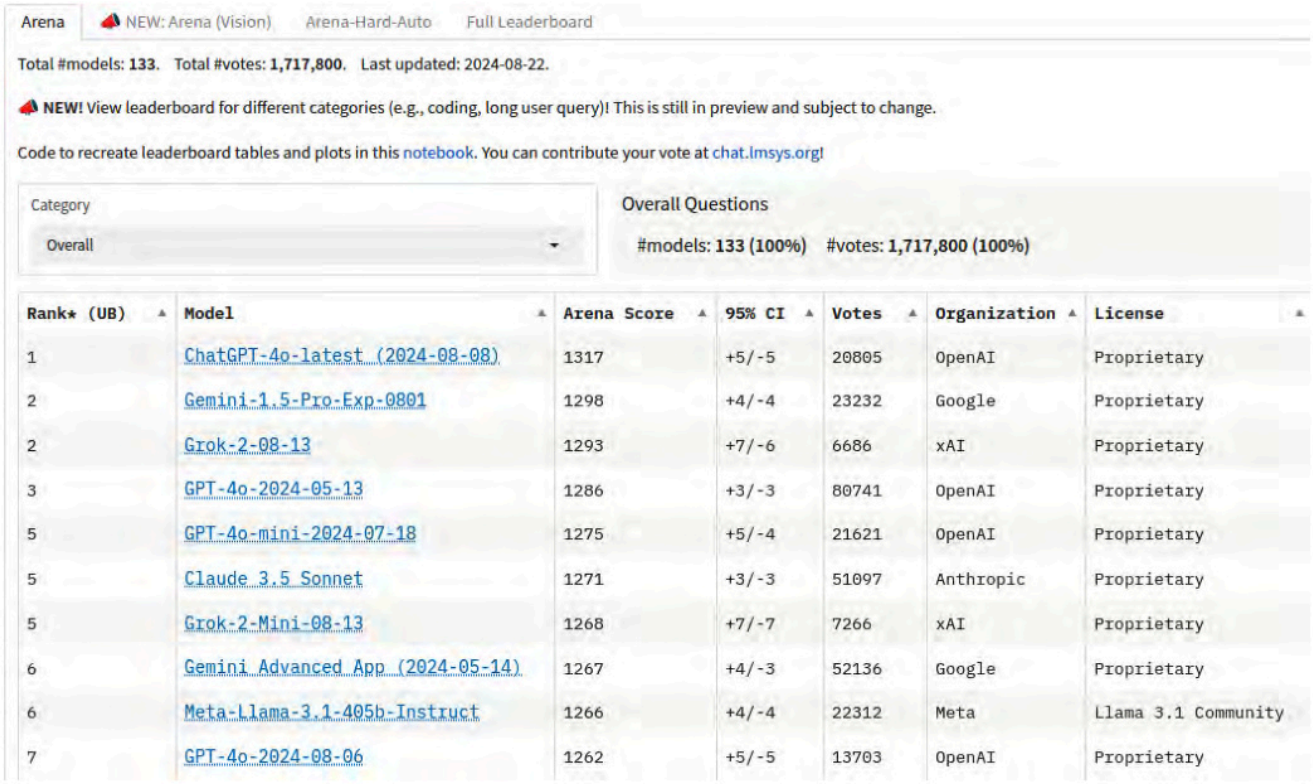
正如你所见，评估基础模型并不简单。在训练过程中，通常会跟踪损失和困惑度，期望两者随着时间的推移而减少。ROUGE 和 BLEU 通常用于具有参考文本的数据集。然而，这两种指标都

依赖于精确匹配，忽略了语义相似性，因此在评估更具创造性或多样性的文本生成时存在局限性。

> 最近的一项研究 Urial 指出，我们在指令微调中看到的大部分收益实际上来自基础模型。作者分析了基础模型和指令微调模型之间的 token 分布变化，发现大多数 token 的变化非常小。发生的变化主要是风格上的，涉及诸如问候语和免责声明等 token，这些在对话模型中很常见。这一发现表明，基础模型已经具备了遵循指令所需的大部分知识，突显了预训练的重要性。此外，无需微调的方法也有可能达到与指令微调相似的性能，尽管这一领域仍在积极研究中。

虽然这些指标提供了定量衡量，但定性评估（包括人类判断）对于评估生成文本的整体连贯性和与预期任务的相关性也至关重要。定量指标在大规模比较中很有用，但它们可能会忽略流畅性、创造性或上下文适当性等细微差别。这就是人类评估的优势所在。平衡定量和定性评估可以确保对文本生成模型进行更全面的评估。

对于最终用户的生成模型（如聊天模型），最好的方法之一是与模型互动。还有一些流行的竞技场，如 LMSYS，用户可以在其中与不同的匿名模型互动并选择最佳结果。然后，这些结果会被汇总到一个排行榜中，如图 6-7 所示。这些竞技场比自动化排行榜更好，因为它们反映了更接近实际使用场景的性能。不幸的是，获取竞技场分数成本高昂，很多时候对于全新模型来说是不可能的。在这种情况下，诸如 MT Bench、IFEval、EQ Bench 和 AGIEval 等基准与竞技场分数有一定的相关性，因此可以用来粗略估计模型在现实世界中的表现。新的基准不断涌现，因此及时了解最新研究非常重要。



Arena | NEW: Arena (Vision) | Arena-Hard-Auto | Full Leaderboard

Total #models: **133**. Total #votes: **1,717,800**. Last updated: 2024-08-22.

**NEW!** View leaderboard for different categories (e.g., coding, long user query)! This is still in preview and subject to change.

Code to recreate leaderboard tables and plots in this notebook. You can contribute your vote at chat.lmsys.org!

Category: Overall

Overall Questions: #models: **133 (100%)** #votes: **1,717,800 (100%)**

| Rank* (UB) | Model | Arena Score | 95% CI | Votes | Organization | License |
|---|---|---|---|---|---|---|
| 1 | ChatGPT-4o-latest (2024-08-08) | 1317 | +5/-5 | 20805 | OpenAI | Proprietary |
| 2 | Gemini-1.5-Pro-Exp-0801 | 1298 | +4/-4 | 23232 | Google | Proprietary |
| 2 | Grok-2-08-13 | 1293 | +7/-6 | 6686 | xAI | Proprietary |
| 3 | GPT-4o-2024-05-13 | 1286 | +3/-3 | 80741 | OpenAI | Proprietary |
| 5 | GPT-4o-mini-2024-07-18 | 1275 | +5/-4 | 21621 | OpenAI | Proprietary |
| 5 | Claude 3.5 Sonnet | 1271 | +3/-3 | 51097 | Anthropic | Proprietary |
| 5 | Grok-2-Mini-08-13 | 1268 | +7/-7 | 7266 | xAI | Proprietary |
| 6 | Gemini Advanced App (2024-05-14) | 1267 | +4/-3 | 52136 | Google | Proprietary |
| 6 | Meta-Llama-3.1-405b-Instruct | 1266 | +4/-4 | 22312 | Meta | Llama 3.1 Community |
| 7 | GPT-4o-2024-08-06 | 1262 | +5/-5 | 13703 | OpenAI | Proprietary |

图 6-7. LMSYS 排行榜

基准虽然是有价值的工具，但也有其局限性。例如，基于知识的基准通常表现出以美国为中心的偏见，包含关于美国历史和法律的题目，如 MMLU 中所见。此外，大多数这些基准都是基于英语的，其他语言的基准很少。一些社区正在努力翻译这些基准，但进展缓慢，翻译可能并不完美。此外，聊天基准（通常也包括竞技场）往往更关注单轮对话而非多轮对话。评估非常长的上下文模型仍然是一个挑战，特别是在聊天场景中，这是一个尚未解决的领域。

最重要的结论是，在你想要解决的任务上测试模型。基准对于选择用于微调或通用聊天模型的初始基础模型很有用，但它们不能替代实际测试。

# 项目时间：检索增强生成（RAG）

LLM 只能使用基于其上下文和训练数据的信息。如果你想向 LLM 询问某个特定主题的信息，它只有在这些信息是其数据的一部分时才能知道答案。例如，如果你尝试询问 Llama 关于新电影的信息，它可能难以提供准确的答案。

检索增强生成（RAG）是一种技术，模型可以通过它访问存储在某个地方的信息（例如段落或文档）。通过 RAG，LLM 使用用户输入和存储的信息来生成响应。这种方法非常强大，因为它允许模型访问大量信息，这使得更新信息比重新训练模型更容易。

不幸的是，可能有数百万份文档，因此你不能简单地将所有文档传递给模型。为了解决这个问题，我们使用嵌入模型（如第 2 章中的句子转换器）将每个文档编码为向量，并将这些向量存储（通常存储到称为向量数据库的地方）。然后，我们使用最近邻搜索来找到与用户输入最相似的文档。最后，我们将用户输入和检索到的文档传递给 LLM。这种方法非常吸引人，因为它允许模型根据需要访问大量信息。

你的目标是构建一个如图 6-8 所示的管道，其中：

1. 用户输入一个问题。
2. 管道检索与问题最相似的文档。
3. 管道将问题和检索到的文档传递给 LLM。
4. 管道生成响应。

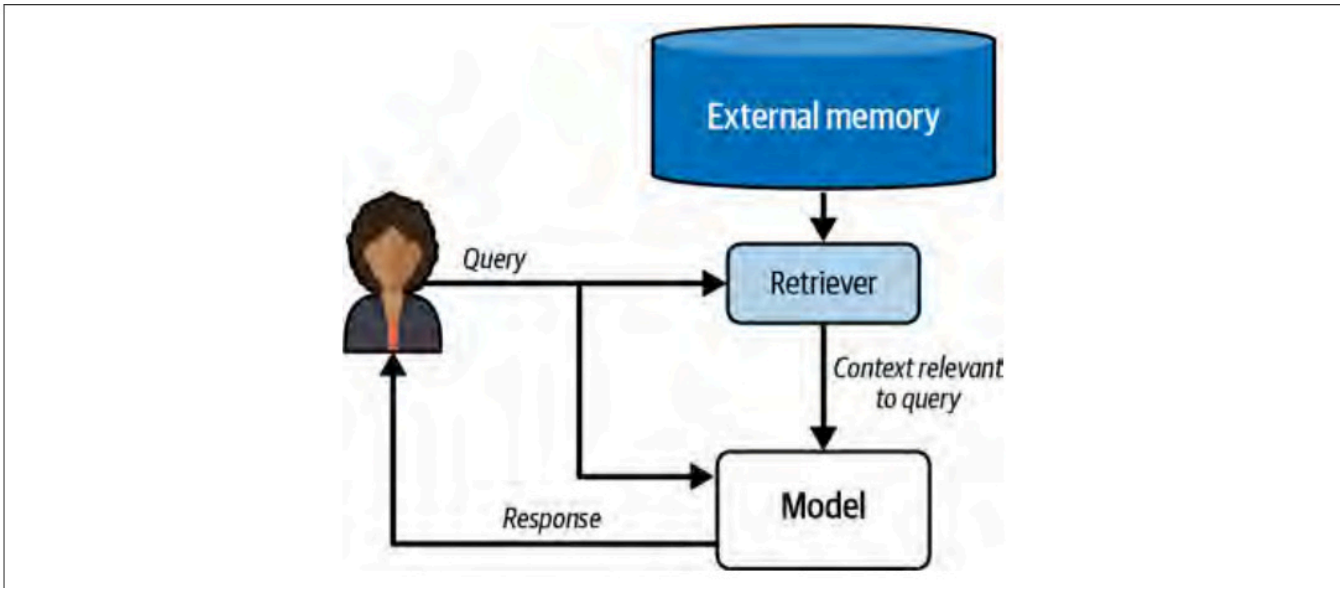



图 6-8. RAG 管道

你不需要为此任务训练任何模型。对于检索，我们建议使用 sentence_transformers 预训练模型。你可以自由选择你喜欢的模型（例如 Mistral 或 Llama）进行生成。请注意，问题 1 和 2 已在第 2 章的挑战 3 中解决，问题 3 和 4 已在本章中解决。目标是将所有这些部分整合在一起。你可以使用以下函数来指导你：

```python
def embed_documents(documents: List[str]):
    # 使用句子转换器模型编码文档
    # 将文档存储到某个地方

def retrieve_documents(query: str):
    # 使用存储的文档检索
    # 与查询最相似的文档

def generate_response(query: str, documents: List[str]):
    # 使用 LLM 生成响应

def pipeline(query: str):
    documents = retrieve_documents(query)
    response = generate_response(query, documents)
    return response
```

使用哪些文档？这取决于你，但我们建议从一个非常简单的设置开始（即选择 5-10 个句子或段落，可能是你自己编写的），然后你可以扩展到更多文档。

> 附录 C 展示了一个端到端的示例，教你如何构建一个最小化的 RAG 管道。我们建议先尝试自己构建管道，然后查看附录以查看完整示例。

# 总结

本章探讨了微调大语言模型（LLMs）的技术。我们首先讨论了传统的编码器模型微调方法，用于文本分类任务。然而，这种方法也可以用于其他任务，例如从给定文本中回答问题或识别文本中的实体。接着，我们探讨了如何微调解码器模型以进行文本生成。我们讨论了微调与零样本或少样本生成的优势和局限性。我们还研究了监督微调如何使生成模型能够开箱即用地解决多个任务。

尽管这些技术非常强大，但将其扩展到最新、越来越大的模型时仍面临挑战。为了解决这个问

题，我们探索了使用量化技术，在较小的GPU上运行大型模型的推理，并讨论了参数高效微调（PEFT）技术，以减少计算和磁盘空间的需求。通过结合这些技术，我们成功微调了一个7B模型，使其能够进行对话。有了这些基础，您可以为特定任务微调大型模型。

虽然本章主要关注模型架构和微调技术，但重要的是要记住，这些模型的成功还取决于训练数据的质量和多样性。需要多少数据并不显而易见：这取决于模型大小、任务复杂性和数据质量。几百个高质量的训练样本通常比数千个低质量的样本更有效。

如需进一步阅读，我们建议以下资源：

- 要了解更多关于数据的内容，我们推荐FineWeb数据集的博客文章，这是一个关于15万亿token数据集的全面介绍，包括对预处理的深入调查、如何创建高质量网络数据集的解释以及如何创建自动注释。
- 关于评估，我们建议阅读《Let's Talk About LLM Evaluation》，它提供了模型评估及其挑战的高层次概述。我们还推荐阅读Open LLM Leaderboard的博客文章，它全面介绍了以社区为中心的模型评估。最后，Eugene Yan写了一篇关于该主题的优秀博客文章。
- 要了解更多关于LoRA的内容，我们建议阅读《Practical Tips for Finetuning LLMs Using LoRA》以及QLoRA发布的博客文章。
- 要了解更多关于量化的内容，我们推荐这篇量化的视觉指南以及Jay Alammar和Maarten Grootendorst的《Hands-On Large Language Models》（O'Reilly出版）。
- 要了解更多关于生产环境中LLM设置的所有组件，我们建议阅读《Building a Generative AI Platform》的博客文章。

# 练习

1. 基础模型和微调模型有什么区别？对话模型属于哪种模型？
2. 在哪些情况下你会选择基础编码器模型进行微调？
3. 解释微调、指令微调和QLoRA之间的区别。
4. 使用适配器会导致模型体积变大吗？
5. 加载一个70B模型需要多少GPU内存，分别在半精度、8-bit量化和4-bit量化的情况下？
6. 为什么QLoRA会导致训练速度变慢？
7. 在哪些情况下我们会在微调期间冻结模型权重？

你可以在本书的GitHub仓库中找到这些练习的答案以及以下挑战的解决方案。

# 挑战

图像分类。尽管本章主要关注微调Transformer模型以完成NLP任务，但Transformer也可以用于其他模态，如音频和计算机视觉。本挑战的目标是微调一个Transformer模型以进行图像分类。我们建议以下内容：

- 使用预训练的ViT模型，例如google/vit-base-patch16-224-in21k。
- 使用一个包含图像和标签的数据集，例如food101。

逻辑几乎相同，但有一些关键区别，例如使用AutoImageProcessor而不是AutoTokenizer。我们建议查阅文档以指导你完成该过程。

参考文献


Abdin, Marah, 等. “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone.” arXiv, 2024年8月30日. https://arxiv.org/abs/2404.14219.

Belkada, Younes, 和 Tim Dettmers. “A Gentle Introduction to 8-bit Matrix Multiplication for Transformers at Scale Using Hugging Face Transformers, Accelerate and bitsandbytes.” Hugging Face博客, 2022年8月17日. https://oreil.ly/FYVTE.

Belkada, Younes, 等. “Making LLMs Even More Accessible with bitsandbytes, 4-bit Quantization and QLoRA.” Hugging Face博客, 2023年5月24日. https://oreil.ly/CbB9a.

Chung, Hyung Won, 等. “Scaling Instruction-Finetuned Language Models.” arXiv, 2022年10月20日. https://arxiv.org/abs/2210.11416.

Dettmers, Tim, 等. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.” arXiv, 2022年8月15日. https://arxiv.org/abs/2208.07339.

Dubey, Abhimanyu, 等. “The Llama 3 Herd of Models.” arXiv, 2024年8月15日. https://arxiv.org/abs/2407.21783.

Honovich, Or, 等. “Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor.” arXiv, 2022年12月19日. https://arxiv.org/abs/2212.09689.

Kocetkov, Denis, 等. “The Stack: 3 TB of Permissively Licensed Source Code.” arXiv, 2022年11月20日. https://arxiv.org/abs/2211.15533.

Lester, Brian, 等. “The Power of Scale for Parameter-Efficient Prompt Tuning.” arXiv, 2021年9月2日. https://arxiv.org/abs/2104.08691.

Li, Xiang Lisa, 和 Percy Liang. “Prefix-Tuning: Optimizing Continuous Prompts for Generation.” arXiv, 2021年1月1日. https://arxiv.org/abs/2101.00190.

Lin, Bill Yuchen, 等. “The Unlocking Spell on Base LLMs: Rethinking Alignment via In-Context Learning.” arXiv, 2023年12月4日. https://arxiv.org/abs/2312.01552.

Liu, Xiao, 等. “GPT Understands, Too.” arXiv, 2023年10月25日. https://arxiv.org/abs/

2103.10385.
Mishra, Swaroop, 等. “Cross-Task Generalization via Natural Language Crowdsourcing Instructions.” arXiv, 2022年3月14日. https://arxiv.org/abs/2104.08773.
Sanh, Victor, 等. “DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter.” arXiv, 2020年3月1日. https://arxiv.org/abs/1910.01108.
Sanh, Victor, 等. “Multitask Prompted Training Enables Zero-Shot Task Generalization.” arXiv, 2022年3月17日, https://arxiv.org/abs/2110.08207.
Wang, Yizhong, 等. “Self-Instruct: Aligning Language Models with Self-Generated Instructions.” arXiv, 2023年5月25日. https://arxiv.org/abs/2212.10560.
Wang, Yizhong, 等. “Super-NaturalInstructions: Generalization via Declarative Instructions on $1600+$ NLP Tasks.” arXiv, 2022年10月24日. https://arxiv.org/abs/2204.07705.
Wei, Jason, 等. “Finetuned Language Models Are Zero-Shot Learners.” arXiv, 2022年2月8日. https://arxiv.org/abs/2109.01652.
Ye, Qinyuan, 和 Xiang Ren. “Learning to Generate Task-Specific Adapters from Task Description.” arXiv, 2021年6月15日. https://arxiv.org/abs/2101.00420.
Zhou, Chunting, 等. “LIMA: Less Is More for Alignment.” arXiv, 2023年5月18日. https://arxiv.org/abs/2305.11206.

# 第7章. 微调Stable Diffusion

在上一章中，我们介绍了如何通过微调教会语言模型以特定风格写作或学习特定领域的概念。我们可以将相同的原则应用于文本到图像模型，使我们能够定制模型，即使只使用单个GPU（与预训练Stable Diffusion等模型所需的多GPU节点相比）。

在本章中，我们将使用你在第5章中学到的基础预训练Stable Diffusion模型，并将其扩展以学习它可能不了解的风格和概念，例如“你的宠物”的概念或特定的绘画风格。我们还将学习如何赋予它新的能力，例如修复图像和提供新的条件作为输入。

在这里，我们不会从头开始编写代码，而是将重点放在理解和运行现有的脚本上，这些脚本是为微调模型而创建的。为此，我们建议你克隆`diffusers`库，因为大多数示例都位于该库的`examples`文件夹中：

```
git clone https://github.com/huggingface/diffusers.git
```

# 完整的Stable Diffusion微调

完整模型是在特定模型定制技术（如LoRA、Textual Inversion和DreamBooth）发展之后出现的微调限定词。这些技术并不完全微调整个模型，而是提供了一种高效的微调方式（正如我们在第6章中学习的LLM的LoRA）或提供了“教”模型新概念的新方法。我们将在本章进一步讨论这些技术。

在这些技术出现之前，完整模型这样的限定词并不存在，因为那时它只被称为微调。在这种上下文中的微调意味着进一步训练扩散模型——正如你在第3章和第4章中学到的那样——但目标是将模型引导到你想要添加的特定知识上。你可以让Stable Diffusion学习一种无法通过提示获得的风格或主题，或者是在模型发布后才发明的风格或主题（见图7-1）。正如完整模型这个限定词可能暗示的那样，一旦模型被微调，它将变得擅长你引入的风格或主题，并且可能专门生成主要那种类型的内容。本节将使用一个预制的脚本来执行完整微调。我们将使用diffusers库中的脚本`diffusers/examples/text_to_image/train_text_to_image.py`。

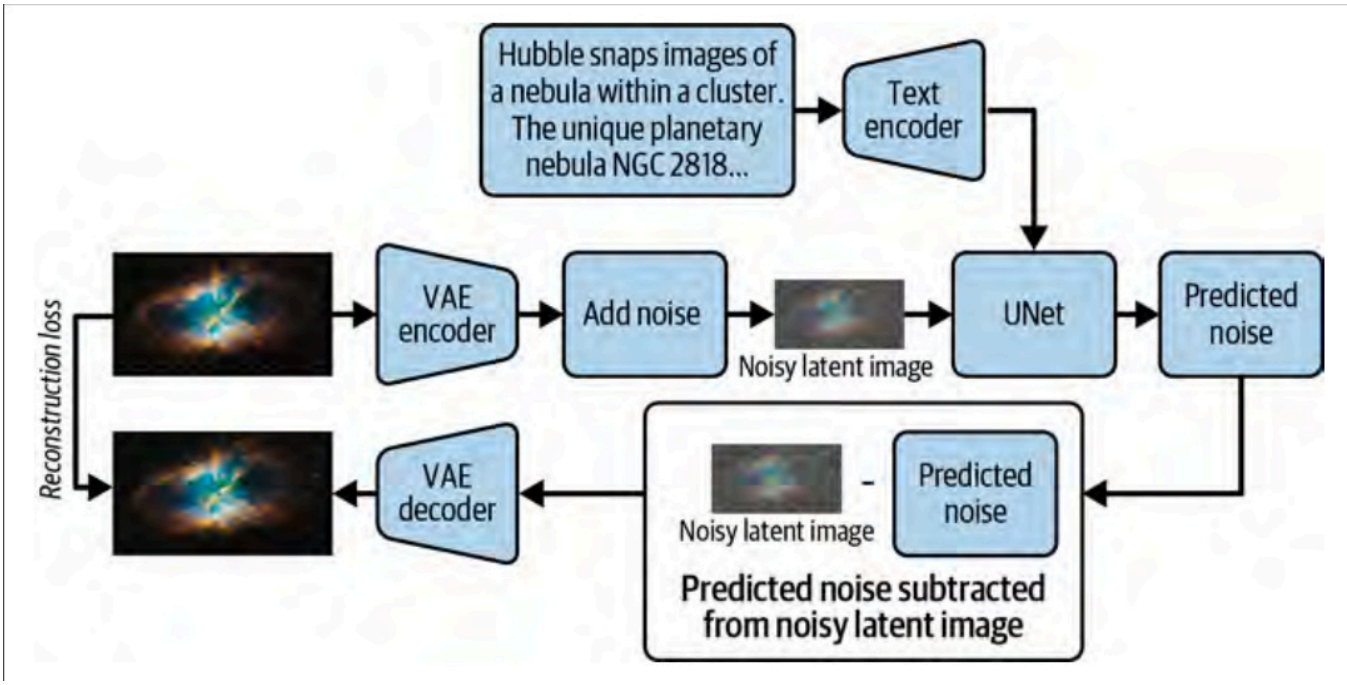



图7-1. Stable Diffusion微调架构

## 准备数据集

数据集中最重要的部分是质量。过滤数据集以保留高质量样本并删除低质量样本可以显著影响微调的质量。

高质量的完整模型微调可能需要一个相对较大的数据集，包含500+张图像。虽然这听起来可能

很多，但与训练整个Stable Diffusion模型所需的数十亿张图像相比，拥有数百张图像的数据集仍然只是很小的一部分。在本章我们将进一步讨论的特定模型定制技术中，我们将学习如何用少至四张图像来定制模型。

回到完整模型微调。由于我们引导的是一个文本到图像模型，我们必须向它展示一个包含图像及其相应描述的数据集，就像模型在预训练期间看到的那样。

如果你需要灵感，这里有一些例子：

- 文艺复兴时期的绘画，用于制作文艺复兴风格的微调
- 你最喜欢的建筑风格的建筑图片，用于制作建筑模型
- 一组风景照片，用于微调模型以生成包含宁静森林、雄伟山脉和宁静湖岸的现实风景场景

> 虽然我们鼓励探索，但除了学习和教育目的之外的微调和部署需要谨慎。如果你使用的图像是某位艺术家的风格、某个人的面孔或任何拥有知识产权（IP）的材料，询问作者或IP持有者你是否可以这样做不仅是一种礼貌，而且在你的司法管辖区可能也是法律要求。

在我们的示例中，我们将使用哈勃望远镜图像，这些图像由NASA在拍摄后六个月放入公共领域。创建哈勃望远镜数据集并在哈勃图像上微调Stable Diffusion是由德克萨斯大学达拉斯分校的研究员Maxwell Weinzierl开创的，他制作了esa-hubble数据集和哈勃Diffusion 1和2模型。在本例中，我们的目标是通过使用esa-hubble数据集微调Stable Diffusion v1.5来重新创建哈勃Diffusion 1模型。

esa-hubble数据集是通过从欧洲航天局网站爬取哈勃望远镜拍摄的图像创建的。幸运的是，描述这些图像中天文现象的标题也可用，因此两者都可以保存并放入与datasets库兼容的格式中，然后用于微调。如何收集数据（通过网络爬取或其他方式）超出了本书的范围，但你可以查看Scrapy或Beautiful Soup等工具以在Python中进行网络爬取。请注意每个网站关于爬取或抓取的政策。

一旦你有了一个包含图像-文本对的数据集，你可以将它们加载到datasets库中。以下部分将向你展示如何做到这一点。如果你没有自己的图像数据集，你可以继续使用提供的esa-hubble数据

集：

```
from datasets import load_dataset
dataset = load_dataset("imagefolder", data_dir="/path/to/folder")
```

`imagefolder`是`load_dataset`的一种特殊模式，允许加载一个图像目录和一个包含每个图像标题的元数据文件。此模式要求你将所有图像放在`/path/to/folder`中，并有一个`metadata.csv`文件，其中包含每个图像的标题。因此，你的文件夹可以具有以下结构：

folder/metadata.csv
folder/0001.png
folder/0002.png
folder/0003.png

而`metadata.csv`文件应如下所示：

file_name,text
0001.png,这是一只金毛猎犬在玩球
0002.png,一只德国牧羊犬
0003.png,一只吉娃娃

一旦你的数据集加载完成，你可以通过一个简单的命令将其推送到Hugging Face Hub与社区分享：

```
dataset.push_to_hub("my-hf-username/my-incredible-dataset")
```

这样你就保存了数据集并准备好微调模型了！在某些情况下，你有一个完美的图像数据集，但尚未为其制作人工标题。在这些情况下，你可以使用图像到文本的标题生成模型来创建标题，然后用于微调模型。Salesforce的BLIP-2或Microsoft的Florence 2等模型广泛用于此任务。要了解更多信息，你可以查看“图像到文本”任务页面。如果你没有数据集，不用担心；你可以使用提供的esa-hubble数据集。

## 微调模型

为了微调模型，我们需要一个数据集，正如前一节所讨论的，还需要一个训练脚本以及我们将要微调的预训练模型的权重。幸运的是，我们可以轻松地使用`diffusers`库来设置这些内容，该库提供了示例训练脚本，以及`accelerate`库（用于高效的训练过程，允许 PyTorch 的训练循环

在多个 GPU、TPU 或不同的精度（如 BF16）上运行）。这个脚本非常有用，因为它包含了高效训练 Stable Diffusion 的 UNet 所需的所有代码，同时通过暴露的超参数为用户提供了控制权，我们将在后面探讨这些超参数。

你需要一个至少具有 16GB 显存的 GPU 来执行此微调，或者使用诸如 Google Colab Pro 之类的服务。我们将在本章中学习的自定义技术允许在更普通的 GPU 或 Google Colab 的免费版本上进行训练。

你可以使用自己的数据集，或者使用 Maxwell Weinzierl 提供的 `esa-hubble` 数据集来跟随并复现 Hubble Diffusion。

要开始微调模型，首先需要从 `diffusers` 仓库克隆必要的训练脚本：

git clone https://github.com/huggingface/diffusers.git
cd diffusers/examples/text_to_image/

然后，使用 `train_text_to_image.py` 脚本。对于显存受限的设置，我们将使用 `use_8bit_adam`，这将需要 `bitsandbytes` 和 `gradient_accumulation_steps` 来使用比通常适合 GPU 内存的更大的批量大小：

```
accelerate launch train_text_to_image.py \
--pretrained_model_name_or_path="stable-diffusion-v1-5/stable-diffusion-v1-5" \
--dataset_name="Supermaxman/esa-hubble" \
--use_ema \
--mixed_precision="fp16" \
--resolution=512 \
--center_crop \
--random_flip \
--train_batch_size=1 \
--gradient_checkpointing \
--gradient_accumulation_steps=4 \
--use_8bit_adam \
--checkpointing_steps=1000 \
--num_train_epochs=50 \
--validation_prompts \
    "Hubble image of a colorful ringed nebula: \
A new vibrant ring-shaped nebula was imaged by the \
NASA/ESA Hubble Space Telescope." \
    "Pink-tinted plumes in the Large Magellanic Cloud: \
The aggressively pink plumes seen in this image are extremely uncommon, \
with purple-tinted currents and nebulous strands reaching out into \
the surrounding space." \
--validation_epochs 5 \
--learning_rate=1e-05 \
--output_dir="sd-hubble-model" \
--push_to_hub
```

在我们深入探讨这些参数之前，我们建议你在阅读本章的同时运行该脚本，因为训练将需要一些时间（一到三个小时，具体取决于你的 GPU）。然后在你阅读完整章后进一步迭代超参数。

详细解释 `train_text_to_image.py` 脚本的工作原理超出了本书的范围；然而，其基本概念与你在第 115 页的“训练扩散模型”中学到的相同，我们涵盖了训练扩散模型的核心概念。关键步骤包括准备图像数据集、定义噪声调度以及创建一个 UNet 模型来预测噪声。训练循环迭代地向干净图像添加噪声，让模型预测该噪声，计算预测噪声与实际噪声之间的损失，并通过反向传播更新模型权重。在这里，我们将了解关键的超参数（这些是你在开始微调模型之前设置的参数），它们仍然非常重要；我们将逐一介绍前面训练脚本中的每个设置。

从这个设置中最重要的概念是 `learning_rate` 和 `num_train_epochs`：

**learning_rate**

表示每个训练步骤中模型权重更新的量。如果你选择较高的学习率，微调模型的优化过程可能无法稳定，而太低的值可能导致欠拟合，你的模型可能永远无法学习。我们建议在 1e-04 (0.0001) 和 1e-06 (0.000001) 之间进行实验。

**num_train_epochs**

表示你的模型将遍历整个数据集的次数。模型通常需要多次遍历数据集才能学习一个概念。另一种设置模型运行训练循环次数的方法是设置 `max_train_steps` 变量，你可以在其中设置模型将经历的确切训练步骤数（即使它在某个 epoch 中途结束）。

我们还将简要介绍其他超参数：

**use_ema**

表示使用指数移动平均来训练模型，这有助于通过平均权重来稳定模型的训练过程。

**mixed_precision**

将以混合精度训练模型。如果设置为 FP16，如前面的代码所示，所有不可训练的权重（如 VAE）将被转换为半精度。这些权重仅用于推理，因此我们不需要它们以全精度存储。这将使用更少的显存并加速训练。

**resolution**

指定训练的图像分辨率。然后图像将根据诸如 `center_crop`（如果图像大于目标分辨率，它们将被中心裁剪）和 `random_flip`（一些图像将在训练期间被翻转以提高鲁棒性）等参数进行调整。

**train_batch_size**

指定同时向模型展示的样本数量。批量越大，显存需求越大，但训练时间越短。

**gradient_checkpointing 和 gradient_accumulation_steps**

这使得用户可以在较少的显存上进行训练。使用 `gradient_checkpointing` 会以额外的计算为代价来节省梯度内存，而 `gradient_accumulation_steps` 允许通过累积多个训练小批次的结果来使用更大的有效批量。

**use_8bit_adam**

是否使用 `bitsandbytes` 的 8 位 Adam 优化器来减少所需的 GPU 内存。此选项使训练更快并使用更少的内存，梯度累积的精度低于 FP16 或 FP32 精度（在更新模型权重之前对多个小批次的梯度求和）。

**checkpointing_steps**

在多少个训练步骤（模型看到的批次）之后保存模型的快照？如果你设置了较高的 `num_train_epochs` 或 `max_train_steps`，保存中间模型非常有用。你可能会发现表现最好的模型是在 30 个 epoch 上训练的，而完整的 50 个 epoch 会导致过拟合。

**validation_prompts**

帮助你在训练期间检查模型表现的提示。每经过 `validation_epochs` 个 epoch，你的模型将使用 `validation_prompts` 生成图像，你可以通过感知分析模型的学习情况。

**output_dir** 模型将保存到的本地目录。

**push_to_hub**

是否在训练后将模型推送到 Hugging Face Hub。

`train_text_to_image.py` 训练脚本有更多的参数和设置。Stable Diffusion XL 也可以类似地进行微调。一旦你的模型训练完成并推送到 Hub，你就可以在其上运行推理。

## 推理

一旦微调完成，模型可以像常规的 Stable Diffusion 模型一样用于推理，正如你在第 5 章中学到的，但这次加载的模型是新训练的。如果你没有计算资源来训练模型，这里有一个在相同 Hubble 数据集上训练的模型，你可以尝试：`Supermaxman/hubble-diffusion-1`。你也可以使用 Hugging Face 平台与他人分享你的模型。以下是 Hubble 微调的 Stable Diffusion 模型的输出：

```python
import torch
from diffusers import StableDiffusionPipeline

from genaibook.core import get_device

model_id = "Supermaxman/hubble-diffusion-1"
device = get_device()
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,  # your-hf-username/your-finetuned-model
    torch_dtype=torch.float16,
).to(device)

prompt = (
    "Hubble reveals a cosmic dance of binary stars: In this stunning new image "
    "from the Hubble Space Telescope, a pair of binary stars orbit each other in a "
    "mesmerizing ballet of gravity and light. The interaction between these two "
    "stellar partners causes them to shine brighter, offering astronomers crucial "
    "insights into the mechanics of dual-star systems."
)

pipe(prompt).images[0]
```

当你使用该模型时，你可能会发现它变得非常擅长生成可能来自哈勃望远镜的图像（或你为其微调的任何内容），正如它生成的前述图像所示。然而，它已经成为了这方面的专家。如果你提示它生成其他内容，它要么会生成一些看起来像星系的输出，要么就是一些无意义的内容。这是因为通过微调整个模型，它会经历灾难性遗忘，整个模型会朝着你引导的方向进行调整。此外，如前所述，你需要相当多的图像来训练它。诸如 DreamBooth 和 LoRA 等技术可以帮助克服这些限制。

## DreamBooth

DreamBooth（图7-2）是一种用于微调Stable Diffusion的定制技术，首次出现在Google

Research的DreamBooth论文中。DreamBooth技术通过使用与触发词相关联的少量样本图像对Stable Diffusion UNet进行完全微调。为了保留模型的先前知识，它还使用代表我们正在微调的对象类型的样本图像，采用一种称为先验保留损失的方法。正如我们将在本章进一步解释的那样，如果我们想微调一只特定的狗（我们的狗），我们将使用这只狗的图像，但也使用其他狗的类别图像。

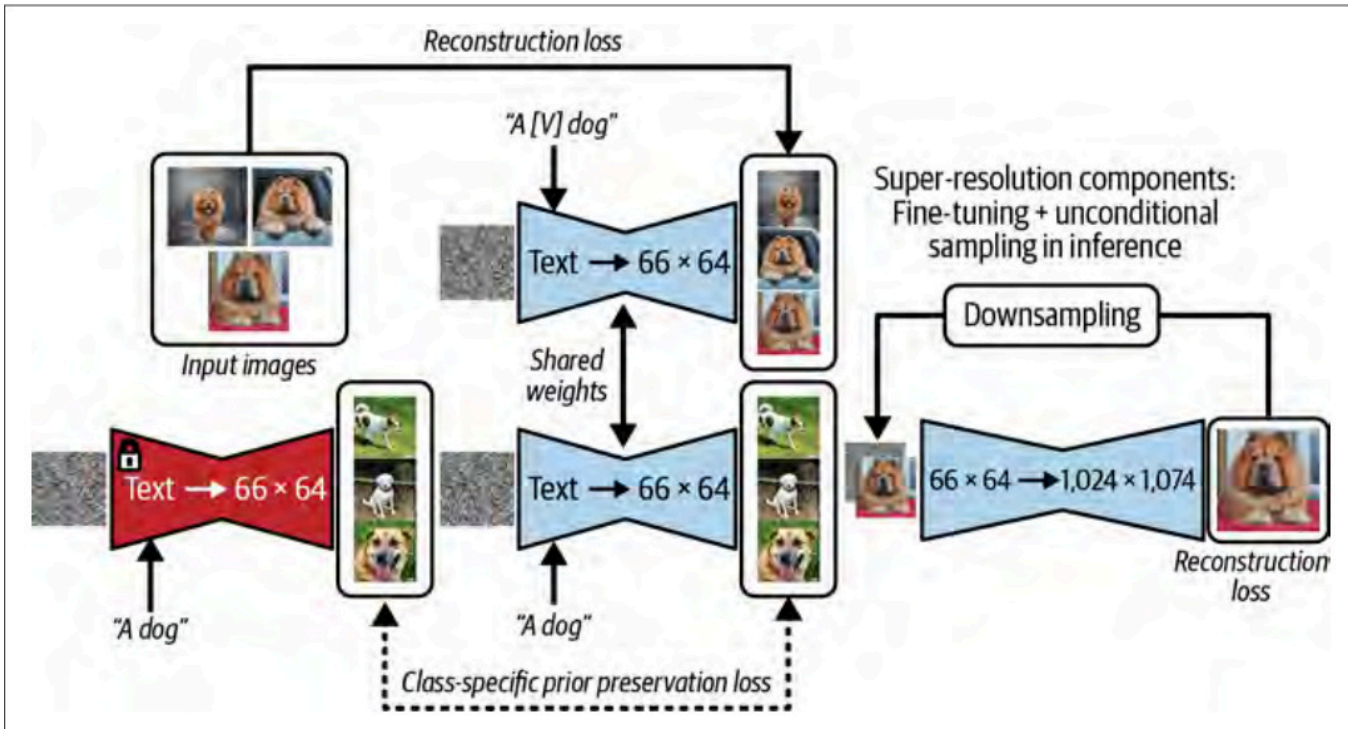



图7-2. DreamBooth架构流程

与完全微调文本到图像扩散模型相比，DreamBooth带来了三个令人兴奋的进步：

- 通过教授模型一个新概念来定制扩散模型，同时保留所有先前的知识（避免上一节中提到的灾难性遗忘特性）。如图7-2所示，DreamBooth将特定的唯一标记或一组标记调整为新添加的概念。因此，如果你想将你的狗包含在模型中，你可以用句子“a [T] dog”来训练它，每次你在模型中引用“a [T] dog”时，它都能够生成具有特定“dreamboothed”狗的图像，同时保持其特征。这可以通过利用模型已经具备的语义知识（例如，它“知道”什么是狗）与新颖的类别特定先验保留损失来实现，使模型在生成新概念时保留该知识。这种组合允许在各种场景、姿势、视角和光照条件下创建主题，而这些条件在参考图像中并不存在。如果没有唯一标记，模型可能会将你试图训练的新知识与与该标记相关联的现有概念混合在一起。
- 仅用3-5个示例定制扩散模型。与使用500+个示例进行完全微调不同，当给定少量示例时，模型仍然可以很好地学习和泛化。这是因为模型可以利用其内部知识来处理与你定制的相同类别的内容（例如，“a [T] dog”包含单词“dog”，然后它将使用模型中狗的内部表示来定

制为你图像中的特定狗）。较少示例的另一个后果是训练速度快，因为模型需要处理的内容更少。
- 为每个实例图像编写标题以供模型学习，就像我们在完全微调中所做的那样，是兼容但可选的，因为使用标记而不为图像添加标题对于DreamBooth来说已经足够。

原始论文中的DreamBooth技术应用于Google的专有扩散模型Imagen。然而，开源社区成员Xavier Xiao将该技术适配到Stable Diffusion，此后出现了许多社区实现，例如TheLastBen和khoya-ss的实现。diffusers库也有DreamBooth训练脚本。

总的来说，通过试验、错误和去中心化的经验，社区发现以下内容：

- 通常三到五张图像足以在Stable Diffusion上训练一个常见主题或风格。
- 使用8到20张图像对于训练独特风格或稀有对象更有效。
- 先验保留损失对于面部有益，但对于其他主题或风格可能不需要。
- 微调模型的文本编码器和UNet可以产生良好的结果。
- 大多数这些见解已被纳入社区的训练脚本中。

我们将使用diffusers库中的脚本diffusers/examples/dreambooth/train_dreambooth.py。

> DreamBooth并不是第一个具有相同目标的定制技术。基于开创性论文“An Image Is Worth One Word”的Textual Inversion展示了如何训练Stable Diffusion模型的文本编码器的新嵌入以包含新主题。该技术仍然相关，因为训练的嵌入可以很小（只有几千字节）。然而，存在大小与质量的权衡，而DreamBooth的质量使该技术在文本到图像社区中占据主导地位。结合Textual Inversion和DreamBooth（称为Pivotal Tuning）的技术的实验也显示出良好的结果。使用DreamBooth，你需要找到一个唯一的触发词。然而，通过利用Textual Inversion，我们可以创建触发词作为新标记，从而更好地注入新概念。

## 准备数据集

通常，5到20个示例足以训练一个新对象或面部。对于风格，如果模型在该范围内难以学习，添加更多示例可能会有所帮助。由于DreamBooth不需要为图像添加标题，你只需将训练图像放在

你喜欢的文件夹中，并可以使用训练代码引用它。例如，你可以下载你宠物的照片。

对于此示例，我们将训练一个模型以识别其中一位作者的面部。如果你想跟随，可以自由地训练一个模型以识别你自己的面部（或可能是宠物）。

## 先验保留

使用DreamBooth，你可以选择利用先验保留类别。这通过在训练期间提供先验保留损失来实现，以便模型理解它正在生成该类别中的元素。例如，在教授模型你自己的面部时，拥有一组面部图像将帮助模型“理解”你试图训练的类别是面部，因此即使你提供少量示例，它也将基于面部图像进行训练。如果模型已经具备你将要创建的类别的知识，你甚至可以自己生成先验图像（训练代码中有一个标志允许你这样做）或将它们上传到特定文件夹。

要启用先验保留，你可以在训练脚本中配置几个参数：

with_prior_preservation

是否使用先验保留。如果设置为True，模型将基于class_prompt和num_class_images参数生成图像。如果你提供了class_data_dir文件夹，该文件夹中的图像将用作类别图像。

class_prompt

当使用先验保留时，描述将用于训练的生成样本图像类别的提示——例如，一个巴西男人的面部。它不包括触发词。

num_class_images

要生成的类别图像的数量。如果你提供了class_data_dir文件夹，该文件夹中的图像将用作类别图像。如果文件夹中的图像少于num_class_images，剩余的图像将由class_prompt生成。

## DreamBooth模型

与微调整个模型一样，最重要的变量是learning_rate和num_train_epochs。低学习率后缓慢增加训练周期或步骤的数量可以提供一个良好的起点，以获得良好的质量。另一种探索路径是固定训练周期或步骤的数量并增加学习率。这两种策略可以结合使用以找到最佳超参数。

让我们回顾一些DreamBooth独有的参数：

instance_prompt

模型将尝试从中学习概念的提示。尝试为你的主题名称找到一个罕见的标记或组合，并用上下文

包围它，例如，以mybtfuart风格，plstps的照片，或sckpto玩具。罕见的组合将是你正在微调的特定实例的触发词。

train_text_encoder

是否也训练文本编码器。它可以产生良好的结果，但消耗更多的VRAM。训练文本编码器与UNet一起可能有用，因为你还在提示的文本解释中注入了关于这个新概念的知识。

with_prior_preservation 是否使用先验保留损失。

class_prompt或class_data_dir

用于生成先验保留的类别图像的提示。如果提供了class_data_dir，它们将从该文件夹中获取。否则，将使用class_prompt中指定的提示生成一些图像。

prior_loss_weight

控制先验保留损失对模型的影响。

让我们使用train_dreambooth.py脚本训练一个模型以识别其中一位作者的面部，如图7-3所示。如果你想跟随，可以自由地训练一个模型以识别你自己的面部。

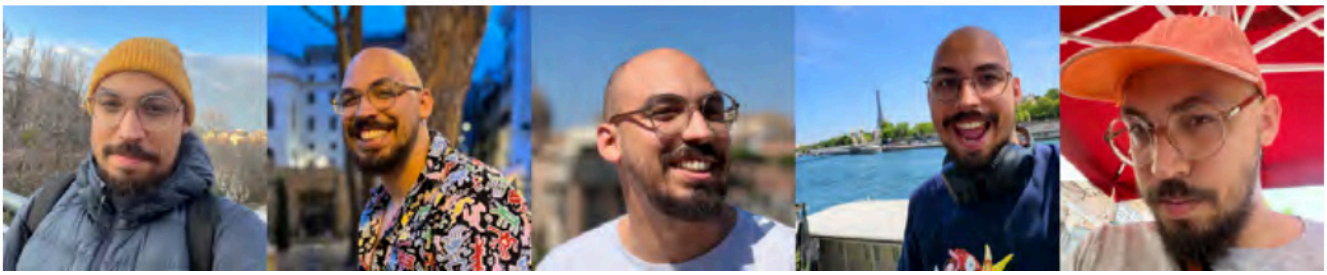


图7-3. 用于DreamBooth训练的“Apolinário Passos”面部图像集

```
accelerate launch train_dreambooth.py \
--pretrained_model_name_or_path="stable-diffusion-v1-5/stable-diffusion-v1-5"  \
--instance_data_dir="my-pictures" \
--instance_prompt="a photo of plstps" \  # <1>
--resolution=512 \
--train_batch_size=1 \
--with_prior_preservation \
--class_prompt="an ultra realistic portrait of a man" \  # <2>
--gradient_accumulation_steps=1 \
--train_text_encoder \
--learning_rate=5e-6 \
--num_train_epochs=100 \
--output_dir="myself-model" \
--push_to_hub
```

[1]plstps 是模型学习概念的独特标记。
[2]在这里，你可以添加关于你正在训练的内容的通用描述。

一旦模型训练完成，你可以使用它来生成新的图像。让我们探索如何做到这一点。

## 推理

尽管模型的整体结构保持不变，仅基于 `instance_prompt` 的新 token 被修改，但新的 DreamBooth 模型仍然是一个完整的 Stable Diffusion 权重。因此，它可以像这样加载以进行推理：

```
model_id = "your-hf-profile/your-custom-dreambooth-model"
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
).to(device)

# Insert here your instance prompt and some custom customization
prompt = "a photo of plstps speaking on a microphone"
pipe(prompt).images[0]
```

插入你的实例提示和一些自定义内容。

图 7-4 展示了模型在作者之一的面部上训练的一些结果。

图 7-4. 模型在 Apolinário 的面部上“DreamBoothed”生成的图像

## 训练 LoRAs

我们在全微调和 DreamBooth 上遇到了一个问题：一旦我们完成了模型的微调，我们最终会得到与原始 Stable Diffusion 模型一样大的新权重。这种情况对于共享、本地托管、堆叠模型、在云端服务模型以及其他下游应用来说并不理想。为此，可以使用 LoRA（图 7-5），就像我们在第 6 章中对 LLMs 所做的那样。

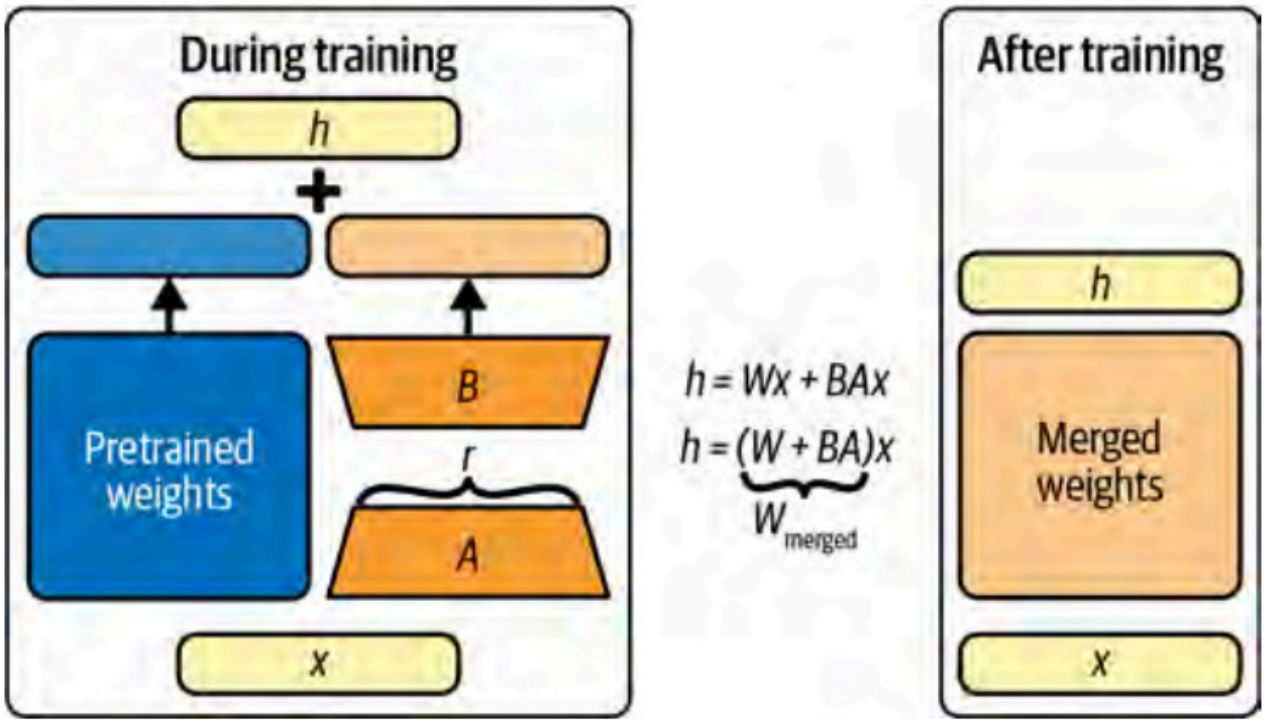



图 7-5. LoRA 架构流程

正如第 6 章所讨论的，LoRAs 允许冻结预训练模型的权重并注入秩分解矩阵，从而显著减少需要训练的参数数量。LoRA 训练的秩也可以作为工件共享，这些工件可以合并到模型中而不会增加推理延迟。

听起来很棒！但我们处于微调扩散模型的章节，而原始的 LoRAs 专注于 transformers。这就是 Simo Ryu 的 Stable Diffusion LoRA GitHub 仓库发挥作用的地方。意识到 LoRA 的秩可以以与 transformers LLMs 相同的方式附加到 Stable Diffusion UNet 和文本编码器上，现在为扩散模型解锁了 LoRAs 的力量。

再次，`diffusers` 库来拯救，它包含了一个用于 LoRA 训练的脚本，适用于全模型微调和 DreamBooth 微调。使用 `diffusers` 训练 LoRA 权重与使用 `diffusers` 进行全模型微调或 DreamBooth 微调几乎相同，但有一些关键区别：

- 数据集格式与用于全模型微调和 DreamBooth 微调的格式相同。
- 训练脚本不同，尽管超参数相同。我们将使用 `diffusers` 库中的 `examples/text_to_image/train_text_to_image_lora.py` 和 `examples/dreambooth/train_dreambooth_lora.py`。
- 对于推理，过程涉及将基础模型加载到管道中，然后添加 LoRA 适配器。这种方法很方便，因为它允许你快速加载和切换不同的 LoRA 适配器，同时保持相同的基础模型。例

如，你可以使用另一个用户共享的预训练 LoRA 微调（你可以在 Hugging Face Hub 上找到预训练选项）。步骤是：选择基础模型（LoRA 将附加到的模型），加载 `diffusers` 管道，将 LoRA 权重加载到模型中，并可选地融合 LoRA 权重以提高效率和速度。

让我们看看如何加载 LoRA 微调模型并对其进行推理。主要区别在于确定基础模型并将 LoRA 权重加载到模型中：

```
from diffusers import DiffusionPipeline
from huggingface_hub.repocard import RepoCard

# We'll use a classic hand drawn cartoon style
lora_model_id = "alvdansen/littletinies"

# Determine which is the base model
# This information is frequently in the model card
# It's CompVis/stable-diffusion-v1-4 in this case
card = RepoCard.load(lora_model_id)
base_model_id = card.data["base_model"]

# Load the base model
pipe = DiffusionPipeline.from_pretrained(
    base_model_id, torch_dtype=torch.float16
)
pipe = pipe.to(device)

# Add the LoRA to the model
pipe.load_lora_weights(lora_model_id)

# Merge the LoRA with the base model
pipe.fuse_lora()

image = pipe(
    "A llama drinking boba tea", num_inference_steps=25,
    guidance_scale=7.5
).images[0]
image
```

## 赋予 Stable Diffusion 新能力

通过微调来教模型新的风格或主题是令人惊叹的。但如果我们能通过微调赋予 Stable Diffusion 比平常更多的能力呢？通过使用一些特殊技术进行微调，我们可以为模型提供修复能力或包含额外的条件。

### 修复

修复涉及屏蔽图像的特定区域，你希望用其他内容替换该区域。它与图像到图像类似，不同之处在于仅在屏蔽区域添加噪声：模型仅对该区域进行去噪，旨在更改或移除图像中的该项目，同时

保持图像的其余部分不变。

通过为 UNet 添加额外的输入通道，可以为预训练的文本到图像扩散模型提供修复能力。在修复专家 Stable Diffusion v1 模型的情况下，他们通过为 UNet 添加五个零填充的输入通道（四个用于编码的屏蔽图像，一个用于屏蔽本身）进行了 $\sim 400\mathrm{K}$ 步的训练。在训练过程中，生成了合成屏蔽，其中 $25\%$ 的内容被屏蔽。由于你有屏蔽后图像的真实情况，模型学会了如何根据提示填充屏蔽区域，成为一个强大的图像编辑工具。

对于更高级的模型，如 Stable Diffusion XL，一些修复能力是开箱即用的，无需进一步微调，这使一些人质疑专门的微调模型是否能提高这种能力。然而，SDXL 专家修复模型发布时带有一些额外的功能，展示了这种技术即使在更大更先进的模型中的潜力。虽然这种技术无法在家庭硬件上进行训练，需要全微调数十万步，但微调模型对所有人都是可访问的。在下一章中，我们将更深入地探索修复（带代码）。

## 特殊条件的额外输入

就像可以为 UNet 添加新的输入通道以让模型学习如何执行修复一样，也可以添加其他条件。这种应用的一个例子是 Stable Diffusion 2 Depth，这是一个从 `stable-diffusion-2-base` 恢复并微调了 $200\mathrm{K}$ 步的模型，带有一个额外的输入通道，该通道处理用户提示和包含由 MiDaS 生成的单目深度（相对于相机的距离）预测的图像。一个例子可以在图 7-6 中看到。

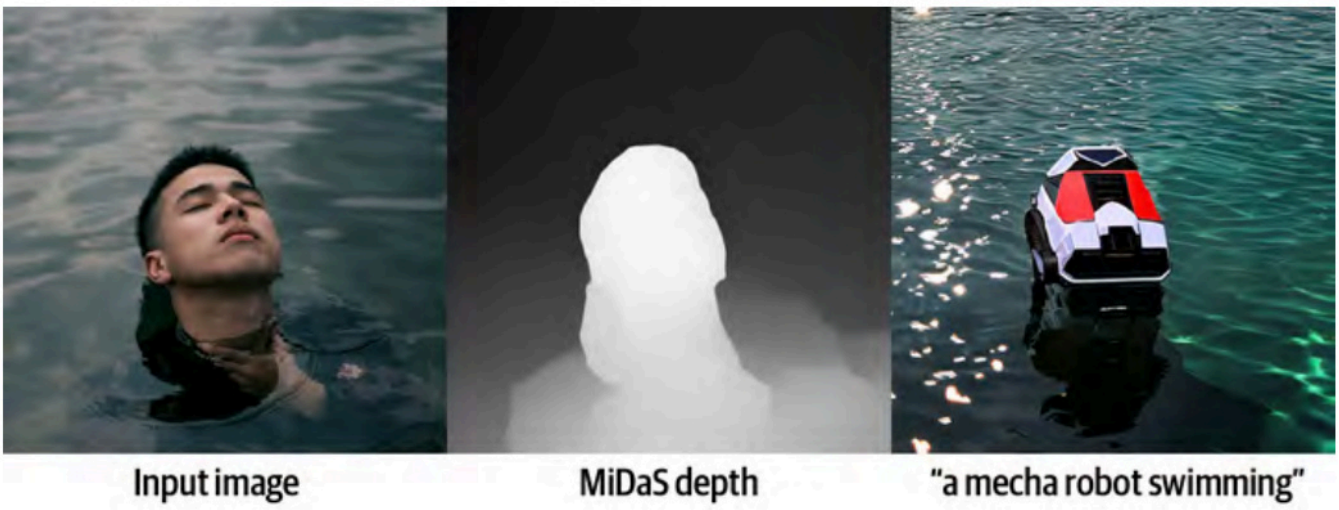



图 7-6. 使用 MiDaS 深度条件微调的 Stable Diffusion 2 的推理

虽然这种技术效果很好，但为了获得新的条件而对基础模型进行数百或数千步的微调，使得这一过程仅限于少数公司和实验室。然而，诸如 ControlNets、ControLoras 和 T2I 适配器之类的技术在模型顶部附加适配器，使这一过程在训练和推理中更加高效；我们将在下一章中探索更多这

些创造性的文本到图像应用。

# 项目时间：自己训练一个SDXL DreamBooth LoRA

微调是为文本到图像扩散模型带来更多知识的好方法，正如你在本章中学到的，DreamBooth允许仅用少量示例图像进行微调，而LoRA训练则允许使用较小的模型，并且与微调整个模型相比，GPU使用率更低。在这个项目中，你将微调一个DreamBooth LoRA。在学习了DreamBooth和LoRA的基础知识后，你可以使用diffusers更高级的脚本。如果你没有至少16 GB显存的GPU，我们建议使用Google Colab或Hugging Face Spaces来完成这个项目。

你的目标是能够提示一个尚未存在的新对象或风格到Stable Diffusion中，并让模型成功生成包含它的新图像。这涉及两个步骤：

1. 数据集创建：

a. 找到一个你想包含在模型中的对象或风格。它可以是你拥有的独特物品（例如，一个木制猫玩具）或你家里的一种家具/绘画/地毯风格。
b. 从不同角度、不同背景拍摄这些对象的几张照片；大约3-8张照片应该足够了。

c. 为每张图片写一个描述性标题，并使用一个独特的标记来描述你的对象（例如，cttoy），例如：

- 一张cttoy正面的照片，白色背景
- 一张cttoy侧面的照片，花盆在背景中
  d. 将数据集上传到Hugging Face Datasets Hub或将其保存在本地文件夹中。

2. 模型训练：

a. 打开任何适合你的训练脚本（基于推荐的脚本或你找到的其他脚本）。
b. 将图像文件夹指向你创建的本地文件夹或Hugging Face数据集。
c. 运行训练。如前所述，你可以尝试调整学习率、batch_size和其他超参数，直到你对你的LoRA满意为止。参考第253页的“训练LoRAs”了解如何将训练好的LoRA加载到Stable Diffusion中进行测试。
d. 使用你的validation_prompts，你可以在训练期间检查样本的生成情况。一旦模型训练完成，你可以使用load_lora_weights加载它，以了解你的模型是如何训练的。

# 总结

由于从头训练一个大型文本到图像模型需要大量的计算资源，微调步骤使得单GPU操作成为可能，以定制现有模型以生成你所需的内容。在本章中，你学习了如何通过微调扩散模型来扩展知识并根据特定需求定制模型，同时保留其整体知识。你学习了如何进行完整微调、使用DreamBooth进行特定角色或风格的定制，以及使用LoRA提高效率。你还了解到，微调扩散模型可以为它们赋予新的能力。总的来说，微调是一个强大的工具。

本章探讨了微调Stable Diffusion模型以教授其风格、主题或能力的技术。从完整模型微调开始，你学习了如何改变模型的行为以生成所需风格或主题的图像。然后我们转向了DreamBooth和LoRA等技术，这些技术允许用更少的示例进行定制，并减少灾难性遗忘的风险。

我们还讨论了微调为模型添加新能力的潜力，例如修复和特殊条件，扩展了Stable Diffusion的实用性，超越了其原始配置。

为了进一步阅读，我们建议查看以下内容：

·“如何微调Stable Diffusion：我们在Lambda制作文本到宝可梦模型的方法”
·“我如何训练LoRA：m3lt风格训练概述”
·“通过微调Stable Diffusion创建无限图标库”
·高级扩散训练指南

- DreamBooth论文本身
- 关于扩散模型中LoRA的详细介绍

# 练习

1. 解释完整模型微调和DreamBooth之间的主要区别。
2. 在计算资源和模型适应性方面，使用LoRA相比完整模型微调有哪些优势？
3. 为什么在进行DreamBooth训练时使用独特标记很重要？
4. 除了教授新概念外，微调还可以为模型添加新能力。列举两种通过应用微调技术模型可以学习的能力。
5. 讨论超参数的选择如何影响微调扩散模型的结果。
6. 描述在偏置数据集上微调文本到图像模型的潜在风险。

你可以在本书的GitHub仓库中找到这些练习的解决方案以及以下挑战。

## 挑战

LoRA与完整微调的比较。使用LoRA和完整微调训练一个DreamBooth模型，并比较结果。尝试修改LoRA的rank超参数，看看它对结果有多大影响。

## 参考文献

Gal, Rinon, 等. “一张图片值一个词：使用文本反演个性化文本到图像生成.” arXiv, 2022年8月2日. https://arxiv.org/abs/2208.01618.
Hu, Edward J., 等. “LoRA：大型语言模型的低秩适应.” arXiv, 2021年10月16日. https://arxiv.org/abs/2106.09685.
Podell, Dustin, 等. “SDXL：改进潜在扩散模型以实现高分辨率图像合成.” arXiv, 2023年7月4日. http://arxiv.org/abs/2307.01952.
Ruiz, Nataniel, 等. “DreamBooth：微调文本到图像扩散模型以进行主题驱动生成.” arXiv, 2023年3月15日. https://arxiv.org/abs/2208.12242.
Ryu, Simo. LoRA GitHub仓库. 2022. https://oreil.ly/L3hEc.

# 第三部分 更进一步

# 第8章 文本到图像模型的创意应用

本章介绍了利用文本到图像模型并扩展其能力的创意应用，超越了仅使用文本控制生成的范围。我们将从最基本的应用开始，然后转向更高级的应用。

## 图像到图像

尽管像Stable Diffusion这样的生成文本到图像扩散模型可以从完全噪声的图像中生成图像，正如你在第4章和第5章中学到的，也可以从已有的图像开始，而不是完全噪声的图像。也就是说，对初始图像添加一些噪声，并让模型通过去噪部分修改它。这个过程被称为图像到图像，因为图像根据其噪声程度和文本提示被转换为另一张图像。

使用diffusers库，我们可以加载一个图像到图像管道来加载类。作为一个例子，让我们探索如何使用SDXL来完成这个任务。以下是主要区别：

- 我们使用StableDiffusionXLImg2ImgPipeline而不是通常的StableDiffusionXLPipeline。

- 我们向管道传递一个提示和一个初始图像。

我们可以使用stabilityai/stable-diffusion-xl-base-1.0或stabilityai/stable-diffusion-xl-refiner-1.0模型来应用我们的图像到图像细化。当你想对图像进行风格化或从现有内容中创建新上下文时，推荐使用基础模型。如果你想要在不进行太多创意转换的情况下细化或添加细节，专门处理图像细节的refiner模型可能是一个不错的选择：

```
import torch
from diffusers import StableDiffusionXLImg2ImgPipeline
from genaibook.core import get_device
device = get_device()
// 加载管道
img2img_pipeline = StableDiffusionXLImg2ImgPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-refiner-1.0", torch_dtype=torch.float16,
    variant="fp16",
)
```

然后，我们可以将管道移动到我们的设备（通常是GPU的cuda）。由于某些示例可能需要太多GPU，另一种选择是使用img2img_pipeline.enable_model_cpu_offload()，它根据需要将子模块移动到GPU。这将使推理变慢，但允许你在较小的GPU上运行模型：

```
// 将管道移动到设备
// 或者，img2img_pipeline.enable_model_cpu_offload()
img2img_pipeline.to(device)
```

现在我们有了管道，让我们尝试一个例子：

```
from genaibook.core import SampleURL, load_image, image_grid

// 加载图像
url = SampleURL.ToyAstronauts
init_image = load_image(url)

prompt = "Astronaut in a jungle, cold color palette, muted colors, detailed, 8k"

// 将提示和图像传递给管道
image = img2img_pipeline(prompt, image=init_image, strength=0.5).images[0]
image_grid([init_image, image], rows=1, cols=2)
```

我们的StableDiffusionXLImg2ImgPipeline接受与普通Stable Diffusion管道相同的输入，外加两个额外参数：

**init_image**
我们将要修改的原始图像。

**strength**

我们将向图像中添加多少噪声。强度为0将返回完全相同的图像，因为没有添加任何噪声。强度为1将完全噪声化图像，完全忽略它，表现得像常规的文本到图像管道。

查看图8-1中的实验，我们在其中对同一图像应用了0到1之间的强度。

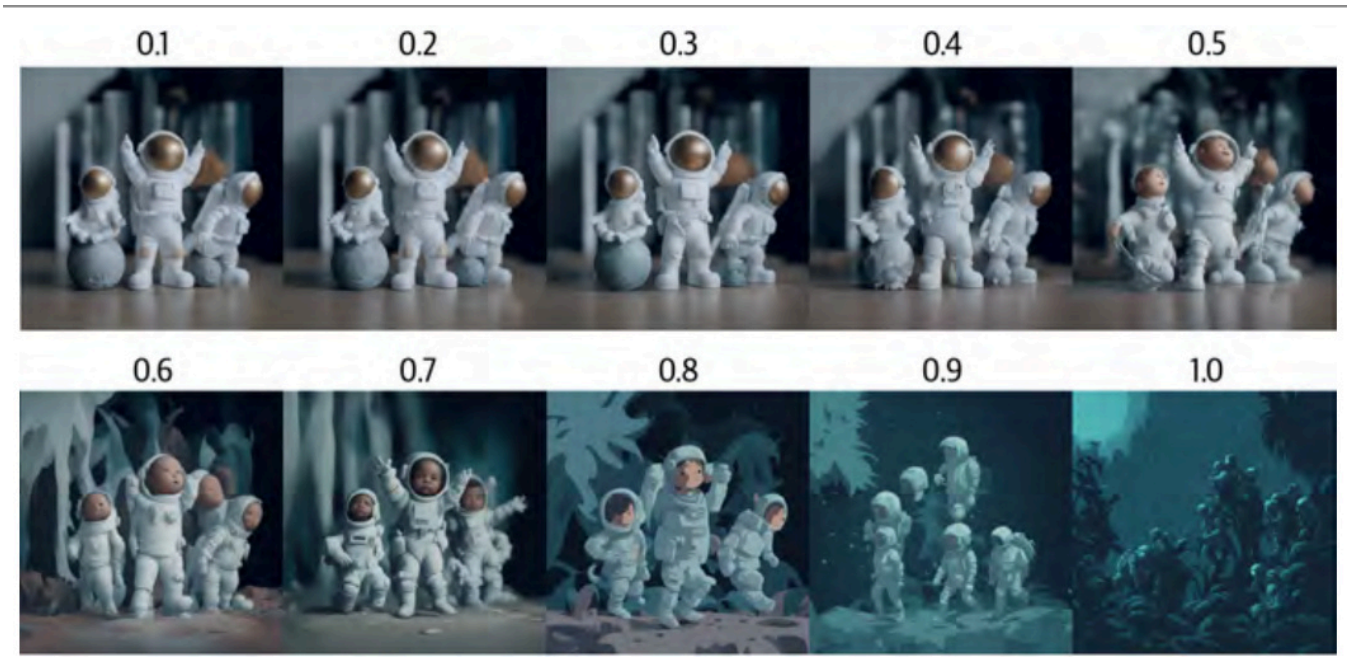


图8-1. 使用图像到图像模型将图像的去噪强度从0.1变化到1.0

# 图像修复

图像修复是基于周围上下文填充图像缺失部分的过程。正如我们在前一章中讨论的那样，可以直接使用模型进行修复，或者微调文本到图像的扩散模型以提升其修复能力。

在我们深入探讨用于修复的文本到图像扩散模型的具体细节之前，值得注意这种文本到图像生成方法与传统的图像处理修复技术之间的区别。传统方法通常依赖于分析周围像素，并使用各种算法基于局部图像统计或基于采样的补丁来填充被遮罩的区域。虽然这些传统方法对于简单背景或小区域可能有效，但它们通常在处理复杂纹理或图像内容的语义理解时表现不佳。相比之下，使用文本到图像扩散模型进行修复具有几个优势。这些模型能够基于视觉和语义上下文理解和生成内容，从而产生更连贯和富有创意的结果。然而，机器学习方法通常需要更多的计算资源。

让我们展示如何进行修复以及它可以利用哪些创意应用。和之前一样，我们可以使用一个管道 `StableDiffusionXLInpaintPipeline` 来处理：

```
from diffusers import StableDiffusionXLInpaintPipeline

// 加载管道
inpaint_pipeline = StableDiffusionXLInpaintPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0", torch_dtype=torch.float16,
    variant="fp16",
).to(device)
img_url = SampleURL.DogBenchImage
mask_url = SampleURL.DogBenchMask
init_image = load_image(img_url).convert("RGB").resize((1024, 1024))
mask_image = load_image(mask_url).convert("RGB").resize((1024, 1024))
// 将图像和提示传递给管道
prompt = "A majestic tiger sitting on a bench"
image = inpaint_pipeline( prompt=prompt, image=init_image,
    mask_image=mask_image, num_inference_steps=50, strength=0.80,
    width=init_image.size[0], heigth=init_image.size[1],
).images[0]
```

在生成的结果网格中，左侧面板表示源图像，中间面板表示遮罩图像，右侧面板表示输出图像：

```
image_grid([init_image, mask_image, image], rows=1, cols=3)
```

`StableDiffusionXLInpaintPipeline` 接收的一些最重要的参数如下：

**init_image**

将被修复的图像。

**mask_image**

一个二值颜色遮罩图像。黑色表示图像中应保持不变的部分，白色表示图像将被生成内容替换的部分。

**strength**

我们将添加到遮罩中的噪声量。类似于图像到图像的强度，但仅应用于遮罩区域。强度为 0 将返回与未添加噪声相同的图像。强度为 1 将完全噪声化遮罩区域，这对于平滑混合来说并不是最佳情况。建议在 0.6 到 0.8 之间进行实验。

除了基础的扩散模型外，一些模型（如 `diffusers/stablediffusion-xl-inpainting`）经过微调，专门用于修复。这些模型明确针对此任务进行了微调，这使得我们可以在推理过程中使用更高的强度。

# 提示权重与图像编辑

正如你在第 4 章中学到的，扩散模型使用类似 Transformer 的注意力机制，使模型能够灵活地关注输入的最相关部分。具体来说，交叉注意力用于在 UNet 层中使用文本提示来调节图像生成。

然而，在某些情况下，你可能希望对生成的图像有更多的控制。例如，我们可能希望执行以下操作：

- 通过调整文本嵌入向量的缩放比例，修改提示词中每个单词所占的权重。
- 结合多个提示生成图像。
- 在保持结构的同时进行图像编辑的生成。

为此，“Prompt-to-Prompt Image Editing with Cross Attention Control” 提出了通过修改和控制交叉注意力来实现可操纵性的想法。该论文还有一个非官方的 `diffusers` 实现。

除了 Prompt-to-Prompt 之外，还出现了其他用于编辑生成图像的技术，例如 Attend-and-Excite 和 Semantic Guidance，两者都有官方的 `diffusers` 实现。在本章中，我们将更深入地探讨 Semantic Guidance 编辑，因为它在可操纵性和编辑质量之间取得了平衡。

## 提示权重与合并

compel 提示增强库实现了提示权重和合并的关键方面，并且易于与 diffusers 库一起使用。它通过预处理字符串并增强 CLIP 嵌入空间中的相应嵌入来工作。由于 Stable Diffusion XL 使用了两个文本编码器，这增加了提示权重过程的复杂性。这种复杂性源于权重必须在两个编码器之间协调，并且第二个文本编码器的输出需要被池化。compel 库抽象了这种复杂性。

控制提示的两种简单方法如下：

- 通过在单词后添加 `+` 符号来增加其权重，使其在图像中更加突出，从而修改文本嵌入的规模。你也可以使用 `-` 来减少权重。通过添加多个 `+` 和 `-` 符号，你可以进一步增加或减少单词的权重。尽管我们可以减少单词的突出程度，但它可能并不总是完全从图像中移除该概念。
- 合并两个提示（将它们放在括号内），然后为每个提示指定权重。

让我们编写一些代码。和往常一样，我们首先加载一个管道：

```
from diffusers import DiffusionPipeline

pipeline = DiffusionPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16, variant="fp16",
).to(device)
```

我们现在初始化一个 `Compel` 类，它需要提供扩散模型的分词器和文本编码器。该类还需要指定哪个文本嵌入将被池化：

```
from compel import Compel, ReturnedEmbeddingsType
// 使用倒数第二层 CLIP 层，因为它更具表现力
embeddings_type = ( ReturnedEmbeddingsType
    .PENULTIMATE_HIDDEN_STATES_NON_NORMALIZED
)
compel = Compel( tokenizer=[pipeline.tokenizer, pipeline.tokenizer_2],
    text_encoder=[pipeline.text_encoder, pipeline.text_encoder_2],
    returned_embeddings_type=embeddings_type, requires_pooled=[False, True],
)
```

最后，我们可以使用不同的 `compel` 增强提示生成图像：

```python
from diffusers.utils import make_image_grid

# Prepare the prompts
prompts = []
prompts.append("a humanoid robot eating pasta")
prompts.append(
    "a humanoid+++ robot eating pasta"
)  # make its humanoid characteristics a bit more pronounced
prompts.append(
    '["a humanoid robot eating pasta", "a van gogh painting"].and(0.8, 0.2)'
)  # make it van gogh!

images = []
for prompt in prompts:
    # Use the same seed across generations
    generator = torch.Generator(device=device).manual_seed(1)

    # The compel library returns both the conditioning vectors
    # and the pooled prompt embeds
    conditioning, pooled = compel(prompt)

    # We pass the conditioning and pooled prompt embeds to the pipeline
    image = pipeline(
        prompt_embeds=conditioning,
        pooled_prompt_embeds=pooled,
        num_inference_steps=30,
        generator=generator,
    ).images[0]
    images.append(image)
image_grid(images, rows=1, cols=3)
```

`+` 相当于将提示权重乘以 1.1，而 `-` 相当于乘以 0.9。除了使用 `+` 和 `-` 之外，我们还可以如下方式对标记进行加权：`a robot eating (pasta)1.2`。有关 `compel` 库的更多参考，请查看其官方参考指南。

## 使用语义引导编辑扩散图像

如前所述，存在一些用于扩散生成图像的编辑技术；虽然通过 Prompt-to-Prompt 进行交叉注意力控制是一种流行的编辑方式，但语义引导（Semantic Guidance,SEGA）允许更细粒度的控制和精确的编辑。

SEGA 通过在反向扩散过程的每一步中操纵模型的噪声估计来工作。这种动态噪声调整使 SEGA 能够基于文本描述在潜在空间中进行语义编辑。通过动态调整预测的噪声，SEGA 确保修改朝着从文本嵌入中得出的语义方向进行。该方法计算文本嵌入相对于潜在空间的梯度，有效地引导图像生成或修改朝着期望的语义结果进行。这一过程无需重新训练或修改模型的原始架构，仅基于文本输入即可实现动态和定向的更改。

让我们首先展示如何使用 `SemanticStableDiffusionPipeline` 生成一张男性面部的照片：

```
from diffusers import SemanticStableDiffusionPipeline

semantic_pipeline = SemanticStableDiffusionPipeline.from_pretrained(
    "CompVis/stable-diffusion-v1-4", torch_dtype=torch.float16, variant="fp16"
).to(device)

generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    generator=generator,
)
out.images[0]
```

正如你将在接下来的示例中看到的，SEGA 包含一些重要的参数，用于其编辑功能：

`edit_guidance_scale` 模型应遵循编辑的强度
`edit_warmup_steps` 模型在应用 SEGA 之前应开始的去噪步骤数
`edit_threshold` 应保留原始图像的像素百分比
`reverse_editing_direction` 编辑是否应包含（False）或移除（True）该概念

让我们通过添加 "smiling, smile" 来引导提示，使男性微笑：

```
generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    editing_prompt="smiling, smile",
    edit_guidance_scale=4,
    edit_warmup_steps=10,
    edit_threshold=0.99,
    edit_momentum_scale=0.3,
    edit_mom_beta=0.6,
    reverse_editing_direction=False,
    generator=generator,
)
out.images[0]
```

让我们进行另一次编辑，这次让男性戴上眼镜：

```
generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    editing_prompt="glasses, wearing glasses",
    reverse_editing_direction=False,
    edit_warmup_steps=10,
    edit_guidance_scale=4,
    edit_threshold=0.99,
    edit_momentum_scale=0.3,
    edit_mom_beta=0.6,
    generator=generator,
)
out.images[0]
```

最后，让我们同时进行多次编辑，如下面的代码片段所示。唯一的区别是 `editing_prompt` 现在是一个提示列表，关键参数（`edit_warmup_steps` 等）也必须是列表：

```
generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    editing_prompt=[
        "smiling, smile",
        "glasses, wearing glasses",
    ],
    reverse_editing_direction=[False, False],
    edit_warmup_steps=[10, 10],
    edit_guidance_scale=[6, 6],
    edit_threshold=[0.99, 0.99],
    edit_momentum_scale=0.3,
    edit_mom_beta=0.6,
    generator=generator,
)
out.images[0]
```

## 通过反演进行真实图像编辑

反演是一种将真实图像带回预训练生成模型的潜在空间的技术。该技术在 GAN 中显示出有希望的结果，并成功应用于引导扩散模型。

这使我们能够回答你在学习 SEGA 时可能提出的一个问题：“但如果我们想编辑现实世界中的图像呢？”一种替代方法是使用前面介绍的图像到图像方法。然而，它只能提供非常有限的编辑，并且并不总是产生预期的结果，因为结果可能会在没有太多控制的情况下发生巨大变化。为了解决这个问题，我们可以将扩散图像编辑技术（如 Prompt-to-Prompt 或 SEGA）与反演技术结合，以提供更细粒度的编辑。

引导扩散模型的反演过程使用去噪器的反演调度器。第一个引入反演的去噪器是DDIM反演，它可以通过以相反的顺序执行 DDIM 采样（从真实图像开始并逐渐向其添加噪声）来预测潜在空间中的先前时间步样本。然后，当你对其进行去噪时，你将得到预期的原始图像。仅此而已，这并不特别有趣；你只是使用了噪声潜在空间来重建你已经拥有的图像。然而，如果你的目标是对图像进行编辑，反演可能会变得非常强大。

通过反演进行编辑的最简单方法如下：

1. 获取真实提示和描述（例如，田野中的马的照片）的 DDIM 反演。
2. 将提示修改为目标图像（例如，田野中的斑马的照片）。
3. 使用修改后的提示重建图像。

结果如图 8-2 所示。

图 8-2. 使用 DDIM 反演技术进行编辑（改编自“An Edit Friendly DDPM Noise Space: Inversion and Manipulations”中的图像）

虽然这些结果很有趣，但它们并不适合真实图像编辑。更先进的技术，如使用 DDPM 反演和编辑友好噪声空间进行更好的反演，可以提供更好的重建，但仍然无法提供最广泛的编辑范围。

我们可以将反演技术与编辑技术结合，为真实图像提供广泛的编辑。例如，“Null-text Inversion for Editing Real Images Using Guided Diffusion Models”和 LEDITS++ 分别利用了 Prompt-to-Prompt 和 SEGA。

这些技术利用了你之前学到的编辑技术，但不是在潜在空间中提供通过提示生成的图像的编辑，而是在图像反演后的重建潜在空间中进行编辑。在这里，你将学习如何使用 LEDITS++ 提供现实世界的图像编辑，利用已经学到的 SEGA 和反演技术。

## 使用 LEDITS++ 进行编辑

LEDITS++ 结合了你刚刚学到的两种技术：SEGA 和反演。它进一步实现了一种技术，通过模型生成的交叉注意力和噪声掩码来锚定你的编辑。这种组合使我们能够使用我们为 SEGA 学到的相同参数来编辑真实图像。

LEDITS++ 的工作方式如下：

1. 应用反演将我们感兴趣的图像转换为模型可以操作的格式。
2. 我们使用 `editing_prompt` 决定编辑列表，并使用 `reverse_editing_direction` 决定编辑方向（是添加还是移除该概念）。
3. 我们应用在 SEGA 中学到的 `edit_guidance_scale` 和 `edit_threshold` 来进行编辑。

让我们使用 LEDITS++ 管道：

```python
from diffusers import LEditsPPPipelineStableDiffusion

# Load the model as usual
pipe = LEditsPPPipelineStableDiffusion.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    variant="fp16"
)
pipe.to(device)

image = load_image(SampleURL.ManInGlasses)

# Invert the image, gradually adding noise to it so
# it can be denoised with modified directions,
# effectively providing an edit
pipe.invert(image=image, num_inversion_steps=50, skip=0.2)

# Edit the image with an editing prompt
edited_image = pipe(
    editing_prompt=["glasses"],
    # tell the model to remove the glasses by editing the direction
    reverse_editing_direction=[True],
    edit_guidance_scale=[1.5],
    edit_threshold=[0.95],
).images[0]

image_grid([image, edited_image], rows=1, cols=2)
```

## 通过指令微调实现真实图像编辑

为扩散模型提供真实图像编辑的另一种方法是专门为此任务微调模型。这种方法由InstructPix2Pix论文首创。训练需要一个包含原始图像、编辑指令和编辑后图像的编辑指令对数据集。

然后，Stable Diffusion模型的第一卷积层会附加额外的输入通道，使其能够接收图像输入。该模型使用与原始Stable Diffusion模型中用于字幕的文本条件机制相同的机制进行训练，但修改为将编辑指令作为其提示（图8-3）。

在原始论文发表后，进一步训练和改进的InstructPix2Pix模型相继出现。最著名的是Stability AI的CosXL Edit，它基于Stable Diffusion XL的变体进行训练，以执行高质量的编辑。

图8-3. 使用InstructPix2Pix技术进行编辑

CosXL仓库是受限制的，因此请访问Hugging Face上的模型页面，阅读许可证，如果同意条款，请点击按钮接受。在终端会话中运行huggingfacecli login以登录。系统会要求您提供一个访问令牌，您可以在设置页面创建该令牌。如果您从Google Colab会话下载模型，可以设置一个HF_TOKEN秘密或环境变量，并授予您的笔记本使用它的权限。以下是一个使用CosXL进行编辑的示例：

```python
from diffusers import (
    EDMEulerScheduler,
    StableDiffusionXLInstructPix2PixPipeline,
)
from huggingface_hub import hf_hub_download

edit_file = hf_hub_download(
    repo_id="stabilityai/cosxl", filename="cosxl_edit.safetensors"
)

# from_single_file loads a diffusion model from a single diffusers file
pipe_edit = StableDiffusionXLInstructPix2PixPipeline.from_single_file(
    edit_file, num_in_channels=8, is_cosxl_edit=True, torch_dtype=torch.float16
)

# The model was trained so that the EDMEulerScheduler
# is the correct noise scheduler for denoising
pipe_edit.scheduler = EDMEulerScheduler(
    sigma_min=0.002,
    sigma_max=120.0,
    sigma_data=1.0,
    prediction_type="v_prediction",
    sigma_schedule="exponential",
)
pipe_edit.to(device)

prompt = "make it a cloudy day"
image = load_image(SampleURL.Mountain)
edited_image = pipe_edit(
    prompt=prompt, image=image, num_inference_steps=20
).images[0]

image_grid([image, edited_image], rows=1, cols=2)
```

## ControlNet

ControlNet是一种通过除了文本提示条件之外的其他条件来控制图像扩散模型的模型。ControlNet模型在原始模型的可训练副本上进行训练。与直接微调不同，ControlNet完全保留了原始模型，并将所有新条件注入到原始模型的可训练副本中。这使得即使您的ControlNet是用相对较少的样本训练的，也能保留模型的能力。

ControlNet模型被训练为接受各种条件，其中两个条件如图8-4所示：canny边缘和人体姿势（OpenPose）。除此之外，还有深度图、涂鸦、分割、线稿等。请查看Hugging Face上所有官方的Stable Diffusion v1-5 ControlNets，并查看Hugging Face Models以获取社区训练的模型。

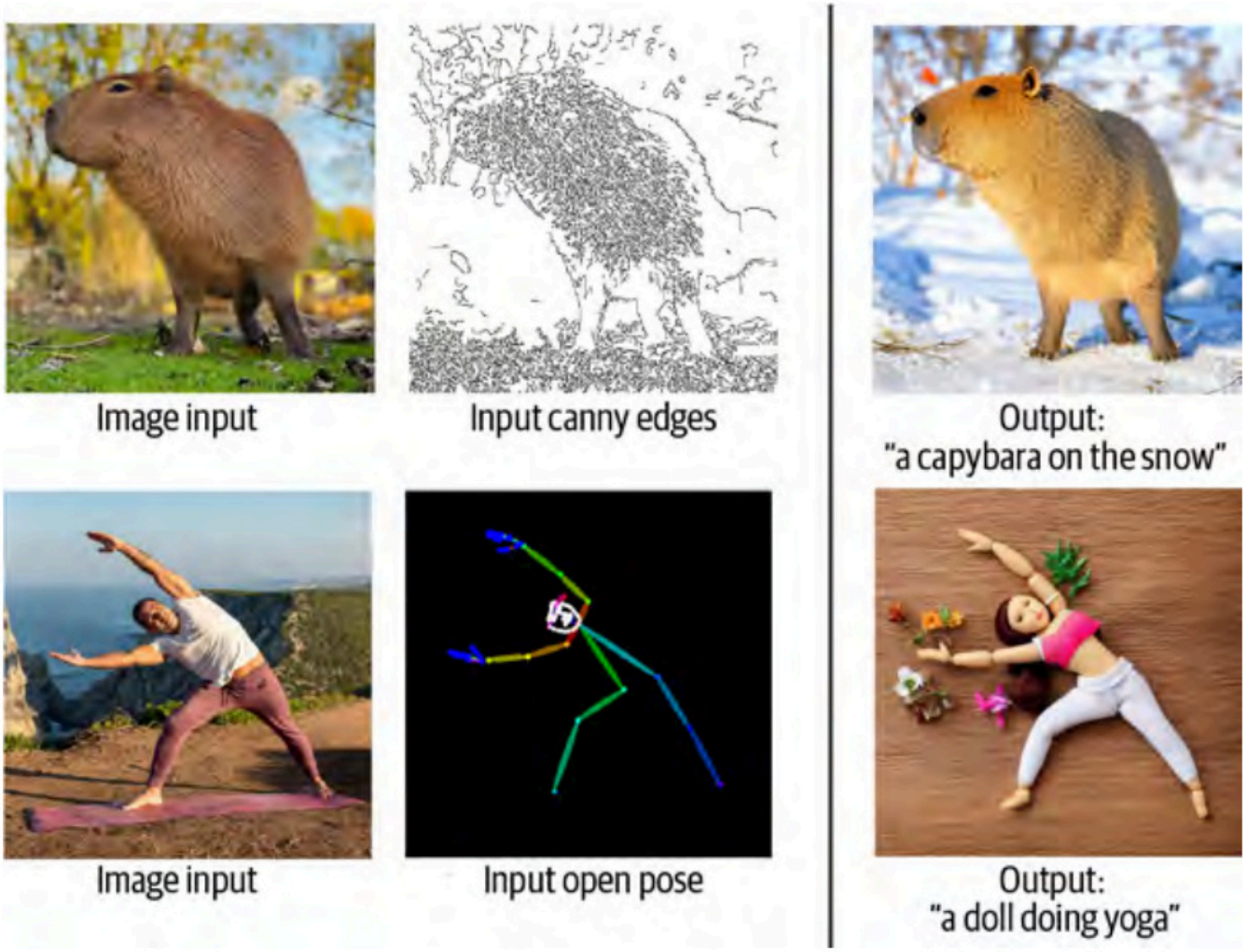



图8-4. 使用图像输入、canny边缘和open pose条件进行ControlNet示例，并生成输出

ControlNet的多功能性和高效性使其成为各种图像生成和图像处理任务的强大工具。通过允许对输出进行细粒度控制，同时保持原始模型的能力，ControlNet为创意和实际应用开辟了新的可能性，如图8-5所示。例如，它可以用于时尚设计领域，以在不同身体姿势上可视化服装；在建筑领域，根据粗略草图生成建筑设计；或在电影预制作中，从简单的线稿快速创建故事板。使用不同类型的控制输入（如边缘、姿势估计或深度图）的能力，为创意人员和开发人员提供了一个灵活的框架，以根据他们的特定需求指导图像生成过程。

图8-5. Duck ControlNet Canny示例

我们将使用一个辅助库controlnet_aux，将输入图像预处理为官方预训练ControlNet模型所需的条件格式。`diffusers`库的作者还为Stable Diffusion XL训练了ControlNets。您可以在此Hugging Face集合中找到它们。

首先，我们使用`StableDiffusionXLControlNetPipeline`加载主模型。该管道还期望一个`ControlNelModel`参数，该参数提供在去噪期间为UNet提供额外条件的模型：

```
from diffusers import ControlNetModel, StableDiffusionXLControlNetPipeline

controlnet = ControlNetModel.from_pretrained(
    "diffusers/controlnet-depth-sdxl-1.0",
    torch_dtype=torch.float16,
    variant="fp16",
)

controlnet_pipeline = StableDiffusionXLControlNetPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    controlnet=controlnet,
    torch_dtype=torch.float16,
    variant="fp16",
)
# controlnet_pipeline.enable_model_cpu_offload()  # Optional, saves VRAM
controlnet_pipeline = controlnet_pipeline.to(device)
```

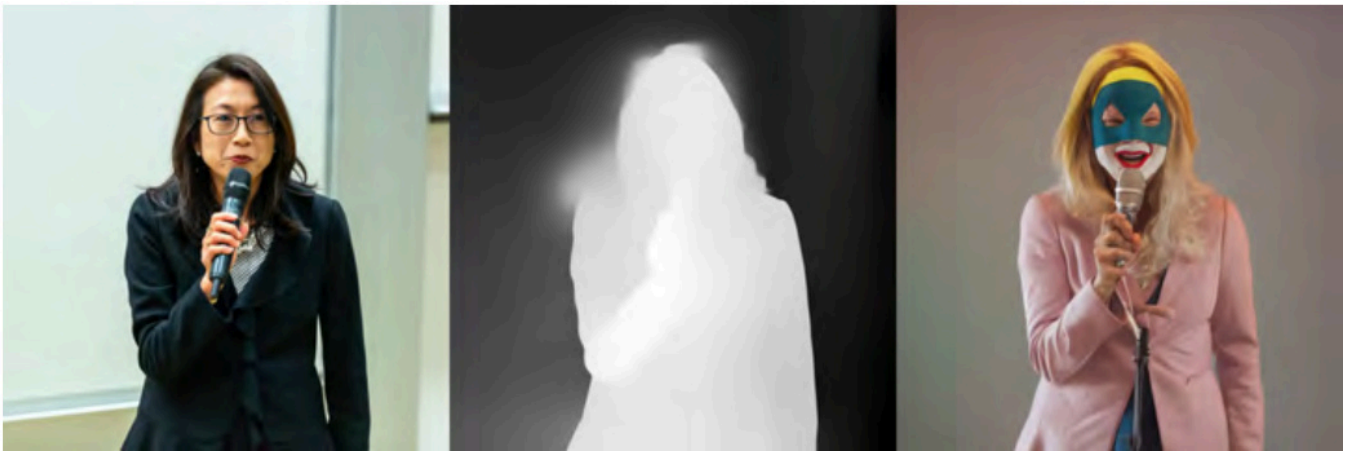
然后，我们可以使用`controlnet_aux`将图像预处理为所需的条件格式。在这里，我们使用`MidasDetector`预处理，因为它是一个深度估计模型。该模型接收图像输入并输出估计的深度

图，这正是我们需要为`diffusers/controlnet-depthsdxl-1.0`模型提供的：

```
from controlnet_aux import MidasDetector
from PIL import Image

original_image = load_image(SampleURL.WomanSpeaking, size=(1024, 1024))

# loads the MiDAS depth detector model
midas = MidasDetector.from_pretrained("lllyasviel/Annotators")

# Apply MiDAS depth detection
processed_image_midas = midas(original_image).resize(
    (1024, 1024), Image.BICUBIC
)
```

最后，我们可以将提示和处理的图像传递给管道以生成新图像。`controlnet_conditioning_scale`将决定条件对最终结果的影响程度：

```
image = controlnet_pipeline(
    "A colorful, ultra-realistic masked super hero singing a song",
    image=processed_image_midas,
    controlnet_conditioning_scale=0.4,
    num_inference_steps=30,
).images[0]
image_grid([original_image, processed_image_midas, image], rows=1, cols=3)
```



虽然训练您的ControlNets超出了本书的范围，但如果您对此主题感兴趣，我们建议您查看Hugging Face博客的使用diffusers训练您的ControlNet。

# 图像提示和图像变体

文本提示很好，但有时需要更多内容来表达我们对模型的意图。通过图像提示扩散模型，我们可以将输入范围扩展到视觉领域。

## 图像变体

为了激发我们的创造力，有时我们需要看一些相似但不同的东西。这就是图像变体的目的：对给定的图像进行重新诠释，提供一个熟悉但不同的图像生成。让我们探索两种方法——使用CLIP图像嵌入和IP-Adapter：

### 使用CLIP图像嵌入

正如你在第5章中学到的，Stable Diffusion使用CLIP作为其文本编码器。除了文本编码器，CLIP还可以用于生成图像嵌入。一些扩散模型被训练成可以使用图像嵌入作为输入来生成新的图像变体。Karlo和Kandinsky就是这种情况。对于Stable Diffusion来说，这并不是开箱即用的。然而，通过微调可以实现这一点。Stable Diffusion Image Variations是一个微调过的Stable Diffusion v1-5模型，它接受CLIP图像嵌入作为输入。你可以在Hugging Face上尝试它的演示。

### 使用IP-Adapter

另一种不需要微调模型的方法是使用预训练的IP-Adapters（图像提示适配器, mage Prompt Adapters）。这些适配器允许使用图像进行提示，从而实现图像变体以及图像提示的其他广泛用例，如风格迁移、主体身份保留和结构控制。

如图8-6所示，IP-Adapter由两个组件组成：一个从图像中提取特征的编码器和附加到预训练Stable Diffusion UNet上的解耦交叉注意力模块。

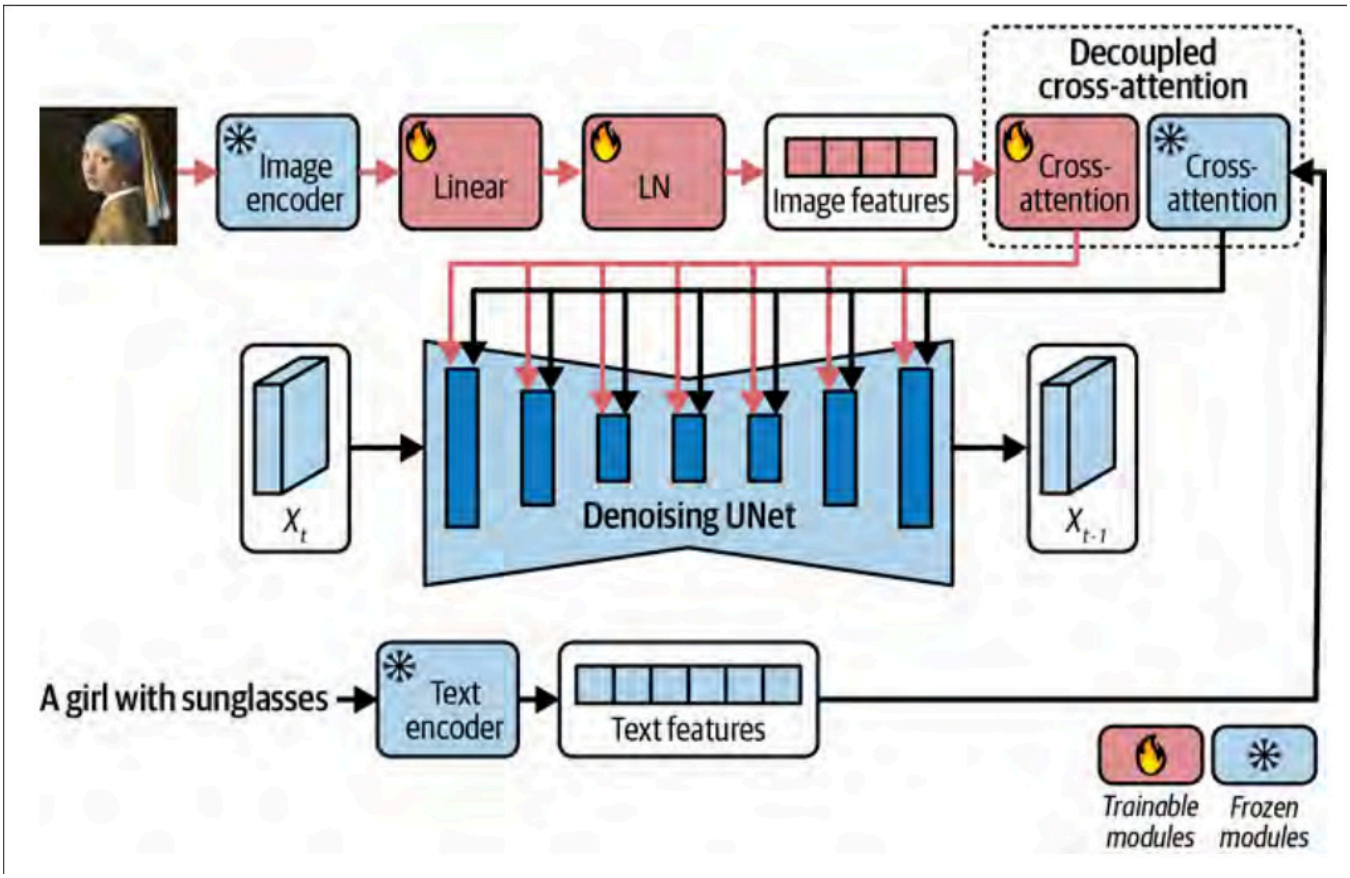



图8-6. IP-Adapter架构（改编自IP-Adapter论文中的图像）

使用IP-Adapter只需要对基础SDXL管道进行最小的更改：

- 使用`load_ip_adapter()`加载IP-Adapter模型。
- 使用`set_ip_adapter_scale()`指定IP-Adapter的缩放比例。

就是这样！要使用IP-Adapter进行图像变体，我们可以提供参考图像和一个空提示，结果如下：

```
from diffusers import StableDiffusionXLPipeline

sdxl_base_pipeline = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16,
    variant="fp16",
)
sdxl_base_pipeline.to(device)

# We load the IP Adapter too
sdxl_base_pipeline.load_ip_adapter(
    "h94/IP-Adapter", subfolder="sdxl_models", weight_name="ip-adapter_sdxl.bin"
)

# We can set the scale of how strong we
# want our IP Adapter to impact our overall result
sdxl_base_pipeline.set_ip_adapter_scale(0.8)

image = load_image(SampleURL.ItemsVariation)
original_image = image.resize((1024, 1024))

# Create the image variation
generator = torch.Generator(device=device).manual_seed(1)
variation_image = sdxl_base_pipeline(
    prompt="",
    ip_adapter_image=original_image,
    num_inference_steps=25,
    generator=generator,
).images

image_grid([original_image, variation_image[0]], rows=1, cols=2)
```

使用IP-Adapter，我们生成了图像的重新诠释（你可以看到它去掉了奶油——我们可以接受！），这非常酷，并且启发了有趣的用例。然而，IP-Adapter比我们目前所做的要强大得多。图像提示允许将IP-Adapter与文本提示以及你在本章中学到的其他控制方法结合起来。

## 图像提示

IP-Adapter不仅仅用于生成图像变体。它允许你将图像作为提示之一，从而使你能够应用风格迁移等技术，以及你在本章中学到的所有其他技术，添加图像提示和文本提示。

风格迁移

虽然IP-Adapter在风格迁移方面开箱即用，但InstantStyle论文的研究人员发现，如果IP-Adapter仅应用于Stable Diffusion模型的UNet的某些块，它可以专门影响图像风格。其思想是，我们可以向模型传递提示和风格图像，模型将生成遵循提示但具有风格图像风格的图像。在这个例子中，我们将应用巴西画家Tarsila do Amaral的作品风格，“O Mamoeiro”。主要区别在于IP-Adapter的缩放比例和输入提示，它不再是空的：

```
# We load the model and the IP Adapter, just as before
pipeline = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0", torch_dtype=torch.float16
).to(device)

# Load the IP Adapter into the model
pipeline.load_ip_adapter(
    "h94/IP-Adapter", subfolder="sdxl_models", weight_name="ip-adapter_sdxl.bin"
)

# We are applying the IP Adapter only to the mid block,
# which is where it should be mapped to the style in SDXL
scale = {"up": {"block_0": [0.0, 1.0, 0.0]}}
pipeline.set_ip_adapter_scale(scale)

image = load_image(SampleURL.Mamoeiro)
original_image = image.resize((1024, 1024))

# Run inference to generate the stylized image
generator = torch.Generator(device=device).manual_seed(0)
variation_image = pipeline(
    prompt="a cat inside of a box",
    ip_adapter_image=original_image,
    num_inference_steps=25,
    generator=generator,
).images

image_grid([original_image, variation_image[0]], rows=1, cols=2)
```

附加控制

现在，为了总结本章，我们可以展示我们学到的多种技术如何组合在一起。我们将把IP-Adapter的“O Mamoeiro”风格添加到我们之前的ControlNet蒙面歌手示例中：

```
controlnet = ControlNetModel.from_pretrained(
    "diffusers/controlnet-depth-sdxl-1.0", torch_dtype=torch.float16
)

# Load the ControlNet pipeline
controlnet_pipeline = StableDiffusionXLControlNetPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    controlnet=controlnet,
    torch_dtype=torch.float16,
    variant="fp16",
)
controlnet_pipeline.to(device)

# Load the IP Adapter
controlnet_pipeline.load_ip_adapter(
    "h94/IP-Adapter", subfolder="sdxl_models", weight_name="ip-adapter_sdxl.bin"
)
# We are applying the IP Adapter only to the mid block,
# which is where it should be mapped to the style in SDXL
scale = {
    "up": {"block_0": [0.0, 1.0, 0.0]},
}
controlnet_pipeline.set_ip_adapter_scale(scale)

# Load the original image
original_image = load_image(SampleURL.WomanSpeaking)
original_image = original_image.resize((1024, 1024))

# Load the style image
style_image = load_image(SampleURL.Mamoeiro)
style_image = style_image.resize((1024, 1024))

# Apply the MiDAS depth estimation
processed_image_midas = midas(original_image).resize(
    (1024, 1024), Image.BICUBIC
)

image = controlnet_pipeline(
    "A masked super hero singing a song",
```

```
    image=processed_image_midas,
    ip_adapter_image=style_image,
    controlnet_conditioning_scale=0.5,
).images[0]
image_grid(
    [original_image, style_image, processed_image_midas, image], rows=1, cols=4
)
```

## 项目时间：你的创意画布

现在是你的创意画布🎨。在本章中，我们提供了多种创意应用和表达的机制。现在是时候练习了；你在本章的挑战是尝试以你自己的创意方式结合至少两种我们在这里介绍的技术——类似于我们如何以“回忆录”风格创建蒙面超级英雄。以下是一些指导你创意之旅的想法：

- 使用 ControlNet 的 canny 边缘检测，以你珍视的参考风格（IP-Adapter 风格）重新构想你的生活空间。
- 在纸上绘制一个城市的粗略草图，拍照后应用图像到图像转换，将这座城市变成一个 Solarpunk 乌托邦。现在，以你通过草图生成的 Solarpunk 乌托邦风格为参考，在你的新城市中创建建筑、交通系统和居民。
- 或者找到其他方式来探索这个潜在空间！

## 总结

在本章中，我们探索了各种创意应用，这些应用通过细粒度的可控性提供了更多的控制，并扩展了文本到图像模型的能力，增加了这些模型可以接受的输入范围。由于这些技术是可组合且相互兼容的，我们允许复杂的创意流程和艺术过程超越仅仅输入文本并获取图像作为输出。通过结合图像变换、变体、风格或结构图像参考，以及更细粒度的提示控制，艺术家和创意专业人士可以将机器学习工作流程纳入他们的工具箱。

然而，超越生成图像，尤其是在将真实图像引入模型的潜在空间时，带来了新的挑战。关键的伦

理问题包括虚假信息、欺骗和所有权。使图像编辑对每个人（而不仅仅是Adobe Photoshop专家等）可访问，创造了新的机遇和挑战，例如图像操纵可能被用来传播虚假信息，并通过深度伪造或不存在的内容欺骗人们。所有权问题也是关键：艺术风格可以被重新混合，但何时这种重新混合是公平的，其法律和伦理边界仍然是一个悬而未决的问题。像水印这样的缓解策略已经在diffusers等库中存在；然而，关于如何处理机器学习赋予我们的这些新能力，社会层面还需要进行更多的讨论。

尽管如此，这些新模型的创意潜力是一个令人兴奋的发展。当以道德和负责任的方式使用时，它们无疑将为创意人士提供超越想象的工具。

为进一步阅读，我们建议以下资源：

- “IP-Adapter: 你需要知道的一切”
- “使用diffusers训练你的ControlNet”
- Diffusers修复指南
- “使用InstructPix2Pix进行指令调优的Stable Diffusion”
- 第290页的“参考文献”中的论文

# 练习

1. 解释修复与图像到图像变换的区别，并提供一个实际应用的例子。
2. 提示权重如何帮助克服扩散模型的局限性？
3. Prompt-to-Prompt编辑与SEGA之间的关键区别是什么？
4. ControlNet如何增强扩散模型的能力？给出可以与ControlNet一起使用的条件的例子。
5. 在文本到图像模型的背景下，“反转”是什么，它允许我们做什么？

你可以在本书的GitHub仓库中找到这些练习的答案。

# 参考文献

Brack, Manuel, 等. “LEDITS $^{++}$ : 使用文本到图像模型进行无限图像编辑.” arXiv, 2023年11月30日. http://arxiv.org/abs/2311.16711.
Brack, Manuel, 等. “SEGA: 使用语义指导的文本到图像模型.” arXiv, 2023年1月29日. http://arxiv.org/abs/2301.12247.
Brooks, Tim, 等. “InstructPix2Pix: 学习遵循图像编辑指令.” arXiv, 2022年11月17日. https://arxiv.org/abs/2211.09800.
Chefer, Hila, 等. “Attend-and-Excite: 基于注意力的文本到图像扩散模型语义指导.” arXiv, 2023

年1月31日. http://arxiv.org/abs/2301.13826.
Hertz, Amir, 等. “Prompt-to-Prompt: 使用交叉注意力控制的图像编辑.” arXiv, 2022年8月2日. http://arxiv.org/abs/2208.01626.
Mokady, Ron, 等. “Null-Text Inversion: 使用引导扩散模型编辑真实图像.” arXiv, 2022年11月17日. http://arxiv.org/abs/2211.09794.
Podell, Dustin, 等. “SDXL: 改进高分辨率图像合成的潜在扩散模型.” arXiv, 2023年7月4日. http://arxiv.org/abs/2307.01952.
Wang, Haofan, 等. “InstantStyle: 文本到图像生成中风格保留的免费午餐.” arXiv, 2024年4月4日. http://arxiv.org/abs/2404.02733.
Ye, Hu, 等. “IP-Adapter: 文本到图像扩散模型的文本兼容图像提示适配器.” arXiv, 2023年8月14日. http://arxiv.org/abs/2308.06721.
Zhang, Lvmin, 等. “为文本到图像扩散模型添加条件控制.” arXiv, 2023年2月11日. https://arxiv.org/abs/2302.05543.

# 第9章. 生成音频

在第1章中，我们通过基于Meta的MusicGen模型的transformers管道瞥见了音频生成的潜力。本章深入探讨生成音频，使用扩散和基于transformer的技术，这将引入一系列新的令人兴奋的挑战和应用。想象一下，如果你可以在通话期间实时去除所有背景噪音，如果你可以获得高质量的会议转录和摘要，或者如果一位歌手可以用其他语言重新生成他们的歌曲。你甚至可以生成莫扎特和Billie Eilish的作曲主题，并赋予其墨西哥风情的转折。嗯，这就是该领域的轨迹，未来令人兴奋。

我们可以用机器学习和音频做些什么？最常见的两个任务是将语音转录为文本（自动语音识别，ASR）和从文本生成语音（文本到语音，TTS）。在ASR中，模型接收某人（或多个人）说话的音频作为输入，并输出相应的文本。对于某些模型，输出还捕获了其他信息，例如谁在说话或某人说话的时间。ASR系统被广泛使用，从虚拟语音助手到字幕生成器。由于近年来许多开放访问模型向公众提供，关于多语言性和在边缘设备上直接运行模型的研究取得了令人兴奋的进展。

在文本到语音（TTS）中，模型生成合成的、希望是逼真的语音。与ASR一样，在TTS中，人们对在设备上运行模型以及多语言性也有相当大的兴趣。TTS也提出了自己的一系列挑战，例如生成多个说话者的音频，使声音听起来更自然，并在生成中引入语调、停顿、情感标记、音高控制、口音和其他特征。

尽管TTS和ASR是最受欢迎的任务，我们还可以用机器学习和音频做许多其他事情（其中一些如

图9-1所示）：

**文本到音频**

文本到语音可以推广到文本到音频（TTA），其中，基于提示，模型可以生成旋律、音效和歌曲。

**语音克隆**

一个人的声音，包括音调、音高和韵律，被保留以生成新的声音。

**音频分类**

模型对提供的音频进行分类。典型的例子是命令识别和说话者识别。

**语音增强**

模型去除音频中的噪音，使声音更清晰。

**音频翻译**

模型接收源语言$X$的音频，并输出目标语言Y的音频。

**说话者分割**

模型识别特定时间的说话者。

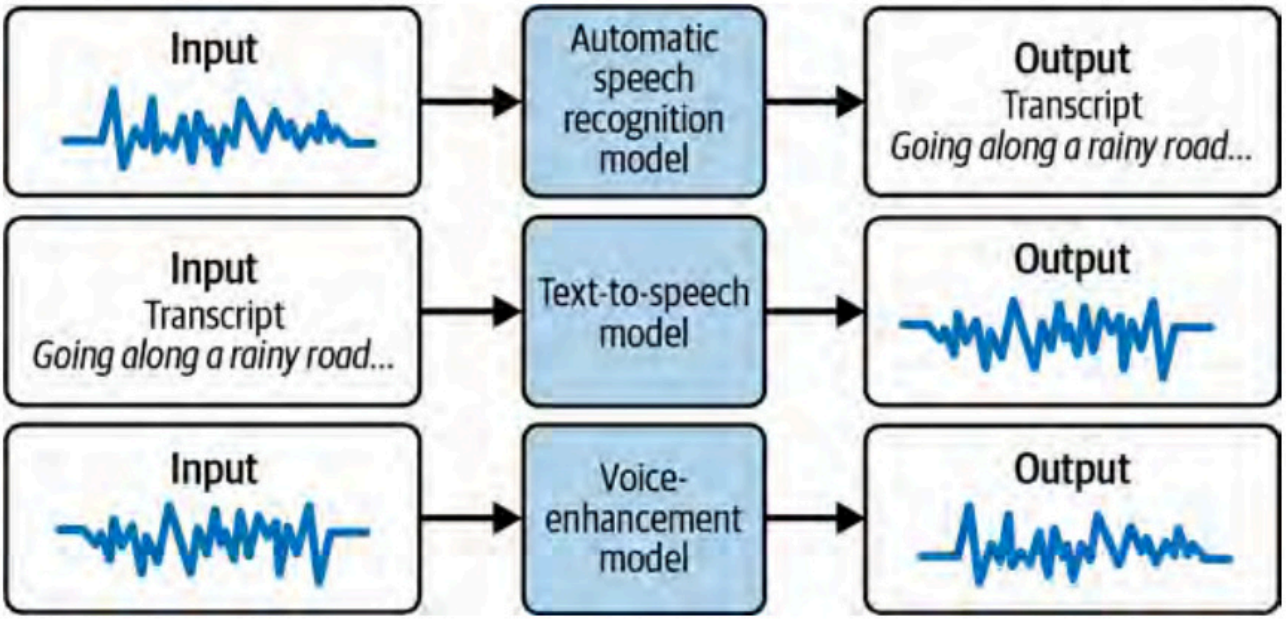


图9-1. 音频任务的例子

音频相关任务具有挑战性，原因有很多。首先，处理原始音频信号比处理文本更复杂且更不直观。许多应用场景要求音频模型能够实时或在设备端运行，这可能会限制模型大小和推理速度。例如，当前的扩散模型如果用于交互式翻译，速度会太慢。最后，评估生成音频模型可能具有挑战性。你如何衡量模型生成的歌曲质量是否好?

我们可以使用多种工具和数百个开放访问的模型和数据集来完成这些任务。Common Voice是Mozilla基金会的一个流行的众包数据集，包含超过2,000小时的音频文件及其对应的文本，涵盖一百多种语言。许多其他流行的音频数据集，如LibriSpeech、VoxPopuli和GigaSpeech，也可用，每个数据集都有其自己的领域和用例。正如有许多开源数据集一样，我们也可以使用许多开放访问的模型。本章将探索基于transformer的模型，如Meta的Wav2Vec2、OpenAI Whisper、Microsoft SpeechT5和Suno Bark。我们还将探索可以生成歌曲的令人兴奋的扩散模型，如Stable Diffusion（但用于歌曲）、Dance Diffusion和AudioLDM。尽管跳入另一种模态可能令人望而生畏，但我们在生成旅程中收集的许多工具即将被使用。

# 音频数据

首先，我们将展示音频数据的结构以及如何使用它。我们将探索 LibriSpeech 数据集，该数据集包含超过 1,000 小时的书籍朗读音频，对于训练和评估语音识别系统非常有用。音频数据集的第一个挑战是它们往往非常大，因此同时加载所有数据可能不可行。音频数据集可能会迅速生成数 TB 的数据，无法存储在硬盘中。

我们可以使用 `load_dataset_builder()` 来更好地了解数据集的结构，而无需加载所有数据：

```
from datasets import load_dataset_builder
ds_builder = load_dataset_builder( "openslr/librispeech_asr",
    trust_remote_code=True
)
ds_builder.info.splits
{'test.clean': SplitInfo(name='test.clean', num_bytes=368449831,
    num_examples=2620, shard_lengths=None, dataset_name=None),
'test.other': SplitInfo(name='test.other', num_bytes=353231518,
    num_examples=2939, shard_lengths=None, dataset_name=None),
'train.clean.100': SplitInfo(name='train.clean.100', num_bytes=6627791685,
    num_examples=28539, shard_lengths=None, dataset_name=None),
'train.clean.360': SplitInfo(name='train.clean.360', num_bytes=23927767570,
    num_examples=104014, shard_lengths=None, dataset_name=None),
'train.other.500': SplitInfo(name='train.other.500', num_bytes=31852502880,
    num_examples=148688, shard_lengths=None, dataset_name=None),
'validation.clean': SplitInfo(name='validation.clean', num_bytes=359505691,
    num_examples=2703, shard_lengths=None, dataset_name=None),
'validation.other': SplitInfo(name='validation.other', num_bytes=337213112,
    num_examples=2864, shard_lengths=None, dataset_name=None)}
```

数据集作者发现，语料库的规模使得处理它变得不切实际，因此他们决定将其分为 100、360 和 500 小时的子集。这是有道理的，毕竟仅训练数据就超过 60 GB。在不加载所有这些数据的情况下，我们可以通过使用 `.info.features` 开始查看特征：

```
ds_builder.info.features

{'file': Value(dtype='string', id=None),
'audio': Audio(sampling_rate=16000, mono=True, decode=True, id=None),
'text': Value(dtype='string', id=None),
'speaker_id': Value(dtype='int64', id=None),
'chapter_id': Value(dtype='int64', id=None),
'id': Value(dtype='string', id=None)}
```

最有价值的特征是 `text` 和 `audio`。我们拥有构建初始语音识别管道所需的所有数据：音频及其对应的转录。在底层，每个特征都有一个类型。例如，`text` 类型是一个 `Value` 特征，它包含数据类型 `string`。`audio` 类型是一个 `Audio` 特征，它包含音频信息。就像图像一样，音频可以用多个通道表示。`mono` 属性指示音频是单声道（单通道，提供统一的听觉体验）还是立体声（双通道，提供方向感）。我们将在下一节讨论 `sampling_rate` 和 `decode` 的含义。

鉴于数据集如此庞大，我们需要找到高效处理它的方法。与其下载整个数据集然后使用它，您可以使用流模式一次加载一个示例，从而不占用磁盘空间，并能够在下载时使用数据集中的样本。当使用 `datasets` 库的流模式时，我们会得到一个 `IterableDataset`，它可以像任何 Python 迭代器一样使用。让我们看看 100 小时分割的第一个示例：

```
ds = load_dataset( "openslr/librispeech_asr", split="train.clean.360",
    streaming=True,
)
sample = next(iter(ds))
sample
{'audio': {'array': array([ 9.15527344e-05, 4.57763672e-04, 5.18798828e-04, ...,
     -4.57763672e-04, -5.49316406e-04, -4.88281250e-04]),
     'path': '1487-133273-0000.flac', 'sampling_rate': 16000},
'chapter_id': 133273,
'file': '1487-133273-0000.flac',
'id': '1487-133273-0000',
'speaker_id': 1487,
'text': 'THE SECOND IN IMPORTANCE IS AS FOLLOWS SOVEREIGNTY MAY BE '
        'DEFINED TO BE THE RIGHT OF MAKING LAWS IN FRANCE THE KING '
        'REALLY EXERCISES A PORTION OF THE SOVEREIGN POWER SINCE '
        'THE LAWS HAVE NO WEIGHT'}
```

样本为我们提供了音频和对应的文本。`audio` 条目是包含以下内容的特征类型：

- 一个包含解码后音频数据的数组。回想一下，`audio` 特征中的 `decode` 设置为 `True`，这意味着音频已经为您解码。否则，音频包含字节，您需要自己解码它。
- 下载的音频文件的路径。
- 数据 `sampling_rate`，这对于正确加载音频至关重要。

如果这些概念对您来说很陌生，不用担心。这是学习音频到底是什么的绝佳机会。

音频是随时间变化的无限值集。计算机无法处理连续数据，因此我们需要处理音频信号并拥有数字离散（有限）表示。为此，我们在给定时间内拍摄许多快照。这称为采样率。例如，对于此音频样本，我们可以在 `audio` 特征中找到采样率为 16,000（单位为赫兹）。这意味着在给定时间内拍摄了 16,000 个样本。如果我们有一分钟的音频文件，那几乎是百万个值——难怪音频数据集如此庞大。图 9-2 显示了使用采样率为 6 的音频波形。

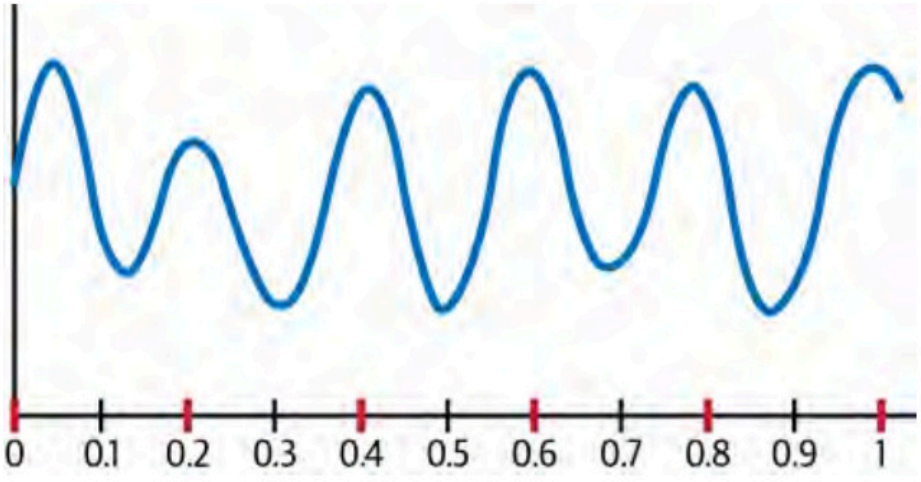



图 9-2. 在此使用采样率为 6 的波形中，离散化发生在间隔 1/6 秒的位置

采样率是一个重要参数：在处理音频机器学习时，我们需要确保所有音频样本具有相同的采样率。模型是在特定采样率下预训练的，因此在微调或运行推理时，您需要确保使用相同的采样率。尽管一些最流行的音频数据集的采样率为 16,000，但这并不总是如此，因此您必须在预处理阶段重新采样数据：

```
array = sample["audio"]["array"]
sampling_rate = sample["audio"]["sampling_rate"]

# Let's get the first 5 seconds
array = array[: sampling_rate * 5]
print(f"Number of samples: {len(array)}. Values: {array}")

Number of samples: 80000. Values:
[9.15527344e-05 4.57763672e-04 5.18798828e-04 ... 7.05261230e-02
 5.92041016e-02 6.50329590e-02]
```

当然，我们无法在书中打印音频文件，但您可以运行一些代码来收听音频。我们可以使用 `IPython.display.Audio()` 函数来实现这一点。或者，您可以访问官方交互式演示以收听本章的所有音频样本。让我们收听 100 小时分割中的第一个音频样本：

```
import IPython.display as ipd
ipd.Audio(data=array, rate=sampling_rate)
```

> 您可能对 `Audio` 特征中的 `decode` 和 `mono` 感到好奇。 `decode` 属性指定数据是否应解码（返回为浮点数数组）或不解码（返回为字节）。 `mono` 属性指定音频是单声道（一个通道）还是多通道。我们将在接下来的章节中探索这些示例。

## 波形

我们刚刚看到，我们使用音频的数字离散表示来能够处理它。在底层，音频只是一个值的数组。这些数组包含了许多任务训练模型所需的信息，因此在进入应用之前，花时间理解这个数组是值得的。

这个数组代表什么？数组中的每个值代表振幅，它描述了声波的强度，并以分贝为单位进行测量。[1] 振幅告诉我们声音相对于参考值的响度；0 dB，即参考值，代表人类耳朵能感知到的最低声音。你的呼吸声大约为 10 dB，一场激烈的音乐会可能达到 120 dB（开始感到疼痛），而 1883 年克拉卡托火山爆发的声音估计为 310 dB，甚至在 3,000 英里（4,800 公里）外都能听到，如图 9-3 所示。

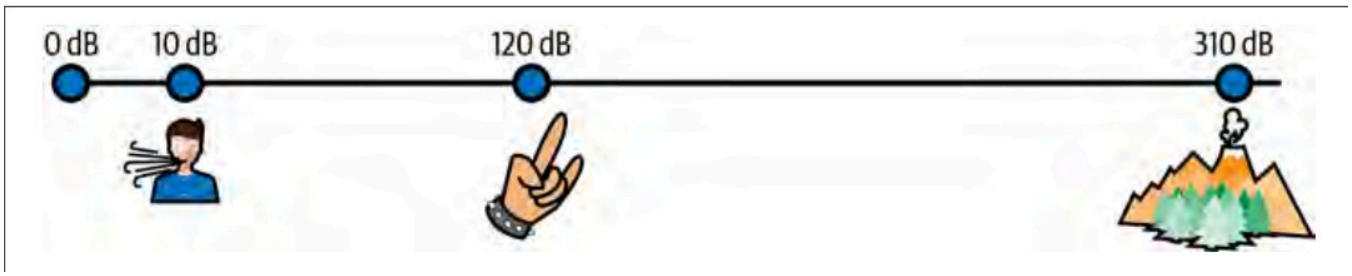
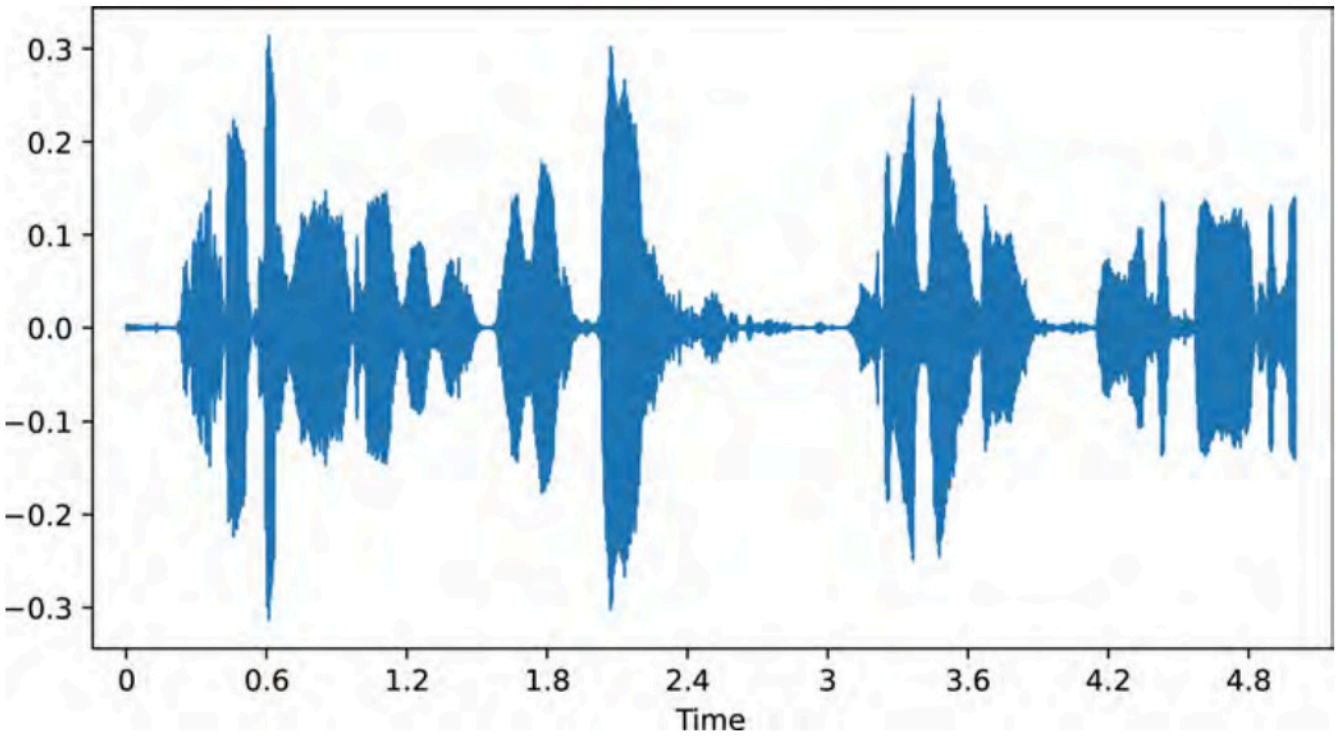



图 9-3. 一些声音在分贝尺度上的振幅

振幅通常以分贝为单位进行测量，但数组经常被归一化，因此数值范围在 -1 到 1 之间。为了可视化音频，我们可以使用波形，即振幅随时间变化的图。让我们使用 librosa 这个流行的音频数据处理库来绘制波形：

```
import librosa.display

librosa.display.waveshow(array, sr=sampling_rate);
```




波形有助于对音频数据进行初步探索。如果你听这段音频，你可以识别出最初的波形对应着朗读者说“the second in importance is as follows”之后的一个短暂停顿。更一般地说，波形是一种直观的方式来识别音频中的不规则性，并对信号及其模式有一个整体的了解。

## 频谱图

波形和频谱图（图9-4）是表示音频信号的不同方式。本节将解释什么是频谱图以及何时使用它们。

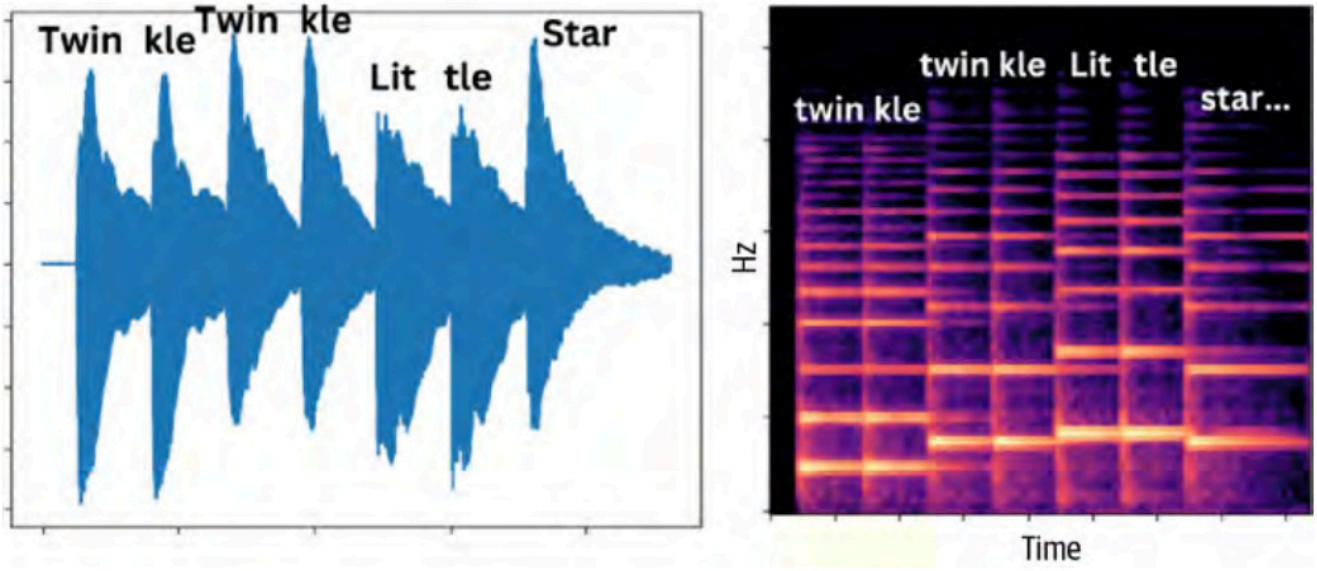



图9-4. 表示同一音频片段的波形（左）和梅尔频谱图（右）

假设我们想训练一个生成模型，以根据输入音频生成新的声音或合成语音。一种直接的方法可能是使用原始音频波形作为模型的输入。波形是声音的直接且直观的表示，包含了重现原始音频所需的所有信息。你可以通过听数据集中的一些样本来轻松验证这一点，无论它们包含人声、音乐还是其他声音。那么，如果波形能够准确表示声音，为什么我们不使用它们来训练模型呢?

第一个挑战在于波形的维度。尽管波形是一维的，但它们由大量数据点组成。每秒钟的音频有数万个样本需要模型处理，这使得学习数据中的模式和结构变得困难。这种高维度使得模型难以有效地学习数据中的模式和结构。

此外，我们的听觉系统对音高或音色等属性的微小差异非常敏感，这些属性与声音的频率特性相关。虽然这些属性被编码在波形的形状中，但仅通过观察波形来识别它们——即使对人类来说——也是具有挑战性的。我们可以从波形中提取的最明显信息是振幅随时间的变化，通常称为时域。然而，与频率相关的属性，例如钢琴和小提琴声音不同的原因，仅从波形中辨别要困难得多（图9-5）。

虽然可以使用原始波形数据训练模型（如我们将在本章后面探讨的那样），但涉及的样本数量巨大，带来了重大挑战。例如，以16,000的采样率录制的一分钟音频包含近一百万个样本。如果我们试图通过平均或组合样本来降低维度，我们可能会丢失在频域中至关重要的细微差异。即使采样率为16,000，对于语音来说是足够的，但对于高质量音乐来说是不够的。从CD到流媒体服务的消费音频标准通常使用$44.1\text{kHz}$或更高的采样率是有原因的。

鉴于这些挑战，一种更有效的方法通常是将音频波形转换为称为频谱图的不同表示形式。这种紧

凑的视觉表示形式描绘了声音的频率和振幅随时间的变化。通过明确捕捉频率细节，频谱图提供了音频信号的结构化和信息丰富的表示，这使得模型更容易从中学习。频谱图将问题从时域转换到时频域，这通常更适合涉及音频的机器学习任务。然而，需要注意的是，频谱图是一种有损表示，意味着它们不会保留原始波形中的所有信息。尽管如此，这种损失通常是可以接受的，因为频谱图保留了最感知相关的特征。

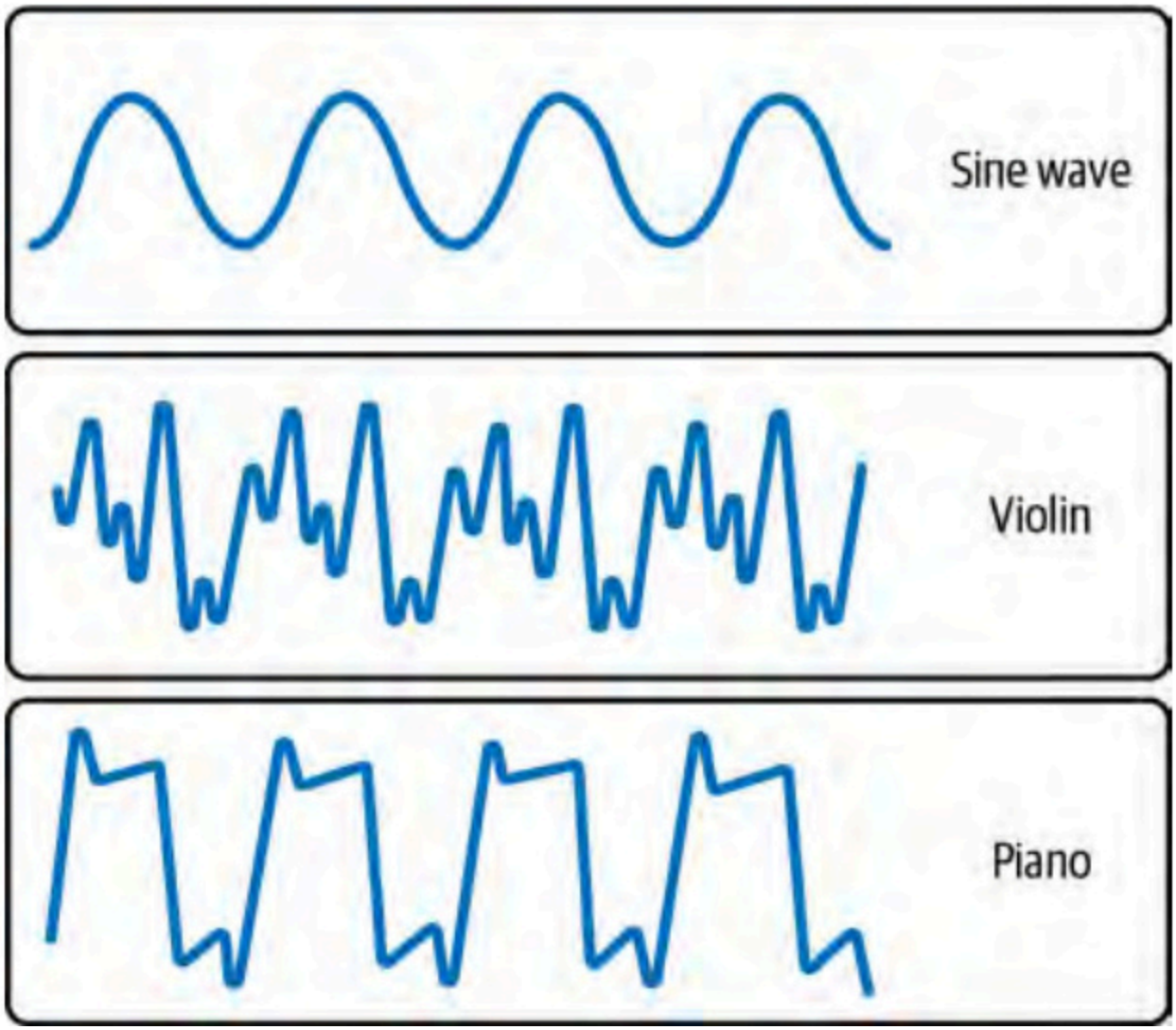
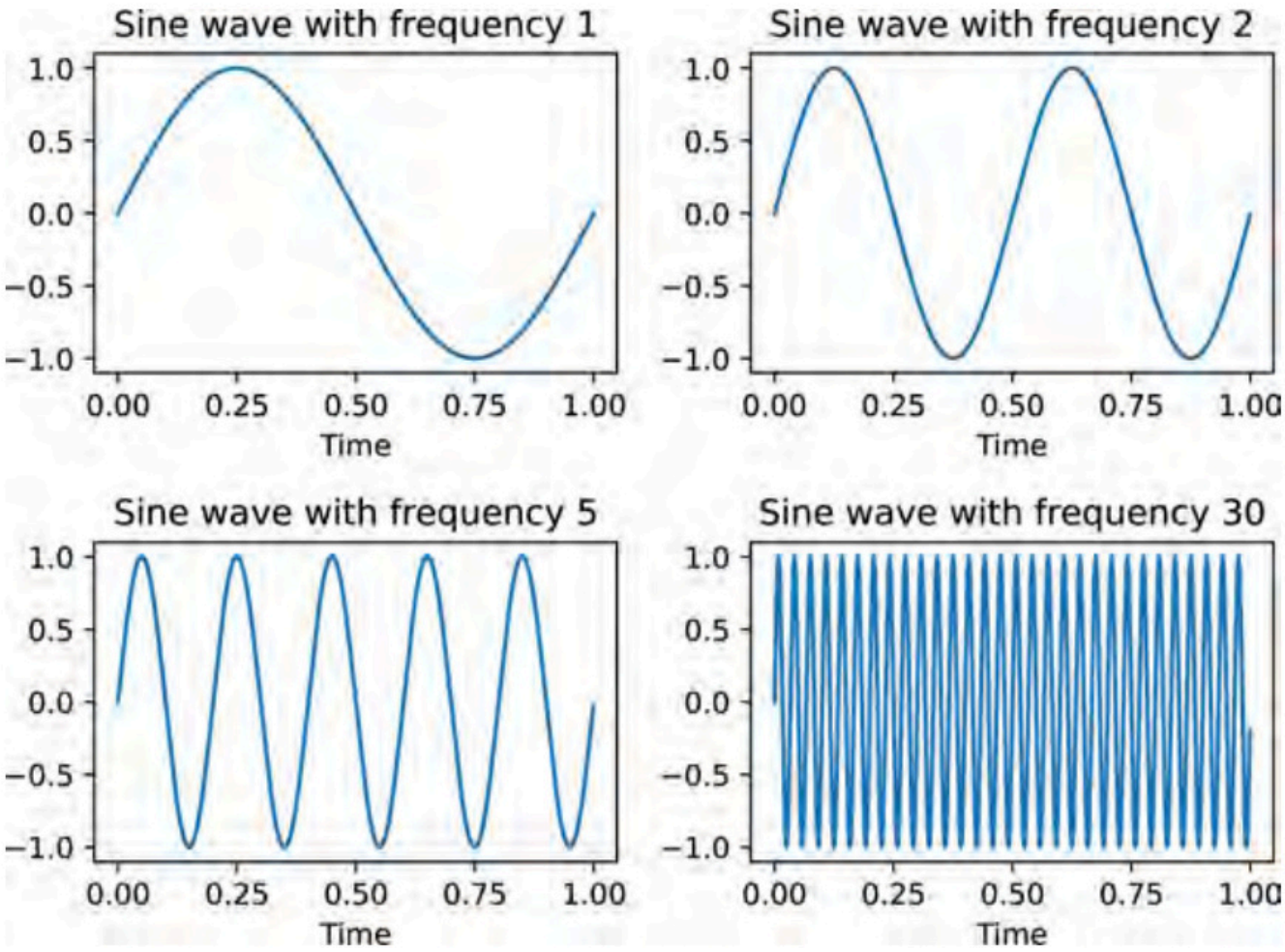



图9-5. 几种乐器演奏的相同音符具有相同的音高（频率）和振幅，但由于波形形状的变化，听起来不同。纯音符是完美的正弦波。

在我们深入探讨频谱图之前，让我们先看看频率。我们将绘制四个具有相同振幅范围但频率不同的波：

```
import numpy as np
from matplotlib import pyplot as plt


def plot_sine(freq):
    sr = 1000  # samples per second
    ts = 1.0 / sr  # sampling interval
    t = np.arange(0, 1, ts)  # time vector
    amplitude = np.sin(2 * np.pi * freq * t)

    plt.plot(t, amplitude)
    plt.title("Sine wave with frequency {}".format(freq))
    plt.xlabel("Time")


fig = plt.figure()

plt.subplot(2, 2, 1)
plot_sine(1)

plt.subplot(2, 2, 2)
plot_sine(2)

plt.subplot(2, 2, 3)
plot_sine(5)

plt.subplot(2, 2, 4)
plot_sine(30)

fig.tight_layout()
plt.show()
```




正如你所观察到的，尽管这些波具有相同的振幅范围，但它们表现出不同的频率。虽然这些是简单的正弦波，但现实世界中的声音更为复杂。通常，声音将是多个波的组合，每个波都有其频率和振幅。因此，我们的第一个任务是将声波分解为多个更简单的组成部分。为什么这有用？这个过程允许我们提取模型可以利用的有价值信息：在不同频率下，振幅如何随时间变化？我们如何分解声音？我们使用傅里叶变换（FT），这是一种数学工具，可以将单个函数分解为多个函数。

让我们从一些简单的正弦函数开始，如图9-6所示。在第一列中，我们有正弦函数。在第二列中，我们有FT的图，即频域中的函数。

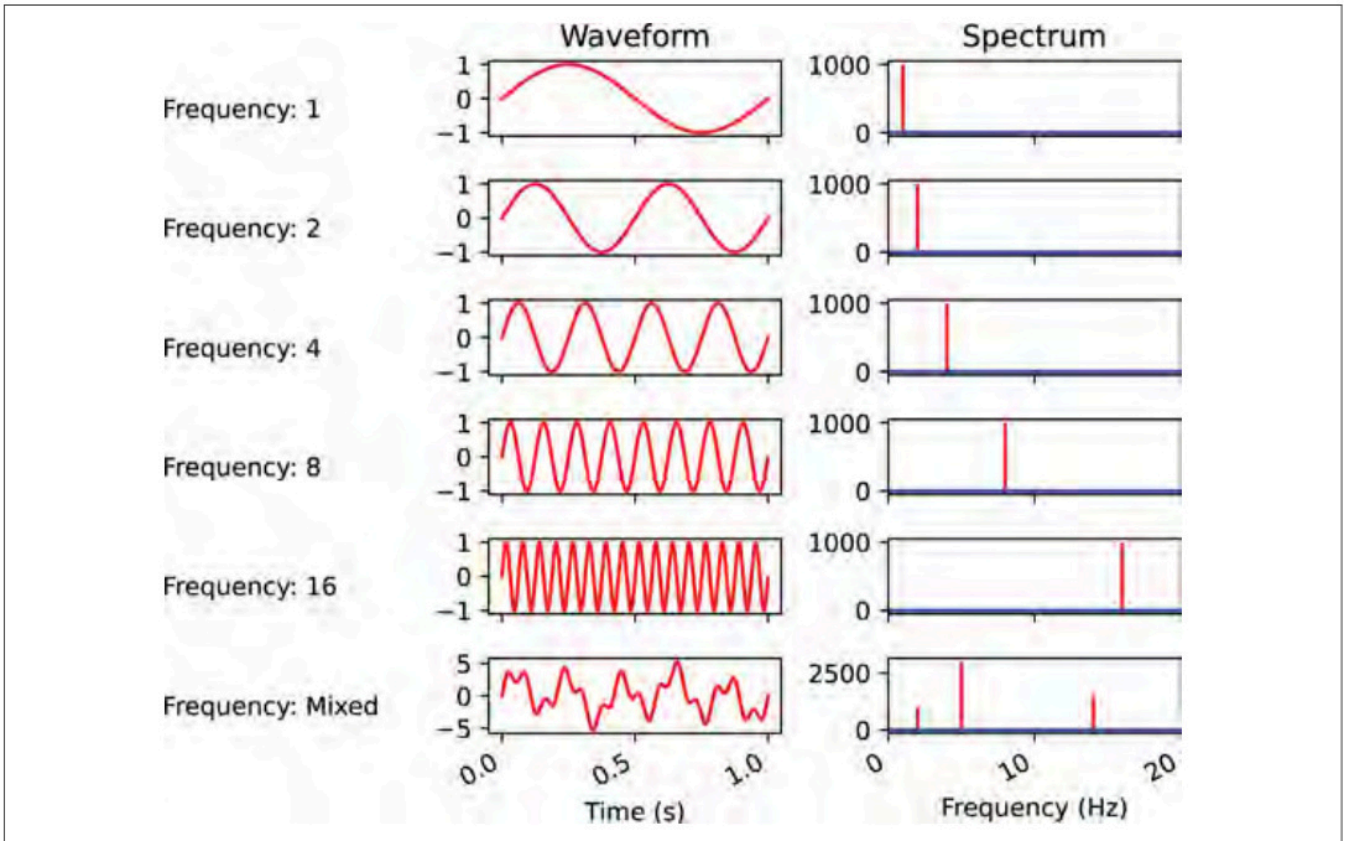



图9-6. 一些正弦函数及其频谱。纯音符（正弦波）在正弦周期处有一个峰值，而更复杂的声音在频域中显示多个峰值。

让我们分析第一行。在左侧，我们有原始波形——一个频率为每秒一个周期的正弦波。在右侧，FT图在x轴上表示频率，在y轴上表示频域中的振幅。观察到在1处的峰值，这与原始波形的频率一致。接下来的几行显示了不同频率的示例，并表现出相同的行为：FT图在原始波形的频率处达到峰值。y值是样本数的一半（在本例中，采样率为2,000，且仅为一秒，因此我们有2,000个样本）乘以原始波形振幅中的振幅（在前五行中为1）。

现在，让我们探讨最后一行——一个更有趣的案例。该波形结合了三个不同的正弦函数，具有不同的振幅（1、3和1.5）和频率（2、5和14）。仅通过观察波形很难辨别其组成，因此这就是频域有用的地方。在频域中，我们可以观察到三个峰值，对应于原始函数的频率：2、5和14。因此，我们可以逆向工程并描述初始波形的三个函数。

在$2\text{Hz}$的频率下，频域振幅（右侧的y轴）为1,000。通过计算$1,000 \times 2/2,000$，我们得到1，即组成波形的第一个正弦函数的振幅。同样，$3\text{Hz}$的频率产生频域振幅为3,000，通过计算$3,000 \times 2/2,000$，我们得到3。分解后的正弦波如图9-7所示。傅里叶变换为我们提供了一种机制，使我们能够分析复杂波形并理解组成它们的频率。我们揭示了声音中隐藏的复杂性，这些信息我们原本以为不存在，但实际上存在。这提供了更多的信息，并将成为转录语音或生成音乐

的模型的关键。

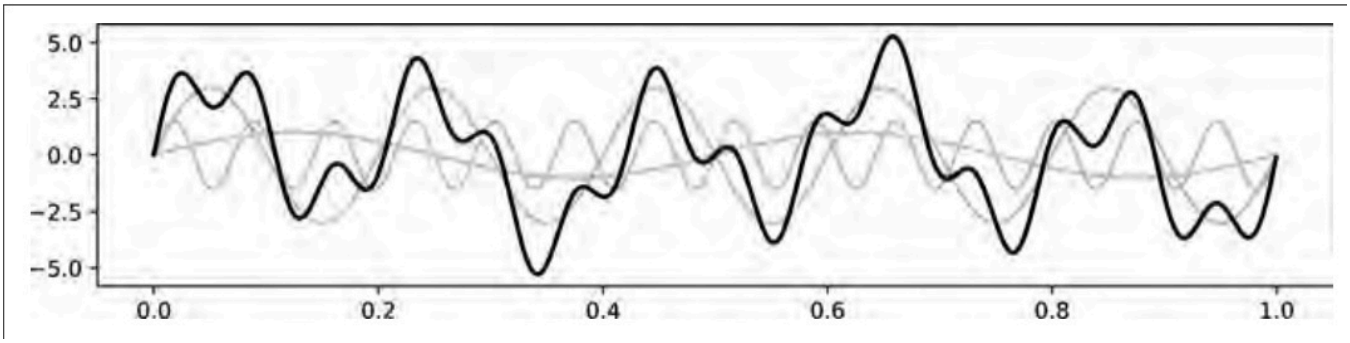
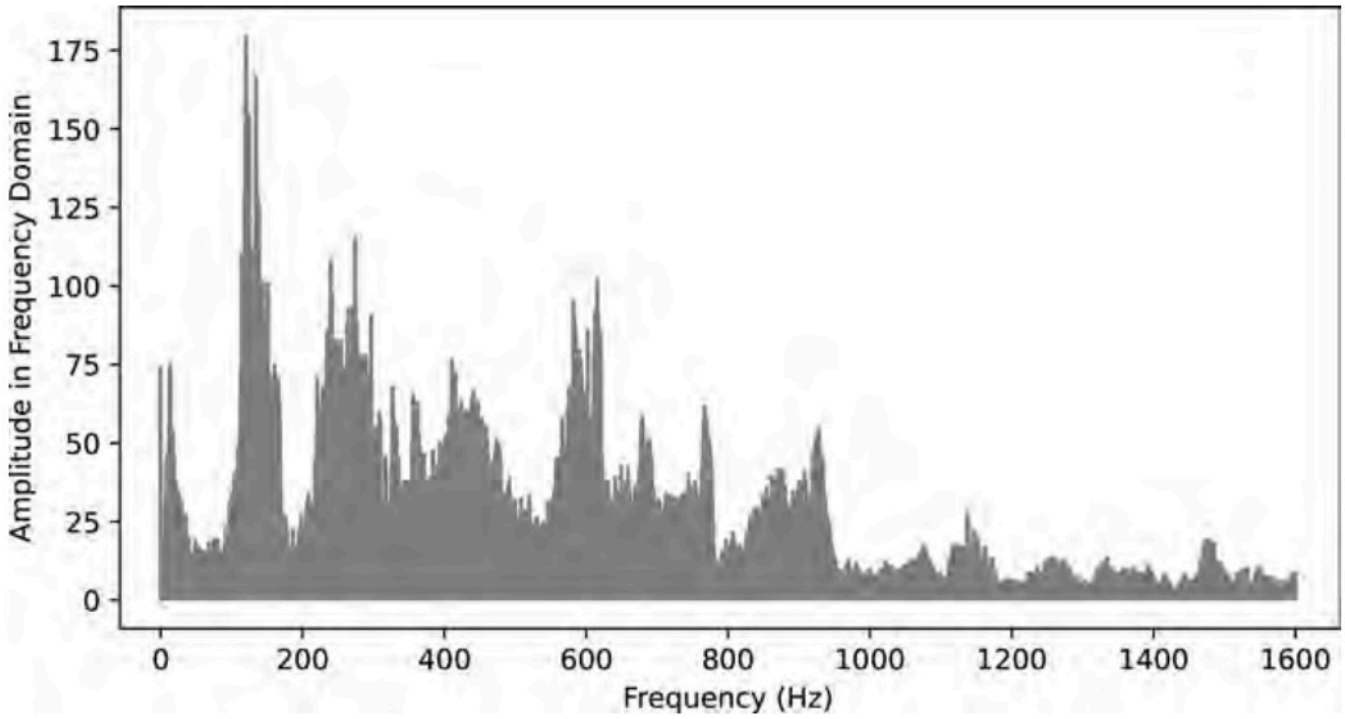



图9-7. 通过分析频谱表示，可以将复杂声波分解为正弦频率

那么本章开头的那个声音呢？与图9-7中的波形一样，该函数可以分解为多个具有其振幅和频率的正弦函数。让我们看看它的频域图：

```
# Compute the Fast Fourier Transform (FFT) of the input signal
X = np.fft.fft(array)

# Length of the FFT result (which is the same as the length of the input signal)
N = len(X)

# Calculate the frequency bins corresponding to the FFT result
n = np.arange(N)
T = N / sampling_rate
freq = n / T

# Plot the amplitude spectrum for the first 8000 frequency bins
# We could plot all the bins, but we would get a mirror image of the spectrum
plt.stem(freq[:8000], np.abs(X[:8000]), "b", markerfmt=" ", basefmt="-b")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude in Frequency Domain")
plt.show()
```




这段音频比之前的例子更具挑战性。我们可以看到，大部分声音集中在 $0-800$ Hz 范围内。我们还可以看到，在 170 Hz 附近有一些较大的噪声。虽然这个图很有趣，但我们在时域中丢失了信息。我们不知道在什么时间出现了特定频率的声音。波形包含幅度和时间信息，而傅里叶变换图包含幅度和频率信息。我们能否将三者同时结合起来呢？

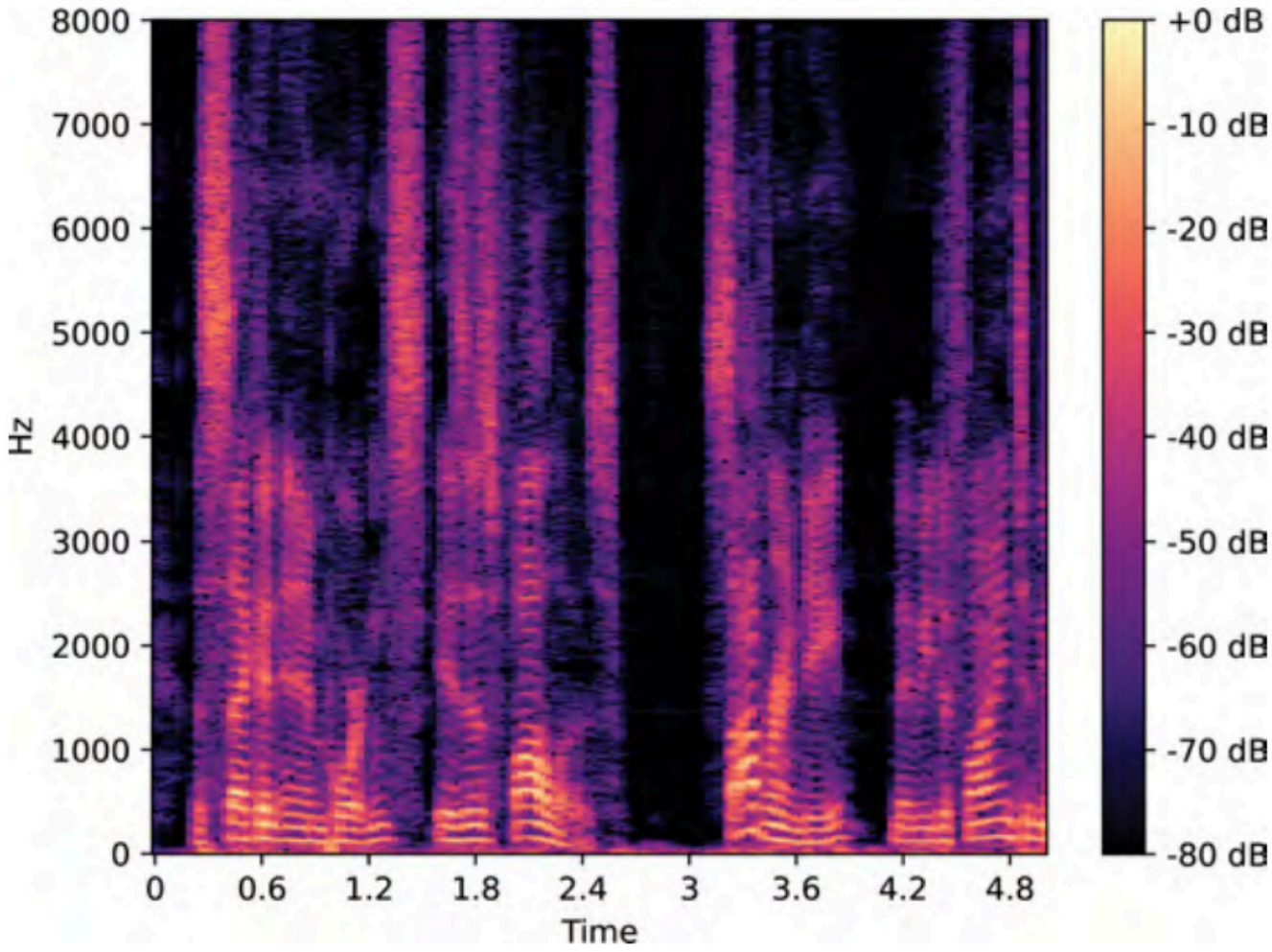
频谱图展示了信号的频率和幅度随时间的变化。它们是信息丰富的工具，能够在一个图中同时可视化时间、频率和幅度。为了创建频谱图，我们会在原始波形上滑动一个窗口，并计算该段的傅里叶变换，以捕捉频率随时间的变化。然后，这些窗口可以堆叠在一起形成频谱图。这种在音频上滑动窗口的方法称为短时傅里叶变换。

使用 librosa，我们可以使用 stft() 来获取短时傅里叶变换。除了计算频谱图外，我们还将幅度转换为分贝尺度，这是对数的，更适合可视化。记住，幅度是声压的差异。因此，声压的数值范围非常广。通过使用对数尺度，我们限制了范围，使图表更具信息性，并且更接近人类感知声音的方式。

> 分贝尺度是对数的，意味着每增加 10 dB，相对声强增加十倍。然而，我们对响度的感知并不直接与这些物理变化一致。虽然分贝尺度和我们对响度的感知都遵循对数模式，但它们的方式不同。具体来说，声强增加 10 dB，人耳感知到的响度大约增加一倍，而不是十倍。这种差异是因为人类听觉系统将声强的变化压缩到一个更易管理的响度感知范围内。

接下来，我们来看一下示例的频谱图：

```
# 计算短时傅里叶变换 (STFT)
# 我们取 STFT 的绝对值以获取每个频率区间的幅度。
D = np.abs(librosa.stft(array))

# 将幅度转换为分贝，这是对数的。
S_db = librosa.amplitude_to_db(D, ref=np.max)

# 生成频谱图显示
librosa.display.specshow(S_db, sr=sampling_rate, x_axis="time", y_axis="hz")
plt.colorbar(format="%+2.0f dB")
```

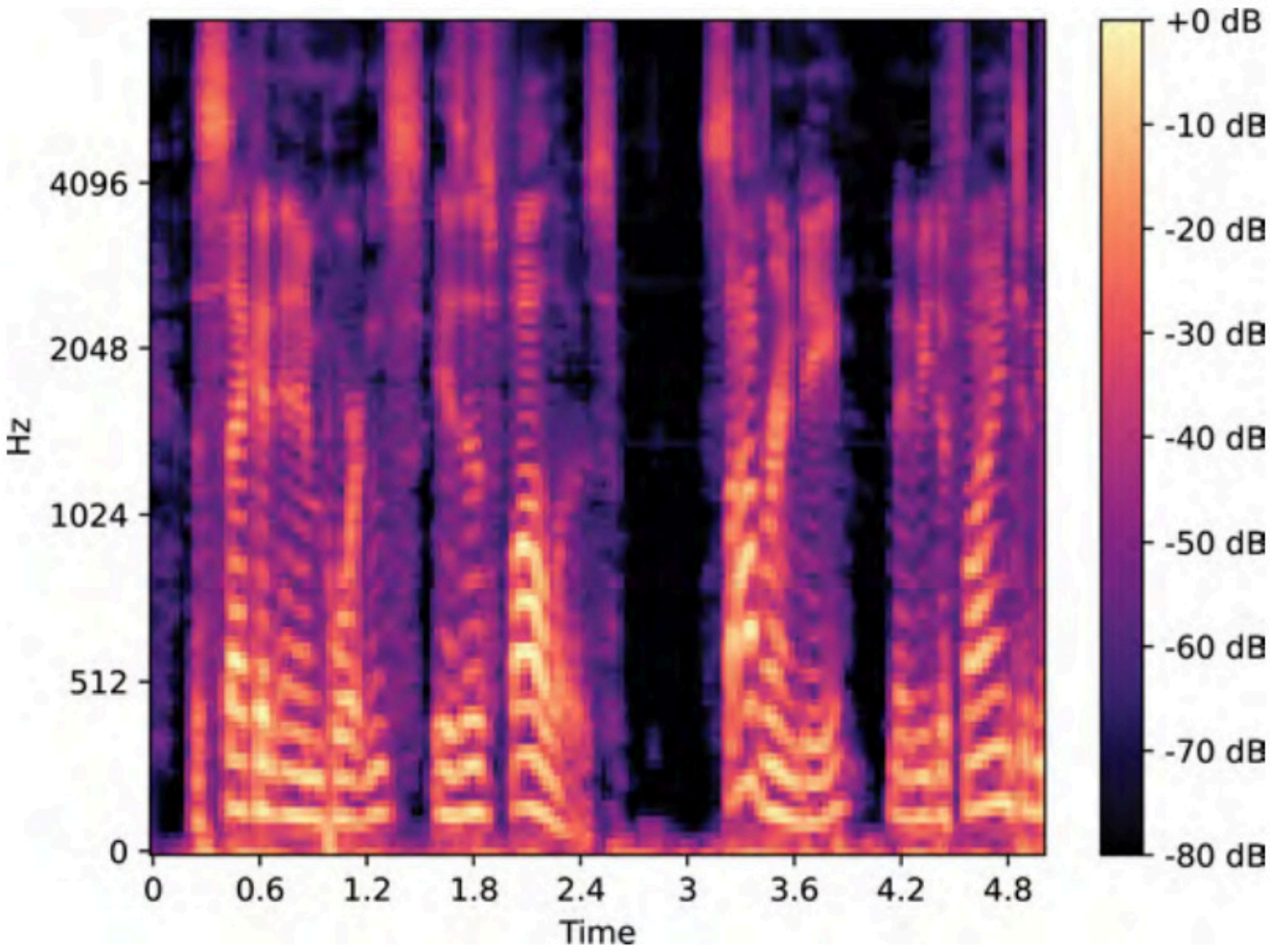



x 轴是时间，与波形图相同。y 轴显示频率（使用赫兹，线性单位），颜色表示特定点的频率强度（分贝）。黑色区域表示没有能量的区域（静音）。与之前一样，我们可以在前 2.4 秒和最后 1.6 秒观察到一些噪声。最响的点出现在低频区域（明亮的颜色和 y 轴上的低值）。这与波形图和频域图一致，我们在低频区域获得了较高的幅度。

> 你可能会好奇为什么会有负的分贝值。由于我们使用了 amplitude_to_db() 并设置了 ref=np.max，频谱图的最大值为 0 dB。其余的值是相对于这个最大值的。例如，–20 dB 的值意味着幅度比最大值低 20 dB。

一种流行的频谱图变体称为梅尔频谱图。在普通频谱图中，频率单位是线性的，而梅尔频谱图使用的尺度类似于我们（人类）感知声音的方式。人类对声音的感知是对数的，意味着我们对低频变化更敏感，而对高频变化不那么敏感。500 和 $1{,}000$ Hz 之间的差异比 5,000 和 5,500 之间的差异更明显。librosa 库再次提供了一个方便的方法来计算梅尔频谱图。在梅尔频谱图中，频率（y 轴）上的等距具有相同的感知距离。让我们绘制梅尔频谱图：

```
# 从音频信号生成梅尔尺度频谱图。
# 结果是一个矩阵，其中每个元素对应于特定时间的频率带（梅尔尺度）的功率。
S = librosa.feature.melspectrogram(y=array, sr=sampling_rate)

# 我们将功率频谱图转换为分贝尺度
S_dB = librosa.power_to_db(S, ref=np.max)

# 显示梅尔尺度频谱图
librosa.display.specshow(S_dB, sr=sampling_rate, x_axis="time", y_axis="mel")
plt.colorbar(format="%+2.0f dB")
```



梅尔频谱图与原始频谱图有相似的模式，但我们可以注意到一些差异。首先，y 尺度不是线性的：512 和 1,024 之间的距离与 2,048 和 4,096 之间的距离相同。其次，低频区域中能量较多（分贝较高）的区域在梅尔频谱图中更加明显。这与人类感知声音的方式一致。

除了作为理解音频信号的优秀视觉表示外，频谱图还经常直接用于机器学习模型。例如，歌曲的频谱图可以用作模型的输入来分类其流派。同样，模型可以接收一些单词作为输入，并输出表示某人说出这些单词的音频的频谱图。

# 基于Transformer架构的语音转文本

现在让我们深入探讨自动语音识别（ASR），即将音频文件转录为文本的任务。与许多其他任务一样，我们可以使用transformers库中的`pipeline()`，它方便地处理了所有的预处理和后处理，是一个有用的推理封装器。让我们使用OpenAI发布的流行开源模型Whisper的最小版本来获取一个初始基线：

```
from transformers import pipeline
pipe = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-small",
    max_new_tokens=200,
)
pipe(array)

{'text': ' The second in importance is as follows. Sovereignty may be defined to be'}
```

使用一个小型版本（2.44亿参数，可以高效地在设备上运行）的结果是可靠的。更令人惊讶的是，如果你听到原始音频，它在“be”这个词的中间被切断了。Whisper可以预测整个单词，即使它没有完成。此外，Whisper还预测了标点符号（例如句号）。接下来的部分将揭示执行ASR的模型是如何工作的。在我们探索Whisper之前，让我们先讨论一下仅使用编码器的模型如何应用于ASR。

## 基于编码器的技术

看待自动语音识别（ASR）任务的一种方式是将其视为文本标记分类。这个想法类似于在自然语言处理（NLP）领域中使用BERT。首先，你使用仅包含编码器的Transformer模型通过掩码语言模型（MLM）进行预训练；也就是说，模型使用大量未标记的数据进行预训练，并且部分输入被掩码。模型的目标是预测应该填充掩码的单词。我们可以在音频中应用相同的原理。不是

掩码文本，而是掩码音频（准确地说，是潜在语音表示），模型学习上下文表示。之后，这个模型可以在不同的音频任务（如说话人识别或ASR）上使用标记数据进行微调。让我们深入探讨如何用这个想法实现ASR。

我们之前提到的第一个挑战是，音频样本可能非常长。一个30秒的音频样本，采样率为$16\ \mathrm{kHz}$，将产生一个包含480,000个值的数组。将480,000个值作为Transformer编码器的单个输入样本，将需要大量的内存和计算资源。为了缓解这个问题，一种方法是使用卷积神经网络（CNN）作为特征编码器。CNN将在输入数据（波形）上滑动，并输出潜在语音表示。例如，假设一个20毫秒的窗口滑动通过1秒的音频，我们将得到50个潜在表示（假设没有重叠窗口；在CNN术语中，我们使用的步幅为20）。这些潜在表示随后被传递给经典的Transformer编码器，编码器为每个表示输出嵌入。在预训练期间，潜在表示的某些部分被掩码，因此模型学习如何预测缺失的部分。

为了进行ASR微调，编码器上添加了一个简单的线性分类头：分类器的目标是估计与编码器处理的每个音频窗口对应的文本。模型将分类的类别数量是一个设计决策。我们可以决定分类整个单词、音节或仅仅是字符，但考虑到我们使用的是20毫秒的窗口，单个单词无法适应一个窗口。在这种情况下，分类字符似乎非常有意义，并且它还有一个额外的好处，即我们可以保持一个非常小的词汇表。例如，对于英语，我们可以使用26个英文字母加上一些特殊标记（例如，表示静音的标记、表示未知的标记等）。为了保持最小的词汇表，我们通常对文本进行预处理，将所有文本大写并将数字转换为单词（例如，“14”转换为“fourteen”）。

> 在本章的第一部分中，我们使用频谱图以简洁的二维视觉表示捕捉输入数据的幅度和频率特征。现在，我们正在探索以不同方式处理数据的架构，使用CNN直接从波形中提取特征，而不将其转换为频谱图。这些方法之间的选择取决于特定任务和架构设计等因素。

正如我们即将讨论的那样，一些模型将频谱图作为输入，而另一些模型则直接处理原始波形。Transformer凭借其注意力机制，特别适合处理序列数据，因此考虑输入的时间结构非常重要。

让我们回顾一下使用基于编码器的模型执行ASR的整个流程：

1. 接收到表示幅度的原始音频数据（一维数组）。

2. 数据被归一化为零均值和单位方差，以标准化不同幅度。
3. 一个小型CNN将音频转换为潜在表示。这减少了输入序列的长度。
4. 这些表示随后被传递给编码器模型，编码器为每个表示输出嵌入。
5. 每个嵌入最终通过分类器，分类器预测每个表示对应的字符。

这种模型的输出可能如下所示：

CHAAAAAPTTERRRSSIXTEEEEENIMMMMIIGHT...

嗯……这看起来像文本，但显然不正确。发生了什么？如果一个字符的声音持续的时间超过单个窗口，它可能会在输出中多次出现。请记住，模型在训练期间不知道每个字符发生的时间，因此无法直接将音频与文本对齐。

最初用于循环神经网络（RNN）的一种解决方法是联结时序分类（Connectionist Temporal Classification, CTC）。在音频中使用CTC的想法是添加填充标记（为了可视化目的，我们将使用字符*），它有助于作为字符组之间的边界，以及一个分隔标记(/)，用于分隔单词。模型将学习预测这些标记。在推理过程中，输出可能如下所示：

CHAAAAA*PTT*ERRR/SS*IX*T*EE*EEN/I/MMM*II*GHT

通过这个输出，我们可以通过合并同一组中连续相等的字符来进行去重，从而得到我们想要的结果：

CHAPTER SIXTEEN I MIGHT

所有这些想法（使用仅编码器模型、使用CNN处理波形以及使用CTC进行分类）构成了基于编码器的架构（如Meta的Wav2Vec2（2020）和HuBERT（2021））的基础。Wav2Vec2使用Librispeech和LibriVox（均为未标记数据集）进行预训练。它只需10分钟的标记数据进行微调，就可以超越使用更多数据训练的模型。这非常有趣，因为基础模型可以轻松地针对特定领域或口音进行微调，而无需大量数据。这里解决的下游任务是ASR，但相同的预训练模型可以微调用于其他任务，如说话人识别和语言检测。以下代码展示了使用Wav2Vec2进行推理的每个步骤（请注意，你也可以使用pipeline()作为高级API）：

```
import torch
from transformers import Wav2Vec2ForCTC, Wav2Vec2Processor

from genaibook.core import get_device

device = get_device()

# The AutoProcessor has the pre and post-processing incorporated
wav2vec2_processor = Wav2Vec2Processor.from_pretrained(
    "facebook/wav2vec2-base-960h"
)
wav2vec2_model = Wav2Vec2ForCTC.from_pretrained(
    "facebook/wav2vec2-base-960h"
).to(device)

# Run forward pass, making sure to resample to 16kHz
inputs = wav2vec2_processor(
    array, sampling_rate=sampling_rate, return_tensors="pt"
)
with torch.inference_mode():
    outputs = wav2vec2_model(**inputs.to(device))

# Transcribe
predicted_ids = torch.argmax(outputs.logits, dim=-1)
transcription = wav2vec2_processor.batch_decode(predicted_ids)
print(transcription)

['THE SECOND IN IMPORTANCE IS AS FOLLOWS SOVEREIGNTY MAY BE DEFINED TO']
```

> 由于模型是使用特定采样率的数据进行预训练的，因此在推理过程中需要使用相同采样率的音频。你可以通过重新采样数据（例如，使用dataset.cast_column("audio", Audio

> (sampling_rate=16_000)))）或在处理器中指定采样率来实现这一点，如前面的代码片段所示。

HuBERT遵循相同的预训练概念以学习有用的潜在表示，但改变了训练过程，使用BERT的原始MLM目标。虽然Wav2Vec2预测字符，但HuBERT使用聚类技术处理波形以学习离散的隐藏语音单元，这些单元可以视为NLP中的标记。模型在第二阶段预测这些隐藏单元在随机掩码位置的值。

请注意，Wav2Vec2和HuBERT仅适用于英语。在Wav2Vec2发布几周后，Meta发布了XLSR-53，它具有与Wav2Vec2相同的架构，但使用53种语言的56,000小时语音进行预训练。XLSR学习了几种语言共有的语音单元。因此，数字数据较少的语言仍然可以获得不错的结果。2021年，Meta发布了XLS-R（是的，它与XLSR不同），这是一个20亿参数的模型，使用类似的设置进行训练，但使用了来自128种语言的近10倍的未标记数据（436,000小时）。

需要注意的是，这些模型是声学模型：它们的输出完全基于输入的声音，缺乏内在的语言信息。例如，模型可能会频繁拼错单词，输出非单词的内容，或混淆同音词（例如，“bear”与“bare”）。缓解这一问题的一种方法是在生成阶段加入语言信息。

通常在分类阶段，我们计算 `argmax(logits)` 来预测最可能的字符。然而，另一种方法是引入一个语言模型，该模型可以根据字符序列预测最可能的单词。n-gram 模型是一种语言模型，它基于给定文本语料库中的前 $n-1$ 个单词来预测单词的可能性。例如，二元模型 $(n=2)$ 考虑单词对，而三元模型 $(n=3)$ 考虑三元组。这种概率模型有助于理解语言的上下文和结构，并且可以非常小巧高效。像第 2 章中提到的基于 Transformer 的完整语言模型也可以使用。然而，n-gram 模型在计算、内存和推理时间上只需一小部分成本，就能实现这些模型的大部分质量提升。

可以将 n-gram 分数添加到束搜索中，以生成 $k$ 个最可能的文本序列。通过将语言模型集成到束搜索中，我们可以增强解码过程。束搜索会评估声学模型和语言模型的分数，这有助于纠正拼写错误并过滤掉无意义的单词。这种组合方法显著提高了生成文本的准确性和连贯性。

> 你可能希望增加某些单词的概率（例如，如果某些单词不在语言模型中，或者你需要提升

> 特定领域的数据）。为此，你可以计算输出中热词的数量并增加其概率。

## 编码器-解码器技术

使用带有CTC头的编码器模型是ASR（自动语音识别）中最流行的方法之一。如前所述，声学模型可能会生成拼写错误，因此需要引入n-gram模型。这些挑战促使人们探索编码器-解码器架构。

我们可以将ASR问题表述为序列到序列问题，而不是分类问题。这就是我们在本节开头介绍的开源模型Whisper所做的。与Wav2Vec2或HuBERT不同，Whisper（图9-8）是在监督设置下使用大量标注数据进行训练的：超过680,000小时的音频及其对应的文本。相比之下，Wav2Vec2是在不到60,000小时的无标注数据上训练的。其中约三分之一的数据是多语言的，这使得Whisper能够为96种语言执行ASR。由于该模型是用标注数据训练的，Whisper在预训练期间直接学习从语音到文本的映射，而无需微调。Whisper的另一个特点是它被训练为在没有注意力掩码的情况下操作：它可以直接推断出忽略哪些输入。

我们如何进行推理？与Wav2Vec2不同，Whisper使用频谱图进行操作。我们首先对一批音频样本进行填充和/或截断，以确保输入长度一致，将其转换为对数梅尔频谱图，并在将其传递给编码器之前使用CNN处理输出。然后，编码器的输出被传递给解码器，解码器以自回归的方式（就像Llama等模型一样）逐个预测下一个标记，直到生成结束标记。虽然编码器-解码器架构可能比仅使用编码器的方法慢，但Whisper可以处理长音频样本，可以预测标点符号，并且在推理过程中不需要额外的语言模型。

在仅使用编码器的模型中，引入语言模型是必要的，以解决声学模型生成的拼写错误，通常需要使用外部n-gram模型。在Whisper的情况下，解码器具有双重作用：生成文本转录，同时充当语言模型。为什么解码器可以充当语言模型？通过学习根据编码器的上下文信息预测转录序列中的下一个标记，Whisper在推理过程中消除了对外部语言模型的需求。

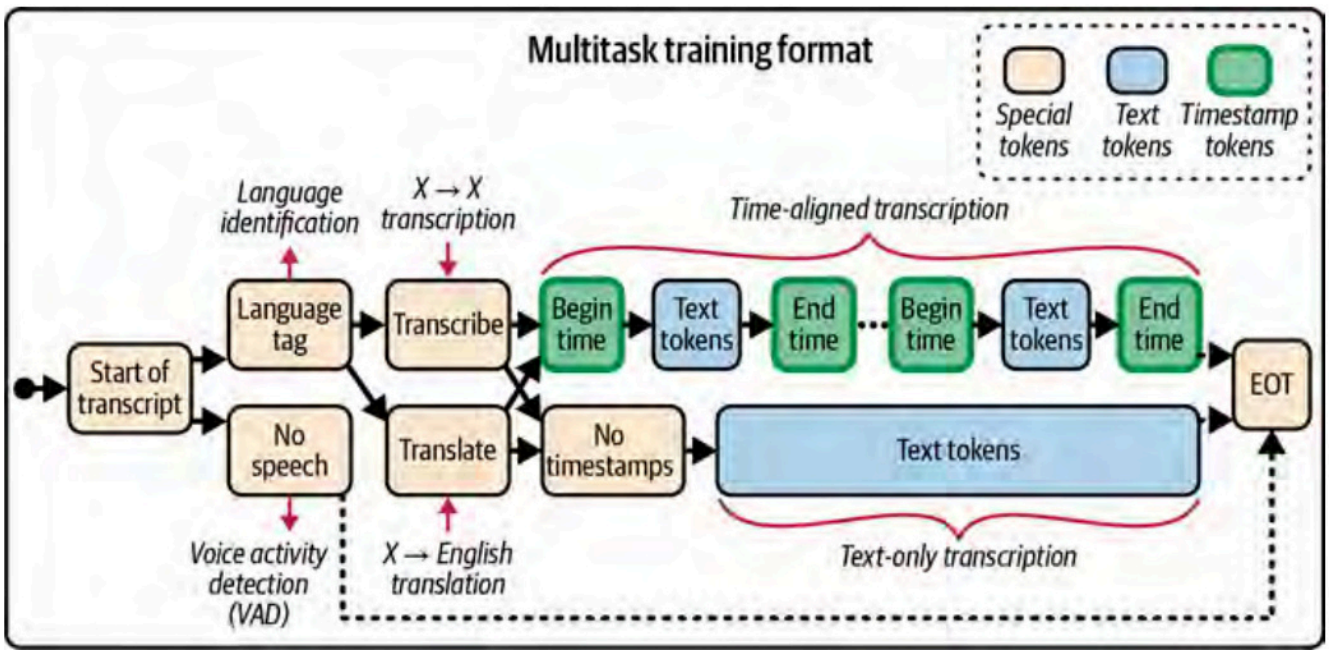



图9-8. Whisper的训练以序列到序列的方式建模，涵盖多种任务，包括翻译、转录、多语言语音识别等。使用特殊标记来标识任务、语言和数据中的关键点，从而引导模型执行所需操作。（改编自Whisper论文中的图像。）

Whisper使用特定的序列格式，因此查看图9-8对于理解其生成过程至关重要。特殊标记用于指示语言或任务，从而引导模型生成所需的输出。这类似于我们在前几章中介绍的调节方法。一些最重要的标记如下：

- 语音以转录开始标记开头。
- 如果语言不是英语，则有一个语言标签标记（例如，hi表示印地语）。
- 使用语言标签，我们可以执行语言识别、转录或翻译成英语。
- 如果有无语音标记，Whisper用于语音活动检测。
  让我们看一个西班牙语的示例及其对应的格式：

```
from transformers import WhisperTokenizer

tokenizer = WhisperTokenizer.from_pretrained(
    "openai/whisper-small", language="Spanish", task="transcribe"
)

input_str = "Hola, ¿cómo estás?"
labels = tokenizer(input_str).input_ids
decoded_with_special = tokenizer.decode(
    labels, skip_special_tokens=False
)
decoded_str = tokenizer.decode(labels, skip_special_tokens=True)  # <4>

print(f"Input:                          {input_str}")
print(f"Formatted input w/ special:    {decoded_with_special}")
print(f"Formatted input w/out special: {decoded_str}")

'Input: Hola, ¿cómo estás?'
'Formatted input w/ special: '
'<|startoftranscript|><|es|><|transcribe|><|notimestamps|>Hola, '
'¿cómo estás?<|endoftext|>'
'Formatted input w/out special: Hola, ¿cómo estás?'
```

1. 加载预训练的标记器。由于Whisper需要添加一些标记，例如语言ID标记和任务标识符，我们需要指定语言和任务参数。
2. 对输入字符串进行标记化。
3. 将标记ID解码回原始字符串，但包括特殊标记。
4. 将标记ID解码回原始字符串，但不包括特殊标记。

使用Whisper创建转录与使用Wav2Vec2并没有太大不同：

1. 我们使用处理器将音频准备为模型期望的格式。在这种情况下，从原始语音中提取梅尔频谱图，然后进行处理，以便模型可以消费。
2. 模型生成与转录对应的标记ID。
3. 处理器解码ID并将其转换为人类可读的字符串。

OpenAI发布了九个Whisper变体，参数范围从3900万到15亿，模型检查点支持多语言和仅英语的设置。在此示例中，我们将使用中等规模的小型多语言模型，它可以在2GB的GPU内存上运

行，并且比最大模型快六倍。

> 新模型，例如大型模型的第二和第三版本，仍在不断发布。此外，Distil Whisper项目已经开发出高质量、更小的原始模型变体，速度提高了六倍，体积缩小了49%。

```
from transformers import WhisperForConditionalGeneration, WhisperProcessor
whisper_processor = WhisperProcessor.from_pretrained("openai/whisper-small")
whisper_model = WhisperForConditionalGeneration.from_pretrained(
    "openai/whisper-small"
).to(device)
inputs = whisper_processor(
    array, sampling_rate=sampling_rate, return_tensors="pt"
)
with torch.inference_mode():
    generated_ids = whisper_model.generate(**inputs.to(device))
transcription = whisper_processor.batch_decode(
    generated_ids, skip_special_tokens=False
)[0]
print(transcription)
('<|startoftranscript|><|en|><|transcribe|><|notimestamps|> The '
'second in importance is as follows. Sovereignty may be defined to '
'be<|endoftext|>')
```

这是一个深入了解WhisperProcessor的好机会。为了更好地理解前面的代码，我们建议：

- 查看其文档。
- 确定处理器的两个组件。
- 识别并检查处理器的输出（前面代码中的输入）。

## 从模型到流水线

在前面的章节中，你学习了执行自动语音识别（ASR）的不同架构和方法。然而，在生产环境

中使用它们时，仍然有三个挑战需要解决：

### 长音频转录

第一个限制是，Transformer 模型通常只能处理有限长度的输入。例如，Wav2vec2 使用了注意力机制，其复杂度是二次的。Whisper 没有注意力机制，但它设计用于处理 30 秒的音频，并且会截断更长的音频。解决这个问题的一个简单方法是分块：将音频分割成较短的样本，对它们进行推理，然后重建输出。虽然这种方法效率高，但在分块边界处可能会导致质量下降。为了解决这个问题，我们可以使用步幅分块，这意味着我们将有重叠的块，然后将它们连接起来。结果不会与模型在完整音频上预测的结果完全相同，但应该接近。我们可以将分块批量处理，并通过模型并行运行它们，因此比顺序转录整个音频文件更高效。分块和分块批处理可以快速使用 ASR 流水线的 `chunk_length_s` 和 `batch_size` 参数来完成。

### 实时推理

对于许多应用来说，执行实时 ASR 是理想的。现在你已经学会了分块，我们可以使用小分块（例如 5 秒）和 1 秒的步幅来使用模型。使用 CTC 模型进行实时推理会更快，因为它是单次通过的架构，而包含解码器的模型则不然。虽然 Whisper 会慢一些，但你可以执行相同的分块逻辑来转录传入的块并获得良好的结果。

### 时间戳

拥有指示短音频段落开始和结束时间的时间戳对于将转录与输入音频对齐非常有用。例如，如果你从视频通话中生成字幕，你会想知道每个转录属于哪个时间段。这可以通过 `return_timestamps` 轻松启用。在底层，我们知道每个输出标记的上下文窗口和 CTC 模型的采样率。

让我们结合所有这些来处理一个较长的（1 分钟）音频：

```
device = get_device()

pipe = pipeline(
    "automatic-speech-recognition", model="openai/whisper-small", device=device
)
pipe(
    long_audio,
    generate_kwargs={"task": "transcribe"},
    chunk_length_s=5,
    batch_size=8,
    return_timestamps=True,
)

{'chunks': [{'text': ' the second in importance is as follows.', 'timestamp': (
    0.0, 3.0)}, {'text': ' Sovereignty may be defined to be the right of '
    'making laws.', 'timestamp': (3.0, 6.33)}, {'text': ' In France, the king
    really exercises a ' 'portion of the sovereign power, since the laws '
    'have no weight till he has given his assent to 'them.', 'timestamp': (6.33,
     16.89)}, {'text': ' He is moreover the executor of the laws, but ' 'he
    does not really cooperate in their ' 'formation since the refusal of his
    asset does ' 'not annul them. He is therefore merely to be ' 'considered as
    the agent of the sovereign ' 'power.', 'timestamp': (16.89, 36.61)},
    {'text': ' But not only does the king of France exercise ' 'a portion of
    the sovereign power, He also ' 'contributes to the nomination of the '
    'legislature, which exercises the other ' 'portion. He has the privilege of
    appointing ' 'the members of one chamber and of dissolving ' 'the United
    States has no share in the ' 'formation of the legislative body',
    'timestamp': (36.61, 59.75)}, {'text': ' and cannot dissolve any part of it
    . The king ' 'has the same right of bringing forward ' 'measures as the
    chambers.', 'timestamp': (59.75, 67.09)}, {'text': ' A right which the
    president does not possess.', 'timestamp': (67.09, 70.43)}],
'text': ' the second in importance is as follows. Sovereignty may ' 'be defined
    to be the right of making laws. In France, the 'king really exercises a
    portion of the sovereign power, 'since the laws have no weight till he has
    given his assent 'to them. He is moreover the executor of the laws, but he
    ' 'does not really cooperate in their formation since the ' 'refusal of his
    asset does not annul them. He is therefore ' 'merely to be considered as
    the agent of the sovereign ' 'power. But not only does the king of France
    exercise a ' 'portion of the sovereign power, He also contributes to the '
```

```
    'nomination of the legislature, which exercises the other ' 'portion. He
    has the privilege of appointing the members of ' 'one chamber and of
    dissolving the United States has no ' 'share in the formation of the
    legislative body and cannot ' 'dissolve any part of it. The king has the
    same right of ' 'bringing forward measures as the chambers. A right which '
    'the president does not possess.'}
```

你可能会注意到，转录大部分是准确的，但音频中存在的一两句话在转录中缺失了。这是较小的 Whisper 模型的常见问题。由于这些模型是生成式的（即它们生成文本而不是直接将声音分类为标记），它们偶尔会遗漏单词甚至生成虚假内容。让我们讨论一些评估这些模型的策略。

## 评估

有如此多的模型可以执行音频任务，决定选择哪一个可能会很复杂。对于预训练模型，我们通常会查看多个任务的下游性能。虽然最常见的评估下游任务是自动语音识别（ASR），但我们也可以评估在其他任务（如关键词检测、意图分类或说话人识别）中微调的预训练模型。除了下游任务的性能外，我们还需要考虑其他因素，如模型的大小、推理速度、训练所用的语言以及训练数据与推理数据的接近程度。例如，如果你在处理特定口音，你可能希望使用该口音的数据微调模型。如果你需要实时推理，你可能会选择较小的变体。

在本节中，我们将对不同的英语语音识别模型进行高级评估。虽然这不会为你提供一个端到端的评估框架来选择最适合你用例的模型，但它确实提供了一些分析模型性能的实用方法。让我们评估两个小型多语言模型：拥有7400万参数的多语言Whisper版本和拥有9400万参数的Wav2Vec2版本。我们可以比较推理速度和GPU使用的峰值来开始：

```
from genaibook.core import measure_latency_and_memory_use

wav2vec2_pipe = pipeline(
    "automatic-speech-recognition",
    model="facebook/wav2vec2-base-960h",
    device=device,
)
whisper_pipe = pipeline(
    "automatic-speech-recognition", model="openai/whisper-base", device=device
)

with torch.inference_mode():
    measure_latency_and_memory_use(
        wav2vec2_pipe, array, "Wav2Vec2", device, nb_loops=100
    )
    measure_latency_and_memory_use(
        whisper_pipe, array, "Whisper", device=device, nb_loops=100
    )

'Wav2Vec2 execution time: 0.009195491333007812 seconds'
'Wav2Vec2 max memory footprint: 1.7330821120000002 GB'
'Whisper execution time: 0.092218232421875 seconds'
'Whisper max memory footprint: 1.6933248 GB'
```

不出所料，两个模型的最大内存占用（使用的显存）非常相似，这很合理，因为它们的参数数量相近。1.7 GB的内存占用相对较小，可以在笔记本电脑甚至一些强大的手机上运行。Wav2Vec2明显更快，这是可以预期的，因为Whisper的解码器一次生成一个标记的文本。

现在让我们看看这两个模型在高质量预测方面的表现如何。评估ASR模型最常见的指标是词错误率（WER），它通过查看预测与原始标签之间的差异来计算错误数量。错误是根据从预测到标签所需的替换、插入和删除次数来确定的。例如，给定源文本“how can the llama jump”和预测“can the lama jump up”，我们有以下情况：

- 一次删除，因为“how”缺失
- 一次替换，因为“llama”被替换为“lama”
- 一次插入，因为“up”仅在预测中出现

WER是错误总数除以标签中的单词数，因此我们的WER为0.6（三个错误除以五个预测）。请注意，尽管“llama”和“lama”之间只有一个字符的差异，但这被计为一个完整的错误。许多替代指

标，如字符错误率（CER），基于每个字符评估差异。尽管如此，WER作为ASR评估的首选指标在业界被广泛采用。`evaluate`库提供了一个高级API接口来使用这些指标。使用示例，让我们学习如何加载和计算WER指标：

```
from evaluate import load
wer_metric = load("wer")
label = "how can the llama jump"
pred = "can the lama jump up"
wer = wer_metric.compute(references=[label], predictions=[pred])

print(wer)

0.6
```

在进行评估之前，第二个需要考虑的方面是，不同的ASR模型根据其训练数据具有不同的输出格式。例如，Whisper在训练时使用了大小写和标点符号，因此其转录包含这些内容。为了公平地评估模型，我们可以在计算WER之前对标签和预测进行归一化处理。这并不完美，因为一个学习大小写和标点符号的模型不会比不学习的模型获得更低的错误率，但这是一个坚实的起点。

`transformers`库提供了归一化器（BasicNormalizer、EnglishTextNormalizer等）。BasicTextNormalizer会删除连续的空白字符和基本标点符号，并将文本转换为小写。EnglishNormalizer更高级，标准化数字（“million”到“10000000”，管理缩写等。让我们使用BasicNormalizer，因为它会很好地工作；重要的是我们要一致地归一化所有文本和转录：

```
from transformers.models.whisper.english_normalizer import BasicTextNormalizer

normalizer = BasicTextNormalizer()
print(normalizer("I'm having a great day!"))

i m having a great day
```

我们将使用Common Voice（一个流行的众包多语言数据集）来比较这些模型。我们将使用该数据集的英语和法语测试部分进行演示，但我们可以使用其他语言（尽管我们需要小心分词）。我们将在每种语言的200个样本上评估WER和CER。

> Common Voice数据集是公开的，但你需要接受条款和条件，并与作者Mozilla基金会分享你的姓名和电子邮件。你可以通过访问数据集页面并点击按钮表示接受来轻松完成此步骤。如果你不同意这些条款，请随意使用其他数据集或你自己的数据进行评估。

让我们从实现评估管道开始：

```python
# This code example is optimized for explainability
# The inference could be done in batches for speedup, for example.
from datasets import Audio

def normalize(batch):
    batch["norm_text"] = normalizer(batch["sentence"])
    return batch

def prepare_dataset(language="en", sample_count=200):
    dataset = load_dataset(
        "mozilla-foundation/common_voice_13_0",
        language,
        split="test",
        streaming=True,
    )
    dataset = dataset.cast_column("audio", Audio(sampling_rate=16000))
    dataset = dataset.take(sample_count)
    buffered_dataset = [sample for sample in dataset.map(normalize)]
    return buffered_dataset

def evaluate_model(pipe, dataset, lang="en", use_whisper=False):
    predictions, references = [], []

    for sample in dataset:
        if use_whisper:
            extra_kwargs = {
                "task": "transcribe",
                "language": f"<|{lang}|>",
                "max_new_tokens": 100,
            }
            transcription = pipe(
                sample["audio"]["array"],
                return_timestamps=True,
                generate_kwargs=extra_kwargs,
            )
        else:
            transcription = pipe(sample["audio"]["array"])
        predictions.append(normalizer(transcription["text"]))
        references.append(sample["norm_text"])
```

```
    return predictions, references
```

1. 实现使用Whisper英语归一化归一化批次的函数。
2. 以流模式加载Common Voice数据集。
3. 将音频数据集重新采样为$16\mathrm{kHz}$。
4. 从数据集中抽取200个样本。
5. 归一化数据集。我们还将数据集缓冲，以便它存储在列表中而不是流数据集中。
6. 对于Whisper，我们为其生成添加了额外的参数（例如，指定语言和任务）。

现在我们有了评估管道，让我们用两个模型和两种语言来尝试它。我们首先指定套件：

```
eval_suite = [
    ["Wav2Vec2", wav2vec2_pipe, "en"],
    ["Wav2Vec2", wav2vec2_pipe, "fr"],
    ["Whisper", whisper_pipe, "en"],
    ["Whisper", whisper_pipe, "fr"],
]
```

现在我们有了所有组件，让我们运行评估：

```
cer_metric = load("cer")

# Pre-process the English and French datasets
processed_datasets = {
    "en": prepare_dataset("en"),
    "fr": prepare_dataset("fr"),
}

for config in eval_suite:
    model_name, pipeline, lang = config[0], config[1], config[2]

    dataset = processed_datasets[lang]

    predictions, references = evaluate_model(
        pipeline, dataset, lang, model_name == "Whisper"
    )

    # Compute evaluation metrics
    wer = wer_metric.compute(references=references, predictions=predictions)
    cer = cer_metric.compute(references=references, predictions=predictions)

    print(f"{model_name} metrics for lang: {lang}. WER: {wer}, CER: {cer}")
```

```
Reading metadata...: 16372it [00:00, 26197.69it/s]
Reading metadata...: 16114it [00:00, 36235.41it/s]
Wav2Vec2 metrics for lang: en. WER: 0.44012772751463547, CER: 0.22138
Wav2Vec2 metrics for lang: fr. WER: 1.0099113197704748, CER: 0.57450
Whisper metrics for lang: en. WER: 0.2687599787120809, CER: 0.14674
Whisper metrics for lang: fr. WER: 0.5477308294209703, CER: 0.27584
```

让我们来讨论一下结果：

- Whisper 在英文和法文上都明显优于 Wav2Vec2（错误率越低越好）。
- 尽管 Wav2Vec2 在英文上的 WER 较高，但你可以注意到 CER 要低得多。正如之前讨论的，Wav2Vec2 是一个声学模型，它可能会产生拼写错误。而 Whisper 则内置了语言模型，因此更有可能生成正确的单词。
- Whisper 在法文上优于 Wav2Vec2 并不令人惊讶；毕竟，Whisper 是用多语言数据训练的，而 Wav2Vec2 是用英文数据训练的。

我们可以尝试使用更大的 Whisper 变体，进一步降低法文的 WER（最大的变体比基础版本大 20 倍）。另一个有趣的点是，Whisper 在英文上的表现优于 Wav2Vec2。Whisper 是多语言的，因此我们可能会预期它在英文上的表现不如完全用英文数据调整的模型。除此之外，OpenAI 还发布了一个相同大小但完全用英文数据训练的 Whisper 模型。如果我们想将 Whisper 投入生产，并且知道所有用户都会说英文，那么最好切换到英文变体。

记住，Whisper 是一个自回归模型。因此，它可能会“幻觉”，有时会不断生成标记。这是与仅编码器方法相比的主要问题之一。由于 WER 没有上限，单个幻觉可能会显著提高 WER。一种方法是限制生成标记的最大数量。由于 Whisper 设计用于转录 30 秒的片段，我们可以假设 60 个单词或大约 100 个标记。另一种方法是通过强制模型返回时间戳来抑制幻觉。在 100 个样本的初步实验中，仅通过应用这种方法，WER 从 1.72 降到了 0.84。如果幻觉是一个问题，为什么 Whisper 在之前的评估中优于 Wav2Vec2？答案是 Whisper 的语言模型、标注数据以及用于预训练的大量数据的结合。

我们建议花一些时间尝试以下内容：

- 不同的模型大小（例如，3900 万参数的 Whisper Tiny 与 15 亿参数的 Large V2，以及蒸馏变体）
- 不同的模型（例如，在法文上微调的 Wav2Vec2 模型或仅用于英文训练的 Whisper 变体）
- 不同的生成参数

> 如果 ASR 评估主题让你感兴趣，我们建议阅读《端到端语音基准测试》论文。该基准测试提出了使用多个数据集对多个端到端系统进行统一评估。还有一个公开的开源语音识别模型排行榜，定期更新最新模型。

这是一次有趣的 ASR 技术探索。现在让我们探索如何做相反的事情：将文本转换为语音，然后推广到音频生成。

# 从文本到语音再到生成音频

到目前为止，你已经学会了如何使用基于Transformer的模型进行高质量的转录。在本章音频部

分的第三部分中，我们将深入探讨音频生成技术、其评估方法以及面临的挑战。你将了解两种流行的TTS模型：SpeechT5和Bark。然后，我们将简要讨论那些不仅限于语音，还能推广到其他形式音频（例如音乐）的模型，如MusicGen、AudioLDM和AudioGen。最后，我们将探索如何使用扩散模型通过Riffusion和Dance Diffusion生成音频。

从头开始训练和评估文本到音频（TTA）模型可能会非常昂贵且具有挑战性。与自动语音识别（ASR）不同，TTA模型可能有多个正确的预测。以TTS模型为例：生成的内容可以有不同的发音、口音和说话风格，而这些都可能是正确的。此外，像Common Voice这样的流行ASR数据集往往包含噪声，因为我们希望为不同条件构建鲁棒的系统。不幸的是，数据集中的噪声对于TTA来说是一个不受欢迎的特性，因为模型也会学会生成它。想象一下，你生成的语音中带有狗叫或汽车喇叭的背景音。因此，TTA的训练数据集需要具有高质量。

## 使用序列到序列模型生成音频

SpeechT5 是微软预训练的一个开源模型，能够执行语音到文本任务（如自动语音识别（ASR）和说话人转换）、语音到语音任务（如增强和语音转换）以及文本到语音（TTS）任务。SpeechT5 的架构如图 9-9 所示，它使用编码器-解码器结构，输入和输出可以是语音或文本。为了处理来自不同模态的输入，语音和文本预网络将输入转换为编码器接收的隐藏表示。更具体地说，编码器预网络将输入波形和文本转换为编码器使用的共同隐藏表示。同样，解码器的输入和输出也通过语音和文本解码器预网络和后网络进行预处理和后处理。这产生了六个额外的网络，它们提供了灵活性，使同一个模型能够完成多种任务：

**文本编码器预网络**

一个文本嵌入层，将输入映射到编码器期望的隐藏表示。

**语音编码器预网络**

与 Wav2Vec2 中的特征提取器相同：一个预处理输入波形的卷积神经网络（CNN）。

**文本解码器预网络**

在预训练期间，它与文本编码器预网络相同。在微调时，会对其进行修改。

**语音解码器预网络**
接收对数梅尔频谱图并将其压缩为隐藏表示。

**文本解码器后网络**
单层线性网络，将输出投影到词汇表上的概率分布。

**语音解码器后网络**

预测输出频谱图并使用额外的卷积层对其进行细化。

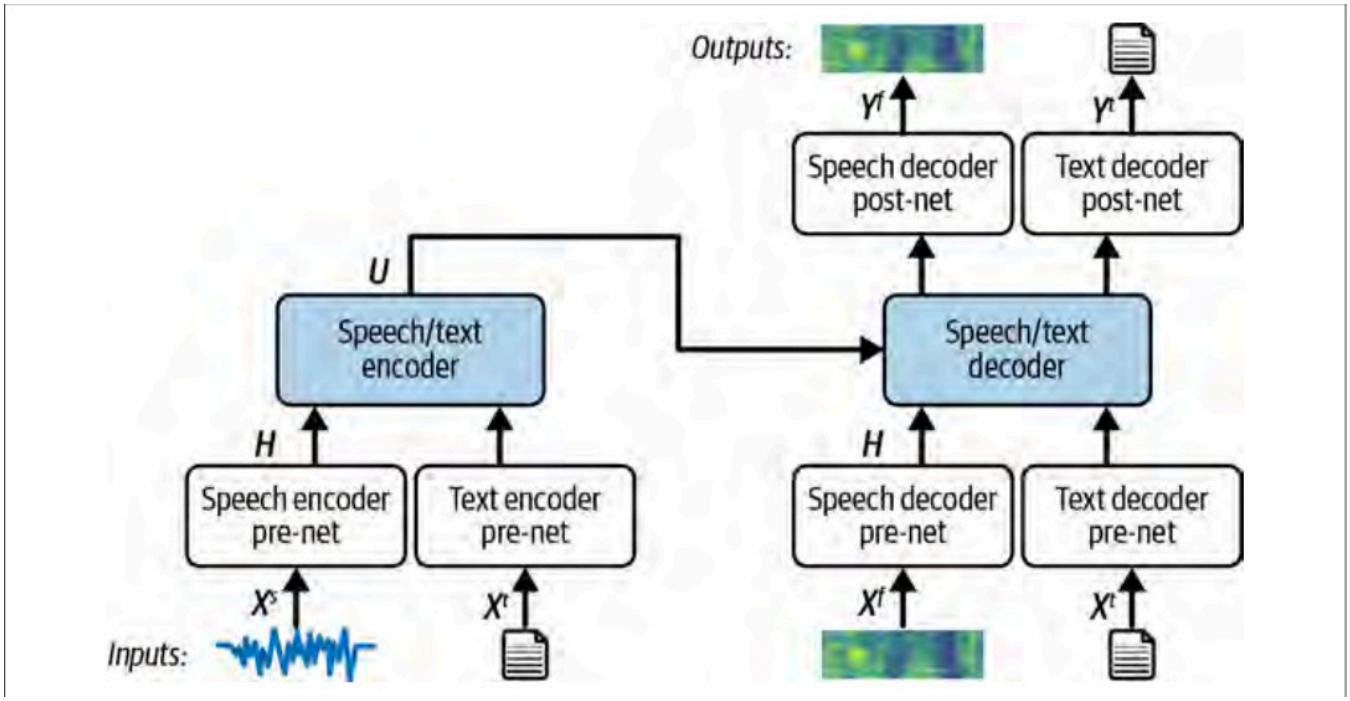



图 9-9. SpeechT5 模型的架构（改编自原论文中的图像）

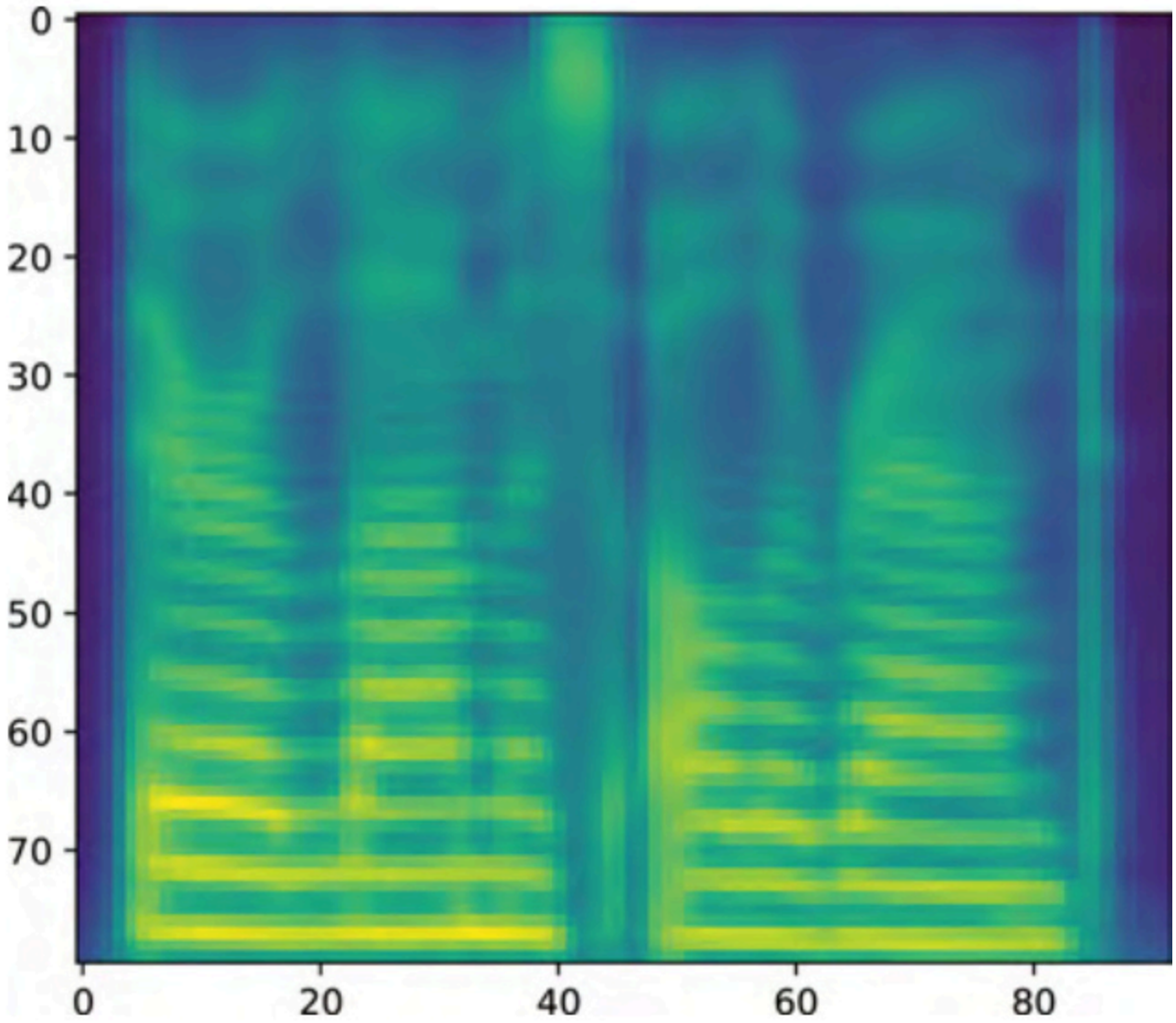
例如，如果我们想执行 ASR，我们的输入将是音频，输出将是文本。在这种情况下，我们将使用语音编码器预网络和文本解码器网络（包括预网络和后网络）。transformers 库提供了一个处理器类（SpeechT5Processor），它提供了处理音频和文本输入和输出的功能。其结构与第 308 页的“基于编码器的技术”非常相似：

```
from transformers import SpeechT5ForSpeechToText, SpeechT5Processor

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_asr")
model = SpeechT5ForSpeechToText.from_pretrained("microsoft/speecht5_asr")

inputs = processor(
    audio=array, sampling_rate=sampling_rate, return_tensors="pt"
)
with torch.inference_mode():
    predicted_ids = model.generate(**inputs, max_new_tokens=70)

transcription = processor.batch_decode(predicted_ids, skip_special_tokens=True)
print(transcription)

['chapter sixteen i might have told you of the beginning i might '
'have told you of the beginning of the beginning of the beginning '
'of the beginning of the beginning chapter sixteen']
```

现在，我们想执行 TTS。我们期望输入是文本，输出是语音，因此我们使用文本编码器预网络和语音解码器网络。为了支持多说话人 TTS，SpeechT5 期望接收说话人嵌入。这些嵌入表示有关说话人的信息，例如他们的声音和口音，从而使 SpeechT5 能够生成具有这种风格的语音。嵌入是通过 x-vectors 提取的，这是一种将任意长度的输入音频映射到固定维度嵌入的技术。SpeechT5 通过将 x-vector 与语音解码器预网络的输出连接起来，巧妙地利用了 x-vector，从而在解码时结合了有关说话人的信息。

我们可以使用随机的说话人嵌入（例如使用 `torch.zeros(1, 512)`），但结果会非常嘈杂。幸运的是，我们可以利用一些现有的在线说话人嵌入：

```
from transformers import SpeechT5ForTextToSpeech

from genaibook.core import get_speaker_embeddings

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")
model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts")

inputs = processor(text="There are llamas all around.", return_tensors="pt")
speaker_embeddings = torch.tensor(get_speaker_embeddings()).unsqueeze(0)

with torch.inference_mode():
    spectrogram = model.generate_speech(inputs["input_ids"], speaker_embeddings)

plt.figure()
plt.imshow(np.rot90(np.array(spectrogram)))
plt.show()
```




正如你所观察到的，模型输出是对数梅尔频谱图而不是波形。频谱图是强大的工具，但它们也有其局限性。虽然使用短时傅里叶变换将波形转换为频谱图是直接的，但将频谱图转换回波形的过程则不然。不幸的是，频谱图并不包含重建原始声音所需的所有信息。为了理解这一限制，让我们看一下正弦波的一般公式：

$$F(t) = A \sin\left(2\pi f t + \phi\right)$$

$A$ 表示振幅，$f$ 是频率，$t$ 是输入时间。所有这些信息都存在于频谱图中。缺少的一个元素是 $\Phi$（相位）。$\Phi$ 表示相位，它提供了有关信号的额外信息。虽然幅度和音高是更相关的属性，但相位信息对于准确的音频重建至关重要。因此，我们需要技术来从频谱图中重建原始波形（包括相位信息）。这是一个学习声码器的绝佳机会。

一种经典的重建方法是 Griffin–Lim 算法，这是一种从预测的频谱图中重建波形的迭代算法。该算法因其简单和快速而流行，但可能会导致低质量的音频。Griffin–Lim 算法对于某些频谱图生

成来说已经足够好，但不幸的是，如果你查看前一个示例中生成的频谱图，你可能会注意到图像质量还有提升空间。使用经典技术将其转换为波形会产生大量噪声和伪影，因此我们必须深入研究涉及神经网络的更高级技术。

获取用于频谱图到波形重建的训练数据非常容易，这导致近年来可训练模型（称为神经声码器）的研究兴起，这些模型接收一些特征表示或频谱图并将其转换为波形。WaveNet 是最早的模型之一——来自 DeepMind 的著名模型，能够实现高质量的重建。WaveNet 虽然质量高，但它是一个自回归模型，结果较慢，无法用于实际应用。在 WaveNet 的基础上进行了工作以使其更快，但不幸的是，它需要大量的优化和强大的 GPU。

基于 GAN 的方法已成为频谱图到波形重建的流行、高质量、实时替代方案。在高层次上，它们使用对抗训练方法，其中一个模型（称为生成器）接收梅尔频谱图并输出波形，而判别器模型则判断音频质量是否足够接近真实值，从而推动生成器和判别器共同改进。MelGAN 和 HiFiGAN 是流行的基于 GAN 的方法。它们快速且可并行化，质量与 WaveNet 相当，并且有良好的开源实现。我们将使用 HiFiGAN 作为 SpeechT5 的声码器。HiFiGAN 的生成器是一个 CNN，带有两个判别器，帮助评估音频的不同方面，推动 CNN 生成高质量的音频。我们可以通过直接传递频谱图来使用 SpeechT5HifiGan（或者，我们可以在生成语音时指定声码器参数）：

```
from transformers import SpeechT5HifiGan

vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan")
with torch.inference_mode():
    # Alternatively
    # model.generate_speech(
    #   inputs["input_ids"],
    #   speaker_embeddings,
    #   vocoder=vocoder)
    speech = vocoder(spectrogram)

import IPython.display as ipd
display(ipd.Audio(speech.numpy(), rate=16000))
```

请记住，你可以在笔记本中通过 `Audio(array, rate=sampling_rate)` 播放音频，或者探索我们编译的一些生成结果。

我们可以使用 SpeechT5 生成频谱图，并使用 HiFiGAN 将频谱图转换为波形。SpeechT5 是在英语上训练的，因此它在其他语言上表现不佳。可以对其进行微调以支持其他语言，但会有一些限制，例如不支持英语以外的字符。对其他语言进行微调还需要为非英语说话人获取说话人嵌

入，因此模型的预期表现会更差。如果你尝试了不同的说话人嵌入，结果的质量在很大程度上取决于说话人嵌入。也许我们可以做得更好?

理想的 TTS 设置是拥有一个单一模型（而不是频谱图生成器加上声码器），对其进行端到端训练，能够灵活地用于多个说话者，生成长音频，并且具有快速的推理能力。这引导我们走向一个名为 VITS 的新模型，它是一种并行的端到端方法。从高层次来看，VITS 可以被视为一种条件变分自编码器（conditional VAE）。VITS 结合了多种技巧，其中一些对我们来说并不陌生。VITS 使用 Transformer 编码器作为主编码器，并使用 HiFiGAN 生成器作为解码器。其他组件有助于提高质量和灵活性，以应对 TTS 中的一对多挑战。在训练过程中，另一个技巧是在计算重建损失时比较梅尔频谱图而不是最终的原始波形。这有助于训练专注于提高感知质量；如果你还记得，梅尔频谱图近似于我们感知声音的方式。通过将其纳入重建损失，模型被推动生成更易感知的语音。

VITS 于 2021 年由 Kakao Enterprise 发布，并成为当时的 SOTA 模型之一。2023 年，Meta 发布了大规模多语言语音数据集（Massively Multilingual Speech, MMS）。这个数据集带来了许多令人兴奋的结果。首先，它包含了用于识别 4,000 种口语语言的数据（音频分类任务）。Meta 还发布了超过 1,100 种语言的 TTS 和 ASR 数据。作者构建了一个新的预训练 Wav2Vec2 模型，并发布了针对这 1,100 种语言的多语言 ASR 微调模型。这一切与 TTS 有什么关系? MMS 的作者还为每种语言训练了单独的 VITS 模型，从而为越南语和荷兰语等多种语言提供了高质量的 TTS 模型，获得了比原始 VITS 模型更好的结果。这是一个很好的例子，展示了如何通过改进训练数据来获得更好的结果。让我们使用这个模型生成一些语音：

```
from transformers import VitsModel, VitsTokenizer, set_seed

tokenizer = VitsTokenizer.from_pretrained("facebook/mms-tts-eng")
model = VitsModel.from_pretrained("facebook/mms-tts-eng")

inputs = tokenizer(text="Hello - my dog is cute", return_tensors="pt")

set_seed(555)  # make deterministic
with torch.inference_mode():
    outputs = model(inputs["input_ids"])

outputs.waveform[0]
```

## 超越语音的Bark

你刚刚学习了如何使用SpeechT5和VITS生成语音。这可以应用于许多用例，但其他生成音频应用超越了语音。例如，你可能需要一个能够生成笑声或哭声的模型。你可能还希望有一个能够生成歌曲的模型。

我们将介绍的第三个生成模型是Suno AI的Bark（图9-10），这是另一个基于Transformer的模型。Bark是目前最流行的开源生成音频模型之一。它不仅可以生成语音，还可以生成其他声音。例如，你可以在提示中加入笑声或叹息声，相应的声音将被整合到语音中。我们甚至可以通过在提示中使用♪符号使其更具音乐性。Bark支持多语言，并包含一个语音预设库（类似于说话者嵌入），这使我们能够生成不同声音的语音。

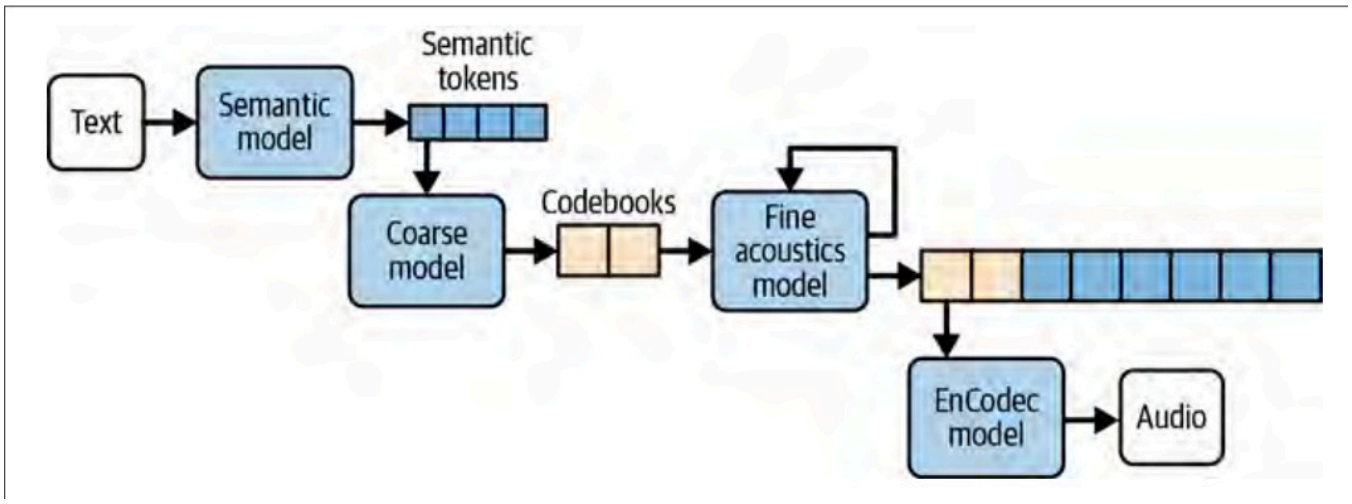



图9-10. Bark管道使用三个组件：文本到语义标记、语义到粗粒度标记和粗粒度到细粒度标记

在深入探讨Bark的细节之前，还有一个概念需要了解：音频编解码器。在压缩音频时，目标是在尽可能保持音质的同时减小文件大小（或比特率）。多年来，研究人员一直在尝试使用神经网络来实现这一目标，从而产生了像SoundStream和EnCodec这样的先进工具。EnCodec由Meta开发，是一个流行的开源神经编解码器，可以实时高质量地压缩音频。

EnCodec使用三步过程。首先，编码器将音频压缩为潜在表示。接下来，量化层将其转换为更紧凑、更高效的格式（压缩表示）。最后，解码器从压缩数据中重建音频。量化潜在空间由码本表示，每个码本包含多个可能的向量。例如，输入音频可能在量化潜在空间中使用32个码本向量（每个有1,024个条目）来表示。

有了编解码器工具，让我们回到Bark。Bark的目标是接收文本并将其映射到码本中。然后，Bark使用神经编解码器的解码器组件将码本转换为音频。实现这一目标有四个组件：

**文本模型**

一个自回归解码器Transformer，顶部有一个语言建模头，将文本提示映射为高级语义标记。因此，Bark可以泛化到训练数据之外的声音，如音效和歌词。

**粗粒度声学模型**

与文本模型相同的架构，但它将文本模型中的语义标记映射到前两个音频码本中。

**细粒度声学模型**

一个非因果自编码器Transformer，基于初始码本的总和迭代预测后续码本。它总共输出两个粗粒度码本和六个生成的码本。

**编解码器**

一旦预测了所有八个码本通道，EnCodec模型的解码器部分将解码输出音频数组。

有多个可配置参数。例如，我们可以修改由粗粒度和细粒度声学模型生成的码本数量。我们还可以调整每个码本的大小，官方实现中为1,024。这种架构使模型能够生成新的声音、多种语言的语音等。让我们来试试：

```
from transformers import AutoModel, AutoProcessor

processor = AutoProcessor.from_pretrained("suno/bark-small")
model = AutoModel.from_pretrained("suno/bark-small").to(device)

inputs = processor(
    text=[
        """Hello, my name is Suno. And, uh — and I like pizza. [laughs]
        But I also have other interests such as playing tic tac toe."""
    ],
    return_tensors="pt",
).to(device)


speech_values = model.generate(**inputs, do_sample=True)
```

这些结果相当不错。如果我们希望输出声音符合预定义的说话者，我们还可以使用作者共享的官方库中的说话者嵌入。也可以在自己的声音上训练这些说话者嵌入，从而能够生成符合你语调的

合成音频。让我们使用一个预定义的声音：

```
voice_preset = "v2/en_speaker_6"

inputs = processor("Hello, my dog is cute", voice_preset=voice_preset)

# 可能需要额外执行下面这一行
# inputs['history_prompt'] = inputs['history_prompt'].to(device)
audio_array = model.generate(**inputs.to(device))
audio_array = audio_array.cpu().numpy().squeeze()
```

## AudioLM和MusicLM

通过附加声音控制语音生成很酷，但我们能生成完整的旋律吗？答案是肯定的。让我们从AudioLM和MusicLM开始，这是Google在2023年推出的两个令人兴奋的模型，能够进行音频和音乐生成。这可以用于各种应用，例如在视频中生成噪音效果、为播客添加背景音乐或为游戏设计声音。

AudioLM接收几秒钟的音频录音，然后生成高质量的延续，保留说话者的身份和说话方式。该模型在没有转录和注释的情况下进行训练，这使其非常令人印象深刻。AudioLM是如何实现这一点的？从概念上讲，AudioLM与Bark类似，但任务不同：声音延续而不是TTA。

AudioLM（图9-11）首先使用w2v-BERT，这是一个预训练模型，将波形映射为丰富的语义标记（类似于我们在Bark中使用LM从文本生成语义标记）。然后，语义模型预测未来的标记，建模音频序列的高级结构。第二个模型，粗粒度声学模型，使用生成的语义标记和过去的声学标记生成新的标记。过去的声学标记是如何生成的？我们可以将输入波形传递给编解码器并检索码本（量化潜在表示）。这有助于保留说话者的特征并生成更连贯的音频。第四个模型，细粒度声学模型，为音频添加更多细节，进行细化，提高质量，并去除前一阶段的有损压缩伪影。最后，标记被输入到神经编解码器中以重建波形。

AudioLM也可以在音乐中进行训练，例如钢琴录音，以生成保持节奏和旋律的连贯延续。尽管底层模型不同，你可能会注意到这些阶段与Bark相似：我们训练一系列模型，最终生成高质量的码本，然后使用神经编解码器（Bark中的EnCodec和AudioLM中的SoundStream）生成最终的波形。

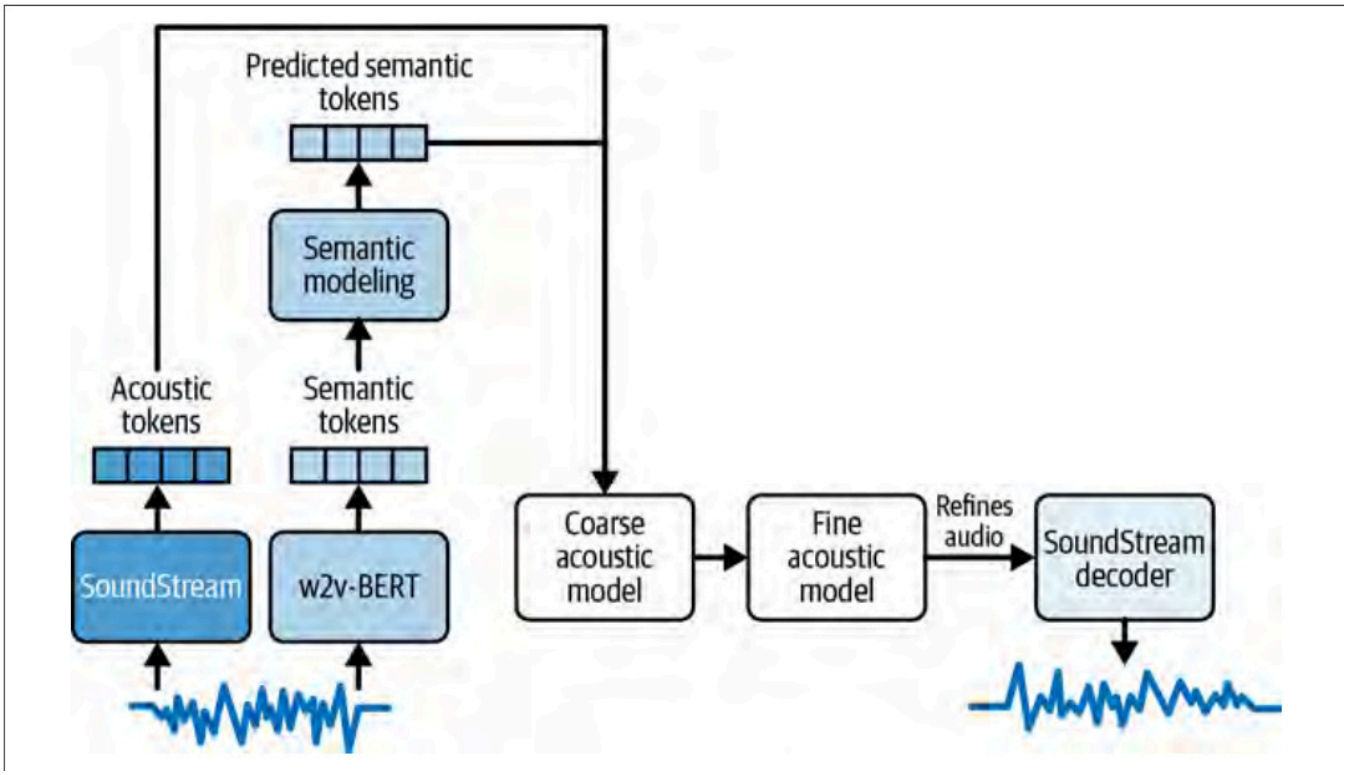



图9-11. 在AudioLM模型管道中，AudioLM将输入音频转换为标记序列，并使用语言建模技术进行音频生成（改编自原论文中的图像）

MusicLM（图9-12）更进一步，专注于高质量的音乐生成，准确匹配文本描述。例如，你可以有一个文本描述，如“一场激烈的小提琴摇滚音乐会”作为提示。MusicLM使用AudioLM的多阶段设置，并加入了文本条件。

如果获取高质量的标注TTS数据很复杂，那么标注TTA可能会更加复杂。TTA系统可能涉及更广泛的音频类型，包括环境声音和音乐。准确标注各种声音可能更具挑战性和强度。为了解决这个问题，MusicLM使用了一个额外的模型MuLan，它类似于CLIP用于图像-文本对，可以将文本及其对应的音频映射到相同的嵌入空间。因此，MuLan在训练期间不需要字幕，并能够在大量音频数据上进行训练。更具体地说，在训练期间，MusicLM使用从音频计算的嵌入来条件化模型，而在推理期间，MusicLM使用文本嵌入。

> 在撰写本文时，这些模型都不是开源的。LAION发布了一个名为CLAP的MuLan替代品。尽管训练数据比MuLan少20倍，但它可以生成多样化的音乐样本。EnCodec是SoundStream的开源替代品。

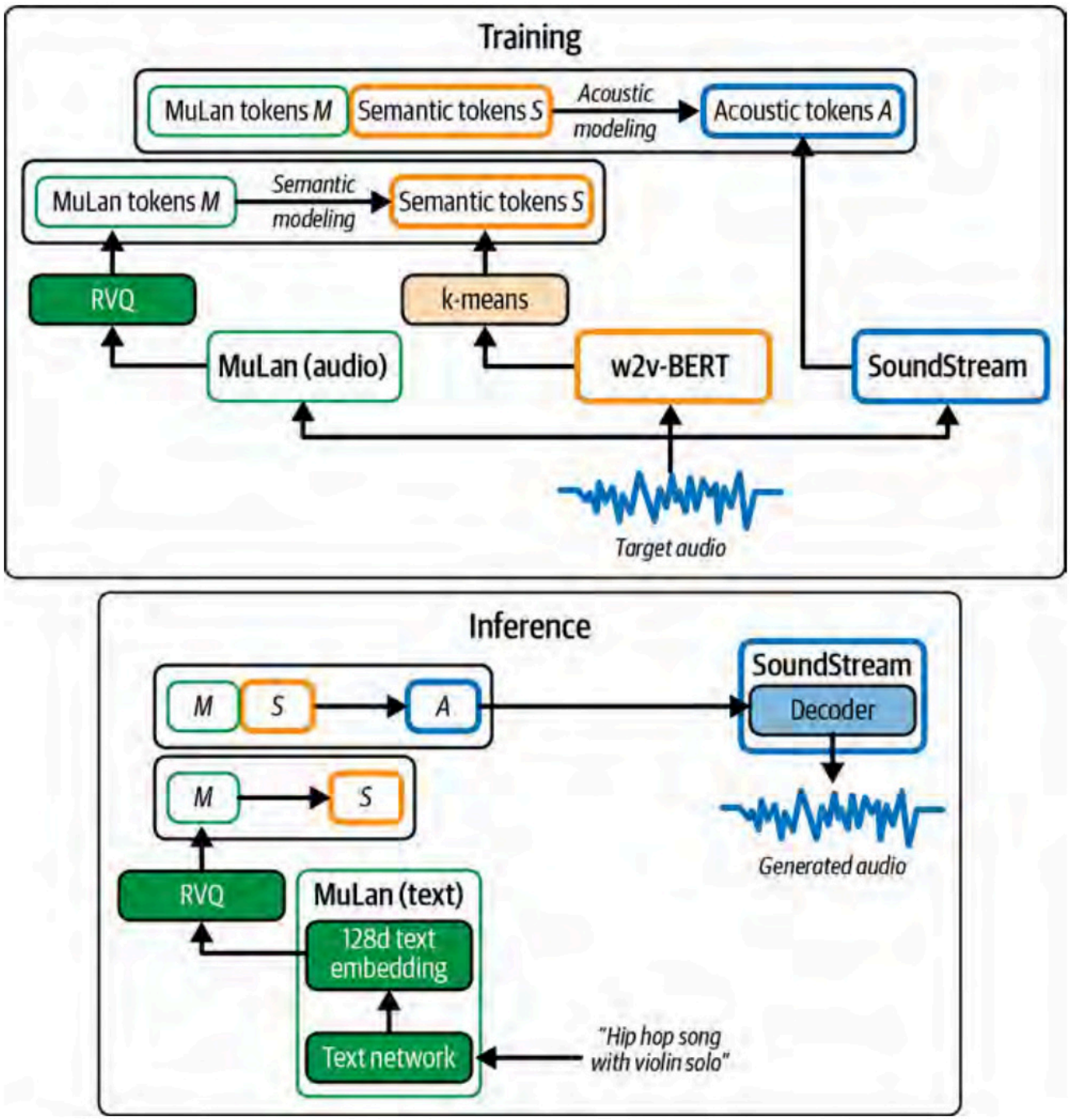



图9-12. MusicLM将文本条件引入AudioLM架构，以根据提示生成音频（改编自原论文中的图像）

## AudioGen 和 MusicGen

与此同时，在2022年和2023年，Meta发布了多个基于文本引导的音频生成开源模型。AudioGen可以生成声音和环境效果（例如，狗叫声或敲门声）。AudioGen遵循与Bark和AudioLM类似的流程。它使用T5的文本编码器生成文本嵌入，并用它们来引导生成过程。解码器自回归地生成基于文本和之前步骤的音频标记的音频标记。最终的音频标记可以通过神经编解码器进行解码。

AudioGen的开源版本是原始架构的一个变体；对于神经编解码器，他们在环境声音数据上重新训练了一个EnCodec。如果你觉得这很熟悉，那是因为确实如此——我们再次执行了与Bark和AudioLM类似的过程。那么，我们可以用AudioGen做什么呢？一个模型可以完成三项任务：

- 使用完整的架构，我们可以创建基于文本的音频，例如“狗在叫，同时有人在吹小号”。
- 如果我们移除文本编码器，我们可以无条件生成声音。
- 我们可以通过使用现有音频的输入音频标记来进行音频延续。

在AudioGen的基础上，Meta发布了MusicGen，它被训练为基于文本生成音乐，并在各种指标上取得了比MusicLM更好的结果。MusicGen通过文本编码器（如T5或Flan T5的编码器）传递文本描述以获得嵌入。然后，它使用一个语言模型生成基于嵌入的音频码本。最后，使用EnCodec解码音频标记以获得波形。发布了多个MusicGen模型。

> AudioGen使用公共数据集（如AudioSet、AudioCaps等）进行训练。而MusicGen则使用了20,000小时的Meta自有和特别许可的音乐进行训练，结合了内部数据集、Shutterstock和Pond5的音乐数据。

让我们尝试最小的MusicGen变体，它有3亿个参数，并加载每个组件：

```
from transformers import AutoProcessor, MusicgenForConditionalGeneration

model = MusicgenForConditionalGeneration.from_pretrained(
    "facebook/musicgen-small"
).to(device)
processor = AutoProcessor.from_pretrained("facebook/musicgen-small")
inputs = processor(
    text=["an intense rock guitar solo"],
    padding=True,
    return_tensors="pt",
).to(device)

audio_values = model.generate(
    **inputs, do_sample=True, guidance_scale=3, max_new_tokens=256
)
```

你已经学会了使用多个音频生成模型：Bark、SpeechT5和MusicGen。根据你期望的API抽象，我们可以使用transformers中提供的文本到音频和文本到语音的管道，而不是独立加载模型和处理器并自己编写所有推理代码。它们抽象了逻辑，非常适合运行推理：

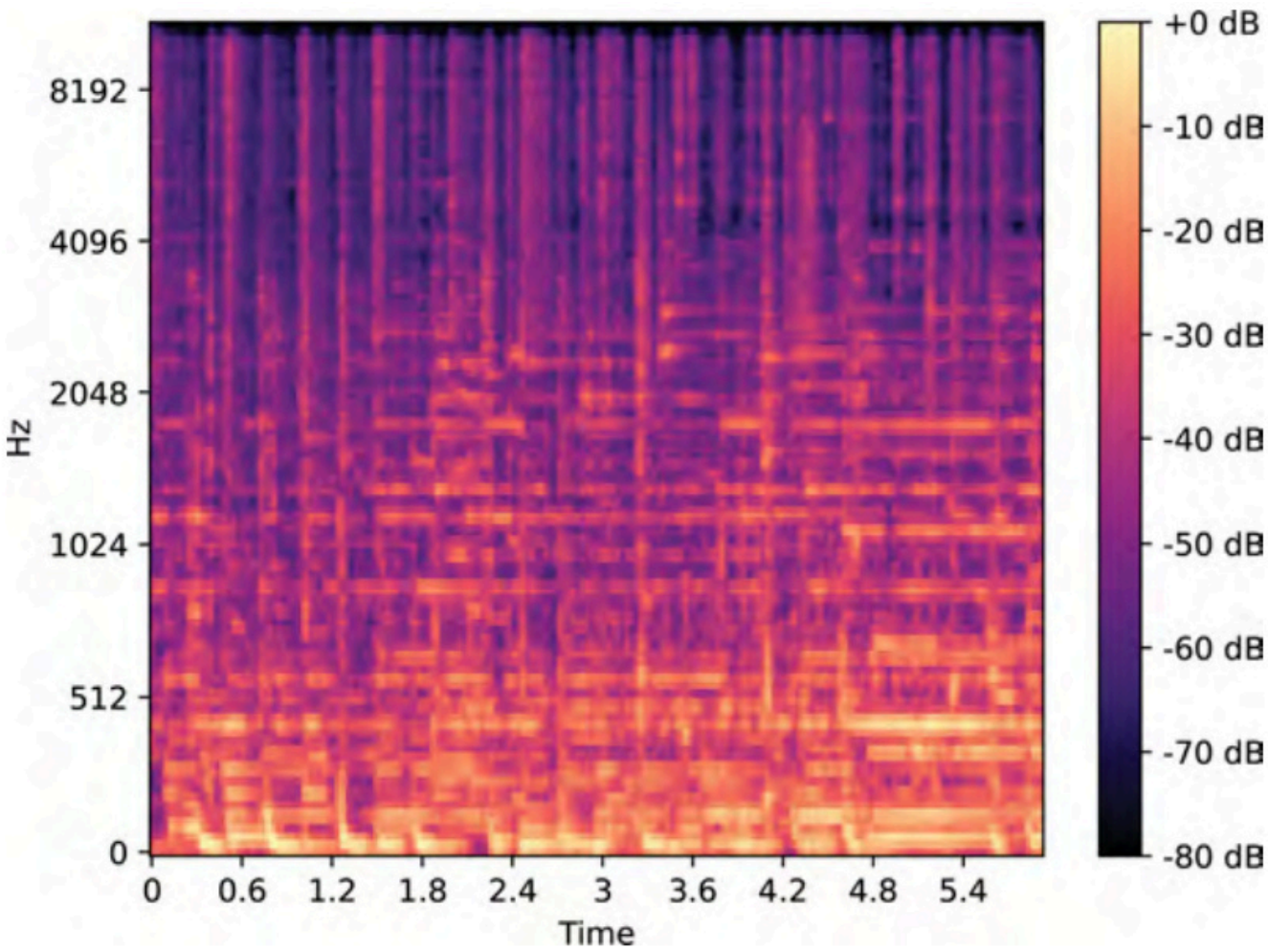
```
from transformers import pipeline

pipe = pipeline("text-to-audio", model="facebook/musicgen-small", device=device)
data = pipe("electric rock solo, very intense")

display(ipd.Audio(data["audio"][0], rate=data["sampling_rate"]))
```

## Audio Diffusion 和 Riffusion

让我们探索用于音频生成的扩散方法。通过频谱图，我们有了音频的信息丰富的视觉表示，它可以作为蓝图转换回声音。如果我们知道一个可以生成图像的模型，我们就可以做一些令人兴奋的事情……等等，我们知道扩散模型。正如第4章所讨论的，我们可以使用扩散管道来创建图像（有条件和无条件的）。

Audio Diffusion将这个简单的想法扩展到音频：我们可以从歌曲数据库中挑选数千个梅尔频谱图，然后训练一个无条件的扩散模型来生成梅尔频谱图图像，之后可以将其转换为音频。尽管这听起来出奇地简单，但它产生了不错的结果。例如，teticio/audio-diffusion-ddim-256是一个使用作者喜欢的歌曲中的20,000张图像训练的模型。让我们用这个模型生成一首歌曲：

```
from diffusers import AudioDiffusionPipeline

pipe = AudioDiffusionPipeline.from_pretrained(
    "teticio/audio-diffusion-ddim-256"
).to(device)

output = pipe()
```

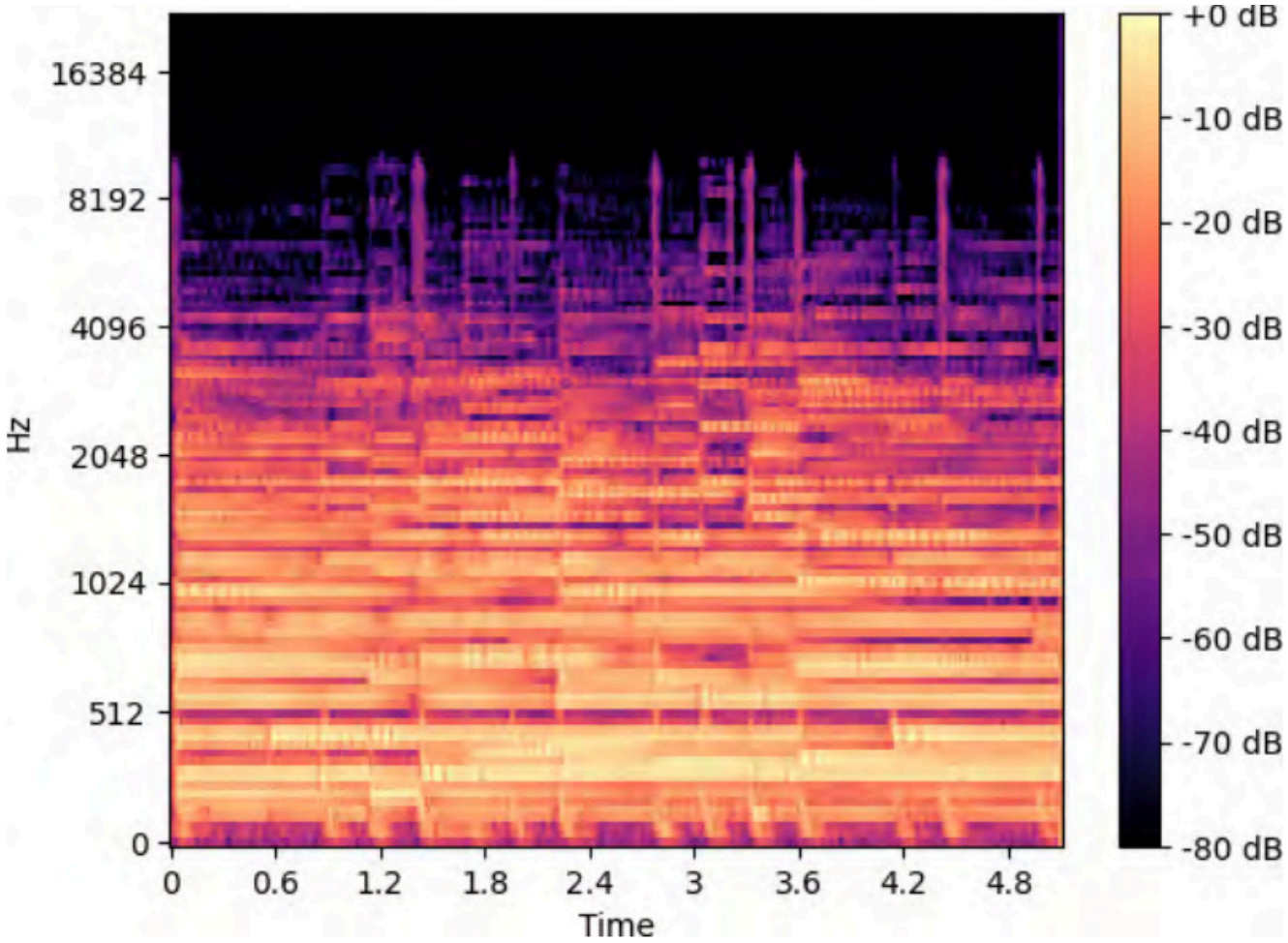



我们可以通过检查AudioDiffusionPipeline的结果（output.images）来访问频谱图。幸运的是，AudioDiffusionPipeline为我们处理了频谱图到音频的转换（通过使用Griffin–Lim，这在频谱图质量足够的情况下有效）并返回相应的音频（output.audios）。请注意，这个模型直接对频谱图进行去噪。或者，我们可以使用AutoEncoder对图像进行编码并在潜在空间中工作（如第5章所述），从而显著加快模型训练和推理速度。如果你想深入研究，这可能是一个很好的练习。

> 当你绘制output.images[0]中的频谱图时，你会观察到与之前从音频输出中获得的频谱图相比的差异。该模型被训练为生成灰度图像，而梅尔频谱图是彩色的。此外，生成的频谱图由于训练数据的结构而水平翻转。显示的频谱图已被翻转以保持与本章中显示的其他频谱图的一致性。

我们可以进一步扩展这个想法，并使用一个基于文本的模型来生成基于文本的频谱图。例如，Riffusion是Stable Diffusion的一个微调版本，可以根据文本提示生成频谱图图像。尽管这个想法听起来很奇怪，但它出奇地有效：

```
from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "riffusion/riffusion-model-v1", torch_dtype=torch.float16
)
pipe = pipe.to(device)
prompt = "slow piano piece, classical"
negative_prompt = "drums"
spec_img = pipe(
    prompt, negative_prompt=negative_prompt, height=512, width=512
).images[0]
```




使用Stable Diffusion的简单性带来了许多优势。常见的工具如图像到图像、修复、负面提示和插值都可以开箱即用。例如，你可以将原声独奏转换为电吉他独奏，或者在流派之间平滑过渡，比如从环境打字声到爵士乐，以创建独特的声音效果。让我们使用之前生成的频谱图，并通过一个新的提示使用图像到图像管道将钢琴曲转换为吉他曲：

```
from diffusers import StableDiffusionImg2ImgPipeline

pipe = StableDiffusionImg2ImgPipeline.from_pretrained(
    "riffusion/riffusion-model-v1", torch_dtype=torch.float16
)
pipe = pipe.to(device)

prompt = "guitar, acoustic, calmed"
generator = torch.Generator(device=device).manual_seed(1024)
image = pipe(
    prompt=prompt,
    image=spec_img,
    strength=0.7,
    guidance_scale=8,
    generator=generator,
).images[0]
```

## Dance Diffusion

尽管生成频谱图并将其转换为音频片段效果不错，但使用图像来训练音频模型并不十分直观。我们可以探索训练一个可以直接处理原始音频数据（数字数组）而不是图像的模型。正如第4章中所做的那样，UNet是一个具有一系列下采样层和上采样层的CNN。尽管到目前为止我们一直在使用适用于2D数据的UNet（通过使用UNet2DModel），但我们也可以使用一个适用于原始音频的UNet，即让UNet处理浮点数数组（UNet1DModel）。

Dance Diffusion是一个用于无条件音频生成的开源模型家族，它直接生成音频波形。有不同模型使用不同数据集进行训练。例如，harmonai/maestro-150k是一个使用200小时钢琴片段训练的模型，因此它可以无条件生成钢琴声音：

```
from diffusers import DanceDiffusionPipeline

pipe = DanceDiffusionPipeline.from_pretrained(
    "harmonai/maestro-150k", torch_dtype=torch.float16
)
pipe = pipe.to(device)
audio = pipe(audio_length_in_s=5, num_inference_steps=50).audios[0]
```

Dance Diffusion的设置与我们在第4章中看到的类似。通过更改UNet2DModel和数据，你可以开

箱即用地获得不错的结果。为Dance Diffusion设置一个最小的训练循环并使用你的数据可以相对简单，并且需要很少的设置。这些模型的质量还可以，但显然还有提升空间。

### 关于生成音频的扩散模型的更多内容

扩散模型的一个显著问题是其推理速度非常慢。受Stable Diffusion的启发，DiffSound和AudioLDM模型被设计为在潜在空间中运行。这个潜在空间是CLAP潜在嵌入空间。就像MusicLM中的MuLan和图像领域中的CLIP一样，CLAP是一个将文本和音频映射到共享潜在空间的模型。这使得扩散过程更快，并且不再需要标注数据。

这与Stable Diffusion类似，但在不同的模态中：我们使用CLAP而不是CLIP；我们仍然使用UNet骨干网络（实际上，与Stable Diffusion相同的规格）；并且我们使用VAE解码潜在空间并生成梅尔频谱图。就像我们早期的TTS探索中所做的那样，我们使用声码器HiFiGAN从频谱图合成音频波形。这种架构设计为AudioLLM提供了一个框架，用于执行多种文本引导的音频操作。例如，它可以进行修复、超分辨率和风格迁移。

MusicLDM对AudioLDM进行了修改，使其完全专注于音乐。CLAP不能直接使用，因为它在以音效和其他声音为主的数据集上进行了预训练，但音乐数据并不多。作者在文本-音乐配对数据集上重新训练了CLAP，增强了对音乐数据的理解。作者还在音乐数据上重新训练了HiFiGAN声码器，以确保重建的波形与音乐相对应。

文本到音乐生成的一个风险是，由于许可数据的稀缺性，扩散过程更有可能从原始训练数据中重建示例。为了避免这种情况，MusicLDM使用数据增强技术来鼓励模型在音乐数据中探索更多空间。例如，他们使用mixup技术，通过插值两个音频样本来创建一个新的样本。这有助于防止过拟合并提高泛化能力。

## 评估音频生成系统

与图像生成一样，TTS和TTA很难评估，因为没有单一的正确生成。除此之外，音频质量在很大程度上取决于人类感知：想象一下，你使用不同的模型根据提示生成一首歌曲；不同的人会有不同的偏好，这使得比较模型变得具有挑战性且成本高昂。

已经有了一些定义客观指标的方法。当没有任何参考音频时，可以使用Frechet音频距离（FAD）来评估音频模型的质量。该指标与人类感知相关，可以识别出合理的音频，但不一定能评估生成的音频是否遵循提示。另一种方法，如果有参考音频，可以使用分类器为预测和参考音频计算类别预测，然后测量两者概率分布之间的KL散度。在缺乏参考数据的情况下，CLAP可以将输入提示和输出音频映射到相同的嵌入空间，并计算嵌入之间的相似性。这种方法会强烈偏向于用于CLAP的预训练数据。

最终，TTS和TTA系统始终需要人类评估。如前所述，生成的质量是主观的，因此理想情况下，应由来自不同背景的多人评估系统的质量。一种方法是计算平均意见分数（MOS），这需要人类在某个尺度上评估生成。另一种方法是向人类展示输入提示和来自不同模型的两个不同生成，然后要求人类选择他们更喜欢哪一个。

# 下一步是什么?

生成音频的世界正在经历前所未有的创新和增长时刻。音频和语音领域的最新进展非常显著，但该领域仍在不断发展，不断提供新的探索前沿。

例如，开发实时高质量的音频生成是一个令人兴奋的新兴领域。实时生成高质量音频的前景为跨领域的应用开辟了许多可能性，从辅助工具到游戏开发。最近的发布，如Coqui的XTTS、ElevenLabs的TTS工具和Kyutai的Moshi，都是很好的例子。

另一个新兴主题是统一建模，即创建可以在不同任务之间灵活切换的模型。像SeamlessM4T这样引入可扩展的统一语音翻译系统的模型是这个领域的一个很好的例子。这个单一模型可以支持各种音频任务，如TTS、语音到语音翻译、文本到语音翻译（生成另一种语言的合成语音）等。本章探讨了流行的高质量语音、音频和音乐生成模型。这三种数据域之间的差异为训练一个统一模型以生成这三种形式的音频带来了挑战。然而，最近的模型技术如AudioLDM 2正在探索这个方向。这些模型提出了一种统一的音频语言，使其能够创建所有这些形式，并获得与其他任务特定模型相当或更好的结果。

音频的挑战和讨论不仅限于建模。关键的伦理问题包括数据来源、版权法、记忆和所有权。虽然克隆一个人的声音在技术上令人印象深刻，但在未经同意的情况下应用于他人的声音时，会引发严重的伦理问题。训练一个音乐模型可能是一个令人兴奋的智力和创造性实验，但确定公平的数据集使用仍然是一个悬而未决的问题。例如，未经同意创建名人声音的合成版本用于广告可能被视为侵犯其个人权利。

此外，最近大多数音频的机器学习研究主要集中在英语上，多语言方面仍有很大的进步空间。如果语音识别模型主要针对少数语言开发，其他语言可能会获得次优或无法访问的服务和工具。这些伦理考虑强调了负责任地开发和应用机器学习技术的重要性。主动解决这些问题有助于建立信任，并确保这些技术的好处得到公平分配。

# 项目时间：端到端对话系统

在本书的各个章节中，你已经学会了使用Transformer模型生成文本（第2章和第6章）、扩散模型生成图像（第4章和第5章）、转录和生成语音的模型（本章），以及构建Gradio演示（第5

章）。在这个挑战中，目标是构建一个端到端的Gradio应用程序，其中：

- 用户可以输入或说出提示
- 提示可以是对话式的，也可以是要求生成图像的
- 根据提示，模型生成响应，可能是图像或文本
- 模型将输出图像和文本，以及在文本情况下生成的相应语音

这个项目需要许多设计决策。你会选择哪个模型？如何平衡质量和速度？如何确定提示是否要求生成图像？这个开放式项目需要你使用在本书中学到的技能。我们建议从一个简单的版本开始，然后逐步迭代。你不需要为这个项目训练任何模型，但如果你愿意，可以微调模型。目标是构建一个有趣且互动的对话系统，可以根据用户提示生成图像、文本和音频。

# 总结

多么精彩的冒险！生成音频领域正处于一个鼓舞人心的时刻，每隔几周就有新模型出现，更高质量的数据集被发布，新的实验室进入生成音频领域。面对音频领域中使用的众多模型，感到不知所措是正常的；毕竟，这个领域进展非常快。来自其他模态的见解，如语言或潜在空间中的扩散模型，激发了我们使用的许多工具。音频的固有复杂性使我们发现了新的组件，如用于频谱图到波形重建的声码器和用于音频压缩和解压缩的神经编解码器。虽然本章仅介绍了音频领域，但它为我们提供了深入研究最新研究的基础知识。

如果你想进一步探索这个领域，我们建议研究以下主题：

**CTC**

要了解Wav2Vec2使用的CTC算法，我们推荐阅读互动博客文章“Sequence Modeling with CTC”。

**ParlerTTS**

ParlerTTS是一个用于TTS的训练和推理库（以及一系列模型）。该库轻量级，可以生成高质量且可定制的语音。我们建议探索ParlerTTS的GitHub仓库，并尝试其推理和训练示例。

**声码器**

在本章中，我们简要介绍了声码器，如HiFiGAN，它可以将梅尔频谱图转换为语音。我们建议阅读关于WaveNet、MelGAN和HiFiGAN声码器的内容，以获得更全面的概述。它们之间有什么区别？如何评估它们？

### 模型优化

不同的模型优化技术可以显著加快音频生成速度，同时最小化质量损失。我们推荐阅读博客文章“AudioLDM 2, but Faster ⚡”和“Speculative Decoding for 2x Faster Whisper Inference”。

### 其他流行模型

我们在本章中涵盖了许多模型。除了深入研究它们，我们建议探索其他流行模型，如Tacotron 2、FastSpeech、FastSpeech 2、TorToiSe TTS和VALL-E。对这些模型的高层次理解将提供生态系统的完整图景。

正如我们遇到了大量的数据集和模型，表9-1和表9-2简洁地总结了进一步探索的关键资源。

表 9-1. 数据集摘要

| Dataset | Description | Training hours | Recommended use |
|---|---|---|---|
| LibriSpeech | Narrated audiobooks | English: 960 | Benchmarking and pretraining models |
| Multilingual LibriSpeech | Multilingual equivalent of LibriSpeech | English: 44,659<br>Total: 65,000 | Benchmarking and pretraining models |
| Common Voice 13 | Crowd-sourced multilingual with varying quality | English: 2,400<br>Total: 17,600 | Multilingual systems |
| VoxPopuli | European Parliament recording | English: 543<br>Total: 1,800 | Multilingual systems, domain-specific uses, nonnative speakers |
| GigaSpeech | Multidomain English from audiobooks, podcasts, and YouTube | English: 10,000 | Robustness over multiple domains |
| FLEURS | Parallel multilingual corpus | 10 hours for each of 102 languages | Evaluation in multilingual settings (including “low digital resource” setting) |
| MMS-labeled | New Testament read out loud | 37,000 hours for a total of 1,100 languages | Multilingual systems |
| MMS-unlabeled | Recordings of stories and songs | 7,700 hours for 3,800 languages | Multilingual systems |

表 9-2. 模型摘要

| Model | Task | Model type | Notes |
|---|---|---|---|
| Wav2Vec2 | English ASR | Encoder transformer with CTC | Trained on unlabeled English data. Can easily be fine-tuned. |
| HuBERT | English ASR | Encoder transformer with CTC | Trained on unlabeled English data. Can easily be fine-tuned. |
| XLS-R | Multilingual ASR | Encoder transformer with CTC | Trained on unlabeled data for 128 languages. |
| Whisper | Multilingual ASR | Encoder-decoder transformer | Trained on a massive amount of multilingual labeled data. |
| SpeechT5 | ASR, TTS, and S2S | Encoder-decoder transformer | Adds pre- and post-nets to map speech and text to the same space. |
| HiFiGAN | Spectrogram to speech | GAN with multiple discriminators | It's one type of vocoder. |
| EnCodec and SoundStream | Audio compression | Encoder-decoder | Uses quantized latent space. |
| Bark | Multilingual TTA | Multistage autoregressive | Predicts codebooks and uses EnCodec to reconstruct. |
| MuLan and CLAP | Map text and audio to the same space | Transformer encoder for text and CNN for audio | CLAP is the open source replication of MuLan. |
| AudioLM | Audio continuation | Multistage autoregressive | Similar conceptually to Bark but uses audio input. |
| MusicLM | Text to music (TTM) | Combines MuLan and AudioLM | Incorporates MuLan to remove need for labeled data. |

| Model | Task | Model type | Notes |
|---|---|---|---|
| AudioGen | TTA | Multistage autoregressive | EnCodec is retrained on environmental sound data. |
| MusicGen | TTM | Same as AudioGen | Multiple variants are open source. |
| AudioLDM | TTA | Latent space diffusion (same as Stable Diffusion) | Uses CLAP for latent space. |
| MusicLDM | TTM | Same as AudioLDM | Retrains CLAP and HiFiGAN on music domain. |

# 练习

1. 使用波形图与频谱图各有什么优缺点?
2. 什么是频谱图，什么是梅尔频谱图？模型中通常使用哪一种?
3. 解释CTC（连接时序分类）的工作原理。为什么基于编码器的ASR模型需要它?
4. 如果推理数据的采样率为$8\text{kHz}$，而模型的训练数据采样率为$16\text{kHz}$，会发生什么情况?
5. 如何将n-gram模型添加到基于编码器的模型中?
6. 基于编码器的模型与基于编码器-解码器的模型在ASR中各有什么优缺点?

7. 在什么情况下你会更倾向于使用CER（字符错误率）而不是WER（词错误率）来评估ASR?
8. SpeechT5使用了哪六个网络？进行语音转换需要哪种设置?
9. 什么是声码器？在什么情况下你会使用它?
10. EnCodec模型的目的是什么?
11. TTA模型如何利用MuLan/CLAP来减少对标注数据的依赖?

你可以在本书的GitHub仓库中找到这些练习和以下挑战的解答。

# 挑战

1. Whisper探索。以下代码片段从头创建了一个随机数组和一个Whisper特征提取器：

```
import numpy as np
from transformers import WhisperFeatureExtractor
array = np.zeros((16000, ))
feature_extractor = WhisperFeatureExtractor(feature_size=100)
features = feature_extractor(
    array, sampling_rate=16000, return_tensors="pt"
)
```

探索改变`feature_size`、`hop_length`和`chunk_length`对输入特征形状的影响。然后，查看Whisper FeatureExtractor文档中的默认值及其含义，并尝试计算一个音频块会生成多少特征。
2. 语音转换。使用SpeechT5实现语音转换，使得输入音频由不同的说话者说出。
3. 训练Dance Diffusion。为Dance Diffusion实现一个小型训练管道。你可以使用第4章的代码作为起点。

# 参考文献


Agostinelli, Andrea, 等. “MusicLM: 从文本生成音乐.” arXiv, 2023年1月26日. http://arxiv.org/abs/2301.11325.

Ao, Junyi, 等. “SpeechT5: 统一模态编码器-解码器预训练用于口语处理.” arXiv, 2022年5月24日. http://arxiv.org/abs/2110.07205.

Ardila, Rosana, 等. “Common Voice: 一个大规模多语言语音语料库.” arXiv, 2020年3月5日. http://arxiv.org/abs/1912.06670.

Babu, Arun, 等. “XLS-R: 大规模自监督跨语言语音表示学习.” arXiv, 2021年12月16日. http://arxiv.org/abs/2111.09296.

Baevski, Alexei, 等. “Wav2vec 2.0: 自监督学习语音表示的框架.” arXiv, 2020年10月22日. http://arxiv.org/abs/2006.11477.
Barrault 等. SeamlessM4T 网页. 2022年8月22日. https://oreil.ly/WTVBw.
Borsos, Zalán, 等. “AudioLM: 一种语言建模方法用于音频生成.” arXiv, 2023年7月25日. http://arxiv.org/abs/2209.03143.
Chen, Ke, 等. “MusicLDM: 使用节拍同步混合策略增强文本到音乐生成的新颖性.” arXiv, 2023年8月3日. http://arxiv.org/abs/2308.01546.
Conneau, Alexis, 等. “FLEURS: 少样本学习评估通用语音表示.” arXiv, 2022年5月24日. http://arxiv.org/abs/2205.12446.
Conneau, Alexis, 等. “无监督跨语言表示学习用于语音识别.” arXiv, 2020年12月15日. http://arxiv.org/abs/2006.13979.
Copet, Jade, 等. “简单且可控的音乐生成.” arXiv, 2023年6月8日. http://arxiv.org/abs/2306.05284.
Défossez, Alexandre, 等. “高保真神经音频压缩.” arXiv, 2022年10月24日. http://arxiv.org/abs/2210.13438.
Gandhi, Sanchit. “音频数据集完全指南.” Hugging Face 博客, 2022年12月15日. https://oreil.ly/CXGff.
Gandhi, Sanchit, 等. “ESB: 多领域端到端语音识别基准.” arXiv, 2022年10月24日. http://arxiv.org/abs/2210.13352.
Gandhi, Sanchit, 等. Hugging Face 音频课程. Hugging Face, 2023年6月14日. https://oreil.ly/Z-IZ0.
Hollemans, Matthijs. “使用 SpeechT5 进行语音合成、识别等.” Hugging Face 博客, 2023年2月8日. https://oreil.ly/xar9H.
Hsu, Wei-Ning, 等. “HuBERT: 通过掩码预测隐藏单元进行自监督语音表示学习.” arXiv, 2021年6月14日. http://arxiv.org/abs/2106.07447.
Huang, Qingqing, 等. “MuLan: 音乐音频和自然语言的联合嵌入.” arXiv, 2022年8月25日. http://arxiv.org/abs/2208.12415.
Kim, Jaehyeon, 等. “条件变分自编码器与对抗学习用于端到端文本到语音.” arXiv, 2021年6月10日. http://arxiv.org/abs/2106.06103.
Kong, Jungil, 等. “HiFi-GAN: 高效高保真语音生成的生成对抗网络.” arXiv, 2020年10月23日. http://arxiv.org/abs/2010.05646.
Kreuk, Felix, 等. “AudioGen: 文本引导的音频生成.” arXiv, 2023年3月5日. http://arxiv.org/abs/2209.15352.
Kucsko, Georg. 2023. Bark GitHub 仓库. 2023年9月17日. https://oreil.ly/Pq2K5.
Liu, Haohe, 等. “AudioLDM: 使用潜在扩散模型进行文本到音频生成.” arXiv, 2023年9月9日.

http://arxiv.org/abs/2301.12503.
Panayotov, Vassil, 等. “Librispeech: 基于公共领域有声读物的 ASR 语料库.” 在 2015 年 IEEE 国际声学、语音和信号处理会议上, 第 5206–10 页. IEEE, 2015. https://oreil.ly/MOIUF.
Patry, Nicolas. “在 Transformers 中使用 Wav2Vec2 处理大文件的自动语音识别.” Hugging Face 博客, 2022年2月1日. https://oreil.ly/gh8O1.
Pratap, Vineel, 等. “将语音技术扩展到 1,000+ 种语言.” arXiv, 2023年5月22日. http://arxiv.org/abs/2305.13516.
Radford, Alec, 等. “通过大规模弱监督实现鲁棒语音识别.” arXiv, 2022年12月6日. http://arxiv.org/abs/2212.04356.
Von Platen, Patrick. “在 Transformers 中使用 N-Grams 增强 Wav2Vec2.” Hugging Face 博客, 2022年1月12日. https://oreil.ly/o0HLV.
Wu, Yusong, 等. “大规模对比语言-音频预训练与特征融合和关键词到字幕增强.” arXiv, 2023年4月7日. http://arxiv.org/abs/2211.06687.
Yang, Dongchao, 等. “Diffsound: 用于文本到声音生成的离散扩散模型.” arXiv, 2023年4月28日. http://arxiv.org/abs/2207.09983.
Zeghidour, Neil, 等. “SoundStream: 端到端神经音频编解码器.” arXiv, 2021年7月7日. http://arxiv.org/abs/2107.03312.

# 第10章. 生成式 AI 中快速发展的领域

生成式 AI 的领域发展非常迅速。自从我们开始编写这本书以来，我们已经见证了 GPT-4、Llama 3、Gemini 和 Sora 等新模型的发布。除此之外，还涌现了许多新的基础大语言模型、音频模型和扩散技术。正如前言中提到的，本书专注于提供可推广的技能和理解的基本原则和基础知识，使您能够随着领域的不断发展而跟进。

在结束本书之前，我们希望为您提供一些生成式 AI 中最令人兴奋且快速发展的领域的一瞥。本章提供了这些主题和资源的高级概述，以便您在感兴趣时进一步深入。与其说是让您精通这些主题，不如将本章视为您继续学习的指南。

## 偏好优化

在第6章中，我们基于Open Assistant对话数据集以监督学习的方式训练了一个聊天模型。我们使用了传统的微调方法，但最近出现了一股整合偏好的模型浪潮。这些模型被训练为生成符合某些期望的响应。例如，有些人可能希望训练非常有帮助的模型，无论请求是什么，它们都会尽力提供帮助。其他公司可能希望训练更加中立的模型，避免生成有害的输出。

当模型表示无法提供帮助时，这是由于它的偏好优化。偏好优化用于引导大型语言模型（LLM）朝着期望的行为发展，这可能是生成更少错误的代码、以对话风格生成文本，或者拒绝生成某些主题的内容。

基于人类反馈的强化学习（RLHF）是用于偏好调优的方法之一，它对传统的微调过程进行了一些调整。正如我们在第6章中所做的那样，第一步是通过监督微调对模型进行微调。在RLHF中，微调后的模型随后被用于训练一个奖励模型。对于每个提示，模型会生成多个潜在的选项。然后，由评判者对选项进行排序，奖励模型被训练为预测评判分数。评判者通常是人类，但最近有使用其他大型模型作为评判者的趋势（例如，RLAIF论文使用LLM进行排序）。最后一步是使用奖励模型进一步微调原始的监督微调模型，使模型学会生成与高分（根据评判者）示例相似的输出。RLHF是Llama 2聊天模型以及ChatGPT的关键组成部分。

虽然在微调过程中引入奖励模型的概念很有趣，但它也增加了一层复杂性，如图10-1所示。这促使了大量研究，旨在用新的损失函数取代奖励模型。最近的研究，如直接偏好优化（DPO）、身份偏好优化（IPO）和卡尼曼-特沃斯基优化（KTO），是这一持续探索的显著例子。这些方法完全移除了RL组件，而RL组件被认为是不稳定且难以训练的。

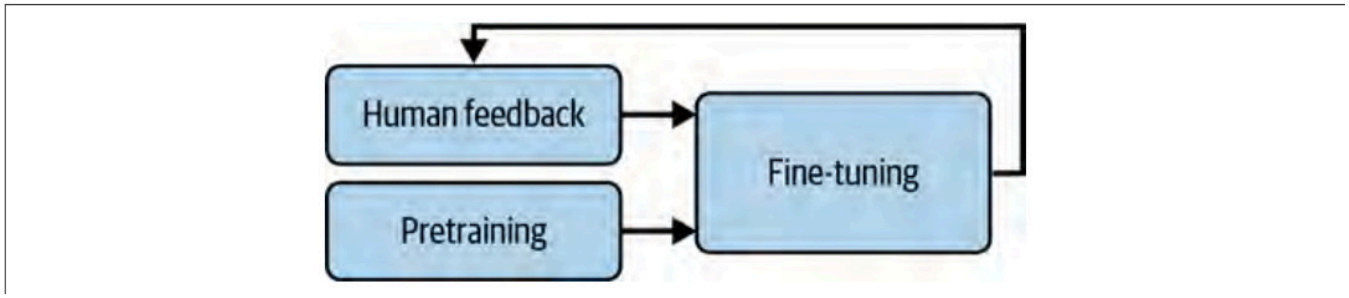



图10-1. RLHF有三个核心组成部分：预训练、微调和人类反馈

RLHF也可以应用于扩散模型。去噪扩散策略优化（DDPO）是一种使用强化学习来微调扩散模型并提高生成图像质量的方法。RLHF也可以应用于扩散模型，如图10-2所示。

图10-2. RLHF应用于扩散模型

以下是一些关于该主题的额外阅读材料：

- Hugging Face的RLHF介绍博客文章是RLHF的逐步概述。
- Sebastian Rashka的RLHF博客文章提供了RLHF及其替代方案的概述。
- 要了解RLHF在扩散模型中的应用，我们推荐阅读DDPO博客文章或Tanishq Abraham的博

客。

- 要了解更多关于DPO、IPO和KTO的信息，我们推荐阅读HF博客文章。
- 宪法AI是另一种将LLM与一组价值观对齐的方法。HF博客文章是深入了解它的好资源。

# 长上下文

本书中讨论的大多数LLM可以处理最多几千个标记的上下文，其中Llama 3.1在128,000个标记上表现出色，有些模型甚至可以处理几十万个标记。专有模型已经实现了更长的上下文，例如Gemini可以处理多达200万个标记，Anthropic Claude支持200,000个标记。处理极长的上下文对于RAG系统（如第6章项目中实现的系统）或进行代码生成或理解的系统非常有用，其中整个代码库可以用作上下文。

随着输入变得过长，会出现多个问题：

- LLM需要更多的VRAM来处理长上下文。
- 随着上下文的增长，生成的质量往往会下降。由于模型没有在如此长的上下文中进行训练，它们可能无法捕捉标记之间的依赖关系。
- 生成速度变慢。传统的注意力机制是一个瓶颈，因为它需要二次复杂度。

一种解决方案是使用窗口注意力，它限制了输入到LLM的标记数量。你可以将窗口注意力视为通过输入文本的滑动窗口。这可以保持GPU使用率的上限，但质量仍然会下降，因为窗口开始之前可能有一些携带重要信息的标记。窗口注意力可以通过诸如Attention Sinks之类的方法进行调整，以考虑初始标记，这对于多轮对话非常有用。另一种方法是使注意力机制更高效。有许多关于次二次缩放的提议，例如Longformer（它将窗口技术与全局注意力特征相结合）和Flash Attention（它通过融合操作优化内存传输）。Flash Attention已成为一种流行的解决方案，因为它在推理过程中使空间需求线性化。

虽然上述方法侧重于使注意力算法更快或计算效率更高，但也有研究通过旋转位置嵌入（RoPE）缩放扩展预训练的LLM。我们能否选择Llama并使其处理比预训练时更多的标记？RoPE改变了位置信息如何被纳入Transformer中，从而使它们能够捕捉长距离依赖关系。通过进行最少的微调，我们可以扩展上下文窗口。例如，Meta能够将原始LLaMA模型的上下文窗口从2,048个标记扩展到32,768个标记。

另一波研究也在探索任意长的上下文。Ring Attention和Infini-Attention是两个研究方向，旨在能够处理无限上下文。关于长上下文模型评估的研究仍处于早期阶段。Needle in a Haystack，如图10-3所示，是一种用于长上下文检索的示例方法。

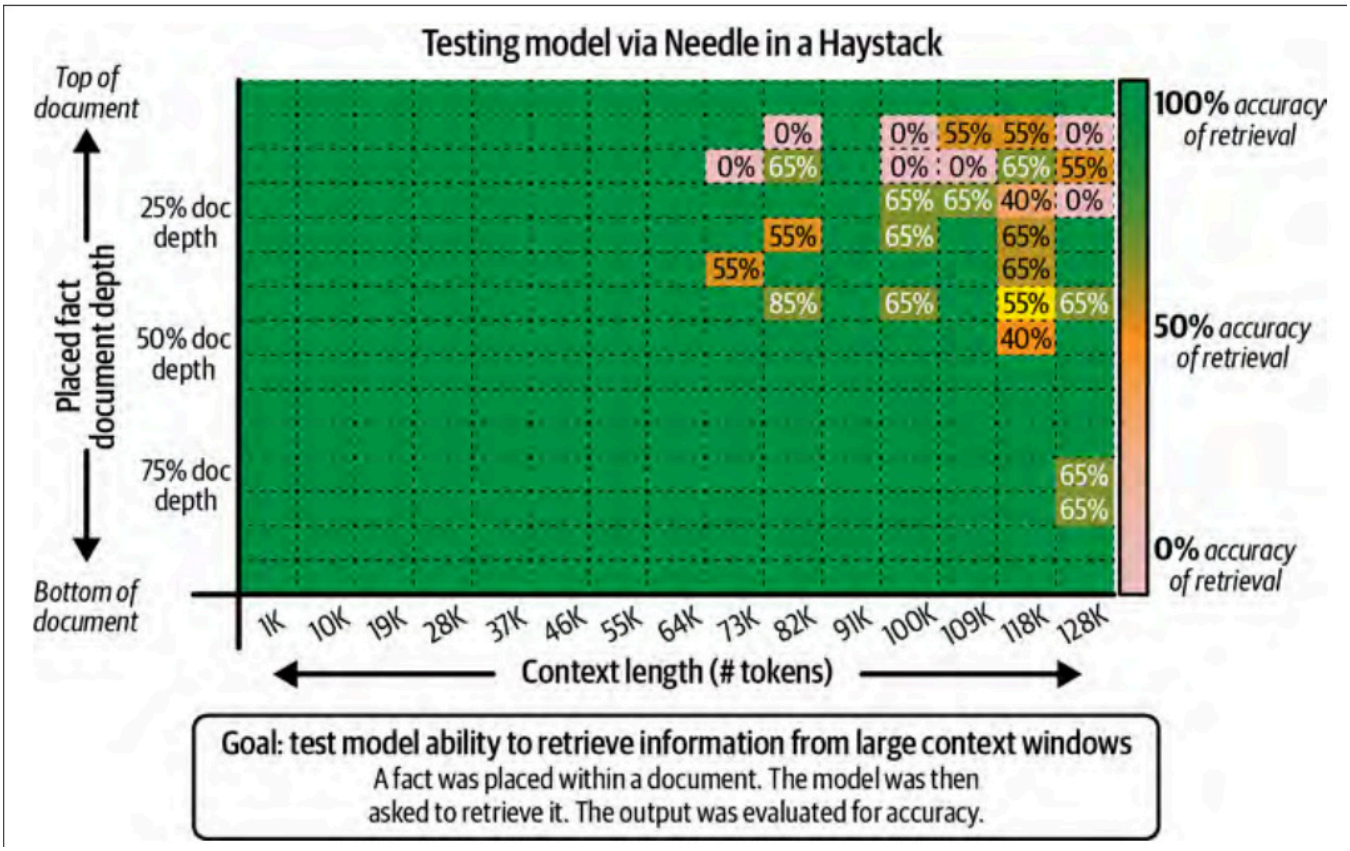



图10-3. Needle in a Haystack评估长上下文中的上下文检索

在生态系统中，还有并行的努力，旨在完全使用替代架构。其中之一是使用RNN：RWKV是一系列开源模型，由于线性注意力，它们实现了高效的推理，同时保持了非常高效和可并行化的训练。另一种替代方案是使用状态空间模型（SSM）。Mamba使用SSM实现了与标记数量相关的线性内存缩放，并且具有非常快的推理速度。

关于该主题的一些推荐阅读材料包括：

- Attention Sinks博客文章解释了Attention Sinks的工作原理以及不同实验的结果。
- Flash Attention和Flash Attention 2论文。
- 有许多并行的努力通过RoPE扩展上下文，从从业者进入ML到Meta的研究。
- 要了解更多关于RWKV的信息，我们推荐阅读其发布博客文章、其论文或一篇关于架构改进的最新论文。
- 要了解SSM和Mamba，我们推荐阅读“The Annotated S4”和“Mamba: The Hard Way”。

## 专家混合模型

近年来，专家混合模型（Mixture of Experts, MoE）已成为大型语言模型（LLMs）中一种引人

注目的方法，其中最著名的发布是Mistral团队在2023年12月推出的Mixtral 8x7B。在Transformer模型的背景下，MoE与传统的密集（dense）Transformer非常相似（见图10-4），但它们在训练效率和生产可扩展性方面具有优势。在给定固定计算资源的情况下，MoE能够比密集模型在训练曲线上走得更远。在大规模使用中，MoE在处理每秒大量请求时可能更加高效。

MoE模型的一个关键元素是将Transformer块中的部分或全部前馈网络替换为稀疏的MoE块。每个MoE块是一组“专家”网络，每个专家都是一个不同的模型（通常是另一个前馈网络）。给定一个token，只有部分专家会动态激活，而其余专家则处于空闲状态。为了学习哪些专家应该激活，MoE使用一个门控网络（或路由器），在训练和推理过程中动态地将token分配给不同的专家。这个门控网络充当流量控制器，确保token在专家之间有效分配。这使得门控网络成为MoE的关键组件，许多研究都致力于训练更好的门控网络。

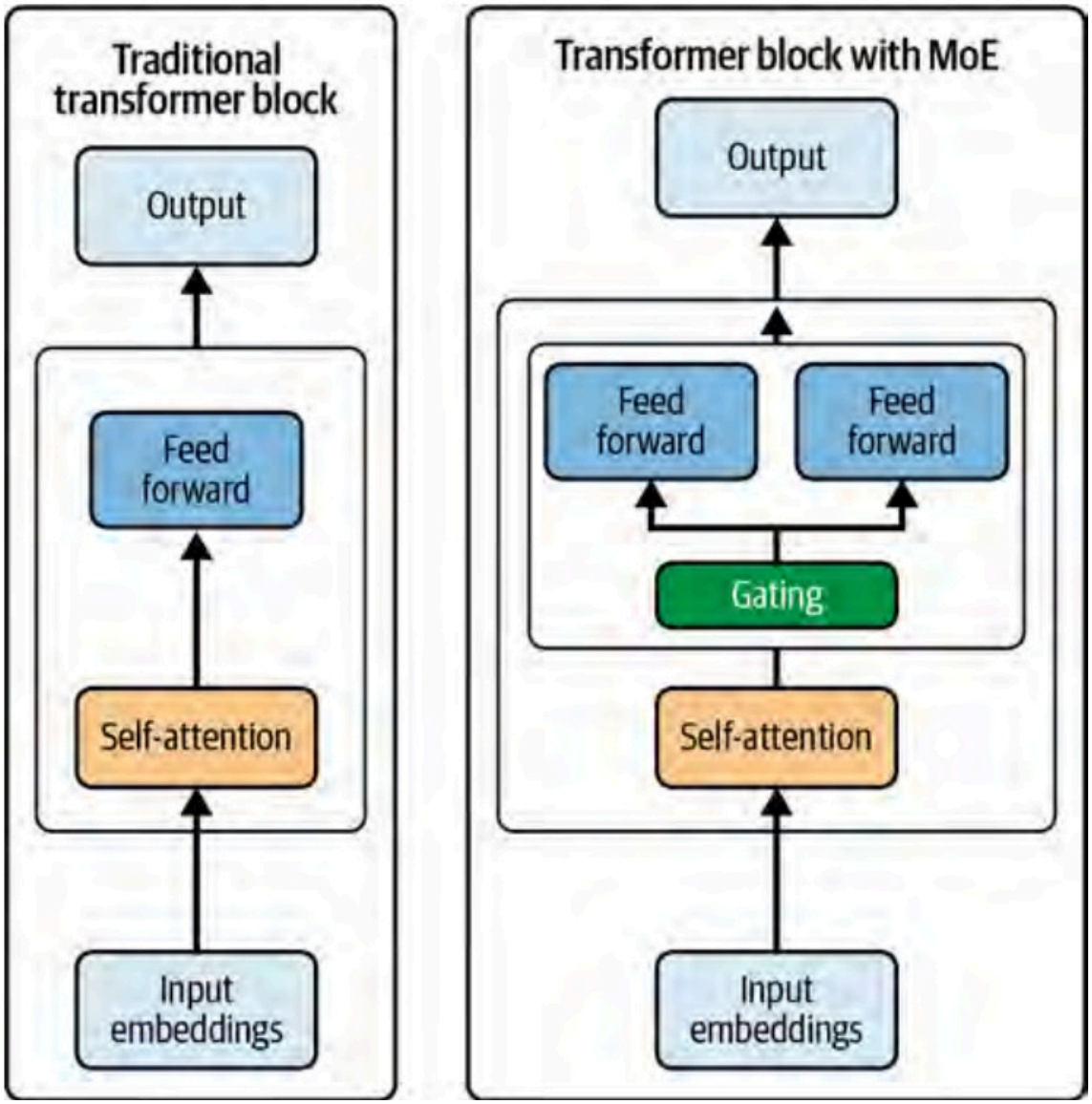


图10-4. 一个简化的传统Transformer块及其具有两个专家的MoE等效结构

需要澄清的是，MoE中的专家数量并不会直接导致参数线性增加。例如，在Mixtral $8x7B$ 的情况下，模型名称可能暗示有八个专家，每个专家有70亿参数，因此总共有560亿参数，但实际上它包含470亿参数，因为存在共享组件，而“8”指的是每个MoE块中的专家数量！请记住，只有前馈网络被替换为MoE块。网络的其他部分，如注意力块，仍然是共享的。

这意味着加载模型时，你需要一个GPU来容纳470亿参数。另一方面，对于给定的token，只有部分专家会激活（Mixtral中为2个）。因此，激活的参数数量要低得多（Mixtral中为120亿）。这使得MoE在本地推理中不太有吸引力（因为需要大量GPU内存），但在生产环境中却非常引人注目，因为你可能会同时收到许多请求。由于每个并行处理的请求中每个token激活的参数较少，MoE应该能够比密集模型处理更多的请求。

关于MoE的第二个误解是，专家会专门化于不同的任务或数据子集。专家通过损失函数进行训练，确保token生成负载在专家之间均匀分布，从而确保所有专家都被使用。门控专家更像是一种负载均衡器，没有证据表明存在高级任务专门化。

由于其令人印象深刻的质量和生产特性，MoE在生态系统中变得非常流行。除了著名的Mixtral 8x7B模型外，还有Snowflake Arctic Instruct、Databricks DBRX、Mixtral 8x22B、DeepSeekMoE和Qwen 1.5 MoE。

要了解更多关于该主题的内容，我们建议阅读以下资源：

- Hugging Face的MoE介绍博客提供了MoE研究的高层次概述以及其工作原理的入门解释。
- Switch Transformers论文深入探讨了构建和训练MoE的许多挑战和设计决策。这是一篇优秀的论文，帮助你理解在使用MoE时面临的设计问题。
- DeepSeekMoE论文介绍了一些MoE的新颖想法，例如将专家分割成更小的专家和始终激活的共享专家。这篇论文是了解该领域前沿研究的好读物。
- Mixtral论文详细介绍了Mixtral $8x7B$ 的训练过程。它没有引入新的架构或训练思想，但它是了解MoE实际训练过程的好读物。考虑到Mixtral是第一个高质量且公开可用的MoE模型，这一点尤其有趣。

# 优化与量化

传统上，优化技术主要出于两个原因被追求：

- 在高负载场景中最大化模型性能，例如由数千用户使用的聊天UI或生成式API。每台服务器

和单位时间内服务更多用户可以最小化成本，并让更多用户享受服务。
- 减少训练时间或训练资源。对于大型模型，适度减少内存使用量或完成训练步骤所需的时间可以显著加速训练，并减少硬件使用量。即使对于小型实验室或个体从业者训练较小模型，更快的训练速度也允许更快的迭代周期和完成更多实验。

此外，过去几年社区对在消费级硬件上运行各种模型（包括LLMs）表现出了极大的兴趣。这有许多原因：在没有API计时器的情况下实验模型，通过查看内部激活来理解模型的工作原理，在计算机上私密地运行任务甚至无需互联网连接，使用自己的数据进行微调而无需花费大量金钱和精力在云服务上，等等。

因此，社区提出了许多优化技术和巧妙的方法。我们已经简要探讨了量化，这是一组旨在减少模型参数精度（对质量影响最小）以减少内存并在消费级硬件上运行模型的技术。我们在本章中提到的另一种广泛使用的技术是Flash Attention，它不仅允许更长的上下文窗口，还减少了所有基于Transformer的模型（包括语言模型和许多其他生成模型，如我们在第4章中看到的）的内存消耗。

另一种有趣的加速技术是推测解码（speculative decoding），也称为辅助生成（assisted generation），适用于回归模型（如LLMs）。该方法使用两个具有相同架构的模型来完成同一任务。其中一个模型小而快，另一个模型大且质量更高。小模型像往常一样生成几个token，在收集到几个token后，它们一起传递给大模型进行确认。大模型在一次通过中验证它们，而不是通过生成循环。只有大模型确认的token才会被使用（其余的会被丢弃），但只要小模型和大模型经常一致，这比单独使用大模型更快。这对于结构化文本（如代码）特别有用。内存消耗会更大，因为我们将使用两个模型而不是一个，但吞吐量会更高。

一种类似的方法是Medusa解码。这是一种微调专门化方法，它在现有模型上添加新的头，以便一次生成多个token的序列。与前面的情况一样，会检查多个候选者，以确保生成的文本与不使用Medusa时相同。

以下是一些关于这一活跃主题的更多学习资源：

- Merve Noyan的这篇文章是量化的温和介绍，并提供了额外的参考资料以深入探讨该主题。
- llama.cpp代码库包含多种量化和优化技术，包括自定义内核，以加速在消费级硬件（包括Windows和Apple Silicon）上的推理。
- 社区成员（如TheBloke，Tom Jobbins）准备并分享了多种LLM的量化、即用版本。这位独立社区成员准备的模型已被下载数百万次。
- Hugging Face关于推测方法的文档讨论了推测解码及其使用方法。
- Joao Gante的这篇博客文章是辅助生成/推测解码的绝佳介绍。

- 推测解码的损失变体也是可能的。在这篇文章中，Vivien Tran-Thien对这一方向进行了精彩的探索。

## 数据

尽管本书的大部分内容都集中在模型及其应用上，但用于训练这些模型的数据是机器学习的一个关键方面。大多数预训练模型都是在大量网络数据上进行训练的。然而，通过过滤这些数据或使用其他大语言模型生成合成数据，较小的模型可以实现与更大模型相当的性能。高质量的数据可以显著提升甚至小型模型的性能，例如微软的Phi-3。FineWeb是一个高质量的开源网络数据过滤数据集，并附有一份技术报告，展示了如何过滤出高质量的token。

尽管合成数据并不新鲜，但Phi为探索完全由大语言模型创建的大规模数据集（数十亿token）打开了大门。Phi的第一个版本是一个13亿参数的模型，使用了60亿token的高质量网络数据和10亿token由GPT-3.5生成的教科书和练习题进行训练。Cosmopedia是一个初步尝试，发布了250亿token的合成数据集，涵盖多个主题，并由Mixtral $8\mathrm{x}7\mathrm{B}$ 创建。

像Argilla的distilabel这样的工具使得创建合成数据和AI反馈变得容易，这些数据可以用于预训练和RLHF。虽然Phi和Cosmopedia专注于用于预训练的大量合成数据，但机器学习模型越来越多地用于创建偏好数据集并扩展模型评估。

除了Cosmopedia的发布博客外，我们建议阅读Eugene Yan关于生成和使用合成数据的博客文章、TinyStories论文以及Phi的原始论文《Textbooks Are All You Need》，以及Phi-1.5和Phi-3的后续报告。

对于其他模态，如图像或音频，我们不仅需要图像和音频数据集，还需要为数据添加字幕或转录的文本数据集。已经发布了包含文本-图像对的大规模数据集，例如LAION-2B（用于训练Stable Diffusion的早期模型）、COYO 700M和DataComp 1B。然而，从开放互联网上抓取数十亿未过滤的图像可能包含不适当或非法内容，因此在这些未过滤数据上训练的模型可能会引发关于版权和合理使用以及安全挑战的开放性问题。为了缓解这些挑战，已经发布了具有开放许可的Creative Commons图像-文本对数据集，例如CommonCanvas。对于音频，开放数据集如Mozilla的Common Voice用于语音，FreeSound和Free Music Archive用于音效，已被广泛采用。虽然也存在通过网络抓取和授权许可的音频数据集，但目前尚未形成类似图像领域COYO或DataComp那样的集中式数据集索引体系。

## 一统天下的模型

使用大语言模型和其他生成模型来满足特定用例的主要方法有三种，按复杂度从高到低排列如

下：

1. 从头开始训练一个新模型。
2. 为你的用例微调一个预训练模型。
3. 使用现有模型进行提示工程或RAG

选择哪种方法取决于你的资源和优先级。由于高昂的成本，从头开始训练模型对大多数公司来说现在都是难以承受的。幸运的是，高质量开源模型的兴起使得通过微调更容易获得高质量的结果。像BloombergGPT（用于金融）、AstroLLaMA（用于天文学）和BioMistral（用于医疗领域）这样的模型已经证明，使用领域特定数据微调一个强大的预训练模型有时比直接使用模型效果更好。

也就是说，随着模型的零样本能力提高，它们的性能可能足以满足许多用例，而无需任何微调。根据项目需求，投入在数据质量和模型迭代上的时间可能更适合投入到最终产品中。例如，当Meta发布Llama 3及其高质量指令版本时，社区很难通过微调获得更好的结果，因为该模型已经非常强大。

另一个重要的考虑因素是如何部署和消费模型。选项包括自托管、使用云提供商的开箱即用解决方案或利用模型训练者提供的API（例如OpenAI或Cohere）。最佳选择取决于你的具体需求。社区越来越多地采用工具（如langchain和llamaindex），这些工具可以轻松在不同解决方案之间切换，避免依赖单一模型提供商。我们建议以允许轻松评估和根据需要交换模型的方式构建你的系统。

# 计算机视觉

计算机视觉是一个拥有悠久历史的广阔领域，早于机器学习技术。它试图从图像中提取有意义的信息，并将其应用于决策或行动，例如引导自动驾驶车辆、检测生产线上的缺陷、计数物体或监控交通。机器学习方法彻底改变了计算机视觉领域，而计算机视觉任务的持续改进又推动了越来越强大的表示学习方法的研究，从而引发了生成图像的爆炸式增长。

传统上，计算机视觉被视为一组不同的任务，因为将这些任务分解为更小的问题比一次性解决视觉理解问题更容易。计算机视觉任务包括以下内容：

**图像分类**

决定一组类别中哪一个最能描述给定图像内容的问题。自2009年开放ImageNet数据集与ImageNet大规模视觉识别挑战赛（ILSVRC）一起发布以来，这一任务的进展显著加快。2012年，由Alex Krizhevsky、Ilya Sutskever和Geoffrey E. Hinton创建的深度神经网络赢得了当年的

挑战赛，表现比第二好的解决方案高出41%，在多年的缓慢进展后，这一表现令人难以置信。这一事件传统上被视为深度学习革命的开始，并彻底改变了机器学习和计算机视觉任务的处理方式。

**目标检测**

在图像中查找特定类型对象或主题的任务（图10-5）。给定输入图像，目标检测模型会生成围绕模型识别的对象的矩形（边界框），并提供关于每个框内对象属于预测类别的概率的置信度分数。

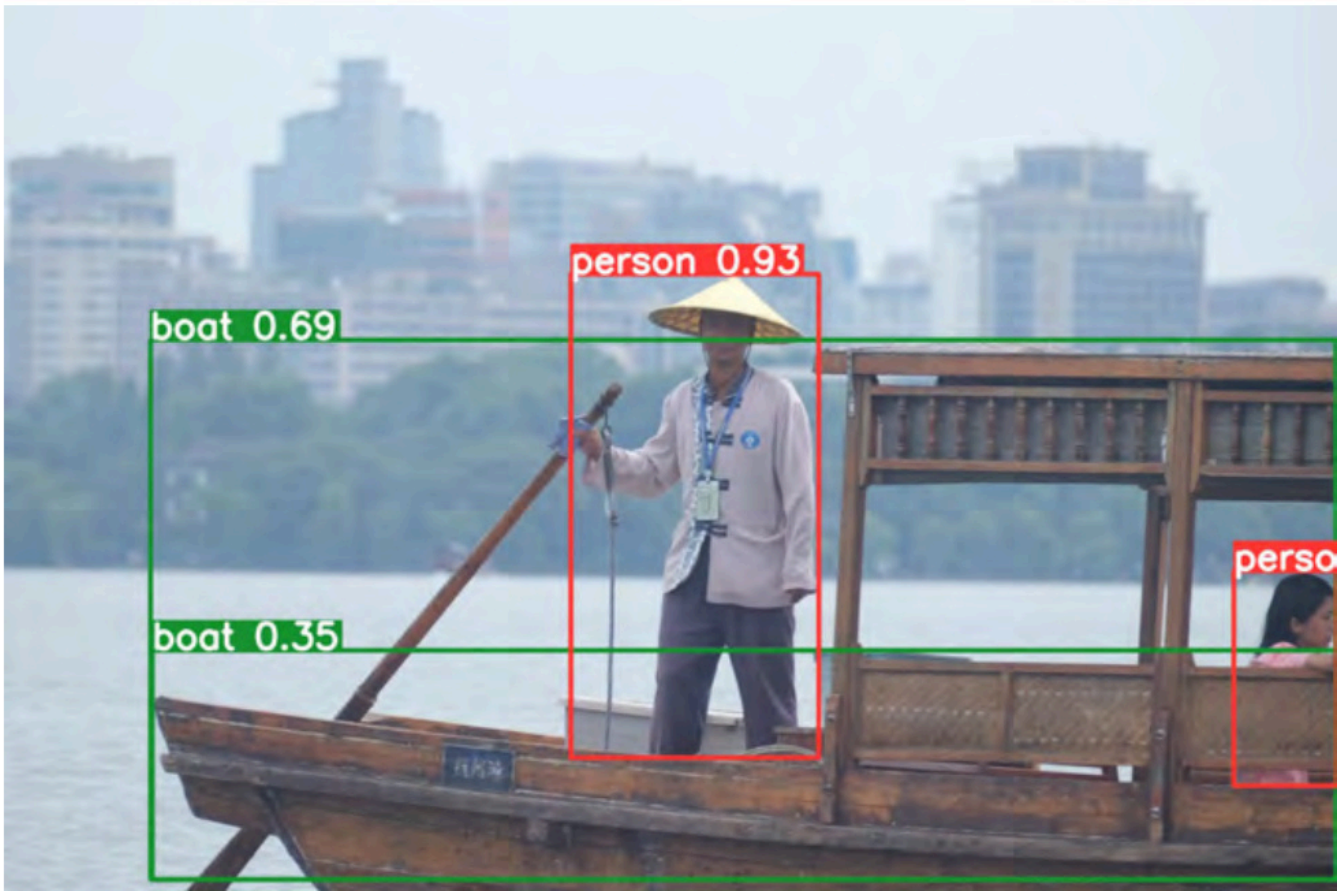



图10-5. YOLOv10模型检测到的一些对象，来自它识别的对象类别集

**分割**

比检测更进一步。该任务试图解决将图像中的每个像素根据一组预定义的类别进行分类的问题。例如，在城市照片上训练的分割模型会预测图像中对应于道路、树木、行人或汽车的像素。当同一类别的多个对象之间没有区别时，这被称为语义分割（所有属于人的像素将被分配相同的标签标识符），或者当模型能够区分属于同一类别的不同对象实例时，这被称为全景分割。

### 深度估计

仅给定一张没有额外信息的图像，估计图像中物体的距离（图10-6）。这被称为单目深度估计，以区别于使用立体输入（两张图像）或其他类型附加数据的系统。单目深度估计在计算机图形学、3D、游戏、摄影和艺术任务中非常有用。

图10-6. 苹果的Depth Pro模型用于从照片中估计深度图

图像分类、检测和分割模型传统上是在一组特定的已知类别上进行训练的。例如，ImageNet挑战赛是在1,000个类别上进行的（尽管原始数据集包含约22,000个类别）。这带来了一个规模问题：要训练理解世界的模型，我们需要使用足够的训练数据来覆盖所有可能的类别及其之间的细微差别。零样本任务指的是模型在没有针对特定类别集进行训练的情况下执行这些任务的能力。例如，CLIP模型是从互联网上下载的图像-文本对进行训练的，因此它很好地理解了通常一起出现的图像和文本特征。正如我们在第3章中看到的，训练好的CLIP模型可以用于对用户提供的一组任意类别中的图像进行分类，而无需在这些类别上进行训练。

随着更大的模型在越来越多的数据上进行训练，它们可以以零样本的方式解决任务，而无需明确针对这些任务进行训练。此外，这些大模型学会了使用丰富且描述性强的表示，使得针对特定任务进行微调变得容易。此外，结合文本和视觉表示的多模态模型（也称为视觉LM）可以回答关于图像数据的自然语言问题，因此它们可以用于各种工作流程，而不是特定任务的专用模型。这引发了一个问题：我们是否正处于另一场计算机视觉革命的边缘，训练在庞大数据集上可能比专注于特定任务更有成效？或者较小的微调模型是否仍将优于通用模型？

## 3D 计算机视觉

虽然传统的计算机视觉主要关注 2D 数据，如图像和视频，但人们对理解、解释和生成 3D 数据

的兴趣日益增长。3D 计算机视觉广泛应用于机器人、增强现实、医疗保健、视频制作、游戏和自动驾驶领域。传统上，3D 数据通过网格（由顶点、边和面组成的集合，用于定义物体的形状）来表示（见图 10-7）。然而，机器学习技术在处理网格时常常遇到困难，这促使人们探索替代表示方法，如神经辐射场（NeRFs）和高斯溅射。

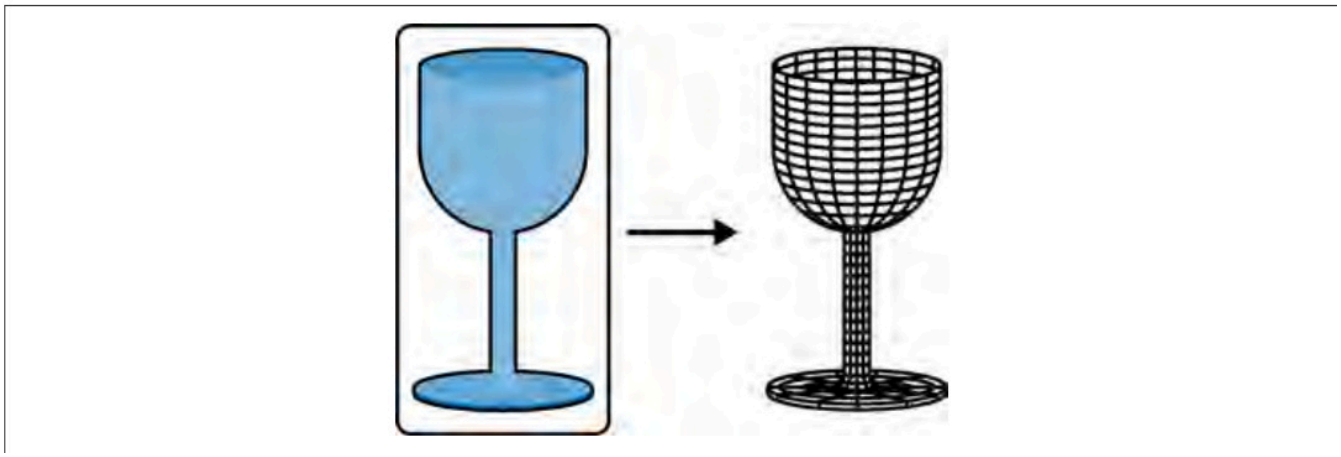


图 10-7. 即使从单张图像也可以生成网格

与自然语言处理（NLP）不同，3D 机器学习生态系统规模较小且高度以研究为导向。因此，许多开源工具都是实验性的，处于早期开发阶段。该领域正在快速发展；例如，第一篇 NeRF 论文发表于 2020 年，而高斯溅射则于 2023 年出现。对于那些有兴趣进一步探索这一领域的人，以下是一些资源：

- Frank Dellaert 在 2020 年、2021 年和 2022 年发表了博客文章，回顾了 NeRF 生态系统中的发展。建议按时间顺序阅读。
- Hugging Face 提供了一门关于 3D 机器学习的免费课程。该课程是对生成式机器学习技术的高层次、实用概述，这些技术可以应用于 3D 渲染管线的不同步骤。

## 视频生成

随着图像生成模型在生成式机器学习中确立了可行的用例，视频自然成为了一个研究前沿。毕竟，视频是一系列快速移动的图像，足以让人类大脑产生它们在运动的错觉。因此，视频生成的一种方法是利用预训练的图像生成模型来生成时间和视觉上连贯的图像序列（见图 10-8）。AnimateDiff 等框架通过推导运动先验来引导像 Stable Diffusion 这样的模型生成时间上连贯的视频。Deforum 等管线则通过相机控制创建动画，同时将帧变化作为美学的一部分。

图 10-8. Genmo 的 Mochi-1 预览是一个文本到视频模型，可以根据文本提示创建短视频

视频到视频技术也是利用生成式模型在视频领域的一种方式。通过利用现有视频并为其提供新风格或新主题，生成式 AI 可以转换视频或将草图转化为动画。RunwayML 的 Gen-1 论文展示了一种高效且性能优异的视频到视频转换方法。该模型目前是该公司的商业产品之一。

然而，原生视频生成技术要实现更高效的新视频创作，仍是发展迅猛的研究前沿领域。训练原生视频生成模型面临的一大挑战在于：难以获取语义表达精准且训练效率高的视频-文本配对数据。

为应对这一挑战，清华大学开发的CogVideo（首个开源文本转视频模型）与Meta AI于2022年发布的Make-A-Video等模型，创新性地采用了将无字幕视频与文本-图像数据集相结合的技术方案。其中Make-A-Video通过以下突破性方法解决了视频-文本配对数据不足的难题：首先从海量图像-文本对中学习视觉-文本理解能力，再将该能力与从无监督非标注视频中习得的运动理解知识相融合。这种创新架构使模型无需显式的视频-文本配对，就能同时掌握物体的外观特征与运动规律，从而同时实现文本生成视频和图像生成视频两大功能。

这种技术在 Stable Diffusion 上的应用使得更高效的文本到视频开源模型得以进一步迭代，例如阿里巴巴的 ModelScope 文本到视频模型。通过 Stable Video Diffusion 模型，视频数据集的规模得以扩大，该模型训练了一个视频潜在扩散模型，并将其与 Stable Diffusion 文本到图像模型配对。Stable Video Diffusion 的图像到视频变体发布了开放权重，并且与 Make-A-Video 和 ModelScope 样本相比，模型质量有所提高。

其他扩展的视频生成方法也已宣布，首先是 OpenAI 的超高质量、逼真且时间上连贯的文本到视频模型 Sora。虽然该模型令人印象深刻，但其技术报告中透露的技术细节很少。在同一领域，Google DeepMind 推出了其 Sora 竞争对手 Veo。快手上线了 Kling，这是第一个公开访问的 Sora 级别模型，而 RunwayML 则在几周后发布了 Gen-3。

在开源和开放权重方面，清华大学的 THUDM 小组发布了 CogVideoX 2B 和 CogVideoX 5B，

这是一系列开放权重模型，可以生成与之前提到的闭源替代品相当的逼真且时间上连贯的文本到视频。还有一个活跃的项目 Open-Sora，旨在提供 Sora 的开源复制，该项目进展迅速。

# 多模态

在前几章中，我们探讨了使用生成式模型处理多种模态，如文本、图像和音频。我们研究的一些模型，如 Stable Diffusion，其输入条件是一种模态（文本），而输出是另一种模态（图像）。然而，输入和输出都是单一模态的模型（这里我们也可以包括文本到语音和语音到文本模型）通常不被社区称为多模态或多模态模型。多模态通常意味着单个模型可以同时处理多种模态的输入或输出。让我们来看看多模态领域的一些最新进展。

CLIP 是 OpenAI 于 2021 年推出的一种模型架构，它基于数百万张图像及其描述性标题进行训练。训练完成后，该模型可以同时接受图像和文本作为输入，并将这些输入编码到相同的语义相关向量空间中。这一特性使得该模型能够执行广泛的零样本任务，如图像分类，以及语义上比较图像与图像或文本与文本。第 3 章介绍了 CLIP，第 5 章展示了 CLIP 文本编码器如何成为 Stable Diffusion 模型的一个组件。

Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation (BLIP) 是 Salesforce 于 2022 年推出的一个框架和模型架构，与 CLIP 一样，它基于图像-文本对进行训练。然而，该模型被训练为将输出进一步解码为文本。无论是文本、图像还是两者的多模态输入，都使得该模型能够执行零样本任务，如图像描述和视觉问答（见图 10-9）。后续工作如 BLIP-2 通过结合冻结的图像编码器和大型语言模型（LLM）进一步优化了这一概念。

图 10-9. BLIP 图像描述

视觉语言模型（VLMs），有时也称为视觉大型语言模型（VLLMs），可以同时接受图像和文本作为输入，并提供文本输出（见图 10-10）。这一领域的开创性工作是 DeepMind 于 2022 年发表的 Flamingo 论文，但该模型并未发布。开源复制品如 IDEFICS 确实存在。然而，从头训练 VLM 已被证明成本高昂。通过冻结（不可训练）的图像编码器进一步微调已有的预训练 LLM 的方法已被证明是高效且性能优异的。BLIP-2 等架构率先采用了这种方法，但它们仍然具有相对狭窄的领域应用（如图像描述、问答等），并且没有保留 LLM 的所有能力。微软研究院于 2023 年推出的 LLaVA 架构允许这种视觉编码器与 LLM 的连接，同时保留其所有推理能力。这种方法在开源社区中引发了技术和模型的爆炸式增长，从轻量高效的模型到与商业模型竞争的 SOTA VLM。要跟上这一领域的发展，可以关注 Open VLM 排行榜或 Hugging Face 上的图像-文本到文本趋势模型。要获得实践经验，可以从 transformers 上的 VLM 推理开始。OpenAI 的

GPT-4 或 Anthropic 的 Claude 等商业模型也具有 VLM 视觉组件。

---

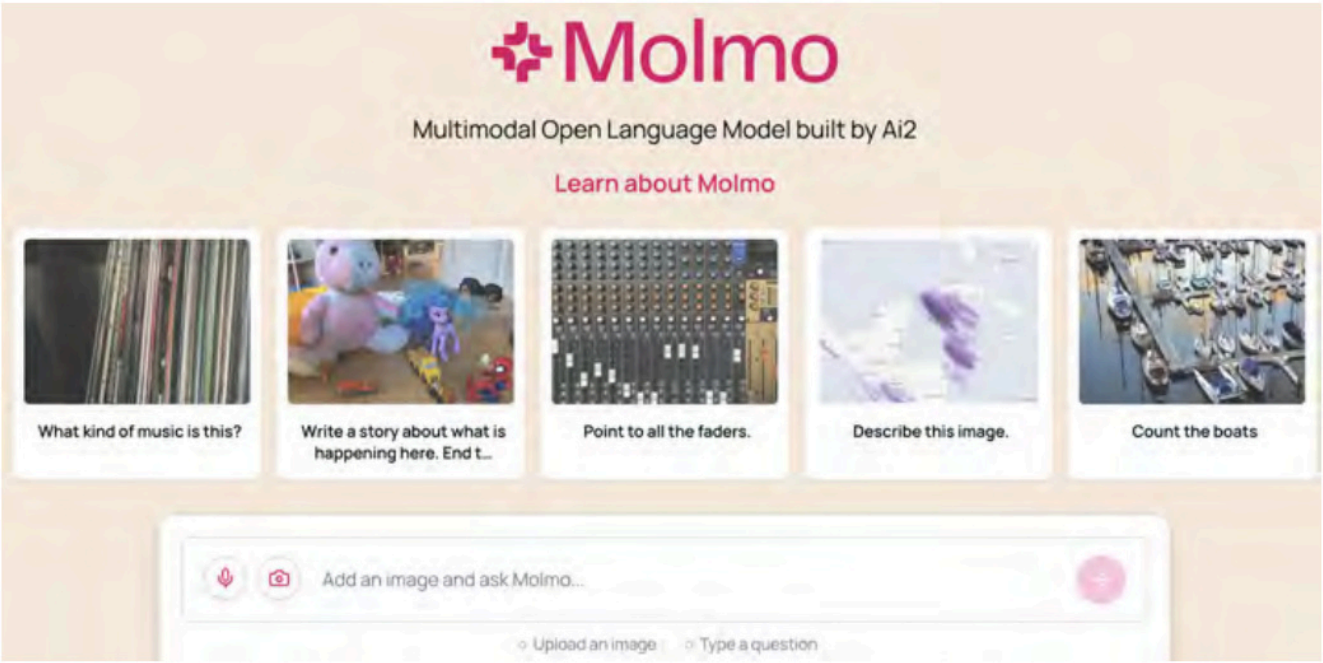



图 10-10. VLM 界面

使用与 VLM 相同的逻辑，可以通过利用冻结的编码器实现其他模态。例如，Gazelle 是一个联合语音语言模型，它利用了预训练的 LLM（Mistral 7B）和冻结的音频编码器（Wav2Vec2）。该模型能够直接从音频输入中处理和推理，从而获得了强大的能力。

多模态输出模型是下一个前沿领域。这一领域的研究和商业模型正在不断进步，我们预计很快将迎来一波新的开源多模态输出模型。在这一领域的一些基础研究中，Meta AI 的 Chameleon 架构可以接收文本和图像输入，并生成文本和图像输出。它在图像指令跟随、图像编辑、图像描述、问答等任务上具有零样本能力。Allen Institute for AI 提出的 Unified-IO 和 Unified-IO 2 等架构具有更广泛的多模态输入/输出能力，但它们尚未大规模应用。至于商业模型，OpenAI 在 2024 年 5 月发布了一款多模态 GPT-4o 模型，可以接收图像、文本和音频，并输出相同模态的图像、文本和音频。

# 社区

正如你可能在本章和整本书中已经意识到的，机器学习生态系统的节奏非常快。跟上最新研究和发展的最佳方式是成为社区的一部分。有很多方法可以做到这一点，比如加入 Discord 服务器或 Reddit 社区，与他人分享你的工作，阅读论文，并在 X（前身为 Twitter）上关注研究人员和实践者。

令人惊讶的是，我们已经从所有研究都在传统实验室中进行的世界，转向了在去中心化设置中进行的非常有影响力的研究。EleutherAI、Nous Research、BigCode 和 LAION 都是后者的例子，参与他们的工作可以像加入他们的 Discord 服务器一样简单。许多这些社区还举办论文阅读会、精彩的异步聊天讨论和黑客马拉松。

其中一些社区是由个人社区贡献者发起的，他们通过摆弄开源模型开始了他们的旅程，以解决对他们来说重要的问题。尽管从头开始训练大模型的成本很高，但对于任何对该领域感兴趣的人来说，仍然存在大量的研究和开发机会。现在比以往任何时候都更适合开始。

# 附录 A. 开源工具

如果没有开源，这本书就不可能完成。我们讨论的大多数主题和大部分机器学习研究都依赖于开源贡献——更不用说我们使用的生产工具链，如 Jupyter Notebook、Quarto、nbdev 等。

在本附录中，我们将探索各种适用于机器学习从业者的开源工具。其中一些我们在书中已经使用过，而另一些则值得了解。通过熟悉这些工具，你将能够很好地扩展你刚刚学到的应用和技术。

## Hugging Face 技术栈

在本书中，你已经熟悉了 Hugging Face 技术栈的核心库。这是我们使用的两个主要库：

**transformers**

用于训练和运行基于 Transformer 的模型的主要库，支持多种模态。它提供了多个抽象层次，从高级的 pipeline 和 Trainer 到支持运行你自己的 PyTorch 训练循环。

**diffusers**

与 transformers 类似，diffusers 库允许运行预训练的基于扩散的模型。尽管它主要以图像生成能力闻名，但该库也支持音频、视频和 3D。

这两个库都采用了以可用性、简单性和可定制性为优先的设计理念。这对最终用户意味着什么？首先，这两个库都旨在提供跨模型的一致性规范。无论你使用的是 Llama 还是 Gemma，在它们之间切换理想情况下应该只需要一行代码的更改。虽然这两个库都有许多用于快速推理的功能，但默认情况下，模型总是以最高精度和最低优化加载。这确保了跨平台的可用性，并避免了复杂的安装，但也意味着除非配置了优化，否则模型在开箱即用时速度会较慢。

另外两个常用的 Hugging Face 库是：

**accelerate**

允许在分布式设置中运行 PyTorch 代码，无论是训练还是推理。无论你是想在 GPU、多个 GPU、带有 CPU 卸载的 GPU 还是完全在 CPU 上运行模型，accelerate 都抽象了所有复杂性。它被 transformers 和 diffusers 在底层使用，因此大多数用户不需要学习太多关于 accelerate API 的知识。

**peft**

该库支持参数高效微调技术，以较低的计算和存储成本微调模型。它与 transformers 和 diffusers 集成良好。尽管本书主要探讨了 LoRA，但还有许多其他方法，如 p-tuning、prefix tuning、IA3、OFT 和 DoRA。

# 数据

近年来，用于数据集处理、标注和生成的开源生态系统显著增长。以下是一些正在构建的用于处理数据的工具的简要列表：

**datasets**

本书大量依赖 datasets，这是一个用于访问、共享和处理多种模态的开放数据集的流行库。与 transformers 和 diffusers 一样，datasets 提供了一致的 API，允许用户轻松交换给定模态的数据集。

**argilla**

Argilla 是一个用于构建高质量数据集的工具。它提供了一个简单的 UI，人类可以在其中对数据进行评分，这对于比较模型生成（对 RLHF 很重要）、创建经典 NLP 任务的数据集（例如实体识别）或创建评估数据集等任务非常有用。

**distilabel**

随着合成数据生成的兴起，像 distilabel 这样的新工具出现了。它允许创建生成合成数据的管道。

# 封装工具

随着生态系统的发展，围绕 transformers 构建了各种社区和研究工具：

**axolotl**

该工具简化了模型微调；你只需要创建一个简单的配置文件来设置你的微调任务。它支持常见的数据集格式和模型架构。

**unsloth**

Unsloth 旨在在 transformers 之上提供极快的 LLM 微调，使用一个包含优化内核的 FastLanguageModel 类。

**sentence-transformers**

如第 2 章所述，Transformer 模型也可以用于计算整个句子、段落或文档的嵌入。sentence-transformers 库提供了简单的 API 来计算预训练模型的嵌入或微调你自己的模型。

**trl**

随着 RLHF 的兴起，trl 提供了一个简单的 API 用于微调和对齐 Transformer 和扩散模型。第 6 章展示了如何进行监督微调（SFT），但 trl 包含许多其他方法，如奖励建模和 DPO。

# 本地推理

开源模型的一个主要优势是，你可以在自己的硬件上本地运行许多模型，提供隐私、可定制性和本地集成（例如，使用代码模型作为本地 IDE 扩展）等好处。根据你的用例，可以使用各种工具：

**llama.cpp**

允许在各种硬件上进行 LLM 推理。它支持多种量化技术（从 1.5 位到 8 位），并得到了社区的广泛采用。它通常用于与本地 LLM 聊天或为另一个本地服务设置本地端点。

**Transformers.js**

允许直接在浏览器中运行模型，无需服务器。这对于轻松部署低延迟且无推理成本的服务，或用于隐私优先的用例（如实时编写通话记录或字幕）非常有用。

## 部署工具

虽然为单个查询运行推理可能很简单，但在生产中部署 LLM 则更为复杂。有许多工具可用于此，包括以下两个流行的工具：

**vLLM**

一个用于 LLM 服务的简单库，灵活且与流行模型集成良好。

**TGI**

一个用于 LLM 部署的生产就绪工具包。

其他选项包括 lmdeploy 和 NVIDIA 的 TensorRT-LLM。有这么多选择，你可能会想知道该选择哪一个。我们的建议是探索它们，找出最适合你用例的工具。所有这些工具都在积极开发中，具有不同的模型覆盖范围、社区采用率、可扩展性、与云服务和自托管环境的集成等。

# 附录 B. LLM 内存需求

模型有各种大小！例如，Llama 3.1 发布了 8B、70B 和 405B 的变体。要加载和使用 LLM，你需要足够的内存来存储模型。参数数量及其精度等因素会影响 LLM 的内存需求。

如果你没有足够的内存，可以尝试以下选项：

- 降低你使用的模型的精度。与其使用 float16，你可以使用 int8。
- 使用较小的模型。有许多高质量的小模型。
- 卸载你不使用的模型部分。这可以通过 CPU RAM 卸载来完成，这是一种常见的减少模型内存需求的技术，但会降低推理速度。如果没有足够的内存怎么办？我们可以将剩余的模型部分存储在磁盘上，并在需要时加载它们。幸运的是，accelerate 库通过 device_map="auto" 自动处理了这一点，它会根据需要自动卸载模型部分。

## 推理内存需求

你可以大致估计内存需求如下：

GPU 内存需求 = 参数数量 × 每个参数的字节数

每个参数的字节数取决于使用的精度。在不深入细节的情况下，表 B-1 显示了使用不同精度级别（float32、float16、int8、int4 和 int2）加载 2B、8B、70B 和 405B 模型所需的内存。

表 B-1. 模型和精度级别的推理内存需求

| Model | float32 | float16 | int8 | int4 | int2 |
|---|---|---|---|---|---|
| 2B | 8 GB | 4 GB | 2 GB | 1 GB | 512 MB |
| 8B | 32 GB | 16 GB | 8 GB | 4 GB | 2 GB |
| 70B | 280 GB | 140 GB | 70 GB | 35 GB | 17.5 GB |
| 405B | 1.62 TB | 810 GB | 405 GB | 202.5 GB | 101.25 GB |

作为参考，H100 有 $80\mathrm{GB}$ 的内存，因此加载 Llama 3.1 405B 至少需要一个完整节点（8 个 H100）来以 8 位整数加载模型。这是一个粗略的估计。你还需要考虑输入和输出张量所需的内存以及中间计算所需的内存。例如，长序列比短序列需要更多的内存，尤其是当我们处理超过 100,000 个 token 时。

在撰写本文时，我们可以将模型量化为 int8，而性能损失最小。低于每个参数 8 位的技术会带来性能下降，并且是一个活跃的研究领域。

# 训练内存需求

计算训练需求可能会变得复杂，因为它们取决于模型和训练脚本的实现细节。内存需求也会根据批量大小、数据集样本中的标记数量、训练技术（例如，全微调与PEFT）以及训练并行设置而显著变化。

训练内存需求的细节超出了本书的范围。然而，我们可以提供大语言模型（LLM）训练内存需求的粗略估计。表B-2展示了微调Llama的粗略GPU需求。

表B-2. 模型和训练技术的训练内存需求

| Model | Full fine-tuning | LoRA | QLoRA |
|---|---|---|---|
| 8B | 60 GB | 16 GB | 6B GB |
| 70B | 500 GB | 160 GB | 48 GB |
| 405B | 3.25 TB | 950 GB | 250 GB |

## 延伸阅读

如果你对LLM的内存需求感兴趣，可以查看以下资源：

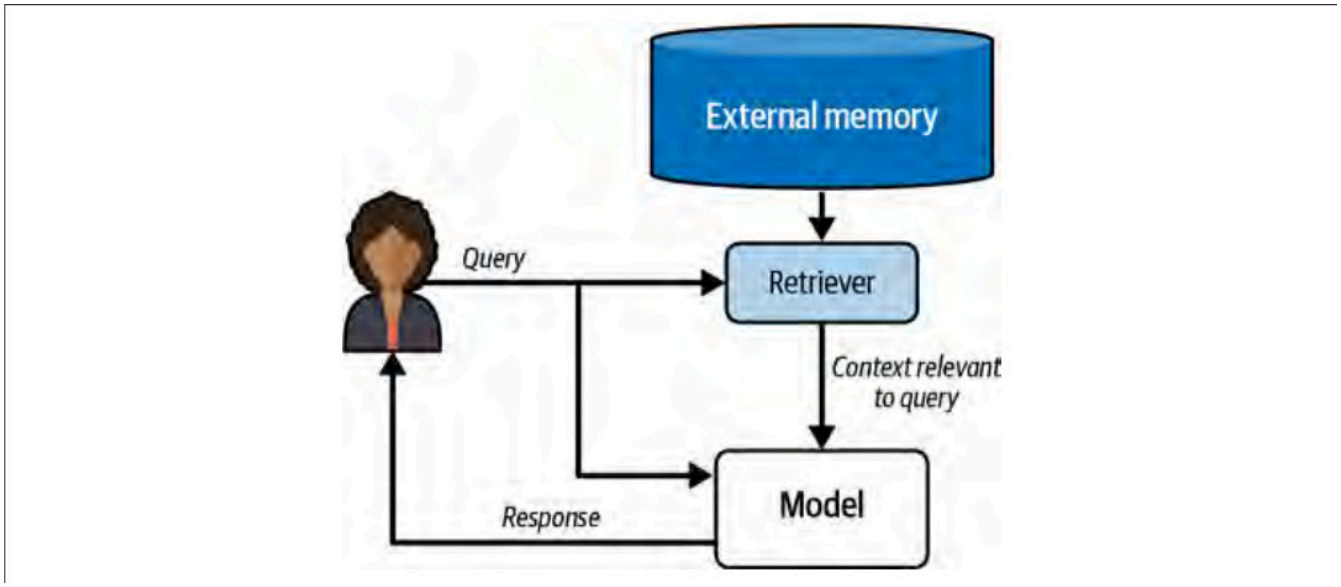
“Transformer Math 101” by EleutherAI，详细解释了不同设置下训练模型的内存需求。
“Breaking Down GPU VRAM Consumption” 是一篇简短的博客文章，解释了消耗GPU内存的组件。

# 附录C. 端到端检索增强生成

LLM的一个流行应用是基于输入提示和外部检索信息生成内容。在本附录中，我们将演示如何构建一个管道，利用预训练的LLM和预训练的句子转换器，基于用户输入和一组文档生成内容。我们在本书中已经探讨了构建这一管道的各个模块。第2章讨论了使用LLM进行文本生成以及如何使用句子转换器对文本进行编码。第6章还包含了一个项目，我们构建了一个最小的RAG管道。

让我们讨论一下RAG系统的组件（如图C-1所示）：

1. 用户输入一个问题。
2. 管道检索与问题最相似的文档。
3. 管道将问题和检索到的文档传递给LLM。
4. 管道生成响应。




图C-1. 简化的RAG管道

# 数据处理

与任何机器学习项目一样，第一步是加载和处理数据。我们将通过专注于一个单一主题来简化流程。假设我们希望模型生成与《欧盟人工智能法案》相关的内容，这不太可能成为LLM训练数据的一部分，因为我们将使用的模型在《人工智能法案》工作开始之前就已经训练好了。首先，我们将加载文档：

```
import urllib.request

# Define the file name and URL
file_name = "The-AI-Act.pdf"
url = "https://artificialintelligenceact.eu/wp-content/uploads/2021/08/The-AI-Act.pdf"

# Download the file
urllib.request.urlretrieve(url, file_name)
print(f"{file_name} downloaded successfully.")

The-AI-Act.pdf downloaded successfully.
```

文档可能太长，无法一次性处理，因此我们将其拆分为较小的块，并分别嵌入每个块。每个块将是一个单独的文档，我们将与用户输入进行比较。为了简化，我们将使用langchain库中的一些预处理工具，该库提供了创建RAG系统的实用函数。例如，它有一个方便的PyPDFLoader类，

可以从PDF中提取文本并处理分块。

首先，安装必要的依赖项：

```
!pip install langchain_community pypdf langchain-text-splitters
```

现在，让我们使用PyPDFLoader加载并预处理文档：

```
from langchain_community.document_loaders import PyPDFLoader
loader = PyPDFLoader(file_name)
docs = loader.load()
print(len(docs))

108
```

PyPDFLoader将PDF按页拆分为文档，在这种情况下生成了108个文档。我们将它们拆分为更小的块。langchain库提供了帮助处理不同类型文本拆分的类。我们将使用RecursiveCharacterTextSplitter，它有两个关键参数：

chunk_size

每个块中的字符数。通常，最好将其与嵌入模型可以处理的最大标记数联系起来，这对于大多数句子转换器来说较低。否则，您可能会面临文档被截断的风险。

chunk_overlap

每个块与前一个块重叠的字符数。这有助于避免在句子中间拆分。我们将其任意设置为100个字符（我们选择的块大小的五分之一）。

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)
chunks = text_splitter.split_documents(docs)
print(len(chunks))

854
```

让我们将文本块保存到一个数组中：

```
chunked_text = [chunk.page_content for chunk in chunks]
```

这是其中一个块的样子：

```
chunked_text[404]

('user or for own use on the Union market for its intended '
'purpose; \n'
'(12) ‘intended purpose’ means the use for which an AI system is '
'intended by the provider, \n'
'including the specific context and conditions of use, as '
'specified in the information \n'
'supplied by the provider in the instructions for use, promotional '
'or sales materials \n'
'and statements, as well as in the technical documentation; \n'
'(13) ‘reasonably foreseeable misuse’ means the use of an AI system '
'in a way tha t is not in')
```

# 文档嵌入

现在我们有了文档（我们的块），我们需要创建它们的嵌入。我们将使用一个句子转换器模型作为检索器，它就像一个搜索引擎，找到与给定问题最相关的片段。这个过程依赖于计算用户查询的嵌入与文档集合中嵌入的相似性。为了预计算所有文档的嵌入，我们将使用一个预训练的句子转换器模型，使用第52页“练习”中的示例。以下代码片段加载了一个预训练的句子转换器模型BAAI/bge-small-en-v1.5，并使用它编码两个句子：

```
from sentence_transformers import SentenceTransformer, util
sentences = ["I'm happy", "I'm full of happiness"]
model = SentenceTransformer("BAAI/bge-small-en-v1.5")
# 计算两个句子的嵌入
embedding_1 = model.encode(sentences[0], convert_to_tensor=True)
embedding_2 = model.encode(sentences[1], convert_to_tensor=True)
```

句子转换器返回整个句子的单个嵌入。尽管转换器模型通常为每个标记输出一个嵌入，但句子转

换器被训练为将标记嵌入池化为一个句子嵌入，以捕捉文本的语义含义：

```
embedding_1.shape
torch.Size([384])
```

然后，您可以基于余弦相似性比较文档：

```
util.pytorch_cos_sim(embedding_1, embedding_2)
tensor([[0.8367]], device='cuda:0')
```

余弦相似性，正如我们在第3章中看到的，只是两个嵌入向量的点积：

```
embedding_1 @ embedding_2
tensor(0.8367, device='cuda:0')
```

或者，也可以使用：

```
import torch
torch.dot(embedding_1, embedding_2)
tensor(0.8367, device='cuda:0')
```

现在我们知道如何嵌入一个句子，让我们嵌入所有文档：

```
chunk_embeddings = model.encode(chunked_text, convert_to_tensor=True)
```

这将为每个块返回一个384维的嵌入向量：

```
chunk_embeddings.shape
torch.Size([854, 384])
```

# 检索

有了嵌入的文档，我们可以检索与给定问题最相关的文档。我们将使用与之前相同的方法来计算问题与每个文档之间的余弦相似性。幸运的是，相似性计算不需要迭代：可以通过使用内置的

PyTorch矩阵乘法原语高效地执行：

```
def search_documents(query, top_k=5):
    # 将查询编码为向量
    query_embedding = model.encode(query, convert_to_tensor=True)
    # 计算查询与所有文档块之间的余弦相似性
    similarities = util.pytorch_cos_sim(query_embedding, chunk_embeddings)
    # 获取最相似的前k个块
    top_k_indices = similarities[0].topk(top_k).indices
    # 检索相应的文档块
    results = [chunked_text[i] for i in top_k_indices]
    return results
```

让我们尝试一个例子。我们将在书中截断输出以保持简短，但你可以在本地环境中运行代码以查看完整输出：

```
search_documents("What are prohibited ai practices?", top_k=2)

('TITLE II \n'
'PROHIBITED ARTIFICIAL INTELLIGENCE PRACTICES \n'
'Article 5 \n'
'1. The following artificial intelligence practices shall be '
'prohibited: \n'
'(a) the placing on the market, putting into service o')
('low or minimal risk. The list of prohibited practices in Title II '
'comprises all those AI systems \n'
'whose use is considered unacceptable as contravening Unio n '
'values, for instance by violating \n'
'fundame')
```

模型能够正确地从输入问题中检索相关信息。

# 生成

下一步是基于问题和检索到的文档生成响应。让我们使用我们的老朋友，SmolLM的指令版本。[2] 随意尝试其他模型：

```
from transformers import pipeline

from genaibook.core import get_device

device = get_device()
generator = pipeline(
    "text-generation", model="HuggingFaceTB/SmolLM-135M-Instruct", device=device
)
```

我们将使用带有聊天模板的指令模型。正如第6章所讨论的，transformers有一些工具可以格式化提示以满足模型的期望。在RAG的情况下，我们希望将检索到的文档添加到提示中：

```
def generate_answer(query):
    # Retrieve relevant chunks
    context_chunks = search_documents(query, top_k=2)

    # Combine the chunks into a single context string
    context = "\n".join(context_chunks)

    # Generate a response using the context
    prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"

    # Define the context to be passed to the model
    system_prompt = (
        "You are a friendly assistant that answers questions about the AI Act. "
        "If the user is not making a question, you can ask for clarification"
    )
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": prompt},
    ]

    response = generator(messages, max_new_tokens=300)
    return response[0]["generated_text"][2]["content"]
```

让我们尝试一个例子：

```
answer = generate_answer("What are prohibited ai practices in the EU act?")
print(answer)

('The EU Act prohibits the use of artificial intelligence practices '
'that are harmful to individuals, such as:\n'
'\n'
'* The placing on the market, putting into service or use of an A I '
'system that is subliminal, that is, it is not intended to be used '
'for any purpose other than to deceive or manipulate individuals.\n'
'* The use of A I systems that are designed to deceive or '
'manipulate individuals, such as those used in advertising, '
'marketing, or customer service.\n')
```

模型根据输入问题生成响应，并正确检索信息。我们使用了一个非常小的生成模型，因此扩展到更大的模型可以帮助我们获得更高质量的生成，并增加上下文长度，从而增加我们可以传递给上下文的检索文档数量。

# 生产级RAG

我们展示的代码是RAG系统的一个简单示例。在生产级系统中，您需要考虑几个额外的因素：

## 分块

真实数据的一个挑战是文档可能非常长。找到合适的分块大小是一个设计决策，取决于数据和模型：太小的片段会截断想法，而太大的片段会稀释它们。您可以在“5级文本分割”中了解更多关于分割的信息。

## 更小的嵌入

大嵌入可能会占用大量内存。目前正在进行积极的研究，以在保持质量的同时减小嵌入的大小。如果您对这些主题感兴趣，我们建议您阅读关于Matryoshhka嵌入模型和嵌入量化的内容。

## 重新排序

检索步骤在RAG系统中至关重要。我们的检索模型速度快，对于比较数千或数百万个文档至关重要，但它们不一定是最准确的。一旦我们检索到top_k文档，我们可以使用更慢但更高质量的模型对它们进行重新排序。您可以在sentence_transformers文档或“深入探讨交叉编码器和重新

排序”中了解更多关于重新排序的信息。

## 嵌入模型评估

有几十种句子转换器模型。为了选择最适合您用例的模型，我们建议查看大规模文本嵌入基准测试，其中包含模型大小、嵌入维度以及跨数十个任务的质量等信息。请注意，对于检索任务，我们通常希望模型非常小且快速，因此这应该是您决策中的关键因素。

## 生产组件

在实际使用中，您可能希望集成其他组件，如查询重写、个人身份信息（PII）脱敏、缓存和输入防护栏，以防止您的模型被不当使用。Chip Huyen有一篇优秀的文章“构建生成式AI平台”，可以了解更多关于这方面的内容。

最后，有许多开源工具可用于构建RAG系统。以下是一些推荐：

- **ColBERT**
  一个快速且准确的基于BERT的检索模型
- **RAGatouille**
  一个用于使用和训练检索模型的系统
- **开源向量数据库**，如Milvus、Weaviate和Qdrant，当您需要处理大规模数据集时可能会很有用。深入探讨向量数据库超出了本书的范围，但它们也是一个快速发展的领域。您可以在2023年的“选择向量数据库”博客文章中看到比较。

# 索引

A

绝对值最大量化 (absmax quantization), 222
加速库 (accelerate library), 242, 370, 373
准确率 (accuracy), 192
激活函数 (activation functions), 70
适配器 (adapters), 213, 216-220, 239, 247, 253-257
AG News 数据集, 187, 198
AlexNet, 101
阿里巴巴 ModelScope 文本到视频模型, 364

艾伦人工智能研究所 (Allen Institute for AI), 367
Alpaca, 216
振幅 (amplitude), 297 (另见音频生成)
AnimateDiff 框架, 363
“带注释的扩散模型”博客文章, 130
Anthropic Claude, 352
Argilla, 358, 370
自动语音识别 (ASR), 291, 307-323
基于编码器的技术, 308-312
编码器-解码器技术, 312-315
评估, 318-323
生产, 315-318
辅助生成 (assisted generation), 357
AstroLLaMA, 359
Attend-and-Excite 图像编辑, 268
音频分类 (audio classification), 292
音频扩散 (Audio Diffusion), 336-338
音频生成 (audio generation), 291-343
端到端对话系统, 342
系统评估, 340
未来, 341
高质量, 341
频谱图, 298-307
语音到文本 (见 ASR)
文本到音频, 324, 332-340
音频扩散和 Riffusion, 336-338
AudioGen 和 MusicGen, 335-336
AudioLM 和 MusicLM, 332-333
Dance Diffusion, 339
文本到语音, 324-331
Bark, 329-331
SpeechT5, 324-329
统一建模, 341
音频翻译 (audio translation), 292
AudioGen, 335-336

AudioLM, 332-333
自编码器 (AutoEncoders), 57-93
数据准备, 57-60
解码器, 64-65
定义, 55
编码器, 61-63
目标, 106
潜在空间, 70-77
维度, 70-74
可视化, 74-77
训练, 65-69
变分自编码器, 78-93
编码器和解码器, 79-80
生成建模, 93
从编码器分布中采样, 80-83
训练, 83-93
自动语音识别 (见 ASR)
自回归模型 (autoregressive models)
decode() 方法, 21
定义, 17
generate() 方法, 20
model() 方法, 20
平均池化 (average pooling), 137
axolotl, 371

B
BAAI/bge-small-en-v1.5 句子转换器, 380
bad_words_ids 参数, 23
Bark, 329-331
BART (双向自回归变换器), 37
基础模型 (base models), 32 (另见扩散模型; 语言模型)
评估, 230-233
微调, 212, 219
冻结, 195, 213
选择, 190, 204-208
批量归一化 (batch normalization), 61, 141

BBH 数据集, 206
束搜索 (beam search), 21, 23
BeautifulSoup, 241
BERT (双向编码器表示变换器), 38, 42, 190, 195
bfloat16 (BF16; 脑浮点) 类型, 221-222
偏差 (bias)
数据集, 9
变换器模型, 46
双向自回归变换器 (BART), 37
双向编码器表示变换器 (BERT), 38, 42, 190, 195
BigCode, 368
BigCode 模型排行榜, 208
BioMistral, 359
BLEU, 231
BLIP (语言-图像预训练引导的统一视觉-语言理解和生成), 104, 242, 365
Bloom, 39
BloombergGPT, 359
脑浮点 (bfloat16; BF16) 类型, 221-222
“构建生成式 AI 平台” (Huyen), 384

C
CapPa, 104

灾难性遗忘 (catastrophic forgetting), 247
因果语言模型 (见自回归模型)
CelebA-HQ 数据集, 112
CelebFaces Attributes 数据集, 93, 108
字符错误率 (CER), 319-322
无分类器引导 (CFG), 172-173
Chameleon, 367
字符级分词 (character-level tokenization), 15
聊天模板 (chat templates), 228-230
ChatGLM2, 216
ChatGPT, 37
分块 (chunking), 383
对比语言-音频预训练 (CLAP), 95, 333, 340

无分类器引导 (CFG), 172-173
Claude, 37, 39
对比语言-图像预训练 (CLIP), 93-105
替代方案, 104
对比损失, 94-95
图像变体, 283
多模态, 365
概述, 107
步骤, 95-102
用例, 103
零样本分类, 101-103
流程, 102
使用, 101-102
零样本任务, 362
CLIP 引导方法, 104
卷积神经网络 (CNNs), 119 (另见 UNets)
概述, 9
语音到文本, 308
迁移学习, 41
视觉变换器 (ViTs), 47
CoCa, 104
编解码器 (codecs), 330
CogVideo, 364
CogVideoX, 171, 365
Cohere Command R+, 208
ColBERT, 384
“冷扩散” (Bansal 等), 127
数据整理器 (collators), 209
“常见扩散噪声计划和采样步骤的缺陷” (Lin 等), 134
Common Voice 数据集, 293, 320, 359
CommonCanvas 数据集, 178, 359
CommonCrawl 数据集, 176
compel 库, 268-270
压缩 (compression), 55
计算机视觉 (Computer Vision), 360-362

冻结层, 195
ILSVRC 基准, 101
3D, 362
条件扩散模型 (conditional diffusion models), 149-158
创建, 152
数据准备, 150-152
采样, 157-158
训练, 153-156
条件生成对抗网络 (conditional GANs), 9
条件 UNet 架构, 167
条件信号, SDXL, 169-170
混淆矩阵 (confusion matrices), 201-202
连接时序分类 (CTC), 309
同意问题 (consent issues), 9, 341
宪法 AI (Constitutional AI), 352
上下文窗口 (context window), 45, 353, 357
上下文嵌入 (contextual embeddings), 35
连续时间方法 (continuous time approach), 129
对比语言-音频预训练 (CLAP), 95, 333, 340
对比语言-图像预训练 (见 CLIP)
对比搜索 (contrastive search), 28
ControlNet, 257, 279-282
controlnet_aux 库, 281
ControLoras, 257
卷积层 (convolutional layers), 61
卷积神经网络 (见 CNNs; UNets)
余弦相似度 (cosine similarity), 95, 100, 380
Cosmopedia 数据集, 358
CosXL Edit, 278
COYO 700M 数据集, 178, 358
CPU RAM 卸载 (CPU RAM offloading), 373
裁剪坐标, SDXL, 169
交叉注意力机制 (cross-attention mechanism), 37
“通过自然语言众包指令的跨任务泛化” (Mishra 等), 215
连接时序分类 (CTC), 309

D
DALL·E, 94, 107, 111, 177

Dance Diffusion, 339
Data Filtering Networks, 105
数据表示（参见 representations）
Databricks DBRX, 356
DataComp 1B 数据集, 178, 358
DataLoader 加载工具, 60, 117
datasets 库, 117, 145, 187, 294
DCGANs（深度卷积 GANs）, 9
DDIM 逆过程, 274-275
DDPM 逆过程, 275
ddpm-celebahq-256, 112
DDPO（去噪扩散策略优化）, 350
分贝, 297（参见音频生成）
decode() 方法, 16, 21
解码器架构和模型序列分类, 189 变压器模型, 44
深度卷积 GANs（DCGANs）, 9
“深入探讨交叉编码器和重排序”, 383
深度融合, 313
《使用 fastai 和 PyTorch 进行深度学习的程序员指南》（Howard 和 Gugger）, 61
深度神经网络, 360
DeepSeekMoE, 356
Deforum 管道, 363
去噪扩散策略优化（DDPO）, 350
深度估计, 361
DETR, 48
diffusers 库, 369 添加噪声, 118, 132 定义, 4 微调模型, 242 图像到图像管道, 263 LoRA 训练, 254 噪声调度, 129 采样, 123 SOTA 扩散模型, 4-6 训练模型, 112, 115 UNets, 119, 141
扩散模型, 111-146 替代架构, 142-143 带注释的采样循环, 173-176 无分类器引导, 172-173 与 GANs 和 VAEs 的比较, 112 条件扩散模型, 149-158

准备数据, 15 采样, 157-158 训练, 153-156 Flux, 171 交互式 ML 演示, 179-180 迭代优化, 112-115 潜在扩散, 159 噪声调度, 126-137 添加噪声, 128-130 输入分辨率和缩放的影响, 135-137 添加噪声的原因, 126-127 基础数学, 130-135 Stable Diffusion, 160-171 无分类器引导,

172-173 SDXL, 169-170 Stable Diffusion 3, 171 文本编码器, 161-164 UNets, 167-169 VAE, 164-167 文本到音频, 336-340 音频扩散和 Riffusion, 336-338 Dance Diffusion, 339 训练, 115-126, 145 添加噪声, 118 评估, 124-126 加载数据, 116-117 采样, 113, 123 步骤, 121-123 UNets, 119-120 UNets, 119-120, 137-141 构建, 138-140 改进, 141 可视化不同目标, 143 直接偏好优化（DPO）, 350 离散时间方法, 129 判别器损失, 164 Distil Whisper, 315 distilabel 工具, 358, 370 DistilBERT, 43, 190-192, 194 DoRA, 218 DPO（直接偏好优化）, 350 DreamBooth, 239, 248-253, 257 推理, 252 准备数据集, 250 先验保留, 250 训练模型, 251-252 dropout, 141

动态阈值方法, 173

E

8 位量化, 222
EleutherAI, 160, 367, 375
EMA（指数移动平均）, 170
嵌入, 34, 42 量化, 383 更小的嵌入, 383
EnCodec, 330
编码器架构和模型序列分类, 189 语音到文本, 308-312 变压器模型, 37-39, 43
编码器隐藏状态, 162
编码器-解码器架构和模型序列分类, 189 语音到文本, 312-315 变压器模型, 35, 44
epsilon (eps) 目标, 143-145
esa-hubble 数据集, 241
eval 模式, 63
EvalPrediction 类, 194
指数移动平均（EMA）, 170
抽取式问答, 43

F

F1 分数, 193
FAD（Fréchet 音频距离）, 340
合理使用, 9, 178
Falcon, 39, 208
Fashion MNIST 数据集创建类条件模型, 152-153 定义, 149 准备数据, 150-152
前馈神经网络, 34
少样本泛化, 30-32
少样本学习, 189

FID（Fréchet 起始距离）分数, 124-125
“微调语言模型是零样本学习者”（Wei 等）, 215
FineWeb 数据集, 203, 358
Flan, 214
Flan T5, 214-216
Flash Attention, 352, 357
float16（FP16；半精度）类型, 221-222
float32（FP32；全精度）类型, 221-222
Florence 2, 242
Flux, 143, 171
傅里叶变换（FTs）, 301-304
Fréchet 音频距离（FAD）, 340
Fréchet 起始距离（FID）分数, 124-125
免费音乐档案数据集, 359
FreeSound 数据集, 359
频率, 299（参见音频生成）
Frey Face 数据集, 93, 108
傅里叶变换（FTs）, 301-304
全模型微调, 239-247（参见 Stable Diffusion）
全精度（float32；FP32）类型, 221-222
G
GANs（生成对抗网络）, 111 与扩散模型的比较, 112-113 概述, 9 频谱图到波形重建, 328
门控循环单元（GRUs）, 36
高斯分布, 78, 83
高斯噪声, 126（参见噪声和噪声调度）
高斯溅射, 363
Gazelle, 367
Gemini, 352
Gemma, 37, 39, 44, 208, 222, 369
Gen-3, 365
genaibook 库, xiv, 4
生成对抗网络（参见 GANs）
生成模型, 3-12 音频生成, 291-343 大模型和开源模型, 10-11 条件扩散模型, 149-158 定义, 3 扩散模型, 111-146 伦理和社会影响, 8 演变和现状, 9-10 图像生成, 4-6 快速进展, 349-368 社区, 367 计算机视觉, 360-362 数据, 358-359 长上下文, 352-354 专家混合, 354-356 多用途模型,

359 多模态, 365-367

优化技术, 356-357 偏好优化, 349-351 量化, 356-357 3D 计算机视觉, 362 视频生成, 363-365 声音片段生成, 8 Stable Diffusion, 160-171 文本生成, 7-8 文本到图像模型, 263-289 迁移学习微调语言模型, 185-234 微调 Stable Diffusion, 239-259 变压器模型, 13-52 GLIDE, 111 Google, 10 Google Colab, xiii Google 数据集搜索, 187 Google DeepMind, 365-366 GPQA 数据集, 206 GPT-2, 14, 44, 208 少样本任务, 32 生成文本, 7 GPT-4, 39 GPT-Neo, 208 GPU 的使用优势, xiii gradient_accumulation_steps 超参数, 245 gradient_checkpointing 超参数, 245 Gradio, 179-180 使用变压器处理图, 49 贪婪解码, 21 贪婪搜索, 23 Griffin–Lim 算法, 327 GRUs（门控循环单元）, 36 引导, 172-173

H

半精度（float16；FP16）类型, 221-222
硬件要求, xiii
HEIM（文本到图像模型的整体评估）, 126
HiFiGAN, 328
热词增强, 312
HuBERT, 310-311
Hugging Face 定义, 3 3D 机器学习课程, 363 开放 LLM 排行榜, 205-208 公共数据集, 187

推送图像-文本数据集, 242
技术栈, 369-370
模型存储, 5

J
JPEG 文件, 55
Jupyter Notebooks, xiv
I
身份偏好优化 (IPO), 350
IFEval 数据集, 206
图解 Transformer (Alammar), 51
ILSVRC (ImageNet 大规模视觉识别挑战赛), 101, 360
图像分类, 360
图像编辑反演, 274-279 提示词加权, 267-273 语义引导, 270-273
图像生成, 4-6, 111-146 替代架构, 142-143 迭代优化, 112-115 噪声调度, 126-137 训练, 115-126 UNets, 137-141

图像提示适配器 (IP-Adapters), 283-287
图像提示, 285-287 额外控制, 287 风格迁移, 286
图像分割, 361
图像变体, 283-285
图像到图像管道, 263-265
imagefolder 模式, 242
ImageNet 数据集, 101, 360
ImageNet 大规模视觉识别挑战赛 (ILSVRC), 101, 360
“通过生成式预训练提升语言理解能力” (Radford 等), 51
Inception 分数, 125
图像修复, 256, 265-267
输入缩放, 135
指令微调, 213-215, 277-279
知识产权 (IP) 权利, 241
Interface 类, 179
反演调度器, 274
反演, 274-279 指令微调, 277-279 LEDITS, 276-277
IP (知识产权) 权利, 241
IP-Adapters (图像提示适配器), 283-287
IPO (身份偏好优化), 350
迭代优化, 112-115

K

Kaggle, xiii, 187
KID (Kernel Inception 距离), 125
KLD (Kullback–Leibler 散度), 83-87
KTO (Kahneman–Tversky 优化), 350
Kuai Kling, 365

L

LAION 数据集, 104, 160, 368
LAION-2B 数据集, 358
LAION-5B 数据集, 176-178
langchain 库, 359, 378
语言模型 (参见 LMs)

“语言模型是少样本学习者” (Brown), 51
“语言模型是无监督多任务学习者” (Radford 等), 51
语言塔, 99
大语言模型 (参见 LLMs)
潜在扩散, 159 (参见 Stable Diffusion)
潜在空间, 56-57, 70-77 维度, 70-74 可视化, 74-77
余弦定律, 95
学习率和学习率调度器, 210
“学习从任务描述生成任务特定适配器” (Ye 和 Ren), 216
LEDITS, 275-277
LibriSpeech 数据集, 293, 310
LibriVox 数据集, 310
librosa 库, 297
LIMA 数据集, 216
Llama, 7, 11, 37, 39, 44, 208, 222, 369, 374
“Llama 3 模型群, The” (Dubey 等), 202
Llama 3.1, 352
llama.cpp, 357, 371
llamaindex 工具, 359
LLaVA 架构, 366
LLM.int8() 量化, 223
LLMs (大语言模型), 13 使用方法, 359 长上下文, 352 内存需求, 373-375 推理, 373 训练, 374
lmdeploy, 372
LMs (语言模型), 14-32 架构, 33-35 少样本泛化, 30-32 微调, 185-234 适配器, 213, 216-220 指令微调, 213-215 多模型, 212 提示, 213 量化, 221-230 RAG, 233-234 文本分类, 186-203 文本生成, 203-212 概率预测, 17-20 文本生成, 20-28, 50 文本分词, 14-16 零样本泛化, 28-30
LMSYS, 232
load_dataset() 方法, 242
logits, 18
长音频转录, 315
长上下文, 190, 352-354
长短期记忆 (LSTM), 36
Longformer, 352
LoRAs (低秩适配), 239, 247 适配器, 217-220 训练, 253-257
低秩矩阵, 218

LSTM (长短期记忆), 36

M

Make-A-Video, 364
Mamba, 353
Marian NMT, 37
掩码语言建模 (MLM), 38
MaskGIT, 127
大规模文本嵌入基准, 383
大规模多语言语音 (MMS) 数据集, 329
MATH 数据集, 206
matplotlib 库, 58, 150
Matryoshhka 嵌入模型, 383
最大池化, 137
平均意见分数 (MOS), 341
均方误差 (MSE), 84, 121
Medusa 解码, 357
梅尔频谱图, 306-307
MelGAN, 328
Meta, 10-11, 367
Midjourney, 177
Milvus 向量数据库, 384
min_p 参数, 28
Mistral, 7, 11, 39, 208, 222
mixed_precision 超参数, 244
Mixtral, 208
Mixtral 8x22B, 356
Mixtral 8x7B, 354
专家混合 (MoE), 208, 354-356
MLM (掩码语言建模), 38
MMLU-Pro 数据集, 205
MMS (大规模多语言语音) 数据集, 329
MNIST 数据集, 57-60, 140

MobileCLIP, 105
ModelScope 文本到视频模型, 364
MoE (专家混合), 208, 354-356
单目深度估计, 361
MOS (平均意见分数), 341
Mosaic MPT, 208
MPS 设备, xiv
MSE (均方误差), 84, 121
MuLan, 333
多语言支持, 342
多模态, 49, 365-367
“多任务提示训练实现零样本任务泛化” (Sanh 等), 215
Muse, 127
MusicCaps 数据集, 333
MusicGen, 8, 335-336
MusicLM, 332-333
MuSR 数据集, 206

N

n-gram 分数, 312
原生视频生成, 364
自然指令数据集, 215
Needle in a Haystack 方法, 353
负面提示, 173
神经辐射场 (NeRFs), 363
神经声码器, 328
自然语言处理 (NLP), 10
噪声和噪声调度, 126-137
添加噪声, 118, 128-130
无分类器引导, 172

输入分辨率和缩放的影响, 135-137 模型训练和, 153-156 添加噪声的原因, 126-127 精炼模型, 170 采样和, 157-158 文本编码器和, 161 底层数学, 130-135 Nous Research, 367
no_repeat_ngram_size 参数, 26 核采样, 26 “使用引导扩散模型编辑真实图像的空文本反演” (Mokady 等), 275 num_train_epochs 超参数, 244

O

目标检测, 360
开放 LLM 排行榜, 205-208
开放模型, 10-11, 359 条件扩散模型, 149-158 扩散模型, 111-146 表示和压缩, 55-107 Stable Diffusion, 160-171 Transformer 模型, 13-52
开源工具, 369-372 数据集, 370 部署工具, 372 Hugging Face 堆栈, 369-370 本地推理, 371 封装器, 371
开放访问模型, 3, 7
OpenAI, 10, 94, 364 (参见 CLIP)
OpenCLIP, 95, 162, 177
牛津宠物数据集, 93, 108

P

Paella, 127
全景分割, 361
Parti Prompts 数据集, 126
参数高效微调 (PEFT),
216-220, 225-230
peft 库, 370
困惑度, 231
相位, 327
(参见音频生成)
Phi, 208
Phi-3, 203, 358

“选择向量数据库”博客文章, 384
PIL库, 48
pipeline API, 38, 43, 200, 211, 307
PixArt- •Σ , 143
池化, 137, 268
位置编码, 34
预训练模型, 3
精度指标, 192
偏好优化, 349-351
先验保留损失, 248, 250

隐私问题, 9
概率预测, 17-20
提示词加权, 174, 267-273 合并与, 268-270 语义引导, 270-273
“通过交叉注意力控制进行提示到提示的图像编辑” (Hertz等), 267
提示词定义, 5 图像提示, 285-287 负面提示, 173 提示词加权, 174, 267-273
push_to_hub超参数, 245
PyPDFLoader类, 378
Python 创建3.10虚拟环境, xiii hypothesis_template, 103 matplotlib库, 58
PyTorch矩阵乘法原语, 381

Q
Qdrant向量数据库, 384
QLoRA, 225-226
量化, 221-230, 356-357
Qwen, 32, 39, 44, 208
Qwen 1.5 MoE, 356
Qwen2, 14

R
RAG (检索增强生成), 53, 233-234, 377-384 组件, 377 数据处理, 378-379 嵌入文档, 380 生成, 382 构建RAG系统的开源工具, 384 生产级RAG, 383 检索, 381
RAGatouille, 384
重排序, 383
召回指标, 193
重建损失, VAEs, 84-91
循环接口网络 (RINs), 143
循环神经网络 (见RNNs)
递归字符文本分割器, 379
精炼模型, SDXL, 170
监管与问责, 9
人类反馈强化学习 (RLHF), 204, 349-351
相对熵, 83-87
ReLU函数, 70
重参数化技巧, 131
repetition_penalty参数, 22
表示, 55-107 自编码器, 57-93 数据准备, 57-60 解码器, 64-65 编码器, 61-63 潜在空间, 70-77 训

练, 65-69 变分, 78-93 CLIP, 93-105 替代方案, 104 对比损失, 94-95 步骤, 95-102 用例, 103 零样本分类, 101-103
resolution超参数, 244
分辨率, SDXL, 170
检索增强生成 (见RAG)
奖励模型, 350
Riffusion, 336-338
RINs (循环接口网络), 143
RLHF (人类反馈强化学习), 204, 349-351
RNNs (循环神经网络) 定义, 9 机器翻译, 36 RWKV, 353
RoBERTa, 43
RoPE (旋转位置嵌入) 缩放, 352
ROUGE, 231
RunwayML, 160, 365
RWKV, 353
S
采样, 23-28
采样率, 295
Scrapy, 241
SDXL (Stable Diffusion XL), 169-170, 257 (另见Stable Diffusion)
SeamlessM4T, 341
种子, 6
SEGA (语义引导), 268, 270-273
SegFormer, 48
自注意力层, UNets, 141
自注意力机制, 变换器, 14, 19, 34
Self-Instruct数据集, 215
自监督预训练, 42, 58
语义引导 (SEGA), 268, 270-273
语义搜索, 43, 103-106
语义分割, 361
句子变换器, 371, 380, 383
seq2seq (序列到序列) 模型 ASR, 312-315 文本到语音, 324-329 变换器模型, 35-37, 44
序列分类, 186-203 定义评估指标, 192-194 定义模型类型, 189 识别数据集, 187-188 预处理数据集, 190-192 相关性, 202 选择基础模型, 190 训练模型, 194-201

顺序处理, 45
set_transform()方法, 117
SFT (监督微调), 213-215, 277-279
浅层融合, 313
短时傅里叶变换 (STFTs), 304
sigmoid函数, 69-70
信噪比, 135
跳跃连接, 137
SmolLM, 14, 208-212, 382
Snowflake Arctic Instruct, 356
softmax()方法, 19
软件要求, xiii-xiv
Sora, 143, 364
SOTA (最先进) 模型 定义, xvi 扩散模型, 4-6
声音与环境效果, 335-335
声音片段生成, 8
声音延续, 332
说话人分离, 292
说话人嵌入, 326-327
频谱图, 298-307
推测解码, 357
语音到文本, 291, 307-323 基于编码器的技术, 308-312 编码器-解码器技术, 312-315 评估, 318-323 生产, 315-318
SpeechT5, 324-329
SSMs (状态空间模型), 353
Stability AI, 11, 160
Stable Diffusion, 111, 160-171 音频生成, 338 微调, 239-259 特殊条件的额外输入, 256 DreamBooth, 248-253 微调模型, 242-245 推理, 246-247 修复, 256 准备数据集, 240-242 训练 LoRAs, 253-256 Flux, 171 图像生成, 4-6 指令微调, 277-278 SDXL, 169-170, 257, 268 Stable Diffusion 2 Depth, 256 Stable Diffusion 3, 143, 171 文本编码器, 161-164 UNets, 167-169 VAE, 164-167
StableDiffusionPipeline, 4, 161
最先进模型 (见SOTA模型)
状态空间模型 (SSMs), 353
STFTs (短时傅里叶变换), 304

扩散过程的随机性, 6
streamlit库, 179
子词分词, 16
“超级自然指令” (Wang等), 215
监督微调 (SFT), 213-215, 277-279
监督学习, 41
Swin Transformer, 48
合成数据, 358

T

T2I适配器, 257
目标宽高比, SDXL, 170

TensorRT-LLM, 372
文本分类, 186-203
定义评估指标, 192-194
定义模型类型, 189
识别数据集, 187-188
预处理数据集, 190-192
相关性, 202
选择基础模型, 190
训练模型, 194-201

文本生成, 7-8, 20-28, 50, 203-212
评估, 230-233
选择模型, 204-208
训练模型, 208-212

文本转音频 (见 TTA)
文本转语音 (见 TTS)
文本分词, 14-16, 164

文本到图像模型, 263-289
ControlNet, 279-282
图像提示, 285-287
附加控制, 287

风格迁移, 286
图像变体, 283-285
图像到图像管道, 263-265
图像修复, 265-267
反演, 274-279
提示权重和图像编辑, 267-273
合并, 268-270
语义引导, 270-273

Textual Inversion, 239, 250
3D 计算机视觉, 362
3D 数据, 使用 Transformer 处理, 49
THUDM 组, 365
时间序列数据, 使用 Transformer 处理, 49
时间戳, 316
时间步, 153-156
分词分类, 42
分词嵌入, 34
分词, 14-16, 34
分词训练, 17
Top-K 采样, 26
Top-p 采样, 26
topk() 方法, 19
torchsummary 库, 66
torchvision 库, 60, 117
总损失, VAEs, 84-86
ToTensor() 函数, 60
Trainer 类, 196-200
train_batch_size 超参数, 245
train_text_to_image.py 脚本, 243-245
Tran-Thien, Vivien, 358
迁移学习, 41
微调语言模型, 185-234
微调 Stable Diffusion, 239-259

Transformer 块, 34

Transformer 模型, 13-52
预训练的优势, 40-43
架构, 33-39, 43
仅编码器模型, 37-39
序列到序列任务, 35-37
简化视图, 33-35
数据整理器, 209
扩散模型, 143
语言模型, 14-32
架构, 33-35
少样本泛化, 30-32
概率预测, 17-20
文本生成, 20-28
文本分词, 14-16
零样本泛化, 28-30
局限性, 45-46
自然语言处理, 10
非文本用例, 46
量化方法, 224
语音转文本, 307-323
基于编码器的技术, 308-312
编码器-解码器技术, 312-315
评估, 318-323
生产, 315-318
训练循环优化, 196

transformers 库, 369
自动类, 17
加载图像模型, 96
标准化器, 320
概述, 7-8
语音处理器, 325
包装器, 371

Transformers.js, 371
转置卷积, 64

trl 库, 371
TTA (文本转音频), 292, 324, 332-340
音频扩散和 Riffusion, 336-338
AudioGen 和 MusicGen, 335-336
AudioLM 和 MusicLM, 332-333
Dance Diffusion, 339
评估, 340

TTS (文本转语音), 291, 324-331
Bark, 329-331
评估, 340
SpeechT5, 324-329
排版约定, xiv

U

UNets, 119-120, 137-141
添加输入通道, 256
音频生成, 339
构建, 138-140
条件扩散模型, 151-153
创建, 120
改进, 141
SDXL, 170
Stable Diffusion, 167-169
文本编码器, 161

统一建模, 341
Unified-IO 架构, 367
Unnatural Instructions 数据集, 215
unsloth, 371
use_8bit_adam 超参数, 245
use_ema 超参数, 244
UViT 架构, 143

V

VAEs (变分自编码器), 78-93
定义, 107
编码器和解码器, 79-80
生成建模, 93
潜在扩散, 159
潜在空间, 57
从编码器分布中采样, 80-83
SDXL 和, 170
Stable Diffusion, 164-167
训练, 83-93

vae_loss 函数, 84
validation_prompts 超参数, 245
方差保持 (VP) 调度, 134
变分自编码器 (见 VAEs)
方差爆炸 (VE) 调度, 134
向量数据库, 384
速度 (v) 目标, 144
Veo, 365
视频生成, 363-365
视频到视频技术, 364
视觉塔, 98
视觉语言模型 (VLMs), 366
VITS, 329
ViTs (视觉 Transformer), 47, 95
vLLM 库, 372

VLMs (视觉语言模型), 366
语音克隆, 292, 341
语音增强, 292
VP (方差保持) 调度, 134

W

Wav2Vec2, 310-311, 318-323
WaveNet, 328
Weaviate 向量数据库, 384

网络爬虫, 241
权重衰减, 210
WER (词错误率), 319-323
Whisper, 312-315, 318-323
窗口注意力, 352
WizardLM, 216
词错误率 (WER), 319-323
词级分词, 15

X

Y Yi, 39

Z

Zalando, 150
Zenodo, 187
零样本分类
CLIP, 101-103
管道, 102
使用, 101-102
图像分类, 48
序列分类, 189
零样本泛化, 28-30
零样本任务, 362
ZIP 文件, 55

XLSR-53, 311

# 关于作者

Omar Sanseviero 曾是 Hugging Face 的首席羊驼官及平台与社区负责人，领导开发者倡导工程、设备端和 moonshot 团队。Omar 在 Google 的 Google Assistant 和 TensorFlow Graphics 团队中拥有丰富的工程经验。Omar 在 Hugging Face 的工作涉及开源、产品、研究和技术社区的交汇点。

Pedro Cuenca 是 Hugging Face 的一名机器学习工程师，专注于扩散软件、模型和应用开发。他拥有超过 20 年的软件开发经验，涉及互联网应用和 iOS 等领域。作为 LateNiteSoft 的联合创始人兼首席技术官，他参与了 Camera+ 的技术开发，这是一款成功的 iPhone 应用，使用自定义机器学习模型来增强摄影效果。他创建了用于摄影增强和超分辨率等任务的深度学习模型。他还参与了 DALL·E mini 的开发和运营工作。他带来了将 AI 研究整合到现实世界服务中的实践视角，并关注其中的挑战和优化。

Apolinário Passos 是 Hugging Face 的一名机器学习艺术工程师，跨团队合作，致力于艺术和创意领域的多种机器学习用例。Apolinário 拥有超过 10 年的专业和艺术经验，曾在艺术展览、编码和产品管理之间交替工作，并曾担任 World Data Lab 的产品负责人。Apolinário 的目标是确保机器学习生态系统支持并适用于艺术用例。

Jonathan Whitaker 是一名数据科学家和深度学习研究员，专注于生成建模。他之前曾参与编写与本书主题相关的多门课程，包括 Hugging Face 的扩散模型课程以及与 Jeremy Howard 在 2022 年共同创建的 Fast.AI 课程《从深度学习基础到 Stable Diffusion》。他还在担任顾问期间将这些技术应用于工业领域，现在全职在 Answer.AI 从事 AI 研究和开发工作。

# 版权页

《动手实践生成式 AI：Transformers 与扩散模型》封面上的动物是非洲巨燕尾蝶（Papilio antimachus）。

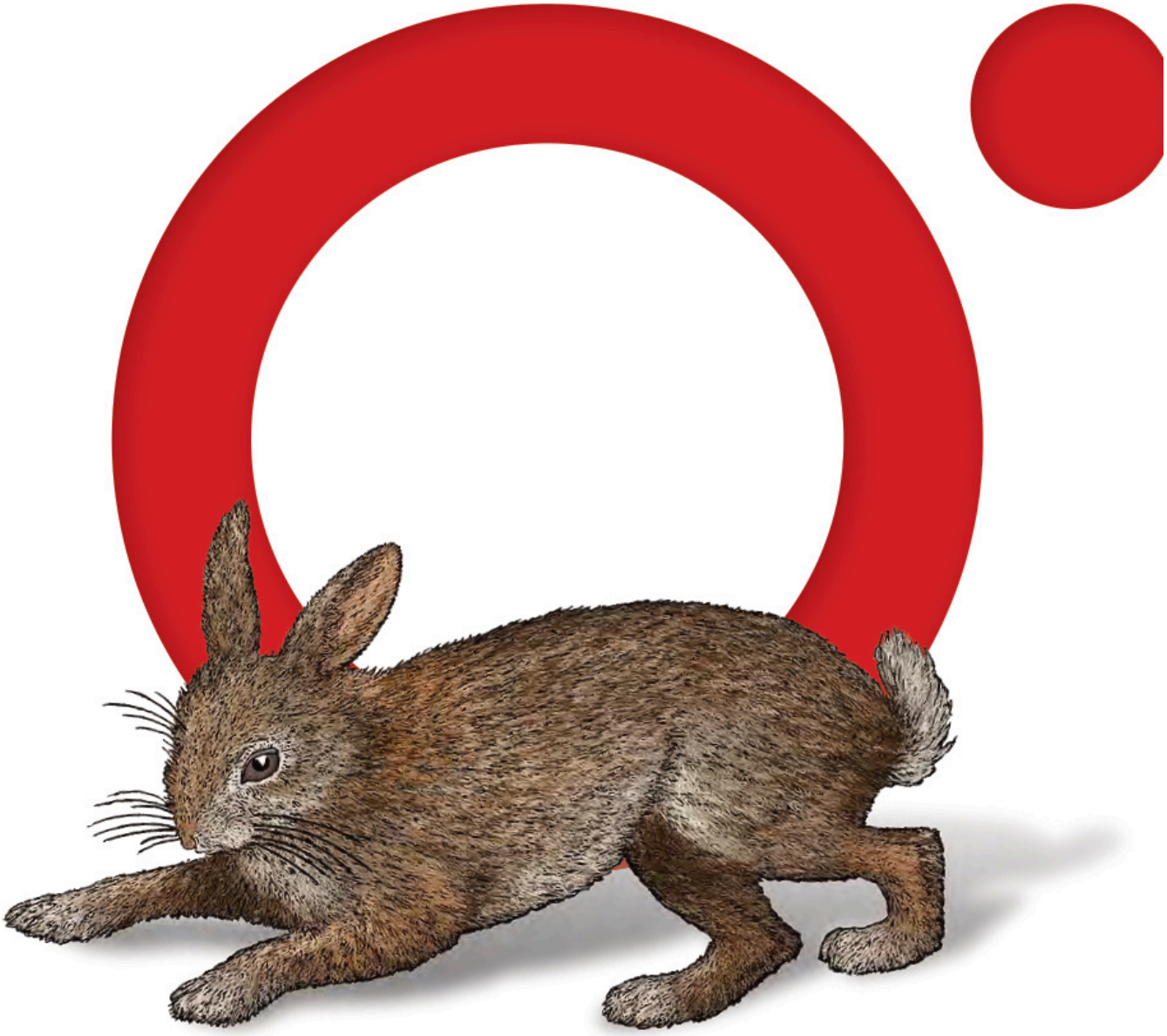
非洲巨燕尾蝶是最大的蝴蝶物种之一，翼展可达 9 至 10 英寸（大约相当于一个餐盘或黑胶唱片的大小）。然而，尽管它们的体型令人印象深刻，但人们对这些巨型昆虫的了解却相对较少。

非洲巨燕尾蝶于 1782 年首次被发现，生活在西非和中非的热带雨林中，大部分时间都在森林冠层中度过；雄性偶尔会被观察到在森林地面上进行泥浆吸水行为，这是一种蝴蝶聚集在湿有机物质（如土壤或粪便）上以获取营养的行为。

由于它们的饮食，非洲巨燕尾蝶具有高度毒性，且没有已知的天敌。尽管有报道称由于栖息地破坏和偷猎（标本非常珍贵，价格可超过 1000 美元）导致种群数量下降，但 IUCN 已将非洲巨燕尾蝶列为数据不足：需要更多信息才能进行保护评估。O’Reilly 封面上的许多动物都濒临灭绝；它们对世界都很重要。

封面插图由 Karen Montgomery 绘制。系列设计由 Edie Freedman、Ellie Volckhausen 和 Karen Montgomery 完成。封面字体为 Gilroy Semibold 和 Guardian Sans。正文字体为 Adobe

Minion Pro；标题字体为 Adobe Myriad Condensed；代码字体为 Dalton Maag 的 Ubuntu Mono。



O'REILLY

向专家学习。
成为专家。

60,000+ 书籍 | 专家现场活动 | 基于角色的课程 互动学习 | 认证准备

免费试用 O’Reilly 学习平台 10 天。